

SLAM Handbook
从定位与建图到空间智能

中文翻译

2026 年 8 月 12 日

版权与翻译说明

本书原著由 Luca Carlone、Ayoung Kim、Timothy Barfoot、Daniel Cremers 和 Frank Dellaert 编辑，© Cambridge University Press。

本 PDF 收录已完成审查的 SLAM Handbook 中文译文。原作者、编辑、出版方及原项目的版权和署名信息均予以保留。中文翻译内容仅在获得相应授权的范围内公开使用。

目录

版权与翻译说明	i
第一章 SLAM 的因子图	1
历史注记	1
1.1 用因子图直观表示 SLAM	2
1.1.1 一个玩具示例	2
1.1.2 因子图视角	3
1.1.3 将因子图作为一种语言	6
1.2 从 MAP 推断到最小二乘	7
1.2.1 用因子图进行 MAP 推断	7
1.2.2 指定概率密度	8
1.2.3 非线性最小二乘	9
1.3 求解线性最小二乘	10
1.3.1 线性化	10
1.3.2 将 SLAM 写成线性最小二乘	11
1.3.3 用矩阵分解求解最小二乘	11
1.4 非线性优化	13
1.4.1 最速下降	13
1.4.2 高斯—牛顿	13
1.4.3 Levenberg—Marquardt	14
1.4.4 Dogleg 最小化	14
1.5 因子图与稀疏性	15
1.5.1 稀疏雅可比矩阵及其因子图	15
1.5.2 稀疏信息矩阵及其图	15
1.5.3 稀疏分解	17
1.6 消元	17
1.6.1 变量消元算法	17
1.6.2 线性高斯消元	20
1.6.3 作为贝叶斯网络的稀疏 Cholesky 因子	23
1.7 增量式 SLAM	23
1.7.1 贝叶斯树	24
1.7.2 更新贝叶斯树	24

1.7.3	增量平滑与建图	25
1.8	延伸阅读与近期趋势	26
第二章	高级状态变量表示	27
2.0.1	流形上的优化	27
2.0.2	旋转与位姿	27
2.0.3	矩阵 Lie 群	28
2.0.4	Lie 群优化	30
2.0.5	不确定性与 Lie 群	32
2.0.6	Lie 群的其他工具	32
2.0.6.1	$\oplus$ 与 $\ominus$ 算子	32
2.0.6.2	逆	33
2.0.6.3	伴随	33
2.0.6.4	雅可比	33
2.1	连续时间轨迹	34
2.1.1	样条	34
2.1.2	从参数化到非参数化	36
2.1.3	高斯过程	37
2.1.4	Lie 群上的样条与高斯过程	38
2.1.4.1	Lie 群上的样条	38
2.1.4.2	Lie 群上的高斯过程	40
2.2	延伸阅读与近期趋势	41
第三章	对错误数据关联和离群值的鲁棒性	43
3.1	离群值的成因及其危害	43
3.2	数据关联与离群值	43
3.3	存在离群值时的最小二乘	44
3.4	在 SLAM 前端检测和剔除离群值	45
3.5	随机采样一致性 (RANSAC)	49
3.6	基于图论的离群值剔除与成对一致性最大化	50
3.7	提高 SLAM 后端对离群值的鲁棒性	53
3.8	迭代重加权最小二乘	64
3.9	Black-Rangarajan 对偶性	65
3.10	交替最小化	73
3.11	渐进非凸性	74
3.12	延伸阅读与近期趋势	85
第四章	可微优化	89
4.1	非线性最小二乘回顾	89
4.2	穿过非线性最小二乘进行微分	90
4.3	展开微分	91
4.4	截断展开微分	92

4.5	隐式微分	93
4.6	流形上的微分	94
4.7	Lie 群导数	95
4.8	流形上的微分运算	97
4.9	自动微分的数值挑战与现代库	98
4.10	可微优化的实现示例	99
4.11	相关开源库	100
4.12	延伸阅读与近期趋势	101
第五章	稠密地图表示	102
5.1	范围感知基础	102
5.2	传感器测量模型	103
5.3	转换为点云	103
5.4	稠密建图基础	105
5.5	占据地图	106
5.6	隐式表面与距离场	106
5.7	占据地图还是距离场?	109
5.8	地图表示	109
5.9	目标空间结构的显式性	109
5.10	空间抽象类型	109
5.11	点	109
5.12	Surfel	112
5.13	网格	112
5.14	体素	113
5.15	连续函数	113
5.16	转换	113
5.17	数据结构与存储	114
5.18	朴素数据存储	114
5.19	哈希表	114
5.20	基于树的数据结构	115
5.21	混合数据结构	116
5.22	构建地图: 方法与实践	116
5.23	点	117
5.24	Surfel	117
5.25	网格	117
5.26	体素	118
5.27	按估计量分类的方法	118
5.28	按数据结构分类的方法	118
5.29	按测量融合算法分类的方法	119
5.30	按可扩展性分类的方法	119
5.31	高斯过程	119

5.32 高斯过程隐式表面	120
5.33 Hilbert map	120
5.34 建图中的深度学习	124
5.35 使用考量	124
5.36 环境因素	124
5.37 下游任务类型	125
5.38 建图方法总结	125
5.39 致谢	125
第六章 SLAM 的可认证最优求解器与理论性质	126
6.1 SLAM 的可认证最优求解器	126
6.2 Shor 松弛	127
6.3 SE-Sync: 可认证正确的位姿图优化	132
6.4 位姿图优化: QCQP 表述	132
6.5 将 Shor 松弛应用于位姿图优化	133
6.6 专栏 6.1: 代数图论要素	133
6.7 通过黎曼阶梯高效求解松弛	136
6.8 舍入解	137
6.9 SE-Sync: 完整算法	137
6.10 基于路标的 SLAM	139
6.11 扩展: 距离测量、各向异性噪声与离群值	142
6.12 面向距离辅助 SLAM 的快速可认证算法	144
6.13 超越 Shor 松弛的可认证算法: 各向异性噪声与离群值	144
6.14 SLAM 问题最优解的准确性如何?	146
6.15 Cramer-Rao 下界与 Fisher 信息矩阵	147
6.16 Fisher 信息矩阵与图拉普拉斯	148
6.17 延伸阅读与近期趋势	149
第七章 视觉 SLAM	151
7.1 历史背景与术语	151
7.1.1 从摄影测量到束调整和视觉 SLAM	151
7.1.2 术语	152
7.2 视觉 SLAM 系统的处理流水线	152
7.2.1 视觉里程计前端	152
7.2.2 建图后端	153
7.2.3 视觉地点识别与重定位	153
7.2.4 计算与数据流	153
7.3 视觉 SLAM 基础	153
7.3.1 相机模型	153
7.3.1.1 几何相机模型	153
7.3.1.2 光度模型	154
7.3.1.3 时间相关效应	155

7.3.1.4	实际考虑	155
7.3.2	关键点	155
7.3.2.1	经典关键点检测器与描述子	156
7.3.2.2	基于深度学习的关键点检测与描述	156
7.3.3	重投影误差	157
7.3.4	基于关键点的视觉 SLAM	157
7.3.5	光度误差与直接法	158
7.3.6	视觉地点识别与全局定位	158
7.3.7	初始化	158
7.3.8	地图表示	160
7.4	关于图像对齐和 BA 的进一步考虑	160
7.4.1	基于关键点的图像对齐	160
7.4.2	直接图像对齐	161
7.4.3	求解 BA	162
7.4.4	再约束调整	163
7.5	完整视觉 SLAM 系统示例	165
7.6	实时稠密重建	166
7.7	基于深度感知相机的 SLAM	170
7.8	将视觉与其他模态相结合	171
7.8.1	惯性测量单元 (IMU)	171
7.8.2	用于全局定位的 GPS 和 WiFi	171
7.9	延伸阅读与近期趋势	171
第八章	LiDAR SLAM	174
8.1	LiDAR 感知基础与分类	175
8.2	LiDAR 里程计	177
8.3	扫描配准基础	178
8.3.1	配准残差中的距离度量	178
8.3.2	确定对应关系	179
8.4	LiDAR 里程计的常见组件	180
8.4.1	点云运动畸变补偿	180
8.4.2	基于特征的 LiDAR 里程计	181
8.4.3	直接逐点 LiDAR 里程计	182
8.4.4	局部建图与位姿估计	182
8.4.5	LiDAR 里程计小结	187
8.5	LiDAR 地点识别	187
8.5.1	问题定义	187
8.5.2	LiDAR 地点识别方法	188
8.5.2.1	局部描述子	188
8.5.2.2	全局描述子	188
8.5.2.3	高层或组合描述子	188

8.5.3	LiDAR 地点识别小结	188
8.6	LiDAR SLAM	189
8.6.1	LiDAR SLAM 系统的结构	189
8.6.2	位姿图优化与地图更新	190
8.6.3	多机器人与多会话 LiDAR SLAM	191
8.6.3.1	多机器人 SLAM	192
8.7	延伸阅读与近期趋势	194
第九章	雷达 SLAM	195
9.1	雷达简介	195
9.1.1	传感器类型	195
9.1.1.1	旋转雷达	195
9.1.1.2	SoC 雷达	197
9.1.2	雷达感知原理	197
9.1.2.1	雷达散射截面	197
9.1.2.2	FMCW 测距	197
9.1.2.3	用锯齿调制确定距离和速度	198
9.1.2.4	用三角调制确定距离和速度	199
9.1.2.5	确定 SoC 雷达的角度	200
9.1.3	雷达应用的挑战	201
9.1.3.1	散斑噪声	201
9.1.3.2	多径	201
9.1.3.3	运动引起的畸变	201
9.1.4	雷达滤波	201
9.2	雷达里程计	203
9.2.1	多普勒里程计	204
9.2.2	示例：多普勒里程计因子表述	204
9.2.3	直接里程计	205
9.2.4	基于特征的里程计	205
9.2.5	基于配准的里程计	206
9.2.6	运动补偿	206
9.3	雷达地点识别	207
9.3.1	雷达地点识别的独特挑战	207
9.3.2	基于学习的雷达地点识别	208
9.3.3	基于描述子的雷达地点识别	208
9.4	雷达 SLAM	209
9.4.1	地图表示	209
9.4.2	雷达 SLAM 流水线	210
9.4.3	雷达 SLAM 中的多模态	211
9.5	雷达数据集	212
9.6	延伸阅读与近期趋势	213

第十章 基于事件的 SLAM	214
10.1 传感器描述	214
10.1.1 工作原理	214
10.1.2 事件相机的优势	216
10.1.3 当前器件与趋势	216
10.2 挑战与应用	216
10.3 基于事件的 SLAM 方法概述与分类	217
10.4 基于事件的 SLAM 系统前端	219
10.4.1 预处理与事件表示	221
10.4.2 间接法	221
10.4.3 直接法	222
10.4.4 基于模型与基于学习的方法	222
10.5 基于事件的 SLAM 系统后端	222
10.6 最先进的系统	223
10.7 数据集、模拟器与基准	223
10.7.1 模拟器	225
10.7.2 数据集与基准	226
10.7.3 指标	228
10.8 延伸阅读与近期趋势	228
10.9 致谢	229
第十一章 面向 SLAM 的惯性里程计	230
11.1 惯性传感与导航基础	230
11.1.1 感知原理与测量模型	230
11.1.2 初始对准	231
11.2 IMU 预积分与因子图	232
11.2.1 运动积分	232
11.2.2 流形上的 IMU 预积分	233
11.2.2.1 预积分 IMU 测量	233
11.2.2.2 噪声传播	234
11.2.2.3 纳入偏置更新	235
11.2.2.4 预积分 IMU 因子与偏置模型	235
11.2.3 高级预积分技术	235
11.2.3.1 数值积分精度	235
11.2.3.2 连续加速度预积分	239
11.2.3.3 连续旋转预积分	239
11.3 辅助惯性导航的可观测性	240
11.3.1 线性化测量模型	240
11.3.1.1 线性化 IMU 运动学模型	240
11.3.1.2 外感测量模型	241
11.3.2 可观测性分析	242

11.3.3 退化运动	242
11.4 视觉惯性里程计与实际考虑	243
11.4.1 视觉惯性里程计	243
11.4.2 外参标定	245
11.4.3 时间同步	245
11.5 延伸阅读与近期趋势	245
第十二章 面向 SLAM 的腿部里程计	247
12.1 历史背景与预备知识	247
12.1.1 历史背景	247
12.1.2 参考坐标系	248
12.1.3 状态定义	248
12.1.4 足式机器人运动学	249
12.1.5 足式机器人动力学	250
12.1.6 关节感知	251
12.1.6.1 旋转编码器	251
12.1.6.2 力与力矩传感器	251
12.1.6.3 接触传感器	252
12.2 运动估计	252
12.2.1 相对位姿估计	253
12.2.2 速度估计	254
12.3 接触估计	254
12.3.1 使用接触传感器	255
12.3.2 使用力/力矩传感器	255
12.3.3 使用 IMU	255
12.3.4 根据关节力矩感知	255
12.4 使用腿部里程计进行状态估计	256
12.4.1 编码器噪声传播	256
12.4.2 因子图平滑	256
12.4.2.1 接触预积分	257
12.4.2.2 速度预积分	257
12.4.2.3 多测量处理	258
12.4.3 与外感传感器融合用于 SLAM	258
12.5 开放挑战	258
12.5.1 腿部变形	259
12.5.2 非刚性接触与滑移	259
12.6 延伸阅读与近期趋势	260
12.7 致谢	260

第十三章 通过深度学习增强 SLAM	261
13.1 深度和相机位姿的深度学习	261
13.1.1 深度预测的深度学习	266
13.1.2 相机位姿预测的深度学习	266
13.1.3 深度和相机位姿的无监督学习	268
13.2 特征匹配和光流的深度学习	269
13.2.1 特征检测器和描述子的学习	269
13.2.2 特征匹配	270
13.2.3 光流与 RAFT	270
13.2.4 作为视觉测量的光流	272
13.2.5 根据光流估计位姿和深度	272
13.3 可微束调整与 DROID-SLAM	272
13.3.1 DROID-SLAM 架构	273
13.3.2 DROID-SLAM 推理	274
13.3.3 泛化到其他模态	277
13.3.4 Deep Patch Visual Odometry (DPVO)	277
13.4 DuSt3R	278
13.4.1 广义立体重建网络	279
13.4.2 回归损失与置信度感知损失	280
13.4.3 下游应用	280
13.4.4 全局对齐	282
13.5 MAST3R	282
13.5.1 匹配头	282
13.5.2 匹配目标	283
13.6 将 MAST3R 扩展到 SfM 和 SLAM	283
13.6.1 MAST3R-SfM	283
13.6.2 MUST3R	283
13.6.3 MAST3R-SLAM	284
13.7 延伸阅读与近期趋势	284
第十四章 可微体渲染的地图表示	286
14.1 三维场景表示与可微渲染	286
14.1.1 可学习三维表示	286
14.1.2 可微渲染	298
14.1.3 使用可微渲染的三维场景表示	298
14.2 Neural Radiance Fields (NeRF)	300
14.2.1 方法概述	300
14.2.2 NeRF 中的数据结构	302
14.2.3 Neural Fields	302
14.2.4 用于视觉 SLAM 的 NeRF/Neural Fields	303
14.2.4.1 Neural Field SLAM 中数据结构的特征与权衡	304

14.3 3D Gaussian Splatting	305
14.3.1 方法概述	305
14.3.2 3D Gaussian Splatting 的应用	316
14.3.3 用于 SLAM 的 3D Gaussian Splatting	321
14.4 延伸阅读与近期趋势	325
第十五章 动态与可变形 SLAM	327
15.1 动态 SLAM 问题的刻画	327
15.1.1 动态特性的刻画	327
15.1.1.1 更多术语	328
15.1.1.2 动态程度	330
15.1.2 状态估计与场景表示	330
15.1.3 在线与离线方法	330
15.2 短期动态与动态 SLAM	331
15.2.1 动态物体移除	331
15.2.2 动态物体跟踪	334
15.2.3 稠密动态 SLAM	335
15.2.3.1 运动物体分割	335
15.2.3.2 参数化模型	336
15.2.3.3 同时跟踪与重建	336
15.3 长期动态与终身 SLAM	337
15.3.1 用于定位的终身 SLAM	338
15.3.1.1 并行地图与摘要地图	338
15.3.1.2 外观不变表示	338
15.3.1.3 内存、扩展与边缘化	338
15.3.2 地图清理与变化检测	339
15.3.2.1 地图清理	339
15.3.2.2 变化检测	340
15.3.2.3 通过变化理解对象	341
15.3.3 变化感知 SLAM	342
15.3.4 时间场景理解	344
15.4 可变形 SLAM	344
15.4.1 NRSfM 与可变形 SLAM	344
15.4.1.1 非刚性运动恢复结构 (NRSfM)	344
15.4.1.2 可变形 SLAM	348
15.4.2 带深度信息的可变形 SLAM	348
15.4.3 使用单目相机的可变形 SLAM 流水线	349
15.4.4 初始化与地图扩展	349
15.5 延伸阅读与近期趋势	350

第十六章 度量-语义 SLAM	351
16.1 从传统 SLAM 到度量-语义 SLAM	351
16.2 稀疏度量-语义表示	352
16.2.1 物体表示与因子图建模	352
16.2.1.1 将物体形状表示为语义路标	352
16.2.1.2 将物体形状表示为二次曲面	354
16.2.1.3 将物体形状表示为网格	356
16.2.1.4 隐式物体形状表示	358
16.2.2 稀疏度量-语义 SLAM 的混合求解器	360
16.3 稠密度量-语义表示	362
16.3.1 基于点和 surfel 的度量-语义 SLAM	362
16.3.2 基于体素的度量-语义 SLAM	363
16.3.3 基于网格的度量-语义 SLAM	367
16.4 分层度量-语义表示与三维场景图	371
16.4.1 分层表示与符号落地	371
16.4.2 三维场景图	372
16.5 延伸阅读与近期趋势	373
第十七章 迈向开放世界空间智能	375
17.1 背景与术语	376
17.1.1 基础模型简史	376
17.1.2 术语与范围	376
17.2 面向空间智能的基础模型	376
17.2.1 基于特征的基础模型	378
17.2.2 生成式基础模型	379
17.2.3 类别无关的图像分割	379
17.3 开放世界建图	379
17.3.1 稠密表示	380
17.3.2 物体地图与三维场景图	381
17.3.3 隐式函数	384
17.3.4 任务驱动的表达	385
17.4 延伸阅读与近期趋势	386
17.4.1 用地图落地基础模型	386
17.4.2 重新审视是否需要地图	387
17.4.3 机器人基础模型?	387
17.4.4 总结性说明	389
第十八章 空间智能系统的计算结构	390
18.1 从 SLAM 走向空间智能	390
18.1.1 具身智能设备需要空间智能	390
18.1.2 场景表示与世界模型	390
18.1.3 SLAM 正在演化为空间智能	391

18.2 总体计算结构	392
18.3 空间智能中的状态估计与机器学习	392
18.4 处理器和传感器硬件的未来格局	393
18.4.1 处理器	393
18.4.2 传感器	395
18.5 将空间智能图映射到硬件	395
18.5.1 世界模型处理	395
18.5.1.1 世界模型	397
18.5.2 实时循环	398
18.5.3 靠近图像平面的处理	398
18.6 使用高斯信念传播的收敛式分布式计算	399
18.7 因子图中的持续学习	400
18.8 性能度量	404
18.9 延伸阅读与近期趋势	404

第一章 SLAM 的因子图

作者： Frank Dellaert、Michael Kaess、Timothy Barfoot

本章介绍因子图，并在高斯先验和高斯测量噪声的条件下，建立最大后验（maximum a posteriori, MAP）推断与最小二乘之间的联系。我们关注 SLAM 后端，即前端已经提取测量并完成数据关联之后的部分。首先，我们使用因子图对 SLAM 问题进行可视化（第 1.1 节）；随后说明 MAP 推断如何导出最小二乘优化（第 1.2 节）。接着讨论线性优化（第 1.3 节）和非线性优化（第 1.4 节）的求解方法，并进一步明确稀疏性、因子图和贝叶斯网络之间的联系（第 1.5 节）。然后介绍变量消元算法及其图模型解释（第 1.6 节），最后据此构建贝叶斯树以及增量平滑与建图（iSAM）算法（第 1.7 节）。本章最后讨论延伸阅读与近期趋势（第 1.8 节）。

历史注记

SLAM 的平滑方法不仅考虑机器人当前时刻的位置，还考虑截至当前时刻的完整机器人轨迹。许多作者只研究机器人轨迹的平滑问题^[172, 701, 700, 420, 594, 312]，这类方法如今称为**基于位姿的 SLAM**（pose-based SLAM）。它尤其适用于激光测距仪等传感器，因为这类传感器会在相邻机器人位姿之间产生两两约束。

更一般地，可以考虑完整 SLAM 问题^[1088]，即在最优意义下，同时估计全部传感器位姿以及环境中所有特征的参数。从计算角度看，这种基于优化的平滑方法具有明显优势：(a) 与基于滤波的协方差矩阵或信息矩阵不同，后两者都会随时间推移逐渐变成完全稠密的矩阵^[849, 1087]，而平滑对应的信息矩阵保持稀疏；(b) 在典型建图场景中（也就是机器人并非反复穿行于一个很小的环境），该矩阵能够更紧凑地表示地图协方差结构。由此，在 2000—2005 年间，大量工作开始将这些思想应用于 SLAM^[287, 340, 339, 1088]。

基于平方根的平滑与建图（square-root smoothing and mapping, SAM），也称为“因子图方法”，是在^[249, 254]中提出的，其依据是：信息矩阵或测量雅可比矩阵可以分别通过稀疏 Cholesky 分解或 QR 分解进行高效分解。这样便得到一个平方根信息矩阵，可立即用于求取最优机器人轨迹和地图。在序贯估计文献中，对信息矩阵进行分解被称为**平方根信息滤波**（square-root information filtering, SRIF）；该方法于 1969 年为 JPL 的水手 10 号（Mariner 10）金星探测任务而发展^[83]。使用平方根能够得到更加准确、稳定的算法。引用 Maybeck^[742]的话说：“许多实践者以充分的逻辑依据主张：平方根滤波器应始终优先于标准的卡尔曼滤波递推。”

下面我们将详细讨论：因子图为何是表示 SLAM 问题固有稀疏性的自然工具；将矩阵进行（稀疏）分解并得到矩阵平方根为何是求解这类问题的核心；以及这些内容最终如何与更一般的变量消元算法联系起来。本章的大部分内容是 Dellaert 等人^[255]长篇文章的压缩版。

1.1 用因子图直观表示 SLAM

本节首先通过一个玩具示例及其因子图表示，引入因子图，直观展示 SLAM 问题的稀疏性。随后说明多种不同形式的 SLAM 都可以采用这种表示，即使面对更大规模的问题，其稀疏结构也通常一目了然。

1.1.1 一个玩具示例

我们先考察一个简单的 SLAM 场景，以说明如何构造因子图。图 1.1 给出了该问题结构的示意图。机器人依次经过三个位姿 p_1 、 p_2 和 p_3 ，并对两个路标 ℓ_1 和 ℓ_2 进行方位观测。为了将解锚定在空间中，假设在第一个位姿 p_1 上还有一个绝对位置/方向测量。没有这一测量，就无法获得绝对位置的信息，因为方位测量都是相对的。¹

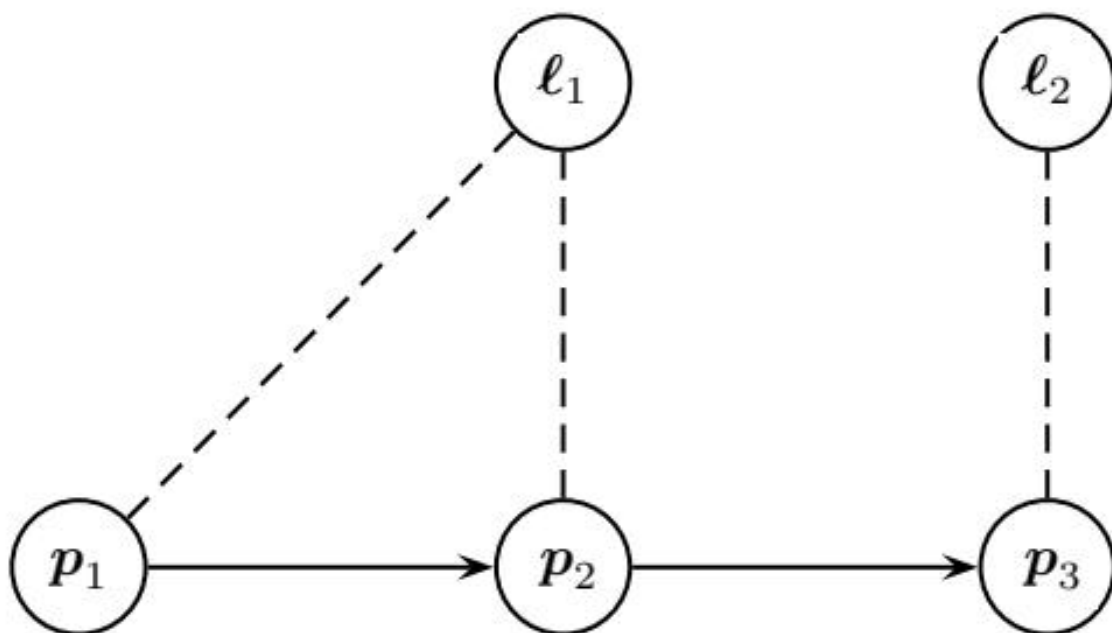


图 1.1: 图 1.1 三个位姿和两个路标构成的玩具同步定位与建图（SLAM）示例。图中用箭头示意机器人运动，用虚线表示方位测量。

由于测量存在不确定性，我们不可能恢复世界的真实状态，但可以得到一个关于测量能够推断出的内容的概率描述。在贝叶斯概率框架中，我们使用概率论的语言，为不确定事件赋予主观的可信程度。具体而言，对未知变量 $\mathbf{x}$ 定义概率密度函数（probability density function, PDF） $p(\mathbf{x})$ 。PDF 是满足下式的非负函数：

$$\int p(\mathbf{x}) d\mathbf{x} = 1, \quad (1.1)$$

这就是全概率公理。在图 1.1 的简单示例中，状态 $\mathbf{x}$ 为

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \\ \ell_1 \\ \ell_2 \end{bmatrix}, \quad (1.2)$$

也就是将各个未知量依次堆叠起来。

在 SLAM 中，给定一组观测测量 $\mathbf{z}$ 后，我们希望刻画关于未知量 $\mathbf{x}$ 的知识；在本例中，未知量包括机器人位姿和未知的路标位置。用贝叶斯概率的语言，这就是条件密度或后验密度：

$$p(\mathbf{x}|\mathbf{z}), \quad (1.3)$$

获得这样的描述称为**概率推断**。在此之前，必须先为感兴趣的变量以及它们如何产生（不确定的）测量指定概率模型。这正是概率图模型发挥作用的地方。

概率图模型利用概率分布中的结构，为复杂概率密度提供紧凑的描述机制^[592]。贝叶斯网络可能是最广为人知的图模型：它由变量节点组成，每个变量节点都关联一个先验概率密度或条件概率密度。图 1.2 展示了图 1.1 玩具示例对应的贝叶斯网络，其中明确给出了依赖于第一个位姿 $\mathbf{p}_1$ 的初始测量 z_1 ，以及 $z_2 \dots z_4$ 四个方位测量，每个测量都同时与一个位姿和一个路标相关。按照惯例，贝叶斯网络中已知量用方形节点表示，如图所示。不过，贝叶斯网络虽然非常适合建模，下面我们还要引入另一种更适合优化的图模型；它只关注问题中的未知变量。

1.1.2 因子图视角

由于局部性，高维概率密度通常可以分解为许多因子的乘积，每个因子只是定义在一个更小域上的概率密度。因子图是一类概率图模型，它允许我们将任意密度表示为因子的乘积。

为了说明因子图的动机，考虑如何对上述玩具 SLAM 示例执行推断。根据贝叶斯定律，后验 $p(\mathbf{x}|\mathbf{z})$ （式 1.3）可写为 $p(\mathbf{x}|\mathbf{z}) \propto p(\mathbf{z}|\mathbf{x})p(\mathbf{x})$ ，于是

$$p(\mathbf{x}|\mathbf{z}) \propto p(\mathbf{p}_1)p(\mathbf{p}_2|\mathbf{p}_1)p(\mathbf{p}_3|\mathbf{p}_2) \quad (1.4a)$$

$$\times p(\ell_1)p(\ell_2) \quad (1.4b)$$

$$\times p(z_1|\mathbf{p}_1) \quad (1.4c)$$

$$\times p(z_2|\mathbf{p}_1, \ell_1)p(z_3|\mathbf{p}_2, \ell_1)p(z_4|\mathbf{p}_3, \ell_2). \quad (1.4d)$$

这里假设位姿轨迹服从典型的马尔可夫链生成模型。每个因子都代表关于未知量 $\mathbf{x}$ 的一部分信息。

为了可视化这种分解，我们使用因子图。图 1.3 展示了该示例对应的因子图：所有未知状态 $\mathbf{x}$ （包括位姿和路标）都有一个对应节点。测量是已知量，因此不显式表示，也不是我们关

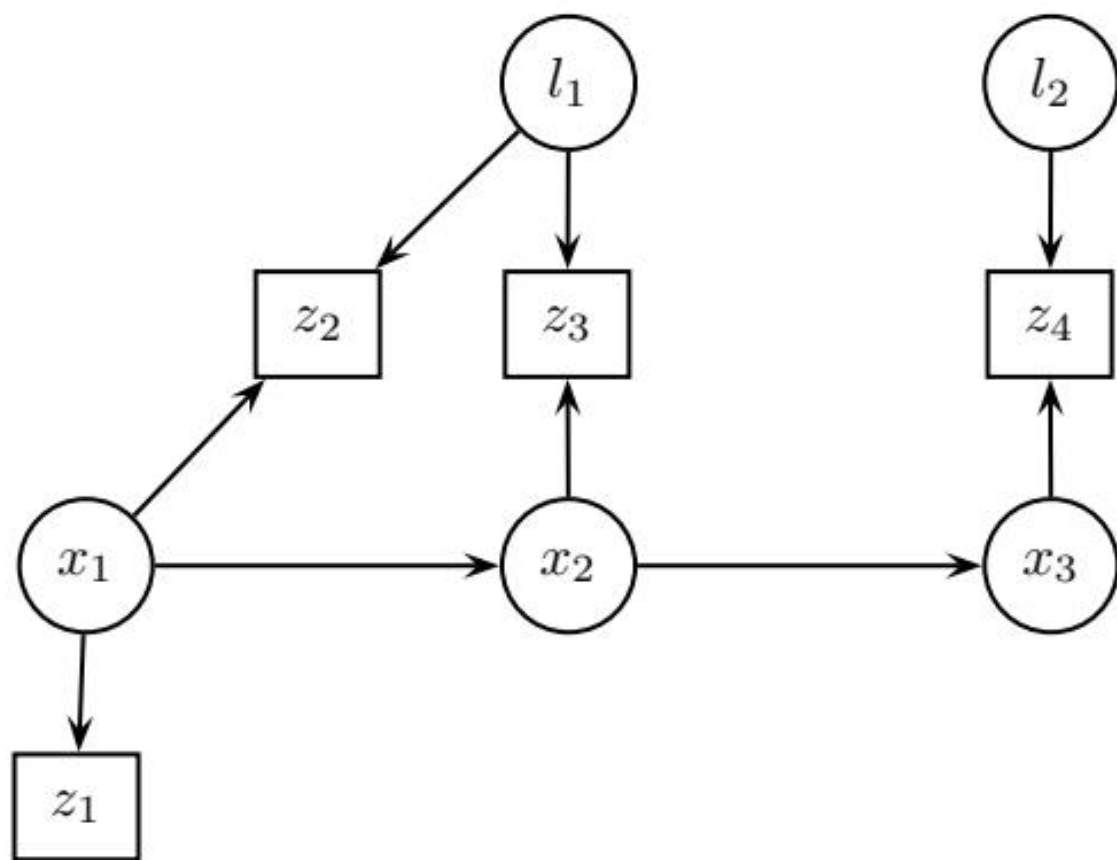


图 1.2: 图 1.2 图 1.1 示例对应的贝叶斯网络，其中将已知测量 $z_1 \dots z_4$ 明确表示为方形节点。

注的对象。在因子图中，我们显式引入另一类节点，用来表示后验 $p(\mathbf{x}|z)$ 中的每个因子。图中每个小黑节点表示一个因子；更重要的是，它只连接到该因子所依赖的状态变量。例如，因子 $\phi_9(p_3, \ell_2)$ 只连接到变量节点 p_3 和 ℓ_2 。具体地，有

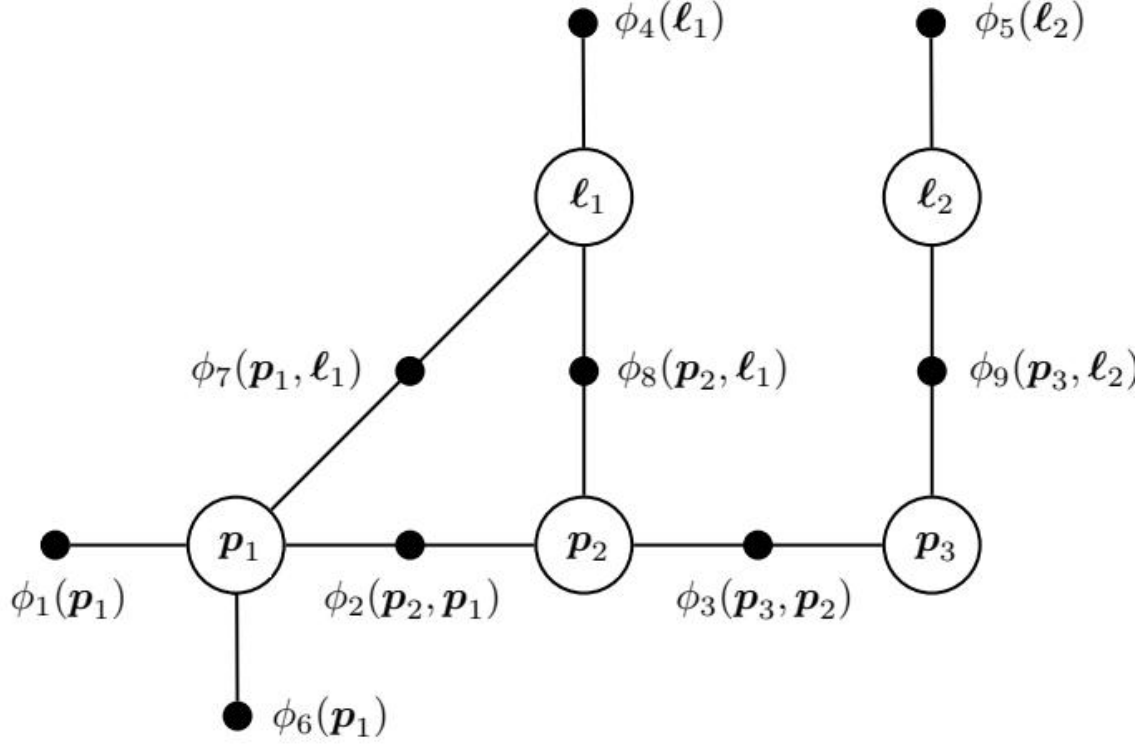


图 1.3: 图 1.3 图 1.1 示例得到的因子图。

$$\phi(p_1, p_2, p_3, \ell_1, \ell_2) = \phi_1(p_1)\phi_2(p_2, p_1)\phi_3(p_3, p_2) \quad (1.5a)$$

$$\times \phi_4(\ell_1)\phi_5(\ell_2) \quad (1.5b)$$

$$\times \phi_6(p_1) \quad (1.5c)$$

$$\times \phi_7(p_1, \ell_1)\phi_8(p_2, \ell_1)\phi_9(p_3, \ell_2), \quad (1.5d)$$

因子与式 (1.4a) — (1.4d) 中原始概率密度之间的对应关系应当是显而易见的。

因子值只需与相应概率密度成正比即可：任何与状态变量无关的归一化常数都可以省略，不会造成影响。在本例中，上述因子要么来自先验，例如 $\phi_1(p_1) \propto p(p_1)$ ；要么来自测量，例如 $\phi_9(p_3, \ell_2) \propto p(z_4|p_3, \ell_2)$ 。虽然测量变量 $z_1 \dots z_4$ 没有在因子图中显式显示，但这些因子都隐式地以它们为条件。有时为了使这一点更加明确，也可以将因子写成（例如） $\phi_9(p_3, \ell_2; z_4)$ ，甚至写成 $\phi_{z_4}(p_3, \ell_2)$ 。

1.1.3 将因子图作为一种语言

因子图除了为推断提供形式化基础之外，还能帮助我们可视化多种不同形式的 SLAM 问题、洞察问题结构，并充当一种通用语言，使不同团队的实践者能够在统一的概念下交流。图 1.3 中的每个因子都可以看作一个涉及其所连接变量的方程。通常，方程数量远多于未知量，因此必须量化先验信息和测量中的不确定性。这将导出最小二乘形式，从而适当地融合多个信息源的信息。

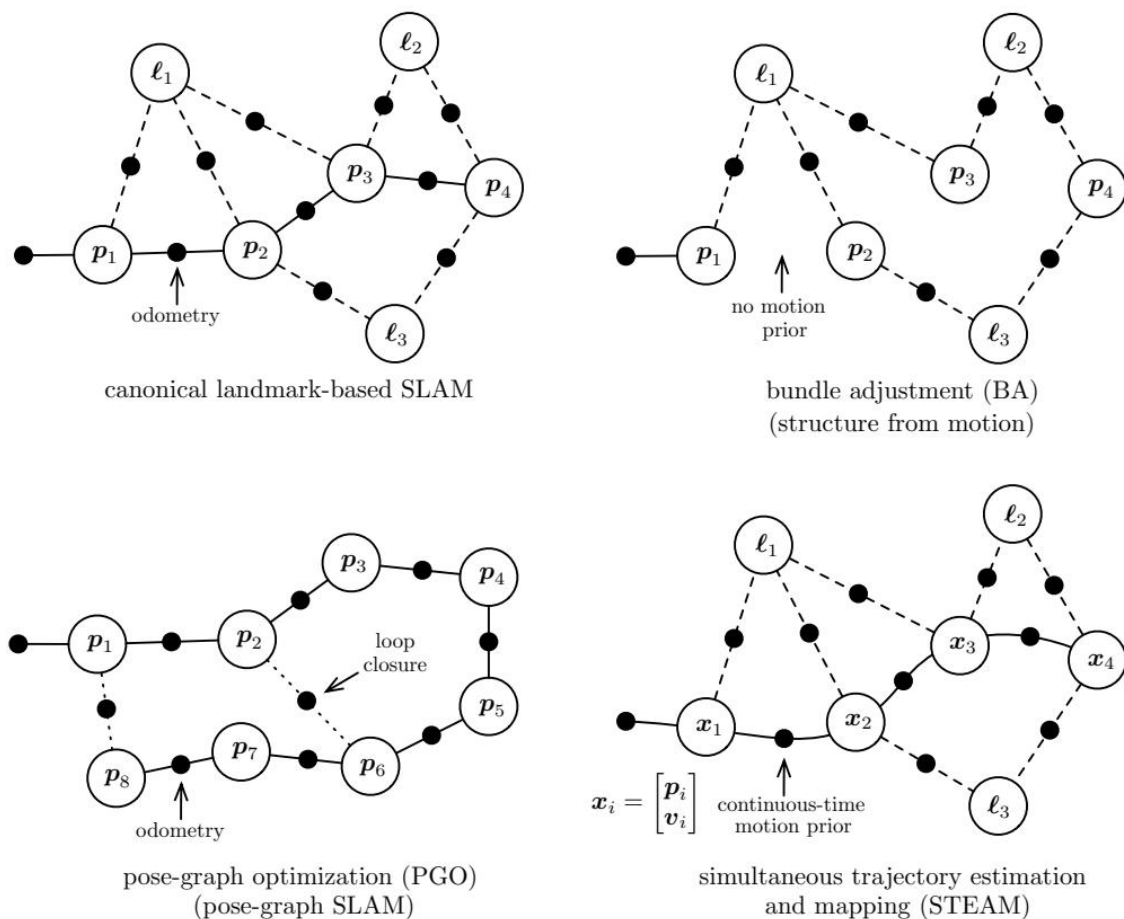


图 1.4: 图 1.4 几种都可以用因子图视角表示的 SLAM 问题变体。典型的基于路标的 SLAM 同时包含位姿变量和路标变量；从位姿观测路标，并且位姿之间通常存在基于里程计的运动先验。束调整 (BA) 与之相同，但不包含运动先验。位姿图优化 (PGO) 不含路标变量，但在位姿之间增加了回环测量。最后，同时轨迹估计与建图 (STEAM) 与基于路标的 SLAM 类似，只是将位姿替换为更高阶状态，并采用平滑的连续时间运动先验。

多种形式的 SLAM 问题都可以很容易地用因子图表示。图 1.1 是基于路标的 SLAM 示例，因为其中同时包含位姿变量和路标变量。图 1.4 给出了其他几种变体，包括束调整 (bundle adjustment, BA；与基于路标的 SLAM 相同，但没有运动模型)、位姿图优化 (pose-graph optimization, PGO；没有路标变量，但包含位姿之间的回环约束)，以及同时轨迹估计与建图 (simultaneous trajectory estimation and mapping, STEAM；在位姿中加入速度等导数项)。

比图 1.1 的玩具示例更接近实际的基于路标的 SLAM 问题，其因子图可能类似图 1.5。该图通过模拟一个在平面内运动约 100 个时间步、同时观测路标的二维机器人生成。为便于可

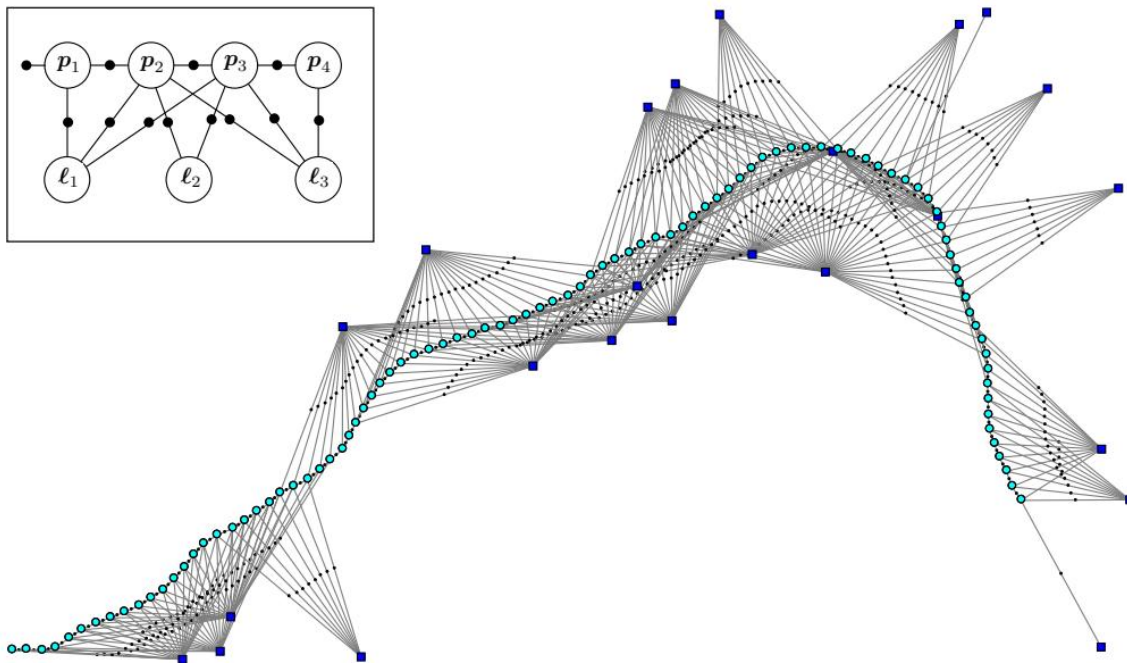


图 1.5: 图 1.5 一个规模更大的模拟 SLAM 示例因子图。

视化，每个位姿和路标都按其真实二维位置绘制。可以看到，里程计因子形成了显著的链状骨架，而在两侧则有二元²似然因子连接到约 20 个路标。在这类 SLAM 问题中，除先验之外，所有因子通常都是非线性的。

检查因子图可以揭示大量结构信息，帮助我们理解某个具体的 SLAM 问题实例。首先，有些路标拥有大量测量，我们预计它们会被非常准确地确定；另一些路标与图的联系很弱，因此估计结果也会更不确定。例如，右下方孤立的路标只有一个关联测量。

1.2 从 MAP 推断到最小二乘

最大后验 (MAP) 推断是确定未知量 $\mathbf{x}$ 取值的过程，使其与不确定测量和先验中包含的信息尽可能一致。在实际情况下，我们并不知道路标的真实位置，也不知道机器人随时间变化的位姿，尽管在许多应用中可能有一个较好的初始估计。下面将说明：在采用高斯测量噪声模型和高斯先验的条件下，MAP 推断对应的优化问题就是我们熟悉的非线性最小二乘问题。

1.2.1 用因子图进行 MAP 推断

给定测量 $\mathbf{z}$ ，我们关注位姿和/或路标等未知状态变量 $\mathbf{x}$ 。对于这些未知状态变量，最常用的估计量是最大后验 (MAP) 估计，其名称源于它最大化了给定测量 $\mathbf{z}$ 时状态 $\mathbf{x}$ 的后验密度 $p(\mathbf{x}|\mathbf{z})$ ：

$$\mathbf{x}^{\text{MAP}} = \arg \max_{\mathbf{x}} p(\mathbf{x}|\mathbf{z}) \quad (1.6a)$$

$$= \arg \max_{\mathbf{x}} \frac{p(\mathbf{z}|\mathbf{x})p(\mathbf{x})}{p(\mathbf{z})} \quad (1.6b)$$

$$= \arg \max_{\mathbf{x}} p(\mathbf{z}|\mathbf{x})p(\mathbf{x}) \quad (1.6c)$$

上式中的第二个等式是贝叶斯定律，它将后验表示为测量密度 $p(\mathbf{z}|\mathbf{x})$ 与状态先验 $p(\mathbf{x})$ 的乘积，再用 $p(\mathbf{z})$ 进行适当归一化。第三个等式省略了 $p(\mathbf{z})$ ，因为它与 $\mathbf{x}$ 无关，不会影响求 $\arg \max$ 的结果。换言之，式 (1.6c) 告诉我们：MAP 估计使似然 $p(\mathbf{z}|\mathbf{x})$ 与先验 $p(\mathbf{x})$ 的乘积最大。

我们使用因子图表示未归一化后验 $p(\mathbf{z}|\mathbf{x})p(\mathbf{x})$ 。形式上，因子图 $F = (\mathcal{U}, \mathcal{V}, \mathcal{E})$ 是一个二部图，包含两类节点：因子 $\phi_i \in \mathcal{U}$ 和变量 $\mathbf{x}_j \in \mathcal{V}$ 。边 $e_{ij} \in \mathcal{E}$ 始终连接因子节点和变量节点。与因子 ϕ_i 相邻的变量节点集合记作 $\mathcal{X}(\phi_i)$ ；我们用 $\Delta_{\mathcal{X}_i}$ 表示对该集合的一次赋值。按照这些定义，因子图 F 将全局函数 $\phi(\mathbf{x})$ 定义为

$$\phi(\mathbf{x}) = \prod_i \phi_i(\mathbf{x}_i). \quad (1.7)$$

换言之，独立关系由因子图中的边 e_{ij} 编码；每个因子 ϕ_i 只依赖于其邻接集合 $\mathcal{X}(\phi_i)$ 中的变量 $\Delta_{\mathcal{X}_i}$ 。

本章其余部分将说明，如何通过对因子图中的未知变量进行优化，找到最优赋值，即 MAP 估计。对于任意因子图，MAP 推断归结为最大化所有因子图势函数的乘积（式 1.7）：

$$\mathbf{x}^{\text{MAP}} = \arg \max_{\mathbf{x}} \phi(\mathbf{x}) \quad (1.8a)$$

$$= \arg \max_{\mathbf{x}} \prod_i \phi_i(\mathbf{x}_i). \quad (1.8b)$$

接下来需要推导因子 $\phi_i(\mathbf{x}_i)$ 的具体形式，而这在很大程度上取决于测量模型 $p(\mathbf{z}|\mathbf{x})$ 和先验密度 $p(\mathbf{x})$ 的建模方式。下面将详细讨论。

1.2.2 指定概率密度

上述密度 $p(\mathbf{z}|\mathbf{x})$ 和 $p(\mathbf{x})$ 的具体形式高度依赖于应用和所使用的传感器。最常用的密度是多元高斯密度，其概率密度为

$$\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}\|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2\right), \quad (1.9)$$

其中， $\boldsymbol{\mu} \in \mathbb{R}^n$ 为均值， $\boldsymbol{\Sigma}$ 为 $n \times n$ 协方差矩阵，并且

$$\|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2 \triangleq (\boldsymbol{\theta} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{\theta} - \boldsymbol{\mu}) \quad (1.10)$$

表示平方马氏距离。归一化常数 $\sqrt{|2\pi\boldsymbol{\Sigma}|} = (2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}$ ，其中 $|\cdot|$ 表示矩阵行列式，它保证多元高斯密度在其定义域上的积分为 1。

未知量的先验通常用高斯密度指定；在许多情况下，将测量建模为受到零均值高斯噪声污染既合理又方便。例如，给定位姿 p 到给定路标 ℓ 的方位测量³ 可建模为

$$\mathbf{z} = \mathbf{h}(p, \ell) + \boldsymbol{\eta}, \quad (1.11)$$

其中 $h(\cdot)$ 是测量预测函数，噪声 $\boldsymbol{\eta}$ 服从协方差为 Σ_R 的零均值高斯密度。于是，关于测量 $\mathbf{z}$ 的条件密度 $p(\mathbf{z}|\mathbf{p}, \ell)$ 为

$$p(\mathbf{z}|\mathbf{p}, \ell) = \mathcal{N}(\mathbf{z}; \mathbf{h}(\mathbf{p}, \ell), \Sigma_R) = \frac{1}{\sqrt{|2\pi\Sigma_R|}} \exp\left(-\frac{1}{2}\|\mathbf{z} - \mathbf{h}(\mathbf{p}, \ell)\|_{\Sigma_R}^2\right). \quad (1.12)$$

在实际机器人应用中，测量函数 $h(\cdot)$ 往往是非线性的。不过，尽管它取决于所使用的传感器和 SLAM 前端，但通常并不难理解或写出。二维方位测量的测量函数就是

$$\mathbf{h}(\mathbf{p}, \ell) = \text{atan2}(\ell_y - p_y, \ell_x - p_x), \quad (1.13)$$

其中 ℓ_x, ℓ_y 是路标的 x, y 坐标， p_x, p_y 是位姿的 x, y 坐标， atan2 是众所周知的双参数反正切函数。因此，最终的概率测量模型 $p(\mathbf{z}|\mathbf{p}, \ell)$ 为

$$p(\mathbf{z}|\mathbf{p}, \ell) = \frac{1}{\sqrt{|2\pi\Sigma_R|}} \exp\left(-\frac{1}{2}\|\mathbf{z} - \text{atan2}(\ell_y - p_y, \ell_x - p_x)\|_{\Sigma_R}^2\right). \quad (1.14)$$

需要注意的是，我们并不总是假设测量噪声为高斯噪声。例如，为了应对偶发的数据关联错误，许多作者提出使用具有更重尾部的鲁棒测量密度；第 3 章将讨论这些方法。

并非所有涉及的概率密度都来自测量。例如，在玩具 SLAM 问题中，轨迹先验 $p(\mathbf{x})$ 由先验 $p(\mathbf{p}_1)$ 和条件密度 $p(\mathbf{p}_{t+1}|\mathbf{p}_t)$ 构成，用于描述在给定已知控制输入 $\mathbf{u}_t$ 的条件下机器人所遵循的概率运动模型。实践中，我们通常采用条件高斯假设：

$$p(\mathbf{p}_{t+1}|\mathbf{p}_t, \mathbf{u}_t) = \frac{1}{\sqrt{|2\pi\Sigma_Q|}} \exp\left(-\frac{1}{2}\|\mathbf{p}_{t+1} - \mathbf{g}(\mathbf{p}_t, \mathbf{u}_t)\|_{\Sigma_Q}^2\right), \quad (1.15)$$

其中 $\mathbf{g}(\cdot)$ 是运动模型， Σ_Q 是维度适当的协方差矩阵；例如，对于在平面内运动的机器人，它是 3×3 矩阵。

我们经常没有已知的控制输入 $\mathbf{u}_t$ ，而是测量机器人如何运动，例如通过里程计测量 $\mathbf{o}_t$ 。若假设里程计只是测量位姿差，并受到协方差为 Σ_S 的高斯噪声影响，则有

$$p(\mathbf{o}_t|\mathbf{p}_{t+1}, \mathbf{p}_t) = \frac{1}{\sqrt{|2\pi\Sigma_S|}} \exp\left(-\frac{1}{2}\|\mathbf{o}_t - (\mathbf{p}_{t+1} - \mathbf{p}_t)\|_{\Sigma_S}^2\right). \quad (1.16)$$

如果同时拥有已知控制输入 $\mathbf{u}_t$ 和里程计测量 $\mathbf{o}_t$ ，就可以将式 (1.15) 与式 (1.16) 结合起来。

对于在三维空间中运行的机器人，需要使用稍微复杂一些的工具，为 SE(3) 这类非线性流形指定概率密度；下一章将对此进行讨论。

1.2.3 非线性最小二乘

现在说明，对于采用上述高斯噪声模型的 SLAM 问题，MAP 推断等价于求解非线性最小二乘问题。假设所有因子都与多元高斯分布成正比，即

$$\phi_i(\mathbf{x}_i) \propto \exp\left(-\frac{1}{2}\|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2\right). \quad (1.17)$$

这既包括简单的高斯先验因子，也包括由零均值正态分布噪声污染的测量所导出的似然因子。将式 (1.17) 代入式 (1.8b)，取负对数并去掉因子 $1/2$ ，便可转而最小化一组非线性最小二乘项之和：

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2. \quad (1.18)$$

最小化这个目标函数，就是将多个由测量得到的因子以及可能存在的多个先验结合起来，执行传感器融合，从而确定未知量的 MAP 解。

一个重要而不太显然的观察是，式 (1.18) 中的因子通常只描述所涉及未知变量 $\Delta \mathbf{x}_i$ 的部分概率。除简单的先验因子外，测量 $\mathbf{z}_i$ 的维度通常低于未知量 $\Delta \mathbf{x}_i$ 的维度。在这种情况下，单个因子会为 $\mathbf{x}_i$ 定义域中的一个无限子集赋予相同的似然。例如，相机图像中的一个二维测量，与一整条能够投影到该图像位置的三维点射线相容。

尽管函数 $\mathbf{h}_i$ 是非线性的，但如果有一个足够好的初始猜测，本章讨论的非线性优化方法通常能够收敛到式 (1.18) 的全局最小值。不过需要提醒的是，式 (1.18) 的目标函数是非凸的，因此当初始猜测较差时，无法保证不会陷入局部最小值。这促成了所谓的**可证明最优求解器**，第 6 章将对此进行介绍。下面我们先关注局部方法，而不是全局求解器；首先考虑更容易的线性化问题。

1.3 求解线性最小二乘

在处理更困难的非线性最小二乘之前，本节首先说明如何将问题线性化，展示线性化如何导出线性最小二乘问题，并回顾如何通过矩阵分解高效求解相应的正规方程。关于这些方法的经典参考是 Golub 和 Van Loan 的著作^[389]。

1.3.1 线性化

可以使用简单的泰勒展开，将非线性最小二乘目标函数 (式 1.18) 中的所有测量函数 $\mathbf{h}_i(\cdot)$ 线性化：

$$\mathbf{h}_i(\mathbf{x}_i) = \mathbf{h}_i(\mathbf{x}_i^0 + \delta_i) \approx \mathbf{h}_i(\mathbf{x}_i^0) + \mathbf{H}_i \delta_i, \quad (1.19)$$

其中，测量雅可比 $\mathbf{H}_i$ 定义为在给定线性化点 $\mathbf{x}_i^0$ 处 $\mathbf{h}_i(\cdot)$ 的 (多元) 偏导数：

$$\mathbf{H}_i \triangleq \left. \frac{\partial \mathbf{h}_i(\mathbf{x}_i)}{\partial \mathbf{x}_i} \right|_{\mathbf{x}_i^0}, \quad (1.20)$$

而 $\delta_i \triangleq \mathbf{x}_i - \mathbf{x}_i^0$ 是状态更新向量。这里假设 $\Delta \mathbf{x}_i$ 位于向量空间中，或者等价地，可以用向量表示。当 $\mathbf{x}$ 中的一些未知状态表示三维旋转或其他更复杂的流形类型时，情况并不总是如此。第 2 章将重新讨论这个问题。

将泰勒展开 (式 1.19) 代入非线性最小二乘表达式 (式 1.18)，得到关于状态更新向量 δ 的线性最小二乘问题：

$$\delta^* = \arg \min_{\delta} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0) - \mathbf{H}_i \delta_i\|_{\Sigma_i}^2 \quad (1.21a)$$

$$= \arg \min_{\delta} \sum_i \|(\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)) - \mathbf{H}_i \delta_i\|_{\Sigma_i}^2, \quad (1.21b)$$

其中, $z_i - h_i(x_i^0)$ 是线性化点处的预测误差, 即实际测量与预测测量之间的差。上式中的 δ^* 表示局部线性化问题的解。

通过一个简单的变量替换, 从这里开始可以去掉协方差矩阵 Σ_i : 将 $\Sigma^{1/2}$ 定义为 Σ 的矩阵平方根, 则平方马氏范数可以改写为:

其中

$$\|e\|_{\Sigma}^2 \triangleq e^{\top} \Sigma^{-1} e = \left(\Sigma^{-1/2} e \right)^{\top} \left(\Sigma^{-1/2} e \right) = \left\| \Sigma^{-1/2} e \right\|_2^2. \quad (1.22)$$

因此, 只需在式 (1.21b) 的每一项中, 用 $\Sigma_i^{-1/2}$ 左乘雅可比和预测误差, 就可以消除协方差矩阵 Σ_i :

$$A_i = \Sigma_i^{-1/2} H_i \quad (1.23a)$$

$$b_i = \Sigma_i^{-1/2} (z_i - h_i(x_i^0)). \quad (1.23b)$$

这个过程称为**白化**。例如, 对于标量测量, 它仅仅意味着将每一项除以测量标准差 σ_i 。注意, 这一步消除了测量的单位 (例如长度、角度), 因此不同的行可以组合到同一个代价函数中。

1.3.2 将 SLAM 写成线性最小二乘

线性化后, 最终得到如下标准最小二乘问题:

$$\delta^* = \arg \min_{\delta} \sum_i \|A_i \delta_i - b_i\|_2^2 \quad (1.24a)$$

$$= \arg \min_{\delta} \|A \delta - b\|_2^2. \quad (1.24b)$$

这里的 A 和 b 分别由收集所有白化雅可比矩阵 A_i 以及白化预测误差 b_i 得到, 并将它们组合成一个大的矩阵 A 和右端项向量 b 。

雅可比矩阵 A 是一个规模很大但稀疏的矩阵, 其分块结构与底层因子图的结构相对应。下面将详细考察这种稀疏结构。不过首先回顾经典线性代数方法, 说明如何求解这个最小二乘问题。

1.3.3 用矩阵分解求解最小二乘

对于一个满列秩的 $m \times n$ 矩阵 A ($m \geq n$), 式 (1.24b) 的唯一最小二乘解可以通过求解正规方程获得:

$$(A^{\top} A) \delta^* = A^{\top} b. \quad (1.25)$$

通常的做法是分解信息矩阵 (也称 Hessian 矩阵) Λ :

$$\Lambda \triangleq A^{\top} A = R^{\top} R. \quad (1.26)$$

这里的 Cholesky 因子 R 是一个 $n \times n$ 的上三角矩阵⁴，通过 Cholesky 分解计算得到；Cholesky 分解是针对对称正定矩阵的 LU（下三角—上三角）分解变体。随后，先求解

$$R^\top y = A^\top b \quad (1.27)$$

得到 y ，再通过前代和回代求解

$$R\delta^* = y \quad (1.28)$$

得到 δ^* 。对于稠密矩阵，Cholesky 分解需要 $n^3/3$ 次浮点运算；整个算法（包括计算对称矩阵 $A^\top A$ 的一半）需要 $(m+n/3)n^2$ 次浮点运算。还可以使用 LDL^{\{top\}} 分解，这是 Cholesky 分解的一个变体，可以避免开平方根。

另一种比 Cholesky 分解更准确、数值稳定性更好的方法是 QR 分解。该方法不必显式计算信息矩阵 $\mathbf{A}$ ，而是直接对 A 及其右端项进行 QR 分解：

$$A = Q \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix}, \quad \begin{bmatrix} d \\ e \end{bmatrix} = Q^\top b. \quad (1.29)$$

其中， Q 是 $m \times m$ 正交矩阵， $d \in \mathbb{R}^n$ ， $e \in \mathbb{R}^{m-n}$ ，而 R 是同一个上三角 Cholesky 三角因子。对于稠密矩阵 A ，通常按列从左到右计算 R 。对每一列 j ，通过用 Householder 反射矩阵 H_j 左乘 A ，将对角线下方的所有非零元素置零。经过 n 次迭代， A 完全分解为

$$H_n \dots H_2 H_1 A = Q^\top A = \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix}. \quad (1.30)$$

通常不会显式构造正交矩阵 Q ，而是将 b 作为额外一列附加到 A ，直接计算变换后的右端项 $Q^\top b$ 。由于 Q 是正交矩阵，有

$$\|A\delta - b\|_2^2 = \|Q^\top A\delta - Q^\top b\|_2^2 = \|R\delta - d\|_2^2 + \|e\|_2^2. \quad (1.31)$$

其中使用了式 (1.29) 的等式。显然， $\|e\|_2^2$ 就是最小二乘残差平方和，而最小二乘解 δ^* 可以通过回代求解三角系统

$$R\delta^* = d \quad (1.32)$$

得到。注意，QR 分解得到的上三角因子 R （对角线上的符号可能不同）与 Cholesky 分解得到的因子相同，因为

$$A^\top A = \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix}^\top Q^\top Q \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix} = R^\top R. \quad (1.33)$$

这里再次使用了 Q 为正交矩阵这一事实。QR 的计算成本主要来自 Householder 反射，为 $2(m-n/3)n^2$ 次浮点运算。与 Cholesky 相比，当 $m \gg n$ 时，两种算法都需要 $O(mn^2)$ 次运算，但 QR 分解大约慢两倍。

总之，与 SLAM 线性化优化问题相关的线性代数可以简洁地概括为：将信息矩阵 $\mathbf{A}$ 或测量雅可比矩阵 A 分解成平方根形式。由于它们都基于 SAM 问题中的矩阵平方根，我们将这类方法称为平方根 SAM，简称 SAM^[249, 254]。

1.4 非线性优化

本节讨论求解式 (1.18) 所定义非线性最小二乘问题的一些经典优化方法。回顾一下，SLAM 中的非线性最小二乘目标函数为

$$J(\mathbf{x}) \triangleq \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2. \quad (1.34)$$

它对应于一个由测量以及部分或全部未知量的先验密度共同构成的非线性因子图。

一般来说，非线性最小二乘问题无法直接求解，而需要从合适的初始估计出发迭代求解。非线性优化方法通过依次求解式 (1.18) 的线性近似，逐渐逼近最小值^[264]。不同算法在局部近似代价函数的方式，以及根据这种局部近似寻找改进估计的方式上有所不同。关于非线性求解器的深入综述见^[814]；^[389] 则从线性代数角度进行讨论。

所有算法都具有如下基本结构：从初始估计 $\mathbf{x}^0$ 出发，在每次迭代中计算更新步 δ ，并将其应用到下一次估计 $\mathbf{x}^{t+1} = \mathbf{x}^t + \delta$ 。当达到某些收敛准则时停止，例如变化量 δ 的范数低于一个很小的阈值。

1.4.1 最速下降

最速下降 (steepest descent, SD) 使用当前估计处的最速下降方向来计算更新步：

$$\delta^{\text{sd}} = -\alpha \nabla J(\mathbf{x})|_{\mathbf{x}=\mathbf{x}^t}, \quad (1.35)$$

其中 $\mathbf{x}^t$ 是当前的 x 估计。这里使用负梯度确定最速下降方向。对于非线性最小二乘目标函数 (式 1.34)，在当前点附近将目标函数近似为二次函数 $J(\mathbf{x}) \approx \|\mathbf{A}(\mathbf{x} - \mathbf{x}^t) - \mathbf{b}\|_2^2$ ，在该线性化点可以得到精确梯度

$$\nabla J(\mathbf{x})|_{\mathbf{x}=\mathbf{x}^t} = -2\mathbf{A}^\top \mathbf{b}.$$

步长 α 必须谨慎选择，以平衡更新的安全性与合理的收敛速度。可以执行显式线搜索，在给定方向上寻找最小值。SD 算法简单，但在最小值附近收敛较慢。

1.4.2 高斯—牛顿

高斯—牛顿 (Gauss—Newton, GN) 通过使用二阶更新实现更快收敛。GN 利用非线性最小二乘问题的特殊结构，用 $\mathbf{A}^\top \mathbf{A}$ 近似 Hessian 矩阵。GN 更新步通过求解正规方程 (式 1.25) 得到：

$$\mathbf{A}^\top \mathbf{A} \delta^{\text{gn}} = \mathbf{A}^\top \mathbf{b}. \quad (1.36)$$

可以使用第 1.3.3 节中的任意方法求解。对于行为良好（即近似二次）的目标函数和较好的初始估计，高斯—牛顿表现出近似二次的收敛速度。如果二次近似很差，GN 步可能使新估计远离最小值，随后发生发散。

1.4.3 Levenberg—Marquardt

Levenberg—Marquardt (LM) 算法允许多次迭代直至收敛，同时控制我们愿意信任高斯—牛顿二次近似的区域。因此，这类方法通常称为**信赖域方法**。

为了结合 SD 和 GN 的优点，Levenberg^[646] 提议在正规方程（式 1.25）的对角线上加入非负常数 $\lambda \in \mathbb{R}^+ \cup \{0\}$ ：

$$(\mathbf{A}^\top \mathbf{A} + \lambda \mathbf{I}) \delta^{\text{lm}} = \mathbf{A}^\top \mathbf{b}. \quad (1.37)$$

当 $\lambda = 0$ 时得到 GN；当 λ 很大时，近似有 $\delta^* \approx \frac{1}{\lambda} \mathbf{A}^\top \mathbf{b}$ ，即沿代价函数 J （式 1.34）的负梯度方向更新。因此，LM 可以看作在 GN 和 SD 之间自然地进行折中。

随后 Marquardt^[731] 提议考虑对角元素的尺度，以获得更快的收敛：

$$(\mathbf{A}^\top \mathbf{A} + \lambda \text{diag}(\mathbf{A}^\top \mathbf{A})) \delta^{\text{lm}} = \mathbf{A}^\top \mathbf{b}. \quad (1.38)$$

这一修改会在梯度较小（目标函数近似平坦的方向）时沿最速下降方向采取更大的步长，因为此时对角元素的逆会较大；相反，在目标函数变化陡峭的方向上，算法会更加谨慎并采取更小的步长。对正规方程的这两种修改，都可以从贝叶斯角度解释为：给问题中的所有未知变量加入零均值先验。

GN 与 LM 的一个关键区别是，LM 会拒绝使残差平方和增大的更新。被拒绝意味着非线性函数在局部表现不佳，需要采取更小的步长。实现方式是启发式地增大 λ ，例如将当前值乘以 10，然后重新求解修改后的正规方程。另一方面，如果某一步使残差平方和降低，则接受该更新，并相应地更新状态估计；此时减小 λ （例如除以 10），然后以新的线性化点重复算法，直至收敛。

1.4.4 Dogleg 最小化

Powell 的 dogleg (PDL) 算法^[885] 可以作为 LM 的更高效替代方案^[697]。Levenberg—Marquardt 算法的一个主要缺点是：如果某一步被拒绝，就必须重新分解修改后的信息矩阵，而这正是算法中最昂贵的环节。因此，PDL 的核心思想是分别计算 GN 步和 SD 步，再将二者适当地组合起来。如果 LM 步被拒绝，GN 步和 SD 步的方向仍然有效，可以用另一种方式组合，直到代价下降。因此，状态估计的每次更新只需进行一次矩阵分解，而不是多次。

图 1.6 展示了 GN 步和 SD 步的组合方式。组合步从 SD 更新开始，随后急转弯（因此称为 dogleg，即“狗腿”）朝向 GN 更新，但在信赖域边界处停止。与 LM 不同，PDL 显式维护一个信赖域，在该区域内我们相信线性假设。线性近似是否合适由增益比决定：

$$\rho = \frac{J(\mathbf{x}^t) - J(\mathbf{x}^t + \delta)}{L(\mathbf{0}) - L(\delta)}, \quad (1.39)$$

其中 $L(\delta) = \mathbf{A}^\top \mathbf{A} \delta - \mathbf{A}^\top \mathbf{b}$ 是在当前估计 $\mathbf{x}^t$ 处对式 (1.34) 非线性二次代价函数 J 的线性化。如果 ρ 很小（即 $\rho < 0.25$ ），说明代价下降幅度没有达到线性化预测，因而缩小信赖域。相反，如果实际下降与预测相符或更好（即 $\rho > 0.75$ ），则根据更新向量的大小扩大信赖域，并接受该步。

1.5 因子图与稀疏性

到目前为止介绍的求解器假设相关矩阵可以是稠密的；但在 SLAM 中，稠密方法无法扩展到实际问题规模。对于图 1.1 的玩具问题，稠密方法能够正常工作。图 1.5 所示的较大模拟问题更接近真实世界，不过对于 SLAM 而言它仍然相对较小，因为实际问题常常包含数千乃至数百万个未知量。我们之所以仍能处理这类问题，正是由于其稀疏性。

直接观察因子图即可认识到这种稀疏性。由图 1.5 可见，该图是稀疏的（远非完全连接图）。连接 100 个未知位姿的里程计链，是由 100 个二元因子构成的线性结构，而不是可能存在的 100^2 个（二元）因子。此外，对于 20 个路标，最多可以有 2000 个因子将每个路标与每个位姿相连，但实际数量更接近 400。最后，路标之间完全没有因子，这反映了我们没有获得它们相对位置的任何信息。这种结构是大多数 SLAM 问题的典型特征。

1.5.1 稀疏雅可比矩阵及其因子图

现代 SLAM 算法的关键在于利用稀疏性。SLAM 中因子图的一个重要性质是：它准确表示了最终稀疏雅可比矩阵 A 的稀疏分块结构。为此，重新考察非线性 SLAM 问题内循环中的关键计算——最小二乘问题：

$$\delta^* = \arg \min_{\delta} \sum_i \|A_i \delta_i - b_i\|_2^2. \quad (1.40)$$

上式每一项都来自原始非线性 SLAM 问题中的一个因子，并在当前线性化点附近进行线性化（式 1.21b）。矩阵 A_i 可以按变量拆分成若干分块，并收集成一个大的分块稀疏雅可比矩阵，其稀疏结构恰好由因子图给出。

尽管这些线性问题通常作为非线性优化的内部迭代出现，下面省略更新量的上标，因为相关结论对其来源无关，适用于一般线性问题。

考虑图 1.1 玩具示例的因子图。线性化后得到具有图 1.7 所示分块结构的稀疏系统 $[A \ b]$ 。将其与因子图进行比较可以发现：每个因子对应一个分块行，每个变量对应 A 的一个分块列。总共有 9 个分块行，分别对应因子分解 $\phi(p_1, p_2, p_3, \ell_1, \ell_2)$ 中的 9 个因子。

1.5.2 稀疏信息矩阵及其图

如第 1.3.3 节所述，使用 Cholesky 分解求解正规方程时，首先要形成 Hessian 矩阵或信息矩阵 $\Lambda = A^\top A$ 。注意， $A^\top A$ 并不是真正的 Hessian，而是由残差的泰勒级数截断得到的“高斯—牛顿”近似。一般而言，由于雅可比 A 是分块稀疏的，Hessian Λ 也应当是稀疏的。按构造， Λ 是对称矩阵；当唯一 MAP 解存在时，它还是正定的。

信息矩阵 Λ 的稀疏模式自然定义了一个无向图 G ：如果两个变量曾经共同出现在同一个因子中，它们之间就存在一条边。在分块层面， $\Lambda = A^\top A$ 的稀疏模式正好是该图的邻接矩阵。这一结论不限于二元因子：一个 n 元因子会在其全部变量之间诱导一个团，因此对应所有变量对之间的非零分块。

图 1.8 展示了 (a) 玩具示例信息矩阵 Λ 的分块结构，以及 (b) 对应的无向图 G 。下面经常使用与 Λ 关联的图 G 来讨论稀疏性和填充。

$$\begin{bmatrix} \Lambda_{11} & & \Lambda_{13} & \Lambda_{14} & \\ & \Lambda_{22} & & & \Lambda_{25} \\ \Lambda_{31} & & \Lambda_{33} & \Lambda_{34} & \\ \Lambda_{41} & & \Lambda_{43} & \Lambda_{44} & \Lambda_{45} \\ & \Lambda_{52} & & \Lambda_{54} & \Lambda_{55} \end{bmatrix}$$

图 1.6: 图 1.7 图 1.1 玩具 SLAM 示例的稀疏雅可比矩阵 A 的分块结构，其中空白项为零。

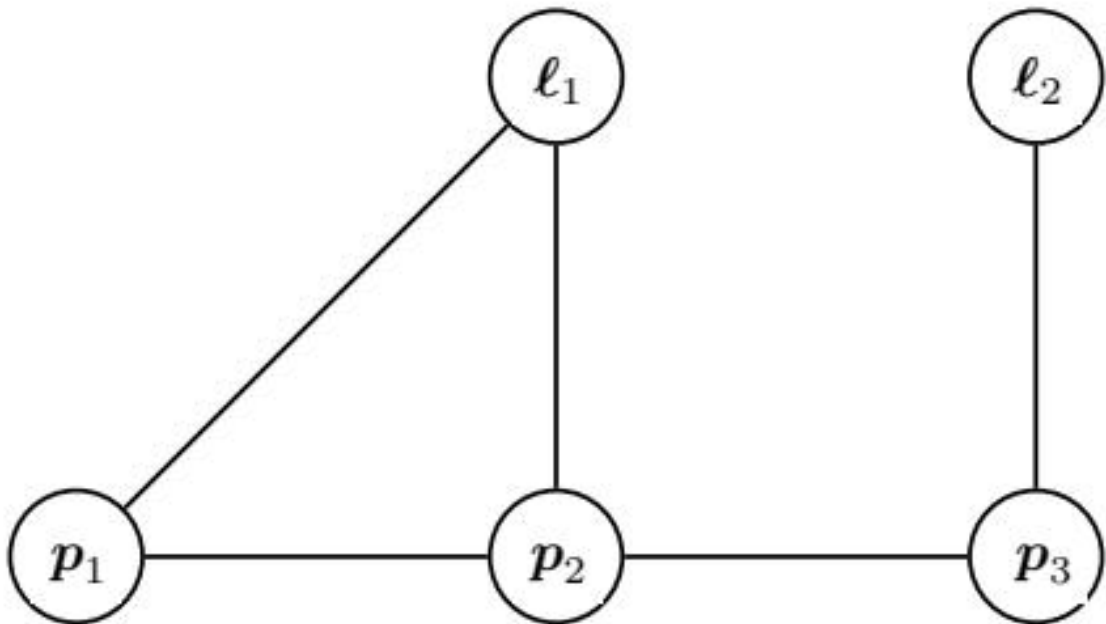


图 1.7: 图 1.8 玩具 SLAM 问题的信息矩阵 Λ 及其关联的无向图 G 。

1.5.3 稀疏分解

我们已经看到，MAP 估计等价于求解第 1.3.3 节所述的线性方程组；对于非线性最小二乘问题，则在迭代过程中反复求解这类系统。前两节说明， A 和 $A^\top A$ 都具有由因子图决定的稀疏性。无需展开细节，这种已知的稀疏模式可以显著加速 Cholesky 分解（处理 $A^\top A$ 时）或 QR 分解（处理 A 时）。例如，CHOLMOD^[189] 和 SuiteSparseQR^[240] 都提供了高效实现，许多软件包在底层使用它们。实践中，在稀疏问题上，稀疏 Cholesky 或 LDU 分解通常也优于 QR 分解，而且优势并非只是一个常数因子。

稀疏分解所需的浮点运算量远低于稠密矩阵。关键在于，稀疏矩阵采用的列排序会显著影响总计算量。任何排序最终都会产生相同的 MAP 估计，但变量排序决定矩阵因子中的填充量（即超出待分解矩阵原有稀疏模式的额外非零项）。已知寻找使矩阵分解填充最小的变量排序是 NP-hard 问题^[1239]，因此必须采用良好的启发式方法，这将进一步影响分解算法的计算复杂度。

仍以图 1.5 所示的较大模拟示例说明这一点。图 1.9 展示了对应稀疏雅可比矩阵 A 的模式，并在右上角给出信息矩阵 $\Lambda = A^\top A$ 的模式。图右侧还给出了两种不同排序下得到的上三角 Cholesky 因子 R 。两者都具有稀疏性，也都满足 $R^\top R = A^\top A$ （变量置换除外），但稀疏程度不同，而这正决定了分解 A 的代价。第一种排序最自然：先排列位姿，再排列路标，得到的稀疏 R 因子有 9399 个非零项。相比之下，右下角按列近似最小度（COLAMD）启发式方法^[29, 241] 重新排序后得到的 R 因子只有 4168 个非零项。然而，回代会为两种排序给出完全相同的解。

还可以使用预条件共轭梯度等工具迭代求解正规方程。在具有特殊稀疏模式的视觉 SLAM 中，幂迭代也已取得成功^[1171]。不过，对于大多数 SLAM 问题，稀疏分解仍然是首选方法，而且它还具有很好的图模型解释，下面将对此进行讨论。

1.6 消元

到目前为止，我们一直从线性代数角度解释 SLAM 推断。本节将视角扩展到更抽象的层次，直接利用图模型思考推断问题。这最终会引出当前最先进的 SLAM 求解器：下一节中用于增量平滑与建图的贝叶斯树。

1.6.1 变量消元算法

存在一种通用算法，可以针对任意（最好是稀疏的）因子图，计算未知变量 $\mathbf{x}$ 上相应的后验密度 $p(\mathbf{x}|\mathbf{z})$ ，并将其表示为易于恢复 MAP 解的形式。如前所述，因子图以因子乘积的形式表示未归一化后验 $\phi(\mathbf{x}) \propto p(\mathbf{x}|\mathbf{z})$ ；在 SLAM 问题中，该图通常直接由测量生成。变量消元算法提供了一套规则，可将因子图转换为另一个称为**贝叶斯网络**的图模型，该模型只依赖未知变量 $\mathbf{x}$ ，从而可以方便地进行 MAP 推断（以及采样、边缘化等其他操作）。

具体来说，变量消元算法可以将如下形式的任意因子图

$$\phi(\mathbf{x}) = \phi(\mathbf{x}_1, \dots, \mathbf{x}_n) \quad (1.41)$$

分解为具有如下形式的贝叶斯网络概率密度：

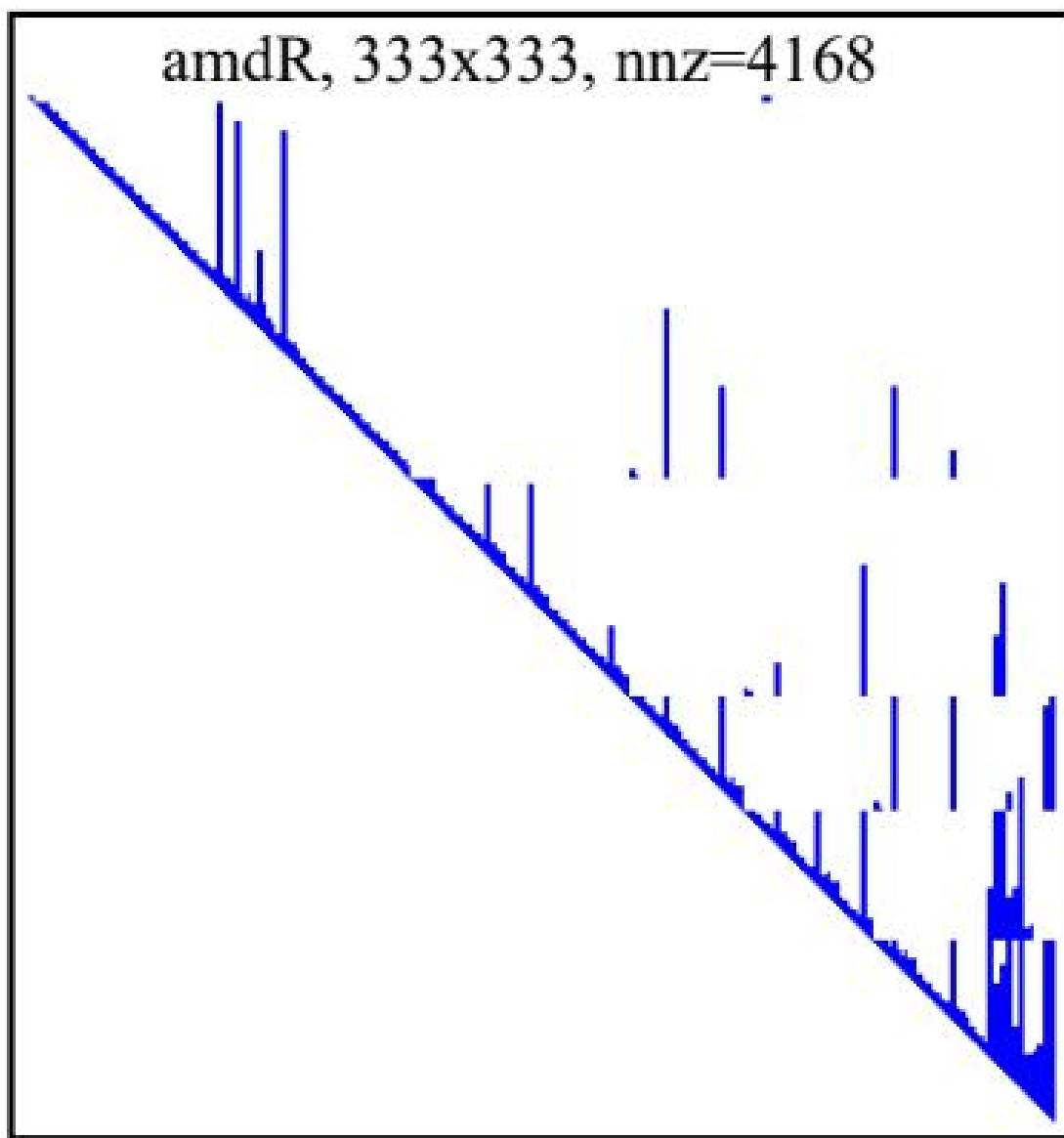


图 1.8: 图 1.9 左侧是图 1.5 问题对应的测量雅可比矩阵 A ；右侧从上到下依次为信息矩阵 $\Lambda = A^T A$ 、其上三角 Cholesky 因子 R ，以及采用更优变量排序（COLAMD）得到的替代因子 R 。

$$p(\mathbf{x}) = p(\mathbf{x}_1 | \mathbf{s}_1) p(\mathbf{x}_2 | \mathbf{s}_2) \dots p(\mathbf{x}_n) = \prod_j p(\mathbf{x}_j | \mathbf{s}_j), \quad (1.42)$$

其中, $\mathbf{s}_j$ 表示在给定变量排序 $\mathbf{x}_1, \dots, \mathbf{x}_n$ 下, 与变量 $\Delta \mathbf{x}_j$ 关联的分隔集 $\mathbf{s}(\mathbf{x}_j)$ 的一次赋值。分隔集定义为消元后 $\Delta \mathbf{x}_j$ 所依赖的变量集合。虽然这种分解类似链式法则, 但对稀疏因子图进行消元通常会得到很小的分隔集。

消元算法每次消去一个变量 $\Delta \mathbf{x}_j$, 从完整因子图 $\phi_{1:n}$ 开始。当消去每个变量时, 生成一个条件密度 $p(\mathbf{x}_j | \mathbf{s}_j)$, 以及只包含剩余变量的约简因子图 $\phi_{j+1:n}$ 。所有变量消元后, 算法返回具有所需分解形式的贝叶斯网络。

给定部分已消元的因子图 $\phi_{j:n}$, 要消除单个变量 $\Delta \mathbf{x}_j$, 首先移除所有与该变量相邻的因子 $\phi_i(\mathbf{x}_i)$, 并将它们相乘得到乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 。随后把 ψ 分解为已消元变量上的条件分布 $p(\mathbf{x}_j | \mathbf{s}_j)$ 与分隔集上的新因子 $\tau(\mathbf{s}_j)$:

$$\psi(\mathbf{x}_j, \mathbf{s}_j) = p(\mathbf{x}_j | \mathbf{s}_j) \tau(\mathbf{s}_j). \quad (1.43)$$

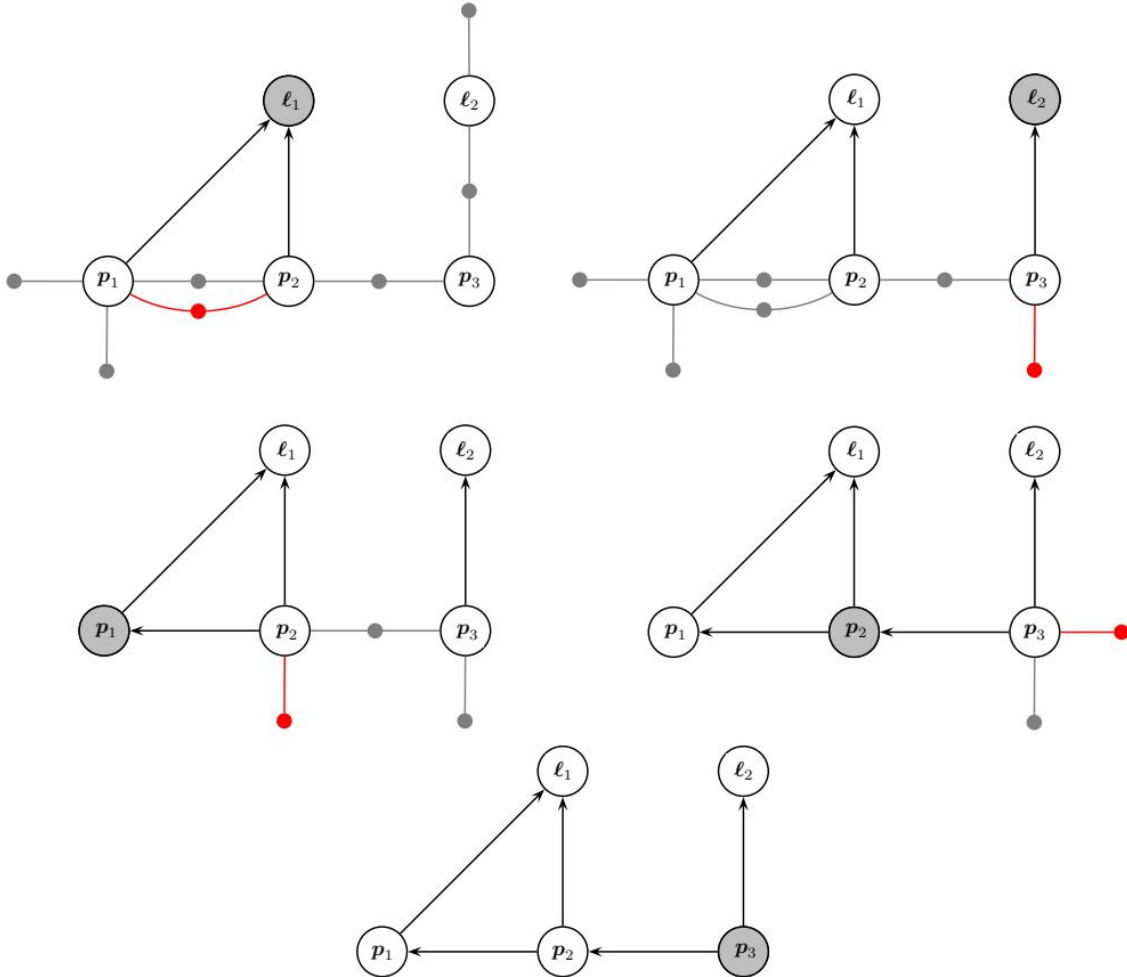


图 1.9: 图 1.10 玩具 SLAM 示例的变量消元: 按 $\ell_1, \ell_2, p_1, p_2, p_3$ 的顺序, 将图 1.3 的因子图转换为贝叶斯网络 (右下)。

因此，从 $\phi(\mathbf{x})$ 到 $p(\mathbf{x})$ 的完整分解，可以看作连续执行 n 个局部分解步骤。当消除最后一个变量 $\mathbf{x}_n$ 时，分隔集 s_n 为空，因此得到的条件密度只是对 $\mathbf{x}_n$ 的先验 $p(\mathbf{x}_n)$ 。

对于玩具示例，一种可能的消元序列如图 1.10 所示，排序为 $\ell_1, \ell_2, p_1, p_2, p_3$ 。每一步中，被消元的变量以灰色阴影表示，分隔集 s_j 上新生成的因子 $\tau(s_j)$ 以红色表示。整体而言，变量消元算法将因子图 $\phi(\ell_1, \ell_2, p_1, p_2, p_3)$ 分解为图 1.10（右下）的贝叶斯网络，对应的分解为

$$p(\ell_1, \ell_2, p_1, p_2, p_3) = p(\ell_1 | p_1, p_2) p(\ell_2 | p_3) p(p_1 | p_2) p(p_2 | p_3) p(p_3). \quad (1.44)$$

1.6.2 线性高斯消元

在线性测量函数和加性正态分布噪声的情况下，变量消元算法等价于稀疏矩阵分解；稀疏 Cholesky 分解和 QR 分解都是该通用算法的特例。

如前所述，消元算法逐个处理变量。对于每个变量 $\Delta \mathbf{x}_j$ ，移除与之相邻的所有因子 $\phi_i(\mathbf{x}_i)$ ，并形成中间乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 。只需将所有矩阵 A_i 累积到一个更大的分块矩阵 $\bar{A}_j$ 中即可：

$$\psi(\mathbf{x}_j, \mathbf{s}_j) \leftarrow \prod_{i \in \mathcal{N}_j} \phi_i(\mathbf{x}_i) \quad (1.45a)$$

$$= \exp \left(-\frac{1}{2} \sum_i \|A_i \mathbf{x}_i - \mathbf{b}_i\|_2^2 \right) \quad (1.45b)$$

$$= \exp \left(-\frac{1}{2} \|\bar{A}_j [\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j\|_2^2 \right), \quad (1.45c)$$

其中，新的右端项向量 $\bar{\mathbf{b}}_j$ 堆叠了所有 $\mathbf{b}_i$ ；符号 $[\cdot; \cdot]$ 表示按列（垂直）堆叠。

考虑在玩具示例中消去变量 ℓ_1 。与它相邻的因子为 ϕ_4, ϕ_7, ϕ_8 ，从而诱导出分隔集 $\mathbf{s}_1 = [p_1; p_2]$ 。乘积因子等于

$$\psi(\ell_1, p_1, p_2) = \exp \left(-\frac{1}{2} \|\bar{A}_1 [\ell_1; p_1; p_2] - \bar{\mathbf{b}}_1\|_2^2 \right), \quad (1.46)$$

其中

$$\bar{A}_1 \triangleq \begin{bmatrix} A_{41} & & \\ A_{71} & A_{73} & \\ A_{81} & & A_{84} \end{bmatrix}, \quad \bar{\mathbf{b}}_1 \triangleq \begin{bmatrix} \mathbf{b}_4 \\ \mathbf{b}_7 \\ \mathbf{b}_8 \end{bmatrix}. \quad (1.47)$$

观察图 1.7 的稀疏雅可比矩阵，这相当于取出第一列中存在非零分块的那些分块行，也就是与 ℓ_1 相邻的三个因子。

接下来可以用多种方式对乘积 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 进行分解。这里讨论 QR 变体，因为它与线性化因子的联系更加直接。具体而言，与乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 对应的增广矩阵 $[\bar{A}_j | \bar{\mathbf{b}}_j]$ 可使用部分 QR 分解^[389] 重写为

$$[\bar{A}_j | \bar{\mathbf{b}}_j] = Q \begin{bmatrix} R_j & T_j & d_j \\ & \tilde{A}_\tau & \tilde{\mathbf{b}}_\tau \end{bmatrix}, \quad (1.48)$$

其中 R_j 是上三角矩阵。由此， $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 可以分解为

$$\begin{aligned}
 \psi(\mathbf{x}_j, \mathbf{s}_j) &= \exp \left\{ -\frac{1}{2} \left\| \bar{\mathbf{A}}_j[\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j \right\|_2^2 \right\} \\
 &= \exp \left\{ -\frac{1}{2} \left\| \mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j \right\|_2^2 \right\} \exp \left\{ -\frac{1}{2} \left\| \tilde{\mathbf{A}}_\tau \mathbf{s}_j - \tilde{\mathbf{b}}_\tau \right\|_2^2 \right\} \\
 &= p(\mathbf{x}_j | \mathbf{s}_j) \tau(\mathbf{s}_j),
 \end{aligned} \tag{1.49a}$$

$$\left\| \bar{\mathbf{A}}_j[\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j \right\|_2^2 = \left\| \mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j \right\|_2^2 + \left\| \tilde{\mathbf{A}}_\tau \mathbf{s}_j - \tilde{\mathbf{b}}_\tau \right\|_2^2. \tag{1.49b}$$

其中利用了旋转矩阵 Q 不会改变相关范数值这一事实。

在玩具示例中，图 1.11 展示了消去第一个变量 ℓ_1 （其分隔集为 $[p_1; p_2]$ ）的结果。图中同时展示了因子图上的操作，以及对图 1.7 稀疏雅可比矩阵的相应影响（省略右端项）。分界线以上的部分对应正在形成的稀疏上三角矩阵 R ；矩阵的新贡献用蓝色表示，新生成的因子用红色表示。为完整起见，图 1.12 展示了其余四个变量消元步骤，给出了 QR 分解如何在一个小型示例上端到端执行。最后一步展示了生成的贝叶斯网络与稀疏上三角因子 R 之间的等价性。

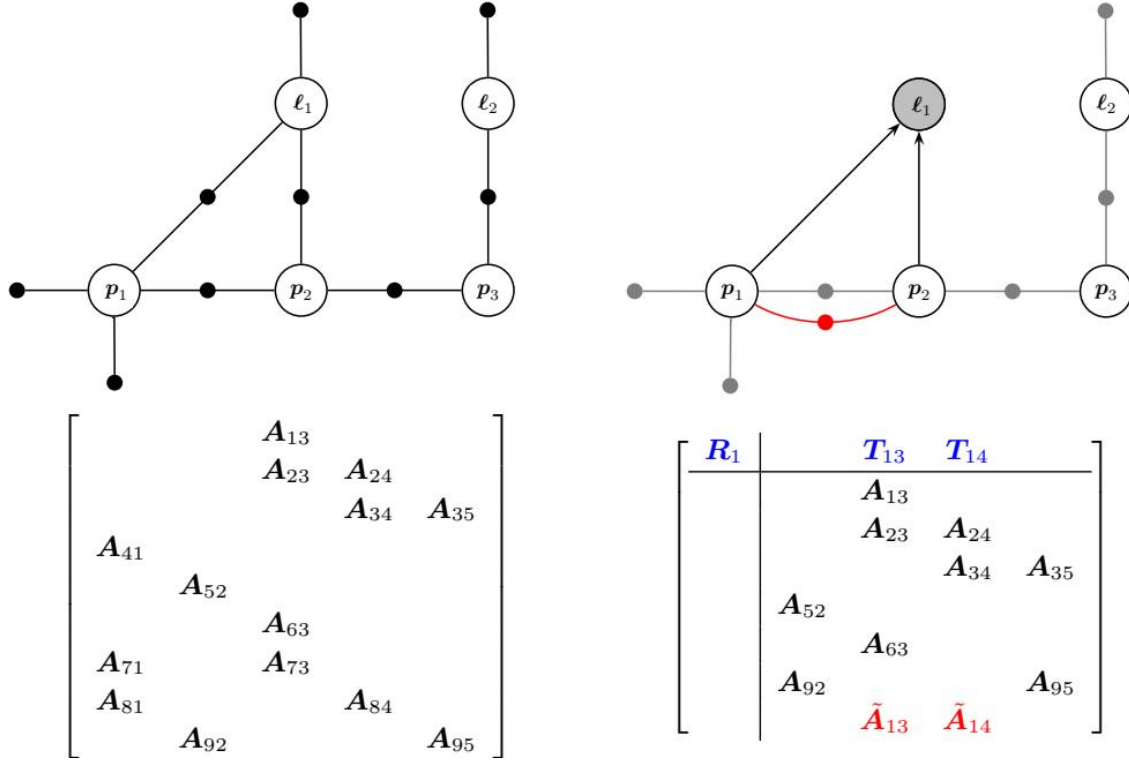


图 1.10: 图 1.11 将变量 ℓ_1 消元作为一次部分稀疏分解步骤。

使用部分 QR 分解消除单个变量的完整消元算法，等价于稀疏 QR 分解。由于上面的处理考虑了多维变量 $\mathbf{x}_j \in \mathbb{R}^{n_j}$ ，因此实际上这是多前沿 QR 分解^[288]的一个实例：每次同时消除多个标量变量，这有利于提高处理器利用率。在这里，标量变量按其在推断问题中的语义进行分组；而稀疏线性代数程序通常会按最大化计算效率的目标分析并分组问题。许多情况下，这两种策略高度一致。

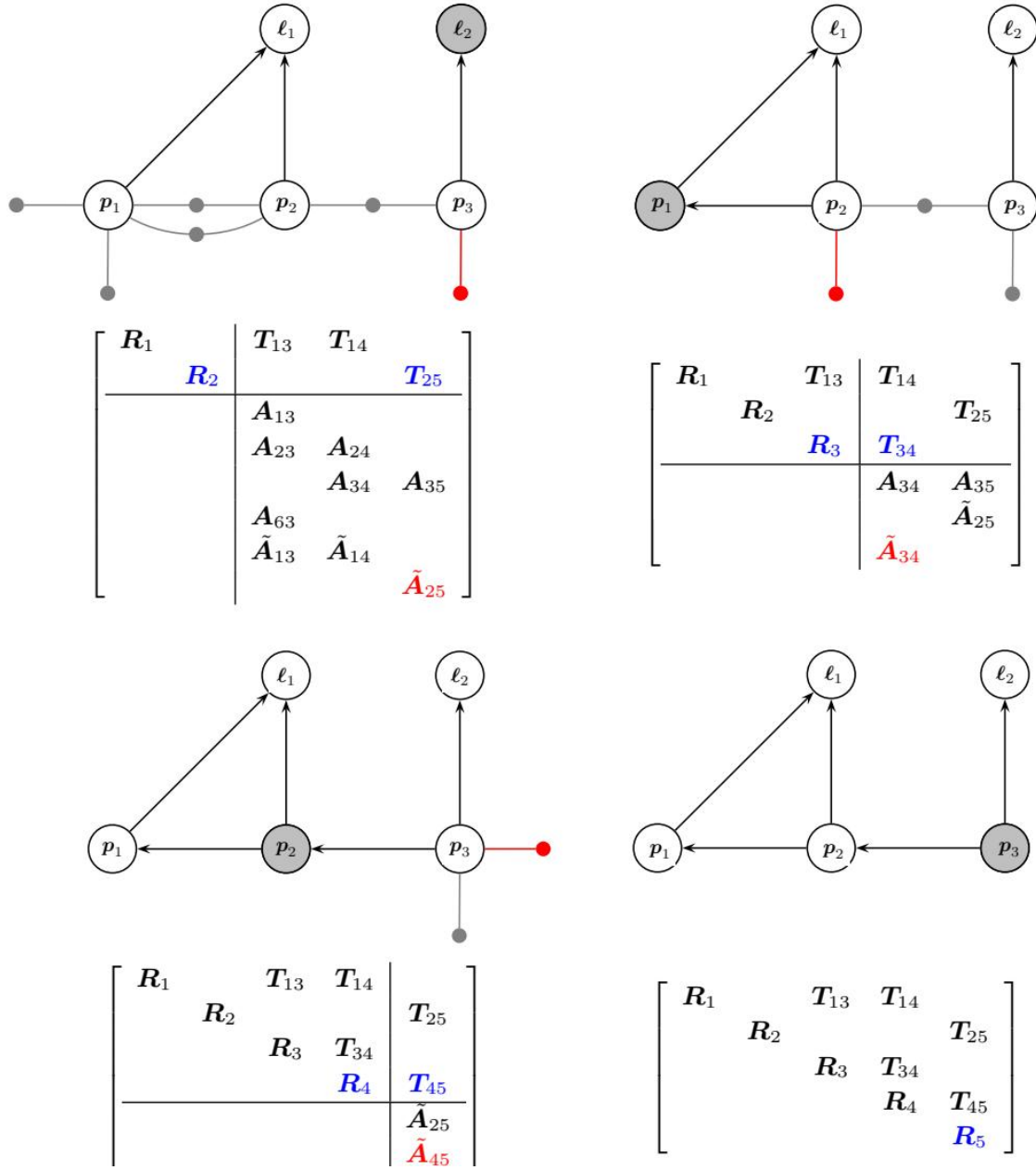


图 1.11: 图 1.12 玩具示例中剩余的消元步骤，完成一次完整 QR 分解。右下角的最后一步展示了最终贝叶斯网络与稀疏 Cholesky 因子 R 的等价性。

1.6.3 作为贝叶斯网络的稀疏 Cholesky 因子

变量消元与稀疏矩阵分解之间的等价性揭示了一个事实：与上三角矩阵关联的图模型就是贝叶斯网络！正如因子图是稀疏雅可比矩阵的图形化表示，贝叶斯网络揭示了 Cholesky 因子的稀疏结构。事后看来，这并不令人意外：贝叶斯网络是有向无环图 (DAG)，而这正对应矩阵的“上三角”性质。

此外，Cholesky 因子对应于高斯贝叶斯网络，即由线性高斯条件分布组成的贝叶斯网络。变量消元算法适用于一般密度；但当因子图只包含线性测量函数和高斯加性噪声时，得到的贝叶斯网络具有非常具体的形式。下面将讨论细节，以及在线性情形下如何求解 MAP 估计。

如前所述，与线性化非线性问题对应的高斯因子图，通过消元转化为密度 $p(\mathbf{x})$ ，其贝叶斯网络分解形式为

$$p(\mathbf{x}) = \prod_j p(\mathbf{x}_j | \mathbf{s}_j). \quad (1.50)$$

无论采用 QR 还是 Cholesky 变体，条件密度 $p(\mathbf{x}_j | \mathbf{s}_j)$ 都为

$$p(\mathbf{x}_j | \mathbf{s}_j) = k \exp \left(-\frac{1}{2} \|\mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j\|_2^2 \right), \quad (1.51)$$

这是关于已消元变量 $\mathbf{x}_j$ 的线性高斯密度。事实上，有

$$\|\mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j\|_2^2 = (\mathbf{x}_j - \boldsymbol{\mu}_j)^\top \mathbf{R}_j^\top \mathbf{R}_j (\mathbf{x}_j - \boldsymbol{\mu}_j) \triangleq \|\mathbf{x}_j - \boldsymbol{\mu}_j\|_{\Sigma_j}^2, \quad (1.52)$$

其中，均值 $\boldsymbol{\mu}_j = \mathbf{R}_j^{-1}(\mathbf{d}_j - \mathbf{T}_j \mathbf{s}_j)$ 线性依赖于分隔集 $\mathbf{s}_j$ ，协方差矩阵为 $\Sigma_j = (\mathbf{R}_j^\top \mathbf{R}_j)^{-1}$ 。因此，归一化常数为 $k = |2\pi \Sigma_j|^{-1/2}$ 。

消元步骤完成后，使用回代获得每个变量的 MAP 估计。如图 1.12 所示，最后被消元的变量不依赖于任何其他变量，因此可以直接从贝叶斯网络中提取该变量的 MAP 估计。按照消元顺序的逆序继续进行时，每个条件分布的所有分隔变量都将在前面的步骤中获得，从而可以计算当前前端变量的估计。

在每一步中，变量 $\Delta \mathbf{x}_j$ 的 MAP 估计就是条件均值：

$$\mathbf{x}_j^* = \mathbf{R}_j^{-1}(\mathbf{d}_j - \mathbf{T}_j \mathbf{s}_j^*), \quad (1.53)$$

因为到此时，分隔集 $\mathbf{s}_j$ 的 MAP 估计 $\mathbf{s}_j^*$ 已经完全确定。

1.7 增量式 SLAM

在增量式 SLAM 场景中，机器人穿行环境并不断接收新测量时，我们希望及时（或至少以固定间隔）计算最优轨迹和地图。一种方式是用新测量更新最近一次矩阵分解，复用已经纳入先前全部测量的计算结果。在线性情况下，可以通过增量分解方法实现这一点，其稠密版本在 Golub 和 Van Loan^[389] 中有详细讨论。然而，矩阵分解作用于线性系统，而实际关心的大多数 SLAM 问题都是非线性的。采用增量矩阵分解时，如何在不重新分解完整矩阵的情况下执行增量式重新线性化，并不显然。为解决这个问题，我们再次借助图模型，引入一种新的图模型——贝叶斯树。随后将说明，如何在系统加入新测量和新状态时增量更新贝叶斯树，从而得到增量平滑与建图 (iSAM) 算法。

1.7.1 贝叶斯树

众所周知，在树结构图中的推断是高效的。相比之下，典型机器人问题对应的因子图包含许多环。不过，我们可以分两步构造树结构图模型：第一步，对因子图执行变量消元，得到具有特殊性质的贝叶斯网络；第二步，利用该特殊性质，找到该贝叶斯网络中各个团之间的树结构。

通过因子图运行消元算法得到的贝叶斯网络具有一个特殊性质：它是弦图（chordal graph），即任意长度大于 3 的无向环都包含一条弦，也就是连接环上两个非相邻顶点的边。在人工智能和机器学习领域，弦图更常称为三角化图。由于它仍然是贝叶斯网络，其联合密度 $p(\mathbf{x})$ 可以在各个变量 $\Delta\mathbf{x}_j$ 上分解：

$$p(\mathbf{x}) = \prod_j p(\mathbf{x}_j | \pi_j), \quad (1.54)$$

其中， π_j 是 $\Delta\mathbf{x}_j$ 的父节点。不过，尽管贝叶斯网络是弦图，在变量层面它仍是一个非平凡图：既不是链式结构，也不是树结构。图 1.10 最后一步给出了当前玩具 SLAM 示例的弦贝叶斯网络；其中存在无向环 $\mathbf{p}_1 - \mathbf{p}_2 - \ell_1$ ，说明它不具备树结构。

通过识别这个弦图中的团，可以将贝叶斯网络改写为贝叶斯树。我们引入这种新的树结构图模型，用来捕捉贝叶斯网络中的团结构。贝叶斯网络中的团之所以能构成一棵树，并不显然；这是弦性质的结果，这里不再证明。将所有这些团列在一棵无向树上，就得到团树，在人工智能和机器学习中也称为连接树。贝叶斯树只是团树的有向版本，同时保留了消元顺序的信息。

更形式化地说，贝叶斯树是一棵有向树，其节点表示底层弦贝叶斯网络中的团 c_k 。具体而言，每个节点定义一个条件密度 $p(\mathbf{f}_k | \mathbf{s}_k)$ ，其中分隔集 $\mathbf{s}_k$ 是团 c_k 与其父团 ϖ_k 的交集 $c_k \cap \varpi_k$ 。前端变量 $\mathbf{f}_k$ 是剩余变量，即 $\mathbf{f}_k \triangleq c_k \setminus \mathbf{s}_k$ 。记号上，将一个团写成 $\mathbf{c}_k = \mathbf{f}_k : \mathbf{s}_k$ 。由贝叶斯树定义的变量联合密度为

$$p(\mathbf{x}) = \prod_k p(\mathbf{f}_k | \mathbf{s}_k). \quad (1.55)$$

对于根节点 f_r ，分隔集为空，即它只是根变量上的简单先验 $p(f_r)$ 。按照贝叶斯树的定义，团 c_k 的分隔集 $\mathbf{s}_k$ 始终是其父团 ϖ_k 的子集，因此图中的有向边与贝叶斯网络中的语义相同：表示条件依赖。

与当前玩具 SLAM 问题（图 1.3）关联的贝叶斯树如图 1.13 所示。根团 $\mathbf{c}_1 = \{\mathbf{p}_2, \mathbf{p}_3\}$ （蓝色）由 p_2 和 p_3 组成，并与另外两个团相交：绿色团 $\mathbf{c}_2 = \{\ell_1, \mathbf{p}_1\} : \mathbf{p}_2$ ，以及红色团 $\mathbf{c}_3 = \{\ell_2\} : \{\mathbf{p}_3\}$ 。颜色还表示平方根信息矩阵 R 的行如何映射到不同团，以及贝叶斯树如何捕捉它们之间的独立关系。例如，绿色和红色行只在属于根团的变量上相交，正如预期。

1.7.2 更新贝叶斯树

增量推断可以看作对贝叶斯树进行简单编辑。这种视角能够更好地解释和理解原本抽象的增量矩阵分解过程，也允许我们以贝叶斯树的形式存储和计算平方根信息矩阵，从而得到具有明确语义的稀疏存储结构。

为了增量更新贝叶斯树，我们有选择地将贝叶斯树的一部分转换回因子图形式。加入新测量时，相当于加入一个新因子；例如，涉及两个变量的测量会产生新的二元因子 $\phi(\mathbf{x}_j, \mathbf{x}_{j'})$ 。

此时，只有连接包含 x_j 和 $x_{j'}$ 的团与根节点的路径会受到影响。这些团下方的子树不受影响，其他不包含这两个变量的子树也不受影响。因此，更新贝叶斯树时，只需将受影响部分转换回因子图，加入与新测量关联的新因子，然后使用任意方便的消元顺序重新消元这个临时因子图，构造新的贝叶斯树，最后重新接回未受影响的子树。

之所以只有树的上部受到影响，可以从贝叶斯树的两个重要性质理解。第一，贝叶斯树由弦贝叶斯网络按照消元顺序的逆序构成，因此每个团中的变量通过消去子团来收集来自子团的信息；任何团中的信息只会向上方根节点传播。第二，一个因子只有在与它相连的第一个变量被消去时才会进入消元过程。结合这两个性质可知，新因子不会影响那些不是其变量后继的变量。但是，如果一个因子涉及分别沿不同（即相互独立）路径连接到根的变量，那么为了表达它们之间新产生的依赖关系，就必须重新消去这些路径。

图 1.14 展示了如何在典型 SLAM 示例中应用这些增量分解/推断步骤。在该示例中，我们在 p_1 与 p_3 之间加入一个新因子，因此只影响树的左分支（左上图红色虚线标记部分）。随后构造右上图所示因子图：为每个团密度 $p(p_2, p_3)$ 和 $p(\ell_1, p_1 | p_2)$ 建立因子，并加入新因子 $f(p_1, p_3)$ 。右下图显示了按 ℓ_1, p_1, p_2, p_3 顺序消元后得到的图。最后，左下图给出重新组装的贝叶斯树，它由消元图得到的新树和原始贝叶斯树中未受影响的团（绿色）构成。

图 1.15 展示了一个小型 SLAM 序列的贝叶斯树示例，数据取自 Olson 等人^[824]著名的“曼哈顿世界”模拟序列第 400 步。机器人探索环境时，新测量通常只影响树的一小部分，因此只需重新计算这些部分。

1.7.3 增量平滑与建图

将上述内容结合起来，并考虑重新线性化等实际问题，就得到一种最先进的增量式非线性机器人 MAP 估计方法 iSAM。第一个版本 iSAM1^[531] 使用了 Golub 和 Van Loan^[389] 的增量矩阵分解方法，但它以次优方式处理线性化：定期在完整因子图上进行线性化，和/或在矩阵填充变得难以处理时进行线性化。第二个版本 iSAM2 使用贝叶斯树表示后验密度^[534]；随后如上所述，每当接收到新测量，就通过贝叶斯树增量更新来处理。

重新消除受影响团时，应该采用怎样的变量排序？只有贝叶斯树受影响部分中的变量需要更新。一种策略是在受影响变量上局部应用 COLAMD。但还可以做得更好：将最近访问的变量强制放到排序末端，即放入根团。对于这种增量变量排序策略，可以使用约束 COLAMD 算法^[241]。它既能将最近访问的变量置于末端，又能保持总体上良好的排序。通常，后续更新只会影响树的一小部分，因此除大型回环外，大多数情况下都能高效完成。

更新树之后，还需要更新解。贝叶斯树中的回代从不依赖其他变量的根节点开始，逐步向叶节点进行。不过，通常不必重新计算所有变量的解：树的局部更新往往不会影响远处的变量。相反，在每个团处检查向下传播的变量估计变化，当该变化低于一个很小的阈值时即可停止。

引入贝叶斯树的动机，是增量求解非线性优化问题。为此，我们有选择地对包含某些变量的因子重新线性化，前提是这些变量偏离线性化点超过一个很小的阈值。与上面的树修改不同，此时必须重新处理包含受影响变量的所有团，不仅包括它们作为前端变量的情形，也包括作为分隔变量的情形。这样会影响树的更大部分，但在大多数情况下仍显著低于重建整棵树的代价。此外，我们还需要回到原始因子，而不能直接把团转换成因子图，这要求在消元时缓存某些量。完整的增量非线性算法 iSAM2 可参见^[534]。

iSAM1 和 iSAM2 已成功应用于许多不同的机器人估计问题，其中变量之间存在非平凡约束，变量数量可达数百万；后续章节还会进一步讨论。两种算法都在 GTSAM 库中实现，代码见 <https://github.com/borglab/gtsam>。

1.8 延伸阅读与近期趋势

希望深入了解因子图的读者，可以参阅 Dellaert 等人^[255]的长篇文章。由于篇幅所限，本章相对简略，并弱化了一些高级概念，以尽可能降低入门门槛。不过，因子图和一般变量消元算法都非常强大，值得系统掌握。之后建议阅读 Kaess 等人^[534]，进一步理解贝叶斯树这一支撑 GTSAM 等现代求解器的核心概念。

如今，因子图范式可以处理流形上的状态变量，融合多种传感器，处理离群测量，甚至纳入深度学习方法的输出。本手册的其余部分将大量讨论这些趋势，读者可继续阅读以了解详情。

第二章 高级状态变量表示

作者： Timothy Barfoot、Frank Dellaert、Michael Kaess、Jose Luis Blanco-Claraco

上一章详细介绍了如何采用因子图范式建立并求解 SLAM 问题。为便于说明，我们有意避开了所估计状态变量的一些细节。本章重新审视状态变量的性质，并介绍现代 SLAM 形式化中普遍存在的两个重要主题。

首先，我们需要更好的工具来处理带有特定约束的状态变量；这些约束为变量定义了一个流形，因此在优化时必须特别处理。SLAM 中存在许多流形，最常见的是与机器人旋转相关的流形（尤其是三维旋转，但平面旋转也属于这一类）。

其次，状态变量的另一个方面来自时间本身的性质。上一章隐式地假设机器人以离散时间步在世界中运动。本章将引入轨迹的平滑连续时间表示，并讨论它们如何与因子图形式完全兼容。我们以 Barfoot^[47] 为主要参考，并采用 Sola 等人^[1022] 的简化记号。

2.0.1 流形上的优化

在某些机器人问题中，可以使用向量值未知量；但在大多数实际场景中，我们必须处理三维旋转以及其他非向量流形。粗略地说，流形是一个拓扑闭合的曲面（例如圆的周长或球面）；重要的是，在每个点附近，流形局部上都类似于欧氏空间。处理流形需要更精密的数学工具，以考虑其特殊结构。本节讨论如何在流形上执行优化，这将在上一章向量空间优化框架的基础上展开。作为示例，图 2.1 对球面流形 $\mathcal{M}$ 及其切空间 $T_x\mathcal{M}$ 进行了可视化；切空间可作为 $x \in \mathcal{M}$ 处用于优化的局部坐标系。

2.0.2 旋转与位姿

在 SLAM 语境中可以讨论许多流形，但最常见的两类用于表示旋转和位姿。旋转通常发生在二维（平面）或三维空间中，因此将旋转流形称为特殊正交群 $\text{SO}(d)$ ，其中 $d = 2$ 或 3 。平面旋转矩阵 $R_a^b \in \text{SO}(2)$ 的形式为

$$R_a^b = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad (2.1)$$

其中 $\theta \in \mathbb{R}$ 是旋转角，也是该情形下唯一的自由度。此外， R_a^b 可将参考坐标系 $\mathcal{F}^a$ 中表示的二维向量（例如路标）旋转到 $\mathcal{F}^b$ 中： $\ell^b = R_a^b \ell^a$ 。

三维旋转矩阵 $R_a^b \in \text{SO}(3)$ 同样用于在两个坐标系之间旋转向量，只是此时向量处于三维空间。三维旋转矩阵有 9 个元素，但只有 3 个自由度（例如滚转、俯仰和偏航）。二维和三维旋转矩阵都必须满足约束

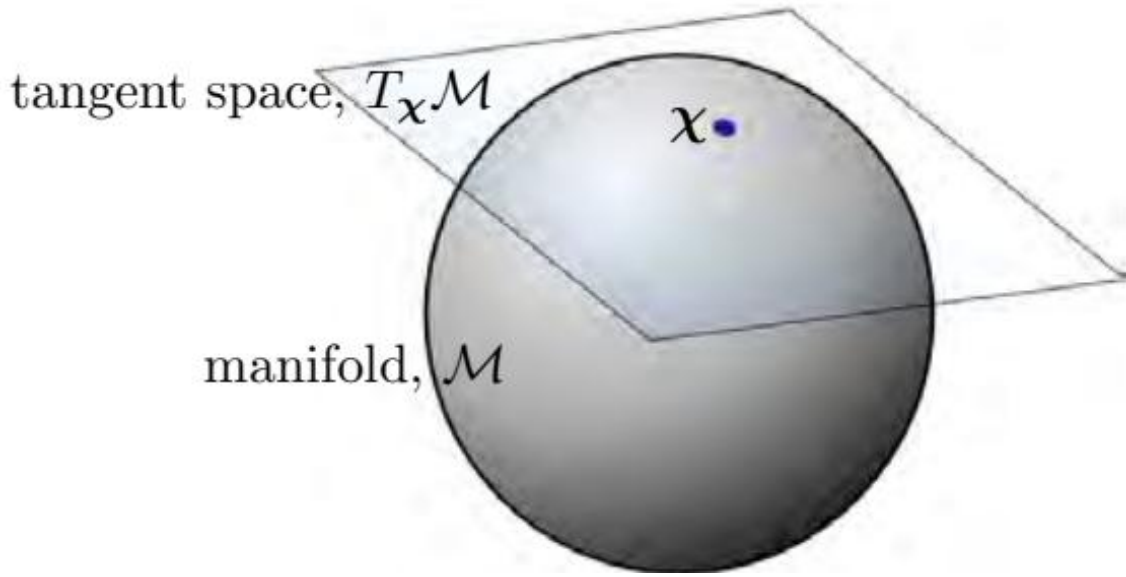


图 2.1: 图 2.1 对于球面流形 $\mathcal{M}$, $T_x \mathcal{M}$ 是局部切平面；配合局部基，可以定义局部坐标。

$$\mathbf{R}_a^{b\top} \mathbf{R}_a^b = \mathbf{I}, \quad \det(\mathbf{R}_a^b) = 1,$$

以适当地限制自由度。

机器人的位姿同时包含旋转变量 $\mathbf{R}_a^b \in \text{SO}(d)$ 和平移变量 $\mathbf{t}_a^b \in \mathbb{R}^d$ ，总共有 $3(d-1)$ 个自由度。有时我们分别跟踪这两类量，并使用 $\{\mathbf{R}_a^b, \mathbf{t}_a^b\} \in \text{SO}(d) \times \mathbb{R}^d$ 表示位姿。另一种方式是将它们组合成一个 $(d+1) \times (d+1)$ 的变换矩阵：

$$\mathbf{T}_a^b = \begin{bmatrix} \mathbf{R}_a^b & \mathbf{t}_a^b \\ \mathbf{0} & 1 \end{bmatrix}. \quad (2.2)$$

所有这类变换矩阵组成的流形称为特殊欧氏群 $\text{SE}(d)$ ，其中 $d=2$ 表示平面运动， $d=3$ 表示三维运动。使用 $\text{SE}(d)$ 的好处是，可以通过一次矩阵乘法同时对路标进行旋转和平移：

$$\underbrace{\begin{bmatrix} \ell^b \\ 1 \end{bmatrix}}_{\tilde{\ell}^b} = \underbrace{\begin{bmatrix} \mathbf{R}_a^b & \mathbf{t}_a^b \\ \mathbf{0} & 1 \end{bmatrix}}_{\mathbf{T}_a^b} \underbrace{\begin{bmatrix} \ell^a \\ 1 \end{bmatrix}}_{\tilde{\ell}^a}, \quad (2.3)$$

其中 $\tilde{\ell}$ 是路标的齐次表示。

由于旋转矩阵和变换矩阵的形式受到约束，它们并不是向量。例如，不能简单地将两个旋转矩阵相加，并期望结果仍是有效的旋转矩阵。不过， $\text{SO}(d)$ 和 $\text{SE}(d)$ 都属于具有额外有用结构的流形，即**矩阵 Lie 群**。我们可以利用这种结构，在因子图 SLAM 中继续执行无约束 MAP 优化（参见 Dellaert 等人^[255]、Boumal^[106] 或 Barfoot^[47]）。关于机器人中的 Lie 群，可参考 Chirikjian 和 Kyatkin^[207] 以及 Chirikjian^[205, 206] 的经典工作。

2.0.3 矩阵 Lie 群

在 $\text{SO}(d)$ 和 $\text{SE}(d)$ 上执行优化的关键，是利用它们的群结构。例如，矩阵 Lie 群在矩阵乘法下封闭：若 $\mathbf{R}_b^c, \mathbf{R}_a^b \in \text{SO}(d)$ ，则乘积也属于该群，且 $\mathbf{R}_a^c = \mathbf{R}_b^c \mathbf{R}_a^b \in \text{SO}(d)$ 。

矩阵 Lie 群还伴随一个非常有用的结构，称为 Lie 代数；它同时也是 Lie 群在单位元处的切空间。对我们而言，Lie 代数最重要的两个性质是：(i) 它是一个向量空间，其维度等于对应 Lie 群的自由度数量；(ii) 存在一个成熟的映射（矩阵指数）将 Lie 代数映射到 Lie 群。因此，可以从 Lie 代数元素相对容易地构造 Lie 群元素。例如，对于 $SO(2)$ ，暂时省略上下标，有

$$\mathbf{R} = \text{Exp}(\theta) = \exp(\theta^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!} (\theta^\wedge)^n = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \in SO(2), \quad (2.4)$$

其中

$$\theta^\wedge = \begin{bmatrix} 0 & -\theta \\ \theta & 0 \end{bmatrix}, \quad \theta \in \mathbb{R}. \quad (2.5)$$

量 $\theta^\wedge$ 属于 Lie 代数 $\mathfrak{so}(2)$ ，通过矩阵指数 $\exp(\cdot)$ 映射为 Lie 群元素 R 。反向也可以使用矩阵对数和 vee 算子完成： $\theta = \text{Log}(R) = (\log R)^\vee$ 。vee 算子 $(\cdot)^\vee$ 是 wedge 算子 $(\cdot)^\wedge$ 的逆。

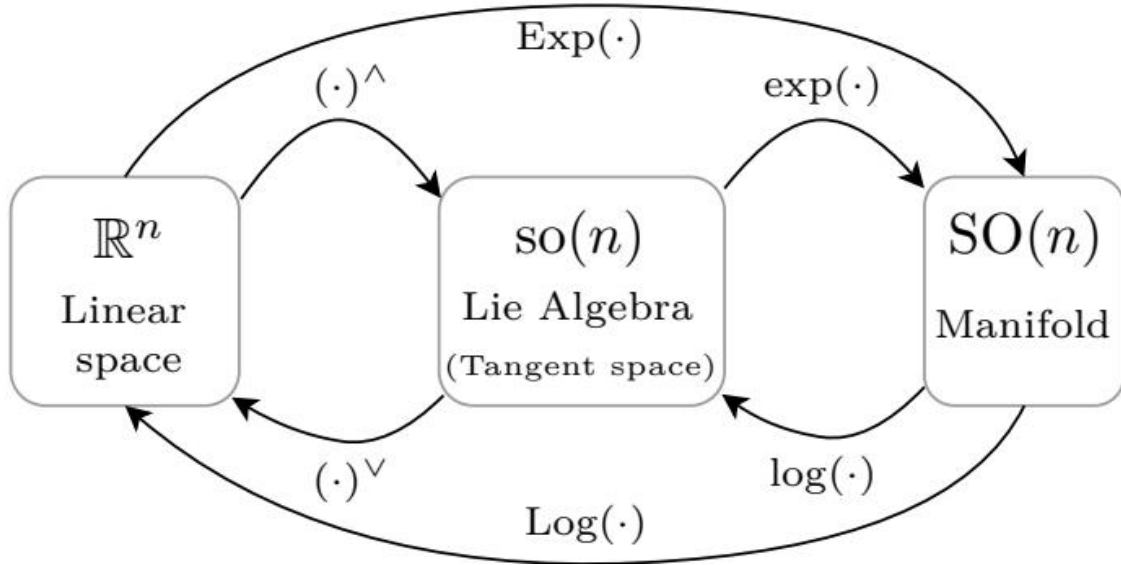


图 2.2: 本章记号概览：wedge 算子 $(\cdot)^\wedge$ 与 vee 算子 $(\cdot)^\vee$ ，指数映射 $\exp(\cdot)$ 与对数映射 $\log(\cdot)$ ，以及便捷记号 $\text{Exp}(\cdot) = \exp((\cdot)^\wedge)$ 和 $\text{Log}(\cdot) = (\log(\cdot))^\vee$ 。图中以 $SO(n)$ 为例，但这些算子对其他流形同样定义。

每个矩阵 Lie 群都有自己的线性 wedge 算子，用于从相应维度的标准向量空间构造 Lie 代数元素。对于 $SO(3)$ ，它是反对称算子：

$$\mathbf{R} = \text{Exp}(\boldsymbol{\theta}) = \exp(\boldsymbol{\theta}^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!} \boldsymbol{\theta}^{\wedge n} \in SO(3), \quad (2.6a)$$

$$\boldsymbol{\theta}^\wedge = \begin{bmatrix} 0 & -\theta_3 & \theta_2 \\ \theta_3 & 0 & -\theta_1 \\ -\theta_2 & \theta_1 & 0 \end{bmatrix} \in \mathfrak{so}(3), \quad \boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix} \in \mathbb{R}^3. \quad (2.6b)$$

对于 $SE(d)$ ，可写成

$$\mathbf{T} = \text{Exp}(\boldsymbol{\xi}) = \exp(\boldsymbol{\xi}^\wedge) \in \text{SE}(d), \quad (2.7a)$$

$$\boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\theta}^\wedge & \boldsymbol{\rho} \\ \mathbf{0} & 0 \end{bmatrix} \in \text{se}(d), \quad \boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\theta} \end{bmatrix} \in \mathbb{R}^{3(d-1)}, \quad \boldsymbol{\theta} \in \mathbb{R}^{2d-3}, \quad \boldsymbol{\rho} \in \mathbb{R}^d. \quad (2.7b)$$

这里 $d = 2$ (平面) 或 3 (三维)。注意, wedge 算子的具体版本可以由输入向量的大小确定。对于这里讨论的每个矩阵 Lie 群, Lie 代数与 Lie 群之间的映射还存在众所周知的闭式表达式, 可以替代矩阵指数的无穷级数形式 [47]。

2.0.4 Lie 群优化

建立矩阵 Lie 群之后, 就可以利用它们将非线性最小二乘项“线性化”, 从而执行 MAP 推断。回顾第 1.3.1 节, 我们仍然要线性化测量函数 $\mathbf{h}_i(\cdot)$, 只是其输入现在可能包含 Lie 群元素。

例如, 假设 $\mathbf{h}_i(\cdot)$ 是一个相机模型, 它接收在相机坐标系中表示的齐次路标 $\tilde{\ell}_i^c$, 并返回图像中的像素坐标 $\mathbf{z}_i \in \mathbb{R}^2$ 。传感器生成模型可写为

$$\mathbf{z}_i = \mathbf{h}_i(\mathbf{T}_w^c \tilde{\ell}_i^w) + \boldsymbol{\eta}_i. \quad (2.8)$$

其中, $\mathbf{T}_w^c \in \text{SE}(3)$ 是相对于相机坐标系的世界坐标系位姿, $\tilde{\ell}_i^w$ 是在世界坐标系中表示的齐次路标, $\boldsymbol{\eta}_i$ 是通常的传感器噪声。我们希望求解

$$\mathbf{T}_w^{c*} = \arg \min_{\mathbf{T}_w^c} \sum_i \left\| \mathbf{z}_i - \mathbf{h}_i(\mathbf{T}_w^c \tilde{\ell}_i^w) \right\|_{\Sigma_i}^2, \quad (2.9)$$

这就是透视 n 点 (perspective-n-point, PNP) 问题。本例假设世界坐标系中的路标位置已知; 当然, 在 SLAM 中也可能需要同时估计它们。

为了线性化传感器模型, 利用 Lie 代数产生位姿的扰动:

$$\mathbf{T}_w^c = \mathbf{T}_w^{c^0} \text{Exp}(\boldsymbol{\xi}). \quad (2.10)$$

这里 $\boldsymbol{\xi} \in \mathbb{R}^6$ 产生一个“小的”位姿变化, 用于扰动初始猜测 $\mathbf{T}_w^{c^0} \in \text{SE}(3)$ 。也常用简写记号

$$\mathbf{T}_w^c = \mathbf{T}_w^{c^0} \oplus \boldsymbol{\xi}, \quad (2.11)$$

它是式 (2.10) 的缩写。由于群的封闭性, 这两个量的乘积必然属于 $\text{SE}(3)$ 。使用 Lie 代数定义位姿扰动, 可以将其维度限制为三维位姿的实际自由度数量, 从而在优化过程中无需额外引入约束。

还可以用矩阵指数的级数形式近似扰动位姿:

$$\mathbf{T}_w^c \approx \mathbf{T}_w^{c^0} (\mathbf{I} + \boldsymbol{\xi}^\wedge). \quad (2.12)$$

其中只保留了关于 $\boldsymbol{\xi}$ 的一次项。将式 (2.12) 代入测量函数 (式 2.8), 得到

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} (\mathbf{I} + \boldsymbol{\xi}^\wedge) \tilde{\ell}_i^w \right) + \boldsymbol{\eta}_i. \quad (2.13)$$

利用齐次点的线性算子 $\odot$ ，还可写成

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} \tilde{\ell}_i^w + \mathbf{T}_w^{c^0} \tilde{\ell}_i^{w\odot} \boldsymbol{\xi} \right) + \boldsymbol{\eta}_i, \quad (2.14)$$

其中

$$\tilde{\ell}^\odot = \begin{bmatrix} \boldsymbol{\ell} \\ 1 \end{bmatrix}^\odot = \begin{bmatrix} \mathbf{I} & -\boldsymbol{\ell}^\wedge \\ \mathbf{0} & \mathbf{0} \end{bmatrix}. \quad (2.15)$$

现在还需对相机函数 $\mathbf{h}_i(\cdot)$ 进行线性化。利用一阶泰勒展开，有

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} \tilde{\ell}_i^w \right) + \underbrace{\frac{\partial \mathbf{h}_i}{\partial \boldsymbol{\ell}} \bigg|_{\mathbf{T}_w^{c^0} \tilde{\ell}_i^w}}_{\mathbf{H}_i \text{ (链式法则)}} \mathbf{T}_w^{c^0} \tilde{\ell}_i^{w\odot} \boldsymbol{\xi} + \boldsymbol{\eta}_i. \quad (2.16)$$

这里两部分相乘得到总体雅可比 $\mathbf{H}_i = \partial \mathbf{h}_i / \partial \boldsymbol{\xi}$ ，其链式法则结构十分清晰。也可以将传感器模型分解成若干基本操作，并使用中间向量状态逐步求导；对于本例，利用链式法则，关于相机位姿的雅可比可以拆成传感器模型、位姿作用、位姿复合和指数映射四项的乘积：

$$\mathbf{H}_i = \frac{\partial \mathbf{h}_i}{\partial \boldsymbol{\xi}}, \quad (2.17)$$

各个单独雅可比都有已知表达式^[87]。注意，每个雅可比的线性化点都考虑了扰动发生在 $\boldsymbol{\xi} = \mathbf{0}$ 附近这一事实。

回到式 (2.1b)，该测量对应的线性化最小二乘项（即负对数因子）为

$$\left\| \left(\mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}_w^{c^0} \tilde{\ell}_i^w \right) \right) - \mathbf{H}_i \boldsymbol{\xi} \right\|_{\Sigma_i}^2. \quad (2.18)$$

此时唯一未知量是位姿扰动 $\boldsymbol{\xi}$ 。将其与其他因子结合并求得状态变量的最优更新（包括 $\boldsymbol{\xi}^*$ ）后，需要按照式 (2.10) 的扰动方案更新初始猜测：

$$\mathbf{T}_w^{c^0} \leftarrow \mathbf{T}_w^{c^0} \oplus \boldsymbol{\xi}^*. \quad (2.19)$$

这样可以确保解始终位于 $\text{SE}(3)$ 中。优化照常迭代，直到所有状态更新（包括 $\boldsymbol{\xi}$ ）的变化足够小。式 (2.19) 适用于右侧位姿扰动；也可以选择左侧扰动，具体讨论见 Barfoot^[47]。

有时测量本身也位于流形上。例如，可能获得一个带噪位姿测量 $\mathbf{Z}_i$ ，其值属于 $\mathbf{T}_w^c \in \text{SE}(3)$ 。此时在 Lie 代数中构造误差：

$$\left\| \text{Log} \left(\mathbf{T}_w^c \mathbf{Z}_i^{-1} \right) \right\|_{\Sigma_i}^2 \approx \left\| \text{Log} \left(\mathbf{T}_w^{c^0} \mathbf{Z}_i^{-1} \right) - \mathbf{H}_i \boldsymbol{\xi} \right\|_{\Sigma_i}^2. \quad (2.20)$$

其中 $\mathbf{T}_w^c$ 按前述方式扰动， $\mathbf{H}_i$ 是该误差对应的雅可比^[47]。这样，任意类型的测量都可以在因子图范式中处理。

总之，对于属于 Lie 群的状态变量，可以通过上述方式执行无约束优化。关键是将测量函数相对于 Lie 代数中的扰动线性化，如式 (2.16) 所示。多数情况下可以解析地完成；也可以利用数值方法或自动微分计算所需雅可比。更一般地，这种针对 Lie 群元素函数的优化方

法属于黎曼优化^[106]：Lie 代数也是流形的切空间，因而优化被限制在位姿（或旋转）流形的切方向上；按式 (2.19) 更新则相当于将更新量重新收回流形。黎曼优化同样适用于并非矩阵 Lie 群的流形，且除了矩阵指数外还可以使用其他回缩映射。

2.0.5 不确定性与 Lie 群

我们通常将估计中的不确定性表示为服从某种分布的随机状态变量。第 1.2.2 节已经讨论过，高斯分布最为常用。对于向量变量 x ，可写为

$$x = \mu + \delta, \quad \delta \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad (2.21)$$

其中 μ 是均值， Σ 是协方差矩阵，即将状态拆分为均值与零均值噪声之和。

对于 Lie 群，不能简单地将噪声加到旋转矩阵 $R \in \text{SO}(d)$ 上，因为这样会破坏群的封闭性，结果不再是有效旋转矩阵。通常使用矩阵指数的满射映射，将高斯噪声 δ 与“均值”旋转 $\bar{R} \in \text{SO}(d)$ 组合：

$$R = \bar{R} \oplus \delta = \bar{R} \text{Exp}(\delta), \quad \delta \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad (2.22)$$

这样可保证 $R \in \text{SO}(d)$ 。对于 $\text{SE}(d)$ 同理：

$$T = \bar{T} \oplus \delta = \bar{T} \text{Exp}(\delta), \quad \delta \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad (2.23)$$

此时可保证 $T \in \text{SE}(d)$ ，并且 δ 与 Σ 的维度必须相应匹配。这种做法本质上是在切空间中定义高斯分布，再通过指数映射将其诱导为旋转上的噪声分布。也可以直接在旋转上定义概率分布，某些情况下这样具有计算优势；第 6 章将再次讨论。

2.0.6 Lie 群的其他工具

关于 Lie 群还可以讨论很多内容^[1033, 106]。为保持简洁，这里只汇集本章及后续章节会用到的几个有用事实。关于 Lie 群元素函数求导的更多细节将在第 4.3 节给出。

2.0.6.1 $\oplus$ 与 $\ominus$ 算子

前面已经使用 $\oplus$ 算子将 Lie 代数向量与 Lie 群元素组合。对于 $\text{SE}(d)$ ，有

$$T = T^0 \oplus \xi = T^0 \text{Exp}(\xi) = T^0 \exp(\xi^\wedge) \in \text{SE}(d). \quad (2.24)$$

还可以定义两个 Lie 群元素之间的“差”运算 $\ominus$ ：

$$T = T^0 \text{Exp}(\xi) \implies \xi = T \ominus T^0, \quad (2.25a)$$

$$\xi = T \ominus T^0 = \text{Log} \left(T^{0^{-1}} T \right) = \log \left(T^{0^{-1}} T \right)^\vee \in \text{se}(d). \quad (2.25b)$$

这些算子可以将相关运算的细节抽象出来。

2.0.6.2 逆

进行扰动时，有时需要扰动旋转或变换矩阵的逆。在 $SE(d)$ 中，直接有

$$(\mathbf{T}^0 \oplus \boldsymbol{\xi})^{-1} = \text{Exp}(\boldsymbol{\xi})^{-1} \mathbf{T}^{0^{-1}} = \text{Exp}(-\boldsymbol{\xi}) \mathbf{T}^{0^{-1}} = (-\boldsymbol{\xi}) \oplus \mathbf{T}^{0^{-1}}. \quad (2.26)$$

可以看到，扰动从右侧移到左侧，同时带有负号。

2.0.6.3 伴随

Lie 群的伴随表示，是将群元素视为其 Lie 代数线性变换的一种方式。对于 $SO(d)$ ，伴随表示与群本身相同，故略去细节。对于 $SE(d)$ ，伴随表示不同于群的基本表示，因此需要进一步说明。

给定位姿 T ， $SE(d)$ 的伴随映射将局部坐标系中的 Lie 代数元素 $\boldsymbol{\xi}^\wedge \in \mathfrak{se}(d)$ 变换为全局坐标系中的元素 $\boldsymbol{\epsilon}^\wedge \in \mathfrak{se}(d)$ ：

$$\boldsymbol{\epsilon}^\wedge \oplus T = T \oplus \boldsymbol{\xi}^\wedge, \quad (2.27)$$

其对应的内自同构（共轭）映射为

$$\boldsymbol{\epsilon}^\wedge = \text{Ad}_T \boldsymbol{\xi}^\wedge = T \boldsymbol{\xi}^\wedge T^{-1}. \quad (2.28)$$

等价地，也可以用伴随矩阵 $\text{Ad}(T)$ 将 $\boldsymbol{\xi} \in \mathbb{R}^6$ 线性变换到 $\mathbb{R}^6$ ：

$$\text{Ad}_T \boldsymbol{\xi}^\wedge = (\text{Ad}(T) \boldsymbol{\xi})^\wedge. \quad (2.29)$$

称 $\text{Ad}(T)$ 为 T 处 $SE(d)$ 的伴随表示。若

$$T = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix},$$

则

$$\text{Ad}(T) = \begin{bmatrix} \mathbf{R} & \mathbf{t}^\wedge \mathbf{R} \\ \mathbf{0} & \mathbf{R} \end{bmatrix}. \quad (2.30)$$

伴随矩阵的一个用途，是把已知变换两侧的扰动互换：

$$T \text{Exp}(\boldsymbol{\xi}) = \text{Exp}(\text{Ad}(T) \boldsymbol{\xi}) T. \quad (2.31)$$

这一等式不需要任何近似。

2.0.6.4 雅可比

每个 Lie 群都有相应的雅可比，用于联系群元素的变化与其 Lie 代数元素。以 $SO(d)$ 为例，旋转矩阵 R_a^b 与角速度 $\boldsymbol{\omega}^b \in \mathbb{R}^{2d-3}$ 之间的运动学方程（Poisson 方程）为

$$\dot{R}_a^b = \boldsymbol{\omega}^{b^\wedge} R_a^b. \quad (2.32)$$

这里 ω^b 表示坐标系 $\mathcal{F}^a$ 相对于 $\mathcal{F}^b$ 的角速度，并在 $\mathcal{F}^b$ 中表达。若将 R_a^b 参数化为 $\exp(\theta)$ ，则等价地有

$$\dot{\theta} = J^{-1}(\theta)\omega^b, \quad (2.33)$$

其中 $J(\theta)$ 是 $SO(d)$ 的（左）雅可比。它在组合矩阵指数乘积时非常有用。例如，当 θ_1 足够小时，

$$\text{Exp}(\theta_1)\text{Exp}(\theta_2) \approx \text{Exp}(\theta_2 + J(\theta_2)^{-1}\theta_1). \quad (2.34)$$

其级数形式为

$$J(\theta) = \sum_{n=0}^{\infty} \frac{1}{(n+1)!} (\theta^\wedge)^n. \quad (2.35)$$

闭式表达式可参阅 Barfoot^[47]。下文也用 $J(\xi)$ 表示 $SE(d)$ 的左雅可比，具体含义由上下文决定。Lie 群雅可比也是利用可微神经网络估计位姿的核心工具（见第 4.2 节）。

2.1 连续时间轨迹

连续时间轨迹可以表示平滑的机器人运动。到目前为止，我们假设需要估计轨迹上的一系列离散位姿；但机器人通常在世界中相当平滑地运动，因此有必要采用更平滑的轨迹表示。连续时间轨迹主要有两类：参数化方法将已知的时间基函数组合成平滑轨迹，通常选择具有局部支撑的基函数（例如分段多项式/样条），从而保持因子图稀疏；非参数化方法利用核函数获得更强的表示能力，具体可以使用以时间为自变量的一维高斯过程（GP）表示轨迹。选择合适且具有物理动机的核函数后，GP 关联的因子图同样可以保持稀疏。

连续时间轨迹不仅能保证平滑性，在处理高频传感器（如 IMU，见第 11.2 节）和/或异步传感器时尤其有用。若每收到一次测量就向因子图加入一个机器人位姿，那么高频传感器或不同传感器的异步采样会迅速产生难以处理的因子图。连续时间方法可以用远少于测量数量的变量表示轨迹，从而保持问题可处理。这对于旋转式激光雷达、雷达以及卷帘快门相机等存在运动畸变的传感器特别有价值，因为可以使用每个点或像素的精确时间戳，将其与该时刻的轨迹关联起来。

完成 MAP 推断后，连续时间轨迹还允许在任意感兴趣时刻查询轨迹，而不仅限于测量时刻；可以进行插值，也可以谨慎地外推。这对 SLAM 输出的下游使用者很有帮助。将测量时刻、估计变量时刻与查询时刻分离，是参数化和非参数化连续时间方法的主要优势。

2.1.1 样条

参数化连续时间轨迹方法的思想，是使用 K 个已知时间基函数 $\Psi_k(t)$ 的加权和表示位姿：

$$p(t) = \sum_{k=1}^K \Psi_k(t) c_k. \quad (2.36)$$

其中 c_k 是未知系数。暂时仍在向量空间中讨论，后面再介绍 Lie 群上的实现。基函数通常选用样条，即分段多项式（例如 B 样条和三次 Hermite 多项式）。样条具有局部支撑，在其

局部影响区域之外取零，因此非常有利。图 2.3 展示了这一设置：在任意时刻最多只有 4 个基函数非零，由此得到稀疏因子图。

与前面的离散时间方法相比，主要区别是使用系数变量而不是位姿变量；但这与一般因子图方法完全兼容。当在时刻 t_i 观测到路标 ℓ 时，传感器模型为

$$z_i = h_i(p(t_i), \ell) + \eta_i. \quad (2.37)$$

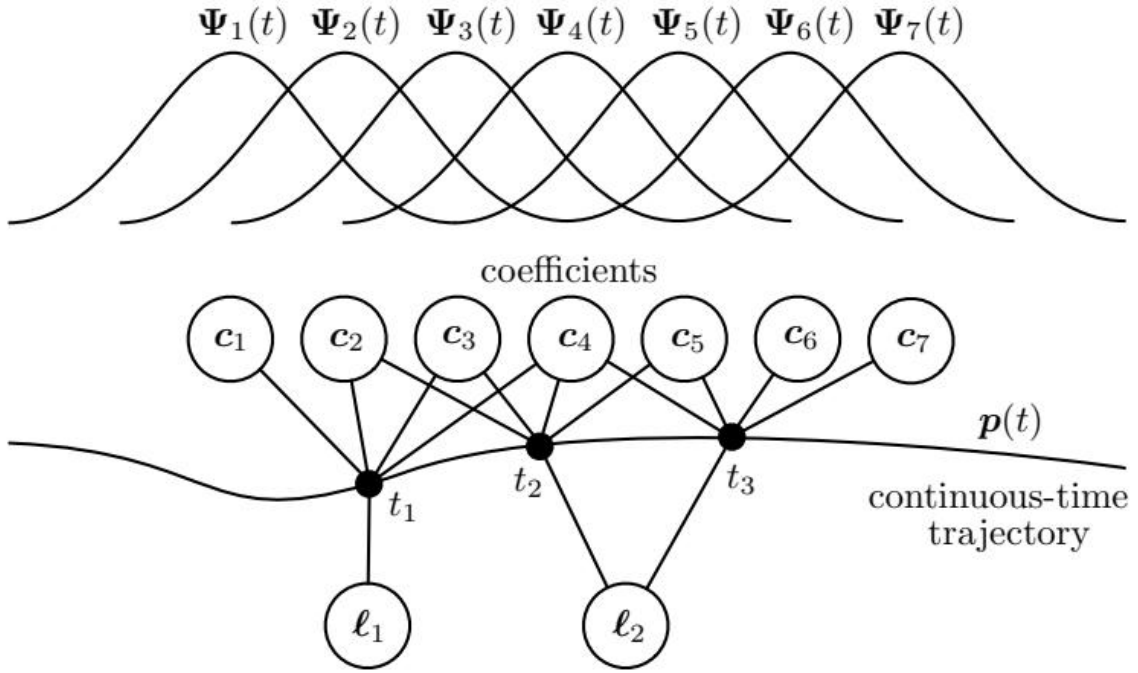


图 2.3: 图 2.3 用参数化样条表示连续时间轨迹。任意时刻的位姿由具有局部支撑的已知基函数加权求和得到；每个路标测量对应一个连接 4 个系数变量和 1 个路标变量的五元因子，整体因子图仍然非常稀疏。

代入式 (2.36)，有

$$z_i = h_i \left(\sum_{k=1}^K \Psi_k(t_i) c_k, \ell \right) + \eta_i. \quad (2.38)$$

如果基函数具有局部支撑，则在 t_i 处只有一小部分系数处于激活状态。令 $\mathbf{x}_i = [\mathbf{c}_i^\top \ell^\top]^\top$ 表示 t_i 处激活的系数变量以及路标变量，就重新得到

$$z_i = h_i(\mathbf{x}_i) + \eta_i. \quad (2.39)$$

于是可以使用一般方法构造非线性最小二乘问题并进行优化。

如果基函数足够光滑，还可以方便地求取轨迹导数：

$$\dot{\mathbf{p}}(t) = \sum_{k=1}^K \dot{\Psi}_k(t) c_k. \quad (2.40)$$

这样就能处理依赖速度甚至更高阶导数的传感器输出，而优化变量仍是同一组系数；只需计算基函数的相应导数即可。

通过 MAP 推断求得最优系数后，可以利用式 (2.36) 或式 (2.40) 在任意时刻查询轨迹及其导数。如果在推断过程中计算了系数估计的协方差（例如通过求信息矩阵的逆），还可以轻松将其映射为查询位姿或导数的协方差，因为式 (2.36) 和式 (2.40) 是线性关系；而基函数的局部支撑意味着只需使用相应的系数边缘协方差。

2.1.2 从参数化到非参数化

基本参数化连续时间方法的主要挑战，是必须决定基函数的类型和数量。基函数太多，容易对测量数据过拟合；太少，又可能没有足够能力表示真实轨迹形状，从而得到过度平滑的解。转向非参数化方法可以部分解决这一问题。

为简化说明，本节暂时假设没有路标变量，只关注位姿变量。采用上一节的参数化方法，线性化最小二乘项（负对数因子）为

$$\|(\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)) - \mathbf{H}_i \Psi_i \delta_{c,i}\|_{\Sigma_i}^2. \quad (2.41)$$

其中， $\mathbf{x}_i^0$ 是当前解（激活的系数）， $\delta_{c,i}$ 是激活系数的更新量， Ψ_i 是在 t_i 处求值并堆叠后的激活基函数，雅可比为

$$\mathbf{H}_i = \left. \frac{\partial \mathbf{h}_i}{\partial \mathbf{x}} \right|_{\mathbf{x}_i^0}. \quad (2.42)$$

像以前一样收集各项，可以写成

$$\delta_c^* = \arg \min_{\delta_c} (\|\mathbf{b} - \mathbf{A} \Psi \delta_c\|^2 + \|\delta_c\|^2). \quad (2.43)$$

这里加入了正则项 $\|\delta_c\|^2$ ，倾向于使解的描述长度保持合理，即偏好接近零的样条系数，从而帮助避免过拟合。最优解满足

$$(\Psi^\top \mathbf{A}^\top \mathbf{A} \Psi + \mathbf{I}) \delta_c^* = \Psi^\top \mathbf{A}^\top \mathbf{b}. \quad (2.44)$$

这可以求得系数的最优更新 δ_c^* 。但我们真正关心的是位姿估计，而不是作为中间量的样条系数。测量时刻位姿变量的最优更新为 $\delta^* = \Psi \delta_c^*$ 。经过简单代数运算，可得

$$(\mathbf{A}^\top \mathbf{A} + \mathbf{K}^{-1}) \delta^* = \mathbf{A}^\top \mathbf{b}, \quad (2.45)$$

这是对式 (1.25) 正规方程的修改版本。核矩阵 $\mathbf{K} = \Psi \Psi^\top$ 起到正则化或平滑函数的作用。式 (2.45) 看起来比式 (2.44) 包含更大的线性系统，因为位姿数量多于样条系数。不过，非参数方法可以利用内置的插值能力，最终减少实际需要求解的系统规模。

为了摆脱显式基函数，可以使用所谓的核技巧，用所选核函数 $\mathcal{K}(t, t')$ （例如平方指数核）的计算结果替代基函数的显式内积。构造核矩阵只需要计算基函数内积，因此令 $\mathbf{K} = [\mathcal{K}(t_i, t_j)]_{ij}$ 即可。此时不再估计样条的系数（参数），因而称为非参数方法；但仍需调节所选核函数的超参数（例如平方指数核的长度尺度），以实现期望的轨迹平滑程度。由于式 (2.45) 需要核矩阵的逆，直接这样做似乎代价很高；下一节将说明如何选择核函数，使逆核矩阵非常稀疏。

2.1.3 高斯过程

下面构造一族经过专门设计的核函数，使逆核矩阵及其对应因子图保持稀疏。上一节说明，可以用核函数替换基函数，从而得到非参数连续时间方法；但若直接进行替换，逆核矩阵可能变得稠密。这里从另一条思路出发：图 2.4 给出了一个由本节思想得到的因子图示例，它既保持稀疏，又产生平滑轨迹。

首先选择一个由白噪声驱动的线性时不变随机微分方程 (SDE)：²

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{L}\mathbf{w}(t), \quad (2.46)$$

其中 $\mathbf{w}(t) = \mathcal{GP}(\mathbf{0}, \mathbf{Q}\delta(t-t'))$ 是零均值白噪声高斯过程， $\mathbf{Q}$ 是功率谱密度矩阵， $\delta(\cdot)$ 是 Dirac delta 函数。该 SDE 将充当运动先验。它可以写出闭式积分解：

$$\mathbf{x}(t) = \Phi(t, t_1)\mathbf{x}(t_1) + \int_{t_1}^t \Phi(t, s)\mathbf{L}\mathbf{w}(s)ds, \quad (2.47)$$

其中 $\Phi(t, s) = \exp(\mathbf{A}(t-s))$ 称为转移函数， t_1 是第一次测量的时间戳。若初始状态均值为零，则后续状态均值也保持为零；状态的协方差函数（即核函数）为

$$\mathcal{K}(t, t') = \Phi(t, t_1)\mathcal{K}(t_1, t_1)\Phi(t', t_1)^\top + \int_{t_1}^{\min(t, t')} \Phi(t, s)\mathbf{L}\mathbf{Q}\mathbf{L}^\top\Phi(t', s)^\top ds. \quad (2.48)$$

在全部测量时刻计算核函数并构造核矩阵时，可以利用简洁关系

$$\mathbf{K} = \Phi\mathbf{Q}\Phi^\top, \quad (2.49)$$

其中 $\mathbf{Q} = \text{diag}(\mathcal{K}(t_1, t_1), Q_1, \dots, Q_M)$ ，且

$$\Phi = \begin{bmatrix} \mathbf{I} & & & & \\ \Phi(t_2, t_1) & \mathbf{I} & & & \\ \Phi(t_3, t_1) & \Phi(t_3, t_2) & \mathbf{I} & & \\ \vdots & \vdots & \vdots & \ddots & \\ \Phi(t_M, t_1) & \Phi(t_M, t_2) & \cdots & \Phi(t_M, t_{M-1}) & \mathbf{I} \end{bmatrix}. \quad (2.50)$$

其中

$$Q_i = \int_{t_{i-1}}^{t_i} \Phi(t_i, s)\mathbf{L}\mathbf{Q}\mathbf{L}^\top\Phi(t_i, s)^\top ds.$$

矩阵 Φ 是由各时间点之间的转移函数构成的下三角块矩阵，其对角块为单位阵。由于我们需要的是逆核矩阵， $\mathbf{K}^{-1} = \Phi^{-\top}\mathbf{Q}^{-1}\Phi^{-1}$ ；而 $\mathbf{Q}$ 是块对角矩阵，逆可以逐个对角块计算。更重要的是， Φ^{-1} 只有主块对角线和紧邻其下的一个块对角线非零：

$$\Phi^{-1} = \begin{bmatrix} \mathbf{I} & & & & \\ -\Phi(t_2, t_1) & \mathbf{I} & & & \\ & -\Phi(t_3, t_2) & \mathbf{I} & & \\ & & -\Phi(t_4, t_3) & \ddots & \\ & & & \ddots & \mathbf{I} \\ & & & & -\Phi(t_M, t_{M-1}) & \mathbf{I} \end{bmatrix}. \quad (2.51)$$

因此，任意长度轨迹的逆核矩阵 K^{-1} 都是块三对角的。根据前面对因子图的讨论，式 (2.45) 左端的稀疏性与因子图结构密切相关；这里 K^{-1} 作为整条轨迹的运动先验，可以用非常稀疏的因子图表示。

K^{-1} 之所以具有这种稀疏因子图，是因为出发点 SDE 的状态具有马尔可夫性。实际含义是：根据式 (2.46) 想表达的运动先验，状态可能需要包含更高阶量，而不只是位姿，还包括其导数。例如，若希望采用“恒速”先验，可以选择

$$\underbrace{\begin{bmatrix} \dot{\mathbf{p}}(t) \\ \dot{\mathbf{v}}(t) \end{bmatrix}}_{\dot{\mathbf{x}}(t)} = \underbrace{\begin{bmatrix} \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} \mathbf{p}(t) \\ \mathbf{v}(t) \end{bmatrix}}_{\mathbf{x}(t)} + \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{I} \end{bmatrix}}_{\mathbf{L}} \mathbf{w}(t). \quad (2.52)$$

这里的状态包含位姿及其导数。这种使用增广状态的形式有时称为同时轨迹估计与建图 (STEAM)，是 SLAM 的一种变体。

上述连续时间轨迹估计实际上是高斯过程回归^[909]的一个例子。建立这一联系后，在测量时刻求解状态，就可以利用 GP 插值在其他时刻查询轨迹的均值和协方差。采用稀疏核方法时，每次查询的代价相对于测量数量 M 为常数，因为只涉及夹住查询时刻的两个估计状态。

还可以利用 GP 插值减少所需控制点的数量，不必在每个测量时刻都设置控制点，这类似于 GP 的诱导点思想。例如，每次激光扫描设置一个控制点，但仍使用扫描过程中每个点的独立时间戳。这样，测量时刻、估计时刻和查询时刻可以彼此不同。GP 的核还提供了适当的正则化，因此无需担心设置过多估计时刻导致过拟合，但仍需有足够的估计时刻来捕捉轨迹细节。

2.1.4 Lie 群上的样条与高斯过程

当状态位于流形上时，同样可以使用样条和 GP 连续时间方法。若流形是 Lie 群，两种方法都利用 Lie 代数，但方式不同。下面先介绍样条，再介绍高斯过程。

2.1.4.1 Lie 群上的样条

使样条适用于 Lie 群的关键，是采用累积形式。对于向量空间，假设所有自由度使用相同基函数，式 (2.36) 可简化为

$$\mathbf{p}(t) = \sum_{k=1}^K \psi_k(t) \mathbf{p}_k, \quad (2.53)$$

其中 $\mathbf{p}_k$ 是样条控制点, $\psi_k(t)$ 是标量基函数。将其改写为累积形式:

$$\mathbf{p}(t) = \psi_1^c(t)\mathbf{p}_1 + \sum_{k=2}^K \psi_k^c(t)(\mathbf{p}_k - \mathbf{p}_{k-1}), \quad (2.54)$$

其中累积基函数为

$$\psi_k^c(t) = \sum_{\ell=k}^K \psi_\ell(t). \quad (2.55)$$

例如, 对时间间隔均匀为 T 的线性插值, 定义 $\alpha_k(t) = (t - (k-1)T)/T$, 普通基函数为

$$\psi_1(t) = \begin{cases} 1 - \alpha_1(t), & 0 \leq t < T \\ 0, & \text{其他情况,} \end{cases} \quad \psi_K(t) = \begin{cases} \alpha_K(t), & (K-1)T \leq t < KT \\ 0, & \text{其他情况,} \end{cases} \quad (2.56)$$

而对 $k = 2, \dots, K-1$,

$$\psi_k(t) = \begin{cases} \alpha_{k-1}(t), & (k-2)T \leq t < (k-1)T \\ 1 - \alpha_k(t), & (k-1)T \leq t < kT \\ 0, & \text{其他情况.} \end{cases} \quad (2.57)$$

对应的累积基函数为

$$\psi_1^c(t) = 1, \quad \psi_K^c(t) = \begin{cases} 0, & t < (K-1)T \\ \alpha_K(t), & (K-1)T \leq t, \end{cases} \quad (2.58)$$

以及对 $k = 2, \dots, K-1$,

$$\psi_k^c(t) = \begin{cases} 0, & t < (k-1)T \\ \alpha_k(t), & (k-1)T \leq t < kT \\ 1, & kT \leq t. \end{cases} \quad (2.59)$$

图 2.5 展示了这些基函数的形状。

累积基函数的主要优点是: 在给定时间戳处, 大多数基函数都不激活。对于线性样条, 当 $(k-1)T \leq t < kT$ 时,

$$\mathbf{p}(t) = \mathbf{p}_{k-1} + \psi_k^c(t)(\mathbf{p}_k - \mathbf{p}_{k-1}). \quad (2.60)$$

此时只需计算一个基函数; 更高阶样条在任意时刻也只会少量激活基函数。

在 Lie 群上使用样条时, 用 Lie 群运算 (矩阵乘法) 替代累积表达式中的加法。以线性样条为例, 在 $(k-1)T \leq t < kT$ 时, $\text{SE}(d)$ 中的元素可以写成

$$\mathbf{T}(t) = \text{Exp}(\psi_k^c(t)\text{Log}(\mathbf{T}_k\mathbf{T}_{k-1}^{-1}))\mathbf{T}_{k-1}. \quad (2.61)$$

这里选择了左侧扰动, 当然也可以采用右侧扰动。将 $\mathbf{T}(t_i)$ 代入任意测量表达式, 在控制点 $\mathbf{T}_k$ 的 Lie 代数扰动上进行线性化, 即可用于 MAP 框架。由于基函数具有紧支撑, 在一个测量时刻只有少量控制点激活。

对于线性样条，式 (2.61) 可写为

$$\mathbf{T}(t) = (\mathbf{T}_k \mathbf{T}_{k-1}^{-1})^{\alpha_k(t)} \mathbf{T}_{k-1}. \quad (2.62)$$

对 $\mathbf{T}(t)$ 中的表达式进行线性化时，分别扰动两个控制位姿，使

$$\text{Exp}(\boldsymbol{\xi}(t)) \mathbf{T}^0(t) = \left(\text{Exp}(\boldsymbol{\xi}_k) \mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \text{Exp}(-\boldsymbol{\xi}_{k-1}) \right)^{\alpha_k(t)} \text{Exp}(\boldsymbol{\xi}_{k-1}) \mathbf{T}_{k-1}^0. \quad (2.63)$$

插值位姿扰动 $\boldsymbol{\xi}(t)$ 与两个控制点扰动 $\boldsymbol{\xi}_k$ 、 $\boldsymbol{\xi}_{k-1}$ 的关系可近似为

$$\boldsymbol{\xi}(t) \approx (\mathbf{I} - \mathbf{A}(\alpha_k(t))) \boldsymbol{\xi}_{k-1} + \mathbf{A}(\alpha_k(t)) \boldsymbol{\xi}_k, \quad (2.64)$$

其中

$$\mathbf{A}(\alpha_k(t)) = \alpha_k(t) \mathbf{J} \left(\alpha_k(t) \mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \right) \mathbf{J} \left(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \right)^{-1}. \quad (2.65)$$

这里 $\mathbf{J}(\cdot)$ 是 $\text{SE}(d)$ 的左雅可比。利用式 (2.64)，即可将测量时刻位姿的变化与两侧控制点位姿关联起来，形成用于 MAP 估计的线性化误差。例如，式 (2.16) 的线性化测量模型可写成

$$\mathbf{e}_i(t) \approx \mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}^0(t) \tilde{\ell}_i \right) - \mathbf{H}_i \boldsymbol{\xi}(t). \quad (2.66)$$

将式 (2.64) 代入即可得到关于两侧控制点的线性化误差：

$$\mathbf{e}_i(t) \approx \mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}^0(t) \tilde{\ell}_i \right) - \mathbf{H}_i (\mathbf{I} - \mathbf{A}(\alpha_k(t))) \boldsymbol{\xi}_{k-1} - \mathbf{H}_i \mathbf{A}(\alpha_k(t)) \boldsymbol{\xi}_k. \quad (2.67)$$

其中还需将 t 时刻的名义位姿 $\mathbf{T}^0(t) = (\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}})^{\alpha_k(t)} \mathbf{T}_{k-1}^0$ 代入 $\mathbf{h}_i$ 和 $\mathbf{H}_i$ 。这相当于沿线性样条链式传播导数；高阶样条也可以采用同样过程。

2.1.4.2 Lie 群上的高斯过程

在 Lie 群上使用高斯过程时，同样利用 Lie 代数。图 2.6 展示了相应的 GP 运动先验因子。根据所选运动先验，控制点状态也可以包含额外的轨迹导数。

我们在一组控制点状态之间使用局部 GP，这与样条类似。对 $\text{SE}(d)$ ，定义控制点之间的局部变量 $\boldsymbol{\xi}_k(t)$ 。例如，对 $\text{SE}(d)$ 的“随机游走”先验，可以选择

$$\dot{\boldsymbol{\xi}}_k(t) = \mathbf{w}(t), \quad \mathbf{w}(t) = \mathcal{GP}(\mathbf{0}, \mathbf{Q} \delta(t - t')), \quad (2.68)$$

这里的变量定义在控制点 $\mathbf{T}_k$ 和 $\mathbf{T}_{k+1}$ 之间。该 SDE 的转移函数为单位阵，随机积分得到

$$\boldsymbol{\xi}_k(t) = \underbrace{\boldsymbol{\xi}_k(t_k)}_{\mathbf{0}} + \int_{t_k}^t \mathbf{w}(s) ds, \quad (2.69)$$

于是运动先验为

$$\boldsymbol{\xi}_k(t) \sim \mathcal{GP}(\mathbf{0}, \min(t, t') \mathbf{Q}). \quad (2.70)$$

若控制点每隔 T 秒均匀分布, 则逆核矩阵为 $\kappa^{-1} = \Phi^{-\top} Q^{-1} \Phi^{-1}$, 其中

$$\Phi^{-1} = \begin{bmatrix} \mathbf{I} & & & & \\ -\mathbf{I} & \mathbf{I} & & & \\ & \ddots & \ddots & & \\ & & & -\mathbf{I} & \mathbf{I} \end{bmatrix}, \quad Q = \text{diag}(\mathcal{K}(t_1, t_1), T\mathcal{Q}, \dots, T\mathcal{Q}). \quad (2.71)$$

局部变量下的误差为

$$e_k = \begin{cases} \text{Log}(\mathbf{T}_1 \check{\mathbf{T}}_1^{-1}) & k = 1 \\ \boldsymbol{\xi}_{k-1}(t_k) - \boldsymbol{\xi}_{k-1}(t_{k-1}) & k > 1 \end{cases}, \quad (2.72)$$

其中 $\check{\mathbf{T}}_1$ 是初始位姿先验。用全局变量表示时, 同一误差为

$$e_k = \begin{cases} \text{Log}(\mathbf{T}_1 \check{\mathbf{T}}_1^{-1}) & k = 1 \\ \text{Log}(\mathbf{T}_k \mathbf{T}_{k-1}^{-1}) & k > 1 \end{cases}. \quad (2.73)$$

图 2.6 给出了这种随机游走 GP 运动先验的因子图。与线性样条类似, 若要在其他时刻查询轨迹, 可以使用 GP 插值; 对于随机游走先验, 结果再次是线性插值:

$$\mathbf{T}(t) = (\mathbf{T}_k \mathbf{T}_{k-1}^{-1})^{\alpha_k(t)} \mathbf{T}_{k-1}, \quad (2.74)$$

其中 $\alpha_k(t) = (t - (k-1)T)/T$, 且 $(k-1)T \leq t < kT$ 。与样条方法不同, 这里的线性插值是 SDE 选择的间接结果, 而不是显式指定的形式; 如果一开始选择高阶 SDE, 就会得到高阶插值样条。

最后, 需要将误差项线性化以用于 MAP 估计。再次利用 Lie 群扰动方法, 对于式 (2.73) 的第二种情形, 有

$$\begin{aligned} e_k &= \text{Log}(\text{Exp}(\boldsymbol{\xi}_k) \mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \text{Exp}(-\boldsymbol{\xi}_{k-1})) \\ &\approx \text{Log}(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}}) + \boldsymbol{\xi}_k - \text{Ad}(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}}) \boldsymbol{\xi}_{k-1}, \end{aligned} \quad (2.75)$$

其中 $\mathbf{T}_k^0$ 和 $\mathbf{T}_{k-1}^0$ 是当前猜测, $\boldsymbol{\xi}_k$ 和 $\boldsymbol{\xi}_{k-1}$ 是待求扰动, $\text{Ad}(\cdot)$ 是 $\text{SE}(d)$ 的伴随表示。式 (2.75) 的线性化形式可以在每次迭代中直接纳入标准 MAP 估计框架。

此外, 若想利用式 (2.74) 减少随机游走示例中的控制点数量, 可以采用式 (2.67) 中针对线性样条的同一方法, 因为两者最终都归结为 Lie 群控制点之间的线性插值。样条与 GP 的根本区别在于: GP 方法使用运动先验项 (见图 2.6) 对问题进行正则化, 而样条方法不使用这种先验。⁴

2.2 延伸阅读与近期趋势

关于流形, 尤其是 Lie 群上的估计, 已有大量研究。机器人领域的奠基性参考是 Chirikjian^[205] 及 Chirikjian 和 Kyatkin^[207]; 这些著作理论基础扎实, 并给出了处理流形不确定性的通用工具, 包括全局不确定性。关于流形优化, Boumal^[106] 是面向机器人应用的优秀参考。Barfoot^[47] 更详细地讨论了不确定性仍较为集中的 Lie 群估计, 这与本章内容相近。连续时

间估计方面, Talbot 等人^[1067] 提供了参数化和非参数化方法的综合综述; Barfoot^[47] 也对非参数方法作了较为详细的说明。

与上一章类似, 离群点处理、使算法可微, 以及利用本章工具支持不同类型的传感器等近期趋势, 将在后续章节中进一步讨论。

第三章 对错误数据关联和离群值的鲁棒性

作者： Heng Yang、Josh Mangelson、Yun Chang、Jingnan Shi、Niko Sunderhauf、Luca Carlone

在第 1 章中，我们已经看到，因子图是用于建模和可视化 SLAM 问题的强大表示；给定一组测量（例如里程计和路标测量）后，最大后验（maximum a posteriori, MAP）估计为推断感兴趣变量（例如机器人位姿和路标位置）提供了一个有理论依据且通用的框架。例如，我们看到，当测量 z_i 受协方差为 Σ_i 的加性零均值高斯噪声影响时，MAP 估计会导出一个非线性最小二乘优化问题：

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2, \quad (3.1)$$

其中， $\mathbf{x}_i$ 表示参与第 i 个测量的状态子集。¹ 本章指出，在实践中，许多测量 z_i 可能由于错误的数据关联而具有很大的误差，远不符合零均值高斯分布（第 3.1 节）；这些测量通常会使得式 (3.1) 中的估计 $\mathbf{x}^{\text{MAP}}$ 产生很大扰动。因此，我们将讨论如何在 SLAM 前端剔除严重离群值（第 3.2 节），然后重点讨论如何提高 SLAM 后端对剩余离群值的鲁棒性（第 3.3 节）。最后，我们将简要回顾近期趋势，并给出相关工作的进一步线索（第 3.4 节）。

3.1 离群值的成因及其危害

本节论证：离群值在大多数 SLAM 应用中不可避免；若未得到恰当处理，则会导致严重错误的估计。

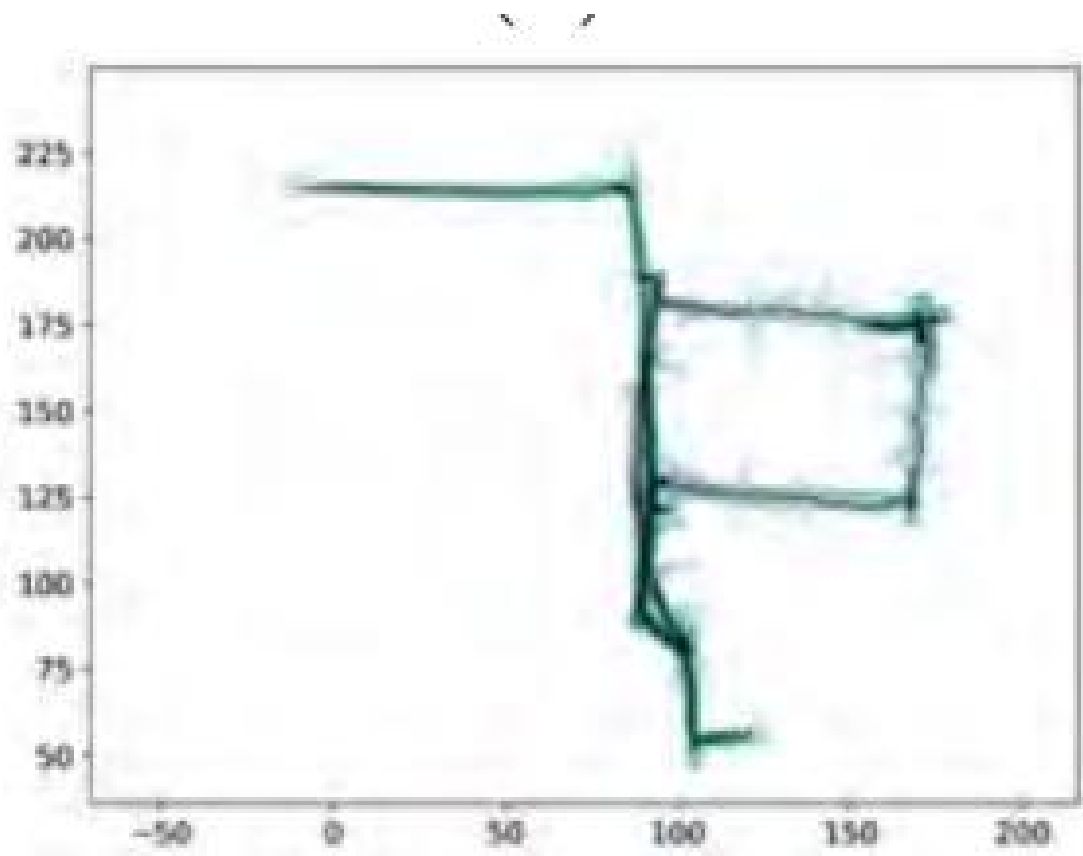
3.2 数据关联与离群值

为了解离群测量的成因，先考虑两个例子。首先，考虑一个基于路标的 SLAM 问题：我们需要根据里程计测量以及若干机器人位姿对路标的相对观测，重建机器人的轨迹和外部路标的位置。若如第 1 章那样假设路标测量具有零均值高斯噪声，则优化中会出现形如 $\|z_{ij} - h(p_i, \ell_j)\|_{\Sigma}^2$ 的项。这些项刻画了如下事实：给定测量 z_{ij} 是从位姿 p_i 对路标 ℓ_j 的观测，并带有高斯噪声，其中 $h(\cdot)$ 是描述相对测量类型（例如距离、方位等）的函数。实践中， z_{ij} 是由 SLAM 前端对原始传感器数据预处理得到的。例如，若机器人装有相机，且 z_{ij} 是对路标 ℓ_j 方位的视觉观测，则可先在图像中进行目标（或更一般地说，特征）检测与匹配，再计算与检测到的像素对应的方位，从而提取该测量。问题在于检测并不完美：图像中被检测为 ℓ_j 的路标，在现实中可能实际是另一个路标。这会使 z_{ij} 大幅偏离所假设的模型。将一个测量关联到

某个路标的问题通常称为**数据关联问题**，它也广泛存在于其他估计问题中（例如目标跟踪）。因此，错误的数据关联会在估计问题中产生离群值。

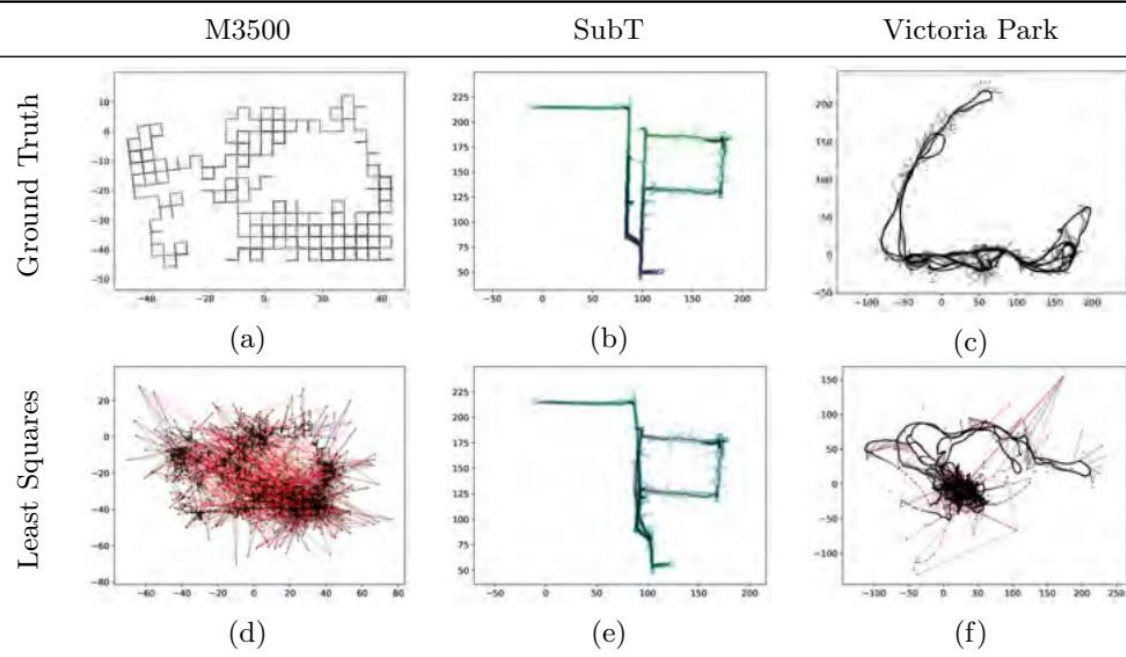
作为第二个例子，考虑位姿图优化问题。此时我们主要关心机器人的轨迹估计（由一组位姿表示），测量则是里程计测量（关联轨迹中相邻的位姿）或回环约束（关联非相邻、且可能在时间上相距很远的位姿）。实践中，回环由基于视觉或 LiDAR 的地点识别方法检测；这些方法负责判定一对位姿 p_i 和 p_j 是否观测到了环境中的同一部分。遗憾的是，当前的地点识别方法容易出错，会在没有观测到同一场景的位姿之间检测出回环。这部分源于现有方法的局限性，但更多时候源于**感知混叠**（perceptual aliasing）：两个外观相似的位置实际上对应不同地点，例如大学建筑中的两个教室，或办公环境中外观相似的隔间。这同样可理解为数据关联失败：我们错误地将回环测量关联到了两个选错的机器人位姿。

需要注意，离群值不仅由错误数据关联造成，也可能由 SLAM 方法中所作假设被违背而引起。例如，大多数 SLAM 方法假设路标是静态的；因此，对运动物体的检测，即使正确地关联到该物体，也可能产生具有大残差的离群测量。类似地，传感器故障和性能退化，例如故障的轮式编码器或相机镜头上的灰尘，也可能导致测量中出现离群值。



3.3 存在离群值时的最小二乘

存在离群值时，最小二乘形式 (3.1) 得到的估计可能严重错误。从理论角度看，我们为测量所假设的高斯噪声是“轻尾”（light-tailed）的，即它实质上排除了误差极大的测量。



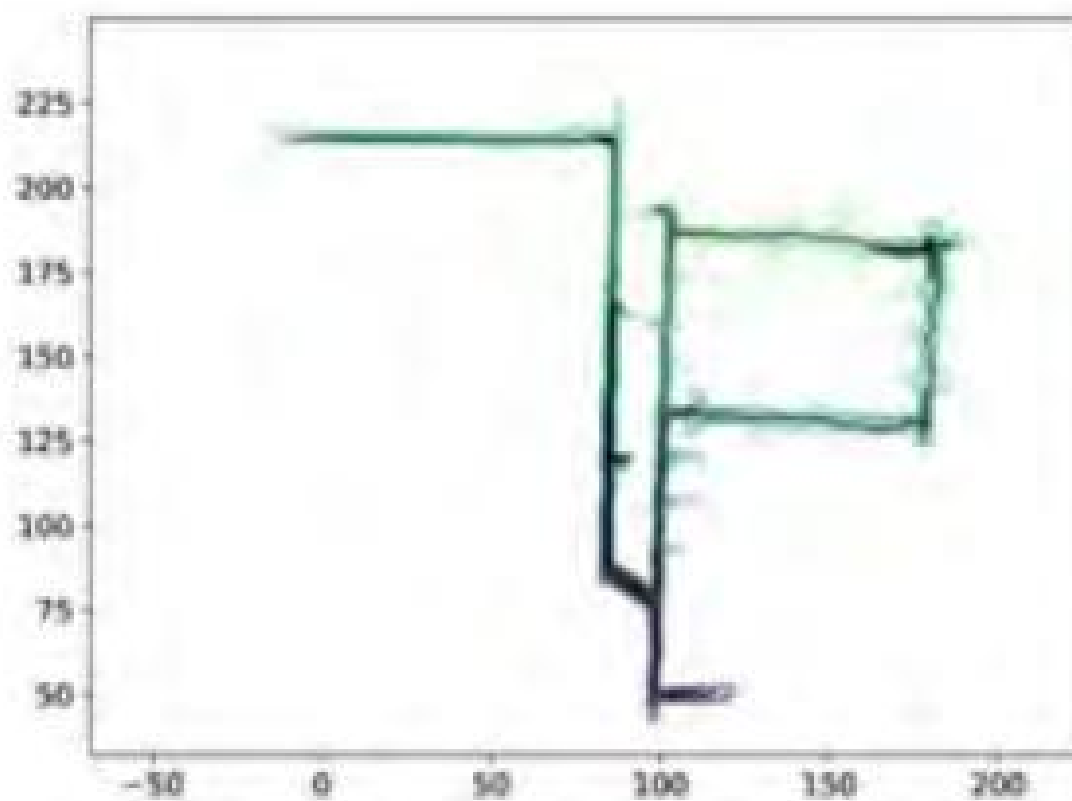
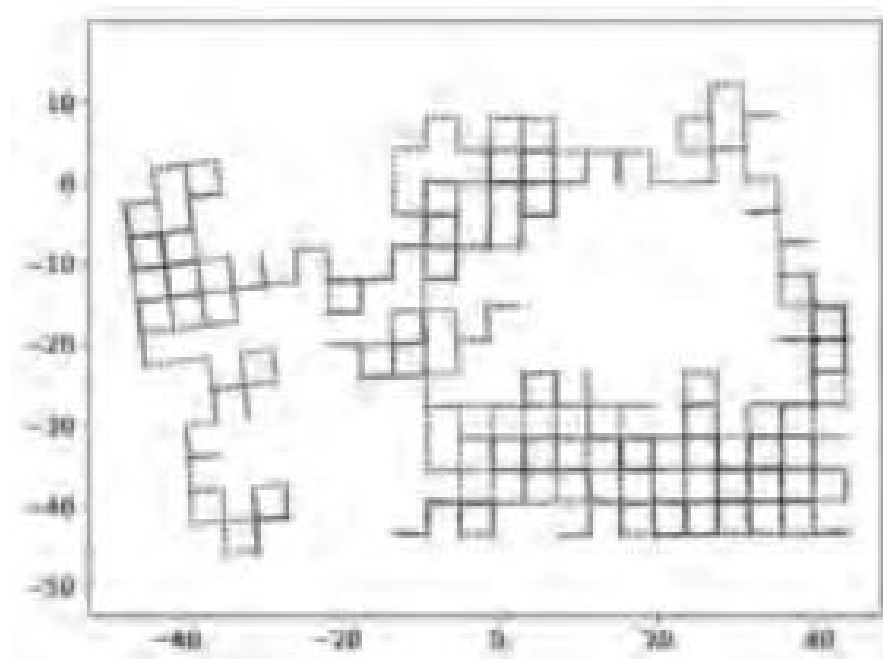
从更实际的角度看，离群值会在目标函数中产生一些项：当在真值附近求值时，其残差误差 $r_i(\mathbf{x}) := \|z_{ij} - h(\mathbf{p}_i, \ell_j)\|_\Sigma$ 非常大。由于优化目标将残差平方，即目标为 $\sum_i r_i(\mathbf{x})^2$ ，这些残差会对代价产生不成比例的巨大影响，优化过程会着重减小由离群值引起的大项，而非充分利用其余的内点 (inlier) 测量。

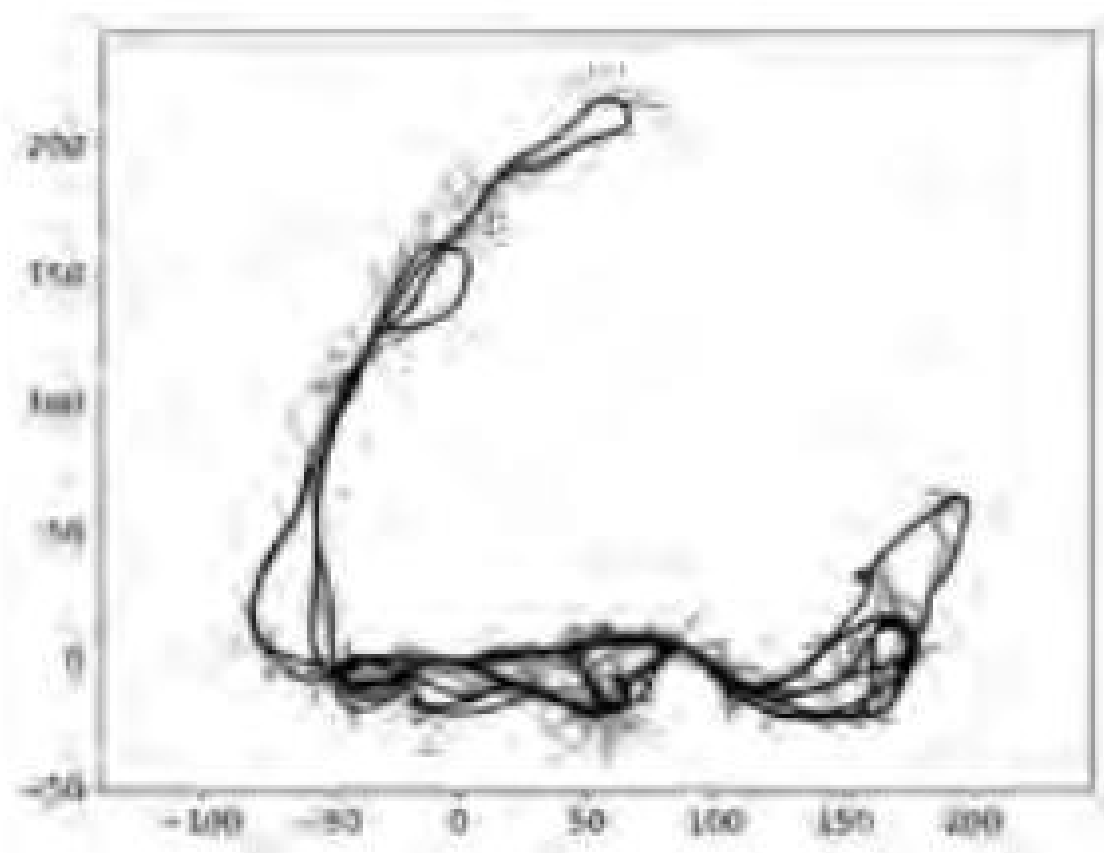
为说明这一点，图 3.1 给出了三个含离群值 SLAM 问题的结果。第一列是一个称为 M3500 的模拟位姿图优化基准，其中位姿按网格状构型排列；数据集包含 3500 个二维位姿和 8953 个测量。第二列是一个称为 SubT 的真实世界位姿图数据集，采集于 DARPA 地下挑战赛期间的一条隧道中^[299]；该数据集包含 682 个三维位姿和 3278 个测量。第三列是一个称为 Victoria Park 的真实世界基于路标的 SLAM 数据集^[809]；该数据集包含 7120 个二维位姿和路标，以及 17728 个测量。图 3.1(a)–(c) 展示这三个问题的真值轨迹。图 3.1(d)–(f) 展示存在离群值时由最小二乘形式产生的估计。具体地，对 M3500 和 Victoria Park，我们向（回环或路标）测量中加入 15% 的随机离群值；SubT 数据集本身已包含离群值。图中以红色显示离群测量。可以看到，离群值会导致完全错误的轨迹和地图估计。此外，离群值常揭示环境中的感知混叠：例如，SubT 数据集中间两条外观相似的竖直走廊会诱发许多伪回环，误导后端构建出只有一条竖直走廊的地图。

3.4 在 SLAM 前端检测和剔除离群值

SLAM 前端的主要作用，是从原始传感器数据中提取中间表示或（伪）测量，之后这些量将转换为后端的因子。典型的 SLAM 前端先计算一个初始测量集合（其中可能混入许多离群值），再对该初始集合进行后处理以去除离群值。本节讨论两种在 SLAM 前端剔除离群值的方法：RANSAC 和成对一致性最大化 (Pairwise Consistency Maximization)。

Ground Truth





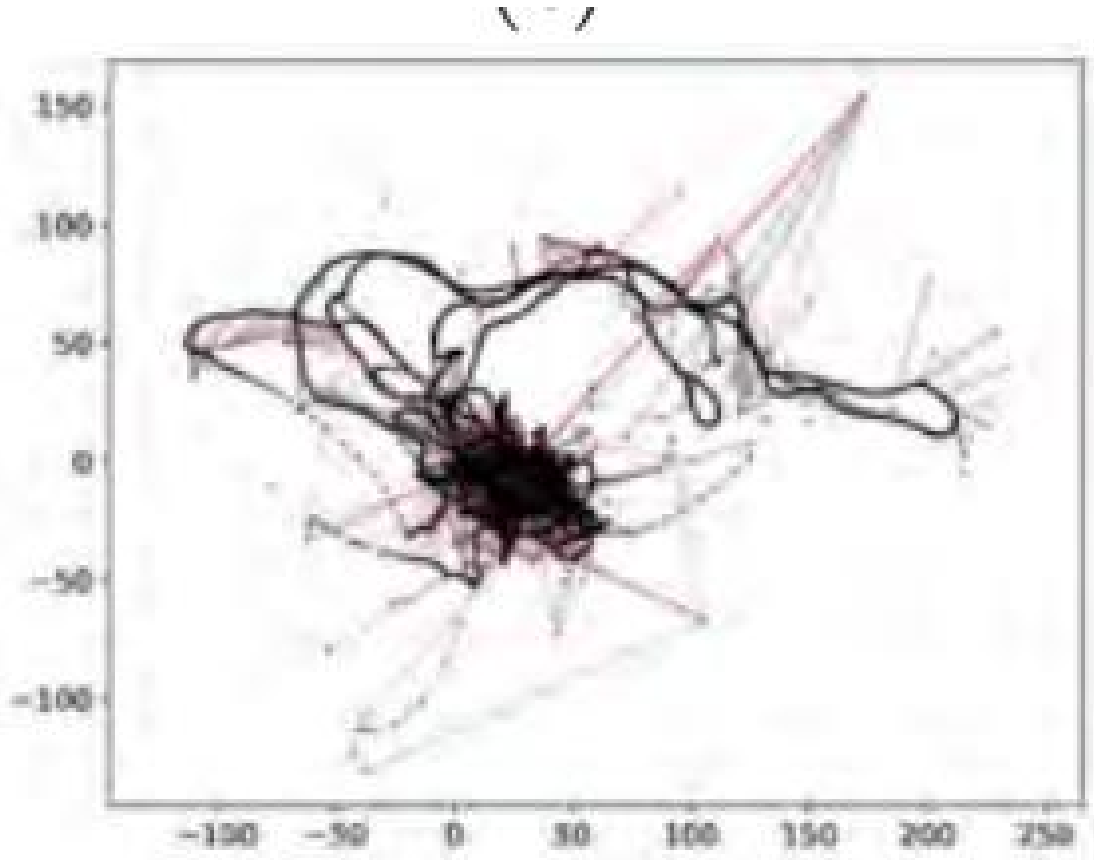


图 3.1: 图 3.1 含离群值的 SLAM 问题: (a)–(c) 分别为 M3500、SubT 和 Victoria Park 数据集的真值轨迹; (d)–(f) 为存在离群值时采用最小二乘形式得到的轨迹估计。内点测量显示为灰色边, 离群值显示为红色边。在 SubT 数据集中, 还展示了由 SLAM 位姿估计构建的稠密地图。

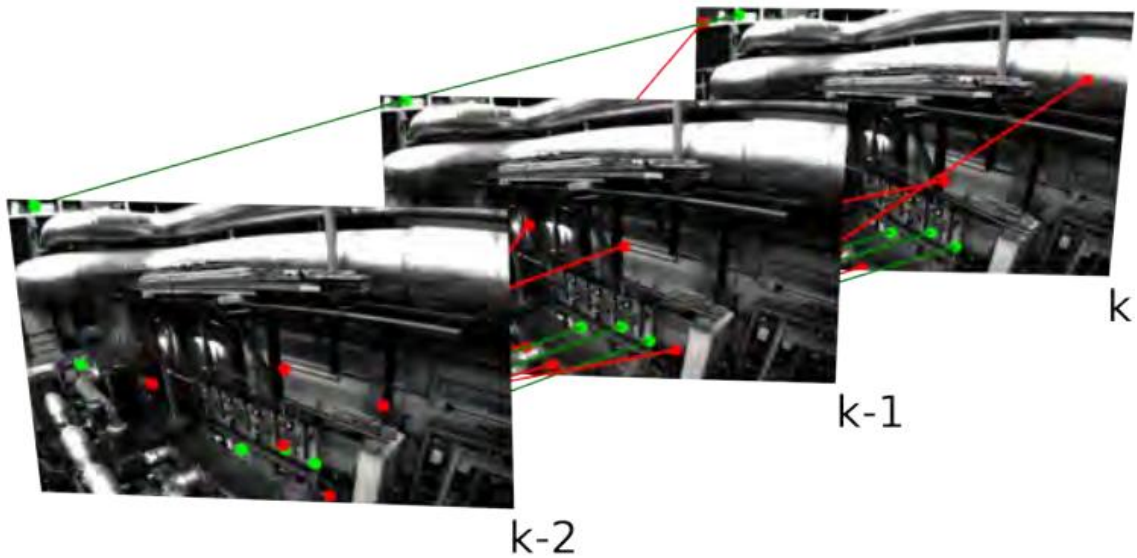


图 3.2: 图 3.2 视觉 SLAM 问题中跨三个帧 (分别采集于时刻 $k-2$ 、 $k-1$ 和 k) 的特征跟踪。内点以绿色显示: 它们是随时间描绘同一 (静态) 三维点的像素。离群值以红色显示。

3.5 随机采样一致性 (RANSAC)

RANSAC 是一种成熟的离群值剔除工具^[328],也是许多基于路标的 SLAM 系统的关键组成部分。为理解 RANSAC 是什么及其在 SLAM 中的作用,考虑一种基于路标的视觉 SLAM 方法。

例 3.1 (基于路标的视觉 SLAM 中的离群值)一种基于路标(或基于特征)的视觉 SLAM 方法会在每幅图像中提取二维特征点,然后通过基于光流的特征跟踪或基于描述子的特征匹配,在连续帧之间关联这些特征点(图 3.2)。具体而言,在时刻 k ,该方法检测二维特征点,并将其与前一帧(例如时刻 $k-1$)观测到的对应点匹配;这些匹配像素通常称为二维-二维对应(2D-2D correspondences)。由于光流或基于描述子的匹配并不精确,这一初始对应集合可能包含离群值。因此,在将其传递给估计机器人位姿和路标位置的后端之前,过滤严重离群值十分重要。

RANSAC 是一种在将对应关系传递给后端之前快速检测并去除其中离群值的工具。检测离群值依赖两个关键洞见。第一个洞见是,在 SLAM 问题中,内点对应必须满足几何约束。例如,在本例中,内点对应描述了相机运动时静态三维点的观测像素运动。由此产生的像素运动不可能任意,而必须遵循精确的几何约束,即**对极约束**(epipolar constraint);该约束规定了两个帧中对应像素如何随相机运动发生关联。具体地,对于已标定相机,对极约束要求对应像素 $z_i(k-1), z_i(k)$, 即分别在时刻 $k-1$ 和 k 描绘路标 i 的像素,满足

$$z_i(k-1)^\top ([t_k^{k-1}]_\times R_k^{k-1}) z_i(k) = 0, \quad (3.2)$$

其中, t_k^{k-1} 和 R_k^{k-1} 是描述相机在时刻 $k-1$ 与 k 之间(未知)运动的相对平移和旋转。²更一般地,若将第 i 个对应记为 z_i (在上述例子中, $z_i = \{z_i(k-1), z_i(k)\}$), 则这类几何约束具有如下形式:

$$C(z_i, x) \leq \gamma, \quad (3.3)$$

它表明对应关系必须满足某个不等式,而该不等式可能是未知状态 x 的函数。式 (3.3) 右侧的参数 γ 通常经过调节以计及噪声的存在。例如,虽然理想情况下式 (3.2) 中的对极约束应严格满足,但实践中像素检测带有噪声,因而可能存在小误差;因此,我们放宽该约束,只要求 $|z_i(k-1)^\top ([t_k^{k-1}]_\times R_k^{k-1}) z_i(k)| \leq \gamma$, 其中 γ 为某个较小的值。

第二个洞见是,假设离群值数量不太多,我们可以将内点找作:对某个 x , 满足几何约束 (3.3) 的最大对应集合:

$$\begin{aligned} S_{\text{CM}}^* &= \operatorname{argmax}_{x, S \subset M} |S| \\ \text{s.t. } C(z_i, x) &\leq \gamma, \quad \forall i \in S \end{aligned} \quad (3.4)$$

其中, M 是初始假定对应 (putative correspondences) 的集合, $|S|$ 表示子集 S 的基数(元素个数)^[728]。换言之,优化问题 (3.4) 在假定对应集合 M 中寻找最大的子集 S , 使得 S 中的测量对同一个 x 都满足几何约束。直观地说,问题 (3.4) 刻画了如下直觉:内点(由集合 S 估计)必须就同一个 x “达成一致”,例如它们必须全都与机器人的真实运动一致。问题 (3.4) 在计算机视觉中称为**一致性最大化**(consensus maximization)。注意,式 (3.4) 不要求求解完整的 SLAM 问题(后者可能包含许多位姿和路标),因为它只涉及 SLAM 状态的

一小部分；例如，对极约束 (3.2) 只涉及两个帧之间的相对位姿，而非整个 SLAM 轨迹。同时，问题 (3.4) 仍然是一个困难的组合问题，这与典型 SLAM 前端对快速运行时间的要求相冲突。因此，人们通常不寻求式 (3.4) 的精确解，而是采用快速启发式方法近似地求解它。

随机采样一致性 (RANDOM SAMPLE CONSENSUS, RANSAC) 可能是为一致性最大化问题 (3.4) 寻找近似解的最著名方法。RANSAC 建立在一个关键假设之上：式 (3.4) 中的 x 维数相对较低，且可利用快速估计器（即所谓的最小求解器，minimal solvers）从一小组测量（即所谓的最小集，minimal set）中估计得到。³ 例如，在我们的视觉 SLAM 示例中，可利用 Nister 的五点法^[810]，仅通过 5 个像素对应估计两个相机帧之间的相对运动。RANSAC 的核心思想是：无需穷尽检查每一个可能的子集 $S \subset M$ ，而可以对最小测量集进行采样以寻找内点。更具体地，RANSAC 迭代执行以下三步：

1. 采样 n 个对应关系的子集，其中 n 是当前问题最小集的大小；⁴
2. 使用最小求解器，根据采样的 n 个对应关系计算估计 $\hat{x}$ ；⁵
3. 对于上一步计算得到的 $\hat{x}$ ，选择满足几何约束 $C(z_i, \hat{x}) \leq \gamma$ 的对应关系 $S \subset M$ 。若该集合 S 大于先前迭代中计算的集合，则将其保存。

最后一步计算得到的集合 S 称为**一致集** (consensus set)。RANSAC 通常在计算到足够大的一致集（由用户参数指定）后，或达到最大迭代次数后停止。RANSAC 本质上试图从测量集合中采样 n 个内点，因为这些内点很可能会与其他所有内点“达成一致”，因而具有较大的一致集。

RANSAC 是众多离群值剔除问题的首选方案。特别是，在离群值数量较少且最小集较小时，它能迅速收敛到良好估计（即良好的对应集合）。若从测量集合中采样一个内点的概率为 ω ，⁶ 则可容易得出，RANSAC 找到一组内点所需的期望迭代次数为 $\frac{1}{\omega^n}$ 。例如，当 $n = 5$ 且 $\omega = 0.7$ （即 70% 的测量为内点）时，期望迭代次数小于 10。加之非最小求解器在实践中极快（常能在数毫秒内完成数百次迭代），RANSAC 极具吸引力。此外，RANSAC 还会提供估计 $\hat{x}$ （例如机器人里程计），可作为后端的初始猜测。

另一方面，RANSAC 未必适合所有问题。尤其是，当内点数量很少或最小集很大时，期望迭代次数会变得不切实际地大。例如，当 $n = 10$ 且 $\omega = 0.1$ 时，找到一组内点所需的期望迭代次数达到 10^{10} ；在更少的迭代次数后终止 RANSAC，很可能返回错误解（即错误的 $\hat{x}$ 和对应关系）。如下一节所述，在许多 SLAM 问题中，“内点很多且最小集很小”的假设并不总是成立。

3.6 基于图论的离群值剔除与成对一致性最大化

如前所述，当离群值比例适中（例如低于 70%）且最小集较小（例如小于 8）时，RANSAC 非常有效。然而，存在严重感知混叠的环境可能具有极高比例的离群值。此外，我们感兴趣的并非所有问题都存在最小集很小的快速最小求解器。例如，考虑一个具有 N 个节点的位姿图 SLAM 问题，最小集至少必须包括 $N - 1$ 个测量（构成位姿图的一棵生成树），而 N 通常有数千之多。

因此，本节介绍一种称为**成对一致性最大化** (Pairwise Consistency Maximization, PCM) 的替代方法。该方法不对最小集采样，而是借助图论寻找内部彼此“一致”的最大测量集合。该方法可处理离群值比例超过 90% 的测量集合，并在传递给后端之前剪除严重离群值。该方法最初由文献^[728]提出，并在^[1005, 332]中推广至超越成对一致性的情况。

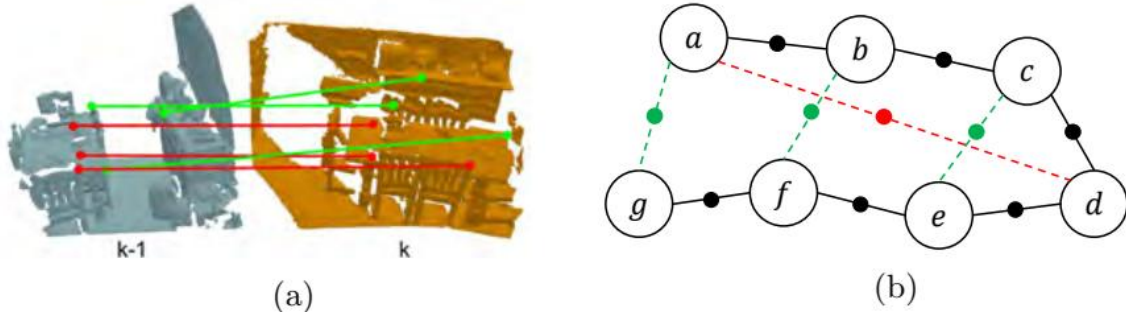


图 3.3: 图 3.3 (a) 来自两幅 RGB-D 扫描的三维-三维对应，它们表示场景的两个局部视图。绿色线条表示内点对应，红色线条表示离群值对应。(b) 回环中含离群值的位姿图。绿色虚线表示内点回环，红色点线表示一个离群回环。

PCM 的关键洞见是：对于许多问题，可以定义一致性函数来描述一对测量是否相互一致。下面通过两个例子说明这一点。

例 3.2 (带 RGB-D 相机的基于路标视觉 SLAM 中的一致性函数) 使用 RGB-D 相机的基于路标视觉 SLAM 方法会在每个 RGB-D 帧中提取三维特征点，再在连续帧之间将它们关联起来 (图 3.3(a))。具体地，在时刻 k ，该方法检测三维特征点，并将其与前一帧 (例如时刻 $k-1$) 观测到的对应点进行匹配；这些匹配的三维点通常称为三维-三维对应 (3D-3D correspondences)。时刻 k 和 $k-1$ 采集的三维点，理想情况下对应于从两个不同视点观测到的同一组三维静态点；因此，对应点对 $\{z_i(k-1), z_j(k-1)\}$ 与 $\{z_i(k), z_j(k)\}$ 之间的距离必须随时间保持不变 (至多存在噪声)：

$$\|z_i(k-1) - z_j(k-1)\| - \|z_i(k) - z_j(k)\| \leq \gamma \quad (3.5)$$

可以看到，与 RANSAC 中使用的几何约束不同，一致性函数 (3.5) (i) 不依赖于状态，因而无需最小求解器即可直接求值；(ii) 无论最小集大小如何，均涉及一对对应关系。尽管前一个例子也可使用 RANSAC 求解，⁷ 现在考虑一个维度更高的问题。

例 3.3 (位姿图 SLAM 中的一致性函数) 考虑一个位姿图 SLAM 问题，其中回环可能因地点识别失败和感知混叠而包含离群值；我们假设里程计可靠且不含离群值。为了判断两个回环是否彼此一致，需要注意：在无噪声情形下，图中沿环的位姿测量复合后必须为单位元 (图 3.3(b))。⁸ 因此，一对回环 T_b^a (位姿 a 与 b 之间) 和 T_d^c (位姿 c 与 d 之间) 必须满足：

$$\text{dist}(T_b^a \cdot \bar{T}_c^b \cdot T_d^c \cdot \bar{T}_a^d, \mathbf{I}) \leq \gamma \quad (3.6)$$

其中， $\bar{T}_c^b$ 和 $\bar{T}_a^d$ 分别是节点 b 到节点 c 、以及从节点 d 到节点 a 的里程计测量链； dist 是合适的距离函数，用于度量 $T_b^a \cdot \bar{T}_c^b \cdot T_d^c \cdot \bar{T}_a^d$ 与单位位姿之间的距离。与往常一样， γ 是为计及噪声而选择的参数：由于里程计和回环中存在噪声，沿一个环的测量未必复合为单位元。⁹

更一般地，一致性函数是关联两个测量并且必须满足给定约束的函数。对于一对测量 z_i 和 z_j ，所得的成对一致性约束形式为：

$$F(z_i, z_j) \leq \gamma, \quad (3.7)$$

其中, F 是一致性函数, γ 是计及测量噪声的用户指定参数。需要强调, 成对一致性约束与状态无关, 因此只需检查每一对测量, 无需求助于最小求解器即可高效验证。

利用式 (3.7), 我们可以给出另一种离群值剔除方法: 选择彼此成对一致的最大测量集合:

$$\begin{aligned} S_{\text{PCM}}^* = \operatorname{argmax}_{S \subset M} |S| \\ \text{s.t. } F(z_i, z_j) \leq \gamma, \quad \forall i, j \in S \end{aligned} \quad (3.8)$$

问题 (3.8) 寻找最大的测量子集 S , 使得 S 中每对测量均成对一致。我们将其称为成对一致性最大化 (PCM) 问题。该问题仍具有组合性质, 但看起来比式 (3.4) 稍易: 它不涉及 x , 且可对 M 中每一对 (i, j) 预先计算约束 $F(z_i, z_j) \leq \gamma$ 。此外, 该问题具有图论解释, 因此可使用图论中的成熟工具, 即最大团算法, 来求解。

为建立问题 (3.8) 与图论的联系, 将离群值剔除问题可视化为一个图 $\mathcal{G}$: 图的节点是假定测量 $i \in M$; 若

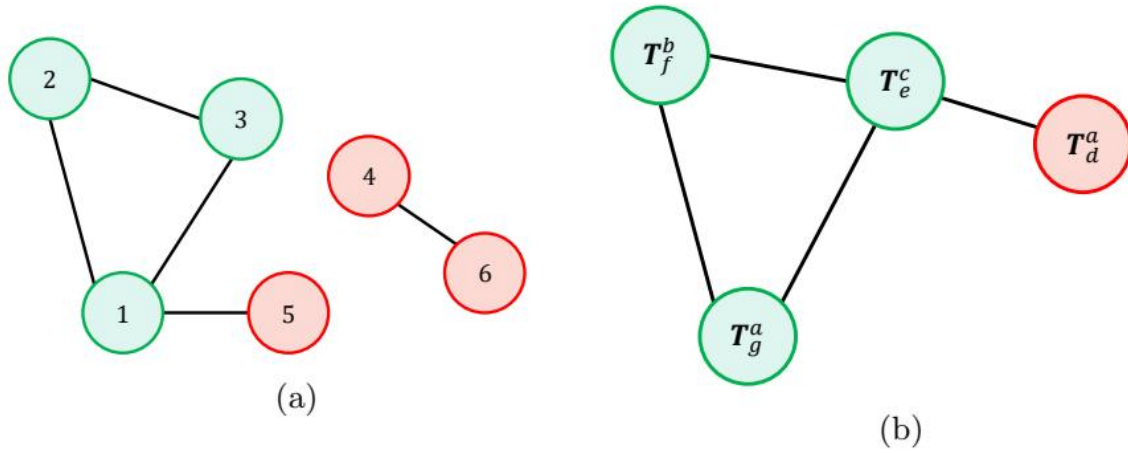


图 3.4: 图 3.4 (a) 图 3.3(a) 中三维-三维对应示例的一致性图。(b) 图 3.3(b) 中位姿图示例的回环一致性图。

$F(z_i, z_j) \leq \gamma$, 则节点 i 和 j 之间存在一条边。这通常称为**一致性图** (consistency graph)。问题 (3.8) 现在要求选择最大的节点子集 S , 使 S 中每对节点之间都由一条边相连; 这正是图中**最大团** (maximum clique) 的定义。更具体地, 在图论中, 团是一个节点子集, 其中每一对节点之间都有边; 最大团则是图中这样的最大节点子集。因此, 问题 (3.8) 的解就是一致性图的最大团。这一图论联系在实践中很有价值, 因为对给定图寻找最大团是图论中研究充分的问题, 称为最大团问题。最大团问题是 NP-hard 问题^[1197], 且难以近似^[1314, 322], 这意味着寻找任意接近真实解的解同样是 NP-hard 的。不过, 已有数十种最大团算法被提出, 其中一些可根据图的稠密度处理规模相当大的问题。所提出的方法大多可分为精确方法与基于启发式的方法。所有精确算法的复杂度均为指数级, 且通常基于分支定界; 启发式算法则常试图利用问题中的某种结构而更快, 但代价是不一定保证最优解^[1197]。相对较新的工作, 如^[853], 提出了可并行化的最大团算法, 能够在大型稀疏图中快速寻找最大团。

总而言之, 使用最大团算法求解 PCM 问题包括以下步骤:

1. 为当前问题选择一致性函数 F 。

2. 对每一对假定测量 $(i, j) \in M$ 求值一致性函数；当 $F(z_i, z_j) \leq \gamma$ 时，在 i 与 j 之间连边，从而构建一致性图。

3. 使用精确或近似最大团算法求解一致性图的最大团。

4. 返回（可能为近似的）最大团中的测量 S 。

需要指出，一致性函数的选择依赖于具体问题；选择好的函数会显著影响离群值剔除质量。例如，可以选择一个无意义函数 $F(z_i, z_j) = 0$ ，它无论输入为何都返回零；此类函数无法剔除任何离群值，从而使 PCM 无效。反之，若构造的函数使得只有内点能通过检验，则恰好剔除所有离群值。文献^[1005, 332]讨论了适用于广泛几何问题的一组潜在一致性函数。

在结束本节前，还需说明几点。尽管 PCM 相较 RANSAC 能处理更高比例的离群值，并且更适合某些问题（例如位姿图 SLAM），但 PCM 与 RANSAC 的权衡更为细致。RANSAC 使用最小求解器计算的估计来评估单个测量的一致性；PCM 则以成对方式评估一组测量彼此之间的一致性。在某些情形中，RANSAC 的单测量一致性不足以评估整个测量集合；位姿图优化常是如此，因为一对回环测量各自的一致性并不必然保证这两个回环之间成对一致。¹⁰另一方面，对于三维-三维位姿估计（例 3.2）等某些问题，PCM 使用的成对一致性函数 (3.5) 可能比 RANSAC 更宽松，从而将某些离群值归类为内点。

在 PCM 的语境下，还应注意，精确最大团求解器在稠密一致性图中（即许多测量对彼此一致时）往往较慢，因此对某些问题而言，基于启发式的最大团解可能是更好的选择。最后，对某些问题很难设计合适的一致性函数；例如，对于二维-二维对应，由于缺乏合适的不变量，难以严格地设计一个通用一致性函数（参见^[1005]的讨论）。

3.7 提高 SLAM 后端对离群值的鲁棒性

包括 RANSAC 和 PCM 在内的前端离群值剔除仍可能漏检离群值，并向后端传递受离群值污染的测量集合。¹¹如第 3.1.2 节所见，采用标准最小二乘估计时，少数离群值即可导致完全错误的结果。因此，提高后端对剩余离群值的鲁棒性十分重要。

第 3.1.2 节指出，使用残差平方会“放大”离群测量对代价函数的影响。本节遵循鲁棒统计中 M-估计 (M-Estimation) 的标准理论^[495]，对 SLAM 优化目标函数作轻微修改，以重新获得对离群值的鲁棒性。

M-估计（“maximum-likelihood-type estimation”）是鲁棒估计的一个框架。它建议将式 (3.1) 中的平方损失替换为恰当选择的鲁棒损失函数 ρ ：

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \mathbf{r}_i(\mathbf{x})^2 \implies \mathbf{x}^{\text{MEST}} = \arg \min_{\mathbf{x}} \sum_i \rho(\mathbf{r}_i(\mathbf{x})). \quad (3.9)$$

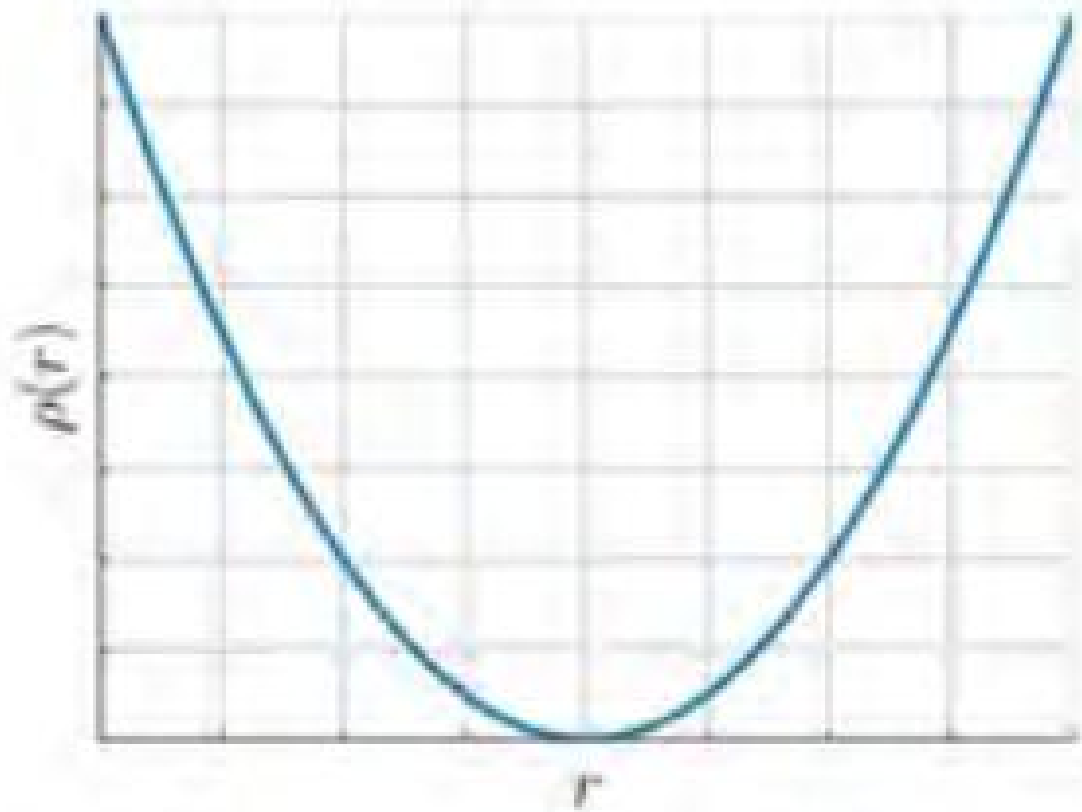
鲁棒损失 ρ 的关键要求是：它为非负函数，且对于大残差的增长速度低于二次；换言之，随着 r_i 变大，鲁棒损失函数需要具有导数 $\frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i} \ll \frac{\partial \|\mathbf{r}_i\|^2}{\partial \mathbf{r}_i} = 2\mathbf{r}_i$ ；在许多情况下，希望当 r_i 变大时， $\frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i}$ 趋于零。为阐明这一要求，考虑使用梯度下降求解 $\min_{\mathbf{x}} \sum_i \rho(\mathbf{r}_i(\mathbf{x}))$ 。由链式法则，目标 $f(\mathbf{x}) \doteq \sum_i \rho(\mathbf{r}_i(\mathbf{x}))$ 的梯度为：

$$\frac{\partial f}{\partial \mathbf{x}} = \sum_i \frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i} \cdot \frac{\partial \mathbf{r}_i(\mathbf{x})}{\partial \mathbf{x}} \quad (3.10)$$

由式 (3.10) 可知，若从一个良好的初始猜测开始（即相对接近真值），离群测量具有较大残差，因而 $\frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i}$ 很小，对总体下降方向的影响也很小。因此它们在估计中几乎没有影响。

实际上, 函数 $\psi(\mathbf{r}_i) := \frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i}$ 通常称为**影响函数** (influence function) [85]。

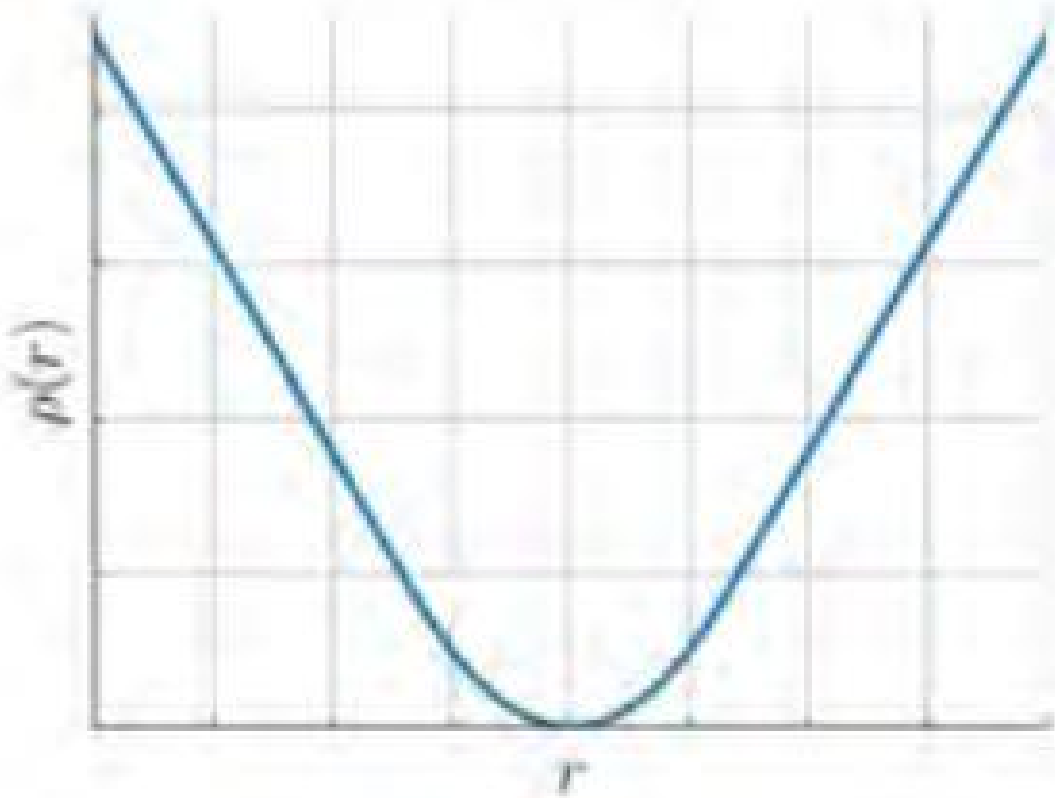
鲁棒估计文献并未给出唯一的鲁棒损失函数, 而是提供了多种可能选择。图 3.5 列出了常见损失函数, 其中包括 Huber、Geman-McClure、Tukey 双权 (Tukey's biweight) 和截断二次损失等常用鲁棒损失, 也包括一种更激进的选择, 即最大一致性损失。后一种通常不列在鲁棒估计文献的损失函数之中, 但这里提及它, 是因为它与我们在式 (3.4) 中讨论的一致性最大化问题相联系。¹² 鲁棒损失的选择高度依赖具体问题 [554, 1075]。例如, 当已知合理的截断阈值 (即内点的预期最大误差) 时, 具有硬截断的损失函数, 例如截断二次损失 (其二次区与函数“平坦”部分之间发生突变), 更为合适。还必须考虑计算因素。例如, Huber 常用于 BA 问题, 因为它是凸函数, 在优化过程中表现更稳定, 尽管它对离群值保留非零影响 (只有当大残差时损失保持常数, 影响才变为零)。另一方面, 截断二次和最大一致性损失对离群值尤不敏感, 但常需要专用求解器。¹³



- (a) 二次损失
- (b) Huber 损失
- (c) Geman-McClure
- (d) Tukey 双权
- (e) 截断二次 (f) 最大一致性损失

图 3.5 二次损失与鲁棒损失函数示例。鲁棒损失函数的形状由一个控制内点和离群值分界的参数确定。

图 3.6(g)-(l) 展示了对上述两种鲁棒损失, 即 Huber 损失和截断二次损失, 应用梯度下



降所得到的 SLAM 轨迹。这里采用与图 3.1 相同的数据集。我们使用启用 gradientDescent 标志的 GTSAM NonlinearConjugateGradientOptimizer^[253] 实现梯度下降求解器，并使用鲁棒噪声模型实例化鲁棒损失函数。最大迭代次数和停止条件阈值（相对与绝对容差）分别设为 10000 和 10^{-7} ，其余参数保留为 GTSAM 默认值。图中，若一条边被优化正确归类为内点或离群值，则显示为灰色（即落入 Huber 或截断二次损失二次区域的内点）；若离群值被错误归为内点，即“假阳性”（false positive），则显示为红色；若内点被错误归为离群值，即“假阴性”（false negative），则显示为蓝色。与非鲁棒最小二乘优化（图 3.1(d)–(f)）相比，鲁棒损失可使 M3500 和 SubT 数据集迅速恢复离群值鲁棒性；但简单的梯度下降仍可能无法正确优化截断二次代价等高度非凸函数，如 Victoria Park 数据集所示。下文将用基于渐进非凸性（graduated non-convexity）的更好求解器处理该问题。此外，尽管梯度下降在许多实例中已改善性能（图 3.6），它仍具有缓慢的收敛尾部；在我们的实验中，往往需要数千次迭代才收敛。因此，本节其余部分讨论同时提高收敛质量和速度的更高级求解器。

M3500

(a) SubT

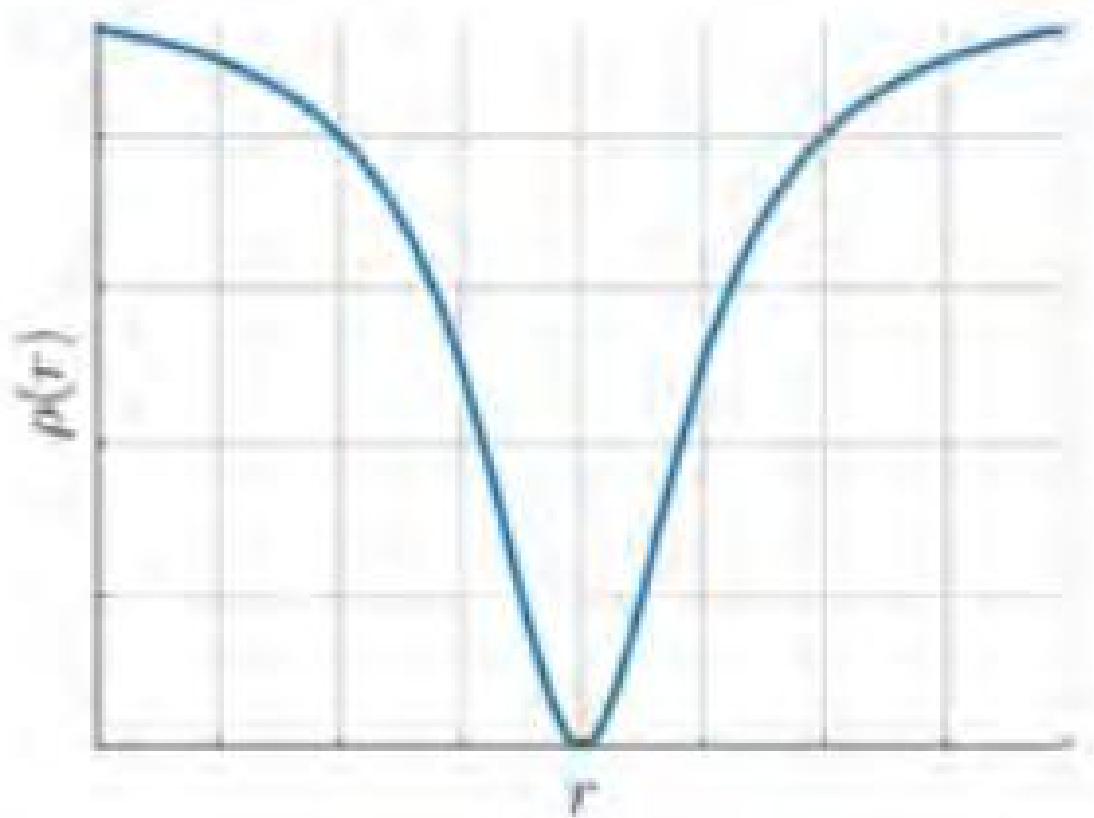
(b) Victoria Park

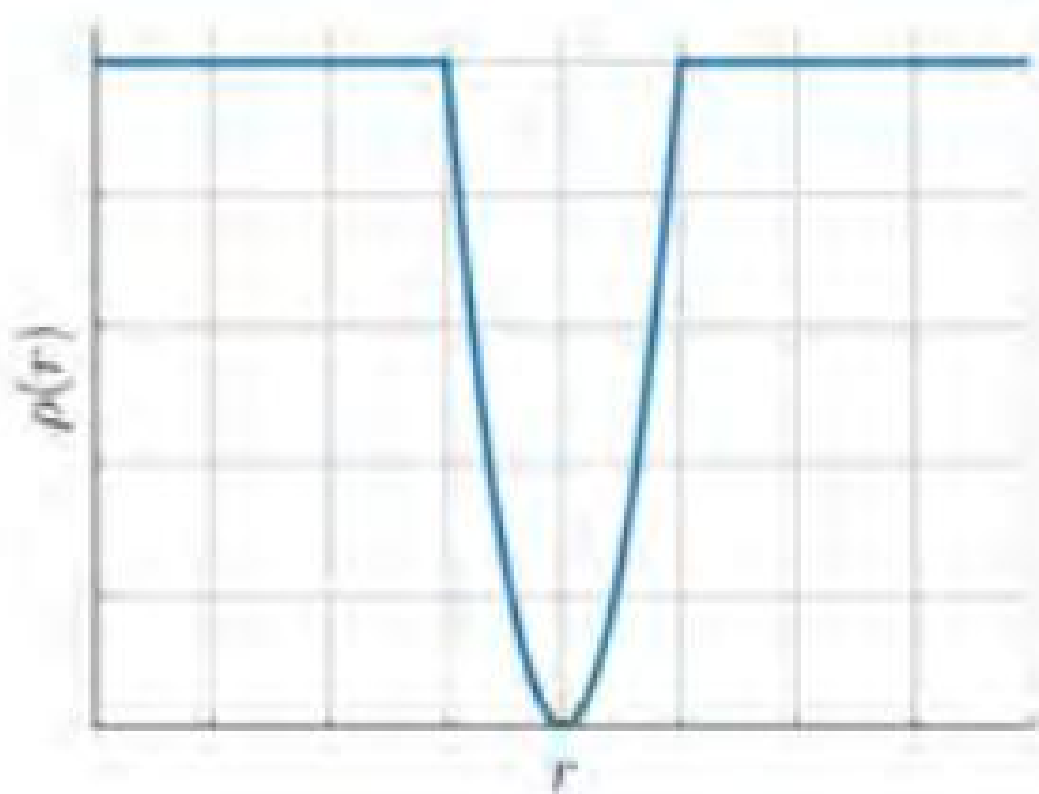
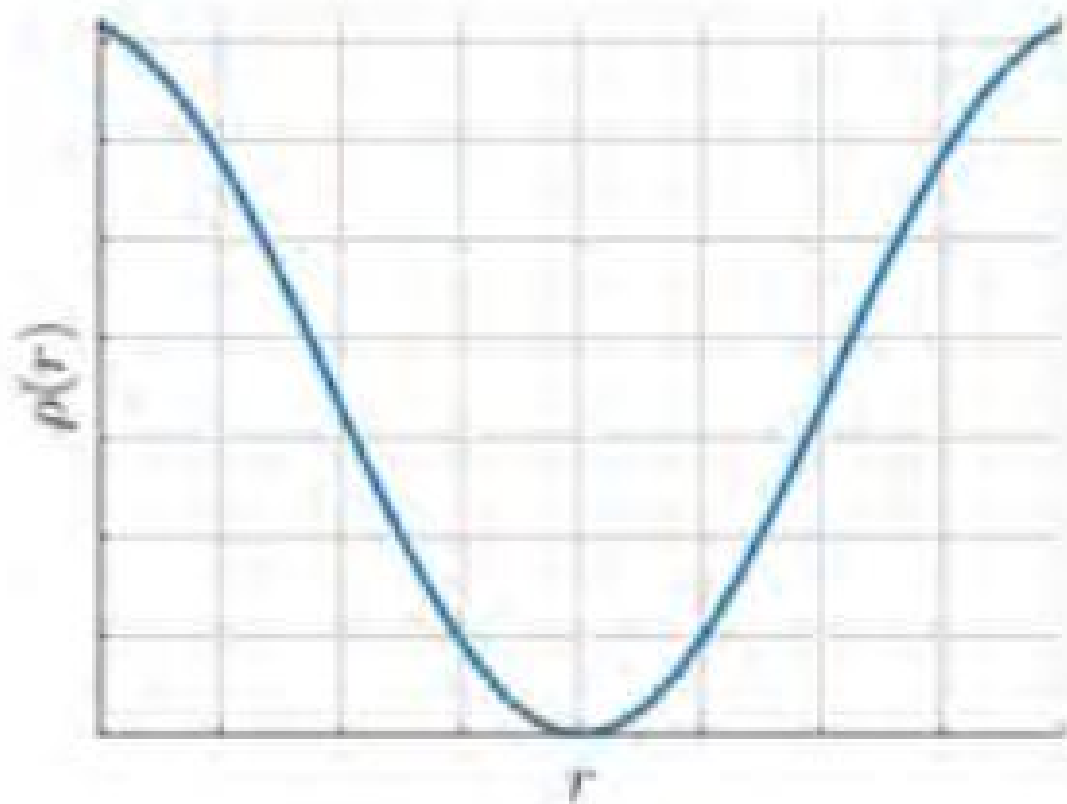
(g)

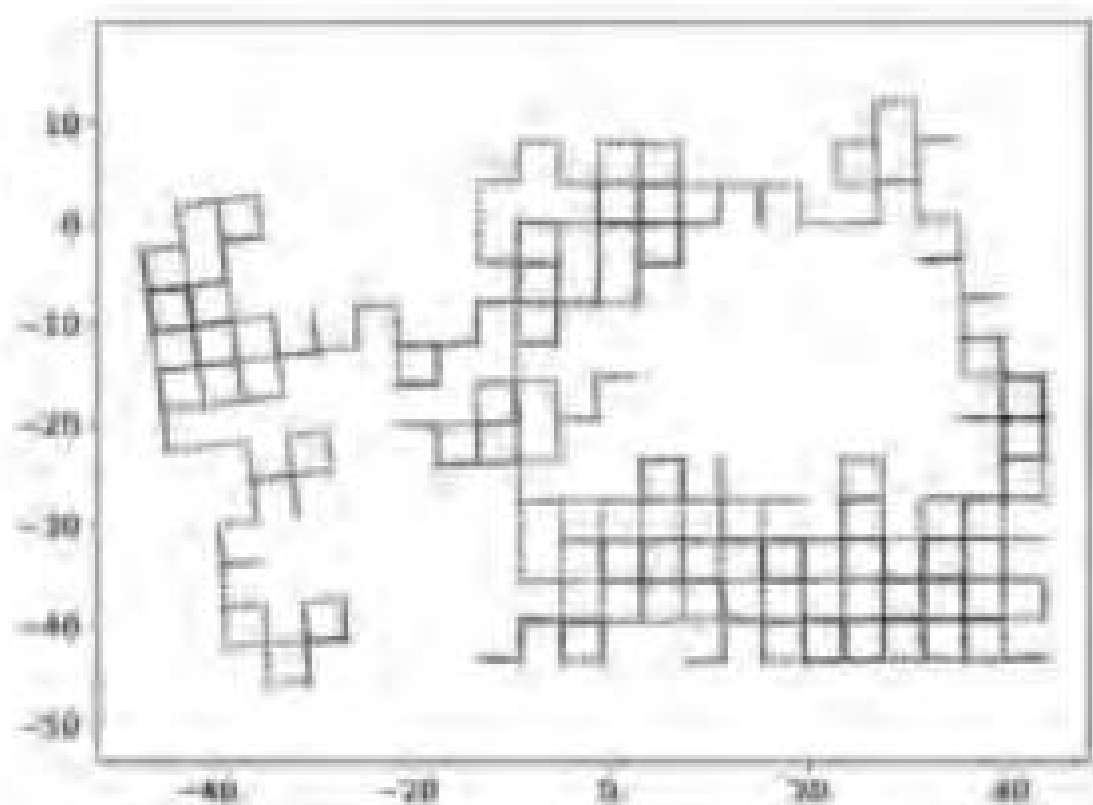
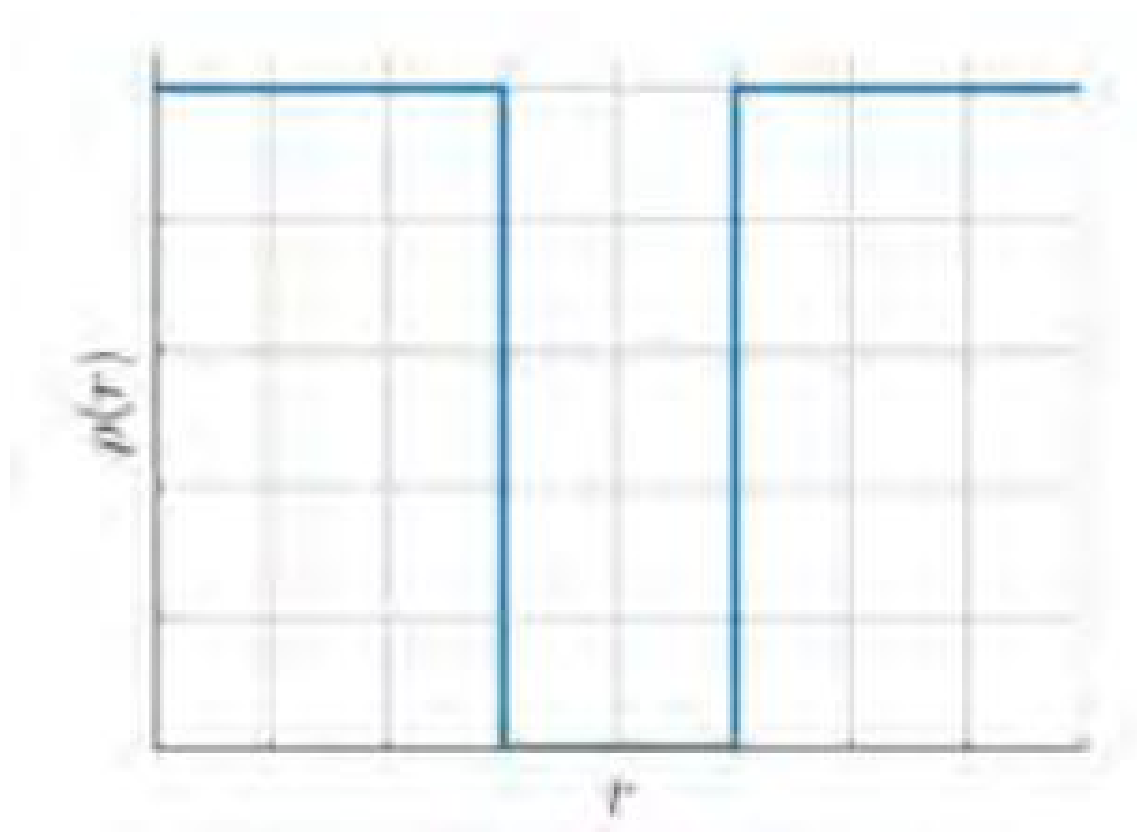
(c)

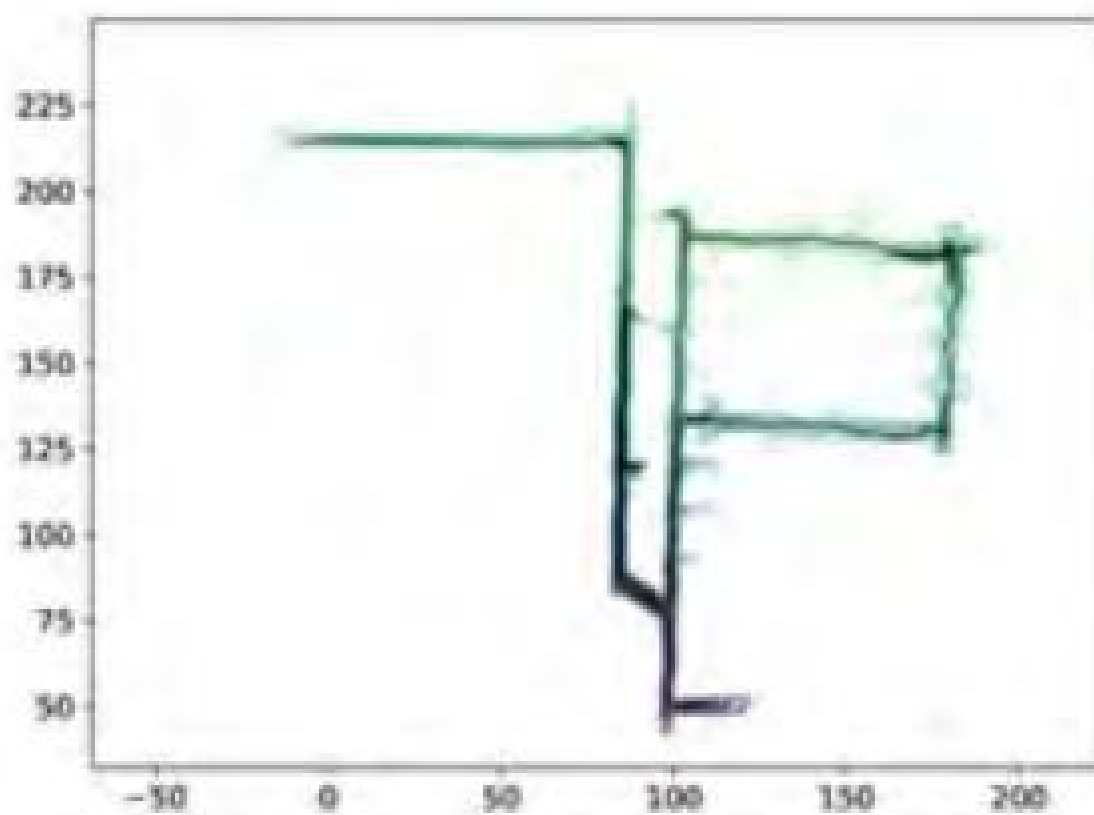
(j)

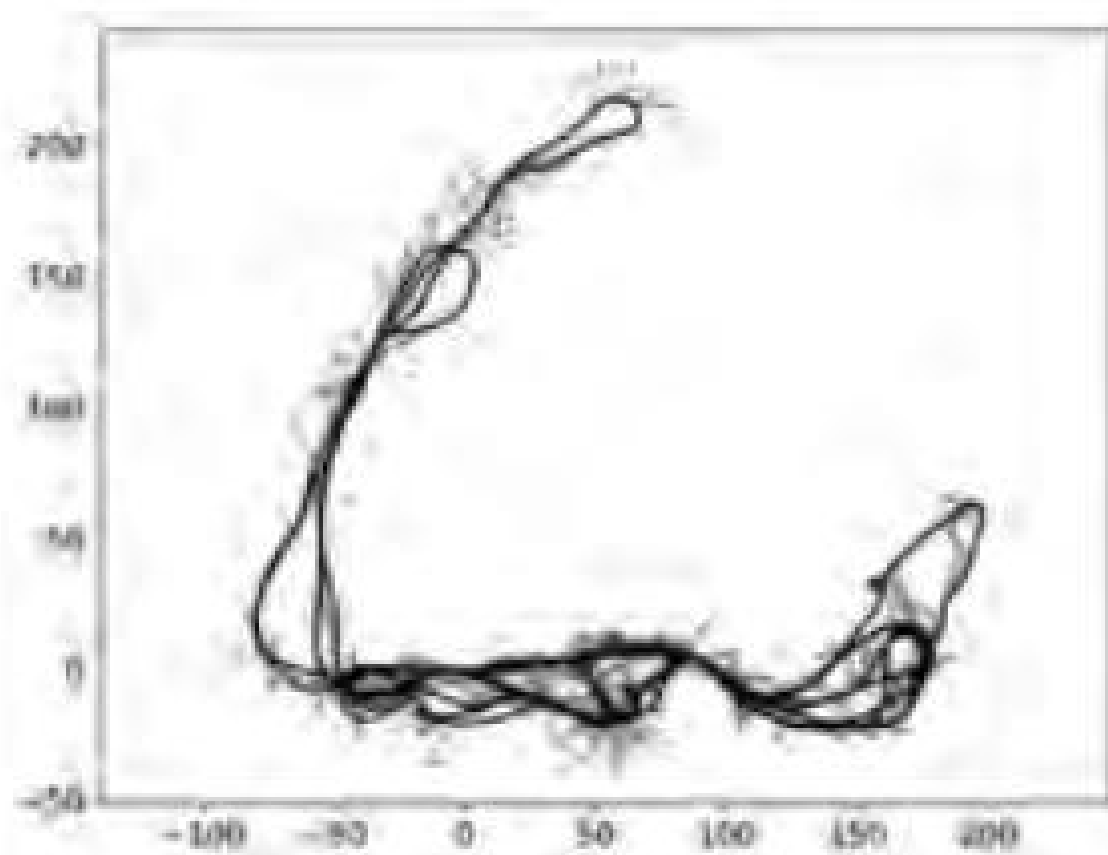
(h) (i)

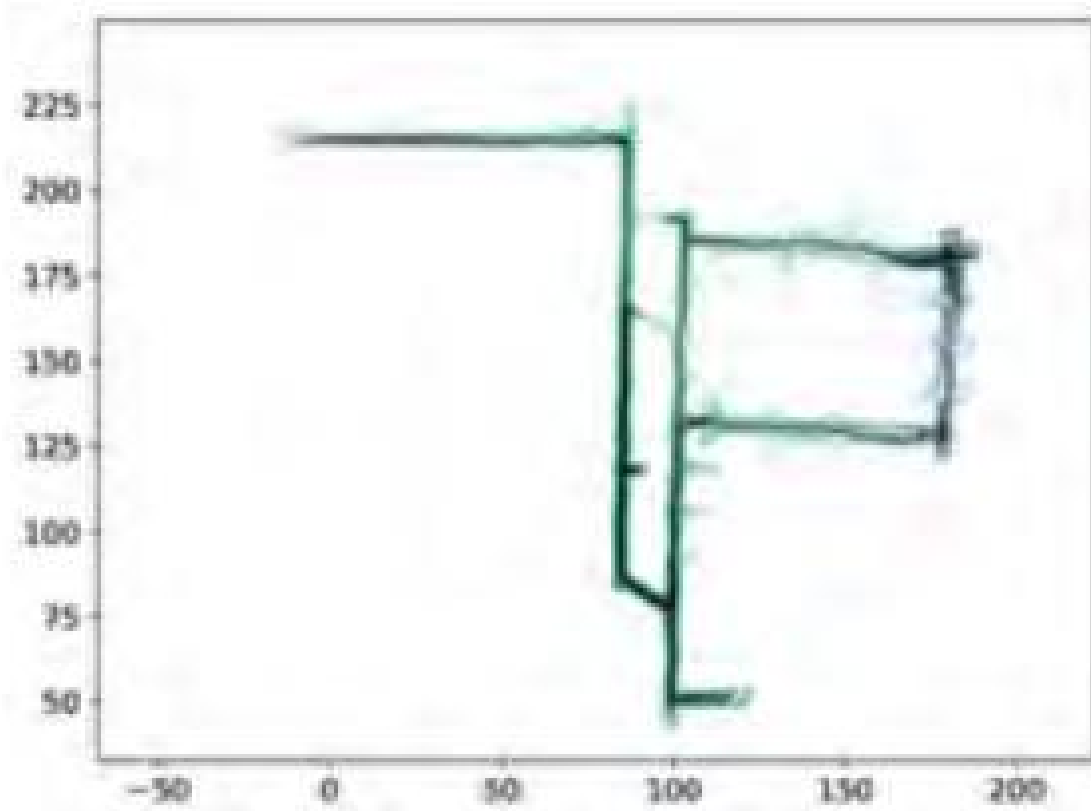
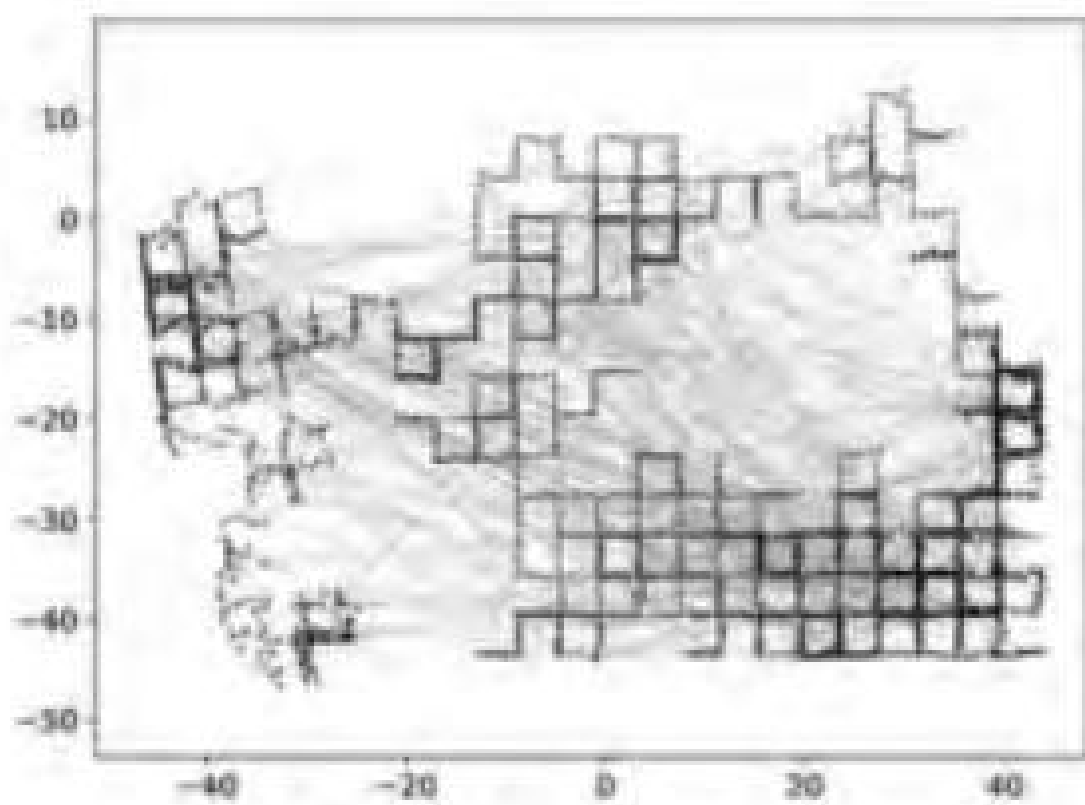


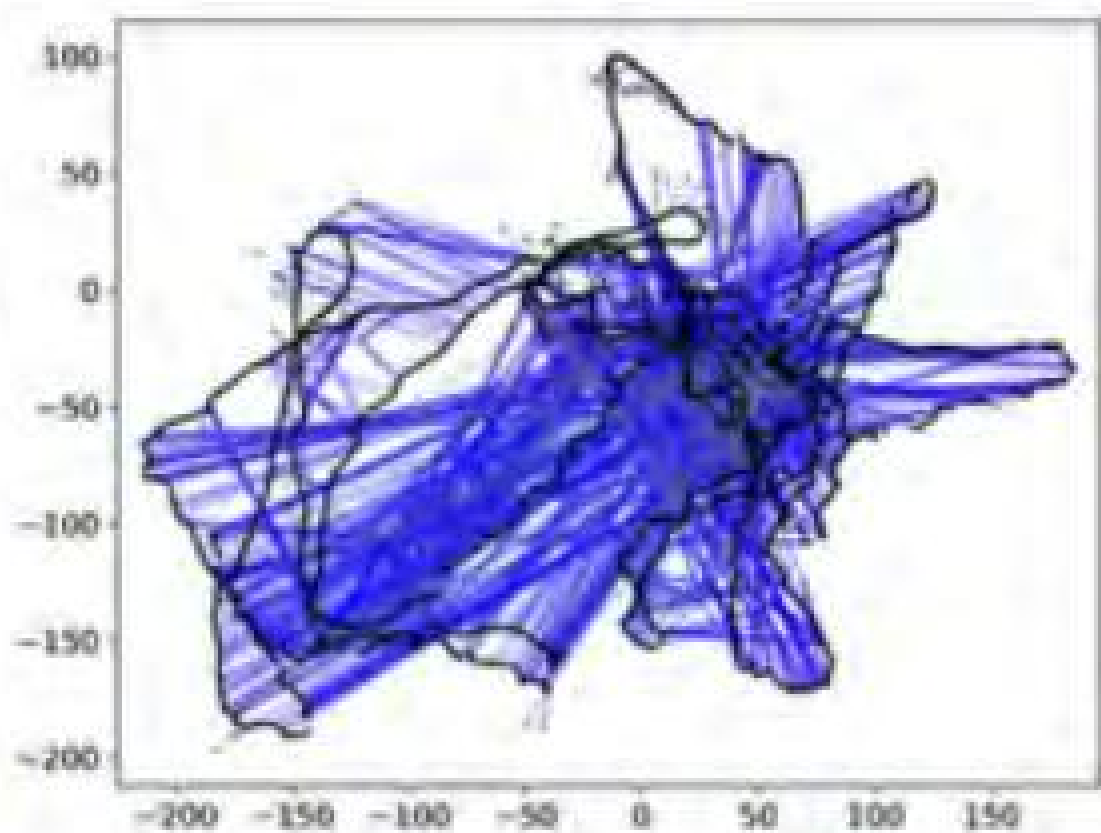


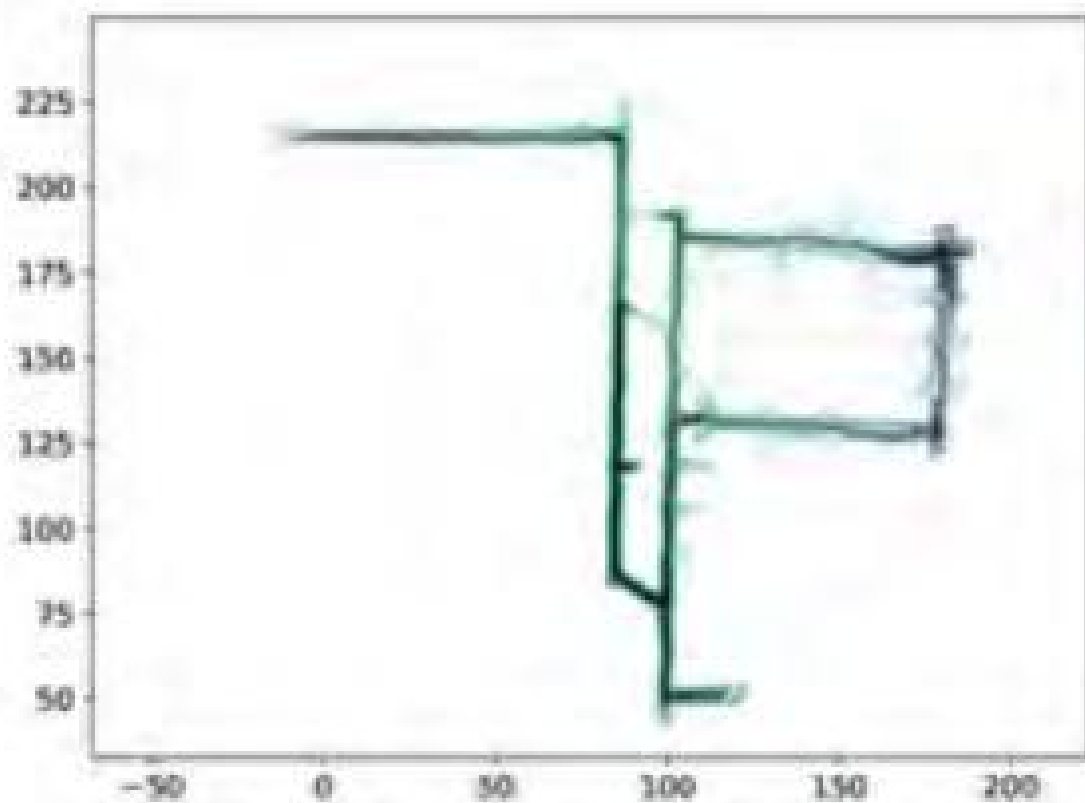
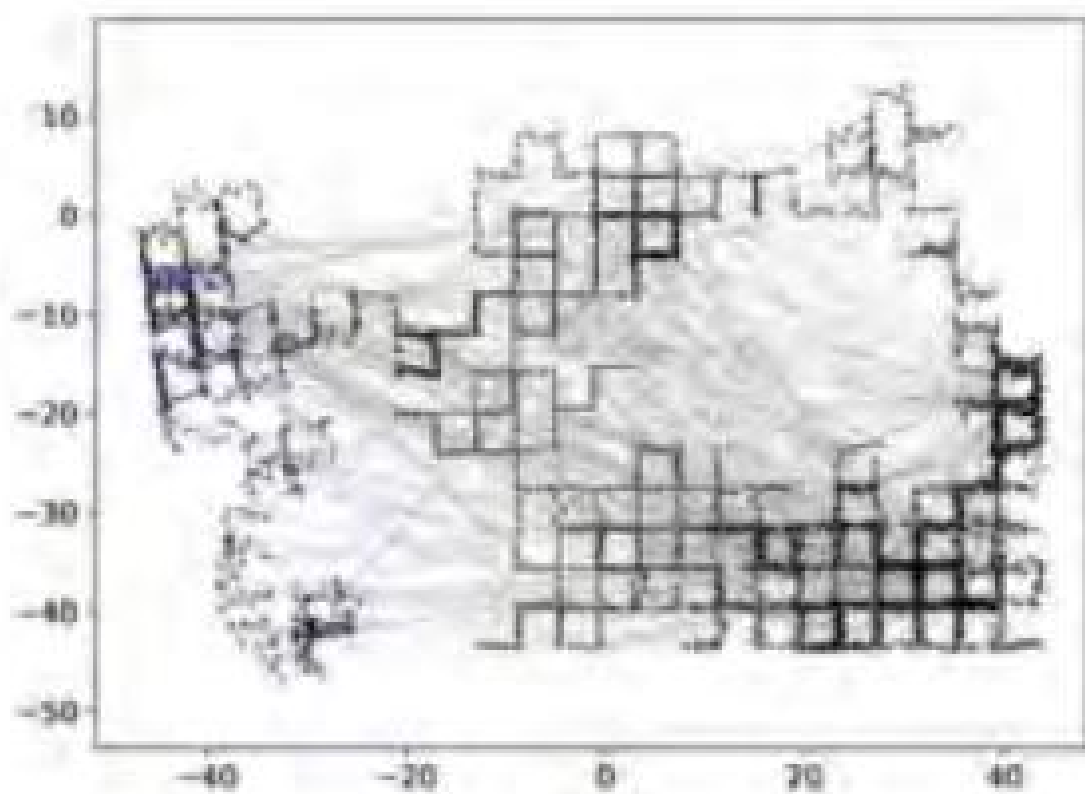




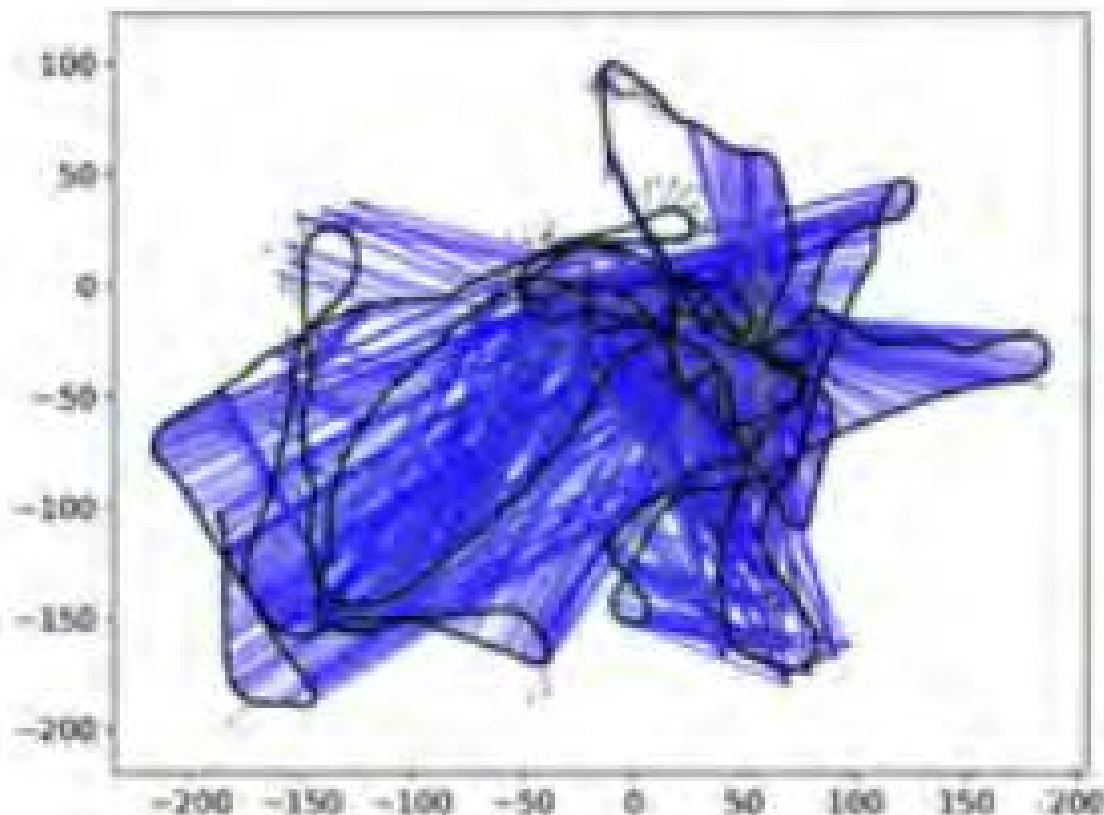








(k)



(l) 图 3.6 使用鲁棒损失函数和梯度下降求解含离群值 SLAM 问题：(a)–(c) 为 M3500、SubT 和 Victoria Park 数据集的真值轨迹；(d)–(f) 为使用梯度下降和 Huber 损失得到的轨迹估计；(g)–(l) 为使用梯度下降和截断二次损失得到的轨迹估计。测量以彩色边显示：灰色边为被优化正确归类为内点或离群值的边；红色边为被错误归类为内点的离群值（假阳性）；蓝色边为被错误归类为离群值的内点（假阴性）。

在深入讨论高级求解器前，最后指出：虽然切换到鲁棒损失函数看似放弃了概率框架，实际上可通过对重尾噪声分布应用 MAP 估计推导出若干鲁棒损失。例如，当假设噪声服从描述内点的高斯密度与描述离群值的均匀分布之间的 max-mixture 分布时，MAP 估计会导出截断二次损失^[34]。

3.8 迭代重加权最小二乘

M-估计在式 (3.9) 中用鲁棒损失 ρ 替换最小二乘损失，即用一个对大残差二次增长的函数替换它。这有两个代价。首先，我们失去了为最小二乘形式开发的高效解法；例如，高斯-牛顿和 Levenberg-Marquardt 方法是为最小二乘问题设计的。其次，由于 M-估计通常具有非凸的代价景观，迭代求解器（例如基于梯度下降的方法）对初始化质量敏感，常收敛至不期望的次优估计。本节介绍一种求解 M-估计的常用算法，即**迭代重加权最小二乘**（iteratively reweighted least squares, IRLS）；顾名思义，它允许复用高效的最小二乘求解器。下一节将

介绍渐进非凸性，以改善 IRLS 的收敛性。

IRLS 的基本思想是：在每次迭代中通过求解加权最小二乘问题来优化式 (3.9)：

$$\mathbf{x}^{(t+1)} = \arg \min_{\mathbf{x}} \sum_i w_i(\mathbf{x}^{(t)}) r_i^2(\mathbf{x}), \quad (3.11)$$

其中，权重 w_i 取决于上一次迭代的估计 $\mathbf{x}^{(t)}$ 。我们希望迭代解 $\mathbf{x}^{(t)}$ 收敛到 M-估计 (3.9) 的最优解。这意味着，式 (3.10) 所示鲁棒损失 ρ 的梯度，必须与式 (3.11) 中损失的梯度相匹配。将式 (3.11) 的梯度写为

$$\sum_i 2w_i(\mathbf{x}^{(t)}) r_i(\mathbf{x}) \frac{\partial r_i(\mathbf{x})}{\partial \mathbf{x}}$$

并与式 (3.10) 比较，可得 IRLS 权重更新规则：

$$w_i(\mathbf{x}^{(t)}) = \frac{1}{2r_i(\mathbf{x}^{(t)})} \frac{\partial \rho(r_i(\mathbf{x}^{(t)}))}{\partial r_i(\mathbf{x}^{(t)})} = \frac{\psi(r_i(\mathbf{x}^{(t)}))}{2r_i(\mathbf{x}^{(t)})}, \quad (3.12)$$

其中， $\psi(r_i) := \frac{\partial \rho(r_i)}{\partial r_i}$ 是影响函数。因而，IRLS 在为每个测量 i 计算权重 $w_i(\mathbf{x}^{(t)})$ 与对加权最小二乘问题 (3.11) 执行优化步骤（即一次 Gauss–Newton 或 Levenberg–Marquardt 迭代）之间交替进行。

图 3.7 给出了 IRLS 在 M3500、SubT 和 Victoria Park 数据集上的表现。IRLS 在数十次迭代内收敛，通常比梯度下降快得多；例如，在我们的 M3500 实验中，梯度下降优化 Huber 损失约需 5 秒，而 IRLS 少于 1.5 秒。另一方面，这种更快的收敛往往以精度略有下降为代价，比较图 3.7 与图 3.6 即可看出。文献 [10, 817] 已研究了更新规则 (3.12) 的收敛性质。

3.9 Black–Rangarajan 对偶性

权重更新规则 (3.12) 在实践中广泛使用，但其推导带有一定启发性；此外，在 ρ 的不可微点（例如截断二次损失的截断点）处，式 (3.12) 没有定义。下面引入一个更有原则的框架，即 Black–Rangarajan (B–R) 对偶性 [85]，用于通过 IRLS 求解 M-估计。

使用截断二次损失说明 B–R 对偶性的直觉：

$$\rho(r_i(\mathbf{x})) := \min\{r_i^2(\mathbf{x}), \beta_i^2\}, \quad (3.13)$$

其中， β_i^2 是第 i 个残差的界：若 $r_i^2(\mathbf{x}) \leq \beta_i^2$ ，第 i 个测量为内点，否则为离群值。通过引入新的权重变量 $w_i \in [0, 1]$ ，可将式 (3.13) 的代价等价地写作两项之和：

$$\rho(r_i(\mathbf{x})) := \min_{w_i \in [0, 1]} w_i r_i^2(\mathbf{x}) + (1 - w_i) \beta_i^2, \quad (3.14)$$

其中第一项恰为加权最小二乘，第二项是与 $\mathbf{x}$ 无关的 w_i 的函数。借助式 (3.14)，使用截断二次损失的 M-估计问题 (3.9) 可改写为

$$\min_{\mathbf{x}, w_i \in [0, 1], i=1, \dots, N} \sum_i [w_i r_i^2(\mathbf{x}) + (1 - w_i) \beta_i^2], \quad (3.15)$$

其中，对每个测量残差 $r_i(\mathbf{x})$ 都引入一个 w_i 。问题 (3.15) 容易解释： $w_i = 1$ 意味着 $r_i^2(\mathbf{x}) \leq \beta_i^2$ ，第 i 个测量为内点； $w_i = 0$ 意味着 $r_i^2(\mathbf{x}) > \beta_i^2$ ，第 i 个测量为离群值。此外，所有 $w_i = 0$ 的残差均被有效地从优化问题 (3.15) 中丢弃，因而保证了鲁棒性。

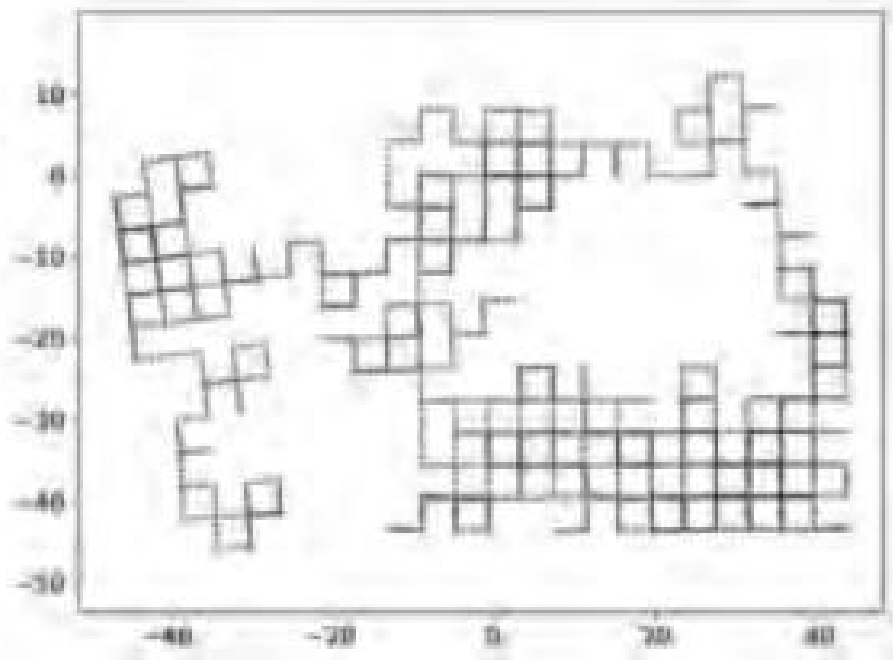
B-R 对偶性将上述推导推广到一族鲁棒损失。

定理 3.4 (Black-Rangarajan 对偶性^[85]) 给定鲁棒损失函数 $\rho(\cdot)$ ，定义 $\phi(z) := \rho(\sqrt{z})$ 。若 $\phi(z)$ 满足 $\lim_{z \rightarrow 0} \phi'(z) = 1$ 、 $\lim_{z \rightarrow \infty} \phi'(z) = 0$ 且 $\phi''(z) < 0$ ，则 M-估计问题 (3.9) 等价于

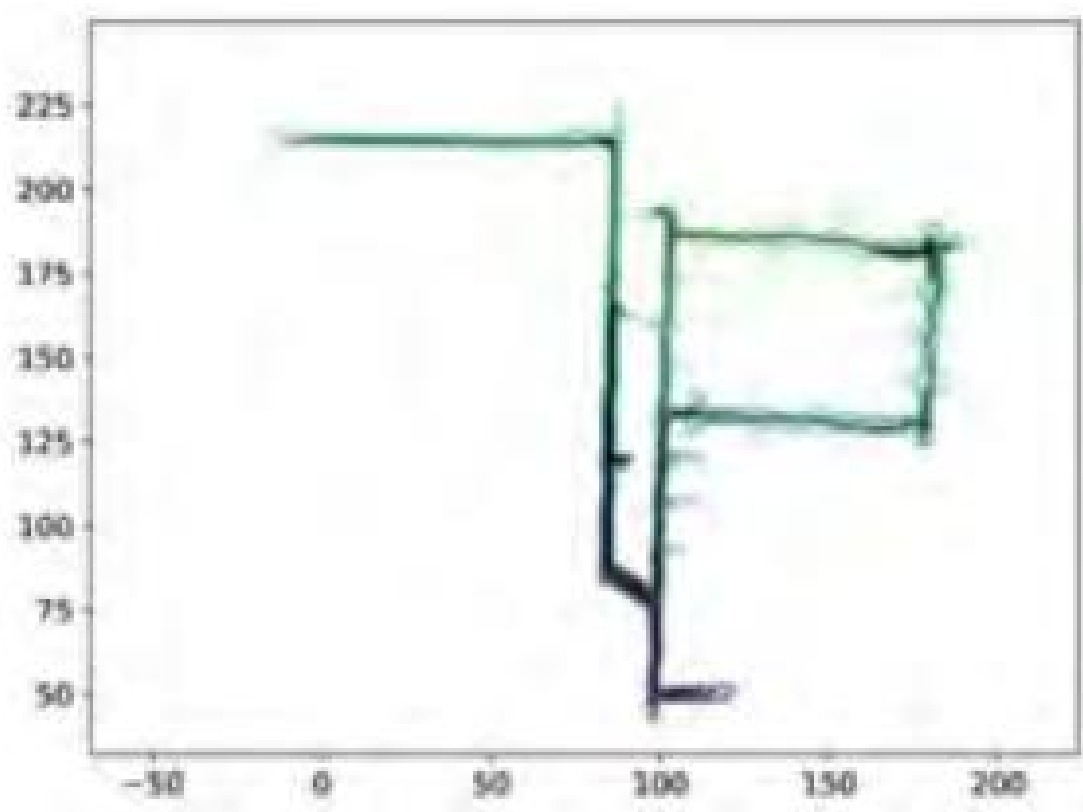
$$\min_{\mathbf{x}, w_i \in [0, 1], i=1, \dots, N} \sum_{i=1}^N [w_i r_i^2(\mathbf{x}) + \Phi_\rho(w_i)], \quad (3.16)$$

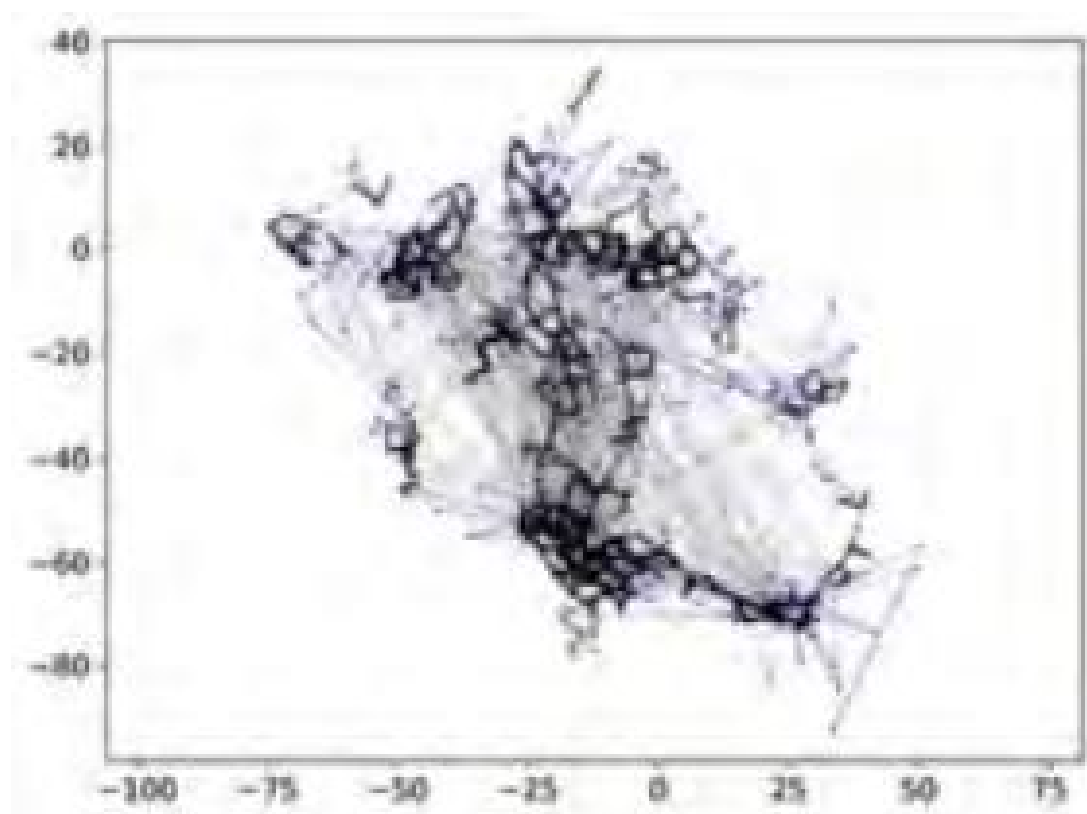
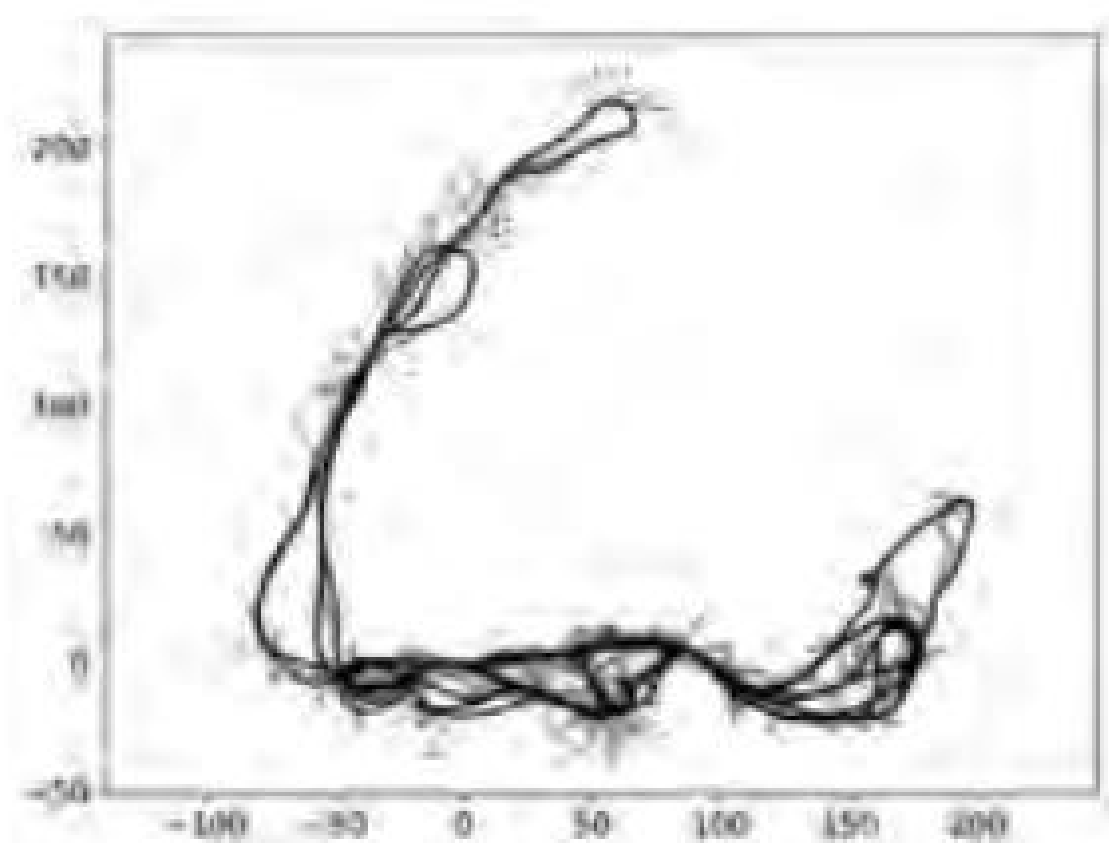
其中， $w_i \in [0, 1], i = 1, \dots, N$ 是与每个残差 r_i 关联的权重变量；函数 $\Phi_\rho(w_i)$ 称为**离群值过程** (outlier process)，它定义对权重 w_i 的惩罚，其形式取决于鲁棒损失 ρ 的选择。

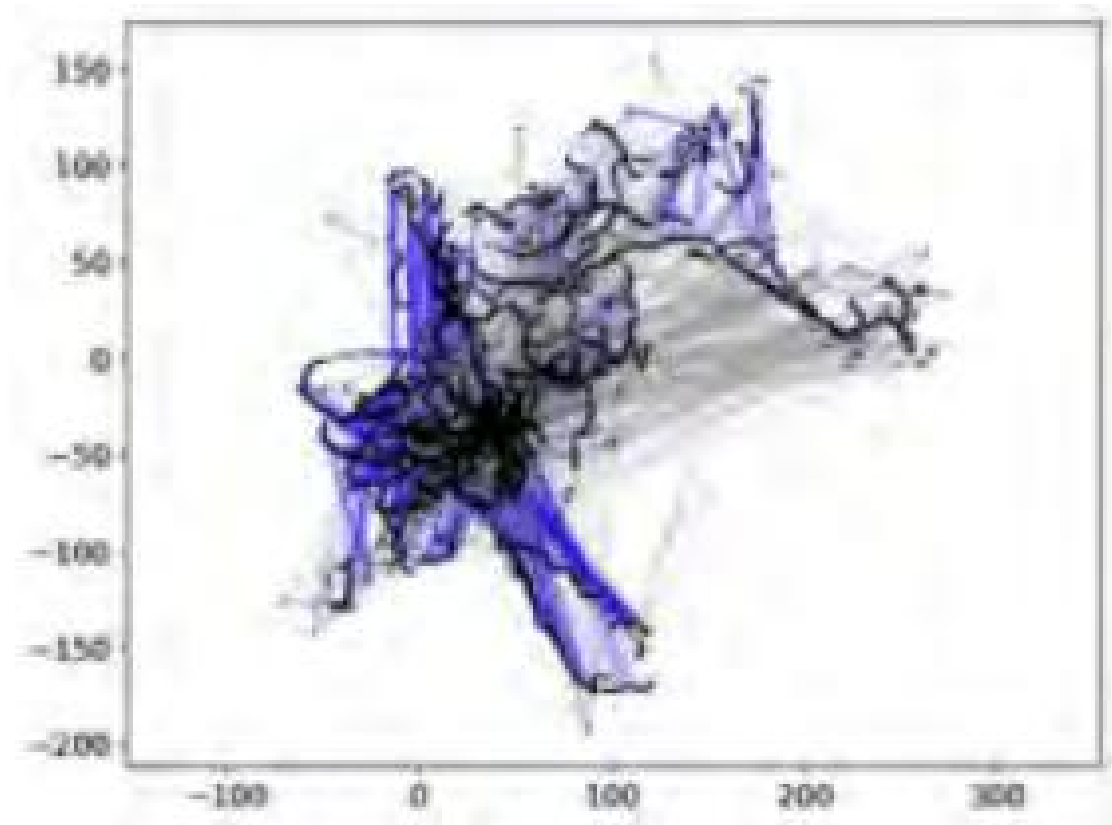
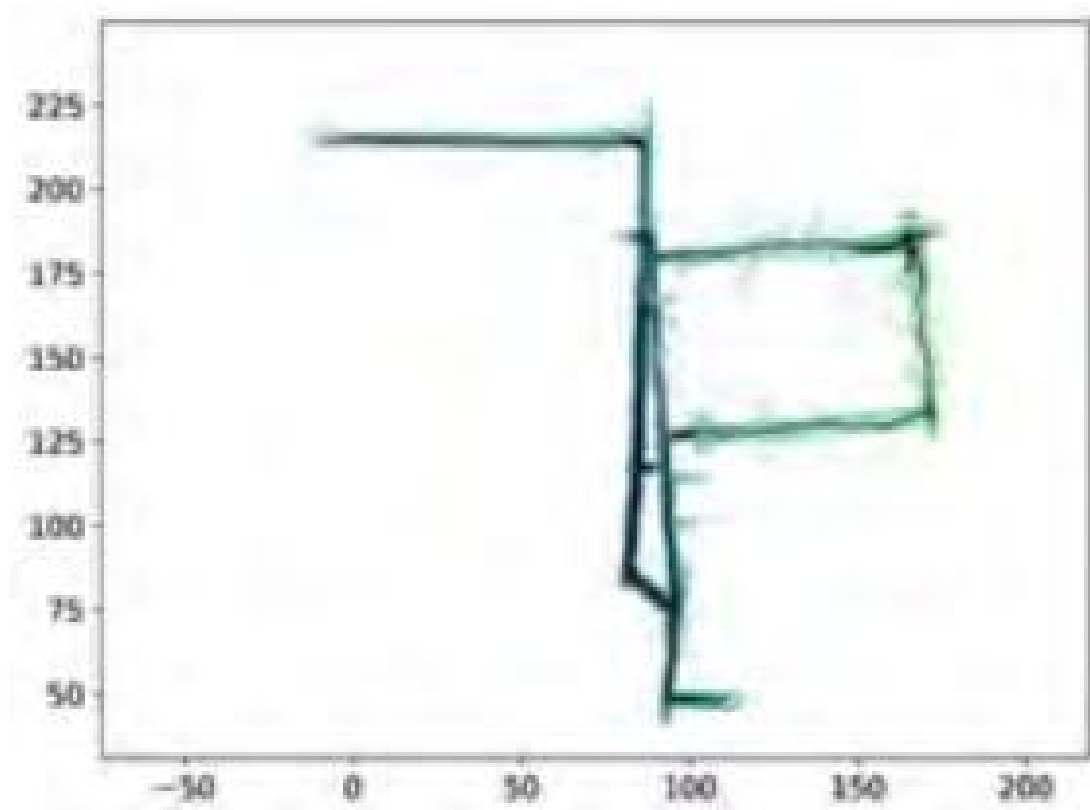
Ground Truth



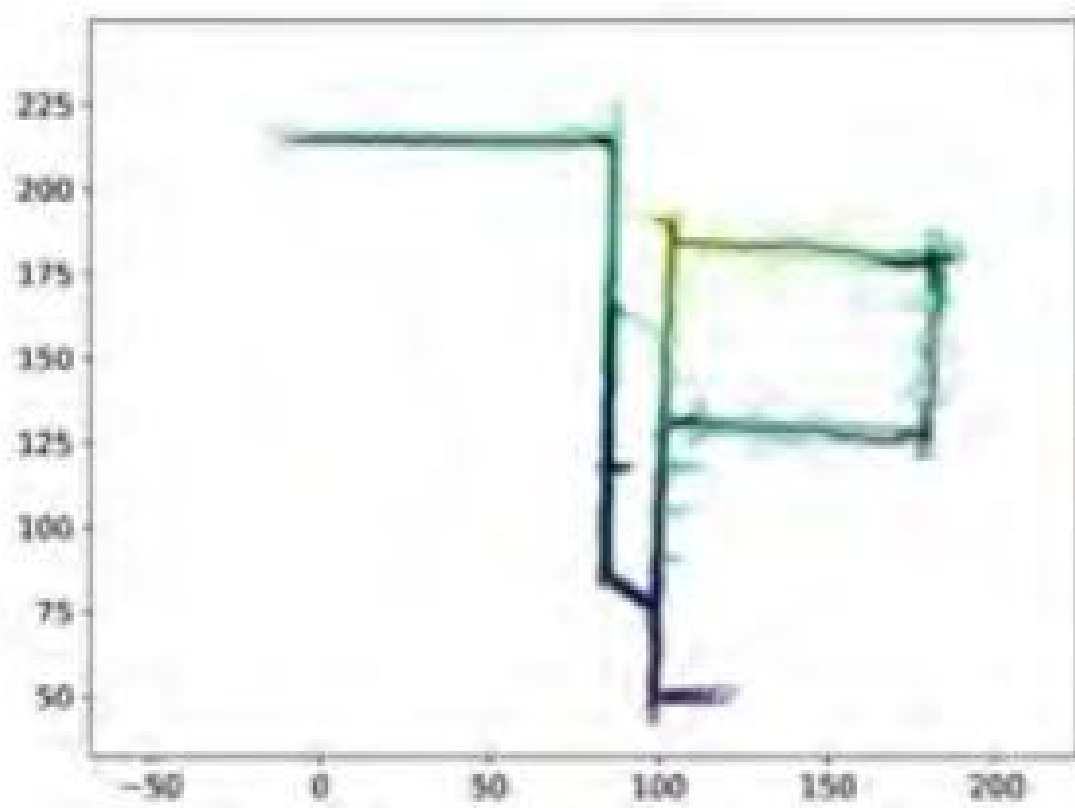
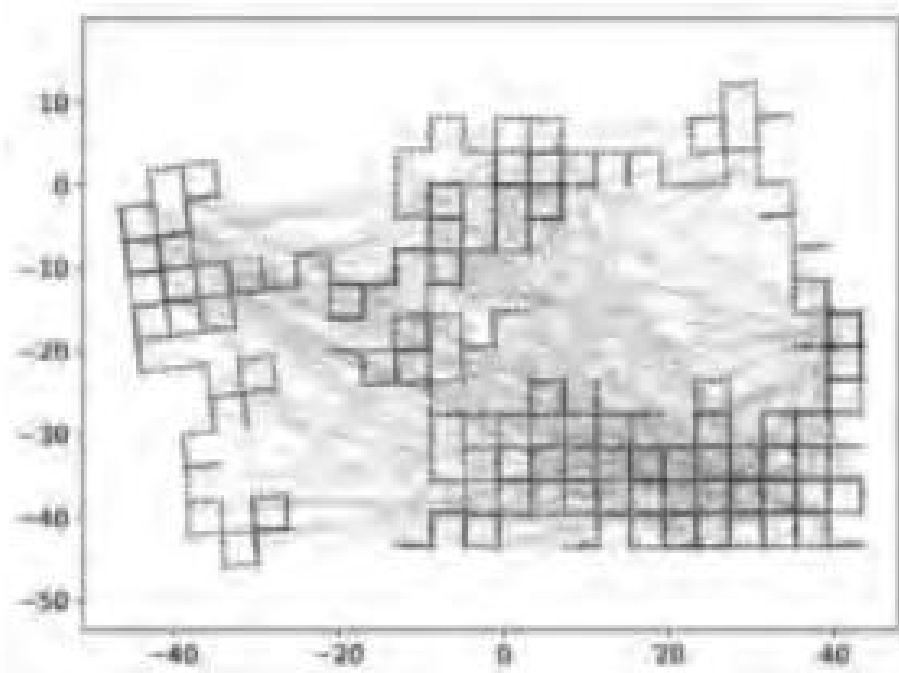
- (a)
- (b)
- (c)
- (d)
- (e)
- (f)
- (g)
- (j)
- (h)
- (k)
- (i)

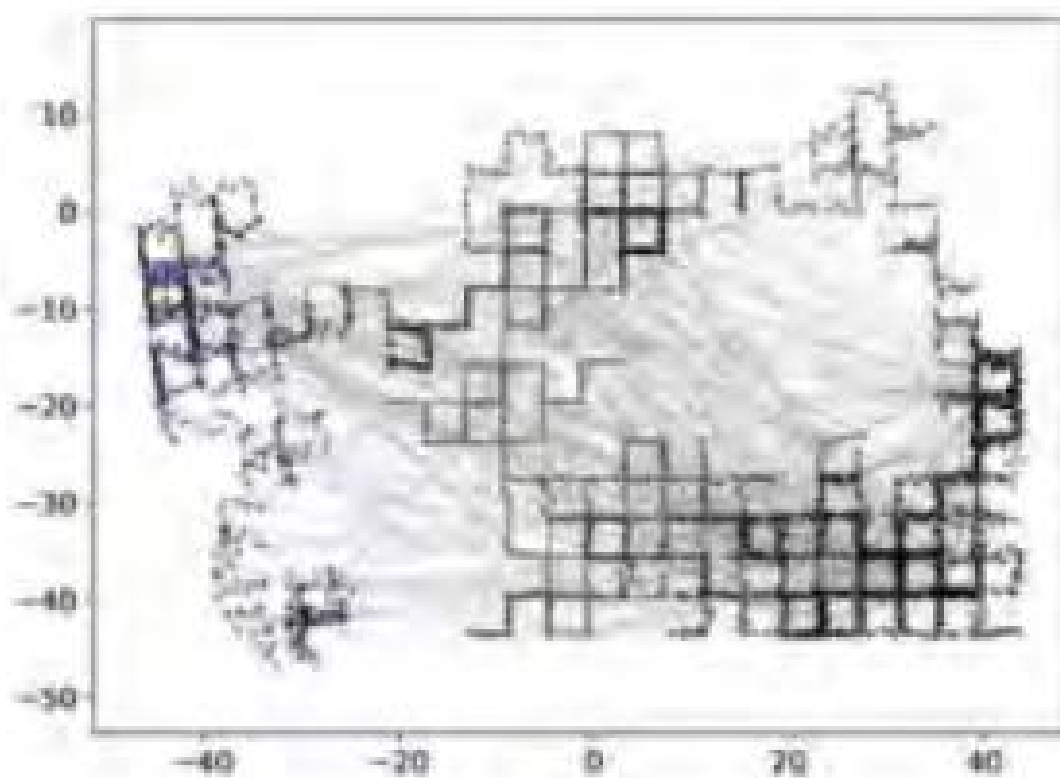
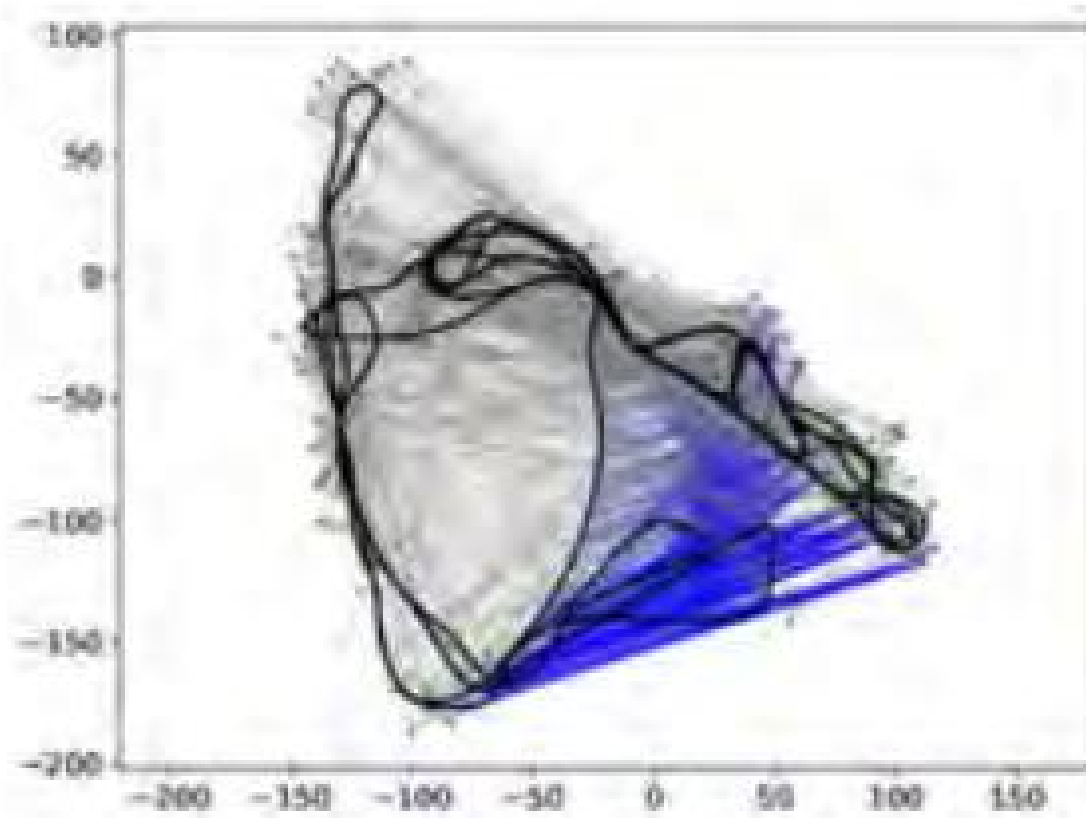


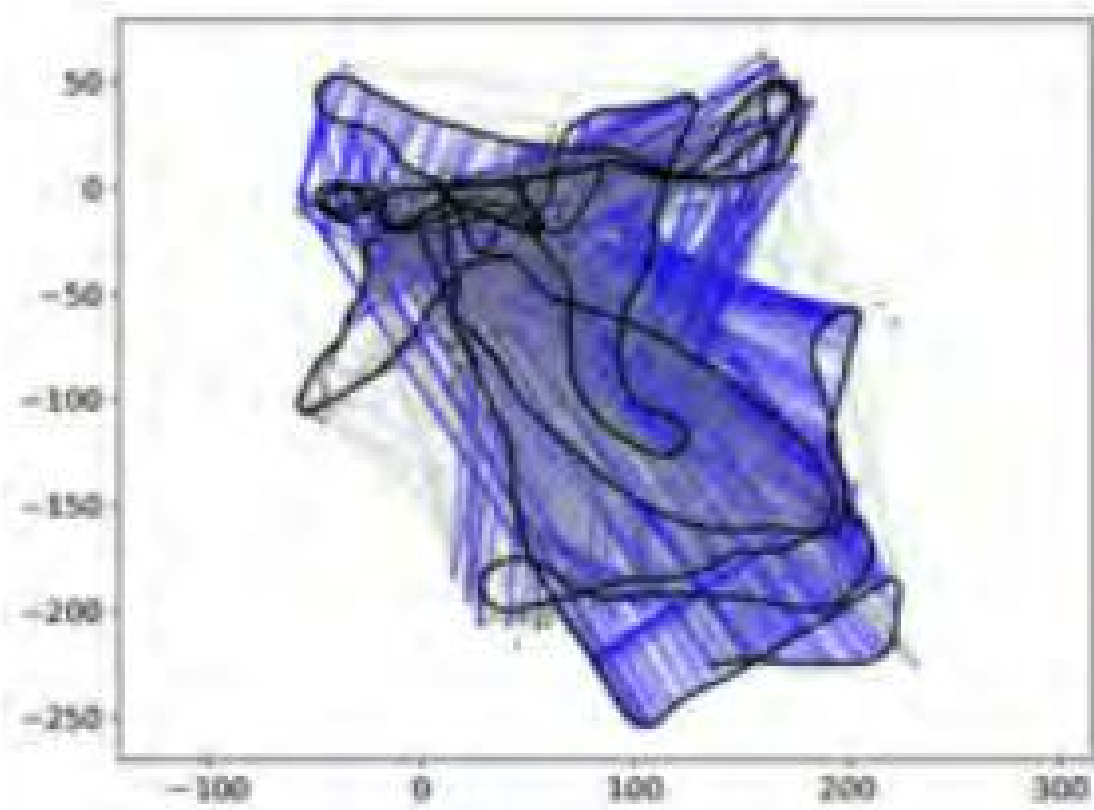
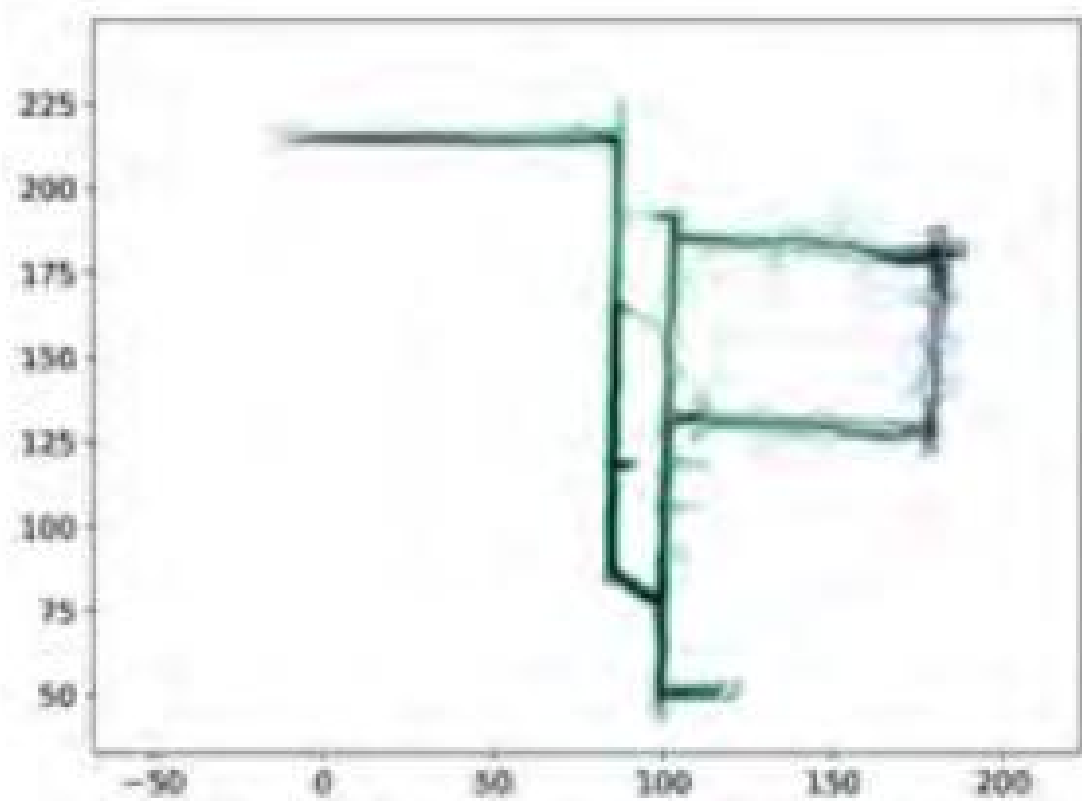




IRLS + GM







(l) 图 3.7 使用鲁棒损失函数和迭代重加权最小二乘 (IRLS) 求解含离群值的 SLAM 问题: (a)–(c) 为 M3500、SubT 和 Victoria Park 数据集的真值轨迹; (d)–(f) 为使用 Huber 损失得到的轨迹估计; (g)–(i) 为使用 Geman–McClure 损失得到的轨迹估计; (j)–(l) 为使用截断二次损失得到的轨迹估计。测量以彩色边显示: 灰色边为正确分类的边; 红色边为被错误归类为内点的离群值 (假阳性); 蓝色边为被错误归类为离群值的内点 (假阴性)。

当 ρ 为截断二次损失时, 我们已由式 (3.14) 轻易得到 $\Phi_\rho(w_i) = (1 - w_i)\beta_i^2$ 。当 ρ 取其他形式时, 文献^[85] 提供了推导 $\Phi_\rho(w_i)$ 的方法。下面给出 Geman–McClure (G–M) 鲁棒损失的例子。

例 3.5 (G–M 损失的 B–R 对偶性) 考虑 G–M 鲁棒损失函数

$$\rho(r_i(\mathbf{x})) = \frac{\beta_i^2 r_i^2(\mathbf{x})}{\beta_i^2 + r_i^2(\mathbf{x})}, \quad (3.17)$$

其中, β_i^2 是与式 (3.13) 类似的第 i 个残差的噪声界。与式 (3.17) 关联的离群值过程为

$$\Phi_\rho(w_i) = \beta_i^2(\sqrt{w_i} - 1)^2. \quad (3.18)$$

为验证式 (3.18) 的正确性, 考虑

$$\min_{w_i \in [0,1]} w_i r_i^2(\mathbf{x}) + \Phi_\rho(w_i), \quad (3.19)$$

其最优解 (令式 (3.19) 的梯度为零) 是

$$w_i^* = \left(\frac{\beta_i^2}{r_i^2(\mathbf{x}) + \beta_i^2} \right)^2. \quad (3.20)$$

将式 (3.20) 代回式 (3.19) 的目标函数, 即可恢复 G–M 鲁棒损失 (3.17)。

3.10 交替最小化

引入 B–R 对偶性后, 使用称为**交替最小化** (alternating minimization) 的通用优化策略^[1112, 80], IRLS 算法便可自然导出。其思想是: 尽管在式 (3.16) 中同时优化 x 和 $w_i \in [0, 1]$, $i = 1, \dots, N$ 很困难, 但固定其中一者而优化另一者很容易。确实, 当 w_i 固定时, 问题 (3.16) 成为加权最小二乘; 类似地, 当 x 固定时, 问题 (3.16) 成为

$$\min_{w_i \in [0,1], i=1, \dots, N} \sum_{i=1}^N \Phi_\rho(w_i) + w_i r_i^2(\mathbf{x}),$$

它可拆为 N 个子问题, 每个子问题优化一个标量 w_i :

$$\min_{w_i \in [0,1]} \Phi_\rho(w_i) + w_i r_i^2(\mathbf{x}). \quad (3.21)$$

问题 (3.21) 易于求解, 且常有闭式解。事实上, 对 G–M 鲁棒损失, 式 (3.21) 的解正是式 (3.20)。对截断二次损失, 问题 (3.21) 为

$$\min_{w_i \in [0,1]} (1 - w_i)\beta_i^2 + w_i r_i^2(\mathbf{x})$$

并具有闭式解

$$w_i^* = \begin{cases} 1, & \text{if } r_i^2(\mathbf{x}) < \beta_i^2, \\ 0, & \text{if } r_i^2(\mathbf{x}) > \beta_i^2, \\ [0, 1], & \text{otherwise.} \end{cases}$$

总之，在 B-R 对偶性的语境下，第 t 次 IRLS 迭代在以下两步之间交替：

1. **变量更新**：使用当前权重 $w_i^{(t)}$ 求解加权最小二乘问题

$$\mathbf{x}^{(t)} \in \arg \min_{\mathbf{x}} \sum_i w_i^{(t)} r_i^2(\mathbf{x}). \quad (3.22)$$

2. **权重更新**：使用 $\mathbf{x}^{(t)}$ 更新权重

$$w_i^{(t+1)} \in \arg \min_{w_i \in [0,1]} \Phi_\rho(w_i) + w_i r_i^2(\mathbf{x}^{(t)}), i = 1, \dots, N. \quad (3.23)$$

通过 B-R 对偶性得到的权重更新规则实际上与广为使用的规则 (3.12) 一致。有趣的是，在 SLAM 中用 G-M 鲁棒损失实例化上述 IRLS 算法，会导出动态协方差缩放 (dynamic covariance scaling) 算法^[11]，该算法已在离群值鲁棒 SLAM 的语境下提出。

3.11 渐进非凸性

上一节利用 Black-Rangarajan 对偶性与交替最小化，导出了在式 (3.22) 和 (3.23) 之间交替求解的 IRLS 框架。然而，由于常见鲁棒损失的非凸性，IRLS 的收敛可能对初始化质量高度敏感，即 $\mathbf{x}^{(0)}$ 与式 (3.9) 最优解的接近程度，或 $w_i^{(0)}$ 对各测量内点/离群值归属的刻画程度。例如，文献^[1006]表明，即使测量中仅有 10% 离群值，采用截断二次损失或 Geman-McClure 损失的 IRLS 也可能失败（亦可参见图 3.7 的结果）。

本节介绍**渐进非凸性** (Graduated Non-Convexity, GNC) 算法，它可使 IRLS 对初始化质量显著不那么敏感。给定鲁棒代价函数 ρ ，GNC 的基本思想是利用标量平滑因子 μ 构造 ρ 的平滑版本 ρ_μ 。调节 μ 可控制 ρ_μ 的非凸程度：在参数范围的一端， ρ_μ 是凸的；在另一端，它恢复原始的 ρ 。下面以两个例子说明。

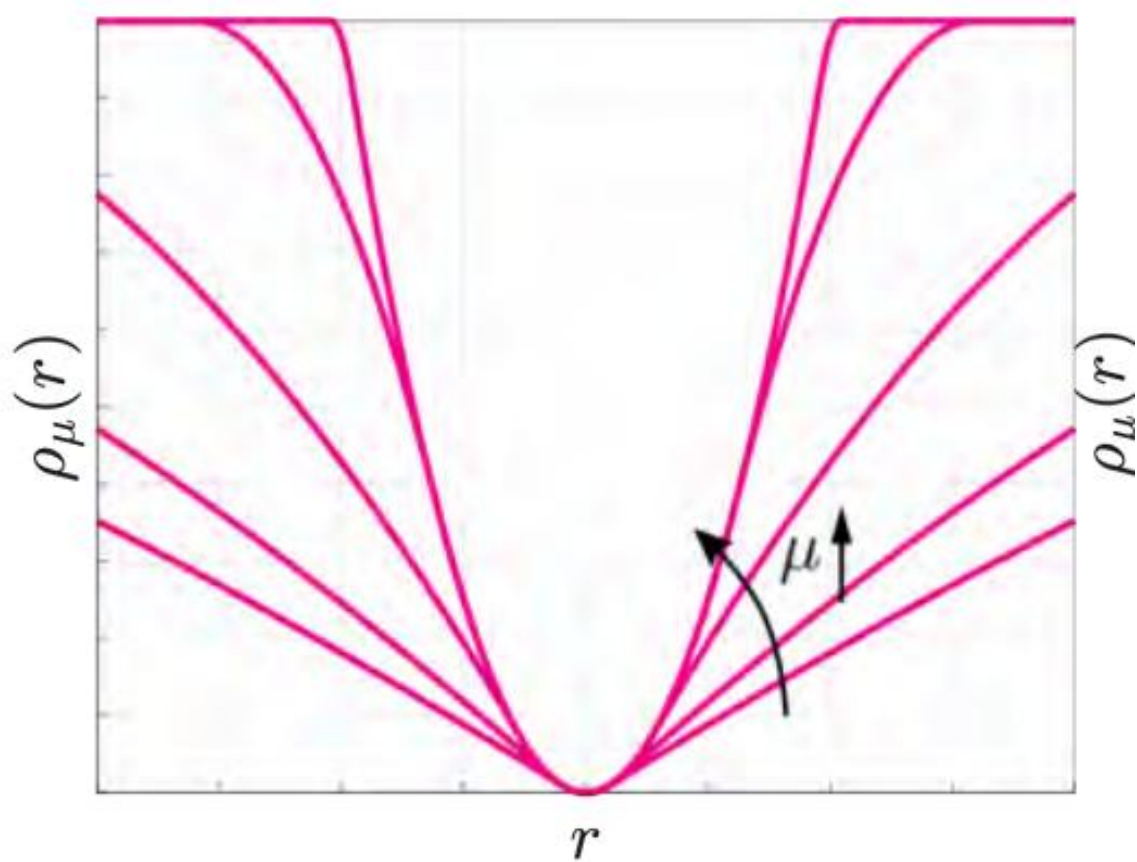
例 3.6 (GNC 截断二次损失) 考虑 GNC 截断二次损失函数

$$\rho_\mu(r_i(\mathbf{x})) = \begin{cases} r_i^2(\mathbf{x}) & \text{if } r_i^2(\mathbf{x}) \in \left[0, \frac{\mu}{\mu+1}\beta_i^2\right] \\ 2\beta_i|r_i(\mathbf{x})|\sqrt{\mu(\mu+1)} - \mu(\beta_i^2 + r_i^2(\mathbf{x})) & \text{if } r_i^2(\mathbf{x}) \in \left[\frac{\mu}{\mu+1}\beta_i^2, \frac{\mu+1}{\mu}\beta_i^2\right] \\ \beta_i^2 & r_i^2(\mathbf{x}) \in \left[\frac{\mu+1}{\mu}\beta_i^2, +\infty\right]. \end{cases} \quad (3.24)$$

当 μ 趋近于零时， ρ_μ 为凸函数；当 μ 趋近于无穷大时，它恢复式 (3.13) 的截断二次损失。

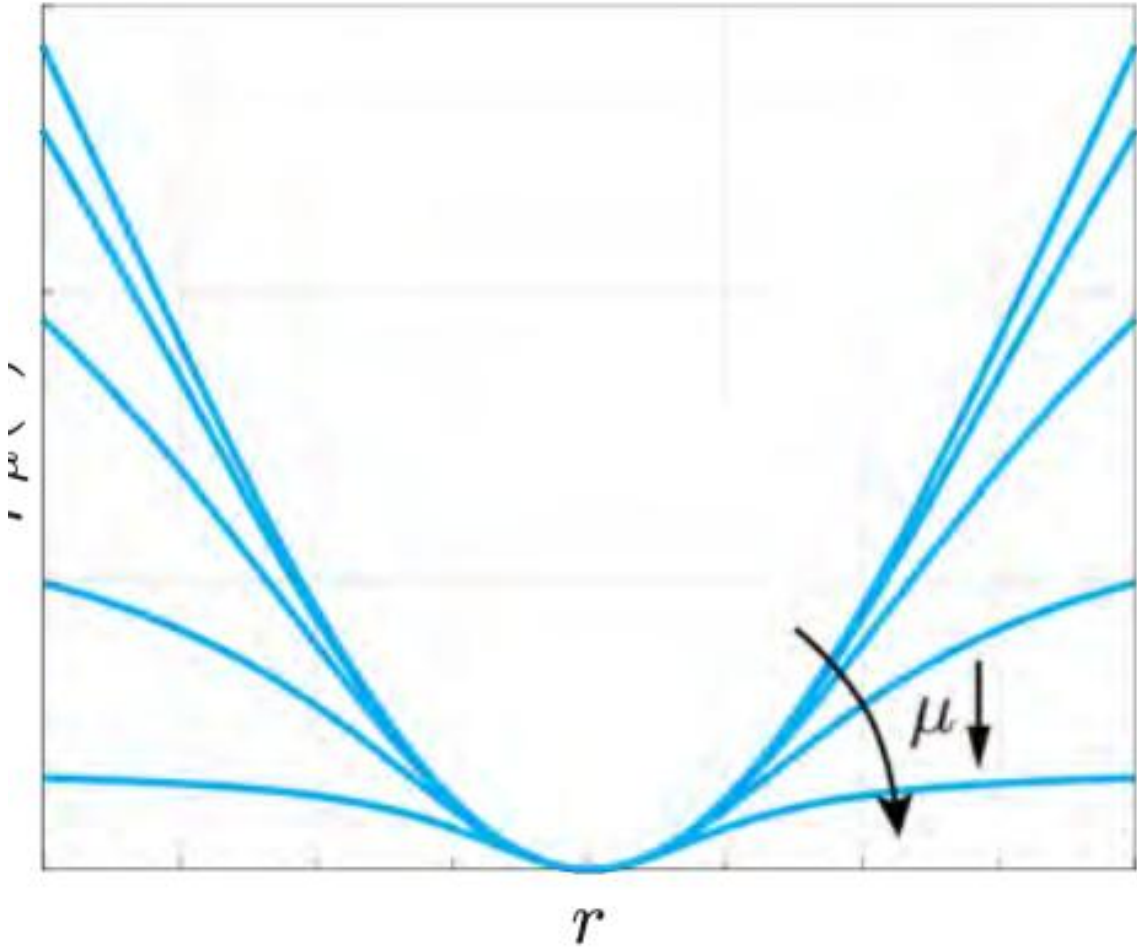
例 3.7 (GNC Geman-McClure 损失) 考虑 GNC Geman-McClure 损失函数

$$\rho_\mu(r_i(\mathbf{x})) = \frac{\mu\beta_i^2 r_i^2(\mathbf{x})}{\mu\beta_i^2 + r_i^2(\mathbf{x})}. \quad (3.25)$$



当 μ 趋近于无穷大时, ρ_μ 为凸函数; 当 $\mu = 1$ 时, 它恢复 G-M 损失 (3.17)。

(a)



(b) 图 3.8 使用控制参数 μ 的渐进非凸性: (a) 截断二次损失; (b) Geman-McClure 损失。^[1222] (©2020 IEEE)

图 3.8(a) 和 (b) 分别绘制了 GNC 截断二次损失和 GNC Geman-McClure 损失。可以看到, 增大或减小控制参数 μ 会为函数引入更多非凸性。

式 (3.24) 和 (3.25) 中平滑 GNC 函数的一个良好性质是, B-R 对偶性仍然适用。对 GNC 截断二次损失 (3.24), 应用 B-R 对偶性得到离群值过程

$$\Phi_{\rho_\mu}(w_i) = \frac{\mu(1-w_i)}{\mu+w_i}\beta_i^2.$$

对 GNC Geman-McClure 损失 (3.25), 应用 B-R 对偶性得到离群值过程

$$\Phi_{\rho_\mu}(w_i) = \mu\beta_i^2(\sqrt{w_i}-1)^2.$$

现在可以给出 GNC 算法。每次迭代执行以下三步:

1. **变量更新:** 使用当前权重 $w_i^{(t)}$ 求解加权最小二乘问题

$$\mathbf{x}^{(t)} \in \arg \min_{\mathbf{x}} \sum_i w_i^{(t)} r_i^2(\mathbf{x}). \quad (3.26)$$

2. **权重更新**: 使用 $\mathbf{x}^{(t)}$ 更新权重

$$w_i^{(t+1)} \in \arg \min_{w_i \in [0,1]} \Phi_{\rho_\mu}(w_i) + w_i r_i^2(\mathbf{x}^{(t)}), i = 1, \dots, N. \quad (3.27)$$

3. **控制参数更新**: 增大或减小 μ , 以向 ρ_μ 加入更多非凸性。

GNC 算法类似 IRLS, 区别在于它从凸的平滑函数 ρ_μ 开始, 然后迭代更新控制参数 μ , 逐渐向 ρ_μ 加入更多非凸性, 以逼近原始损失函数 ρ 。依据平滑损失 ρ_μ 的定义, 通过增大或减小 μ 可恢复原始 ρ 。例如, GNC 截断二次损失在 μ 较大时恢复原始截断二次损失, 因此每次 GNC 迭代将 μ 乘以常数因子 $\gamma > 1$ (例如 [1222] 中的 $\gamma = 1.4$)。反之, GNC Geman–McClure 损失在 μ 接近 1 时恢复原始 G–M 损失, 因此每次 GNC 迭代将 μ 除以 $\gamma > 1$ 。

图 3.9 展示在图 3.6 的相同三个数据集上应用 GNC 所得的 SLAM 轨迹, 采用两种不同的鲁棒损失: Geman–McClure 损失和截断二次损失。我们用 GTSAM 的 GNCOptimizer 实现 GNC。比较图 3.6 和图 3.7 可知, GNC 保证了更好的收敛性 (即较不易陷入局部极小值), 并在全部三个数据集中恢复出相当准确的轨迹。尽管 GNC 已表现出对离群值极强的耐受性 (例如, 在真实问题中可耐受约 80%–90% 的错误回环 [1222, 165]), 但该方法不提供收敛保证。此外, 实验表明其性能依赖具体问题; 它在位姿图优化问题中表现优异, 但在其他感知问题中性能会显著退化 [1005]。

下面展示一个 GNC 失败的问题, 并说明组合前端和后端离群值剔除可能是有益的。为此, 考虑一个比此前更具挑战性的 SLAM 设置。本章前面讨论位姿图优化问题 (例如图 3.1) 时, 假设里程计可靠, 而回环或路标测量中可能存在离群值。该设置本质上假定存在一条“里程计骨架” (odometry backbone): 它提供一组可信测量, 并可构建机器人位姿的初始猜测。虽然这一假设在许多问题中现实可行, ¹⁴ 但某些 SLAM 应用可能缺乏里程计骨架。例如, 由于轮式里程计中的车轮打滑或 LiDAR 里程计中的错误对齐, 某些里程计传感器可能产生错误测量。另一个例子是多机器人 SLAM: 每个机器人都有里程计骨架, 但包括全部机器人轨迹的整体 SLAM 问题却没有 [728]。

M3500

(a) SubT

Victoria Park

(b) (c)

(d)

(g)

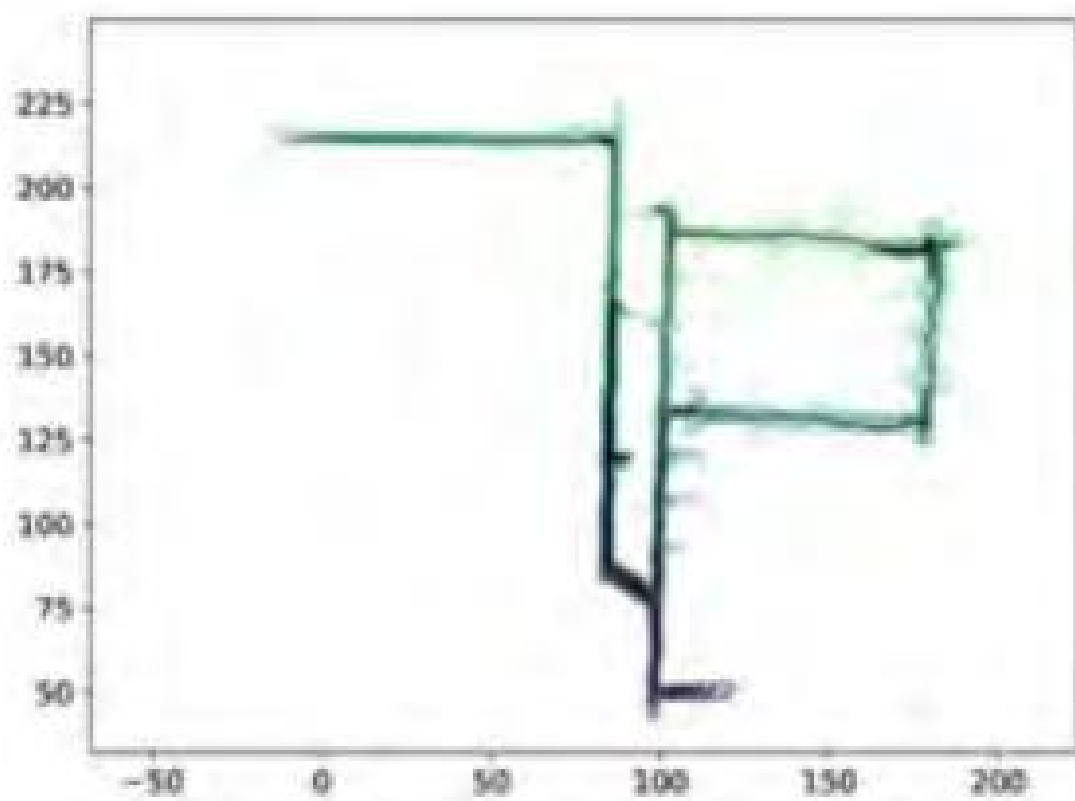
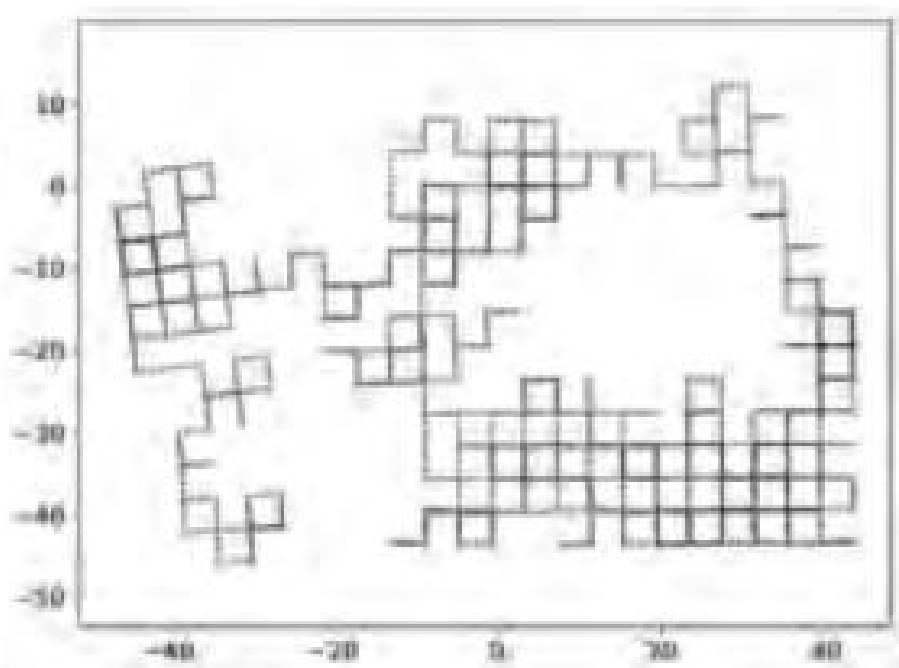
(f) (e)

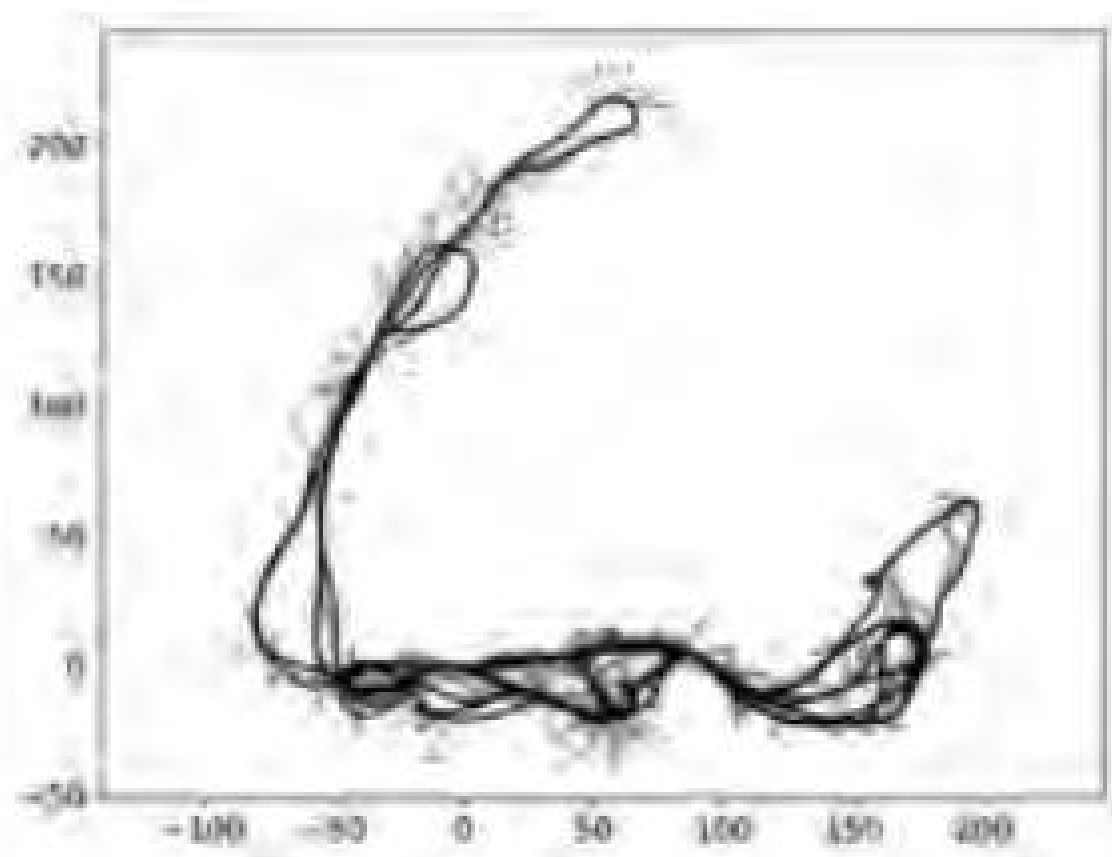
(h)

(i) 图 3.9 使用鲁棒损失函数和渐进非凸性 (GNC) 求解含离群值的 SLAM 问题: (a)–(c) 为 M3500、SubT 和 Victoria Park 数据集的真值轨迹; (d)–(f) 为使用 Geman–McClure 损失得到的轨迹估计; (g)–(i) 为使用截断二次损失得到的轨迹估计。测量以彩色边显示: 灰色边为正确分类的边; 红色边为被错误归类为内点的离群值 (假阳性); 蓝色边为被错误归类为离群值的内点 (假阴性)。

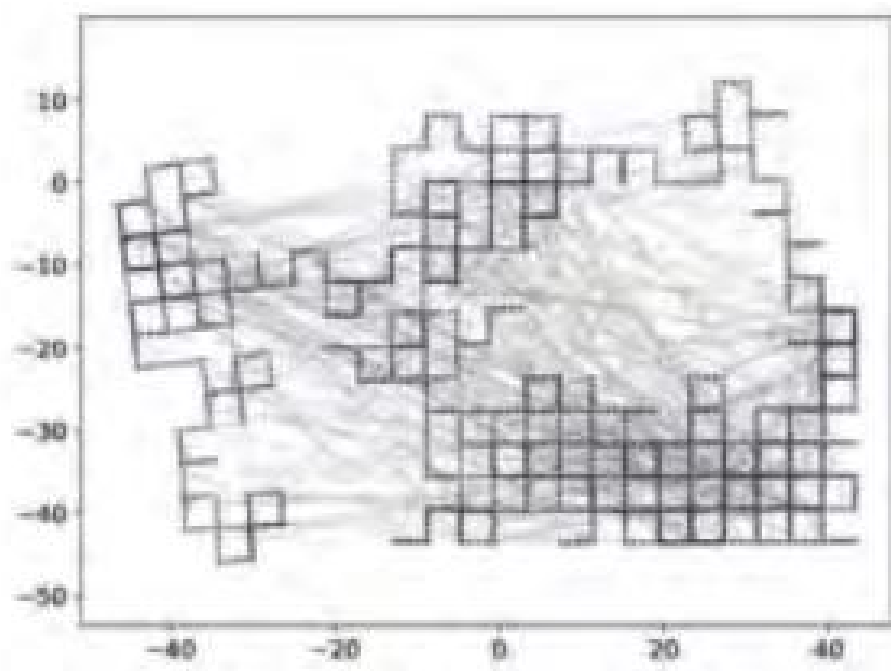
M3500

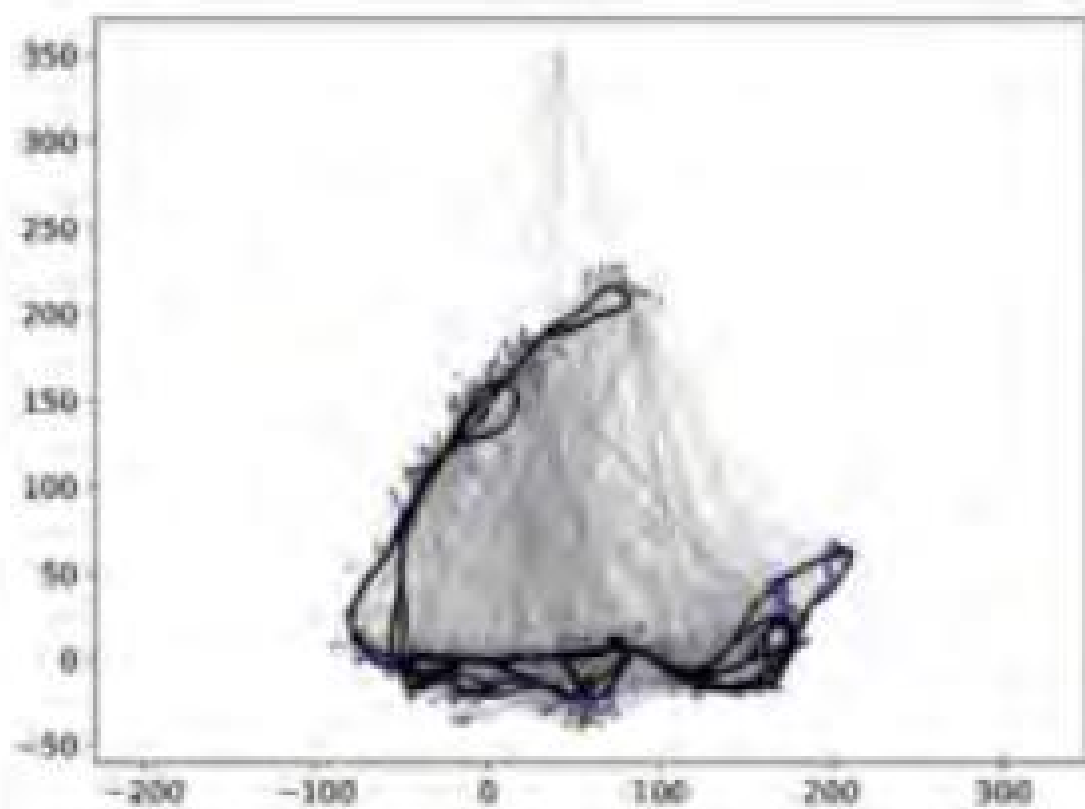
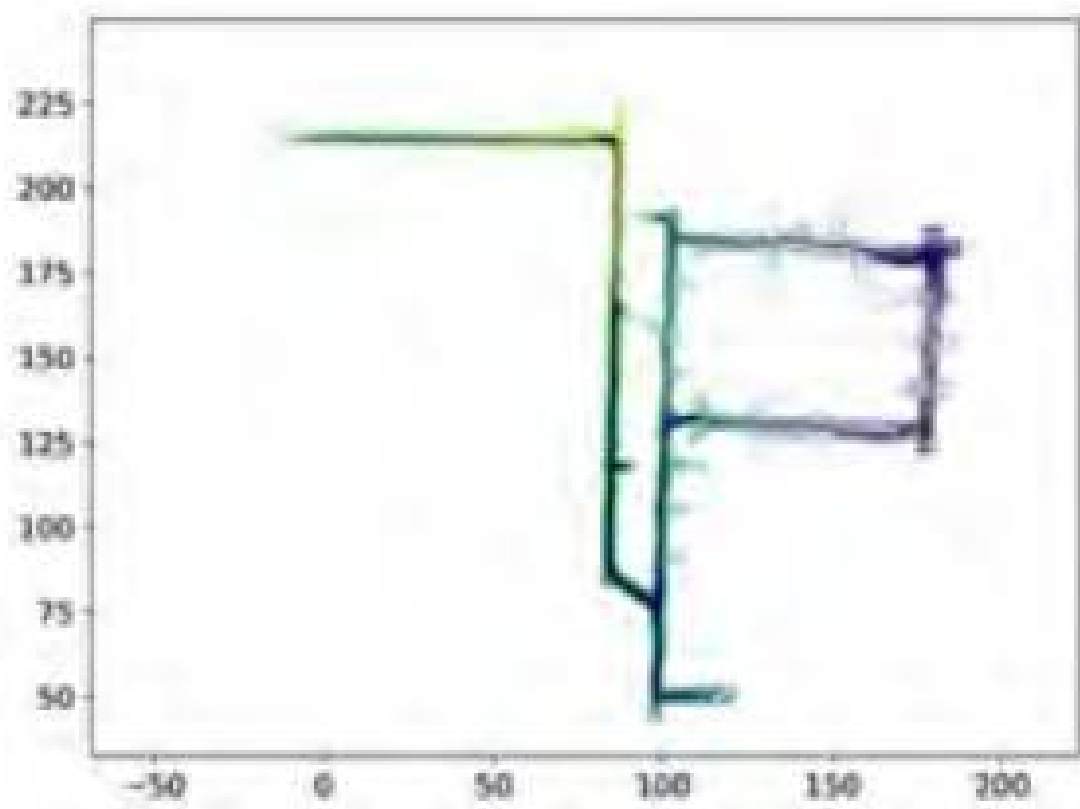
Ground Truth

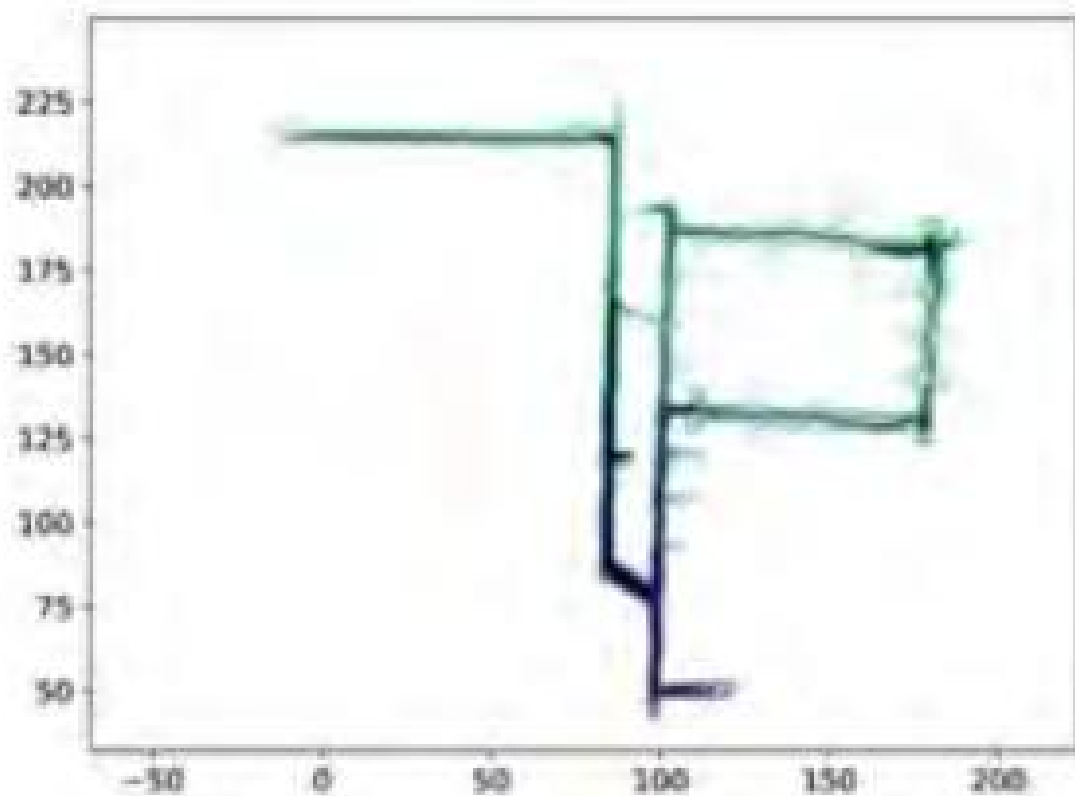
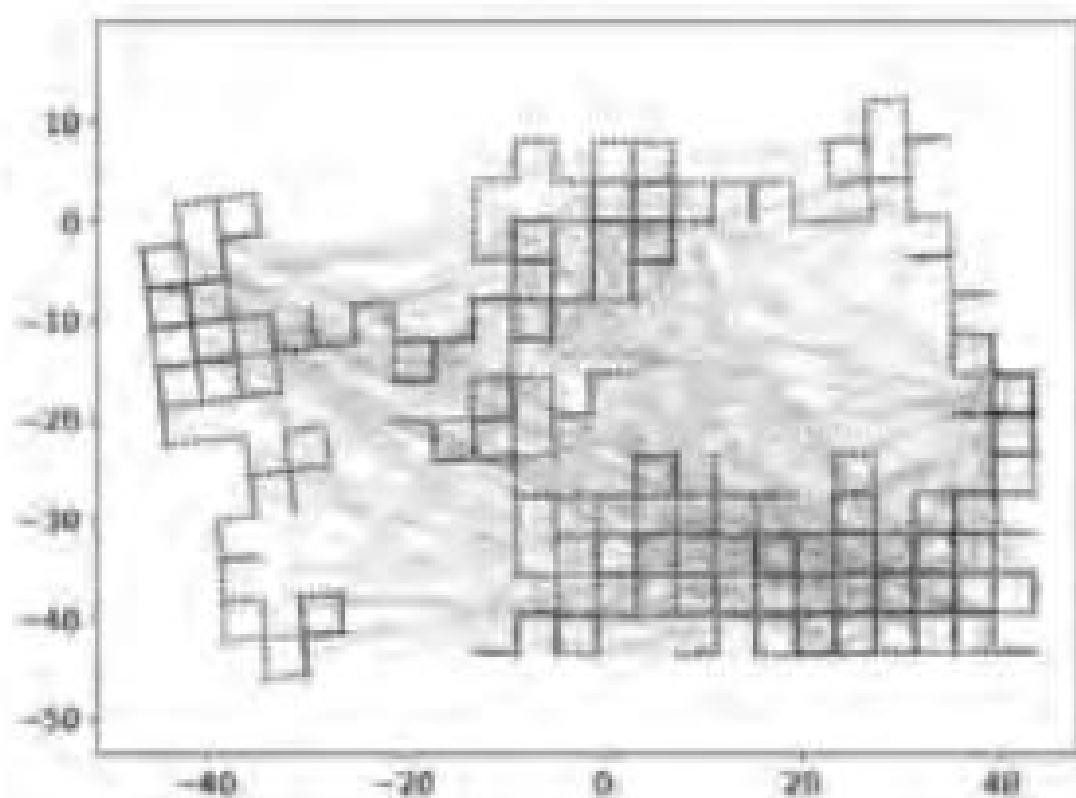


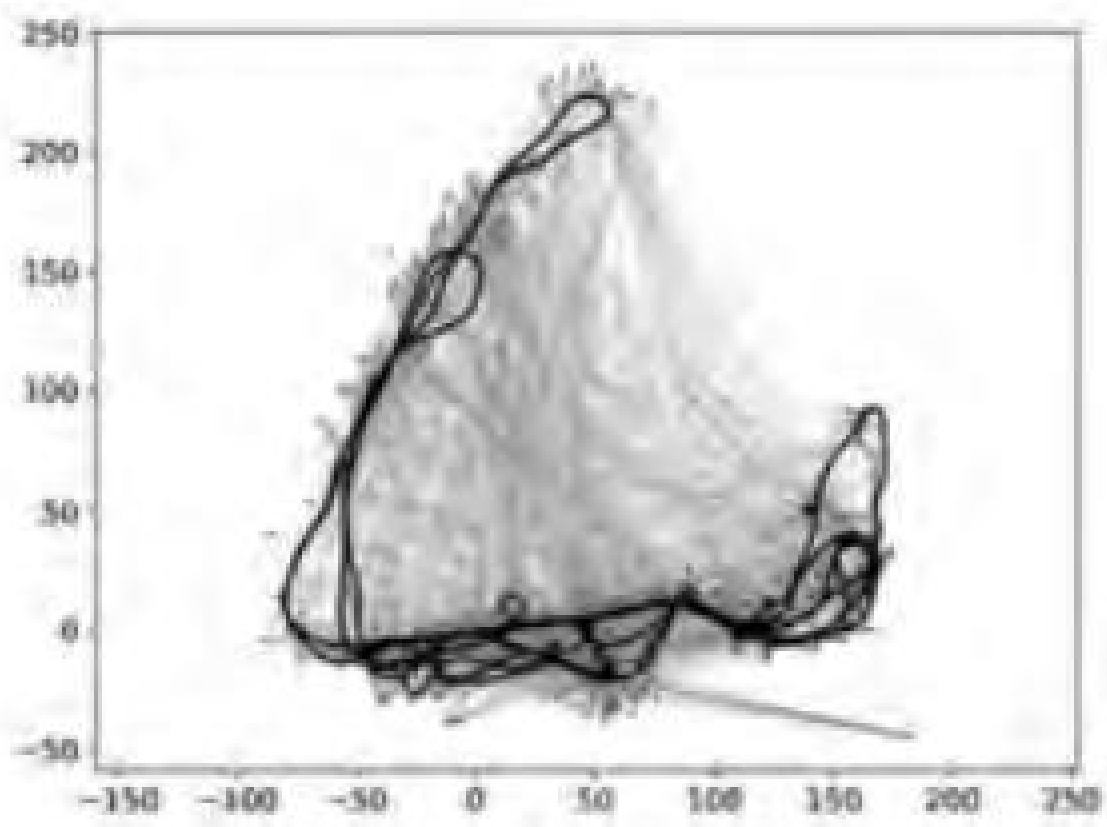


GNC + GM

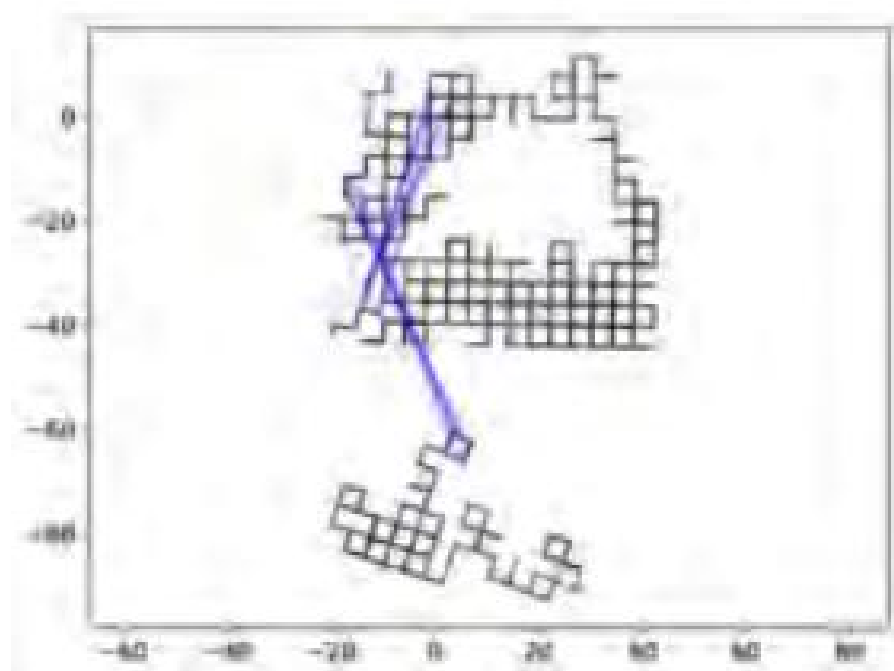




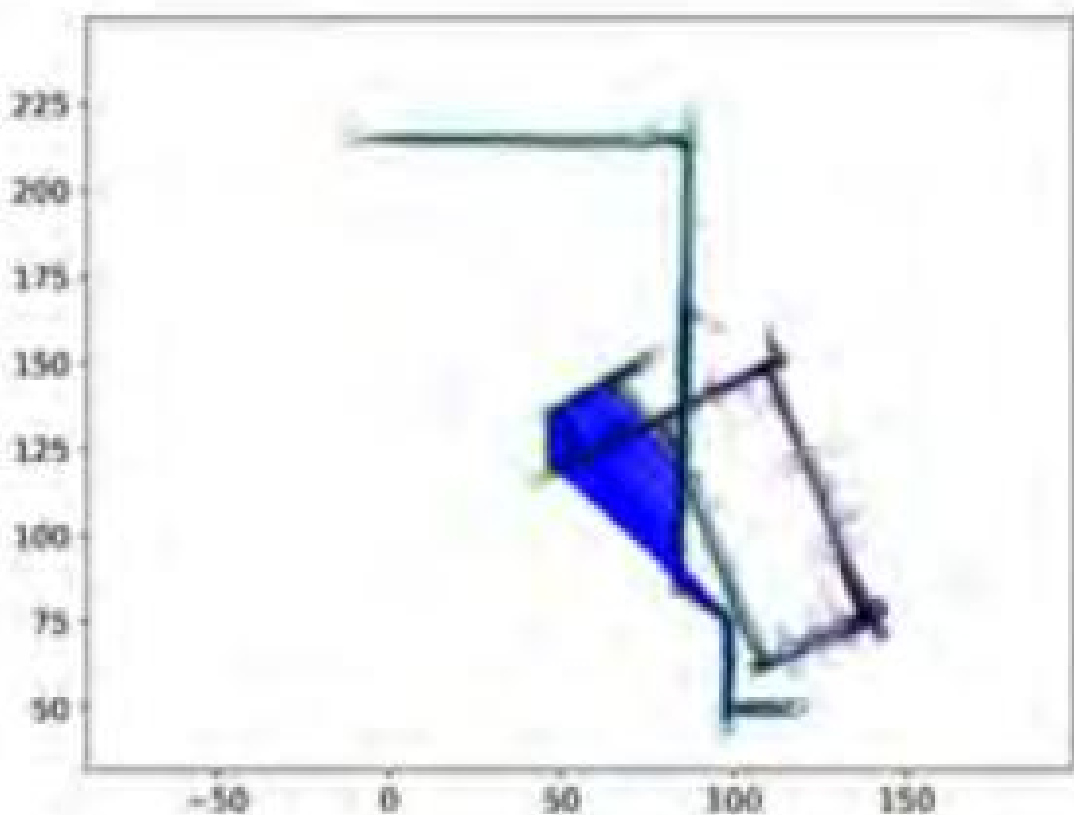




GNC



(a)
SubT

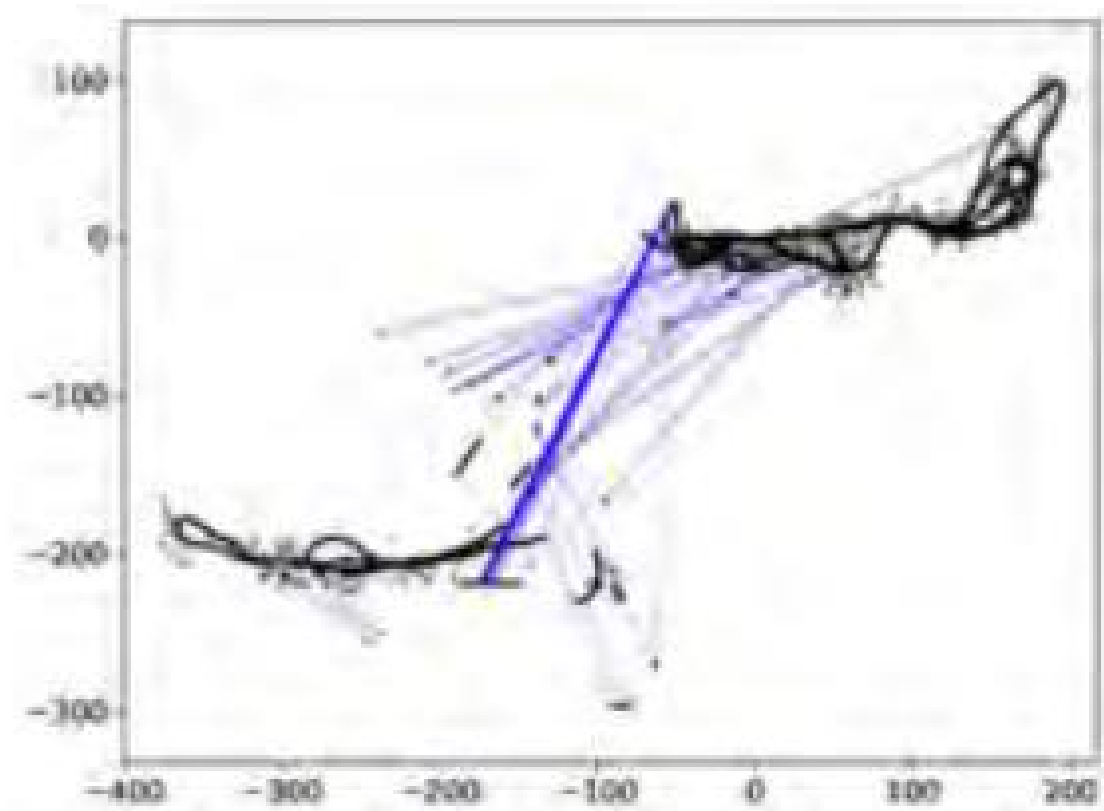


Victoria Park

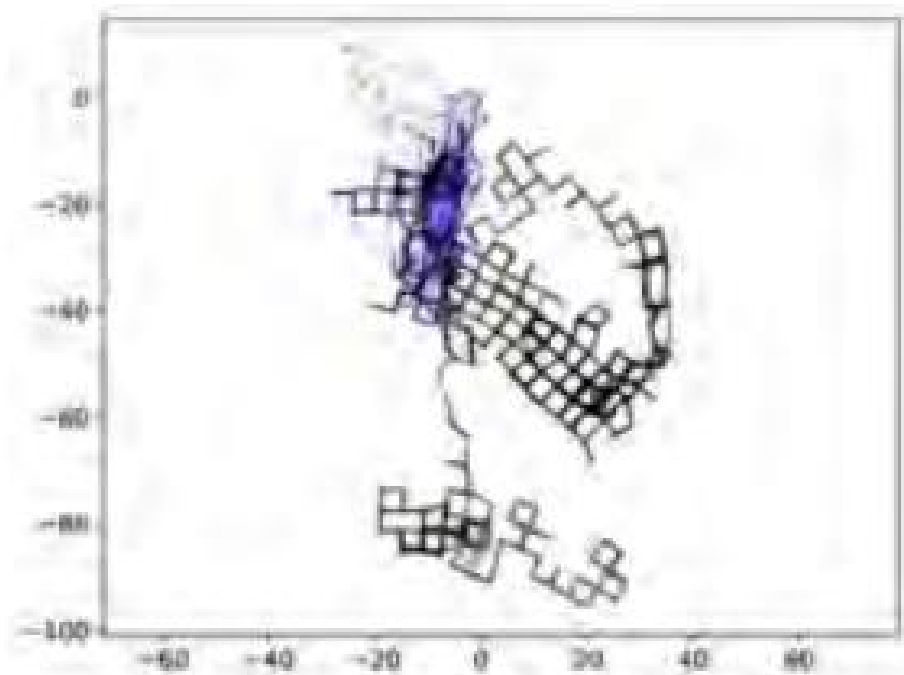
- (b)
- (c)
- (d)
- (g)
- (e)
- (h)
- (f)

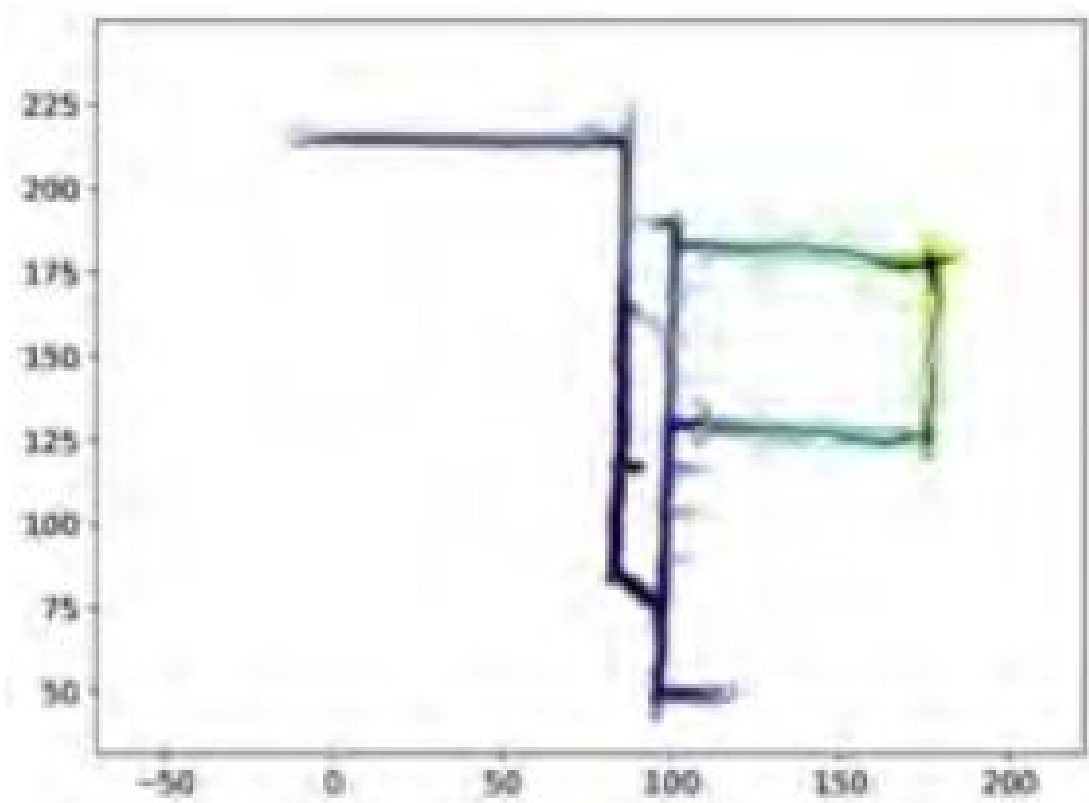
(i) 图 3.10 回环、路标测量和里程计中均含离群值的 SLAM 问题：(a)–(c) 为使用截断二次损失的 GNC 得到的轨迹估计；(d)–(f) 为使用 PCM 进行前端离群值剔除、随后采用最小二乘优化得到的轨迹估计；(g)–(i) 为先使用 PCM 进行前端离群值剔除、再使用带截断二次损失的 GNC 得到的轨迹估计。

在里程计与回环测量均可能为离群值时，鲁棒地求解 SLAM 问题极其困难。为展示所讨论方法在这个更复杂设置下的不足，我们先通过随机扰动两条里程计测量，修改图 3.1 中的位姿图问题，然后尝试用 GNC 求解。具体地，在 GNC 中，除了两条可能被扰动的测量之外，所有里程计测量均固定为内点。图 3.10 展示了 GNC 求解得到的轨迹。由于初始化受到扰动，GNC 失败并收敛到一个局部极小值，将所有测量（包括内点）都归类为离群值（蓝色边）。同



PCM





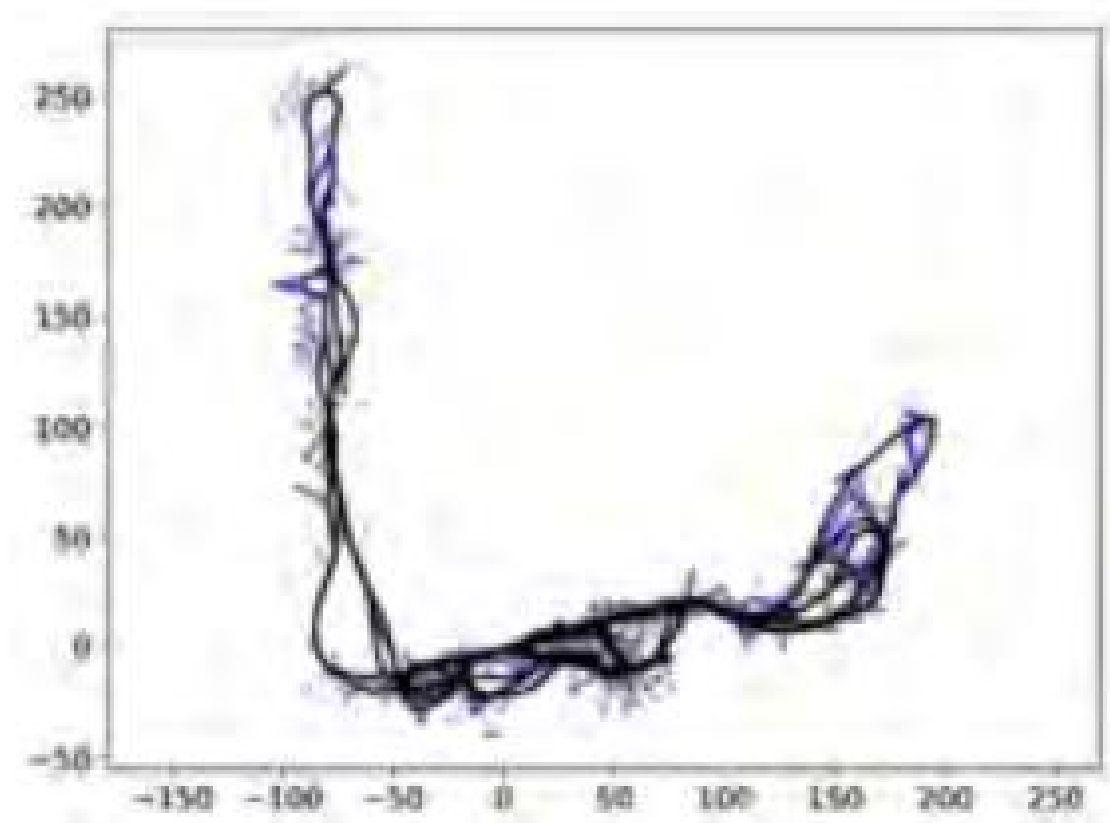
一图还展示了在采用最小二乘后端之前使用 PCM 过滤严重离群值的结果。PCM 与初始猜测无关，因此在 SubT 和 Victoria Park 数据集上能够收敛到更好的解（图 3.10(e) 和 (f)）。有趣的是，在 SubT 和 Victoria Park 数据集上，组合前端离群值剔除（PCM）与 GNC 获得了最佳结果。与此同时，三种方法（GNC、PCM 以及 PCM+GNC）在 M3500 数据集上均未能收敛到可接受的解，这证实了这一 SLAM 设置的困难性，以及现有 SLAM 算法在离群值剔除方面的局限。

3.12 延伸阅读与近期趋势

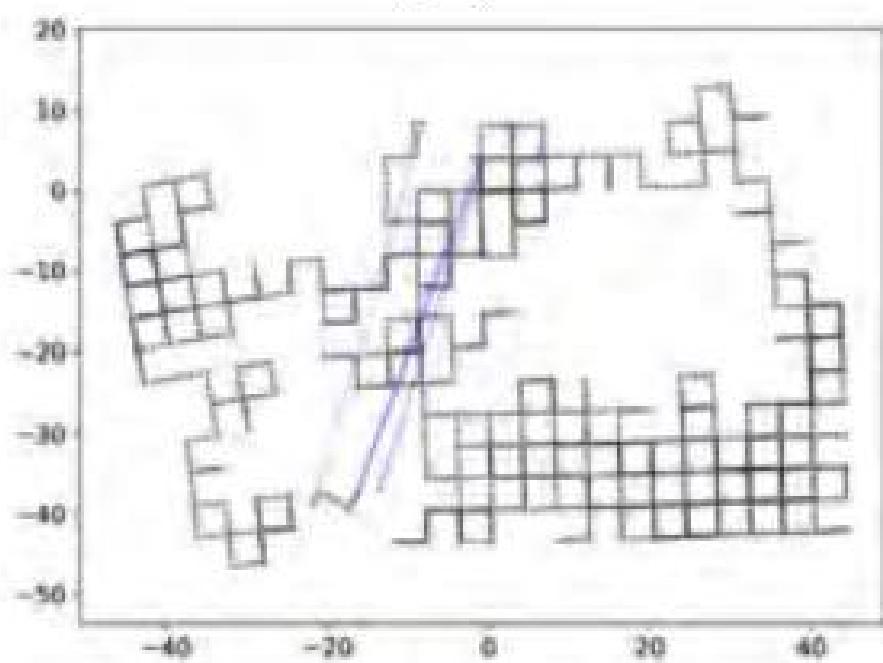
一致性最大化。尽管本章讨论的是 RANSAC 算法最基本的实例（依照 [328] 的最初提议），但值得一提的是，文献中给出了许多 RANSAC 变体，包括通过局部优化精化估计的变体 [216]、在 RANSAC 迭代中使用更好评分（而非一致集大小）的变体（例如 MLESAC [872]），以及在 RANSAC 迭代中对采样施加偏置的变体（例如 PROSAC [215]）。近期文献还包含可微 RANSAC 变体 [1174]，以及在式 (3.4) 中的参数 γ 未知时尝试寻找内点的变体（例如 [46]）。文献 [1109] 对 RANSAC 变体进行了近期综述与评估。

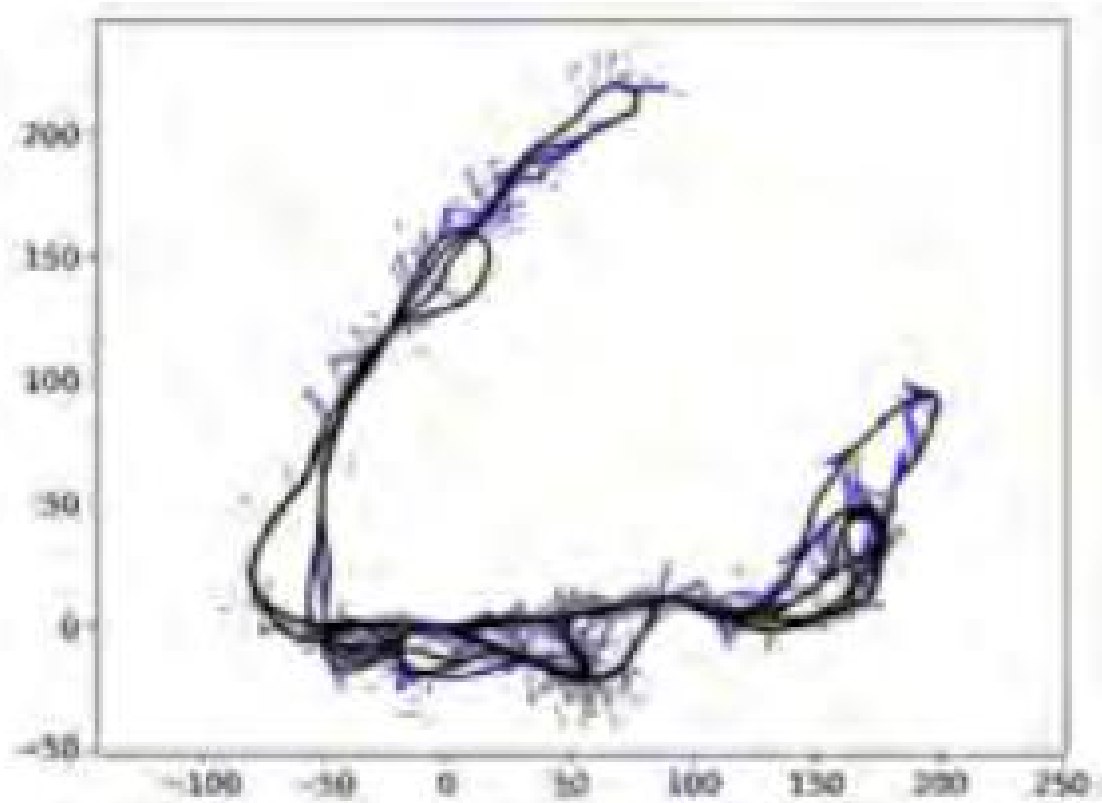
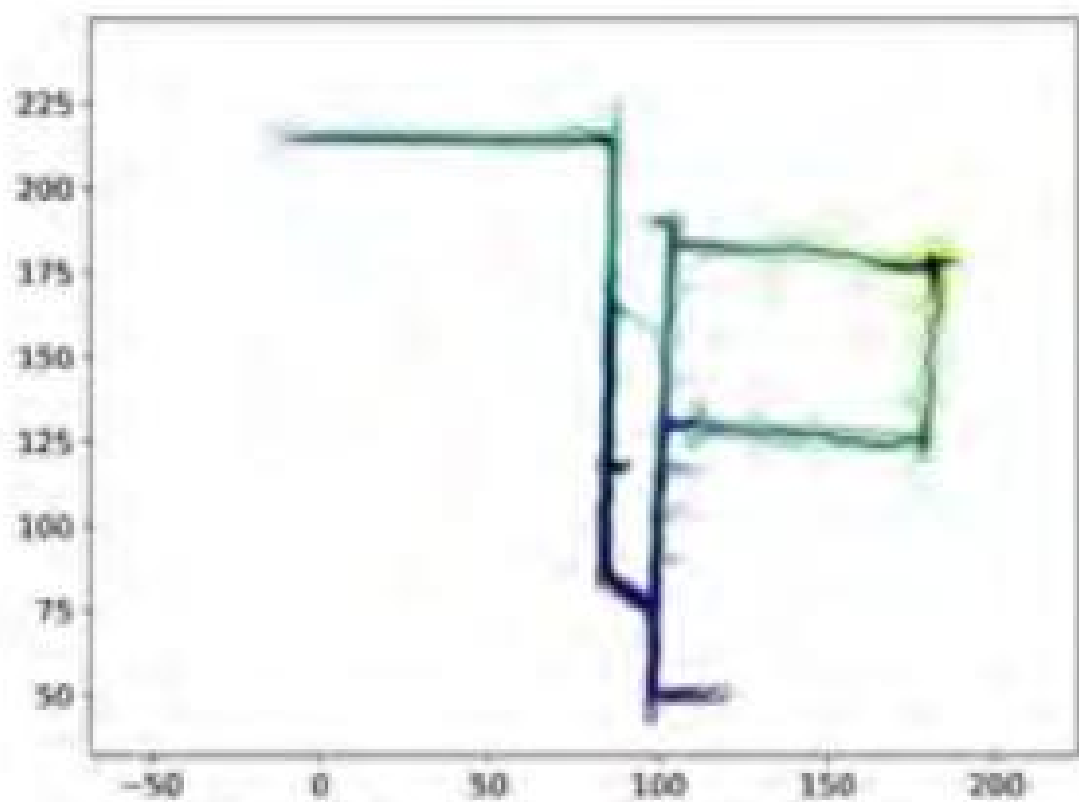
除 RANSAC 外，文献还包含精确一致性最大化方法，通常基于分支定界 [64, 436, 1290, 651, 1026, 203, 507, 132, 1226, 1]。尽管具有全局最优性保证，分支定界在最坏情形下具有指数运行时间，无法扩展到高维问题。

成对一致性最大化。第 3.2.2 节所述的 PCM 方法最初由 Mangelson 等人 [728] 在多机器人 SLAM 的语境下提出，并显示出特别的潜力。计算机视觉中也研究了基于图论的离群值剔



PCM + GNC





除。San Segundo 和 Artieda^[967] 构建关联图, 并为二维图像特征匹配寻找最大团。Perera 和 Barnes^[869] 用团形式化方法分割刚体运动下的对象。Leordeanu 和 Hebert^[639] 通过近似谱方法, 在对应图中寻找强连接簇来建立图像匹配。Enqvist 等人^[309] 开发了基于近似点覆盖的三维-三维和二维-三维配准离群值剔除算法。Yang 和 Carlone^[1217]、Yang 等人^[1223] 与 Bustos 等人^[133] 研究了基于最大团的三维-三维配准图论离群值剔除。检查一个测量子集内部一致性的思想也出现在 Latif 等人^[628] 中: 该工作通过聚类测量, 并用基于卡方的检验检查一致性, 从而剔除回环离群值。PCM 论文^[728] 与本章讨论一样聚焦于成对一致性。近期, PCM 已在^[1005, 1006, 332] 中推广到群组 k 一致性, 即一致性约束 (3.7) 涉及 k 个测量而非仅两个测量的情形。这些论文本质上将一致性图的概念推广为一致性超图, 其中每条超边涉及 k 个节点。相关工作还考虑最大团问题的软变体: 将二元条件 (3.7) 放宽, 以在一致性图的边上产生连续权重^[712, 711]。这些方法已应用于包括地下探索^[298]、LiDAR 点云定位^[713]、多机器人度量-语义建图^[1095] 和非结构化环境全局定位^[33] 在内的实际应用。

交替最小化与渐进非凸性。 M-估计一直是机器人学^[99] 和视觉^[978, 173] 中常用的鲁棒估计方法。Tavish 和 Barfoot^[1075] 研究了不同损失函数的性能。若干论文研究了式 (3.16) 那样带辅助变量的形式化, 却未认识到 Black-Rangarajan 对偶性 (定理 3.4) 所提供的与 M-估计的联系。例如, Sunderhauf 和 Protzel^[1059]、Sunderhauf 和 Protzel^[1058] 以及 Agarwal 等人^[11] 用负责禁用离群值的潜在二元变量扩展问题。Lee 等人^[637] 使用期望最大化。Olson 和 Agarwal^[827] 使用最大混合分布近似多模态测量噪声。近期, Barron^[57] 提出一个单参数函数, 概括 M-估计中一族鲁棒损失函数。Chebrolu 等人^[174] 设计了一个期望最大化算法, 以同时估计未知量 x 并选择合适的鲁棒损失函数 ρ 。渐进非凸性算法最初由^[86, 85] 为早期视觉应用中的离群值剔除提出; 近期该算法已用于点云配准^[1297, 1223]、SLAM^[1222] 和其他应用^[34]。近期, Peng 等人^[860] 提出了一种与 GNC 和 IRLS 相似的算法, 基于优化中的平滑主化 (smooth majorization) 思想, 适用于广泛的鲁棒损失。此外, Peng 等人^[860] 还为 GNC 推导了全局和局部收敛保证。

可认证算法。至此所述算法大致可分为两类: (i) 快速启发式方法 (例如 RANSAC 或 M-估计的局部求解器), 它们高效但性能保证很少; (ii) 全局求解器 (例如分支定界), 它们提供最优性保证但难以随问题规模扩展。近年来出现了一类新方法, 称为**可认证算法** (certifiable algorithms), 试图在可处理性与最优性之间取得平衡。可认证算法将非凸鲁棒估计问题松弛为凸半定规划 (semidefinite programs, SDP),¹⁵ 其解可在多项式时间内获得, 并提供可直接检验的事后全局最优性证书。鲁棒估计的可认证算法已在旋转估计^[1218]、三维-三维配准^[1223] 和位姿图优化^[618, 153] 的语境下提出。文献^[1219, 1221] 描述了一种为含离群值问题推导可认证算法的相当通用的方法, 而^[147] 讨论了其与统计学并行工作的联系。除少数显著例外外, 这些算法尽管在多项式时间内运行, 计算代价仍高, 且通常比启发式方法慢得多。在一些情形中, 这些算法背后的洞见可用于认证快速启发式方法获得的解的最优性^[1219], 从而兼得二者之长。

第四章 可微优化

作者：Chen Wang、Krishna Murthy Jatavallabhula、Mustafa Mukadam

如第 I 章所述，当代 SLAM 系统的设计通常遵循前端与后端架构。在这一架构中，前端通常负责预处理传感器数据，并生成机器人轨迹和环境地图的初始估计；后端则细化这些初始估计，以提升整体精度。机器学习的近期进展带来了基于神经网络的新方法，有望增强 SLAM 前端的部分功能。例如，基于深度学习的方法在特征检测与匹配^[971, 267, 1267]以及前端运动估计^[1167, 1082]中表现出令人瞩目的性能。这些方法从大量样例构成的数据集训练神经网络，而无须针对任务显式编程。与此同时，基于几何的方法仍然是 SLAM 后端不可或缺的组成部分，主要原因在于它们具有普适性，并能通过求解优化问题产生全局一致的估计^[1208]。

原则上，可以将基于学习的 SLAM 前端直接接入 SLAM 架构，再将相应输出送入后端；但学习技术的使用也为不再单向的信息交换打开了空间。具体而言，后端现在可以向前端提供反馈，使前端能够直接从后端估计中学习，从而让两个模块以更协调的方式共同降低估计误差。调和几何方法与深度学习、利用两者的互补优势，是大量近期 SLAM 工作的一条主线。特别是，一个新兴趋势是对 SLAM 后端中出现的基于几何的优化问题进行微分。直观而言，对一个优化问题进行微分，可以理解该问题的最优解（例如 SLAM 估计）如何依赖于该问题的参数；在这里，这些参数就是学习型前端产生的测量。由此可以优化前端以最大化 SLAM 精度。也可以将其视为一个双层优化问题：上层优化过程受制于下层优化；具体来说，用于训练前端的基于神经网络的优化受制于根据给定前端输出计算 SLAM 解的基于几何的优化。

能够端到端地穿过优化过程计算梯度，是求解双层优化问题的核心；这使神经模型能够利用优化所编码的几何先验。这一方案的灵活性已在诸多应用中带来很有前景的先进结果，例如运动恢复（structure from motion）^[1080]、运动规划^[81, 1216]、SLAM^[511, 1082]、束调整（BA）^[1068, 1267]、状态估计^[1244, 182]和图像配准^[716]。

本章介绍如何对 SLAM 中出现的非线性最小二乘问题进行微分的基础知识。具体地，第 4.1 节重述非线性最小二乘（nonlinear least squares, NLS）问题；第 4.2 节说明如何穿过 NLS 问题进行微分；第 4.3 节讨论定义在流形上的问题如何微分；第 4.4 节讨论上述微分的数值挑战，并介绍相关机器学习库；最后，第 4.5 节给出当代 SLAM 系统中可微优化的示例。

4.1 非线性最小二乘回顾

非线性最小二乘（NLS）通过最小化观测值与模型预测值之差的平方和来估计模型参数。与线性最小二乘不同，NLS 中的模型关于参数是非线性的。除第 1 章中的因子图外，这一方法也广泛用于统计学、物理学和工程学等领域；当变量之间的关系并不直接时，它有助于将复杂模型拟合到数据，从而实现更准确且更鲁棒的预测。

具体地，NLS 通过求解下式来寻找变量 $\mathbf{x} \in \mathbb{R}^n$ ：

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}) = \arg \min_{\mathbf{x}} \frac{1}{2} \sum_i \underbrace{\|w_i \mathbf{c}_i(\mathbf{x}_i)\|^2}_{\mathbf{r}_i(\mathbf{x}_i)}, \quad (4.1)$$

其中，目标函数 $\mathcal{L}(\mathbf{x})$ 是向量值残差项 $\mathbf{r}_i$ 的平方和；每个残差项都是优化变量 $\mathbf{x} = \{\mathbf{x}_i\}$ 的（可能相互重叠的）子集 $\mathbf{x}_i \subset \mathbf{x}$ 的函数。当前假定 $\mathbf{x}_i$ 为向量；在本章后续内容中，将讨论推广至变量属于流形的情形。为保持灵活性，这里将残差 $\mathbf{r}_i(\mathbf{x}_i) = w_i \mathbf{c}_i(\mathbf{x}_i)$ 表示为权重 w_i 与向量代价 $\mathbf{c}_i$ 的乘积。

如第 1 章所述，NLS 通常通过迭代方式求解：先在当前变量附近线性化非线性目标，得到线性系统 $(\sum_i \mathbf{J}_i^\top \mathbf{J}_i) \delta \mathbf{x} = -\sum_i \mathbf{J}_i^\top \mathbf{r}_i$ ；再求解该线性系统以获得更新量 $\delta \mathbf{x}$ ；最后按 $\mathbf{x} \leftarrow \mathbf{x} + \delta \mathbf{x}$ 更新变量，直至收敛。第 2 章还指出，对于属于流形的变量，更新步骤中的加法更一般地是一个回缩映射（retraction mapping）。在线性系统中，

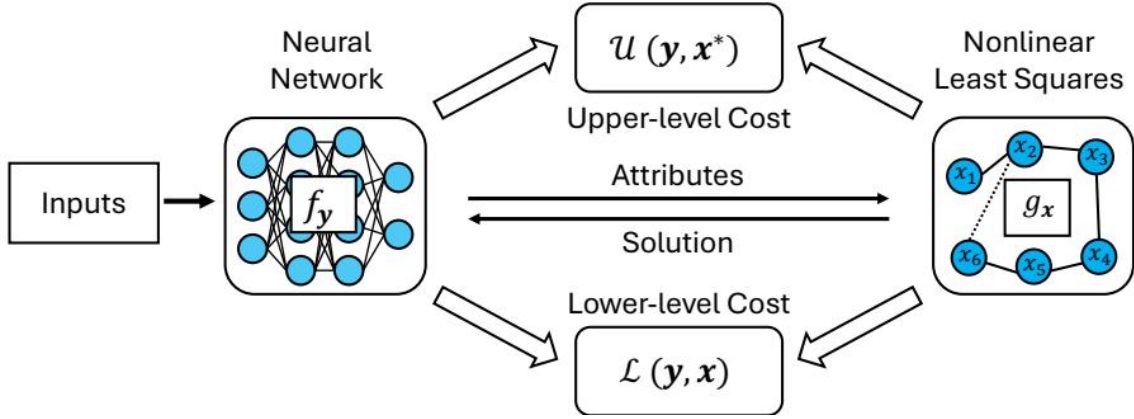


图 4.1: 图 4.1 现代 SLAM 系统通常同时包含神经网络和非线性最小二乘。为消除分别优化两个模块所引入的复合误差，可将整个系统表述为包含上层代价和下层代价的双层优化，从而进行端到端优化。

$\mathbf{J}_i = [\partial \mathbf{r}_i / \partial \mathbf{x}]$ 是残差关于变量的雅可比矩阵。上述迭代方法称为 Gauss-Newton (GN)，是一种（近似）二阶的非线性优化器，因为 $\sum_i \mathbf{J}_i^\top \mathbf{J}_i$ 是 Hessian 矩阵的近似。为改善鲁棒性和收敛性，Levenberg-Marquardt (LM) 等变体会对线性系统施加阻尼；另一些方法则利用线搜索调整更新步长，例如第 1 章介绍的 Dogleg 方法。

4.2 穿过非线性最小二乘进行微分

为将深度学习与非线性最小二乘无缝结合，通常需要可微非线性最小二乘（differentiable nonlinear least squares, DNLS）来求解图 4.1 所示的优化问题。这要求计算解 $\mathbf{x}^*$ 关于上层神经模型任意参数 $\mathbf{y}$ 的梯度；这些参数决定目标函数 $\mathcal{U}(\mathbf{x}; \mathbf{y})$ ，进而决定任意代价 $c_i(\mathbf{x}_i; \mathbf{y})$ 或变量初始化 $\mathbf{x}_{\text{init}}(\mathbf{y})$ 。目标是利用以 $\mathbf{x}$ 为函数定义的下层学习目标 $\mathcal{L}$ ，端到端地学习参数 $\mathbf{y}$ 。这构成一个双层优化（Bilevel Optimization, BLO）问题，可写为：

$$\mathbf{y}^* = \arg \min_{\mathbf{y} \in \Theta} \mathcal{U}(\mathbf{y}, \mathbf{x}^*), \quad (4.2a)$$

$$\text{s. t. } \mathbf{x}^* = \arg \min_{\mathbf{x} \in \Psi} \mathcal{L}(\mathbf{y}, \mathbf{x}), \quad (4.2b)$$

其中, $\mathcal{L} : \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}$ 是下层 (LL) 代价, $\mathcal{U} : \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}$ 是上层 (UL) 代价, $\mathbf{x} \in \Psi$ 和 $\mathbf{y} \in \Theta$ 为可行集。

在实践中, 变量 x 通常是具有明确物理含义的参数, 例如相机位姿; 而 y 通常是没有明确物理含义的参数, 例如神经网络的权重。下面用两个例子说明。

例 4.1 (具有学习特征的视觉 SLAM) 设想一个 SLAM 系统使用由 y 参数化的神经网络进行特征提取/匹配, 同时利用由 x 参数化的 BA 进行位姿估计, 并将特征匹配结果作为输入。在该示例中, 上层代价 (4.2a) 可以是用于优化网络的特征匹配误差, 而下层代价 $\mathcal{L}$ (4.2b) 可以是 BA 的重投影误差。直观地说, 相机位姿和路标位置的最优解 $\mathbf{x}^*$ 在神经网络训练中起监督信号的作用。因此, 优化 BLO (4.2) 可通过反向传播 BA 重投影误差, 进一步降低匹配误差 [1267]。

例 4.2 (PGO) 设想一个 SLAM 系统使用神经网络进行前端位姿估计, 并使用 PGO 作为后端消除里程计漂移。在此示例中, 上层和下层代价都可以是位姿图误差。区别在于, 上层代价优化由 y 参数化的网络, 而下层代价优化由 x 参数化的相机位姿。因此, 前端网络可通过从后端 PGO 反向传播位姿残差, 利用位姿图优化所获得的全局几何知识 [346]。

BLO 是一个历史悠久且研究充分的问题 [1148, 517, 679]。求解 BLO 通常依赖梯度下降技术。具体而言, 上层优化按 $\mathbf{y} \leftarrow \mathbf{y} + \delta \mathbf{y}$ 的形式更新, 其中 $\delta \mathbf{y}$ 是沿负梯度方向的一步。因此, 需要计算目标函数关于上层变量 $\mathbf{y}$ 的梯度, 可写为

$$\nabla_{\mathbf{y}} \mathcal{U} = \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}^*)}{\partial \mathbf{y}} + \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}^*)}{\partial \mathbf{x}^*} \frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}, \quad (4.3)$$

其中, 项 $\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}$ 涉及间接梯度计算。由于式 (4.3) 中其他直接梯度项易于获得, 求解 BLO (4.2) 的难点在于计算 $\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}$ 。为此, 人们从显式和隐式两个角度发展出一系列技术, 分别涉及穿过动力系统的递归微分与隐式微分理论, 通常称为展开微分 (unrolled differentiation) 和隐式微分 (implicit differentiation)。这些算法在 [679, 1148] 中已有总结; 这里的算法 1 给出包含两类方法的通用框架。下面分别介绍展开微分和隐式微分。

4.3 展开微分

展开微分需要穿过下层优化过程执行自动微分 (Automatic Differentiation, AutoDif), 以求解 BLO 问题。具体地, 给定 $t = 0$ 时的初始化 $\mathbf{x}_0 = \Phi_0(\mathbf{y})$, 展开的下层优化迭代过程为

$$\mathbf{x}_t = \Phi_t(\mathbf{x}_{t-1}; \mathbf{y}), \quad t = 1, \dots, T, \quad (4.4)$$

算法 1 通过展开微分或隐式微分求解 BLO。

- 1: 初始化: y_0, x_0 。
- 2: 当未收敛 ($\|y_{k+1} - y_k\|$ 足够大) 时:
- 3: 使用通用优化器 O 经 T 步求解 (4.2b), 获得 x_T 。
- 4: 按下列方式高效估计式 (4.3) 的上层梯度:
- 5: 展开微分: 通过式 (4.7) 中的 AutoDif, 计算 $\hat{\nabla}_{y_k} U = \frac{\partial U(y_k, x_T)}{\partial y_k}$ 。
- 6: 隐式微分 (算法 2): 计算

$$\hat{\nabla}_{y_k} \mathcal{U} = \left. \frac{\partial \mathcal{U}}{\partial y_k} \right|_{x_T} + \left. \frac{\partial \mathcal{U}}{\partial x^*} \frac{\partial x^*}{\partial y_k} \right|_{x_T},$$

其中隐式导数 $\frac{\partial x^*}{\partial y_k}$ 可通过求解由下层最优性条件导出的方程获得（后续各节将作概述）。

7:

8: 使用 $\hat{\nabla}_{y_k} U$ ，通过梯度计算 y_{k+1} 。

9: 结束循环

其中， Φ_t 表示第 t 步中基于下层问题的更新方案， T 为迭代次数。一种更新方案是梯度下降：

$$\Phi_t(\mathbf{x}_{t-1}; \mathbf{y}) = \mathbf{x}_{t-1} - \eta_t \cdot \frac{\partial \mathcal{L}(\mathbf{x}_{t-1}, \mathbf{y})}{\partial \mathbf{x}_{t-1}}, \quad (4.5)$$

其中， η_t 是学习率，项 $\frac{\partial \mathcal{L}(\mathbf{x}_{t-1}, \mathbf{y})}{\partial \mathbf{x}_{t-1}}$ 可通过 AutoDif 计算。¹ 因此，可用 $\mathbf{x}_T$ 近似替代 $\mathbf{x}^*$ ，进而计算 $\nabla_{\mathbf{y}} \mathcal{U}(\mathbf{y})$ ；完整的展开系统可定义为

$$\mathbf{x}^* \approx \mathbf{x}_T = \Phi(\mathbf{y}) = (\Phi_T \circ \cdots \circ \Phi_1 \circ \Phi_0)(\mathbf{y}), \quad (4.6)$$

其中符号 $\circ$ 表示函数复合。因此，只需考虑下列问题，而不必考虑式 (4.2) 中的双层优化：

$$\min_{\mathbf{y} \in \Theta} \mathcal{U}(\mathbf{y}, \Phi(\mathbf{y})), \quad (4.7)$$

该问题需要经由 AutoDif 计算 $\frac{\partial \Phi(\mathbf{y})}{\partial \mathbf{y}}$ ，而不是计算式 (4.3)。值得注意的是，计算递归梯度有两种途径：一种对应反向模式反向传播^[857]，另一种对应前向模式^[928]。这里省略 AutoDif 两种途径的细节，读者可参阅用于深度学习的 AutoDif 库（如 PyTorch^[850]），以及用于 SLAM 的 PyPose^[1145] 和 Theseus^[873]。Liu 等人^[679] 也对这些方法作了综述。

4.4 截断展开微分

反向模式和前向模式是精确的递归梯度计算方法，但完整迭代传播会耗费大量时间。这是因为上层问题对 $\mathbf{x}_t$ （其中 $t = 0, 1, \dots, T$ ）具有复杂的长期依赖；当 $\mathbf{y}$ 和 $\mathbf{x}$ 都是高维向量时，困难会进一步加剧。为克服这一挑战，研究者提出截断展开微分，以显著更少的计算时间和内存得到高质量的近似梯度。具体地，通过忽略长期依赖，并以部分历史近似式 (4.5) 的梯度，即仅保存最后 M 次迭代 ($t = T, T-1, \dots, T-M$)，可以显著降低时间和空间复杂度。Shaban 等人^[992] 已证明，用更少的反向步计算梯度，能够取得与精确梯度优化相当的性能，同时所需内存和计算量小得多。

在计算和内存约束更严格的情形下，截断展开微分仍常是现代机器人应用中的瓶颈。因此，研究者也尝试进一步简化截断微分：在式 (4.4) 中仅执行一次迭代，以移除递归结构^[676]，即

$$\nabla_{\mathbf{y}} \mathcal{U} = \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}_1(\mathbf{y}))}{\partial \mathbf{y}} + \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}_1(\mathbf{y}))}{\partial \mathbf{x}_1} \frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}}, \quad (4.8)$$

其中，项 $\frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}}$ 是由混合二阶导数给出的雅可比矩阵，可由式 (4.5) 计算为

$$\frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}} = -\frac{\partial^2 \mathcal{L}(\mathbf{x}_0, \mathbf{y})}{\partial \mathbf{x}_0 \partial \mathbf{y}}. \quad (4.9)$$

由于在某些应用中计算 Hessian 矩阵代价很高，可以采用数值解法：对变量 x 施加微小扰动，并将式 (4.8) 的第二项整体近似为

$$\frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}_1(\mathbf{y}))}{\partial \mathbf{x}_1} \frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}} \approx \frac{\frac{\partial \mathcal{L}(\mathbf{x}_0^+, \mathbf{y})}{\partial \mathbf{y}} - \frac{\partial \mathcal{L}(\mathbf{x}_0^-, \mathbf{y})}{\partial \mathbf{y}}}{2\epsilon}, \quad (4.10)$$

其中， ϵ 是一个小标量， $\mathbf{x}_0^\pm = \mathbf{x}_0 \pm \epsilon \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}_1(\mathbf{y}))}{\partial \mathbf{x}_1}$ 是微小扰动。该方法避免显式计算雅可比矩阵 $\frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}}$ 。尽管如此，若包含非欧氏变量，例如属于 Lie 群的变量，就需关注扰动模型。幸运的是，现代库已支持针对 Hessian-向量积和 Jacobian-向量积的 Lie 群 AutoDif，例如将在第 4.4 节介绍的 PyPose^[1145]。

4.5 隐式微分

直观地说，式 (4.3) 中的项 $\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}$ 依赖于下层代价 (4.2b)，因此可使用隐式微分推导该梯度的解。

在微积分中，隐式微分是利用链式法则对隐函数求导的方法。对于由方程 $R(x, y) = 0$ 定义的隐函数 $y(x)$ ，通常无法先显式解出 y 再进行微分；取而代之的是，对 $R(x, y) = 0$ 关于 x 作全微分，再解所得线性方程，得到以 x 和 y 表示的导数 $\frac{dy}{dx}$ 。例如，考虑隐函数 $x + y + 5 = 0$ ，对等式两边关于 x 求导，可得 $\frac{dy}{dx} + \frac{dx}{dx} + \frac{d}{dx}(5) = 0 \Rightarrow \frac{dy}{dx} + 1 + 0 = 0$ 。解出 $\frac{dy}{dx}$ ，得到 $\frac{dy}{dx} = -1$ 。

假定下层代价关于 $\mathbf{x}$ 和 $\mathbf{y}$ 至少二次可微，则由最优性条件可知， $\mathbf{x}^*$ 为驻点，从而有 $\frac{\partial \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y})} = 0$ 。将该方程两边关于 $\mathbf{y}$ 求导，得到

$$\frac{\partial^2 \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y}) \partial \mathbf{y}} + \frac{\partial^2 \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y}) \partial \mathbf{x}^*(\mathbf{y})} \cdot \frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}} = 0. \quad (4.11)$$

这给出间接梯度 $\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}$ ：

$$\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}} = -\left(\frac{\partial^2 \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y}) \partial \mathbf{x}^*(\mathbf{y})}\right)^{-1} \frac{\partial^2 \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y}) \partial \mathbf{y}}, \quad (4.12)$$

式 (4.12) 的优点是，以求逆一个 Hessian 矩阵为代价，将变量 y 和 x 之间的间接梯度转化为 $\mathcal{L}$ 的直接梯度。其缺点是 Hessian 矩阵通常过大，难以计算；因此，常见做法是利用高效的 Hessian-向量积求解一个线性系统。

例 4.3 (存储和求逆 Hessian 矩阵的代价) 假定上层和下层代价均涉及一个仅有 10^6 万 (10^6) 个参数 (32 位浮点数) 的网络，则每个网络只需 $10^6 \times 4\text{Byte} = 4\text{MB}$ 的存储空间；而其 Hessian 矩阵需要 $(10^6)^2 \times 4\text{Byte} = 4\text{TByte}$ 的存储空间。这表明，低功耗计算机的内存甚至无法显式存储 Hessian 矩阵，因此直接计算其逆不可行。

回顾目标是计算式 (4.3) 的梯度，将式 (4.12) 代入式 (4.3) 可得：

$$\begin{aligned}\nabla_{\mathbf{y}}\mathcal{U} &= \frac{\partial\mathcal{U}(\mathbf{y},\mathbf{x}^*)}{\partial\mathbf{y}} - \underbrace{\frac{\partial\mathcal{U}(\mathbf{y},\mathbf{x}^*)}{\partial\mathbf{x}^*}}_{\mathbf{v}^\top} \underbrace{\left(\frac{\partial^2\mathcal{L}(\mathbf{x}^*(\mathbf{y}),\mathbf{y})}{\partial\mathbf{x}^*(\mathbf{y})\partial\mathbf{x}^*(\mathbf{y})}\right)^{-1}}_{(\mathbf{H}^\top)^{-1}} \frac{\partial^2\mathcal{L}(\mathbf{x}^*(\mathbf{y}),\mathbf{y})}{\partial\mathbf{x}^*(\mathbf{y})\partial\mathbf{y}} \\ &= \frac{\partial\mathcal{U}(\mathbf{y},\mathbf{x}^*)}{\partial\mathbf{y}} - \mathbf{q}^\top \cdot \frac{\partial^2\mathcal{L}(\mathbf{x}^*(\mathbf{y}),\mathbf{y})}{\partial\mathbf{x}^*(\mathbf{y})\partial\mathbf{y}}\end{aligned}\quad (4.13)$$

随后可通过优化下式，求解线性系统 $\mathbf{H}\mathbf{q} = \mathbf{v}$ 得到 $\mathbf{q}^*$ ：

$$\mathbf{q}^* = \arg\min_{\mathbf{q}} Q(\mathbf{q}) = \arg\min_{\mathbf{q}} \frac{1}{2}\mathbf{q}^\top \mathbf{H}\mathbf{q} - \mathbf{q}^\top \mathbf{v}, \quad (4.14)$$

可使用简单的梯度下降或共轭梯度法^[459]等高效线性求解器。对于梯度下降，需要计算 Q 的梯度 $\frac{\partial Q(\mathbf{q})}{\partial\mathbf{q}} = \mathbf{H}\mathbf{q} - \mathbf{v}$ ，其中 $\mathbf{H}\mathbf{q}$ 可通过高效 Hessian-向量积计算；即 Hessian-向量积是梯度-向量积的梯度：

$$\mathbf{H}\mathbf{q} = \frac{\partial^2\mathcal{L}}{\partial\mathbf{x}\partial\mathbf{x}} \cdot \mathbf{q} = \frac{\partial(\frac{\partial\mathcal{L}}{\partial\mathbf{x}} \cdot \mathbf{q})}{\partial\mathbf{x}}, \quad (4.15)$$

其中， $\frac{\partial\mathcal{L}}{\partial\mathbf{x}} \cdot \mathbf{q}$ 是一个标量。这意味着无需显式计算或存储 Hessian 矩阵 H 。算法 2 总结了通过线性系统计算隐式微分的过程；使用共轭梯度法的算法与之类似。

算法 2 通过线性系统计算隐式微分。

1: 输入：当前上层变量 $\mathbf{y}$ 和最优下层变量 $\mathbf{x}^*$ 。

2: 初始化： $k = 1$ ，学习率 η 。

3: 当未收敛 ($\|\mathbf{q}_k - \mathbf{q}_{k-1}\|$ 足够大) 时：

4: 执行梯度下降：

$\mathbf{q}_k = \mathbf{q}_{k-1} - \eta(\mathbf{H}\mathbf{q}_{k-1} - \mathbf{v})$ ，式 (4.16)，

其中 $\mathbf{H}\mathbf{q}_{k-1}$ 通过高效 Hessian-向量积计算。

5: 结束循环

6: 令 $\mathbf{q} = \mathbf{q}_k$ 。

7: 按式 (4.3) 计算 $\nabla_{\mathbf{y}}\mathcal{U}$ ：

$$\nabla_{\mathbf{y}}\mathcal{U} = \frac{\partial\mathcal{U}(\mathbf{y},\mathbf{x}^*)}{\partial\mathbf{y}} - \underbrace{\left(\frac{\partial^2\mathcal{L}(\mathbf{x}^*(\mathbf{y}),\mathbf{y})}{\partial\mathbf{y}\partial\mathbf{x}^*(\mathbf{y})} \cdot \mathbf{q}\right)^\top}_{(\mathbf{H}_{\mathbf{y}\mathbf{x}} \cdot \mathbf{q})^\top}, \quad \text{式 (4.17),}$$

其中 $\mathbf{H}_{\mathbf{y}\mathbf{x}} \cdot \mathbf{q}$ 也可通过 Hessian-向量积高效计算。

近似。隐式微分实现复杂，但有一种近似方法是忽略隐式成分，仅使用直接部分 $\hat{\nabla}_{\mathbf{y}}\mathcal{U} \approx \frac{\partial\mathcal{U}}{\partial\mathbf{y}}\Big|_{\mathbf{x}_T}$ 。这等价于在上层问题中将下层优化得到的解 $\mathbf{x}_T$ 视为常数。该近似效率更高，但会引入误差项

$$\epsilon \sim \left| \frac{\partial\mathcal{U}}{\partial\mathbf{x}^*} \frac{\partial\mathbf{x}^*}{\partial\mathbf{y}} \right|. \quad (4.18)$$

不过，当隐式梯度包含较小二阶导数的乘积时，这一近似仍然有用，具体取决于所处理的 NLS 问题。

4.6 流形上的微分

典型 SLAM 系统的状态必然在流形上演化，因此，流形上的优化对于求解 SLAM 后端问题至关重要。下面推导对属于 Lie 群的变量求导时所需的雅可比矩阵，这是流形微分的关

键步骤。

4.7 Lie 群导数

第 2 章已经介绍 Lie 群、Lie 代数及其基本运算（例如指数映射和对数映射）的基本概念。本节将简要回顾这些概念，但主要关注其导数的定义，因为这是求解可微优化问题所必需的。

考虑一个 Lie 群的流形。该光滑流形上的每一点 χ 都有唯一的切空间，记为 $T_\chi \mathcal{M}$ ，微积分的基本原理在其中成立。Lie 代数 $\mathfrak{m}$ 是一个向量空间，并可在点 χ 的邻域局部定义为 $\mathfrak{m} = T_\chi \mathcal{M}$ 。指数映射 $\exp : \mathfrak{m} \rightarrow \mathcal{M}$ 将 Lie 代数中的元素映射至 Lie 群；对数映射 $\log : \mathcal{M} \rightarrow \mathfrak{m}$ 则是其逆，建立双向关系：

$$\chi = \exp(\tau^\wedge) \Leftrightarrow \tau^\wedge = \log(\chi), \quad (4.19)$$

其中 hat 算子 $\wedge$ 是线性可逆映射，且 $\tau^\wedge \in \mathfrak{m}$ 。若将 Lie 代数中的坐标表示为 $\mathbb{R}^n$ 中的向量 τ ，便可定义向量 τ 与 Lie 群 χ 之间的映射：

$$\chi = \text{Exp}(\tau) \Leftrightarrow \tau = \text{Log}(\chi), \quad (4.20)$$

这里重新定义指数映射和对数映射，使其分别直接以向量为输入和输出。

要计算 Lie 群上的导数，首先必须理解两个流形元素（例如 x_1 和 x_2 ）之间的相对变化。通过先定义 $\oplus$ 和 $\ominus$ 算子可对这些变化进行量化；它们各组合一个指数/对数映射和一个复合运算。由于复合通常不可交换，这些算子依操作数顺序分为右形式和左形式。右形式算子为：

$$\begin{aligned} \text{right-}\oplus : \quad \chi_2 &= \chi_1 \oplus \tau \triangleq \chi_1 \circ \text{Exp}(\tau), \\ \text{right-}\ominus : \quad \tau &= \chi_2 \ominus \chi_1 \triangleq \text{Log}(\chi_1^{-1} \circ \chi_2). \end{aligned} \quad (4.21)$$

式 (4.21) 中 τ 位于右侧，表示它在 x_1 处的局部坐标系中表示。相反，式 (4.22) 的左形式算子反映全局坐标系视角：

$$\begin{aligned} \text{left-}\oplus : \quad \chi_2 &= \varepsilon \oplus \chi_1 \triangleq \text{Exp}(\varepsilon) \circ \chi_1, \\ \text{left-}\ominus : \quad \varepsilon &= \chi_2 \ominus \chi_1 \triangleq \text{Log}(\chi_2 \circ \chi_1^{-1}), \end{aligned} \quad (4.22)$$

其中， ε 在全局坐标系中表示。 τ 和 ε 都可看作流形元素的增量扰动；借助相应的复合算子 $\oplus$ 和 $\ominus$ ，这些变化以切空间中的向量表示。

有了右形式的 $\oplus$ 和 $\ominus$ 算子后，使用雅可比矩阵 J 描述流形上的扰动。雅可比矩阵刻画了切空间 $\mathfrak{m}$ 中无穷小扰动 τ 的本质：

$$\begin{aligned} \frac{\partial f(\chi)}{\partial \chi} &\triangleq \lim_{\tau \rightarrow 0} \frac{f(\chi \oplus \tau) - f(\chi)}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{f(\chi \circ \text{Exp}(\tau)) - f(\chi)}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{\text{Log}(f(\chi)^{-1} \circ f(\chi \circ \text{Exp}(\tau)))}{\tau}. \end{aligned} \quad (4.23)$$

令 $g(\tau) = \text{Log}(f(\chi)^{-1} \circ f(\chi \circ \text{Exp}(\tau)))$ ，则右雅可比 J_R 可表示为 $g(\tau)$ 在 $\tau = 0$ 处的导数：

$$\frac{\partial f(\chi)}{\partial \chi} = J_R = \left. \frac{\partial g(\tau)}{\partial \tau} \right|_{\tau=0}. \quad (4.24)$$

这样，流形中 $f(\chi)$ 关于 χ 的导数由雅可比矩阵 $J_R \in \mathbb{R}^{m \times n}$ 表示，其中 m 和 n 分别是流形 $\mathcal{M}$ 和 $\mathcal{N}$ 的维数。右雅可比矩阵执行从切空间 $\mathfrak{m}$ 到切空间 $\mathfrak{n} = T_{f(\chi)}\mathcal{N}$ 的线性映射。

类似地，考虑施加于 Lie 群元素 χ 的无穷小扰动 $\epsilon \in T_\chi\mathcal{M}$ ，可利用左形式的加、减算子定义左雅可比 J_L ：

$$\begin{aligned} \frac{\partial f(\chi)}{\partial \chi} &\triangleq \lim_{\epsilon \rightarrow 0} \frac{f(\epsilon \oplus \chi) \ominus f(\chi)}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} \frac{f(\text{Exp}(\epsilon) \circ \chi) \ominus f(\chi)}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} \frac{\text{Log}(f(\text{Exp}(\epsilon) \circ \chi) \circ f(\chi)^{-1})}{\epsilon} \\ &= \left. \frac{\partial \text{Log}(f(\text{Exp}(\epsilon) \circ \chi) \circ f(\chi)^{-1})}{\partial \epsilon} \right|_{\epsilon=0}. \end{aligned} \quad (4.25)$$

所得左雅可比 $J_L \in \mathbb{R}^{n \times m}$ 同样是一个线性映射，但它在全局切空间中从 $T_\chi\mathcal{M}$ 映射至 $T_{f(\chi)}\mathcal{N}$ 。

为考察点 x_1 周围的局部扰动，将 τ 取作 $\tau = \chi \ominus \chi_1$ ，其中 x 是 x_1 的扰动版本。定义在切空间上的协方差矩阵 Σ_x 由期望算子 $\mathbb{E}$ 导出，从而表征不确定性及其传播：

$$\Sigma_\chi \triangleq \mathbb{E}[\tau \tau^\top] = \mathbb{E}[(\chi \ominus \chi_1)(\chi \ominus \chi_1)^\top]. \quad (4.26)$$

这些协方差矩阵可用于在流形上建立高斯分布，即 $\chi \sim \mathcal{N}(\chi_1, \Sigma_\chi)$ 。需要注意，协方差矩阵 Σ_χ 定义在切空间 $T_{\chi_1}\mathcal{M}$ 上，因此流形上的不确定性可以由向量表示，并以协方差矩阵形式传播。

例 4.4 (视觉惯性旋转估计) 考虑一个配备惯性测量单元 (IMU) 和相机的机器人。给定两个传感器的带噪观测 R_{IMU} 和 R_{Cam} ，可通过最小化测量之间的差异估计机器人的朝向；这可表述为流形 $\text{SO}(3)$ 上的非线性最小二乘问题：

$$\hat{R} = \arg \min_{R \in \text{SO}(3)} f(R, R_{\text{IMU}}, R_{\text{Cam}}), \quad (4.27)$$

其中， $f(\cdot)$ 是量化估计朝向 R 与传感器测量 R_{IMU} 和 R_{Cam} 之间差异的代价函数。有了雅可比矩阵后，便可以有效处理流形 $\text{SO}(3)$ 上用于位姿估计的优化。代价函数 $f(\cdot)$ 可写为：

$$f(R) = \|\text{Log}(R_{\text{IMU}}^{-1}R)\|^2 + \|\text{Log}(R_{\text{Cam}}^{-1}R)\|^2. \quad (4.28)$$

要最小化 $f(R)$ ，需要计算其关于流形 $\text{SO}(3)$ 上 R 的梯度。该梯度可由右雅可比 J_R 推导为：

$$\begin{aligned} \nabla f(R) &= 2 \left(\frac{\partial \text{Log}(R_{\text{IMU}}^{-1}R)}{\partial R} \right)^\top \text{Log}(R_{\text{IMU}}^{-1}R) \\ &\quad + 2 \left(\frac{\partial \text{Log}(R_{\text{Cam}}^{-1}R)}{\partial R} \right)^\top \text{Log}(R_{\text{Cam}}^{-1}R). \end{aligned} \quad (4.29)$$

梯度 $\nabla f(R)$ 可与梯度下降等优化算法结合使用：沿切空间移动并重新投影回流形，以迭代更新位姿 R ：

$$R_{k+1} = R_k \text{Exp}(-\alpha \nabla f(R)), \quad (4.30)$$

其中 α 为步长。该迭代过程持续到代价函数 $f(R)$ 收敛至极小值，从而提供利用两个传感器测量的位姿估计 $\hat{R}$ 。

4.8 流形上的微分运算

对于典型流形运算，可以推导与求逆、复合和群作用相关的雅可比矩阵闭式表达式。这些表达式能够通过链式法则计算流形上的函数导数，从而支持 SLAM 中完整的优化过程：

$$\frac{\partial \mathcal{Z}}{\partial \chi} = \frac{\partial \mathcal{Z}}{\partial \mathcal{Y}} \frac{\partial \mathcal{Y}}{\partial \chi}, \quad (4.31)$$

其中， $\mathcal{Z} = g(\mathcal{Y})$ ，且 $\mathcal{Y} = f(\chi)$ 。

求逆的雅可比可通过将函数 $f(\chi) = \chi^{-1}$ 代入式 (4.23) 的右雅可比 J_R 推导，得到：

$$\begin{aligned} \frac{\partial \chi^{-1}}{\partial \chi} &\triangleq \lim_{\tau \rightarrow 0} \frac{\text{Log}((\chi^{-1})^{-1}(\chi \text{Exp}(\tau))^{-1})}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{\text{Log}(\chi \text{Exp}(\tau)^{-1} \chi^{-1})}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{(\chi(-\tau)^\wedge \chi^{-1})^\vee}{\tau}. \end{aligned} \quad (4.32)$$

复合的雅可比可通过将函数 $f(\chi) = \chi \circ \chi_1$ 代入式 (4.23) 推导。复合算子 $x \circ x_1$ 关于 x 的导数为：

$$\begin{aligned} \frac{\partial(\chi \circ \chi_1)}{\partial \chi} &\triangleq \lim_{\tau \rightarrow 0} \frac{\text{Log}((\chi \chi_1)^{-1}(\chi \text{Exp}(\tau) \chi_1))}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{\text{Log}(\chi_1^{-1} \text{Exp}(\tau) \chi_1)}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{(\chi_1^{-1} \tau^\wedge \chi_1)^\vee}{\tau}. \end{aligned} \quad (4.33)$$

复合算子 $x \circ x_1$ 关于 x_1 的导数为：

$$\begin{aligned} \frac{\partial(\chi \circ \chi_1)}{\partial \chi_1} &\triangleq \lim_{\tau \rightarrow 0} \frac{\text{Log}((\chi \chi_1)^{-1}(\chi \chi_1 \text{Exp}(\tau)))}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{\text{Log}(\text{Exp}(\tau))}{\tau} \\ &= \mathbf{I}. \end{aligned} \quad (4.34)$$

流形的雅可比可由 χ 的右雅可比表征，该雅可比从 $\tau \in \mathbb{R}^m$ 的指数映射导出，表示为：

$$\mathbf{J}_r(\tau) \triangleq \frac{\partial \text{Exp}(\tau)}{\partial \tau}. \quad (4.35)$$

右雅可比将 τ 的微小变化传递为 $\text{Exp}(\tau)$ 处局部切空间中的变化。类似地， x 的左雅可比将 τ 的变化映射为流形全局切空间中的变化，表示为：

$$\mathbf{J}_l(\tau) \triangleq \text{Ad}_{\text{Exp}(\tau)} \mathbf{J}_r(\tau). \quad (4.36)$$

群作用的雅可比取决于具体的群作用集合 $v \in \mathcal{V}$ 。群作用的雅可比定义为：

$$\begin{aligned} J_\chi^{X \cdot v} &\triangleq \frac{\partial(\chi \cdot v)}{\partial \chi}, \\ J_v^{X \cdot v} &\triangleq \frac{\partial(\chi \cdot v)}{\partial v}, \end{aligned} \quad (4.37)$$

其中， $\chi \in \mathcal{M}$ 且 $v \in \mathcal{V}$ 。

例 4.5 (机械臂) 考虑一个具有两个关节 R_1 和 R_2 的机械臂，每个关节均由 $\text{SO}(3)$ 中的一个元素表示。机器人末端执行器的最终朝向由关节旋转的复合决定：

$$\mathbf{R} = \mathbf{R}_1 \circ \mathbf{R}_2. \quad (4.38)$$

为了评估 $\mathbf{R}_1$ 和 $\mathbf{R}_2$ 中的微小扰动 τ 对末端执行器朝向 R 的影响, 可使用复合的雅可比对其量化:

$$\begin{aligned}\frac{\partial(\mathbf{R}_1 \circ \mathbf{R}_2)}{\partial \mathbf{R}_1} &= \lim_{\tau \rightarrow 0} \frac{(\mathbf{R}_2^{-1} \tau \wedge \mathbf{R}_2)^\vee}{\tau}, \\ \frac{\partial(\mathbf{R}_1 \circ \mathbf{R}_2)}{\partial \mathbf{R}_2} &= \mathbf{I}.\end{aligned}\quad (4.39)$$

该示例表明, 对第一个关节 $\mathbf{R}_1$ 的调整, 会经由受第二个关节 $\mathbf{R}_2$ 当前状态影响的变换作用于最终朝向 R 。而对第二个关节 $\mathbf{R}_2$ 的改变会直接影响 R , 不受第一个关节 $\mathbf{R}_1$ 的影响。

4.9 自动微分的数值挑战与现代库

AutoDif 是在包括流形微分在内的各种优化情境中准确、高效计算导数的核心技术。由于流形结构固有的复杂几何性质, 流形上的微分面临独特挑战, 这些性质可能影响 AutoDif 的性能和适用性。在可微优化中, 当 AutoDif 与流形的曲空间相互作用时, 这些挑战尤为突出, 可能引入数值不稳定性和不准确性的。

本节给出在基于流形的优化任务中使用自动微分时出现数值问题的例子, 尤其关注在流形约束下保持数值稳定性和精度的复杂性, 例如约束优化以及由流形上微分方程定义的系统。这里以 PyPose 库^[1145]为例; 它为 Lie 群和 Lie 代数定义了一种通用数据结构 LieTensor。具体而言, 将说明一些数值挑战, 以及 PyPose 如何处理这些挑战。

例 4.6 (指数映射和四元数) 指数映射是 Lie 群理论中的基本概念, 在以四元数表示的 Lie 代数和 Lie 群之间转换时尤为关键。该映射可将 $\mathbb{R}^3$ 代数结构中的角速度转换为单位四元数群 $\mathbb{S}^3$ 中的旋转朝向。从解析角度看, 四元数的指数映射由 Rodrigues 旋转公式导出; 该公式将 $\mathbb{R}^3$ 中的向量关联到相应旋转。给定 $\mathbb{R}^3$ 中向量 x , 它表示由旋转角缩放的旋转轴, 则旋转的四元数表示为:

$$\text{Exp}(\boldsymbol{\nu}) = \left[\sin\left(\frac{\|\boldsymbol{\nu}\|}{2}\right) \frac{\boldsymbol{\nu}^\top}{\|\boldsymbol{\nu}\|}, \cos\left(\frac{\|\boldsymbol{\nu}\|}{2}\right) \right]^\top \quad (4.40)$$

其中, $\|\boldsymbol{\nu}\|$ 表示 $\boldsymbol{\nu}$ 的模, 对应旋转角; $\frac{\boldsymbol{\nu}}{\|\boldsymbol{\nu}\|}$ 是沿 $\boldsymbol{\nu}$ 方向的单位向量。

实现可微 LieTensor 的一个挑战是, 通常必须为 Exp 和 Log 映射计算式 (4.40) 中的 $\frac{\sin(\|\boldsymbol{\nu}\|/2)}{\|\boldsymbol{\nu}\|}$ 等数值上存在问题的项^[1083]。当角度非常小时, 直接计算正弦和余弦函数会由于计算机中浮点数的有限表示而导致精度问题。为处理这些问题并保持数值稳定性, PyPose 采用 Taylor 展开, 以避免除以零。

$$\text{Exp}(\boldsymbol{\nu}) = \begin{cases} \left[\boldsymbol{\nu}^\top \gamma_e, \cos\left(\frac{\|\boldsymbol{\nu}\|}{2}\right) \right]^\top & \text{if } \|\boldsymbol{\nu}\| > \text{eps}, \\ \left[\boldsymbol{\nu}^\top \gamma_o, 1 - \frac{\|\boldsymbol{\nu}\|^2}{8} + \frac{\|\boldsymbol{\nu}\|^4}{384} \right]^\top & \text{otherwise.} \end{cases} \quad (4.41)$$

当 $\|\boldsymbol{\nu}\|$ 显著时, $\gamma_e = \frac{\sin(\|\boldsymbol{\nu}\|/2)}{\|\boldsymbol{\nu}\|}$; 当 $\|\boldsymbol{\nu}\|$ 很小时, $\gamma_o = \frac{1}{2} - \frac{\|\boldsymbol{\nu}\|^2}{48} + \frac{\|\boldsymbol{\nu}\|^4}{3840}$, 从而可在整个旋转幅值范围内确保精确计算。这里, eps 是使 $1 + \text{eps} \neq 1$ 成立的最小机器数。这一从解析到数值的过程说明: 在需要高保真旋转表示的应用中, 准确稳定地计算指数映射十分重要。

LieTensor 与现有库相比存在若干区别: (1) PyPose 支持任意阶梯度的 AutoDif, 且兼容 CPU、GPU、TPU 和 Apple silicon GPU 等主流设备; 而 LieTorch^[1083] 等库实现定制 CUDA 核函数, 只支持一阶梯度。(2) LieTensor 支持通过 vmap 算子并行计算梯度, 因此能更快地计算雅可比矩阵。(3) LieTorch、JaxLie^[1244] 和 Theseus 等库只支持 Lie 群, 而

PyPose 同时支持 Lie 群和 Lie 代数。因此，可直接从 LieTensor 实例调用 Exp 和 Log 映射，更加灵活易用；两类变量的梯度也可自动计算和反向传播。读者可在 [1144] 找到受支持 LieTensor 运算的列表，在 [1147] 找到 PyPose 教程。有关 LieTensor 及其自动微分的用法，参见 <https://github.com/pypose/slambook-snippets/blob/main/lietensor.ipynb>。

4.10 可微优化的实现示例

为使用双层优化实现端到端学习，需要集成除神经方法所需的 SGD [930] 和 Adam [579] 等基于梯度的方法之外的通用优化器，因为 SLAM 中的许多问题还需要约束优化或 2nd 阶优化 [51] 等其他优化算法。此外，实际问题中存在离群值，因此需要像第 3 章所述那样使损失函数鲁棒化。下面考虑第 3.3 节介绍的一种用于 SLAM 的 IRLS 方法，并说明 PyPose 面向优化的接口背后的思想，包括求解器 (solver)、核函数 (kernel)、校正器 (corrector) 和策略 (strategy)，以使用 2nd 阶 Levenberg-Marquardt (LM) 优化器。

首先考虑一个加权最小二乘问题：

$$\min_{\mathbf{x}} \sum_i (\mathbf{h}_i(\mathbf{x}_i) - \mathbf{z}_i)^T \boldsymbol{\Sigma}_i (\mathbf{h}_i(\mathbf{x}_i) - \mathbf{z}_i), \quad (4.42)$$

其中， $h(\cdot)$ 是回归模型（模块）， $\mathbf{x} \in \mathbb{R}^n$ 是待优化参数， $\mathbf{h}_i$ 表示第 i 个输入样本的预测， $\boldsymbol{\Sigma}_i \in \mathbb{R}^{d \times d}$ 是信息矩阵。LM 算法通过迭代更新估计 $\mathbf{x}_t$ 得到式 (4.42) 的解，即 $\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} + \boldsymbol{\delta}_t$ ，其中更新步 $\boldsymbol{\delta}_t$ 计算为：

$$\sum_i (\boldsymbol{\Lambda}_i + \lambda \cdot \text{diag}(\boldsymbol{\Lambda}_i)) \boldsymbol{\delta}_t = - \sum_i \mathbf{J}_i^T \boldsymbol{\Sigma}_i \mathbf{r}_i, \quad (4.43)$$

其中， $\mathbf{r}_i = \mathbf{h}_i(\mathbf{x}_i) - \mathbf{z}_i$ 是第 i 个残差； $\mathbf{J}_i$ 是在 $\mathbf{x}_{t-1}$ 处计算的 h 的雅可比矩阵； $\boldsymbol{\Lambda}_i = \mathbf{J}_i^T \boldsymbol{\Sigma}_i \mathbf{J}_i$ 是近似 Hessian 矩阵； λ 是阻尼因子。为得到步长 $\boldsymbol{\delta}_t$ ，需要一个线性求解器：

$$\mathbf{A} \cdot \boldsymbol{\delta}_t = \boldsymbol{\beta}, \quad (4.44)$$

其中， $\mathbf{A} = \sum_i (\boldsymbol{\Lambda}_i + \lambda \text{diag}(\boldsymbol{\Lambda}_i))$ ， $\boldsymbol{\beta} = - \sum_i \mathbf{J}_i^T \boldsymbol{\Sigma}_i \mathbf{r}_i$ 。实践中，方阵 $\mathbf{A}$ 通常是正定的，因此可使用 Cholesky 等标准线性求解器。若雅可比 $\mathbf{J}_i$ 很大且稀疏，也可使用稀疏 Cholesky [161] 或预条件共轭梯度 (PCG) [459] 等稀疏求解器。

实践中，常在式 (4.42) 中引入鲁棒核函数 $\rho: \mathbb{R} \mapsto \mathbb{R}$ ，以减小离群值的影响：

$$\min_{\mathbf{x}} \sum_i \rho(\mathbf{r}_i^T \boldsymbol{\Sigma}_i \mathbf{r}_i), \quad (4.45)$$

其中， ρ 用于降低具有较大残差 $\mathbf{r}_i$ 的测量的权重。此时需要调整式 (4.43) 以考虑鲁棒核的存在。一种常用方法是采用 IRLS 方法 Triggs 校正 [1107]，Ceres [15] 库也采用该方法。然而，它需要核函数 ρ 的二阶导数，而该导数始终为负，可能导致包括 LM 在内的二阶优化器不稳定 [1107]。作为替代方案，PyPose 提出 IRLS 方法 FastTriggs；它仅涉及一阶导数，比 Triggs 更快也更稳定：

$$\mathbf{r}_i^\rho = \sqrt{\rho'(c_i)} \mathbf{r}_i, \quad \mathbf{J}_i^\rho = \sqrt{\rho'(c_i)} \mathbf{J}_i, \quad (4.46)$$

其中, $c_i = \mathbf{r}_i^T \Sigma_i \mathbf{r}_i$; $\mathbf{r}_i^\rho$ 和 J_i^ρ 分别是引入核函数后校正的模型残差和雅可比。FastTriggs 的更多细节及其证明参见 [1143]; IRLS 已在第 3.3 节介绍。

若初始猜测距离最优解过远, 简单的 LM 优化器可能无法收敛到全局最优解。因此, 常需采用自适应阻尼、Dogleg 和信赖域方法 [697] 等策略约束每一步, 避免步长“过大”。要采用这些策略, 只需将策略实例 (例如 TrustRegion) 传给优化器。总之, PyPose 只需在构造函数中传入优化器参数, 即可方便地扩展上述算法; 这些参数包括 solver、strategy、kernel 和 corrector。可用算法列表和示例见 [1146]。二阶优化的用法参见 <https://github.com/pypose/slambook-snippets/blob/main/optimization.ipynb>。

4.11 相关开源库

与可微优化相关的开源库可分为三类: (1) 线性代数库; (2) 机器学习库; (3) 专用优化库。

线性代数库是机器学习和机器人研究的基础。Python 线性代数库 NumPy [823] 提供全面的向量和矩阵操作, 并得益于底层经过良好优化的 C 代码而具有较高运行速度。高性能 C++ 线性代数库 Eigen [412] 已被 TensorFlow [2]、Ceres [15]、GTSAM [252] 和 g2o [402] 等众多项目采用。面向 C、C++、Fortran 和 Python 的 GPU 加速库 ArrayFire [727] 包含简洁 API, 并提供针对 GPU 优化的函数。

机器学习库更关注张量 (即高维矩阵) 上的运算和自动微分。Torch [228]、OpenNN [830] 和 MATLAB [738] 等早期机器学习框架为研究人员开发神经网络提供了基础工具, 但它们只支持 CPU 计算且缺少简洁 API, 限制了可用性。几年后, Chainer [1101]、Theano [24] 和 Caffe [518] 等深度学习框架出现, 以应对神经网络规模和复杂度的增长; 它们支持多 GPU 训练, 并为用户构建和训练神经网络提供了便捷 API。进一步地, TensorFlow [2]、PyTorch [850] 和 MXNet [184] 等近期框架提供了完整而灵活的生态系统, 例如多种编程语言 API、分布式数据并行训练, 以及基准测试和部署辅助工具。Gvnm [430] 将可微变换层引入基于 Torch 的框架, 从而实现端到端几何学习。JAX [110] 可自动微分原生 Python 和 NumPy 函数, 并是一个可组合函数变换的可扩展系统。这些框架的出现以多种方式促进了深度学习的发展。近来, 人们进一步努力将标准优化工具与深度学习结合。这些工作如 Theseus [873] 和 Cvxpy-Layer [18] 展示了如何将可微优化嵌入深度神经网络。PyPose [1145] 集成 Gauss-Newton 和 Levenberg-Marquardt 等二阶优化器, 并可计算 Lie 群和 Lie 代数的任意阶梯度, 这些能力对于机器人学至关重要。

其他专用优化库也已在机器人学中开发和应用。举例而言, Ceres [15] 是面向大规模非线性最小二乘优化问题的开源 C++ 库, 已广泛用于 SLAM。Pyomo [434] 和 JuMP [292] 是优化框架, 因其灵活支持构建、求解和分析优化模型的多种工具而被广泛使用。CasADi [32] 凭借对最优控制多种数值方法的快速有效实现, 已用于求解机器人学中的许多现实控制问题。位姿优化和因子图优化也在机器人学中发挥重要作用。例如, g2o [402] 和 GTSAM [252] 是用于基于图的非线性优化的开源 C++ 框架, 提供构建新问题的简洁 API, 并已用于解决 SLAM 中的多个优化问题。

优化库也已广泛用于机器人控制问题。举例而言, IPOPT [1139] 是基于内点法的非线性规划开源 C++ 求解器, 广泛用于机器人与控制。类似地, OpenOCL [590] 支持连续时间、离散时间、约束、无约束、多阶段和轨迹优化等多类优化问题, 可用于实时模型预测控制。另一种面向大规模最优控制和估计问题的库是 CT [383], 它为不同的最优控制求解器提供标准接口,

并可扩展到机器人应用中的广泛动力学系统。Drake^[1079] 为常见控制问题提供求解器，可直接与其仿真工具箱集成；其系统完整性使其受到研究人员青睐。

4.12 延伸阅读与近期趋势

近年来，深度学习方法取得显著发展^[1286]。作为数据驱动方法，相较于传统手工特征，深度学习被认为在视觉跟踪中表现更好。该领域的多数研究采用端到端结构，包括 DeepVO^[1163] 和 TartanVO^[1167] 等有监督方法，以及 UnDeepVO^[656] 和 Unsupervised VIO^[1173] 等无监督方法。一般认为，有监督方法能够从位姿、光流和深度等多样的真值中学习，因此性能优于无监督方法；但在真实世界中获取这类真值是一项劳动密集型工作^[1166]。

近来，混合方法日益受到关注，因为它们整合了基于几何方法与深度学习方法的优点。一些研究探索了将 BA 与深度学习结合，以在帧间施加拓扑一致性，例如为学习网络附加 BA 层的 BA-Net^[1068] 和 DROID-SLAM^[1082]。此外，另一些工作侧重于将图像特征压缩为编码（嵌入式特征），并在推理期间优化位姿-编码图，例如 DeepFactors^[236]。DiffPoseNet^[842] 则使用网络预测位姿和法向流，并通过 Cheirality 层微调粗略预测。然而，在这些工作中，基于学习的方法与基于几何的优化是解耦的，分别用于不同子模块。前端和后端缺少集成可能导致次优性能；此外，它们仅将位姿误差“穿过”BA 反向传播，因此监督信号来自真值位姿。在这种情形下，BA 只是网络的一个特殊层。近期，iSLAM^[346] 以双向方式连接前端和后端，并通过双层优化框架迫使学习模型从几何优化中学习，在不使用外部监督的条件下实现性能提升。其他任务也可表述为双层优化，例如强化学习^[1027]、局部规划^[1216]、全局规划^[186]、特征匹配^[1267] 和多机器人路径规划^[419]。

第五章 稠密地图表示

作者： Victor Reijgwart、Jens Behley、Teresa Vidal-Calleja、Helen Oleynikova、Lionel Ott、Cyrill Stachniss 和 Ayoun Kim

本章将关注 SLAM 的另一个方面，即建图组件。我们假设机器人位姿已经完成估计，目标是构建其周围环境的稠密地图。典型方法首先使用 SLAM 后端（前几章已讨论）求解机器人的运动轨迹，然后在给定轨迹的情况下重建稠密地图。本章将详细介绍稠密地图表示，重点讨论地图元素、数据结构和构建方法。

早期建图方法主要采用第 1 章所述的稀疏、基于路标的方案，仅从环境中提取少量显著特征。随着计算能力提升，以及机械式 3D LiDAR 或 RGB-D 相机等高精度 3D 范围传感器的出现（这些传感器能够以高频率提供详细的 3D 范围测量），稠密地图表示逐渐受到研究关注。对于规划、导航、操作和精确定位等需要详细理解环境的下游任务，稠密地图至关重要。本章解释这些稠密方法如何利用范围传感器数据的完整信息来细化和更新综合地图。

本章首先在第 5.1 节介绍支持稠密建图的主要传感器类型，重点是能够产生详细范围测量的范围传感器。第 5.2 节介绍基本表示元素和数据结构，并在第 5.4 节针对具体应用展开。与稀疏的基于路标建图相比，稠密地图表示的选择取决于所使用的传感器类型和预期的下游应用。影响表示类型选择的关键因素将在第 5.5 节总结。

5.1 范围感知基础

在深入讨论稠密地图表示之前，我们先简要总结一种 SLAM 中常用的关键感知模态，即范围传感器，为后续讨论提供必要背景。这类传感器测量环境中物体的距离，包括 LiDAR 传感器、飞行时间（time-of-flight, TOF）相机、RGB-D 相机和双目相机。这里我们重点介绍最常用的 LiDAR 传感器和 RGB-D 相机，它们主要用于室内外环境中的 SLAM 与稠密建图。

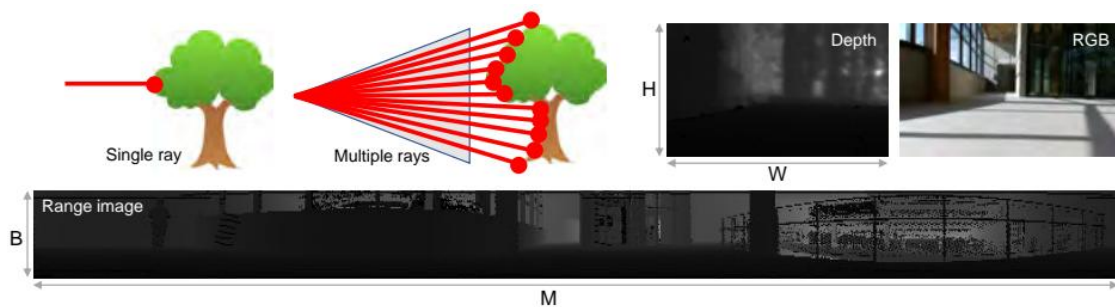


图 5.1: 图 5.1 LiDAR 传感器的单射线和多射线类型。图中给出了真实场景中的 RGB、深度和 LiDAR 范围图像示例。

5.2 传感器测量模型

先简要回顾感知机制及其测量模型。对于 LiDAR 传感器，¹ 激光束由传感器发射，经环境反射后被探测器接收^[935]。设激光束发射时刻为 t_{emit} ，探测时刻为 t_{detect} ，利用光速 c 可按下式得到范围 r ：

$$r = \frac{c(t_{\text{detect}} - t_{\text{emit}})}{2}. \quad (5.1)$$

单条射线测量可以通过按特定模式移动的射线阵列扩展为二维（2D）或三维（3D）点集合，例如进行 360 度旋转或沿特定形状扫描。传感器生成的点集合称为点云，它是构建地图的基本元素。LiDAR 测量也可以表示为范围图像 $\mathbf{R} \in \mathbb{R}^{B \times M}$ ：其中存储传感器完成一次完整旋转（即 360°）时 B 条射线各自的范围。因此，对于每条射线，在传感器的水平视场内有 M 个测量值；图 5.1 给出了示意和示例。

机器人应用中另一类常用范围传感器是 RGB-D 相机，例如 Microsoft Kinect、Azure Kinect DK 和 Intel RealSense。RGB-D 相机除了提供高为 H 、宽为 W 的 RGB 图像 $I_{\text{RGB}} \in \mathbb{R}^{3 \times H \times W}$ 外，还提供一个尺寸相同的深度图 $I_D \in \mathbb{R}^{H \times W}$ ，其中每个像素位置包含深度或范围。早期 RGB-D 相机使用具有已知图案的结构化红外（IR）光源，将该图案投射到环境中来生成深度图 I_D ；随后根据图案的畸变确定各像素的距离。由于阳光通常会干扰这种感知机制，采用投射光的 RGB-D 相机主要用于室内环境。新一代 RGB-D 传感器引入 IR 纹理投影器或飞行时间（TOF）技术，受干扰影响较小，因此可以支持室外应用。

5.3 转换为点云

利用范围传感器（例如 LiDAR 或 RGB-D 相机）的内参，可以将范围图像 R 或深度图 I_D 转换为点云 $\mathbf{P} = \{p_1, \dots, p_N\}$ 。其中点 $\mathbf{p}_i \in \mathbb{R}^3$ 在传感器的局部坐标系中表示。点是地图表示中最基本的单元，本章后续将进一步讨论。

对于 LiDAR，传感器为每条射线提供内参标定 $(\phi_{i,j}, \theta_{i,j})$ ，其中 $1 \leq i < B$ 且 $1 \leq j < M$ ；它们分别是方位角 $\phi_{i,j} \in [0, 2\pi]$ 与极角（俯仰角） $\theta_{i,j} \in [-\pi, \pi]$ ，如图 5.2 所示。利用每条射线的已知角度，可以将 $R_{i,j}$ 中的范围测量 $r_{i,j}$ 转换为三维点 $\mathbf{p} = (x, y, z)$ ：

$$x = r_{i,j} \cos(\theta_{i,j}) \cos(\phi_{i,j}) \quad (5.2)$$

$$y = r_{i,j} \cos(\theta_{i,j}) \sin(\phi_{i,j}) \quad (5.3)$$

$$z = r_{i,j} \sin(\theta_{i,j}). \quad (5.4)$$

对于 RGB-D 相机，通常采用针孔相机模型，将像素位置 (u, v) 处的范围 $r_{u,v} = R_{u,v}$ 转换为三维坐标。利用相机内参 $\mathbf{K} \in \mathbb{R}^{3 \times 4}$ ，将图像坐标中的齐次坐标 $\mathbf{x} = (u, v, r_{u,v})$ 通过伪逆转换到相机坐标中的点 $\mathbf{p}$ ：

$$\mathbf{p} = \mathbf{K}^\dagger \mathbf{x}. \quad (5.5)$$

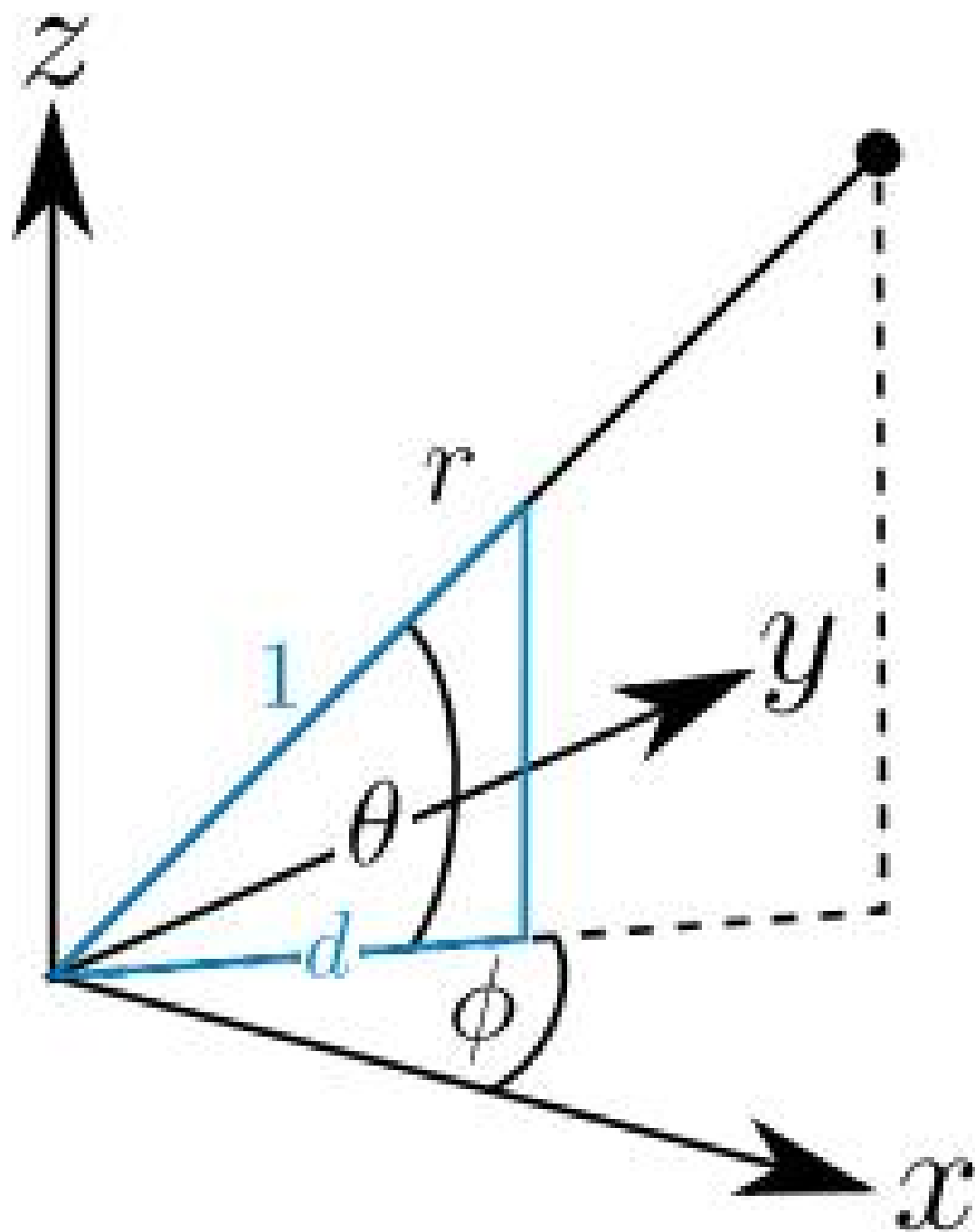


图 5.2: 图 5.2 球面坐标。

根据点的组织方式, 所得点云可分为有组织点云和无组织点云。从深度图转换得到的点云是有组织的, 每个点的位置与对应深度图的像素位置保持结构关系, 因此可通过简单索引计算相邻点。LiDAR 传感器生成的点云则更复杂。例如, 静止的 3D 机械式 LiDAR 生成的点云具有组织结构; 而对于非重复图案固态或 flash LiDAR, 以及运动中的机械旋转式 LiDAR, 这种组织结构不再成立。很多情况下, 无组织点云会因传感器运动而发生畸变, 范围图像中的测量邻域也不一定与空间邻域相关。因此, 需要估计传感器在完成一次扫描期间的运动 (例如利用惯性测量单元 (IMU) 测量或里程计信息), 结合每条射线的时间信息, 对 LiDAR 点云进行去畸变, 以补偿传感器运动^[1133, 256, 1231]。关于运动畸变及其补偿的更多细节, 请参阅第 8 章。

122

Dense Map Representations

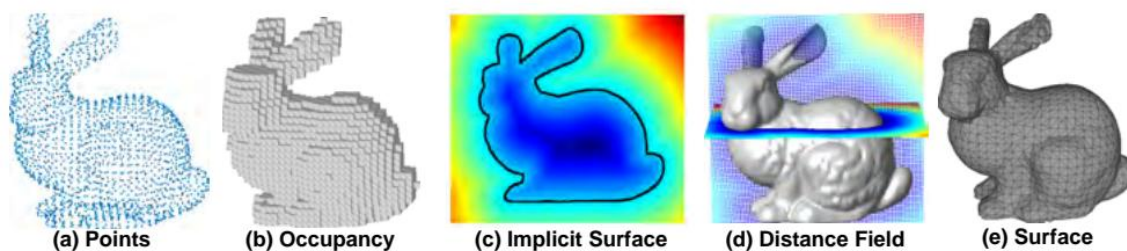


图 5.3: 图 5.3 常见稠密表示的示例。

5.4 稠密建图基础

地图是利用传感器信息和数据处理方法生成的、用于建模环境的符号结构^[1091, 136]。需要注意的是, 地图表示具有多样性, 同一空间信息可以有許多不同表示方式 (如图 5.3 所示)。场景表示的选择和精度会显著影响具体任务的性能, 因此表示方式应由使用场景决定。例如, 机器人中的运动估计和定位倾向于采用稀疏表示, 如 3D 点特征^[787, 896], 以利用这些特征实现一致的机器人位姿估计。另一方面, 场景重建的关键目标是获得准确、稠密且高分辨率的地图, 例如用于检查任务^[506, 822, 922]。类似地, 路径规划需要稠密信息, 例如障碍物占据状态或与障碍物的最近距离^[774, 304, 1110]。总体而言, 本章围绕以下三个问题组织内容。

问题 1: 稠密建图需要估计什么量? 表示环境时最常用的量是占据状态和距离。占据状态是建图中的关键属性, 用于区分自由空间与被占据空间。距离估计则通过测量到邻近表面或物体的范围, 为自由和占据空间提供更符合机器人视角的解释。

稠密建图基础。

问题 2: 应如何表示世界? 这是关于采用何种空间抽象进行表示的问题。概括而言, 表示可以是显式的, 也可以是隐式的。显式空间抽象还可根据所使用的几何类型分类, 而隐式表示可根据所选函数分类。图 5.4 展示了这些抽象类型。

问题 3: 应使用什么数据结构和存储方式? 选定的表示需要存储在内存中供后续使用。数据结构和存储方法应根据具体应用及预期用途进行选择。

文献中存在大量利用范围传感器生成场景稠密表示的方法, 它们在估计量、空间抽象、存储结构、连续性和应用领域等方面各不相同。

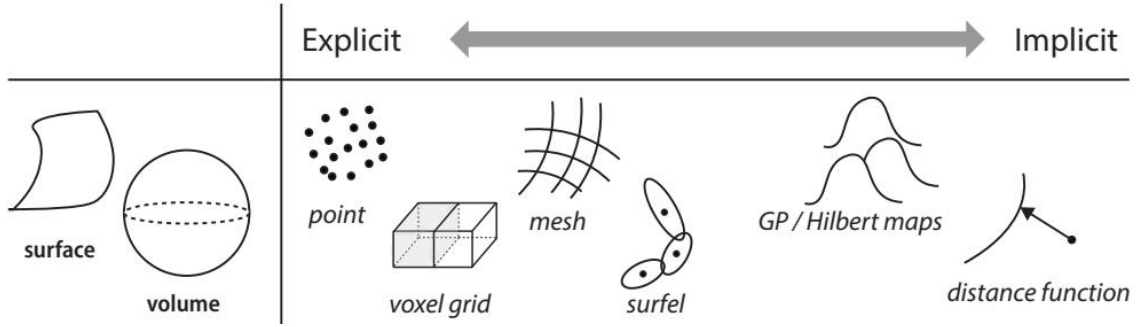


图 5.4: 图 5.4 表示可以是显式的，也可以是隐式的。为清晰起见，图示已作简化；在实际中，抽象并非严格可分，通常可以组合使用。

在讨论不同表示之前，我们先研究由范围测量估计的主要量，重点理解建图阶段最常估计的量。下面简要概述每个量的基本定义，并说明其重要性及发挥作用的具体场景。

5.5 占据地图

占据栅格由 Elfes 和 Moravec^[304, 774] 在三十多年前提出，此后得到广泛使用。占据栅格² 简单且计算高效，因此在室内（甚至室外）环境建图中很受欢迎。最简单的情形是估计某个栅格单元被占据的概率：将单元含有障碍物建模为概率，已占据单元的占据概率为 1，被判定为空的单元为 0。本质上，占据建图是一个二元分类问题，用于预测每个单元属于二元类别的概率。

给定传感器测量集合 $\mathbf{z}_{1:t}$ 和传感器位姿集合 $\mathbf{x}_{1:t}$ ，地图 m 中每个单元被占据的概率可表示为 $p(m|\mathbf{z}_{1:t}, \mathbf{x}_{1:t})$ 。图 5.5 给出了示例地图。假设每个单元 m_k 相互独立且测量条件独立，占据状态可用著名的对数几率形式高效更新^[774]：

$$l(m_k|\mathbf{z}_{1:t}) = l(m_k|\mathbf{z}_{1:t-1}) + l(m_k|z_t), \quad (5.6)$$

其中 $l(\cdot) = \log(o(\cdot))$ ， $o(\cdot)$ 为几率形式：

$$o(m_k|\mathbf{z}_{1:t}) = \frac{p(m_k|\mathbf{z}_{1:t})}{1 - p(m_k|\mathbf{z}_{1:t})}. \quad (5.7)$$

占据建图的主要优势体现在式 (5.6) 中：只需使用先前占据值和逆传感器模型 $l(m_k|z_t)$ ，通过简单加法即可更新概率。尽管如此，占据栅格要实现高效运行，依赖于对环境作出的很强假设。特别是，假设一个单元的占据似然与其他单元无关，会忽略空间相关性，而空间相关性对于推断附近未观测区域的占据状态可能十分重要。此外，传统占据栅格要求预先定义环境离散化方式，使地图各处空间分辨率保持不变。

5.6 隐式表面与距离场

表示机器人周围几何形状的另一方法不是使用占据概率，而是描述自由空间与占据空间之间的边界。在理想情况下，可以用关于三个变量 x, y, z 的解析函数描述该边界的形状和位置；当点 $\mathbf{p} = (x, y, z)$ 位于表面上时，函数值为 0。换言之，该函数的零交叉对应表面本身。

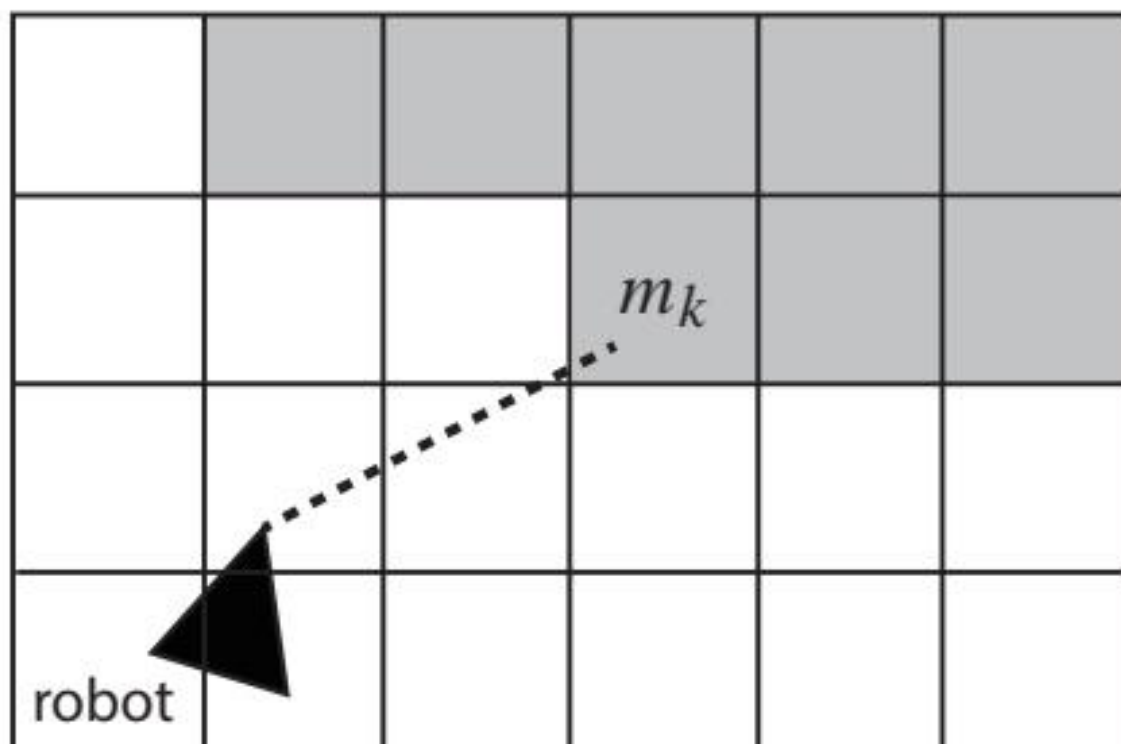


图 5.5: 图 5.5 二维简单射线投射示例。给定某个（计算或估计的）方位角处的范围测量，范围测量的回波表示存在障碍物。

这种函数称为隐式表面。按照约定，当 $\mathbf{p}$ 位于物体内部时函数符号为负，在外部时为正。图 5.6 给出了一个简单示例，说明如何确定隐式函数值。

连续隐式表面表示有许多优点。由于不存在固定分辨率，它可以在任意细节层次表示任意形状的物体。此外，只需计算函数符号，就能判断空间中的给定点位于障碍物内部还是外部。

可以使用不同类型的函数对隐式表面建模，其中欧氏有符号距离函数（Euclidean Signed Distance Function, ESDF）是一种常见选择。在任意查询点处，ESDF 的幅值表示到最近表面的距离，符号表示该点是否位于障碍物内部。ESDF 表示常用于加速几何算法，例如碰撞检查。此外，还可以从 ESDF 推导高质量的邻近梯度，用于基于优化的运动规划和形状配准。

ESDF 通过为地图中的每个点寻找（欧氏）最近表面点来计算。对于已知表面，可使用 Fast Marching 等技术高效完成。然而，在表面估计中，更常使用投影有符号距离，因为它可以从测量中高效计算，也更适合滤波。给定一条穿过查询点的测量射线，投影距离定义为沿射线从射线端点到查询点的距离，因此不必显式搜索最近点。尽管投影距离会高估欧氏距离，但其零交叉（估计表面）仍然正确。Curless 和 Levoy^[234] 提出的、后来由机器人领域 KinectFusion^[801] 推广的标准隐式表面估计方法，将所有测量的投影有符号距离进行加权平均。为减小高估的影响，投影有符号距离函数通常被截断到固定范围内，称为截断带，此时称为截断有符号距离函数（Truncated Signed Distance Function, TSDF）。注意，ESDF 可以从 TSDF 和占据地图高效计算，详见第 5.4.4 节。

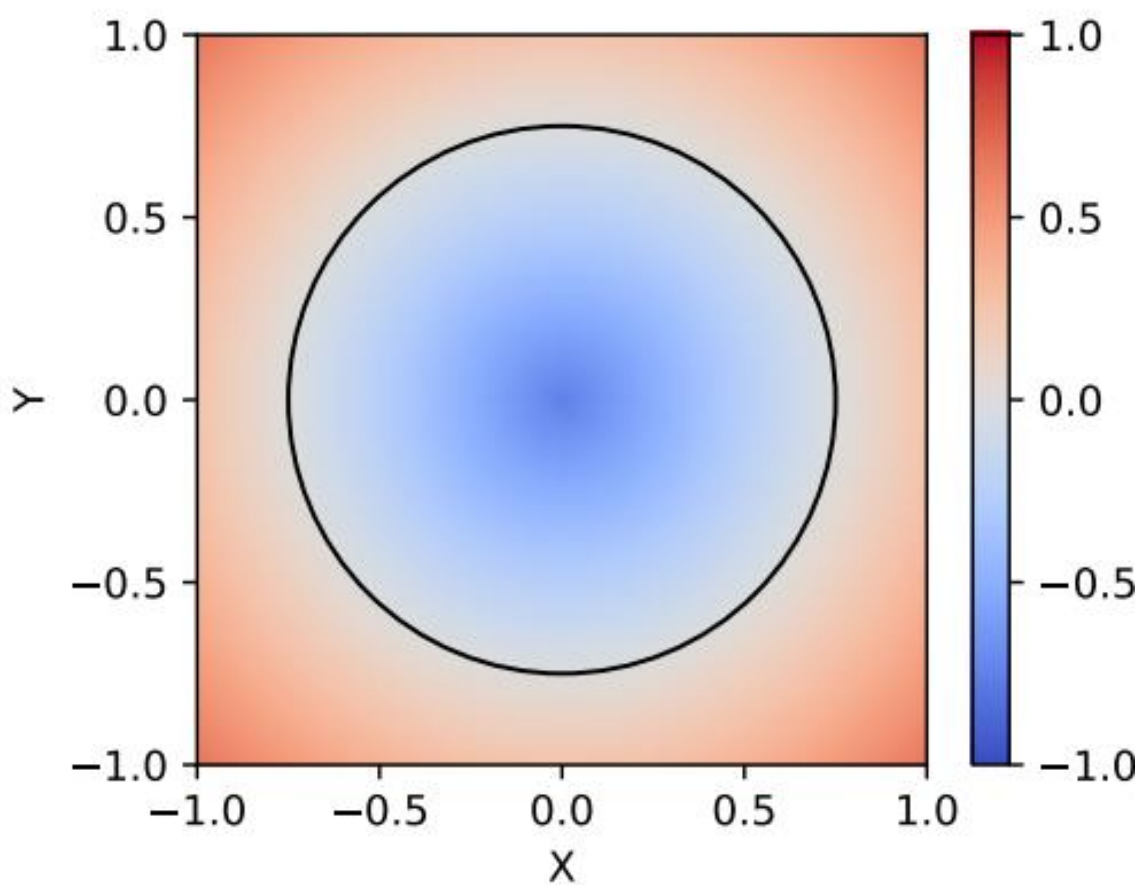


图 5.6: 图 5.6 用显式表面（黑色轮廓）和隐式表面（彩色场）表示的实心二维圆盘。隐式表面函数在物体内部（蓝色）为负，在外部（红色）为正。注意该函数的零交叉对应表面本身。

5.7 占据地图还是距离场？

占据表示和隐式表示都是体积方法，它们的共同点是：通过在观测体积内处处估计感兴趣的量来建模几何形状。然而，两种表示的优先目标不同，哪一种更好取决于具体应用。下面简要总结两个关键差异。

建模直接性：测量射线可以直接告诉我们空间的哪些部分是自由、被占据或未观测的，因此基于占据概率的地图更新所需的启发式方法和假设更少。相比之下，隐式表面通常对到表面的距离建模。对于已知表面，该距离可以精确计算；但对于部分测量，只能依赖前述 TSDF 等距离代理。

平滑性：隐式表面地图天然比占据地图平滑，后者建模的是二元属性。隐式表面的平滑性带来许多好处，最重要的是使其可微，从而产生对许多应用有价值的邻近梯度。平滑性还能降低离散化导致的近似误差，并通过插值获得良好的亚像素分辨率估计。然而，由于不连续性无法平滑表示，隐式表面往往会漏掉细薄障碍物。

5.8 地图表示

5.9 目标空间结构的显式性

如图 5.4 所示，可以根据表示的显式性及其目标空间进行分类。在 3D 建图中，表示体积很直接；但在机器人建图中，表面同样重要，因为它支持各种下游任务。我们可以考虑四类主要表示：显式表面、隐式表面、显式体积和隐式体积表示。

对于表面，可以采用显式或隐式方式表示。显式表示将表面视为二维流形，旨在用场景中物体的边界刻画空间。能够表示边界的最简单抽象就是范围传感器生成的点云。另一种通用表面表示是多边形网格（第 5.4.3 节），由顶点、边和面组成。网格通过形成相连的闭合多边形（更常见的是三角形）来编码体积的定向表面。Surfel（第 5.4.2 节）也是常用抽象，广泛用于建图。表面表示是可视化应用的基础，也用于渲染仿真环境、增强现实、计算机辅助设计和 3D 打印。

3D 体积建模采用类似策略。基于点的朴素表示常用于 LiDAR SLAM；此外，基于占据或距离的体素（第 5.4.4 节）也是显式表示的常见选择。存储体积地图时，需要仔细考虑数据存储，以降低计算成本。体积建图也使用隐式表示，通常通过函数实现。高斯过程（Gaussian process, GP，第 5.4.5 节）和 Hilbert map（第 5.4.6 节）是隐式表示的典型例子。

5.10 空间抽象类型

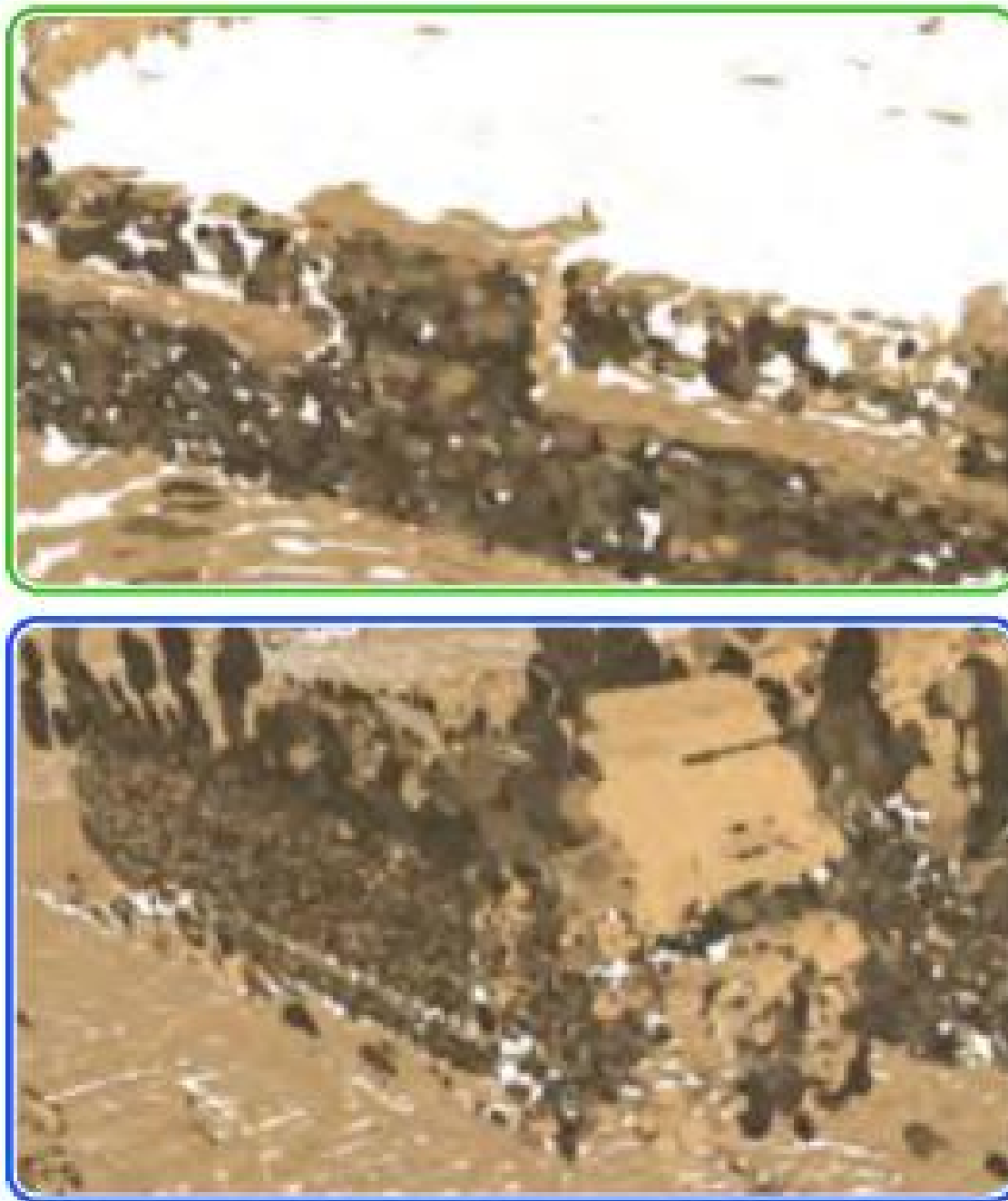
5.11 点

给定范围测量，一种直接的稠密地图表示是使用点云。构建时，我们利用估计的全局位姿 T_t^1 ，将时刻 t 在局部坐标系 $\mathcal{F}^t$ 中记录的点云 P_t 累积到全局地图点云 P_M 中。通常假设 P_M 的全局坐标系取为第一个点云的坐标系 $\mathcal{F}^1$ 。

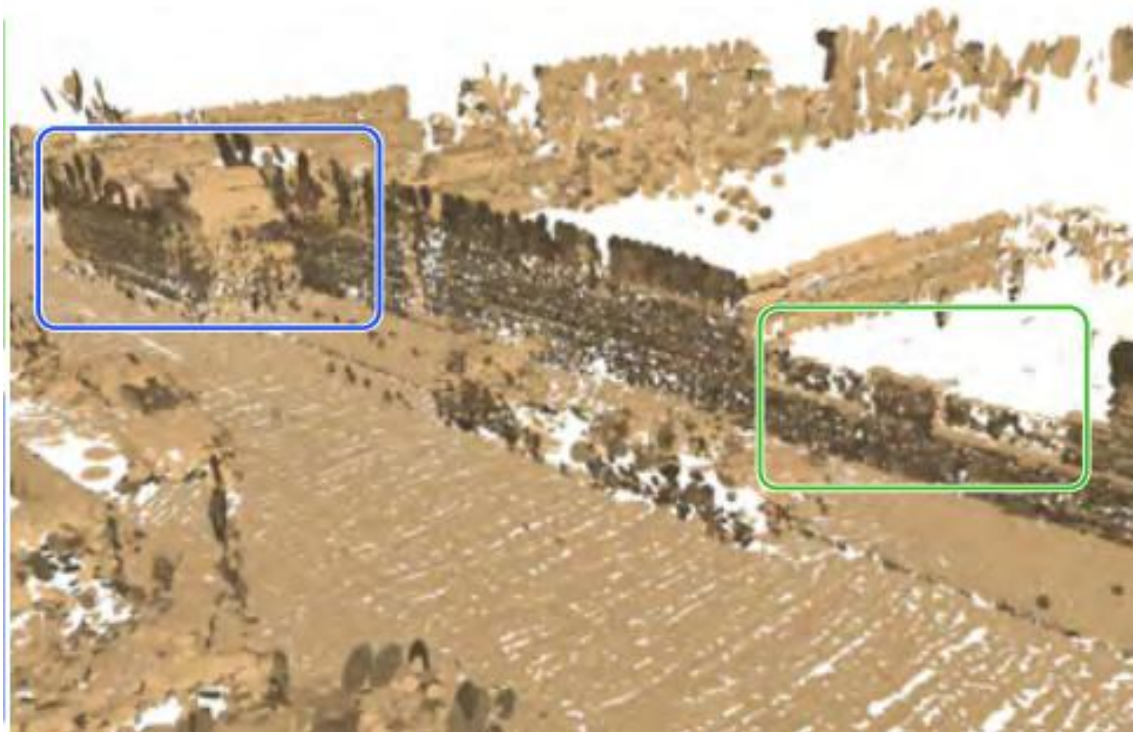
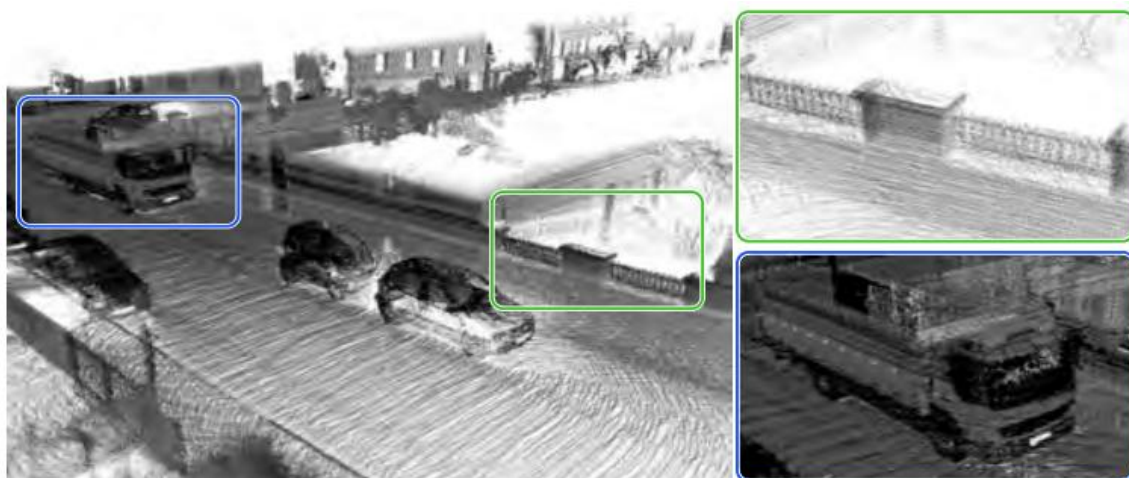
简单累积点云 $P_1, \dots, P_t$ 无法扩展到大型环境，因此常见策略是丢弃同一空间位置的冗余测量。为此，大多数方法^[1269, 256, 1133]使用高效的最近邻搜索（例如体素栅格或基于层次树的

表示, 见第 5.3.3 节) 对点云进行下采样和存储, 例如每个体素只存储有限数量的点^[256, 1133], 或仅保存满足特定准则的点^[1269]。也可以只保存关键帧, 即显式存储少量点云, 但这需要确定何时生成关键帧或子地图。

点云是最基本的表示形式, 可以转换为其他表示, 例如通过 Poisson 表面重建^[1131] 转换为网格, 或通过 Marching Cubes^[1132] 转换为有符号距离函数 (Signed Distance Function, SDF)。但这需要额外数据, 例如点的观测视点; 将多次测量合并为点云地图时, 该信息可能丢失。因此, 通常需要对表面方向作出假设, 以区分物体内部和外部区域。



(a)



(b) 图 5.7 对 KITTI 数据集^[373] 第 07 序列中一组 LiDAR 扫描, 通过累积点云和 surfel 生成的地图进行定性比较。(a) 点云地图, 点的亮度表示 LiDAR 测量的反射率。(b) 基于圆盘的对应 surfel 地图。累积所有点云的完整地图占用 2.95 GB, 而 SuMa^[66] 的对应 surfel 地图仅占用 160 MB。

5.12 Surfel

点云地图直接表示测量, 但存储的点不包含表面信息, 也无法表示点是从哪个方向测得的。通过 surfel^[870], 可以向点添加方向信息来编码这些内容。Surfel 通常表示为圆盘或椭圆^[549, 66, 1184, 104, 103], 更一般地也可以表示为用高斯模型描述的椭球^[1041, 1042, 282, 1318]。Splatting^[1318, 103] 不仅可以整合纹理信息, 还能将重叠 surfel 混合为特定视点下连贯的渲染结果; 更详细的讨论见第 14 章。

一个常用的圆形 surfel (表示圆盘) 由位置 $\mathbf{p} \in \mathbb{R}^3$ 、法向方向 $\mathbf{n} \in \mathbb{R}^3$ 和半径 $r \in \mathbb{R}$ 定义。利用现代图形处理器 (GPU) 的能力可以高效渲染这些几何基元, 从而加速点到 surfel 的关联并显著减少内存占用。

图 5.7 对比了基于点云和基于 surfel 的地图表示。稠密点云能够以很高的细节准确表示环境, 但代价是占用大量内存。相反, 将多次测量聚合为单个 surfel 会损失细节, 但能在保留大尺度表面主要结构的同时显著降低内存使用量。

与使用 surfel (即小型圆形表面片元) 显式表示表面几何密切相关的是正态分布变换 (normal distributions transform, NDT)^[82, 1035]。NDT 将空间划分为体素, 并用正态分布 $\mathcal{N}(\mu, \Sigma)$ 近似体素内的点, 其中均值 μ 和协方差 Σ 由所含点估计。协方差的特征值 $\lambda_1 < \lambda_2 < \lambda_3$ 及其对应特征向量 $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ 可用于估计体素内的表面属性。对于平面表面 ($\lambda_1 \ll \lambda_2$), 最小特征值 λ_1 对应的特征向量 $\mathbf{v}_1$ 即表面法向。因此, 对平面表面而言, NDT 表示一个 surfel, 同时也能更准确地表示无法用单个 surfel 近似的点分布。从这个意义上说, NDT 是一种混合表示: 空间被划分为体素栅格, 因此具有显式性; 而体素由正态分布表示, 能够连续表示体素内部空间, 因此又具有隐式性。

5.13 网格

点云和 surfel 地图虽然描述了局部表面属性, 但由于没有建模表面连通性, 仍然相对稀疏。为了获得更完整的表示, 可以使用网格, 将表面描述为一组相互连接形成多边形集合的点。这样就能表示水密表面、查询并插值新的表面点, 并沿相连表面高效遍历。

在网格术语中, 每个多边形称为一个面, 每个角点称为一个顶点。最常见的是三角形网格 (每个面由三个顶点围成) 和四边形网格 (每个面由四个顶点围成)。注意, 任意多边形 (面) 都可以拆分为等价的三角形集合; 因此, 三角形网格不仅最简单, 也是最通用的网格类型。

网格是一种非常灵活且节省内存的表示, 因为面和顶点的数量可以直接适应表面复杂度及所需细节层次。例如, 任意大小的平面只需两个三角面和四个顶点即可表示。此外, 网格适合并行处理和渲染。网格常用于与计算机图形学交叉的应用, 如渲染、表面分析、操作和形变, 也广泛用于数字模型、仿真或基于表面的算法, 例如地面机器人的路径规划。

5.14 体素

点云和网格适合表示沿表面定义的环境属性。然而，占据状态和有符号距离等估计量是在整个体积内定义的。一种直接存储和处理体积属性的方法，是在规则栅格上对其进行离散化。离散化的占据地图和有符号距离地图分别称为占据栅格和有符号距离场。

类比二维像素，3D 栅格中的单元称为体素。由于栅格结构规则，每个体素都可以分配唯一索引并存入数据结构。需要注意，体素只是一个容器，更形式地说是一个空间划分；其内容的含义因方法而异。在经典占据栅格中，体素值表示该体素内任意点被占据的可能性，因此地图任意点的占据值等于包含该点的体素值。但体素值不必表示一个常量小立方体。例如，有符号距离场中的体素值估计体素中心处的有符号距离；要获得任意点处的距离，需要查询该点邻近体素并通过插值得到其值。一些应用甚至使用（稀疏）体素栅格存储和高效查询非体积属性，例如点或表面颜色。

5.15 连续函数

函数是连续建图的关键抽象。在这种情况下，建图问题归结为拟合参数化或非参数化函数，即求解回归问题。上面提到的许多空间抽象都要求预先离散化环境，通常使地图各处空间分辨率固定；而连续函数（无论参数化还是非参数化）提供更大灵活性，可以重新计算分辨率，并通过插值填补数据缺口。

某些参数化函数，例如二维无穷直线^[1106]和三维平面^[533, 374, 1106]，需要对环境作出严格假设，从而限制场景表示。但它们在内存和计算复杂度方面高效。基于控制点的函数（如 B-spline^[932]）或非参数化函数（如基于 GP 的函数）能够在更少假设下连续建模环境。从占据状态^[816]、隐式表面^[1186, 636]、距离场^[1195]到表面本身^[1122]，尽管计算复杂度较高，基于 GP 的表示因其概率性质而成为表示环境的常用选择；概率性质可以对观测区域和未观测区域进行不确定性量化与推断^[378]。

连续函数用于建图的一个关键优势是：如果选取至少一次可微的函数，就能提供梯度。梯度信息对于定位（计算表面法向）^[1196]、闭环检测（计算地形特征）^[377]、数据融合^[636]和规划^[1196]都很重要。

5.16 转换

上述抽象并不总是单独使用；它们经常相互转换，或以多种形式同时使用。

例如，可以将点和网格等显式几何转换为隐式表面。一种灵活的距离计算方法是最近点查询：可以针对任意显式几何执行，并且只在需要的时间和位置按需计算。另一种方法是在规则栅格的所有点上使用波前传播计算有符号距离，这可以通过 Fast Marching 方法高效实现^[991]。

反过来，将隐式表面转换为网格表示也很常见。将距离场转换为网格的原始技术称为 Marching Cubes^[696]。该算法将隐式表面划分到固定大小的立方体栅格中，并独立处理每个立方体。每个立方体根据八个角点处的隐式表面值生成一组三角形元素（图 5.8），再通过线性插值细化顶点位置。包括 BIM（Building Information Modeling）和 CAD（Computer-aided Design）模型在内的网格，又可以采样生成点或 surfel。

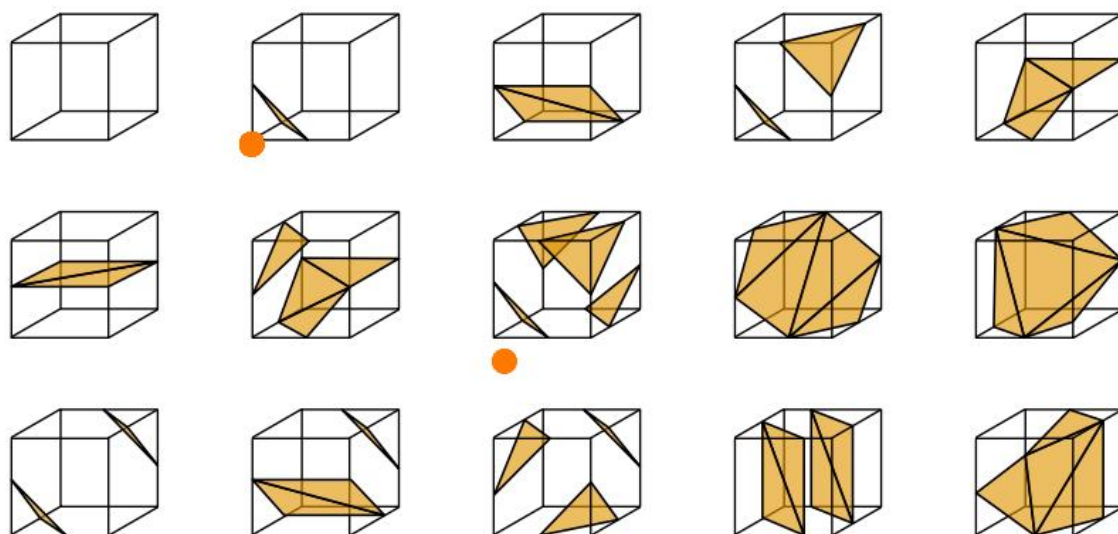


图 5.7: 图 5.8 Marching Cubes 算法将立方体投影到隐式表面上, 查询立方体顶点处函数值的符号, 并查表确定这些值对应的 15 种配置之一。(图片来源: Ryoshoru, “Marching cubes”, CC BY 4.0 许可, 来源: <https://commons.wikimedia.org/wiki/File:MarchingCubesEdit.svg>)

有时还需要将离散表示转换为连续表示。这通常通过对连续表示的参数求解优化问题来实现, 使相对于离散数据的拟合误差最小。

5.17 数据结构与存储

前几节介绍的抽象都需要存储在内存中。本节通过考察数据结构的选择及其优缺点, 讨论各种抽象如何存储。

5.18 朴素数据存储

对于许多表示, 简单的动态可调整大小数组是合理的起点。对于具有预定义空间划分的数据, 需要确定要存储的数据类型, 以及将空间坐标转换为空间索引坐标的函数。这常用于构建表示占据值或有符号距离值的地图。对于不规则数据, 只需确定存储的数据类型, 例如点云或 surfel。图 5.9 展示了使用映射函数存储有序数据以及存储无序点云的朴素方法。

这种朴素方法的优点是简单且支持快速随机访问。代价是表示可能需要大量内存。此外, 读写和修改操作虽然很快, 但改变表示的空间维度可能代价高昂, 因为需要复制整个数据结构的内容。

5.19 哈希表

解决上述朴素存储局限的一种自然扩展是使用哈希表。该方法将地图划分为多个分片, 对每个分片的坐标应用哈希函数得到单个值, 并将分片数据存入以该哈希值为索引的表中。分片通常表示具有明确坐标的地图子区域, 例如规则栅格中的立方体; 这些立方体可以对应单

Dense Map Representations

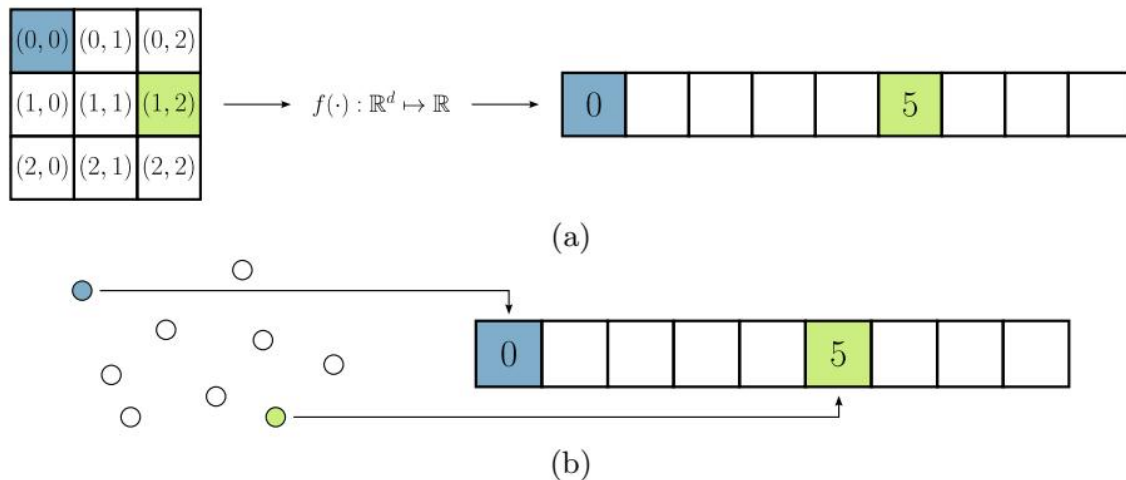


图 5.8: 图 5.9 (a) 通过函数 $f(\cdot)$ 将逻辑栅格坐标映射到朴素数组存储, 函数将坐标唯一映射为线性索引。(b) 将无序数据直接存入数组结构。

个体素或固定大小的体素组 (称为体素块), 也可以存储各自子区域中的点、surfel 或网格片段。

哈希表保留固定数组的 $O(1)$ 查找时间, 同时允许地图动态增长而无需重新分配。使用哈希表存储稠密地图时需要考虑三点:

1. **分片粒度**: 较小分片只在需要的位置分配数据, 可以提高稀疏性。但分片数量不应过多, 否则会降低哈希表的插入性能和内存效率。当哈希表存储仅沿表面存在的属性时, 这一权衡尤为重要。
2. **哈希函数**: 理想的哈希函数应在表中均匀分布键, 即使数据在空间上相邻 (建图中经常如此)。
3. **冲突处理**: 处理哈希冲突的方法 (例如链式法, 即每个条目包含链表, 或开放寻址) 会显著影响哈希表性能。

在大多数情况下, 哈希表在快速访问与高效插入、删除之间取得了良好平衡。但针对特定应用, 通常需要初始调参才能获得良好性能。

5.20 基于树的数据结构

另一种只为环境中相关部分分配内存、同时高效存储空间数据的方式, 是使用分层树结构表示。与哈希表类似, 树通常支持高效访问和插入; 其独特优势在于分层结构, 可高效存储多分辨率数据, 并加速最近邻搜索等空间操作。最常见的树变体包括 kD-tree^[72]、包围体层次结构 (bounding volume hierarchy, BVH)^[223] 和八叉树^[752]。

其中, 八叉树支持高效邻域搜索和增量整合新测量。八叉树中的每个节点由一个称为八分体的子空间表示。八分体由中心 $\mathbf{c} \in \mathbb{R}^3$ 和边长 $e \in \mathbb{R}$ 定义, 对应一个轴对齐包围盒。每个八分体最多有 8 个子八分体, 其边长为 $\frac{1}{2}e$, 如图 5.10 所示。

通常只在叶八分体 (即没有子节点的八分体) 中存储点, 并通过遍历树结构确定内部八分体中的点子集。

构建八叉树时, 在包含点云 P 的轴对齐包围盒内迭代地将空间划分为八分体。每次划分都将 P 分成子集 $P_1, \dots, P_8$, 对应范围为 $\frac{1}{2}e$ 的 8 个子八分体。非空子集 P_i 形成中心为 $\mathbf{c}$ 、

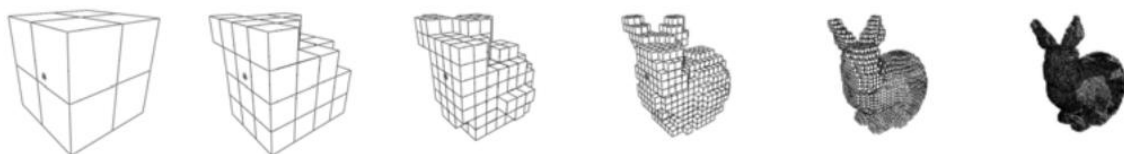


图 5.9: 图 5.10 不同树层级的八叉树及其八分体示例。八叉树的每一层都将空间细分为更细粒度的八分体。注意，八叉树的更深层级只表示被占据的空间。

范围为 $\frac{1}{2}e$ 的子八分体；当达到指定八分体大小或最小点数时停止。构建完成后，通过遍历树结构并按需添加内部节点，可以高效执行更新和插入。当新数据落在根八分体之外时，可以创建新的根节点，并将新数据和旧根节点分配给其子节点，从而扩展树。

与体素栅格相比，八叉树只表示包含数据的子空间，因此能够高效存储被占据空间。但这种内存效率需要通过树遍历访问特定叶八分体，可能增加定位一个点所需的时间；同时必须显式表示树结构，带来额外内存开销。近期一些方法针对这些内存开销进行了改进 [305, 820, 70]。

5.21 混合数据结构

为平衡内存需求与数据访问运行时间，一些数据结构以特定方式结合不同数据结构的优点，形成混合表示。在这种权衡中，我们接受较低的内存使用效率，以获得更高效的内存访问。

例如，哈希体素栅格 [808] 将环境划分为固定大小的稠密块（如 $8 \times 8 \times 8$ 个体素），再把这些块存储在哈希表中，从而结合稠密体素栅格和哈希表的优点。借助哈希表的灵活性，只需为包含有意义信息（如靠近表面的位置）分配块；同时，在每个块内使用普通 3D 数组存储体素，使操作简单、高效，并适合 GPU 加速。

另一种选择是将哈希表与树结合。类似于哈希体素栅格，VDB³ 数据结构 [792, 791] 将空间划分为哈希块，但在每个块内存储分层树。该结构提供层次树的全部优点，包括多分辨率表示和高效最近邻查询。然而，由于每个块大小固定，无论环境规模如何，树的最大高度都保持不变。因此，查找和插入可以在常数时间内完成，并显著快于纯树结构。

5.22 构建地图：方法与实践

节	空间抽象类型	表示的地图实体
5.4.1	点	表面
5.4.2	Surfel	表面
5.4.3	网格	表面（相连）
5.4.4	体素	占据状态或隐式表面
5.4.5–5.4.6	连续函数	占据状态或隐式表面

表 5.1 所介绍建图方法的总结。

到目前为止，我们讨论了需要估计的目标量以及建图可用的各种空间抽象。本节将详细考察用于构建这些地图元素的方法。方法按其主要空间抽象分类，如表 5.1 所示。注意，一些方法会使用额外的空间抽象来提高性能，例如将点分组到体素中，以实现更高效的存储和更快的查询。

5.23 点

如第 5.3.2.1 节所述, 简单累积测得点云来存储点无法扩展到大规模环境, 还会造成测量冗余。因此, 大多数方法^[1269, 256, 1133]将基于点的表示与体素栅格或八叉树结合, 用于表示稠密地图。此外, 数据结构的选择由高效最近邻搜索的需求驱动; 最近邻搜索对于通过迭代最近点 (iterative closest point, ICP) 进行扫描配准至关重要, 因为点对应关系需要迭代建立。

为了处理大规模环境, 某些方法 (例如著名的 LiDAR SLAM 方法 LOAM^[1269]) 会滤波原始点云, 提取角点和平面点, 从而显著缩小点云规模。它们使用体素栅格只存储地图表示中的点子集, 以剪枝冗余测量。基于 LOAM 中基于点的体素化, 后续方法^[998, 1150, 839, 667, 894, 999]分别改进点提取^[998, 1150, 839]、优化流程^[839, 667], 或整合 IMU 信息^[894, 999]。

另一类方法采用不同方式处理点云数据量, 以避免依赖能力很强的特征提取方法。通过体素栅格^[256]对点云进行规则采样, 可以显著减少每次 LiDAR 扫描的点数并去除潜在冗余信息。关键在于, 体素栅格中的点并不聚合和平均, 而是保留原始测量。沿着这一思路, Dellenbach 等人^[256]和 Vizzo 等人^[1133]使用该策略对输入点云下采样, 在体素栅格地图中仅存储每个体素内受限数量的点。

总的来说, 如第 5.3.2.1 节所述, (哈希) 体素栅格具有双重作用: 通过每个体素存储有限数量的点测量来抽象空间, 并通过直接索引相邻体素加速最近邻搜索。

5.24 Surfel

Surfel 可以采用与点云类似的策略。值得注意的是, Stückler 和 Behnke^[1041]以及 Droschel 和 Behnke^[282]的后续工作使用八叉树, 在八叉树层级的多个层次表示 surfel, 以进行数据关联。所谓多分辨率 surfel 地图通过累积的均值和协方差统计量间接表示 surfel, 类似 NDT。

相比之下, Whelan 等人^[1184]将 surfel 存储为简单列表, 并利用高效渲染技术生成投影, 用于室内 RGB-D SLAM 的数据关联。在这种情况下, surfel 是显式几何基元, 因此需要直接处理以相应更新 surfel 属性 (即大小和方向)^[549]。Whelan 等人^[1184]的一项关键贡献是利用地图形变直接形变 surfel, 而不是依赖 PGO; 检测到闭环时, 可以利用 surfel 表示的测量来形变地图。Park 等人^[846]也采用了类似的 surfel 地图形变策略。

同样, Behley 和 Stachniss^[66]面向室外环境, 需要用多个空间范围为 $100\text{ m} \times 100\text{ m}$ 的子地图表示 surfel, 并可将其从 GPU 内存换出。与 ElasticFusion^[1184]不同, 该方法依赖 PGO, 但利用 surfel 可以自由定位这一点, 将 surfel 绑定到位姿, 从而可以用位姿图优化后的位姿直接形变地图; 其他方法^[1158]也采用了这一策略。

5.25 网格

如前文所述, 网格以富有表现力且灵活的方式表示相连表面。网格生成方法可分为两类。第一类直接将测得的点转换为网格; 第二类将问题分为两步: 重建隐式表面, 然后提取等值面得到最终网格 (见第 5.3.2.6 节)。

第一类方法通常直接计算输入点集的 Delaunay 三角剖分, 并识别位于表面上的 Delaunay 三角形子集;^[157]给出了详细综述。直接由点构建时, 网格会隐式适应采样密度。这既可能是优点 (提供自适应分辨率), 也意味着方法对采样不规则和孔洞更加敏感。实践中, 当整个表面可以由非常精确的深度传感器进行稠密采样时 (例如使用测量设备), 通常选择直接网格化方法。

第二类方法使用隐式表面作为中间步骤, 在提取最终表面网格前简化数据融合和滤波。根据数据生成隐式表面的一种直观方法, 是在规则栅格的每个点估计到表面的距离。如第 5.2.2 节所述, 还必须根据每个点位于物体内部还是外部来设置隐式表面的符号。这一信息通常依据估计的表面法向确定, 例如在小邻域上应用主成分分析 (Principal Component Analysis, PCA)。然而, 正如^[476]所指出的, 这类方法可能产生不连续的隐式表面。为解决该问题, Carr 等人^[155]用一组径向基函数 (Radial Basis Functions, RBF) 对隐式表面建模, 并通过求解全局优化问题拟合输入点。所得隐式表面在构造上是平滑的, 并根据全局上下文可

靠填补孔洞；但求解底层的大规模稠密优化问题计算代价很高。Shen 等人^[1002]使用移动最小二乘（moving least squares, MLS）对输入点进行局部近似，克服了这一限制。进一步地，Poisson 表面重建^[545]通过求解偏微分方程（partial differential equation, PDE）将隐式函数拟合到测量点的法向，得到稀疏且计算可处理的优化问题，对噪声尤其鲁棒。

在机器人应用中，通常希望从实时传感器流构建网格。通过增量更新，可以使表面重建达到实时效率。基于 TSDF 的表面重建因其天然的增量特性和总体简单性而广受实践采用。该方法属于第二类方法，通过平均投影距离估计隐式表面。由于更新 TSDF（或一般隐式表面）的代价远高于网格提取，实时方法主要集中于优化前者。

5.26 体素

基于体素的方法是 3D 重建和机器人学中最常用的体积表示之一。本节不按时间顺序罗列大量文献，而是聚焦实践中常见的概念，并依据三个基本决策标准组织内容：选择的估计量、数据结构和可扩展性。

5.27 按估计量分类的方法

基于体素的建图框架首先要选择估计量，最常见的选项是占据状态（见第 5.2.1 节）或距离度量（见第 5.2.2 节）；第 5.2.3 节的讨论可用于在两者之间作出选择。

自声呐的连续概率占据测量模型^[774]提出后，人们开发了简化的分段常数模型来降低计算成本^[478]。这一转变受到 LiDAR 技术出现以及从 2D 地图向 3D 地图发展的研究兴趣推动。最近，Loop 等人^[692]提出连续概率模型：该模型不膨胀物体，而是使物体表面处的占据概率收敛到 0.5。占据估计由于召回率高而常用于碰撞规避，但与基于距离的方法相比，其不连续性和缺乏信息的梯度限制了性能（见第 5.2.3 节）。

对于距离度量，不仅要估计距离场的正值部分，还要在表面后方外推负距离，因为表面由有符号距离场的零交叉表示。为限制融合不完美正、负距离估计对精度的影响（见第 5.2.2 节），更新通常被限制在表面边界附近的小截断带内。然而，基于距离的方法仍容易擦除几何形状。例如，从相反两侧观测细薄物体时，对观测到的正距离和虚构的负距离求平均，会使零交叉翻转或消失。一些工作分析了截断带和权重衰减对最终重建质量的影响^[134]；从根本上说，该问题可以减轻但无法消除。总体而言，在平滑表面上，TSDF 估计的表面优于占据方法，但对细薄物体的召回率较低。

TSDF 提供的距离信息本身非常宝贵。然而，TSDF 并不保守，而是严格高估欧氏距离。为解决这一安全问题，voxblox^[822]推广了增量构建 ESDF 的方法：它将传感器数据融合到 TSDF 中，然后使用 brushfire 算法^[629]更新 ESDF。随后，FIESTA^[426]提出一种混合方法，直接从占据地图增量更新 ESDF。

5.28 按数据结构分类的方法

体积建图最简单的数据结构是静态 3D 数组。KinectFusion^[801]表明，这种结构对于小型、固定大小的场景能够取得良好结果。但许多应用要求地图在运行时动态扩展，并且只在需要的位置分配体素以节省内存。

为此，Nießner 等人^[808]提出体素块哈希方案，将体素分组为块（例如 $8 \times 8 \times 8$ 个体素），并将块存入哈希表。该结构很快被 TSDF 采用，提供常数时间（ $O(1)$ ）查找和动态插入；当然也可用于存储占据概率，如 FIESTA^[426]所示。与单独哈希每个体素相比，将体素分组为块可以在哈希表大小和体素分配粒度之间提供可调权衡。

体素也可以使用树结构存储。Octomap^[478]首先推广了使用八叉树存储占据概率，多年来一直是体积建图的事实标准。树结构的主要优点是天然支持多分辨率；主要限制是编码树结构会产生显著内存开销，且单元查找时间与树高成正比。近期方法通过混合数据结构缓解这一限制。例如，Supereight^[1125]在前几层使用标准（动态）八叉树，在最后几层使用静态八叉树。静态八叉树可视作使用固定大小数组存储的八叉树，这消除了用指针编码父子关系的内存开销，但由于静态八叉树按块分配，粒度有所降低。VDB^[792]数据结构最初为视觉特效（VFX）行业提出，后来被多个体积建图框架采用^[721, 1132]。如第 5.3.3.4 节所述，它将块哈希与树结合，兼具两者优点：良好的内存效率、类似哈希表的常数时间查找和插入，以及类似树的多分辨率。

实际应用中,下游任务往往还要求在占据概率或 TSDF 旁存储颜色、语义^[398, 942]或 ESDF^[822, 426]等附加信息。尽管几乎所有数据结构都可以扩展以支持附加通道,但实现工作量会随底层数据结构的复杂度增加。这进一步说明应优先使用简单数据结构(如像素块哈希)或灵活的第三方库。

5.29 按测量融合算法分类的方法

根据深度测量更新地图的算法称为测量融合器。它通过应用测量模型,更新每个观测体素的估计量。融合测量主要有两类方法:射线追踪和基于投影的方法。

对于每个测得的点,射线追踪融合器从传感器向该点发射射线,并更新射线穿过的所有体素。这种方法非常通用,只需知道传感器原点的位置。然而,一个体素可能被多条射线击中,尤其是靠近传感器的体素,这会造成重复计算;在并行实现中处理由此产生的竞争条件还会带来实现和性能开销。

相反,基于投影的方法直接遍历观测体素,并将每个体素投影到传感器坐标系,查询用于计算更新的射线。遍历地图而不是射线天然避免了竞争条件,因此基于投影的方法广泛用于多线程和 GPU 加速的体积建图框架。直接遍历地图产生的可预测访问模式还能减少内存瓶颈。但该方法的主要缺点是必须明确知道传感器的完整位姿和投影模型;对于无组织点云(包括 LiDAR 运动去畸变后的点云),该方法也更难使用。

5.30 按可扩展性分类的方法

内存和计算成本是体积建图的两大瓶颈。对于固定分辨率方法,内存和计算复杂度随地图总体积线性增长,并随所选分辨率的倒数立方增长。降低这些复杂度是重要研究方向,因为只有这样才能构建超越小型受限体积的高细节地图。

体积建图早期工作主要关注降低内存使用。例如,Octomap 使用八叉树内部节点存储子节点占据值的最大值或平均值。通过递归剪枝估计量接近父节点的叶节点,地图中的均匀区域会自动用更少、更低分辨率的节点表示。这种适应环境几何的策略在实践中非常有效,因为环境通常主要由自由空间组成。此外,在八叉树内部节点存储最小、最大或平均值对下游任务也有价值,可支持低分辨率地图查询,并使用层次算法执行快速碰撞检查或探索规划。但 Octomap 的核心限制是所有测量都以最高分辨率融合,因此其计算复杂度仍呈立方增长。

多分辨率还可用于降低测量更新的计算成本。由于测量射线以固定角度发射,击中远处几何体的射线更少,因此随着距离增加降低更新分辨率是合理的。可以通过多分辨率射线追踪^[286]或多分辨率投影融合^[1125]实现。Supereight2^[347]进一步根据测量更新的熵调节更新分辨率,从而降低计算复杂度。这些方法显著提高了更新性能,但不同分辨率层仍需显式同步。消除同步需求的一种方法是不在每个八叉树节点存储绝对值,而只编码各分辨率层之间的差异;形式上可通过小波分解实现。小波编码地图可以在任意时刻以任意分辨率高效查询。利用这一性质,wavemap^[923]以从粗到细的方式更新地图,进一步降低计算复杂度。除根据测量熵调整更新分辨率外,它还跳过无信息更新,例如地图某区域的占据状态已经收敛为空闲且所有测量一致时。

5.31 高斯过程

如第 5.3.2.5 节所述,将建图问题表述为回归问题有利于获得连续表示。此外,若希望尽量减少对环境作出的假设,采用非参数方法求解非线性回归问题是理想选择。高斯过程(Gaussian process, GP)^[909]是一种随机、非参数、非线性回归方法。给定其他位置处噪声且稀疏的测量,它可以估计任意查询点处未知函数的值。我们已在第 2.2 节介绍如何使用 GP 表示连续时间轨迹。显然,GP 也是连续量建图的有吸引力方案,并已广泛用于机器人学文献中,以利用深度传感器建模连续空间现象^[1122, 816, 377, 574]。

GP 模型中的信息由均值 $\mathbf{m}(\mathbf{x})$ 和核函数 $\kappa(\mathbf{x}, \mathbf{x}')$ 给出,对所估计的连续量建模为:

$$f(\mathbf{x}) \sim \mathcal{GP}(\mathbf{m}(\mathbf{x}), \kappa(\mathbf{x}, \mathbf{x}')). \quad (5.8)$$

设 $\mathbf{X} = \{\mathbf{x}_j \in \mathbb{R}^D\}$ 是具有测量值 $\mathbf{y}$ 的位置集合,在位置 $\mathbf{x}_j$ 处测得的估计量为 $y_j = f(\mathbf{x}_j) + \epsilon_j$ 。对于 J 个训练对 $(\mathbf{x}_j, y_j)$,假设噪声 ϵ_j 独立同分布,且满足高斯分布 $\epsilon_j \sim \mathcal{N}(\mathbf{0}, \sigma_j^2)$ 。给定测试位置集合

$\mathbf{X}^* = \{\mathbf{x}_n^* \in \mathbb{R}^D \mid n = 0, \dots, N\}$, 函数值与观测目标值的联合分布可写为:

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_* \end{bmatrix} = \mathcal{N} \left(\mathbf{1}, \begin{bmatrix} \mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_f^2 \mathbf{I} & \mathbf{K}(\mathbf{X}, \mathbf{X}^*) \\ \mathbf{K}(\mathbf{X}^*, \mathbf{X}) & \mathbf{K}(\mathbf{X}^*, \mathbf{X}^*) \end{bmatrix} \right), \quad (5.9)$$

其中, $\mathbf{K} = [\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)]_{ij}$ 。因此, 条件分布 $(f_* \mid \mathbf{X}, \mathbf{y}, \mathbf{X}^*) \sim \mathcal{N}(\bar{\mathbf{f}}_*, \text{cov}(\mathbf{f}_*))$, 其均值为:

$$\bar{\mathbf{f}}_* = \mathbf{K}(\mathbf{X}^*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_f^2 \mathbf{I}]^{-1} \mathbf{y}, \quad (5.10)$$

协方差为:

$$\text{cov}(\mathbf{f}_*) = \mathbf{K}(\mathbf{X}^*, \mathbf{X}^*) - \mathbf{K}(\mathbf{X}^*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_f^2 \mathbf{I}]^{-1} \mathbf{K}(\mathbf{X}, \mathbf{X}^*). \quad (5.11)$$

式 (5.10) 和式 (5.11) 是所估计量的预测方程。GP 对表示具有空间相关性的数据尤其有效, 因此能够克服环境建图中占据栅格方法所特有的单元独立性假设。高斯过程占据地图 (Gaussian Process Occupancy Map, GPOM) [816] 将传感器观测及其对应标签 (自由或被占据) 收集为训练数据; 地图单元构成测试位置, 并如式 (5.9) 所示与训练数据相关。利用式 (5.10) 和式 (5.11) 完成回归后, 通过二元分类函数将回归输出“压缩”为单元的占据概率。

在原始形式中, GPOM 是一种批处理建图技术, 计算复杂度为 $\mathcal{O}(J^3 + J^2 N)$ 。为应对这一计算复杂度, 尤其是实现增量 GP 建图, 后续提出了多种方法, 例如 [574, 575, 1157, 378]。

基于 GP 函数建图的一个关键优点是, 可以对估计量进行线性运算 [969], 结果仍为 GP。由于求导及梯度都是线性运算, 估计量的微分输出具有概率性。环境不确定性的连续表示可用于突出未探索区域, 并优化机器人的搜索规划 [378, 677]。GP 地图的连续性也能提高规划器的灵活性, 使其可以直接基于已收集的传感器数据推断, 而不受栅格或体素单元分辨率限制。

5.32 高斯过程隐式表面

隐式表面也可以由 GP 表示。高斯过程隐式表面 (Gaussian process implicit surface, GPIS) 技术 [1186, 736, 677, 505] 利用 GP 方法, 根据噪声测量估计隐式表面的概率连续表示。此外, GPIS 也可连续地估计表面和距离场 [576, 1034, 636, 1195]。

在 GPIS 表述中, 设要估计的距离场为 d 。给定表面上的点、这些点到最近表面的距离 d_i , 以及通过线性算子计算得到的相应梯度 ∇d [969], 由于表面上的距离为零, d 与 ∇d 可以建模为零均值的联合 GP:

$$\begin{bmatrix} d \\ \nabla d \end{bmatrix} \sim \mathcal{GP}(\mathbf{0}, \mathbf{K}(\mathbf{X}, \mathbf{X}')). \quad (5.12)$$

GPIS 方法能够估计连续隐式表面及其由梯度得到的法向, 并同时给出不确定性。一些工作考虑使用参数函数先验以更准确地捕捉给定形状 [736, 505]; 另一些方法不仅估计隐式表面, 还估计完整距离场。就标准 GPIS 表述而言, 距离在测量附近 (即表面上) 拟合良好, 但在远离表面处会回归到均值, 此处均值为零。为了在远离表面的位置以连续且概率的形式估计完整距离场, 一些工作将非线性操作应用于类似 GPIS 的表述 [1195, 1196, 634]。

所有这些方法都必须处理基于 GP 表述的计算复杂度; 作为交换, 仅凭点云就能获得环境的连续、生成式和概率表示。

5.33 Hilbert map

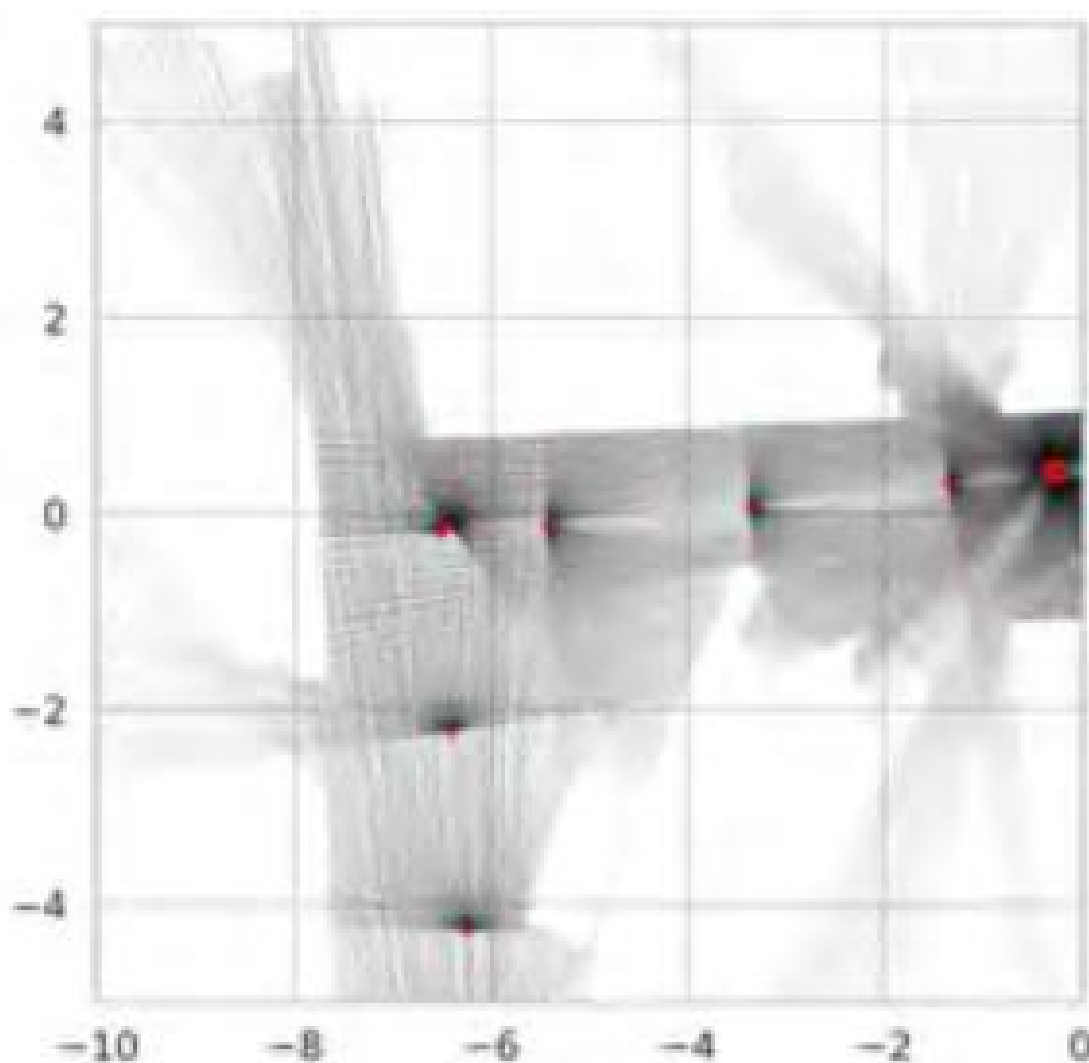
Hilbert map (HM) [904] 在许多方面与 GPOM [816] 相似。二者都是连续概率模型, 与基于体素的方法不同, 不对空间离散化; 与基于点的方法相比, 能够插值缺失数据。如前所述, GPOM 的主要挑战是计算开销高。因此, Hilbert map 的设计目标是: (i) 以在线方式连续处理数据; (ii) 建模观测之间的依赖; (iii) 纳入测量不确定性。

为实现这些目标，通常在投影特征空间中使用随机梯度下降训练逻辑回归器。分类器与优化器的组合支持利用大量数据在线更新模型；特征投影则使简单分类器也能表示复杂的空间细节。

特征投影与 GP 中核函数的思想相同，但这里使用近似而不是完整协方差。有多种选择，包括 Nystroem^[1185]、随机 Fourier 特征 (Random Fourier Features)^[901] 和稀疏核 (Sparse Kernel)^[755]，这里采用稀疏核。稀疏核的目标是限制观测产生影响的范围，从而提高收敛速度和计算效率；得到的核函数会在特定距离处严格降为 0。

该核函数通过在待建图空间中按规则间隔放置诱导核，可以将二维或三维空间中的点投影到显著更高的维度。这样就能计算输入数据的高维特征空间表示，并使用小批量随机梯度下降训练逻辑回归分类器。最后，训练通过沿范围测量采样自由空间点，同时将回波加入为障碍物点来完成。

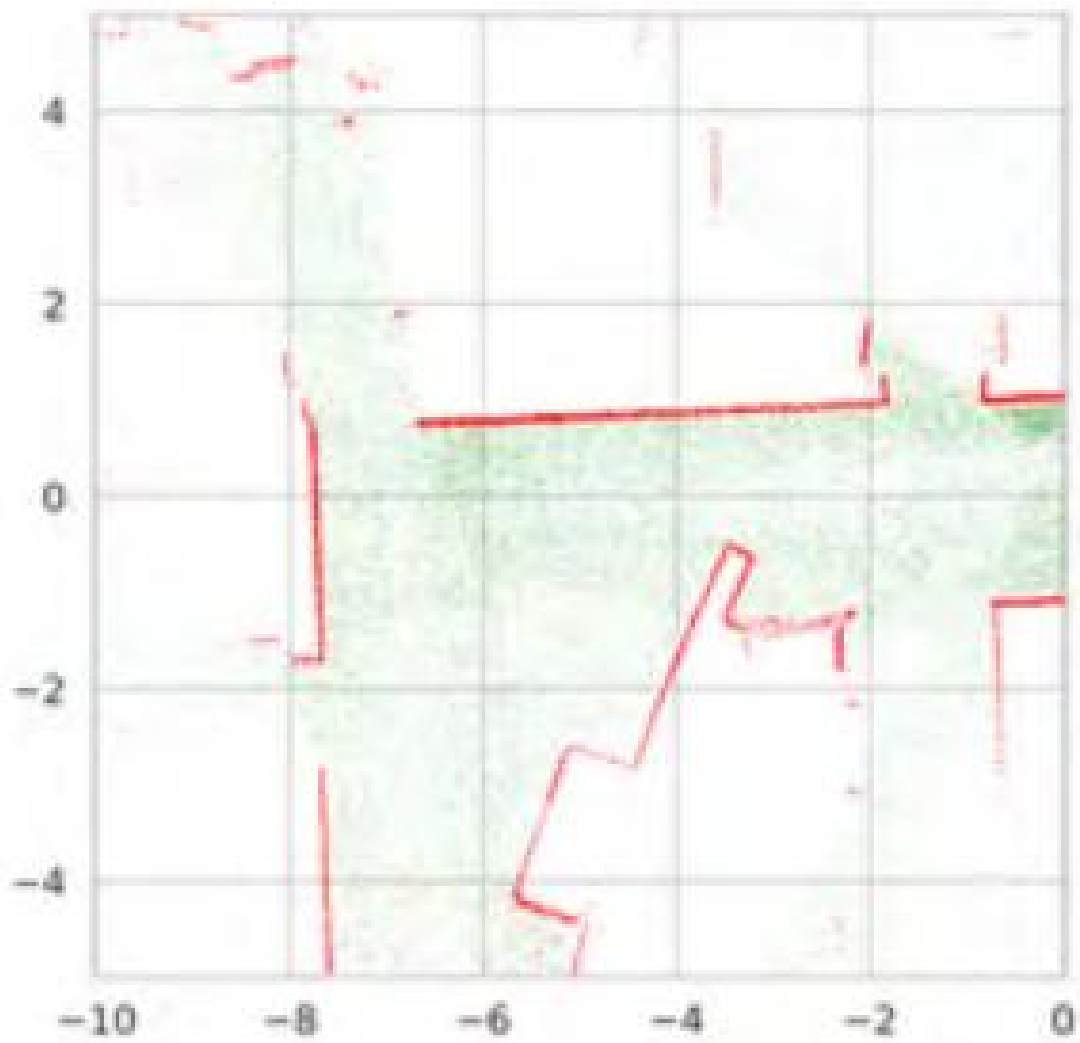
HM 面临的一个挑战是所用核的表达能力。图 5.11 所用的径向基函数 (radial basis function, RBF) 在二维中是圆、在三维中是球，需要组合多个函数值才能重建环境的复杂细节。因此，核的数量和长度尺度之间存在权衡，二者会影响计算成本、重建细节和插值能力。

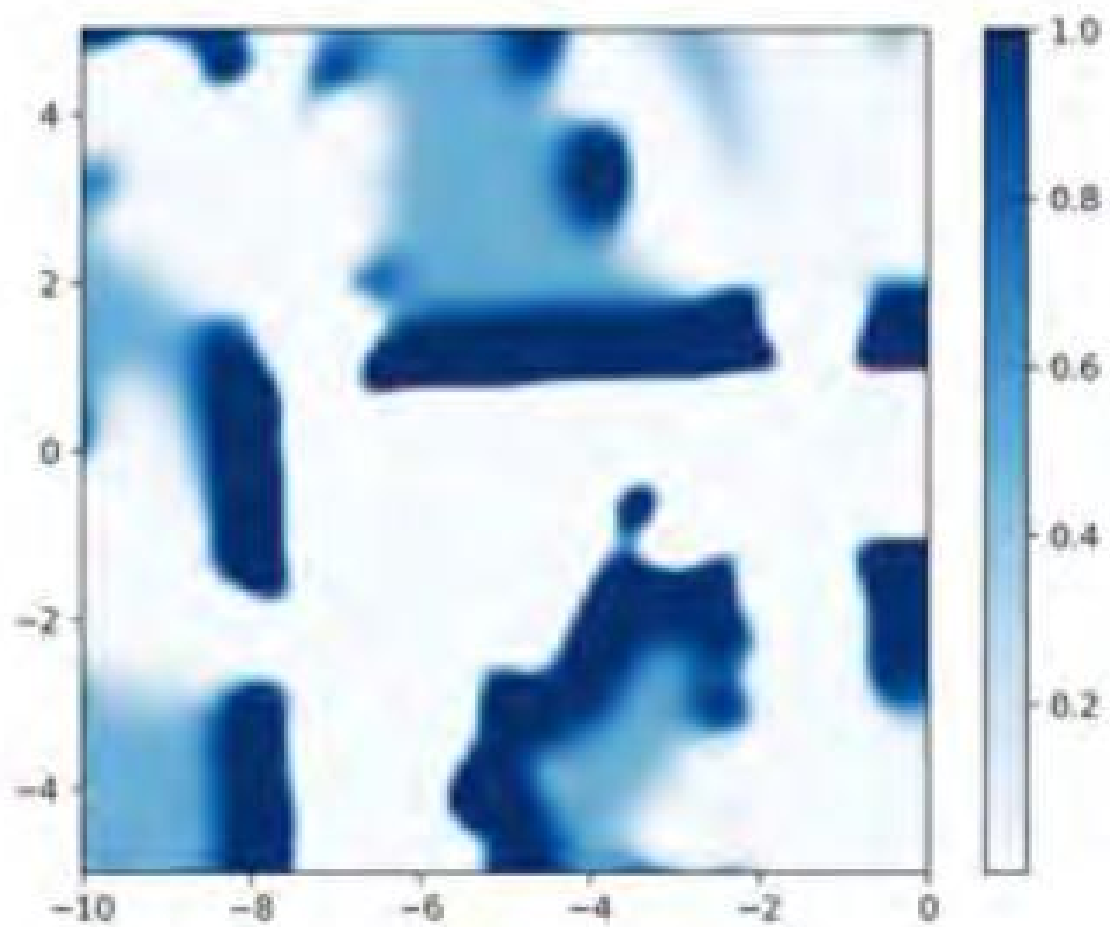


(a)

(b)

(c) 图 5.11 (a) 机器人使用二维 LiDAR 传感器获得的观测，随后转换为 (b) 自由 (绿色) 和被占据 (红





色) 点的数据集, 再用来训练 (c) 所示的 Hilbert map。

5.34 建图中的深度学习

近年来, 利用神经辐射场 (neural radiance fields, NeRF, 见第 14 章及 [764]) 进行新视图合成受到关注; 它通过简单的多层感知机 (multi-layer perceptron, MLP) 实现出色的图像生成结果。一些方法因此研究使用神经表示来估计 SDF。在任意空间位置学习预测 SDF, 可以得到连续表示, 既能以任意分辨率转换为网格, 也能凭借学习函数的插值能力生成更完整的表示。

与隐式表示相似, 这种表示从输入数据学习并近似为可在任意位置查询的连续函数。神经表示通常是在给定位姿下离线学习的; 近来, 人们也开始关注增量方法 [1047, 1293], 以及使用神经表示在线估计位姿的方法 [1047, 968]。

特别地, Sucar 等人 [1047] 的方法基于 RGB-D 帧, 利用神经网络预测场景中任意点的 SDF 值。后续方法将特征的空间表示 (如体素栅格 [862, 1309]、八叉树 [1293]、点 [968, 261] 等) 与神经表示分离。这些方法在空间表示中存储小而具有描述力的特征, 再利用小型神经网络确定场景中任意点的 SDF 值。这样就将学习函数与空间表示解耦, 既可以依靠小型神经解码器将特征转换为有符号距离值, 也能够有效应用于室外等大尺度场景。

基于深度学习的建图、重建和 SLAM 领域发展迅速, 并结合了经典表示中的思想, 例如 surfel splatting [548, 740], 在重建质量和效率方面获得显著成果。第 13 章将介绍 Dust3r 等前馈网络如何从单目视频生成稠密表示。特别是, 渲染新视图并在任意位置生成新数据的能力, 可能在不依赖仿真环境的情况下用于机器人学习。例如, NeRF 和 Gaussian splatting [1318, 103] 已广受关注, 并在多项 SLAM 相关工作中展示出显著潜力; 第 14 章将进一步讨论这些方法。

5.35 使用考量

所有地图表示都需要在各自鲜明且往往互补的优点与缺点之间权衡。因此, 为给定应用或机器人系统选择地图表示时, 必须仔细考虑该地图在所有下游任务中的使用方式。其他需要考虑的因素是运行环境和可用传感器。下面先讨论环境因素, 它们会促成一些明确的选择; 然后讨论更细致、依赖任务的考量; 最后就第 5.4 节所述现有方法给出简要使用建议。

5.36 环境因素

运行环境可以分为结构化和非结构化两类。在自动化工厂等受严格控制的空间中, 针对机器人任务及其将遇到的特定对象量身定制的地图表示, 在效率和精度方面通常优于通用稠密表示。相反, 本章讨论的稠密表示能够对任意形状的物体建模, 并可在任何环境中工作。在变化或部分未知的环境中, 机器人通常必须能够区分已观测自由空间和未观测空间。该信息使路径规划器能够避免穿越未观测空间的不安全运动, 因为这些空间可能被占据; 也可用于探索规划。包括点、surfel 和网格在内的显式表面表示, 通常无法区分自由空间和未观测空间, 而所有基于占据的方法都能做到。基于隐式表面的方法也能提供这种区分, 但在许多更专注于重建的应用中, 为节省计算和内存成本, 远离表面的此类信息经常被丢弃。

另一个考虑因素是可扩展性。显式表示通常比隐式表示更节省内存, 因为它们只描述表面本身, 并且可以轻易地使保真度适应场景各部分所需的细节。需要自由空间信息时, 与固定分辨率体素表示相比, 多分辨率方法可以在精度、内存以及捕捉极细物体的能力方面带来显著改进。

最后需要考虑环境中是否包含大量动态物体, 以及应在多大程度上对其建模。从地图表示角度看, 现有方法大多可归为三类。第一类不考虑动态性, 直接将所有测量融合进本章介绍的一种表示。实践中, 使用隐式表示时这可能已经足够, 因为其自由空间更新通常能很好地消除物体移动后留下的残余。第二类方法试图只将环境静态元素融合到地图中, 显式检测并丢弃所有对应动态物体的测量。这类方法在使用显式地图时尤其流行, 因为显式地图中的残余更难移除; 此类方法通常即使在高度动态的空间中也能生成干净的地图。最后一类方案不仅表示背景, 还表示场景中的运动元素。注意, 这通常通过混合表示完成, 将本章介绍的基本

几何表示与对象层级的专用表示结合。关于动态环境中的 SLAM（包括实现上述方法和更先进方法的具体技术）的详细讨论，请参阅第 15 章。

5.37 下游任务类型

除环境外，同样重要的是考虑机器人所需任务需要何种地图信息。任意给定操作通常都能在各种表示上执行，但效率和实现复杂度往往差异很大。最大的差别在于操作沿表面执行，还是在笛卡尔空间中执行。如表 5.2 所示，隐式表示通常可以简单高效地滤波以笛卡尔坐标表达的属性，例如占据状态；相比之下，显式表示适合滤波沿表面表达的属性，例如视觉纹理。这解释了显式表示通常对深度测量质量更敏感，但能够创建非常细致、视觉效果出色的 3D 重建；而隐式方法适合融合 RGB-D 相机数据等噪声深度测量。

就查询而言，显式表示可以直接沿表面遍历，这解释了它们在渲染、图形应用和覆盖路径规划等任务中的流行。不过，要回答笛卡尔坐标中的查询，需要进行最近邻查找等额外步骤。隐式表示恰好相反，因此常用于碰撞检查任务。

操作	显式表示中高效	隐式表示中高效
滤波测量	沿表面（纹理等）	在笛卡尔空间中（占据状态等）
查询和遍历	在表面坐标中（覆盖规划等）	在笛卡尔坐标中（碰撞检查等）
修改表面	几何（形变等）	拓扑（合并、切割、简化等）

表 5.2 显式与隐式表面表示的互补优点和缺点。

最后，显式表示可以高效修改表面的几何形状，包括形变。实践中，地图通常根据不完美且存在漂移的里程计系统提供的位姿估计，通过融合深度测量构建。随着时间推移，累积位姿误差也会导致稠密地图不一致。与 SLAM 系统一样，检测到闭环时可以通过形变稠密地图来消除这些误差。显式和隐式表面都可以形变，但使用显式表示时该操作在本质上更简单、更高效。相反，使用隐式表示可以简化并提高影响表面拓扑或连通性的操作效率。因此，隐式表示常用于合并表面估计、组合或相减物体形状以及简化表面。

还应记住，不同表示可以配合使用，以发挥各自优势。一个很好的混合方法示例是基于 TSDF 的网格化（第 5.4.3 节）：先使用隐式表面表示（TSDF）方便地滤波噪声深度测量，再用 Marching Cubes 将其转换为显式表示（网格）。在判断混合表示的优势是否抵消其带来的开销时，值得考虑如何将转换限制为仅在局部、低频发生。

5.38 建图方法总结

最后总结本章介绍的现有方法的关键差异。先看显式表示：用点集合描述表面简单，所需假设最少，但信息量也最少。除无穷小点外，surfel 在小邻域或片元上表示表面属性。最后，网格显式表示表面连通性，并允许平滑插值其属性。然而，估计表面连通性需要最多假设，有时还伴随显著计算成本。

对隐式表示而言，隐式表面相对于占据地图的一项特殊优势是能提供快速、高质量的距离信息和梯度，有利于基于优化的规划。然而，滤波占据估计所需的假设更少；对于基于体素的方法，占据地图也更擅长捕捉细薄障碍物。在深度测量特别噪声大或稀疏时，基于 GP 和 Hilbert map 的非体素隐式表示能够提供特别好的不确定性估计。由于它们显式考虑几何形状的空间相关性，通常也更擅长插值部分观测到的表面。

学习型方法是一个快速发展的研究领域。对于基于学习的隐式表示，NeRF 已被证明能够支持很有前景的新能力，尤其是语义建模和空间推理。最近，Gaussian splatting^[552]（一种与 surfel 相似的显式表示）推动了人们对使用 splatting 的方法^[548, 740, 1261]的兴趣。研究人员正积极改善这些方法的计算和内存占用，测试其可支持的新能力，并探索如何将其整合到完整机器人系统中。展望未来，我们预计基于学习的方法将通过学习先验，提高稠密表示的通用性和表达能力，并改善其处理噪声测量、不完整观测和动态物体的能力。

5.39 致谢

作者感谢 Lan Wu 在本章准备过程中提供的支持。

第六章 SLAM 的可认证最优求解器与理论性质

作者：David M. Rosen、Kasra Khosoussi、Connor Holmes、Gamini Dissanayake、Timothy Barfoot、Luca Carlone

第 1–3 章讨论了如何利用最大后验估计，或更一般地利用 M-估计，将 SLAM 中出现的估计问题形式化为优化问题。此外，这些章节还介绍了迭代式局部求解器（例如 Gauss-Newton、Levenberg-Marquardt 和梯度下降），它们通过迭代细化给定初始猜测来寻找所得到优化问题的解。

本章将更深入地考察 SLAM 中出现的优化问题，并回答两个基本问题。第一，能否设计高效算法，在可能不需要初始猜测的情况下，保证计算出这些问题的全局最优解？第二，作为真值的估计，最优解有多准确，又有哪些因素影响其准确性？

第一个问题关乎计算与可靠性：SLAM 需要求解具有多个局部极小值的非凸优化问题。迭代算法可能因初始猜测的质量而陷入局部极小值，进而产生错误估计（图 6.1）。此外，迭代方法没有用于检测是否收敛到次优解的工具，这会在实际应用中引发可信性与可靠性问题。在第 6.1 节中，我们将说明：尽管典型 SLAM 问题具有非凸性，仍可设计可认证最优算法，将 SLAM 优化求解到可证明的最优性，并能区分次优解与最优解。这些算法基于一种称为半正定松弛的数学工具，第 6.1.1 节将对此进行回顾。本章将给出针对 PGO 的可认证算法（第 6.1.2 节的 SE-Sync）、针对带距离和方位测量的基于路标 SLAM 的可认证算法（第 6.1.3 节），随后讨论向其他 SLAM 问题的扩展，包括仅距离测量、各向异性噪声和离群值问题（第 6.1.4 节）。

第二个问题涉及基本极限与估计误差：设计 SLAM 系统时，人们通常希望机器人位姿估计和地图估计接近真值，因为较大误差可能导致包括运动规划在内的下游任务失败。理解基本极限不仅对分析有用，也具有十分实际的意义：如后文所述，这些极限受因子图结构影响，而我们可以通过谨慎驱动机器人（例如迫使它重访地点或路标）来主动控制该结构。因此，第 6.2 节讨论 SLAM 估计可达精度的信息论极限。具体而言，第 6.2.1 节引入 Cramer–Rao 下界，用于量化 SLAM 中的估计误差；第 6.2.2 节则建立该下界与 SLAM 问题图结构之间的联系。

我们将看到，对这些问题的研究揭示了 SLAM 问题所编码的代数、几何和图论结构，与求解该问题的计算与统计难度之间的深刻联系。照例，本章将在第 6.3 节以近期趋势与参考文献的展望作结。

6.1 SLAM 的可认证最优求解器

SLAM 通常被表述为高维非凸问题。由于存在大量局部极小值，求解一般非凸优化问题的全局解本身极具挑战（图 6.1）。特别地，SLAM 的许多特殊情形（例如角同步、旋转平均和 PGO）已知为 NP-困难问题。因此，除非 $P = NP$ ，不存在能够在一般情形下高效求解这些问题的算法^[938]。

尽管具有这种理论复杂性，早期 SLAM 研究却持续显示出向接近真值的解收敛这一令人惊讶的现象。SLAM 算法通常以由里程计计算得到的机器人位姿初始化。在经过良好标定的移动机器人中，里程计漂移可以低于行驶距离的百分之一。因此，SLAM 异乎寻常的可靠收敛最初被归因于高质量初始估计的可得性。

然而，后续研究表明，即使在初始化较差或不一致的情况下，包括随机梯度下降、Levenberg–Marquardt 和预条件共轭梯度法在内的一系列优化技术，也常能恢复近似最优解。这些发现说明，尽管 SLAM 是非线性非凸问题，它仍具有使其特别适合借助专用优化策略求解的内在结构。同时，这些结果也表明，即使底层

SLAM 实例是良定的，简单套用现成局部优化方法（例如梯度下降或拟牛顿法）仍可能产生极其错误的估计。这一认识推动了算法框架的发展：它们显式利用 SLAM 问题的图论与几何基础，以获得更好的收敛性质和保证。

SLAM 近期最令人振奋的进展之一，是实用优化算法的发展。尽管该问题在一般意义上难解，这些算法仍能在温和条件下，可证明地恢复常见 SLAM 表述（例如 PGO）的可认证全局最优解。这类技术称为**可认证正确优化方法**（certifiably correct optimization methods），它们基于凸松弛而非局部优化。此外，它们之所以称为可认证最优，是因为在计算出优化问题的解后，能够量化该估计的次优程度，并可能认证其最优性。这并不与问题的 NP-困难性矛盾：在最坏情形下，这些算法仍可能无法为所计算的解给出最优性证书，只能提供次优性界。不过，这些算法仍有实际价值，主要有两个原因：(i) 在实践中，它们几乎能为所有实际关注的 SLAM 问题（例如具有合理测量噪声的问题）给出最优性证书；(ii) 无法给出最优性证书本身也具有信息价值，因为它可能触发对下游任务的警告，使其不要信任该 SLAM 估计，或促使机器人采取失效保护措施。



- (a)
- (b)
- (c)
- (d)

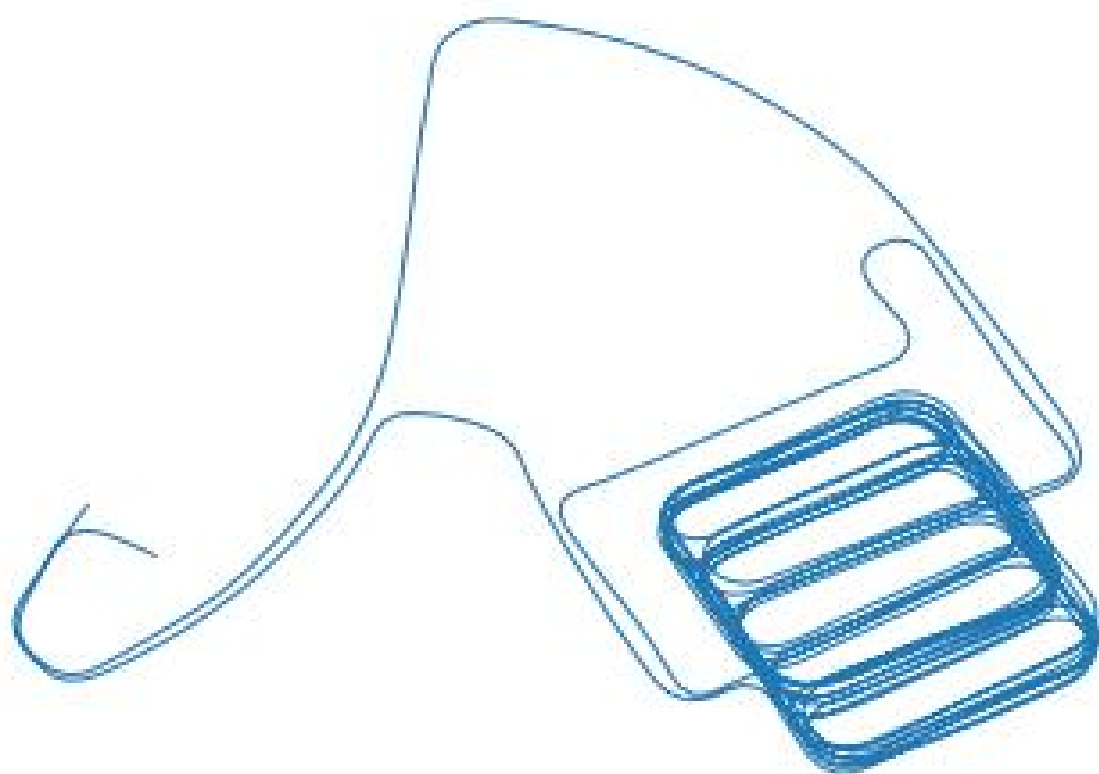
图 6.1 SLAM 最大似然估计中局部极小值的示例。图中展示了位姿图 SLAM 问题 (6.9) 的一个真实世界实例的多个局部极小值；机器人在 (a) 所示的多层停车场中导航。(b) 为使用 SE-Sync^[937] 计算的 (6.9) 的正确解，即全局最优解；(c) 和 (d) 则展示了局部优化方法以随机采样的起点初始化时得到的次优局部极小值示例。((a) 图由^[400]转载。)

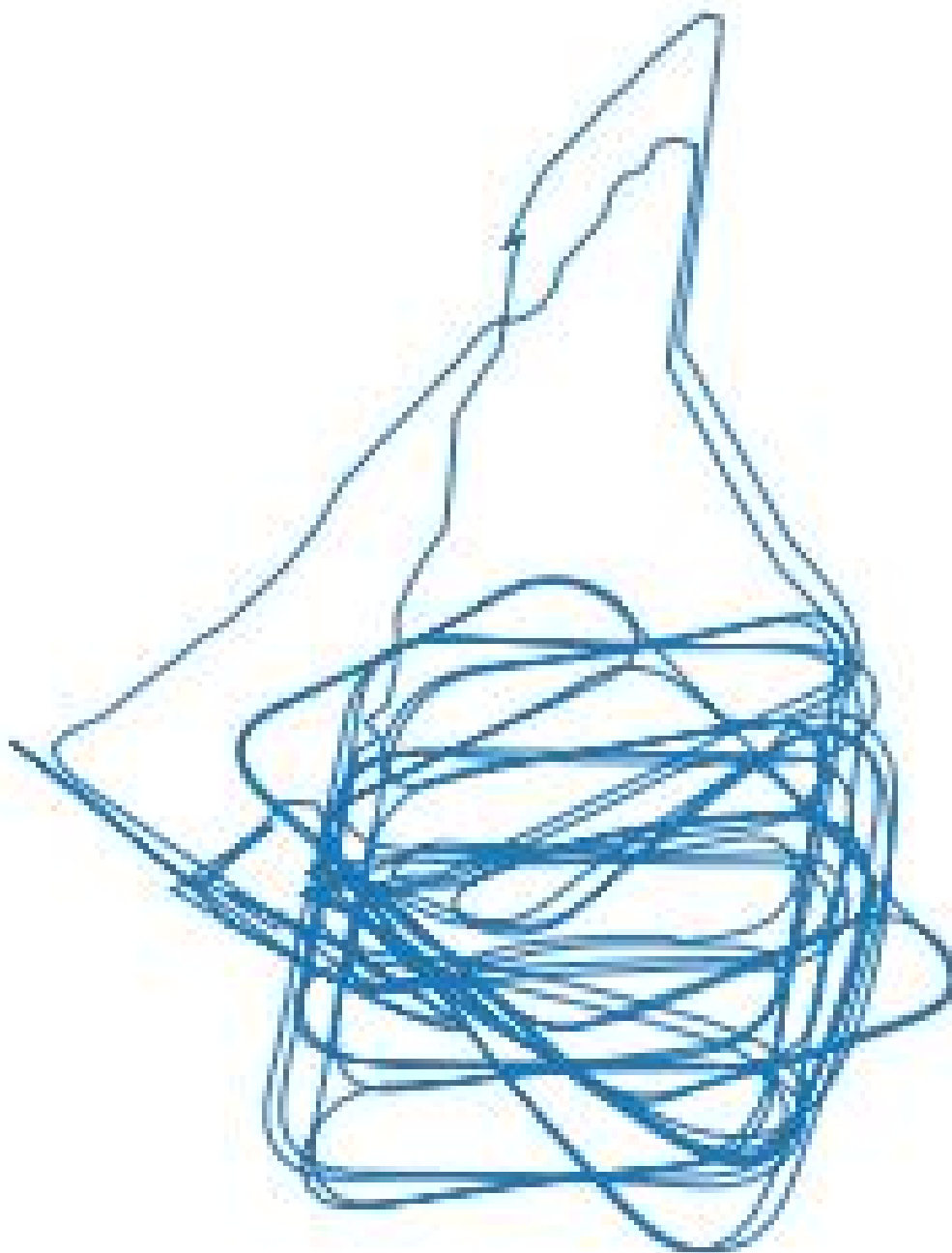
本节简要介绍 SLAM 的可认证正确估计方法。首先考察 Shor 松弛，它是构造支撑可认证估计技术之凸松弛的基本工具之一。接着说明如何应用 Shor 松弛，针对 PGO 这一基本问题构造可认证正确估计方法 SE-Sync。最后讨论 SE-Sync 和 Shor 松弛向更广泛 SLAM 问题的扩展。

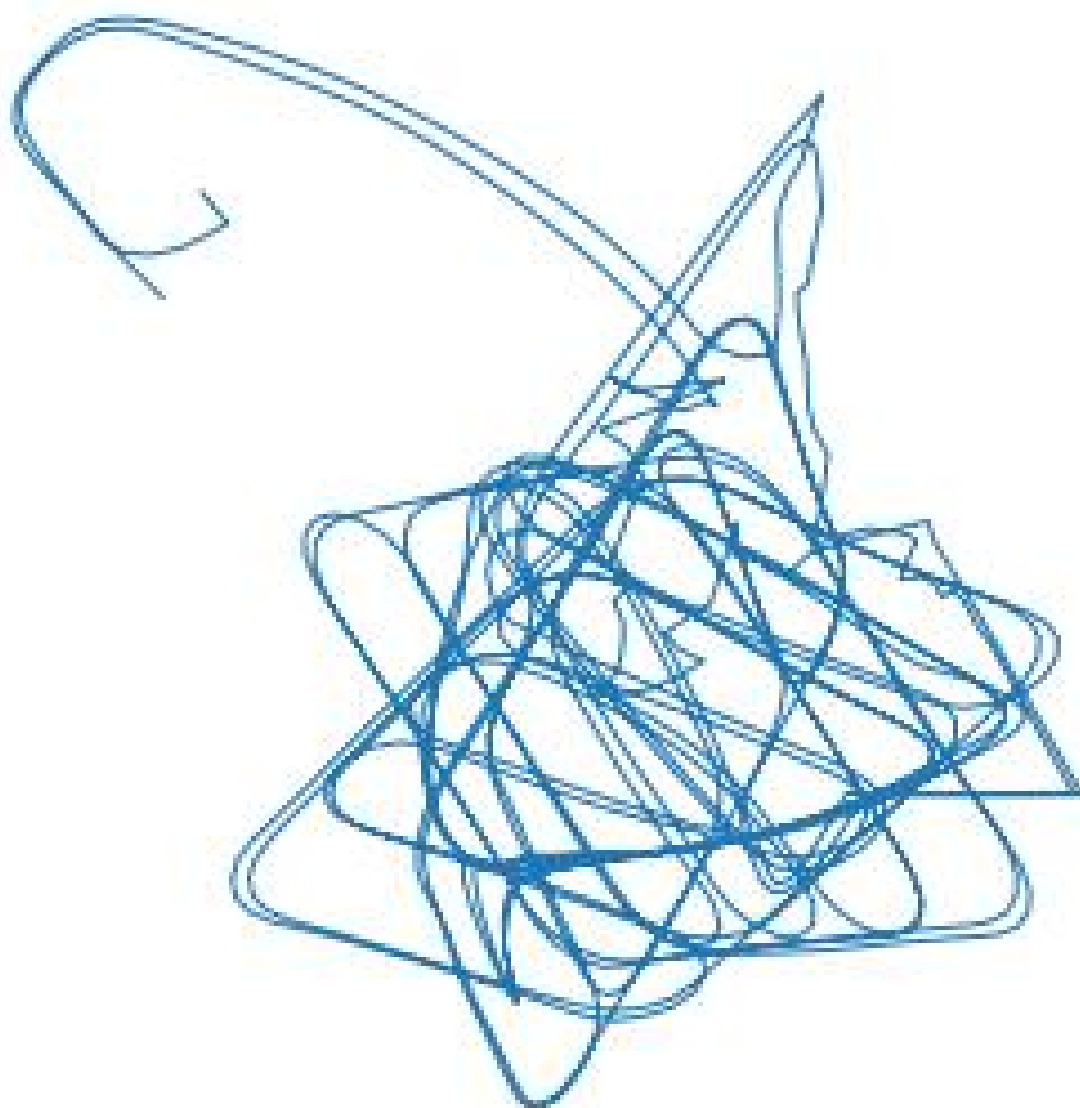
6.2 Shor 松弛

本小节介绍 Shor 松弛。这是我们将用来构造支撑可认证估计器之凸松弛的基本工具之一。

简言之，Shor 松弛是一种为二次约束二次规划（quadratically constrained quadratic program, QCQP）构造凸松弛的过程；QCQP 是指目标函数与约束函数均为二次函数的优化问题。如下一节将看到，许多常见







SLAM 表述均可写成 QCQP。

我们先说明 Shor 松弛如何应用于一般 QCQP，再在下一节将其具体化到 SLAM。考虑以下 QCQP：

$$\begin{aligned} p^* = \min_{\mathbf{x} \in \mathbb{R}^n} & \mathbf{x}^\top \mathbf{C} \mathbf{x} \\ \text{s.t. } & \mathbf{x}^\top \mathbf{A}_i \mathbf{x} = b_i \quad i = 1, \dots, m. \end{aligned} \quad (\text{QCQP})$$

其中， $\mathbf{C}, \mathbf{A}_1, \dots, \mathbf{A}_m \in \mathbb{S}^n$ 是对称矩阵，且 $\mathbf{b} = (b_1, \dots, b_m) \in \mathbb{R}^m$ 是向量。我们将通过一系列简单代数变换，说明如何产生 (QCQP) 的凸松弛。

首先，若 $\mathbf{M} \in \mathbb{S}^n$ 为任意对称矩阵，则可利用迹的循环性质，将由 $\mathbf{M}$ 确定的二次型 $\mathbf{x}^\top \mathbf{M} \mathbf{x}$ 重写为

$$\mathbf{x}^\top \mathbf{M} \mathbf{x} = \text{tr}(\mathbf{x}^\top \mathbf{M} \mathbf{x}) = \text{tr}(\mathbf{M} \mathbf{x} \mathbf{x}^\top). \quad (6.1)$$

将式 (6.1) 应用于 (QCQP)，可得其等价形式：

$$\begin{aligned} p^* = \min_{\mathbf{x} \in \mathbb{R}^n} & \text{tr}(\mathbf{C} \mathbf{x} \mathbf{x}^\top) \\ \text{s.t. } & \text{tr}(\mathbf{A}_i \mathbf{x} \mathbf{x}^\top) = b_i, \quad i = 1, \dots, m. \end{aligned} \quad (6.2)$$

现在注意到，决策变量 $\mathbf{x}$ 仅通过形如 $\mathbf{X} \triangleq \mathbf{x} \mathbf{x}^\top$ 的外积进入问题 (6.2)；按构造，每个这样的矩阵 $\mathbf{X}$ 都是对称、秩为 1 且半正定的。反之，若 $\mathbf{X} \in \mathbb{S}_+^n$ 是半正定矩阵且 $\text{rank}(\mathbf{X}) = 1$ ，则容易证明（考虑对称特征分解即可）对某个 $\mathbf{x} \in \mathbb{R}^n$ ， $\mathbf{X}$ 可作形如 $\mathbf{X} = \mathbf{x} \mathbf{x}^\top$ 的对称分解。综合这些观察，可得等价关系

$$\mathbf{X} \in \mathbb{S}_+^n \text{ and } \text{rank}(\mathbf{X}) = 1 \iff \exists \mathbf{x} \in \mathbb{R}^n \text{ such that } \mathbf{X} = \mathbf{x} \mathbf{x}^\top. \quad (6.3)$$

根据式 (6.3)，问题 (6.2) 等价于

$$\begin{aligned} p^* = \min_{\mathbf{X} \in \mathbb{S}^n} & \text{tr}(\mathbf{C} \mathbf{X}) \\ \text{s.t. } & \text{tr}(\mathbf{A}_i \mathbf{X}) = b_i, \quad i = 1, \dots, m, \\ & \mathbf{X} \succeq 0, \\ & \text{rank}(\mathbf{X}) = 1. \end{aligned} \quad (6.4)$$

至此，问题 (6.2) 和 (6.4) 完全等价；但表述 (6.4) 的优势在于，它揭示了大量有用结构。事实上，(6.4) 中的目标函数和约束函数都是关于（矩阵）决策变量 $\mathbf{X}$ 的线性函数，而半正定约束 $\mathbf{X} \succeq 0$ 是凸约束。因此，求解 (6.4) 的唯一困难来自（非凸的）秩约束。

Shor 松弛^[1011] 只需舍弃 (6.4) 中的秩约束，便得到 (QCQP) 的如下凸松弛：

$$\begin{aligned} d^* = \min_{\mathbf{X} \in \mathbb{S}^n} & \text{tr}(\mathbf{C} \mathbf{X}) \\ \text{s.t. } & \text{tr}(\mathbf{A}_i \mathbf{X}) = b_i, \quad i = 1, \dots, m, \\ & \mathbf{X} \succeq 0. \end{aligned} \quad (\text{SDP})$$

注意，(SDP) 是在一组线性等式约束下，在半正定矩阵集合上最小化线性函数；这种形式的问题称为半正定规划。半正定规划 (SDP) 是凸优化问题，可在多项式时间内求解。

现在考察问题 (QCQP) 与其凸松弛 (SDP) 的关系。首先注意到，我们通过扩展 (QCQP) 的可行集，即去除 (6.4) 中的秩约束，得到 (SDP)；确实，易知 (QCQP) 中的每个可行点 $\mathbf{x} \in \mathbb{R}^n$ 都可提升为 (SDP) 的相应可行点 $\mathbf{X} \triangleq \mathbf{x} \mathbf{x}^\top$ 。¹ 因此，(QCQP) 和 (SDP) 的最优值满足：

$$d^* \leq p^*, \quad (6.5)$$

因为后一个问题在更大的可行集上最小化同一目标函数。

不等式 (6.5) 已经提供了一种非常有用的方法，用于评估 (QCQP) 候选解的质量。设 $\hat{\mathbf{x}} \in \mathbb{R}^n$ 是 (QCQP) 的一个可行点；例如， $\hat{\mathbf{x}}$ 可能由局部优化得到。令 $f(\mathbf{x}) \triangleq \mathbf{x}^\top \mathbf{C} \mathbf{x}$ 为目标函数，则不等式 (6.5) 意味着，可按下式界定 $\hat{\mathbf{x}}$ 作为 (QCQP) 解的次优性 $f(\hat{\mathbf{x}}) - p^*$ ：

$$f(\hat{\mathbf{x}}) - p^* \leq f(\hat{\mathbf{x}}) - d^*. \quad (6.6)$$

注意, (QCQP) 的最优值 p^* 一般极难计算, 而通过求解松弛 (SDP) 可以高效计算最优值 d^* 。因此, 不等式 (6.6) 给出了一种实用途径: 无须知道 p^* 本身, 即可界定 $\hat{x}$ 的次优性。特别是, 若 (6.6) 右端很小, 就可以断定 x 是 (QCQP) 的近似最优解。

此外, 若求解松弛 (SDP), 且所得极小化器 $X^* = x^* x^{*\top}$ 恰好秩为 1, 则立刻可知向量 $\mathbf{x}^* \in \mathbb{R}^n$ 是原始 (非凸) 问题 (QCQP) 的全局极小化器 (因为 $\mathbf{x}^*$ 在 (QCQP) 中可行, 且在 (6.6) 中满足 $f(\mathbf{x}^*) = d^*$)。如后文所示, 这种有利情形实际上在许多机器人状态估计任务中经常出现, 使我们能够由松弛 (SDP) 的解恢复非凸问题 (QCQP) 的精确全局最优解。

在实践中, 使用现成求解器求解大规模 SDP 仍可能很慢或占用大量内存, 因此常为大规模机器人应用开发专用求解器, 例如下文将介绍的 SE-Sync。

6.3 SE-Sync: 可认证正确的位姿图优化

本小节说明如何应用 Shor 松弛, 为求解位姿图优化 (pose-graph optimization, PGO) 这一基本问题开发可认证正确算法 SE-Sync。PGO 是最简单且最常用的 SLAM 表述之一, 因而为说明可认证估计算法的构造提供了自然且具体的例子。此外, PGO 是首个被证明适用于凸松弛的 SLAM 表述^[149, 151, 939]; 相应思想也已被证明可推广到广泛的 SLAM 问题, 后文将予以讨论。

我们的推导分为三个阶段。首先说明如何将 PGO 形式化为最大似然估计实例, 并将其化简为 QCQP。接着推导 PGO 的 Shor SDP 松弛, 并且至关重要地说明: 当测量噪声充分小时, 该松弛实际上是精确的; 这意味着可以通过求解其 (凸) SDP 松弛恢复 PGO 的全局最优解。最后介绍一种利用结构的专用优化算法, 使我们能在实践中求解这一 SDP 松弛的大规模实例。

6.4 位姿图优化: QCQP 表述

位姿图优化 (PGO) 在 d 维空间中 (SLAM 通常取 $d = 2$ 或 3), 根据它们之间一组相对位姿的带噪测量 $\tilde{\mathbf{T}}_{ij} \approx \mathbf{T}_i^{-1} \mathbf{T}_j$, 估计一组 n 个未知位姿 $T_1, \dots, T_n \in \text{SE}(d)$ 的取值。实践中, 未知位姿 $T_1, \dots, T_n$ 描述机器人轨迹 (即在机器人轨迹的离散时刻进行采样), 而测量 $\tilde{\mathbf{T}}_{ij}$ 由 SLAM 前端获得, 例如通过 LiDAR 扫描匹配、轮式里程计或三维计算机视觉技术获得。本小节说明如何通过最大似然估计将该估计问题形式化。将会看到, 在关于噪声作出适当假设时, 所得优化问题是 QCQP。

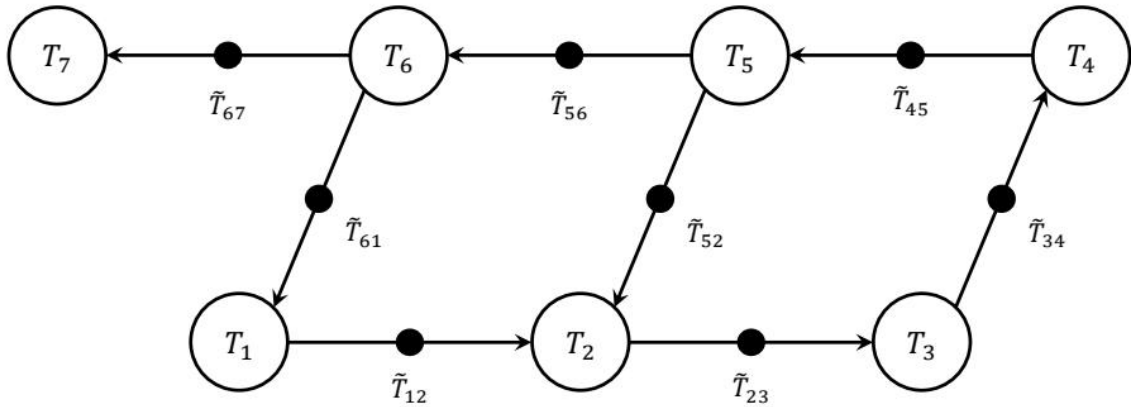


图 6.1: 图 6.2 位姿图示例。这里, 顶点与待估未知位姿 $T_i = (t_i, R_i) \in \text{SE}(d)$ 一一对应; 有向边则与它们之间相对位姿的一组带噪测量 $\tilde{\mathbf{T}}_{ij} \approx \mathbf{T}_i^{-1} \mathbf{T}_j$ 一一对应。

首先, 常可方便地使用按如下方式构造的位姿图 $\vec{\mathcal{G}}$ 来建模定义此估计问题的数据。令 $\mathcal{G} = (\nu, \mathcal{E})$ 为简单无向图, 其节点 i 与未知位姿 T_i 一一对应, 其边 $\{i, j\} \in \mathcal{E}$ 与可用测量集合一一对应。³ 不失一般性, 假定 $\mathcal{G}$ 是连通的。⁴ 随后, 对 $\mathcal{G}$ 的每条边指定方向 (参见图 6.2), 即可从 $\mathcal{G}$ 得到位姿图 $\vec{\mathcal{G}} = (\nu, \vec{\mathcal{E}})$ 。按约定, 描述位姿 T_j 在位姿 T_i 坐标系中取值的 (带噪) 测量 $\tilde{\mathbf{T}}_{ij}$, 对应于从 i 指向 j 的有向边。

为将 PGO 形式化为最大似然估计，必须为可用测量 $\{\tilde{T}_{ij}\}$ 假设一个噪声模型。为此，我们使用各向同性 Langevin 分布 $\mathcal{L}(M, \kappa)$ ：它是定义在 $\text{SO}(d)$ 上的指数族分布，其概率密度函数为

$$p(\mathbf{R}; \mathbf{M}, \kappa) = \frac{1}{c_d(\kappa)} \exp\left(\kappa \text{tr}\left(\mathbf{M}^\top \mathbf{R}\right)\right), \quad (6.7)$$

其中 $M \in \text{SO}(d)$ 和 $\kappa \geq 0$ 为参数， $c_d(\kappa)$ 是归一化常数。注意， M 起位置参数（称为众数）的作用，而 $\kappa \geq 0$ 是标量集中参数。各向同性 Langevin 分布在二维和三维中具有特别简单的生成描述：为生成样本 $\tilde{\mathbf{R}} \sim \mathcal{L}(\mathbf{M}, \kappa)$ ，先从圆周上的 von Mises 分布采样旋转角 $\theta \sim \text{vonMises}(0, 2\kappa)$ ，然后在 $d = 2$ 时令 $\tilde{\mathbf{R}} = M \cdot R(\theta)$ ，或在 $d = 3$ 时令 $\tilde{\mathbf{R}} = M \exp(\theta \mathbf{v}^\wedge)$ ，其中 $\mathbf{v} \sim \mathcal{U}(S^2)$ 是均匀采样的旋转轴^[937]。直观上，可将此分布看作定义在旋转这一非欧流形上的高斯分布类比。

给定一个位姿图 $\vec{\mathcal{G}} = (\nu, \mathcal{E})$ ，假设每个测量 $\tilde{T}_{ij} = (\tilde{\mathbf{t}}_{ij}, \tilde{\mathbf{R}}_{ij}) \in \text{SE}(d)$ 均由以下概率生成模型采样得到：

$$\begin{aligned} \tilde{\mathbf{t}}_{ij} &= \bar{\mathbf{t}}_{ij} + \mathbf{t}_{ij}^\epsilon, & \mathbf{t}_{ij}^\epsilon &\sim \mathcal{N}(0, \tau_{ij}^{-1} I_d), & \forall (i, j) \in \vec{\mathcal{E}}, \\ \tilde{\mathbf{R}}_{ij} &= \bar{\mathbf{R}}_{ij} \mathbf{R}_{ij}^\epsilon, & \mathbf{R}_{ij}^\epsilon &\sim \mathcal{L}(\mathbf{I}_d, \kappa_{ij}), \end{aligned} \quad (6.8)$$

其中， $\bar{T}_{ij} = (\bar{\mathbf{t}}_{ij}, \bar{\mathbf{R}}_{ij}) \in \text{SE}(d)$ 是位姿 T_i 与 T_j 间相对位姿的真实（潜在）值。模型 (6.8) 假定第 ij 个测量的平移分量 $\tilde{\mathbf{t}}_{ij}$ 受集中参数为 $\tau_{ij} > 0$ 的加性零均值各向同性高斯噪声污染，而旋转分量 $\tilde{\mathbf{R}}_{ij}$ 则受众数为 $\mathbf{I}_d$ 、集中参数为 $\kappa_{ij} \geq 0$ 的乘性各向同性 Langevin 噪声污染。

采用噪声模型 (6.8)，而非第 2 章提到的定义在一般李群上的更“通用”的指数化高斯噪声模型，主要是因为其关联的最大似然估计具有特别简单的代数形式。实际上，给定从 (6.8) 采样得到的一组带噪测量 $\tilde{\mathbf{T}}_{ij}$ ，直接计算可得关联的最大似然估计为

$$p_{\text{MLE}}^* = \min_{\substack{\mathbf{t}_i \in \mathbb{R}^d \\ \mathbf{R}_i \in \text{SO}(d) \ (i, j) \in \vec{\mathcal{E}}}} \sum_{(i, j) \in \vec{\mathcal{E}}} \kappa_{ij} \|\mathbf{R}_j - \mathbf{R}_i \tilde{\mathbf{R}}_{ij}\|_F^2 + \tau_{ij} \|\mathbf{t}_j - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{t}}_{ij}\|_2^2. \quad (6.9)$$

特别地，注意 (6.9) 中的目标函数是简单的（二次）线性最小二乘损失。此外，约束 $\mathbf{R}_i \in \text{SO}(d)$ 在平面情形 ($d = 2$) 和三维情形 ($d = 3$) 中均可写成二次约束。下一小节将利用这一事实，经由 Shor 松弛推导问题 6.1 的凸松弛。

6.5 将 Shor 松弛应用于位姿图优化

本小节说明如何应用 Shor 松弛，导出 PGO 问题 6.1 的凸松弛。

化简最大似然估计器。第一步是将问题 6.1 改写为只涉及旋转的更紧凑形式，并揭示优化问题 (6.9) 与构造它的底层图 $\mathcal{G}$ 、 $\vec{\mathcal{G}}$ 之间的对应关系。为此，引入由这些图构造的若干矩阵；关于（代数）图论的入门知识，特别是关联矩阵和拉普拉斯矩阵，请参阅下一页的专栏。

6.6 专栏 6.1：代数图论要素

代数图论研究如何使用代数工具（例如矩阵、向量）表示、分析和从图中提取信息。这里回顾一些基本概念，以支撑本章所作的说明和联系。同时也请读者参阅第 1 章，那里讨论了图与概率图模型之间的联系。

有向图 $\vec{\mathcal{G}}$ 是二元组 $(\nu, \mathcal{E})$ ，其中 ν 是有限节点集合， $\mathcal{E}$ 是边的集合，每条边包含一对有序节点。边 $e \in \mathcal{E}$ 形如 $e = (i, j)$ ，表示连接节点 i 和 j 的边 e 离开节点 i 并指向节点 j （ i 是边的尾部， j 是边的头部）。

对于具有 n 个节点和 m 条边的图，有向图 $\vec{\mathcal{G}}$ 的关联矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 是元素属于 $\{-1, 0, +1\}$ 、描述图结构的矩阵。 $\mathbf{A}$ 的每一行对应一条边，而对应边 $e = (i, j)$ 的行仅有两个非零元素：第 i 列为 -1 ，第 j 列为 $+1$ 。尽管更常见的关联矩阵定义将边置于列中，即使用这里 $\mathbf{A}$ 的转置，但为保持与第 1 章雅可比矩阵的一些对称性，这里采用上述定义。例如，图 6.2 的关联矩阵为

$$\mathbf{A} = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \\ -1 & +1 & & & & & \\ & -1 & +1 & & & & \\ & & -1 & +1 & & & \\ & & & -1 & +1 & & \\ & & & & -1 & +1 & \\ & & & & & -1 & +1 \\ & +1 & & & -1 & & \\ +1 & & & & & -1 & \end{bmatrix} \begin{matrix} e_{12} \\ e_{23} \\ e_{34} \\ e_{45} \\ e_{56} \\ e_{67} \\ e_{52} \\ e_{61} \end{matrix} \quad (6.10)$$

拉普拉斯矩阵 $L \in \mathbb{R}^{n \times n}$ 定义为 $L \triangleq A^T A$, 同样刻画图的连通性。具体而言, L 的第 i 个对角元素对应图中第 i 个节点的度, 即与节点 i 相连的节点数; 而位置 (i, j) 的非对角元素, 若存在一条不论方向如何均连接节点 i 和 j 的边, 则等于 -1 , 否则为零。例如, 图 6.2 的拉普拉斯矩阵为

$$\mathbf{L} = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \\ +2 & -1 & & & & -1 & \\ -1 & +3 & -1 & & -1 & & \\ & -1 & +2 & -1 & & & \\ & & -1 & +2 & -1 & & \\ & -1 & & -1 & +3 & -1 & \\ -1 & & & & -1 & +3 & -1 \\ & & & & & -1 & +1 \end{bmatrix} \begin{matrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{matrix} \quad (6.11)$$

需要指出, 拉普拉斯矩阵不再保留图中边的方向信息, 因为只要存在连接相应节点的边, 不论其方向, 非对角元素即为 -1 。

拉普拉斯矩阵的最小特征值始终等于零, 对应特征向量为所有元素均为一的向量 (每行元素之和为零, 故易知 $L \cdot \mathbf{1} = \mathbf{0}$)。零特征值的数目恰对应图的连通分量数: 连通图, 即任意一对节点之间不顾边方向均存在边路径的图, 只有一个零特征值; 由两个不连通子图组成的图有两个零特征值, 依此类推。此外, 对连通图而言, 第二小特征值衡量图的连通程度, 也称为图的代数连通度或 Fiedler 值。

定义若干与图拉普拉斯矩阵相关、后续推导将使用的关键矩阵。令平移权重图 $\mathcal{W}^T = (\mathcal{V}, \mathcal{E}, \{\tau_{ij}\})$ 为加权无向图, 其节点集为 $\mathcal{V}$ 、边集为 $\mathcal{E}$, 且对 $\{i, j\} \in \mathcal{E}$ 的边权重为 τ_{ij} ; 以 $L(\mathcal{W}^T) \in \mathbb{S}_+^n$ 表示其拉普拉斯矩阵:

$$L(\mathcal{W}^T)_{ij} = \begin{cases} \sum_{\{i, k\} \in \mathcal{E}} \tau_{ik}, & i = j, \\ -\tau_{ij}, & \{i, j\} \in \mathcal{E}, \\ 0, & \{i, j\} \notin \mathcal{E}. \end{cases} \quad (6.12)$$

这只是上文所定义拉普拉斯矩阵的加权版本。类似地, 以 $L(\tilde{G}^\rho) \in \mathbb{S}_+^{dn}$ 表示由旋转测量 $\tilde{R}_{ij}$ 和精度 κ_{ij} 确定的 connection Laplacian; 它是如下定义的对称 $(d \times d)$ 分块结构矩阵:

$$L(\tilde{G}^\rho)_{ij} \triangleq \begin{cases} \left(\sum_{\{i, k\} \in \mathcal{E}} \kappa_{ik} \right) \mathbf{I}_d, & i = j, \\ -\kappa_{ij} \tilde{\mathbf{R}}_{ij}, & (i, j) \in \vec{\mathcal{E}}, \\ -\kappa_{ji} \tilde{\mathbf{R}}_{ji}^T, & (j, i) \in \vec{\mathcal{E}}, \\ \mathbf{0}_{d \times d}, & \{i, j\} \notin \mathcal{E}. \end{cases} \quad (6.13)$$

还定义若干由平移观测集合 $\tilde{t}_{ij}$ 构造的矩阵。令 $\tilde{V} \in \mathbb{R}^{n \times dn}$ 为 $(1 \times d)$ 分块结构矩阵, 其 (i, j) 分块定义为

$$\tilde{V}_{ij} \triangleq \begin{cases} \sum_{\{k \in \mathcal{V} | (j, k) \in \vec{\mathcal{E}}\}} \tau_{jk} \tilde{\mathbf{t}}_{jk}^T, & i = j, \\ -\tau_{ji} \tilde{\mathbf{t}}_{ji}^T, & (j, i) \in \vec{\mathcal{E}}, \\ \mathbf{0}_{1 \times d}, & \text{otherwise,} \end{cases} \quad (6.14)$$

令 $\tilde{D} \in \mathbb{R}^{m \times dn}$ 为 $(1 \times d)$ 分块结构矩阵, 其行和列分别以 $e \in \vec{\mathcal{E}}$ 和 $k \in \mathcal{V}$ 索引, 且其 (e, k) 分块由下式给出:

$$\tilde{D}_{ek} \triangleq \begin{cases} -\tilde{\mathbf{t}}_{kj}^\top, & e = (k, j) \in \vec{\mathcal{E}}, \\ \mathbf{0}_{1 \times d}, & \text{otherwise,} \end{cases} \quad (6.15)$$

并令 $\Omega \triangleq \text{Diag}(\tau_{e_1}, \dots, \tau_{e_m}) \in \mathbb{S}^m$ 表示由平移测量精度构成的对角矩阵。最后, 还将旋转和平移状态估计汇总到分块矩阵 $R \triangleq (R_1 \ \dots \ R_n) \in \text{SO}(d)^n \subset \mathbb{R}^{d \times dn}$ 和 $\mathbf{t} \triangleq (\mathbf{t}_1 \ \dots \ \mathbf{t}_n) \in \mathbb{R}^{dn}$ 中。

有了这些定义, 回到问题 6.1。注意, 若固定旋转状态 $\mathbf{R}_1, \dots, \mathbf{R}_n$ 的取值, 则问题 (6.9) 化为关于剩余平移决策变量 $\mathbf{t}_1, \dots, \mathbf{t}_n \in \mathbb{R}^d$ 的线性最小二乘问题。因此, 可以将平移状态的最优赋值 $\mathbf{t}^*(R)$ 解为旋转状态 R 的函数:

$$\mathbf{t}^*(R) = -\text{vec} \left(R \tilde{V}^\top L (\mathcal{W}^\tau)^\dagger \right). \quad (6.16)$$

将最优赋值 (6.16) 代入 (6.9), 即可从位姿图 SLAM 的 MLE 中解析消去平移状态, 得到下列仅涉及旋转状态的简化 (但等价) 问题:

问题 6.2 (仅旋转位姿图优化)

$$p_{\text{MLE}}^* = \min_{R \in \text{SO}(d)^n} \text{tr} \left(\tilde{Q} R^\top R \right) \quad (6.17a)$$

$$\tilde{Q} = L(\tilde{G}^\rho) + \tilde{D}^\top \Omega^{\frac{1}{2}} \Pi \Omega^{\frac{1}{2}} \tilde{D}, \quad (6.17b)$$

其中, $\Pi \in \mathbb{R}^{m \times m}$ 是到 $\vec{\mathcal{G}}$ 的加权关联矩阵 $A(\vec{\mathcal{G}})\Omega^{\frac{1}{2}}$ 的核空间 $\ker(A(\vec{\mathcal{G}})\Omega^{\frac{1}{2}})$ 的正交投影矩阵。

注意, (6.17) 只涉及 n 个旋转矩阵, 而非 n 个位姿; 该问题现在类似于标准多旋转平均问题^[435], 但数据矩阵 $\tilde{Q}$ 的表达式更为复杂。从更技术性的角度看, 尽管 Π 一般是稠密的, 利用它由图 $\vec{\mathcal{G}}$ 导出的事实, 可证明它具有如下分解:

$$\Pi = \mathbf{I}_m - \Omega^{\frac{1}{2}} \bar{A}(\vec{\mathcal{G}})^\top L^{-\top} L^{-1} \bar{A}(\vec{\mathcal{G}}) \Omega^{\frac{1}{2}} \quad (6.18)$$

其中, $\bar{A}(\vec{\mathcal{G}})\Omega^{\frac{1}{2}} = LQ$ 是 $\bar{A}(\vec{\mathcal{G}})\Omega^{\frac{1}{2}}$ 的薄 LQ 分解, $\bar{A}(\vec{\mathcal{G}})$ 是从 $\vec{\mathcal{G}}$ 的关联矩阵 $A(\vec{\mathcal{G}})$ 删除一行得到的约化关联矩阵。注意, 表达式 (6.18) 仅需要下三角因子 L ; 只要底层图 $\vec{\mathcal{G}}$ 是稀疏的, 该因子也是稀疏的, 并可在实践中高效获得。数据矩阵 $\tilde{Q}$ 的稀疏分解 (6.17b)–(6.18) 将在实现高效优化方法时发挥关键作用 (参见第 6.1.2.3 节)。

构造松弛。现在利用简化形式 (6.17), 推导实践中要求解的问题 6.1 的半正定松弛。

首先将条件 $R \in \text{SO}(d)^n$ 松弛为 $R \in \text{O}(d)^n$ 。后一条件相较前一条件的优势在于, 正交矩阵由一组 (二次) 正交归一约束定义, 故问题 6.2 的正交松弛是齐次 QCQP。事实上, 对于提取 $(d \times d)$ 分块结构矩阵 M 的 n 个对角块的线性映射

$$\begin{aligned} \text{BlockDiag}_{d \times d}: \mathbb{R}^{dn \times dn} &\rightarrow \mathbb{R}^{d \times dn} \\ \text{BlockDiag}_{d \times d}(M) &\triangleq (M_{11}, \dots, M_{nn}) \end{aligned} \quad (6.19)$$

可将 (6.17) 的正交松弛写成具有外在约束的形式:

$$p_O^* = \min_{R \in \mathbb{R}^{d \times dn}} \text{tr} \left(\tilde{Q} R^\top R \right) \quad \text{s.t.} \quad \text{BlockDiag}_{d \times d}(R^\top R) = (\mathbf{I}_d, \dots, \mathbf{I}_d). \quad (6.20)$$

现在注意到, (6.20) 具有与 (QCQP) 相似的结构, 但其中向量 x 被矩阵 R 替代。在 (QCQP) 中, 我们通过将项 $xx^\top$ 替换为合适的秩 1 矩阵 X 并舍弃秩 1 约束来获得松弛; 同样, 可将 (6.20) 中的项 $R^\top R$ 替换为秩 d 矩阵 Z 并舍弃秩约束, 由此得到简化 PGO 问题 (6.17) 的如下半正定松弛:

问题 6.3 (位姿图优化的半正定松弛)

$$p_{\text{SDP}}^* = \min_{Z \in \mathbb{S}_+^{dn}} \text{tr} \left(\tilde{Q} Z \right) \quad \text{s.t.} \quad \text{BlockDiag}_{d \times d}(Z) = (\mathbf{I}_d, \dots, \mathbf{I}_d). \quad (6.21)$$

如第 6.1.1 节所见, 该构造立即蕴含 $p_{\text{MLE}}^* \geq p_O^* \geq p_{\text{SDP}}^*$ 。此外, 若求解 SDP 松弛 (6.21) 后, 恢复的极小化器 $Z^* \in \mathbb{S}_+^{dn}$ 具有形如 $Z^* = R^{*\top} R^*$ 的秩 d 分解, 且 $R^* \in \text{SO}(d)^n$, 则 R^* 本身就是 PGO 问题 6.2 的全局最优解。使我们关注松弛 (6.21) 的显著事实在于, 这种有利情形实际上会在实践中出现。具体而言, 有如下定理 (证明参见 [937]):

定理 6.1 (由问题 6.3 精确恢复 PGO 解) 令 $\bar{Q}$ 为使用 (6.8) 中的真值相对变换 $\bar{T}_{ij}$ 构造的、形如 (6.17b) 的矩阵。存在常数 $\beta \triangleq \beta(\bar{Q}) > 0$, 它依赖于 $\bar{Q}$, 使得若 $\|\bar{Q} - \bar{Q}\|_2 < \beta$, 则:

1. 半正定松弛问题 6.3 有唯一解 Z^* ; 且
2. $Z^* = R^{*\top} R^*$, 其中 $R^* \in \text{SO}(d)^n$ 是最大似然估计问题 6.2 的一个极小化器。

简言之, 定理 6.1 保证, 只要污染可用测量 $\bar{T}_{ij}$ 的噪声不太大, 就可通过求解 SDP 松弛 (6.21) 恢复问题 6.2 的全局极小化器 R^* , 并经由 (6.16) 恢复问题 6.1 的全局极小化器 (R^*, t^*) 。

6.7 通过黎曼阶梯高效求解松弛

作为半正定规划, 问题 6.3 原则上可以高效求解, 即在多项式时间内求解。然而实践中, 存储和操作 (6.21) 中出现的稠密矩阵决策变量 Z 的高计算代价, 使通用内点法无法有效扩展到 Z 维度超过几千的问题。不幸的是, 机器人学和计算机视觉应用中出现的 (6.21) 实例, 其规模通常比这一有效上限大一到两个数量级, 远远超出通用技术的能力。因此, 本小节开发一种利用结构的专用优化过程, 以高效求解问题 6.3 的大规模实例。

利用低秩结构。我们方法背后的主要思想是利用问题 6.3 存在低秩解这一事实。具体而言, 注意松弛 (6.21) 中的决策变量 Z 虽是一般的高维半正定矩阵, 但定理 6.1 保证, 每当松弛精确时, 我们所寻求的解 Z^* 都可写成简洁的因子形式 $Z^* = R^{*\top} R^*$ 。此外, 即使精确性不成立, 问题 6.3 的极小化器通常也具有不比 d 大很多的秩 r , 因而同样可以写成简洁的对称秩分解 $Z^* = Y^{*\top} Y^*$, 其中 $Y^* \in \mathbb{R}^{r \times dn}$ 。

Burer 和 Monteiro [125, 124] 的奠基性工作提出了一种利用这类低秩解的优雅通用方法: 将 (6.21) 中决策变量 Z 的每一次出现都替换为秩 r 的对称分解 $Y^\top Y$ (其中 $Y \in \mathbb{R}^{r \times dn}$), 从而得到 (6.21) 的 Burer–Monteiro 分解:

$$p_{\text{SDPLR}}^*(r) = \min_{Y \in \mathbb{R}^{r \times dn}} \text{tr}(\bar{Q} Y^\top Y) \quad \text{s.t.} \quad \text{BlockDiag}_{d \times d}(Y^\top Y) = (\mathbf{I}_d, \dots, \mathbf{I}_d). \quad (6.22)$$

注意, 由于 $Y^\top Y$ 按构造是对称且半正定的, (6.22) 不再需要显式施加原始 SDP (6.21) 中的半正定约束。此外, 还可注意到, 该问题与所松弛的 (6.20) 高度相似, 重要区别在于: 现在矩阵 Y 的大小为 $r \times dn$, 而非 $d \times dn$, 其中 $r > d$; 换言之, 与 (6.20) 相比, (6.22) 在更高维空间中重新表述了该问题。

若将 (6.22) 中的最大秩参数 r 选得“较小” (即 $r \ll dn$), 则 $\dim(\mathbb{R}^{r \times dn}) = rnd \ll (dn + 1)dn/2 = \dim(\mathbb{S}_+^{dn})$; 也就是说, (6.22) 的搜索空间远低于 (6.21) 的搜索空间。因此, Burer 和 Monteiro 建议对低维非线性规划 (6.22) 应用快速非线性规划算法, 搜索原始 SDP (6.21) 极小化器 $Z^* = Y^{*\top} Y^*$ 的低秩因子 $Y^* \in \mathbb{R}^{r \times dn}$ 。

利用几何结构。进一步将 Y 划分为 $r \times d$ 个分块, 即 $Y = (Y_1, \dots, Y_n) \in \mathbb{R}^{r \times dn}$, 则 (6.22) 中的块对角约束等价于对所有 $i \in [n]$ 满足 $Y_i^\top Y_i = \mathbf{I}_d$; 从几何上看, 该条件表示每个分块 $Y_i \in \mathbb{R}^{r \times d}$ 的列构成一个正交归一标架。一般地, $\mathbb{R}^p$ 中所有正交归一 k -标架的集合为

$$\text{St}(k, p) \triangleq \left\{ Y \in \mathbb{R}^{p \times k} \mid Y^\top Y = \mathbf{I}_k \right\}, \quad (6.23)$$

它构成一个光滑紧致矩阵流形, 称为 Stiefel 流形。这意味着, 等式约束非线性规划 (6.22) 等价于定义在 Stiefel 流形乘积上的如下无约束优化问题:

问题 6.4 (Burer–Monteiro 分解的 SDP 松弛作为流形优化)

$$p_{\text{SDPLR}}^*(r) = \min_{Y \in \text{St}(d, r)^n} \text{tr}(\bar{Q} Y^\top Y). \quad (6.24)$$

表述 (6.22) 和 (6.24) 是等价的, 但后者具有重要的计算优势。特别是, 认识到可行集是 Stiefel 流形的乘积后, 便可以应用针对光滑流形优化的专用算法; 与通用等式约束非线性规划技术相比, 这些算法明显更简单、更快且更准确 [106]。

确保全局最优性。从问题 6.3 化简到问题 6.4, 大幅缩小了待求解优化问题的规模, 但代价是重新引入了二次正交归一约束 (6.23), 而这些约束是非凸的。因此, 将问题 6.2 松弛为问题 6.4 是否真的带来收益并不显然, 因为看起来我们可能只是用另一个困难的非凸优化问题替换了原来的问题。下面这个显著结果 (改编自 Boumal 等人 [107]) 为该方法提供了依据:

定理 6.2 (问题 6.4 全局最优性的一个充分条件) 若 $Y \in \text{St}(d, r)^n$ 是问题 6.4 的一个 (行) 秩亏二阶临界点, 则 Y 是问题 6.4 的全局极小化器, 且 $Z^* = Y^\top Y$ 是半正定松弛问题 6.3 的一个解。

定理 6.2 直接启发出一种简单过程, 即黎曼阶梯 (Riemannian Staircase): 对问题 6.4 的一系列实例应用快速局部优化算法, 以恢复问题 6.3 的解。简言之, 从某个 (较小的) 初始最大秩 $r \geq d$ 开始, 对 (6.24) 应用局部求解器 (更准确地说, 是二阶黎曼优化算法), 恢复二阶临界点 $Y^* \in \text{St}(d, r)^n$ 。若 Y^* 秩亏, 则定理 6.2 证明了 Y^* 是 (6.24) 的全局极小化器, 且 $Z^* = Y^{*\top} Y^*$ 是 (6.21) 的解。另一方面, 若 Y^* 不秩亏, 则只需提高最大秩 r 后重试。注意, 对 $r \geq dn + 1$, 每个 $Y \in \mathbb{R}^{r \times dn}$ 都 (按行) 秩亏, 因此黎曼阶梯保证在有限次迭代后以最优解 Y^* 终止。不过, 通常只需一到两级 “阶梯”。

最后从实践角度说明, 黎曼阶梯充当一种轻量级元算法, 围绕实践中 SLAM 常用的同类快速 (二阶) 局部优化算法运行。因此, 该方法在保证恢复全局最优解的同时, 保留当前最先进 SLAM 技术的速度, 从而兼得二者优势。

6.8 舍入解

刚才已经看到, 黎曼阶梯提供了一种高效算法, 用于恢复松弛问题 6.3 的解 $Z^* = Y^{*\top} Y^*$ 的低秩因子 $Y^* \in \text{St}(d, r)^n$ 。然而, 只要松弛 (6.21) 精确, 我们理想上希望从 Z^* 中提取 PGO 问题 6.2 的最优解 $R^* \in \text{SO}(d)^n$; 否则, 希望提取可行的近似解 $\hat{R} \in \text{SO}(d)^n$ 。本小节描述一种高效舍入过程: 它直接在低秩因子 Y^* 上操作, 而无须显式构造稠密的高维矩阵 Z^* 。

该方法的核心洞见是: 当松弛 (6.21) 精确时, R^* 、 Z^* 和 Y^* 满足

$$Y^{*\top} Y^* = Z^* = R^{*\top} R^*. \quad (6.25)$$

此时, 式 (6.25) 意味着低秩因子 $Y^* \in \mathbb{R}^{r \times dn}$ 实际具有秩 d , 因而可以对它计算薄奇异值分解, 从 Y^* 恢复 R^* 。更一般地, 当 (6.21) 不精确时, 仍可通过截断奇异值分解从 Y^* 恢复最优的秩- d 近似 $\hat{R} \in \mathbb{R}^{d \times dn}$, 然后将 $\hat{R}$ 的每个 $d \times d$ 分块投影到 $\text{SO}(d)$ 上 (同样使用 SVD), 从而得到问题 6.2 的可行近似解。

算法 3 SE-Sync 算法

输入: 初始点 $Y \in \text{St}(d, r_0)^n$, $r_0 \geq d + 1$ 。

输出: 最大似然估计问题 6.1 的可行估计 $\hat{T} \in \text{SE}(d)^n$, 以及问题 6.1 最优值的下界 p_{SDP}^* 。

- 1: 函数 SE-SYNC(Y)
- 2: 设置 $Y^* \leftarrow \text{RIEMANNIANSTAIRCASE}(Y)$ 。
- 3: 设置 $p_{SDP}^* \leftarrow F(\tilde{Q} Y^{*\top} Y^*)$ 。
- 4: 设置 $\hat{R} \leftarrow \text{ROUND SOLUTION}(Y^*)$ 。
- 5: 通过 (6.16) 恢复与 $\hat{R}$ 对应的最优平移估计 $\hat{t}$ 。
- 6: 设置 $\hat{T} \leftarrow (\hat{t}, \hat{R})$ 。
- 7: 返回 $\{\hat{T}, p_{SDP}^*\}$ 。
- 8: 结束函数

6.9 SE-Sync: 完整算法

将第 6.1.2.3 节的高效 SDP 优化方法与第 6.1.2.4 节的舍入过程结合起来, 即得到 *SE-Sync* (算法 3), 也就是我们的位姿图优化可认证正确算法 [937]。

应用于 PGO 实例时, SE-Sync 返回最大似然估计问题 6.1 的可行点 $\hat{T} \in \text{SE}(d)^n$, 以及其最优值的下界 $p_{SDP}^* \leq p_{MLE}^*$ 。该下界又可按下式给出任意可行点 $T = (t, R) \in \text{SE}(d)^n$ 作为问题 6.1 解时次优性的上界:

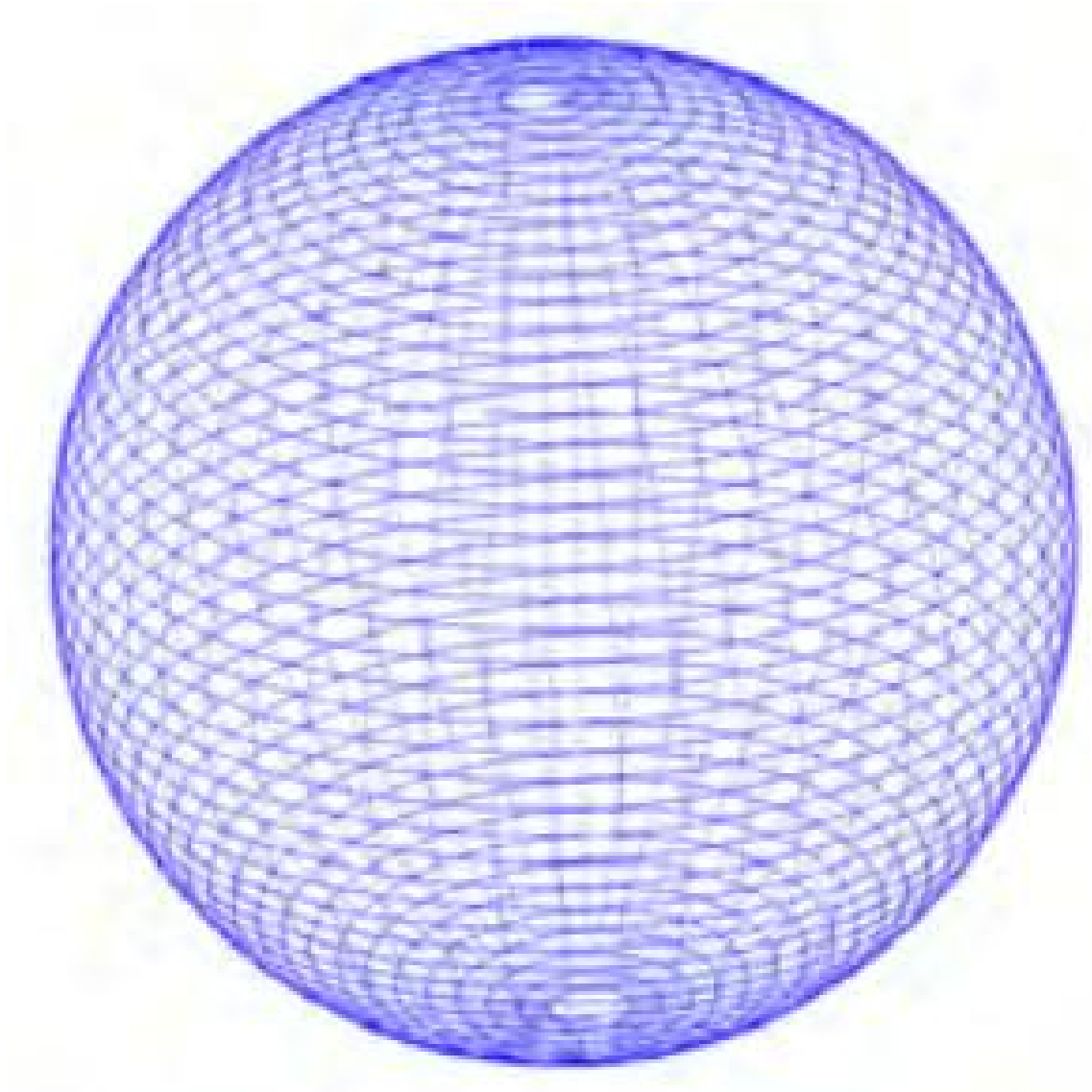
$$F(\tilde{\mathbf{Q}}\mathbf{R}^\top\mathbf{R}) - p_{\text{SDP}}^* \geq F(\tilde{\mathbf{Q}}\mathbf{R}^\top\mathbf{R}) - p_{\text{MLE}}^*. \quad (6.26)$$

此外，当问题 6.3 的松弛精确时，算法 3 返回的估计 $\hat{\mathbf{T}} = (\hat{\mathbf{t}}, \hat{\mathbf{R}}) \in \text{SE}(d)^n$ 达到这一界：

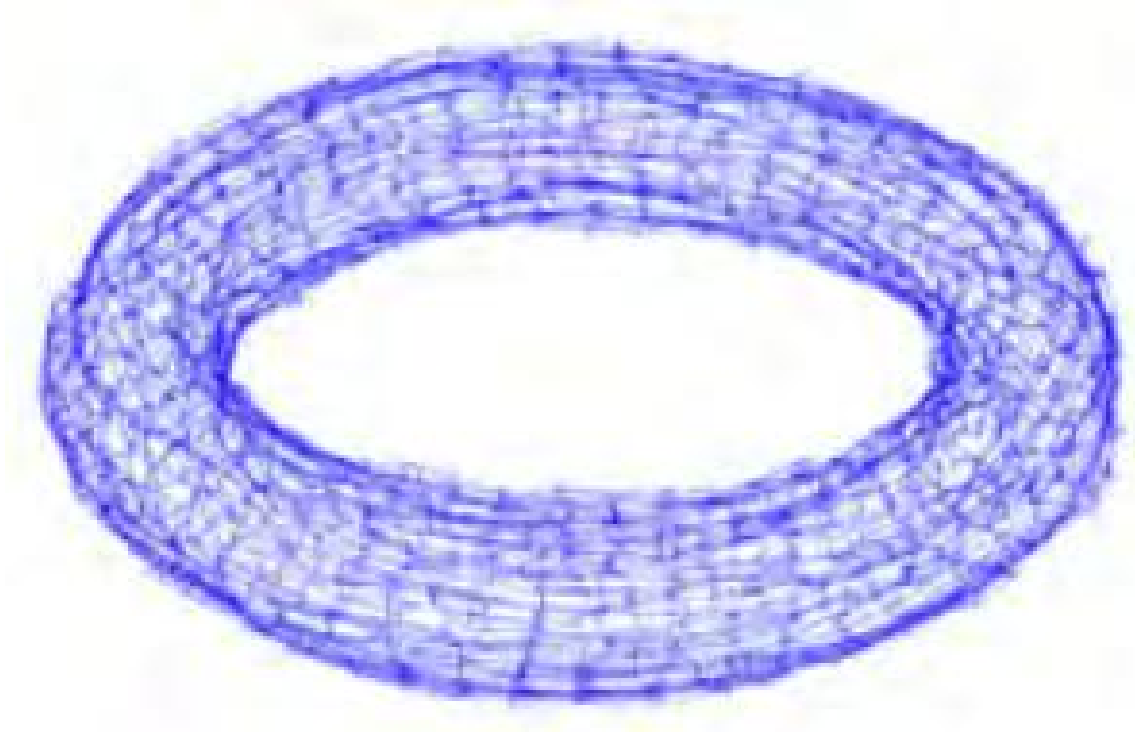
$$F(\tilde{\mathbf{Q}}\hat{\mathbf{R}}^\top\hat{\mathbf{R}}) = p_{\text{SDP}}^*. \quad (6.27)$$

因此，事后验证 (6.27) 成立，即可为 $\hat{\mathbf{T}}$ 作为问题 6.1 解的正确性提供计算证书。由此，SE-Sync 正如所声称的那样，是位姿图优化的可认证正确算法。

SE-Sync 获得的可认证最优结果示例见图 6.3，详细讨论见 [937]。文献 [937] 还报告了运行时间分析，表明该算法至少与传统局部求解器一样快，甚至更快。



- (a) 球面
- (b) 环面
- (c) 网格
- (d) 车库
- (e) 隔间



(f) 环

图 6.3 位姿图优化基准数据集的全局最优解 (源自 [937])。

6.10 基于路标的 SLAM

上一节说明了如何为 PGO 获得快速可认证算法；本节说明同一推导可扩展到基于路标的 SLAM，具体是机器人对路标进行方位和距离，即相对位置测量的情形。

令 $\mathbf{m}_{n+1}, \dots, \mathbf{m}_{n+\ell} \in \mathbb{R}^d$ 为除 n 个机器人位姿外还需估计的 ℓ 个路标位置。事实证明，这些新的路标变量可以方便地并入 SE-Sync 的现有实现。为此，将地图点变量视为纯平移变量，即没有旋转分量的位姿。

假设有 N_m 个路标测量 $\{\tilde{\mathbf{m}}_{ik}\}$ ，它们描述相对于机器人第 i 个位姿的路标 $\mathbf{m}_k$ 的相对位置。还假设这些测量受加性零均值各向同性高斯噪声污染，其形式与 (6.8) 中的平移测量相同：

$$\tilde{\mathbf{m}}_{ik} = \mathbf{m}_{ik} + \mathbf{m}_{ik}^\varepsilon, \quad \mathbf{m}_{ik}^\varepsilon \sim \mathcal{N}(0, \mu_{ik}^{-1} I_d). \quad (6.28)$$

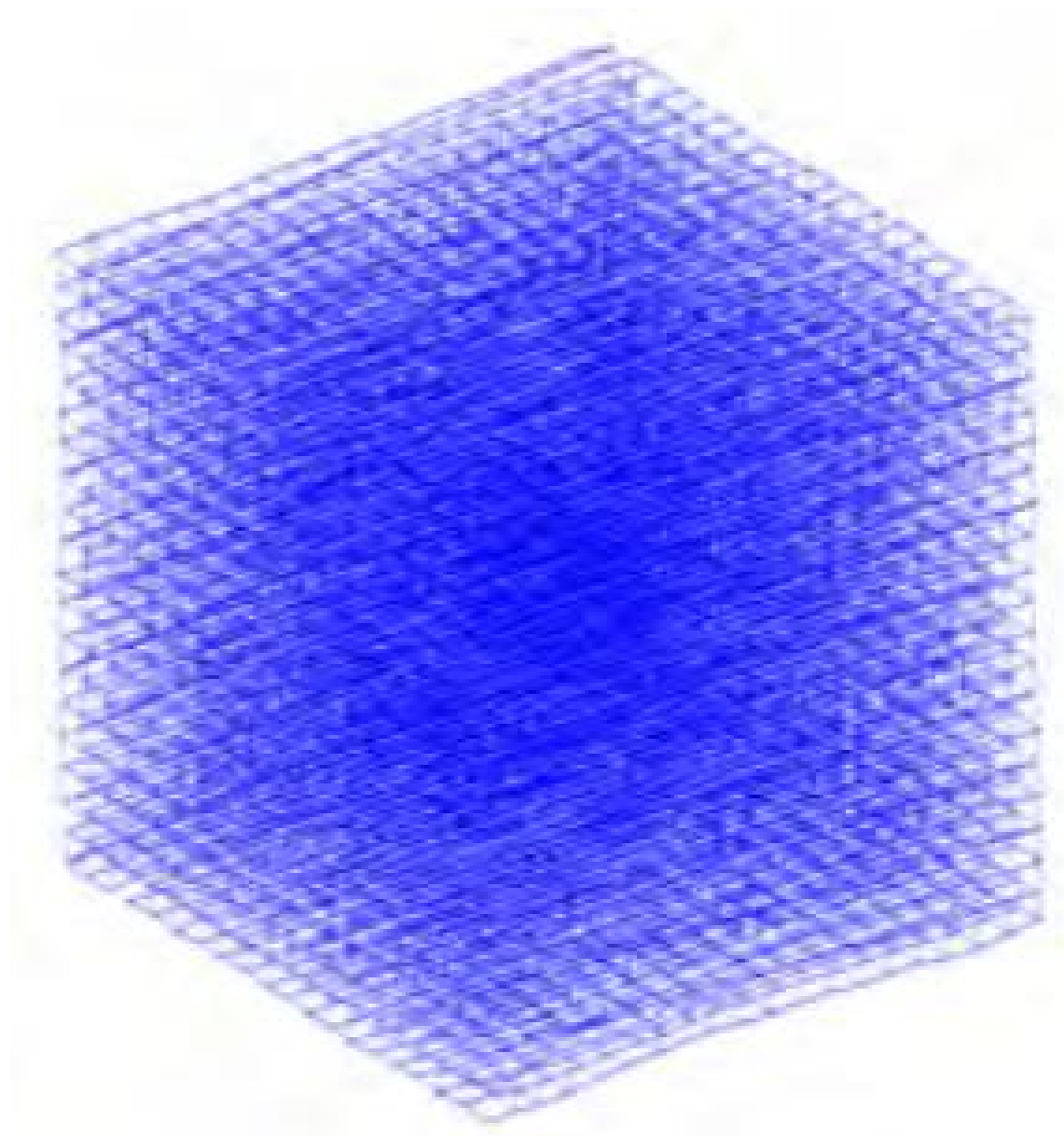
为跟踪这些新测量，向位姿图 $\vec{\mathcal{G}}$ 添加一组新顶点 ν_m 和新边 $\vec{\mathcal{E}}_m$ ，它们分别与地图变量和路标测量一一对应。此外，以 $\vec{\mathcal{E}}_r$ 表示对应机器人位姿相对位姿测量（例如里程计）的边；假设这些测量遵循与 (6.8) 相同的测量模型。

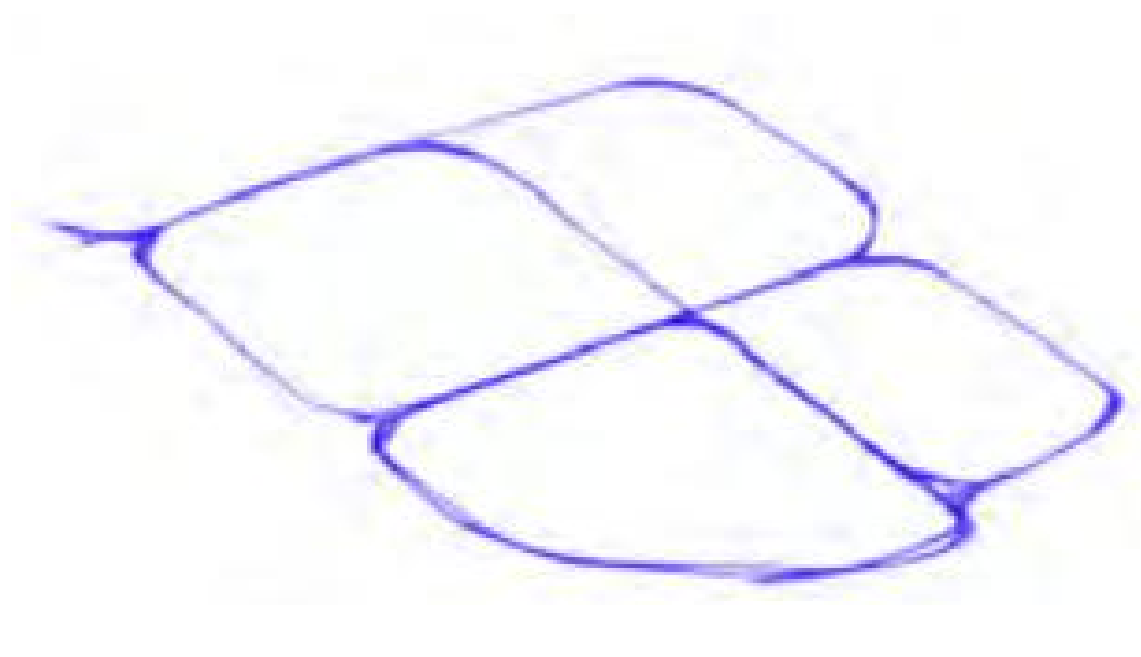
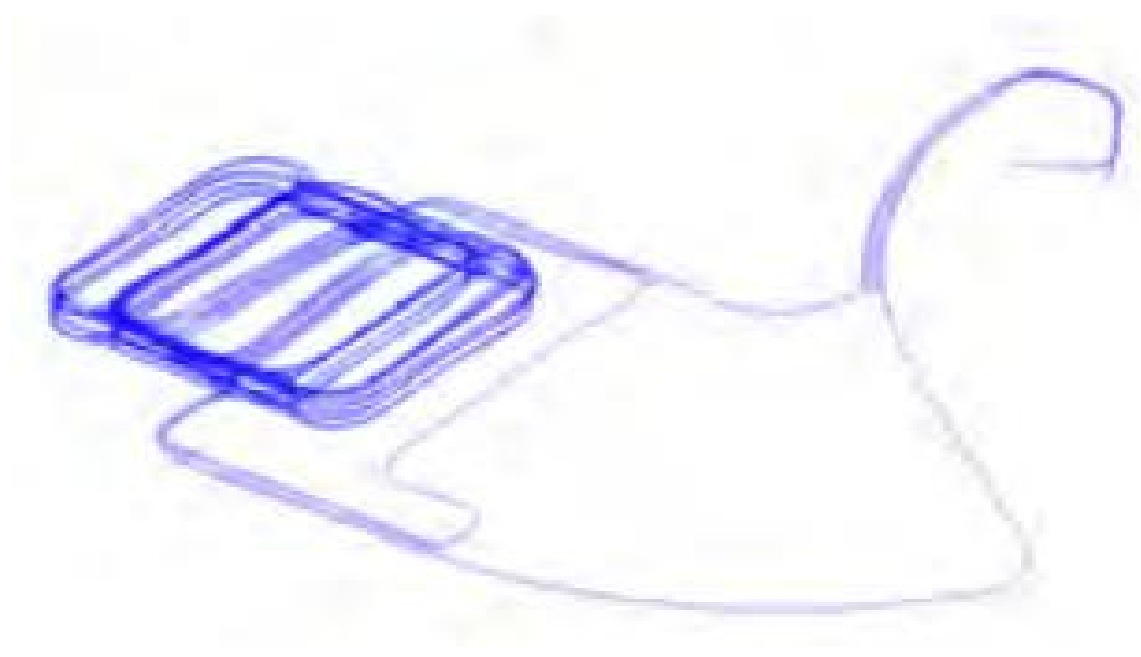
给定路标测量和相对位姿测量，最大似然估计问题变为：

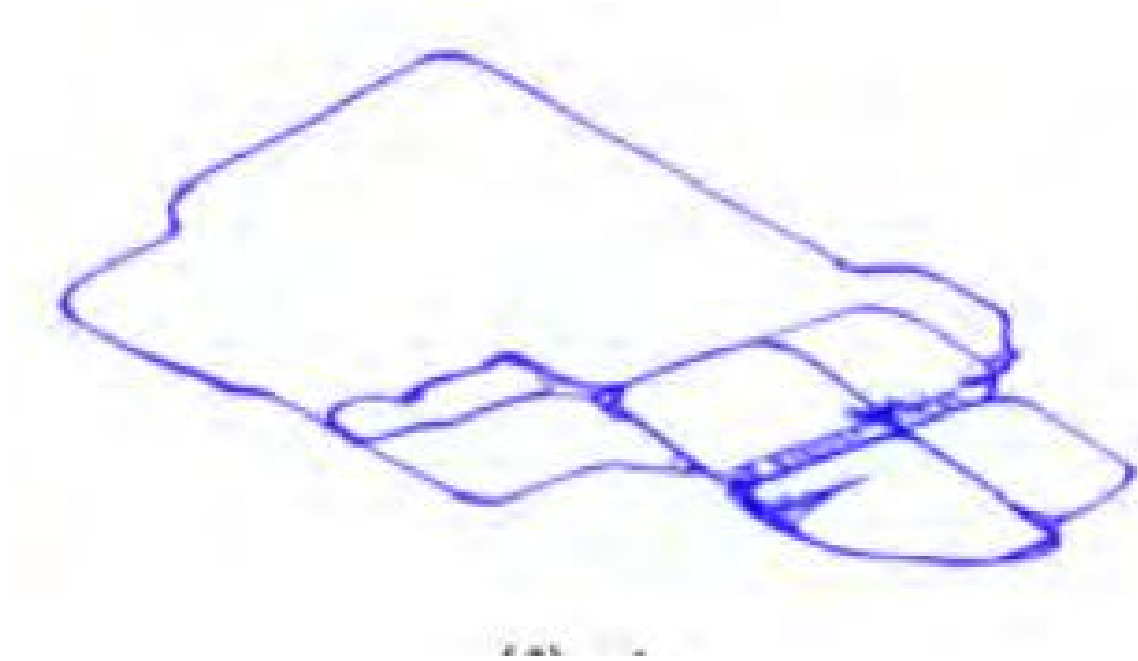
问题 6.5 (基于路标的 SLAM)

$$\begin{aligned} \min_{\substack{\mathbf{R}_i \in \text{SO}(d) \\ \mathbf{t}_i \in \mathbb{R}^d, \mathbf{m}_k \in \mathbb{R}^d}} & \sum_{(i,j) \in \vec{\mathcal{E}}_r} \kappa_{ij} \|\mathbf{R}_j - \mathbf{R}_i \tilde{\mathbf{R}}_{ij}\|_F^2 + \tau_{ij} \|\mathbf{t}_j - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{t}}_{ij}\|_2^2 \\ & + \sum_{(i,k) \in \vec{\mathcal{E}}_m} \mu_{ik} \|\mathbf{m}_k - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{m}}_{ik}\|_2^2. \end{aligned} \quad (6.29)$$

若将地图变量当作位姿平移变量，则上一节的其余推导可原样使用。通过应用 SE-Sync 算法可找到最优旋转变量，其中代价矩阵 $\tilde{Q}$ 经过修改以纳入地图变量的影响。特别地，权重矩阵 Ω 和测量矩阵 $\tilde{D}$ 重定义为：







$$\begin{aligned} \Omega &= \text{BlockDiag}(\Omega_\tau, \Omega_\mu), \\ \Omega_\tau &= \text{Diag}(\tau_1, \dots, \tau_{N_p}), \\ \Omega_\mu &= \text{Diag}(\mu_1, \dots, \mu_{N_m}), \end{aligned} \quad \tilde{D}_{ek} = \begin{cases} -\tilde{\mathbf{t}}_{kj}^T, & e = (k, j) \in \vec{\mathcal{E}}_r, \\ -\tilde{\mathbf{m}}_{kj}^T, & e = (k, j) \in \vec{\mathcal{E}}_m, \\ \mathbf{0}_{1 \times d}, & \text{otherwise} \end{cases} \quad (6.30)$$

此外，投影矩阵 Π 也被修改，以计入新测量图 $\vec{\mathcal{G}} = (\mathcal{V} \cup \mathcal{V}_m, \vec{\mathcal{E}}_r \cup \vec{\mathcal{E}}_m)$ 的形式。注意，connection Laplacian $L(\tilde{G}^\rho)$ 不受新地图测量影响。

一旦找到最优旋转，位姿平移和地图点均可类似 (6.16) 以闭式恢复：

$$\begin{bmatrix} \mathbf{t}^{*T} & \mathbf{m}^{*T} \end{bmatrix}^T = -\text{vec} \left(\mathbf{R}^* \tilde{\mathbf{V}}^T \mathbf{L}(\mathcal{W}^\tau)^\dagger \right), \quad (6.31)$$

这里假设位姿平移变量和地图点已按适当次序排列，且 $\tilde{\mathbf{V}}$ 和 $\mathbf{L}(\mathcal{W}^\tau)^\dagger$ 也已类似 (6.30) 更新，以纳入地图点测量。

典型 SLAM 问题中地图点常远多于位姿。因此，在 SE-Sync 表述中加入地图变量时，必须确保算法对问题中的地图点数仍保持高效。地图点数很大时，SE-Sync 的瓶颈成为构造代价矩阵 $\tilde{Q}$ 。研究表明，可采用经典 Schur 补技巧，确保该矩阵的构造时间复杂度相对于路标数是线性的^[465]。图 6.4(b) 清楚展示了这种线性关系，同时表明 SE-Sync 的其他组成部分的计算代价不随路标数增加。

最后，已证明在该表述中加入路标会影响 SDP 松弛的精确性。特别是，增加路标数和位姿到地图测量图的连通性均能改善松弛精确性。更具体地说，它们提高给定问题拥有精确松弛时所允许的噪声水平。图 6.4(c) 展示了这些参数对一个简单 SLAM 问题的影响。

6.11 扩展：距离测量、各向异性噪声与离群值

本节说明，上工具，即 Shor 松弛和黎曼阶梯求解器，可扩展至其他 SLAM 问题（第 6.1.4.1 节）。此外，讨论用于得到半正定松弛的更一般工具，可将其理解为 Shor 松弛的推广；它们进一步扩展 SLAM 可认证算法的范围，但在求解所得 SDP 时带来额外挑战（第 6.1.4.2 节）。

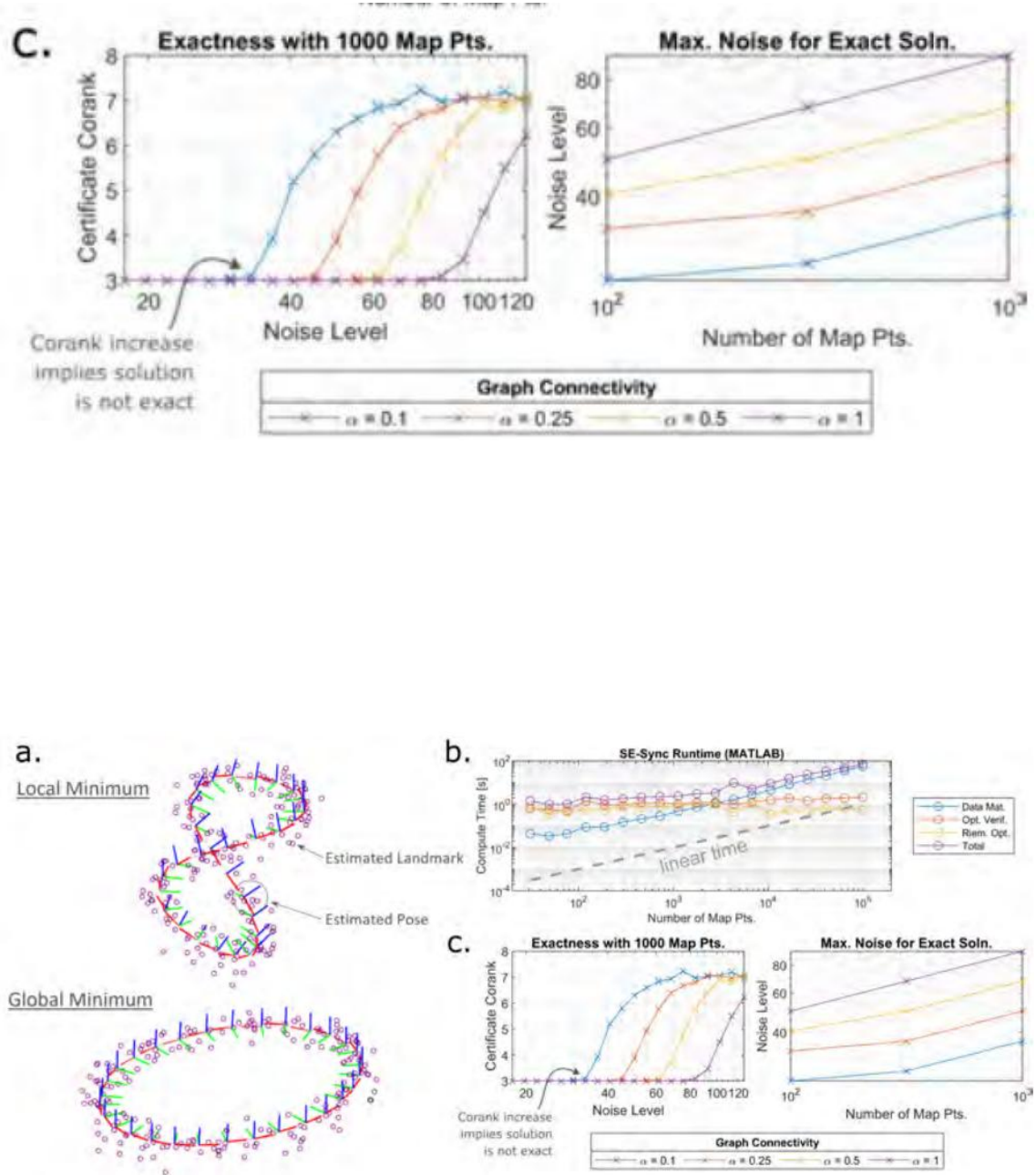


图 6.2: 图 6.4 (a) 简单基于路标 SLAM 问题中局部极小值与全局极小值的示例。(b) (a) 所示示例的运行时间。运行时间随地图规模线性增加, 瓶颈是数据矩阵 $\tilde{Q}$ 的构造 (蓝色所示)。(c) 通过证书矩阵的余秩研究松弛精确性 (当该度量为三时精确)。测量噪声设置为基准标准差, 即平移 0.866 米、旋转 0.573 度, 并按图中 “Noise Level” 乘数缩放。通常, 问题保持精确时的噪声水平随地图点数和位姿到地图连通性增加而提高。源自 ^[465] (©2023 IEEE)。

6.12 面向距离辅助 SLAM 的快速可认证算法

上一节展示了如何为测量为相对位置和相对旋转的 SLAM 问题开发可认证算法。这里依据^[840]的结果说明，同一方法可应用于包含距离测量的问题。具体地，假设可测量变量 i 和 j 之间的距离 $\tilde{r}_{ij}$ ，例如两个机器人位姿之间，或路标与机器人位姿之间的距离。距离辅助 SLAM 的优化问题可表述如下：

$$\begin{aligned} \text{问题 6.6 (距离辅助 SLAM)} \\ \min_{\substack{\mathbf{R}_i \in \text{SO}(d) \\ \mathbf{t}_i \in \mathbb{R}^d}} \sum_{(i,j) \in \tilde{\mathcal{E}}_d} \kappa_{ij} \|\mathbf{R}_j - \mathbf{R}_i \tilde{\mathbf{R}}_{ij}\|_F^2 + \tau_{ij} \|\mathbf{t}_j - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{t}}_{ij}\|_2^2 \\ + \sum_{(i,j) \in \tilde{\mathcal{E}}_d} \gamma_{ij} (\|\mathbf{t}_j - \mathbf{t}_i\| - \tilde{r}_{ij})^2 \end{aligned} \quad (6.32)$$

问题 6.6 目标函数的第一行与前几节所用相同，即相对位姿测量，例如对应机器人里程计。该问题的区别是第二行包含距离测量 $\tilde{r}_{ij}$ 的项（ $\tilde{\mathcal{E}}_d$ 是存在距离测量的节点对 (i, j) 的集合），并以这些测量的方差倒数 γ_{ij} 加权。仅距离测量 $\tilde{r}_{ij}$ 的独特之处在于，它给出位姿 i 和 j 之间相对距离的信息，却不提供方位或相对朝向信息。问题 6.6 可建模多种实际 SLAM 问题，从对路标进行距离测量的基于路标 SLAM，到机器人之间进行相对距离测量的多机器人 SLAM。⁵

将 Shor 松弛应用于问题 6.6 的困难在于，(6.32) 不是 QCQP：展开平方项 $(\|\mathbf{t}_j - \mathbf{t}_i\| - \tilde{r}_{ij})^2$ 可知，该表达式因未平方范数项而不是关于变量 $\mathbf{t}_i$ 和 $\mathbf{t}_j$ 的二次式。为解决此问题，文献^[840]提出一个优雅重表述，引入辅助单位向量 $\mathbf{b}_{ij}$ ，满足 $\|\mathbf{b}_{ij}\| = 1$ ：

$$\begin{aligned} \min_{\substack{\mathbf{R}_i \in \text{SO}(d) \\ \mathbf{t}_i \in \mathbb{R}^d \\ \mathbf{b}_{ij} \in S^{d-1}}} \sum_{(i,j) \in \tilde{\mathcal{E}}_d} \kappa_{ij} \|\mathbf{R}_j - \mathbf{R}_i \tilde{\mathbf{R}}_{ij}\|_F^2 + \tau_{ij} \|\mathbf{t}_j - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{t}}_{ij}\|_2^2 \\ + \sum_{(i,j) \in \tilde{\mathcal{E}}_d} \gamma_{ij} \|\mathbf{t}_j - \mathbf{t}_i - \tilde{r}_{ij} \mathbf{b}_{ij}\|_2^2, \end{aligned} \quad (6.33)$$

直观地，表述 (6.33) 也会推断具有距离测量 $\tilde{r}_{ij}$ 的变量 i 与 j 之间的方位 $\mathbf{b}_{ij}$ ，进而将目标函数中相应项 $\|\mathbf{t}_j - \mathbf{t}_i - \tilde{r}_{ij} \mathbf{b}_{ij}\|_2^2$ 重写成距离-方位测量，与前几节所见测量相似。这样做的优点是 (6.33) 现在是 QCQP：目标函数最后一项关于未知量是二次的，新增的 $\mathbf{b}_{ij}$ 单位范数约束 $\|\mathbf{b}_{ij}\|^2 = 1$ 也是二次的。随后可用 Shor 松弛将新表述松弛为 SDP，并用黎曼阶梯高效求解，得到 CORA^[840]，即面向距离辅助 SLAM 的快速可认证算法。

与先前考虑的位姿和路标 SLAM 问题不同，距离辅助 SLAM 问题 (6.33) 的 Shor 松弛在有界测量噪声下通常并不精确。具体而言，^[840]的经验研究显示，在多机器人情形中，除非机器人之间存在相对位姿测量，否则该松弛通常不精确，即机器人之间仅距离测量不足以获得精确松弛。不过，该松弛在多种实际 SLAM 问题中仍然精确，且能以可与局部求解器相比的运行时间计算可认证最优解。更一般地，Papalia 等人^[840]表明，SLAM 问题底层图的连通性在很大程度上影响 SDP 松弛的精确性，^[937, 465]也观察到这一现象。第 6.2 节将看到，图连通性还影响 SLAM 估计的精度，从而带来关于 SLAM 问题非常有意义的图论见解。

6.13 超越 Shor 松弛的可认证算法：各向异性噪声与离群值

迄今所回顾的是可重表述为 QCQP 的 SLAM 问题的可认证算法及快速求解器。下面考察更广一类 SLAM 问题，即具有各向异性测量噪声的问题和含离群值的问题。其有趣之处在于：严格说来它们不再是 QCQP，却常可表述为多项式优化问题 (polynomial optimization problems, POPs)，其中目标函数和约束均为多项式函数，而不只是二次函数。这一点很重要，因为 Shor 松弛存在一种推广，即矩 (或 Lasserre) 松弛，可为 POP 推导 SDP 松弛，从而能够为更广泛问题设计可认证算法。下面回顾可写为 POP 的 SLAM 问题示例，随后概述矩松弛及实际考量。

示例：具有各向异性噪声的基于路标 SLAM。前几节假设测量噪声各向同性。然而，常见传感模式产生的测量，例如双目相机、LiDAR 和雷达，通常具有各向异性噪声。例如，双目相机观察到的路标位置测量，通常沿相机视线方向更不确定，这是由双目匹配和三角化过程产生的不确定性所致。形式上，路标测量的测量模型变为 $\tilde{m}_{ik} = \mathbf{R}_i^\top (\mathbf{m}_k - \mathbf{t}_i) + \epsilon_{ik}$ ，其中 $\epsilon_{ik} \sim \mathcal{N}(\mathbf{0}, \mathbf{W}_{ik})$ ，且 $\mathbf{W}_{ik}$ 是各向异性协方差，即不能写成单位矩阵的标量倍数。在各向异性噪声下，须将基于路标的 SLAM 问题 6.5 推广为^[465]：

问题 6.7 (具有各向异性噪声的基于路标 SLAM)

$$\begin{aligned} \min_{\substack{\mathbf{R}_i \in \text{SO}(d) \\ \mathbf{t}_i \in \mathbb{R}^d, \mathbf{m}_k \in \mathbb{R}^d}} \quad & \sum_{(i,j) \in \vec{\mathcal{E}}} \kappa_{ij} \|\mathbf{R}_j - \mathbf{R}_i \tilde{\mathbf{R}}_{ij}\|_F^2 + \tau_{ij} \|\mathbf{t}_j - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{t}}_{ij}\|_2^2 \\ & + \sum_{(i,k) \in \vec{\mathcal{E}}_m} \|\mathbf{R}_i^\top (\mathbf{m}_k - \mathbf{t}_i) - \tilde{\mathbf{m}}_{ik}\|_{\mathbf{W}_{ik}}^2 \end{aligned} \quad (6.34)$$

在 (6.34) 中, $\vec{\mathcal{E}}_m$ 表示对应路标测量的边集合; 对于维度适当的一般向量 $\mathbf{a}$ 和矩阵 $\mathbf{W}$, 记号 $\|\mathbf{a}\|_W^2 = \mathbf{a}^\top \mathbf{W} \mathbf{a}$ 表示标准 Mahalanobis 平方范数。相对于问题 6.5, 这一看似无害的改变在实践中影响重大。实际上, 若 $\mathbf{W}_{ik}$ 各向同性, 例如 $\mathbf{W}_{ik} = \mu_{ik}^{-1} \mathbf{I}_3$, 可将 (6.34) 变换为问题 6.5 的 QCQP; 而若 $\mathbf{W}_{ik}$ 各向异性, 则该问题为四次问题, 即涉及变量的四次多项式。⁶

示例: 含离群值的 SLAM。至此假设所有测量均受零均值、但可能各向异性的高斯噪声影响。不幸的是, 如第 3 章所述, 真实 SLAM 问题的部分测量可能为离群值。这通常发生在回环或路标测量中: 错误地点识别或数据关联会使不正确测量加入 SLAM 后端。如第 3 章所述, 缓解离群值影响的有效方法是使用鲁棒损失函数。例如, 存在离群值时, 基于路标的 SLAM 问题 6.5 变为:

问题 6.8 (含离群值的位姿图优化)

$$\begin{aligned} \min_{\substack{\mathbf{R}_i \in \text{SO}(d) \\ \mathbf{t}_i \in \mathbb{R}^d, \mathbf{m}_k \in \mathbb{R}^d}} \quad & \sum_{(i,j) \in \vec{\mathcal{E}}_o} \kappa_{ij} \|\mathbf{R}_j - \mathbf{R}_i \tilde{\mathbf{R}}_{ij}\|_F^2 + \tau_{ij} \|\mathbf{t}_j - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{t}}_{ij}\|_2^2 \\ & + \sum_{(i,k) \in \vec{\mathcal{E}}_m} \rho(\sqrt{\mu_{ik}} \|\mathbf{m}_k - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{m}}_{ik}\|_2). \end{aligned} \quad (6.35)$$

在 (6.35) 中, $\rho(\cdot)$ 是鲁棒损失函数 (参见第 3 章), 其设计目标是降低路标测量中潜在离群值的影响。由第 3 章可知, 可用 Black-Rangarajan 对偶性将 (6.35) 重表述为带辅助变量 w_{ik} 的最小二乘问题, 每个鲁棒损失项对应一个辅助变量。例如, 当选取 $\rho(\cdot)$ 为截断二次损失时, (6.35) 可改写为

$$\begin{aligned} \min_{\substack{\mathbf{R}_i \in \text{SO}(d) \\ \mathbf{t}_i \in \mathbb{R}^d \\ \mathbf{m}_k \in \mathbb{R}^d \\ w_{ik} \in [0, 1]}} \quad & \sum_{(i,j) \in \vec{\mathcal{E}}_o} \kappa_{ij} \|\mathbf{R}_j - \mathbf{R}_i \tilde{\mathbf{R}}_{ij}\|_F^2 + \tau_{ij} \|\mathbf{t}_j - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{t}}_{ij}\|_2^2 \\ & + \sum_{(i,k) \in \vec{\mathcal{E}}_m} w_{ik} \mu_{ik} \|\mathbf{m}_k - \mathbf{t}_i - \mathbf{R}_i \tilde{\mathbf{m}}_{ik}\|_2^2 + (1 - w_{ik}) \beta^2, \end{aligned} \quad (6.36)$$

其中 β 是截断二次损失指定的最大内点误差。注意, (6.36) 现在包含辅助变量 w_{ik} , 它们指示一个测量被划分为内点还是离群值。有趣的是, (6.36) 的目标函数包含最高三次多项式, 而约束仍至多为二次函数。文献^[1221]表明, 对多种鲁棒损失选择和其他问题变体, 包括位姿图优化与多旋转平均, 同一结论均成立。下面讨论如何为涉及多项式的优化问题获得半正定松弛。

多项式优化问题与矩松弛。上述例子表明, 广泛的 SLAM 问题可以重表述为涉及多项式的优化问题。形式上, 它们可以写为 POP:

$$\begin{aligned} \min_{\mathbf{x}} \quad & p(\mathbf{x}) \\ \text{subject to} \quad & h_i(\mathbf{x}) = 0, i = 1, \dots, n_h \\ & g_i(\mathbf{x}) \leq 0, i = 1, \dots, n_g, \end{aligned} \quad (\text{POP})$$

其中函数 p, h_i, g_i 是变量 $\mathbf{x}$ 的实多项式, n_h 和 n_g 分别是等式和不等式约束的数量。

将 SLAM 问题重写为 (POP) 并不立即意味着计算优势: POP 是非常一般的优化问题类别, 也包含 QCQP, 且一般难以求解。我们关注 (POP), 是因为存在标准过程, 即矩 (或 Lasserre) 松弛, 可获得 (POP) 的半正定松弛。更有意义的是, 该过程提供构造松弛层级的工具, 并保证在温和假设下, 该层级中的某些松弛精确。关于矩松弛的更全面介绍, 请参阅^[147]和奠基性工作^[627, 626]; 下面给出一个简单例子来说明其基本思想。

为说明如何获得 POP 的松弛, 考虑问题:

$$\begin{aligned} & \min_{\mathbf{x}} p(\mathbf{x}) \\ & \text{subject to } h_i(\mathbf{x}) = 0, i = 1, \dots, n_h \end{aligned} \quad (6.37)$$

为简单起见, 假设 $\mathbf{x} = [x_1; x_2]$, 且 $p(\mathbf{x})$ 和 $h(\mathbf{x})$ 都是次数至多为 4 的多项式; 在这个简化例子中只有等式约束。

为导出 (POP) 的矩松弛, 定义次数至多为 $r = 2$ 的单项式向量, 其中 r 称为松弛阶:

$$[\mathbf{x}]_2 = [1; x_1; x_2; x_1^2; x_1x_2; x_2^2]. \quad (6.38)$$

随后可以如下外积构造矩阵:

$$\mathbf{X}_4 \triangleq [\mathbf{x}]_2 [\mathbf{x}]_2^\top = \begin{bmatrix} 1 & x_1 & x_2 & x_1^2 & x_1x_2 & x_2^2 \\ x_1 & x_1^2 & x_1x_2 & x_1^3 & x_1^2x_2 & x_1x_2^2 \\ x_2 & x_1x_2 & x_2^2 & x_1^2x_2 & x_1x_2^2 & x_2^3 \\ x_1^2 & x_1^3 & x_1^2x_2 & x_1^4 & x_1^3x_2 & x_1^2x_2^2 \\ x_1x_2 & x_1^2x_2 & x_1x_2^2 & x_1^3x_2 & x_1^2x_2^2 & x_1x_2^3 \\ x_2^2 & x_1x_2^2 & x_2^3 & x_1^2x_2^2 & x_1x_2^3 & x_2^4 \end{bmatrix}. \quad (6.39)$$

关键观察是, 次数至多为 4 的任意多项式均可写成 $\mathbf{X}_4$ 元素的线性组合。因此, 可将 (6.37) 重写为:

$$\begin{aligned} & \min_{\mathbf{X}, \mathbf{x}} \text{tr}(\mathbf{C}\mathbf{X}) \\ & \text{subject to } \text{tr}(\mathbf{H}_i\mathbf{X}) = 0, i = 1, \dots, n_h \\ & \mathbf{X} = [\mathbf{x}]_2 [\mathbf{x}]_2^\top \end{aligned} \quad (6.40)$$

完全类似 Shor 松弛, 现在可将约束 $\mathbf{X} = [\mathbf{x}]_2 [\mathbf{x}]_2^\top$ 替换为 $\mathbf{X} \succeq 0$ 和 $\text{rank}(\mathbf{X}) = 1$, 然后松弛约束以获得半正定松弛。重要的是, 矩松弛还会加入冗余约束⁷来提高质量。这些约束刻画矩阵中包含重复元素这一事实, 例如项 x_1x_2 在 (6.40) 中出现 3 次; 以及当 $h_i(\mathbf{x}) = 0$ 时, $x_1 \cdot h_i(\mathbf{x}) = 0$ 和 $x_2 \cdot h_i(\mathbf{x}) = 0$ 也必须成立这一事实。矩松弛提供系统方法识别所有这些冗余约束, 并扩展到不等式约束情形。此外, 尽管上文导出的是 $r = 2$ 阶松弛, 但可对任意整数 $r \geq 2$ 重复该过程, 获得更大但更好的 SDP 松弛, 即一个松弛层级。奠基性工作^[806]表明, 在温和假设下, 矩松弛会在有限阶 r 产生精确松弛, 即恢复原始 POP 的可认证最优解。有趣的是, 相关工作^[1221, 465]观察到, 各向异性噪声和离群值问题的矩松弛在较低松弛阶即已精确。即使在单项式基 (6.38) 中只使用部分变量, 即使用稀疏单项式基时, 它们仍保持紧致, 这进一步缩小所得 SDP 的规模。

“代价”: 求解 SDP 矩松弛。前几节的松弛与本节松弛的一个重要区别是, 后者的 SDP 松弛包含大量冗余约束。该差异看似很小, 却对 SDP 求解器造成深远后果。特别地, 带冗余约束的 SDP, 例如矩松弛通常得到的 SDP, 是退化的^[27], 这使黎曼阶梯求解器因两个原因而难以应用。第一, 这些问题通常不满足约束资格条件, 而该条件是保证黎曼阶梯收敛所必需的^[841]。第二, 退化 SDP 具有无穷多个对偶解, 这在实现黎曼阶梯时造成计算障碍。换言之, 黎曼阶梯不再是通常由矩松弛产生的退化 SDP 的可行求解器。尽管近期文献已包含 POP 矩松弛的专用求解器^[1220], 它们相较局部求解器仍然较慢。

6.14 SLAM 问题最优解的准确性如何?

上一节讨论了如何为某些 SLAM 问题获得可认证最优解。然而, 另一个基本问题仍然存在: 相对于真值, 最优解究竟有多准确? 理解这一基本极限并识别影响估计准确性的关键因素, 可为系统设计人员和最终用户提供重要指导。例如, 在设计阶段, 这些认识可指导机器人所配备传感器的选择; 而在部署阶段, 则可根据此引导机器人的运动 (以及相应的测量采集), 以确保鲁棒的 SLAM 性能。在给出具体答案之前, 首先需要形式化该问题。现代 SLAM 流水线较为复杂, 许多相互作用的子系统会影响整体性能。因此, 我们从估计理论的角度研究此问题, 聚焦于 SLAM 后端。

一些术语与事实。SLAM 后端的目标是根据含噪测量估计未知量, 例如机器人位姿和路标位置。由于受到随机传感器噪声的污染, 这些测量是随机变量。估计器是将收集到的含噪测量映射为未知参数估计值的函

数。由于估计器依赖于随机测量，它本身也是随机变量。均方误差 (Mean Squared Error, MSE) 是评估估计器性能的常用指标。顾名思义，MSE 表示对所有可能测量求平均的平方估计误差。在单变量情形下，MSE 可表示为估计器的偏差平方与方差之和。偏差是指在所有可能测量上求平均后，真值与估计器输出之间的差异。方差刻画估计器输出在其均值附近的变动性。偏差与方差之间的关系及 MSE 的定义可自然推广至多变量情形，此时以协方差矩阵替代方差。MSE 仍为偏差的平方范数与协方差矩阵迹之和。显然，对于无偏估计器（即偏差为零的估计器），唯一影响 MSE 的量是协方差矩阵。下文将给出 SLAM 估计协方差的下界，从而得到 SLAM 系统可达到准确性的基本极限。

6.15 Cramer–Rao 下界与 Fisher 信息矩阵

Cramer–Rao 下界 (Cramer–Rao Lower Bound, CRLB) 给出了任意无偏估计器所能达到的最佳协方差的理论极限。形式上，

$$\text{Cov}(\hat{\mathbf{x}}) \succeq \mathcal{I}(\mathbf{x}_{\text{true}})^{-1}, \quad (6.41)$$

其中， $\hat{\mathbf{x}}$ 是 $\mathbf{x}_{\text{true}} \in \mathbb{R}^m$ 的任意无偏估计器； $A \succeq B$ 表示 $A - B$ 为半正定矩阵。式 (6.41) 右侧的矩阵 $\mathcal{I}(\mathbf{x}_{\text{true}})$ 是 Fisher 信息矩阵 (Fisher information matrix, FIM)，其定义为⁸

$$[\mathcal{I}(\mathbf{x}_{\text{true}})]_{i,j} \triangleq \mathbb{E}_{\mathbf{z}} \left[\frac{\partial}{\partial x_i} \log p(\mathbf{z}; \mathbf{x}) \frac{\partial}{\partial x_j} \log p(\mathbf{z}; \mathbf{x}) \right]. \quad (6.42)$$

这里的期望是对从概率密度函数 $p(\mathbf{z}; \mathbf{x}_{\text{true}})$ 中抽取的测量 $\mathbf{z}$ 的可能实现取的，且对数似然函数的偏导数在真参数值 $\mathbf{x}_{\text{true}}$ 处求值。在某些正则性条件下，FIM 也可表示为对数似然 Hessian 矩阵的期望：

$$[\mathcal{I}(\mathbf{x}_{\text{true}})]_{i,j} = -\mathbb{E}_{\mathbf{z}} \left[\frac{\partial^2}{\partial x_i \partial x_j} \log p(\mathbf{z}; \mathbf{x}) \right]. \quad (6.43)$$

与 Hessian 的这一联系为理解 CRLB 提供了直观视角：CRLB 以真参数值 $\mathbf{x}_{\text{true}}$ 附近的参数局部敏感性（曲率）来表示任意无偏估计器协方差的下界，这里的曲率针对（期望）对数似然函数而言。若在不同测量实现下（即在期望意义上），对数似然在真参数值附近相对平坦，则任意无偏估计器都难以根据观测数据准确定位真参数。在这种情况下，无偏估计器将具有较大的方差，从而导致较大的 MSE。总之，FIM 刻画了利用任意无偏估计器可从测量中获得的、关于真参数值的信息量。

在某些正则性条件下，已有结果表明，当测量数量趋于无穷时，最大似然估计器 $\hat{\mathbf{x}}_{\text{mle}}$ 渐近地（即在分布意义下）收敛到 $\mathcal{N}(\mathbf{x}_{\text{true}}, \mathcal{I}(\mathbf{x}_{\text{true}})^{-1})$ 。因此，最大似然估计器是渐近无偏的，并达到 CRLB（即在所有无偏估计器中具有最小方差）。由于参数的真值 $\mathbf{x}_{\text{true}}$ 未知，通常以 $\mathcal{I}(\hat{\mathbf{x}}_{\text{mle}})$ 近似 FIM。此外，常以 $\mathcal{I}(\hat{\mathbf{x}}_{\text{mle}})^{-1}$ 近似最大似然估计器 $\hat{\mathbf{x}}_{\text{mle}}$ 的协方差。

作为示例，考虑常见情形：测量由平滑（可能非线性）函数叠加加性高斯噪声生成。此时测量模型可写为

$$\mathbf{z} = \mathbf{h}(\mathbf{x}_{\text{true}}) + \epsilon, \quad (6.44)$$

其中， $\epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma)$ 为噪声。因此，在 $\mathbf{x}$ 处求值的似然函数为 $p(\mathbf{z}; \mathbf{x}) = \mathcal{N}(\mathbf{h}(\mathbf{x}), \Sigma)$ 。将此似然函数代入式 (6.42)，可得 FIM（在 $\mathbf{x}$ 处求值）：

$$\mathcal{I}(\mathbf{x}) = \mathbf{J}(\mathbf{x})^\top \Sigma^{-1} \mathbf{J}(\mathbf{x}), \quad (6.45)$$

其中， $\mathbf{J}(\mathbf{x})$ 表示在 $\mathbf{x}$ 处求值的测量模型 $\mathbf{h}$ 的雅可比矩阵。⁹

不确定性的标量度量：最优实验设计准则。在多数应用中，我们希望估计尽可能确定且准确。FIM 量化了确定性，但通常为矩阵值；理想情况下，需要一个能够量化不确定性、以用于决策或评估估计器的单一数值。因此，在许多应用中（例如系统设计、主动 SLAM），需要将作为 PSD 矩阵的 FIM 映射为实数，以捕获估计误差不确定性的某个有意义且“可优化”的方面。最优实验设计领域已研究了标准的选择，包括 FIM 的行列式 (D-最优性)、其逆的迹 (A-最优性) 和其最小特征值 (E-最优性)。这些准则均为 FIM 的谱函数（即可由 FIM 的特征值计算）。每项准则反映估计误差的不同方面：D-最优性准则量化不确定性的超体积，A-最优性准则衡量平均方差，E-最优性准则表示最坏情况下的估计方差。

6.16 Fisher 信息矩阵与图拉普拉斯

本节研究 SLAM 问题的图结构如何影响 FIM。考察式 (6.45) 可得到一个直观事实：测量噪声协方差矩阵 Σ 会影响 FIM；不出所料，较高的测量噪声协方差会提高 CRLB 中所有无偏估计器所能达到的最小 MSE 下界。此外，测量的雅可比矩阵也出现在式 (6.45) 中，但仅由该式难以直观理解测量雅可比矩阵如何影响 CRLB。¹⁰ 下文将把雅可比矩阵及其所得 FIM 与 SLAM 问题底层图的性质联系起来，从而深入理解其结构。

正如第 1 章和本章前文所述，SLAM 的所有变体都天然具有图表示。该图本质上编码了“谁观测谁”，并简洁地概括 SLAM 问题。例如，在位姿图优化和基于路标的 SLAM 中，每个变量（如机器人位姿或路标位置）由一个顶点表示，成对测量（位姿-位姿或位姿-路标）则对应于相应顶点之间的边。

接下来研究 SLAM 问题底层图的性质如何影响所得估计的准确性。特别地，图内的连通程度反映测量中的冗余性。直观而言，“连通性更好”的 SLAM 图由于存在冗余测量，预期对噪声更具鲁棒性，即使在较高噪声水平下也能得到准确估计。尽管由式 (6.45) 容易证明，加入额外测量（即边）总能降低 CRLB 中的下界（按 Loewner 偏序），¹¹ 但不同测量的影响取决于额外测量涉及哪些变量（即所形成的图连通性）。在 SLAM 的闭环情境中这一点尤其明显：闭合“更大”的闭环会更显著地提高 SLAM 解的准确性。一系列工作已将该一直觉形式化，建立了 SLAM 的图结构与估计、优化中理想性质之间的联系。下面简要概述这些发现。

简化 PGO 问题中 FIM 与图拉普拉斯的联系。基于路标的 SLAM 与位姿图优化中的 FIM 与图拉普拉斯密切相关^[561, 560, 877, 190]。鉴于这些框架中的测量由顶点之间的成对相对观测组成，这一关系是直观的。为说明这一概念，下面推导一个较简单问题的 FIM：在机器人朝向已知的三维位姿图优化问题中估计机器人位置。令 $\mathbf{z}_k$ 表示第 k 个相对测量，其中位姿 i_k 在自身局部坐标系中观测位姿 j_k ：

$$\mathbf{z}_k = \mathbf{R}_{i_k}^\top (\mathbf{t}_{j_k} - \mathbf{t}_{i_k}) + \boldsymbol{\epsilon}_k, \quad (6.46)$$

其中， $\boldsymbol{\epsilon}_k \sim \mathcal{N}(\mathbf{0}, w_k^{-1} \mathbf{I}_3)$ ，而 $\mathbf{I}_3$ 是 3×3 单位矩阵。令 $\mathbf{z}$ 、 $\mathbf{t}$ 和 $\boldsymbol{\epsilon}$ 分别为测量、位置和噪声变量的堆叠向量。此外，令 $\mathbf{R}$ 为（已知）旋转矩阵的块对角矩阵，其第 k 个块是第 k 个测量所涉及的旋转矩阵：

$$\mathbf{R} \triangleq \text{BlockDiag}(\mathbf{R}_{i_1}, \mathbf{R}_{i_2}, \dots, \mathbf{R}_{i_m}). \quad (6.47)$$

堆叠测量模型可表示为：

$$\mathbf{z} = \mathbf{R}^\top (\mathbf{A} \otimes \mathbf{I}_3)^\top \mathbf{t} + \boldsymbol{\epsilon}, \quad (6.48)$$

其中， $\mathbf{A}$ 表示位姿图的约化关联矩阵，¹² $\otimes$ 表示 Kronecker 积。因此，堆叠测量模型的雅可比矩阵为

$$\mathbf{J} = \mathbf{R}^\top (\mathbf{A} \otimes \mathbf{I}_3)^\top. \quad (6.49)$$

堆叠噪声向量的信息矩阵为

$$\Sigma^{-1} = \text{BlockDiag}(w_1 \mathbf{I}_3, w_2 \mathbf{I}_3, \dots, w_m \mathbf{I}_3). \quad (6.50)$$

令 $\mathbf{W}$ 表示边权重的对角矩阵：

$$\mathbf{W} \triangleq \begin{bmatrix} w_1 & 0 & \dots & 0 \\ 0 & w_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & w_m \end{bmatrix}. \quad (6.51)$$

使用式 (6.45)，FIM 可按如下方式计算：

$$\mathcal{I} = \mathbf{J}^\top \Sigma^{-1} \mathbf{J} \quad (6.52a)$$

$$= (\mathbf{A} \otimes \mathbf{I}_3) \mathbf{R} \Sigma^{-1} \mathbf{R}^\top (\mathbf{A} \otimes \mathbf{I}_3)^\top \quad (6.52b)$$

$$= (\mathbf{A} \otimes \mathbf{I}_3)(\mathbf{W} \otimes \mathbf{I}_3)(\mathbf{A} \otimes \mathbf{I}_3)^\top \quad (6.52c)$$

$$= (\mathbf{A}\mathbf{W}\mathbf{A}^\top) \otimes \mathbf{I}_3 \quad (6.52d)$$

$$= \mathbf{L}_w \otimes \mathbf{I}_3, \quad (6.52e)$$

这里注意到 Σ^{-1} 的块对角元素在旋转下保持不变, 因此 $\mathbf{R}\Sigma^{-1}\mathbf{R}^\top = \Sigma^{-1}$; 并使用了 $\Sigma^{-1} = \mathbf{W} \otimes \mathbf{I}_3$ 这一事实。在式 (6.52e) 中, $\mathbf{L}_w = \mathbf{A}\mathbf{W}\mathbf{A}^\top$ 是图的约化加权拉普拉斯矩阵, 其边权重为 $w_1, w_2, \dots, w_m$ 。这清楚表明, 在此情形下 (即假定机器人朝向已知且噪声各向同性的简化 PGO), FIM 完全由底层图的约化加权拉普拉斯刻画。

其他 SLAM 问题的 FIM。上述结果可推广到 PGO 和基于路标的 SLAM 问题 [561, 877, 190]。在这些问题中, FIM 同时涉及图的约化加权拉普拉斯矩阵和依赖于机器人轨迹的附加项。特别地, 三维位姿图优化的 FIM 可写为 [877, Eq. 29]:

$$\mathcal{I}(\mathbf{x}) = \sum_{k=1}^m \mathbf{L}_k \otimes \left(\text{Ad}(\mathbf{T}_{i_k}^{-1})^\top \Sigma_k^{-1} \text{Ad}(\mathbf{T}_{i_k}^{-1}) \right), \quad (6.53)$$

其中, $\mathbf{T}_{i_k}$ 是进行第 k 个测量的机器人的位姿, Σ_k 是污染第 k 个测量的噪声协方差矩阵, $\mathbf{L}_k$ 是第 k 条边的约化基本拉普拉斯矩阵, 定义为 $\mathbf{L}_k \triangleq \mathbf{a}_k \mathbf{a}_k^\top$, 其中 $\mathbf{a}_k$ 是图的约化关联矩阵的第 k 列。实证结果与理论分析表明, 在某些条件下, 由约化拉普拉斯得到的 (近似) 最优设计准则与由 FIM 得到的准则高度一致 [561, 877, 190]。

实践考量。FIM 与图拉普拉斯之间的联系允许使用约化加权拉普拉斯的谱来近似最优设计准则, 即 FIM 的谱函数。例如, D-最优性准则可由约化加权拉普拉斯行列式的函数近似。根据 Kirchhoff 矩阵树定理, 该行列式等于图的加权生成树数量 [561]。由于加权生成树数量可度量边加权图的连通性, 这一结果形式化了图的连通性直接影响估计准确性的直觉。类似地, E-最优性准则与图的代数连通度相关 [275, 561, 877]。总体而言, 使用图拉普拉斯而非 FIM 度量不确定性有两项主要优势: 其一, 可得到计算效率更高的技术 (例如, 在三维位姿图优化中, 拉普拉斯的维度仅为 FIM 的六分之一); 其二, 无需求解 SLAM 问题或采集实际测量, 因为计算仅基于图结构。这些设计准则的图近似已成功用于主动 SLAM [190, 582, 877], 其中机器人规划其轨迹以最大化预期的 SLAM 准确性; 也用于测量选择与剪枝 [275, 561, 1094], 目标是在终身 SLAM 问题中仅选择并保留信息量最大的测量。

6.17 延伸阅读与近期趋势

可认证算法。首个用于二维 SLAM 的可认证算法可追溯至 [149], 其建立在计算机视觉及相关领域的早期工作之上, 包括 [338, 1014]。其后很快出现了向三维 SLAM 的扩展和各种变体 [151, 1108, 937, 939, 114, 152]; 其中 SE-Sync 首次给出了构建快速可认证算法的蓝图, 并首次在有界测量噪声下给出了全局最优性保证 (即松弛的精确性) [937]。

此后, 针对多种与 SLAM 有关的问题均已设计可认证算法, ¹³ 包括旋转平均 [122, 972, 251]、PGO [937, 149]、基于路标的 SLAM [465]、多机器人 SLAM [1093]、距离辅助 SLAM [840]、三维配准 [115, 1223]、多集合配准 [501]、双视图几何 [116, 1285, 367, 1097, 538]、perspective-n-point 问题 [1053]、标定 [382]、单帧位姿与形状估计 [1006]、多帧位姿与形状估计 [994], 以及带学习深度的运动恢复 [1256]。近期工作还将可认证算法扩展至应对各向异性噪声 [466] 和离群值 [1221]。

尽管过去十年中该领域取得了显著进展, 仍存在三个令人关注的开放问题。首先, 仍有一些 SLAM 问题无法用可认证算法处理。例如, 在进行视觉 SLAM (第 7 章) 时, 目标函数中出现的透视投影是有理函数而非多项式; 尽管可通过引入额外变量将该问题重新表述为 POP, 但通常无法为一幅图像中的每个关键点测量增加一个变量。同样地, 第 11 章将详细讨论的 IMU 测量建模, 也尚不利于设计可认证算法。第二, 除某些问题外, 例如 [937, 538, 1256, 840], 其中可将黎曼阶梯方法用作快速求解器, 否则可能需要通过内点法或其他特设求解器求解 SDP。内点法在求解 SLAM 前端中产生的低维优化问题时十分有效且快速 (例如

[115, 1223, 116, 1285, 367, 1097, 538]), 但应用于 SLAM 后端中的大规模问题时会慢到不切实际 (例如 [937]); 特设求解器虽然更具可扩展性, 但相较局部求解器仍然较慢 [1220]。为解决此问题, 近期工作不仅研究更快的 SDP 求解器, 还聚焦于如何缩小 SDP 的规模 (例如稀疏化矩松弛的底层单项式基) [1220], 或如何减少约束数目, 以尝试使 SDP 不退化或加快求解 [290]。最后, 尽管当前工作为每个实例计算最优性证书 (即计算估计值, 并可能为其提供最优性证书), 文献中仍缺乏对何时预期松弛精确的基本理解, 只有少数论文给出了特定问题精确性的条件 [1218, 859, 937, 310, 147]。

含离群值的问题。现实世界的 SLAM 问题通常饱受离群值困扰, 第 3 章已对此作了详细讨论。尽管一些工作在无离群值的可认证算法外层套用 Graduated Non-Convexity 迭代 [1223, 1006, 994], 以在经验上获得对离群值的鲁棒性, 近期工作则直接尝试为涉及离群值的鲁棒估计问题开发可认证算法。该方向的研究处理了 PGO [618, 153]、旋转估计 [1218]、三维配准 [1223]、多重旋转平均、绝对位姿估计, 以及位姿与形状估计 [1221]。文献 [1221] 对这些结果作了良好总结, 而与鲁棒统计并行工作的联系则见 [147]。这些算法依赖于本章前面介绍的矩松弛, 所得 SDP 仍然相对较慢。另一种方法是通过局部求解器获得解, 并使用矩松弛背后的相同洞见, 仅推导检查最优性的方法 [1219]。

在这种情形下也仍有许多值得关注的开放问题。¹⁴ 在存在离群值时, 仍面临上一段所述的相同挑战与机遇, 包括如何将可认证算法扩展至其他含离群值的问题、如何设计更快的求解器, 以及如何导出精确性的条件 (例如作为噪声量和离群值数量的函数)。同时, 还有与离群值存在有关的额外挑战。首先, 上述 SLAM 方法假定存在无离群值的里程计骨干: 换言之, 只有回环测量被置于鲁棒损失函数中。在许多 SLAM 问题中, 具有可靠里程计来源的假设可以接受, 但在某些情形下会形成限制。例如, 在视觉 SLAM 问题中, 若特征跟踪失败, 里程计可能变得不可靠; 而在多机器人 SLAM 问题中, 不存在连接不同机器人的里程计骨干, 所有机器人间测量都可能是离群值。尽管当前可认证算法的表述可推广至也将里程计置于鲁棒损失中的情形, 但所得松弛已知是不紧的, 例如 [618], 且如何改进仍不明确。第二, 即使在求解鲁棒估计问题时计算出最优估计, 若大多数测量都是离群值, 估计仍可能严重错误。一个有趣且尚未充分探索的方向是, 设计能够恢复多个假设、同时保证至少一个假设正确的可认证算法; 统计学中将该设定称为 list decodable regression [147]。

不确定性量化与下游应用。尽管第 6.2 节提供了计算工具, 用以界定 SLAM 估计的不确定性, 并可能根据底层图的结构预测其演化, 但它仍留下许多开放问题。首先, 计算 SLAM 估计的协方差依赖于已知测量协方差; 若测量协方差矩阵不准确, 所得不确定性界也会不可靠。近期工作使用学习来估计测量协方差, 甚至整个测量模型 [1020, 889, 1244] (这也与第 4 章中关于可微优化的讨论相关)。第二, 计算估计协方差的传统方法未考虑可能存在的离群值。存在离群值时, 估计协方差可能不正确, 潜在估计 (例如机器人轨迹) 的分布可能变得高度多模态, 从而限制本章前文所述协方差估计的用途。最后, 越来越多的文献使用不确定性量化来引导主动感知和主动 SLAM (综述见 [876]), 并用于确定在终身 SLAM 期间或存在资源约束时如何选择测量子集 [275, 561, 1094, 150]。

第七章 视觉 SLAM

作者： Jakob Engel、Juan D. Tardós、Javier Civera、Margarita Chli、Stefan Leutenegger、Frank Dellaert 和 Daniel Cremers

从相机观测中重建世界及传感器运动，是一项称为视觉同步定位与建图（Visual Simultaneous Localization and Mapping, Visual SLAM 或 VSLAM）的挑战。相机无处不在、价格低廉且功耗高效，因此视觉 SLAM 在自主机器人、自动驾驶汽车、混合现实和增强现实中具有无穷潜力。本章先介绍一些历史背景和术语（第 7.1 节），并概述典型的视觉 SLAM 流水线（第 7.2 节）。随后回顾视觉 SLAM 表述的关键要素（第 7.3 节），并介绍图像对齐和束调整（bundle adjustment, BA）的进阶主题（第 7.4 节）。之后描述完整视觉 SLAM 系统的实例（第 7.5 节）、实时三维重建（第 7.6 节）和基于深度相机的 SLAM（第 7.7 节），讨论将视觉与包括 IMU、GPS 和 WiFi 在内的其他感知模式结合的优势（第 7.8 节）。最后在第 7.9 节简要讨论新近趋势。

7.1 历史背景与术语

7.1.1 从摄影测量到束调整和视觉 SLAM

视觉 SLAM 的历史建立在通用 SLAM 方法之上，同时也根植于摄影测量和计算机视觉领域的进展。这里重点介绍它与后两个领域的历史联系；与 SLAM 的联系已在本书各处阐述。

1822 年，Nicéphore Niépce 发明了现代摄影；现存最早的照片是他于 1827 年拍摄的《Le Gras 窗外的景色》（A View from a Window at Le Gras）。利用相机重建世界的历史，在摄影发明数十年后开始。法国陆军军官 Aimé Laussedat 于 1851 年前后开创了将地面照片用于地形测绘的方法，因而常被视为摄影测量学的创始人。1867 年，德国工程师 Albrecht Meydenbauer 进一步将摄影测量发展用于建筑测量。自 1890 年起，德国数学家兼登山家 Sebastian Finsterwalder（自 1915 年起任德国数学会会长）开创了利用航空影像对阿尔卑斯冰川进行摄影测量重建，并倡导使用射影几何技术^[357]。其博士生 Otto von Gruber 则将束调整（BA）的数学框架形式化，用以从多幅图像中观测到的一组对应点重建结构和运动。这些概念形成于 20 世纪初，远早于计算机出现。之后，德国火箭科学家 Hellmut H. Schmid 开创了在计算机上部署这些方法：他发展了用于 BA 的矩阵计算技术，并于 20 世纪 50 年代与美国人 Duane C. Brown 合作，在当时规模最大的计算机上实现了这些方法。

重建方法和运动恢复结构（structure from motion, SfM）研究领域建立在多种相机模型之上，起点是针孔相机模型，以及用于刻画二维点观测与其相应三维世界坐标关系的射影几何。20 世纪 90 年代初，Tomasi 和 Kanade^[1102] 在正交投影的简化假设下，提出了用于静态场景重建的矩阵分解技术。早期方法通常聚焦于两视图重建；到 21 世纪初，研究群体转向多视图运动恢复结构问题。传统重建流水线包含特征提取、对应关系估计、利用最小解获得初始相机配置，以及随后进行 BA 以获得全局一致的重建。因此，大量工作致力于开发特征提取与匹配算法，使用 SIFT^[699]、SURF^[62] 等特征描述子，近年又出现了大量基于学习的描述子。为应对错误的点对应关系，研究者提出 RANSAC^[328] 等采样策略，使方法能够在模型拟合之间交替重新估计对应关系（参见第 3 章）。

运动恢复结构通常侧重于从无序图像集合中准确地（一般为离线地）重建大规模三维世界；而视觉 SLAM 通常侧重于从移动相机进行在线、实时重建。因此，此类在线实时方法的前提是发展因果方法，例如^[209]：它只使用过去图像（而非完整图像集或视频）解决最优运动恢复结构问题。具备实时能力的首批运动恢复结构/视

觉 SLAM 方法大约在 2000 年出现^[524, 244]。

7.1.2 术语

下列术语常可互换地描述相近过程；但在不同应用和社群中，它们的侧重点不同。

摄影测量 (photogrammetry) 是从二维照片中提取精确测量、空间信息和三维重建的科学。通过分析从不同视点拍摄的重叠图像，摄影测量能够创建物体或环境的几何表示。这一技术广泛用于制图、测量和三维建模，是许多视觉里程计与 SLAM 算法的基础。

束调整 (bundle adjustment, BA) 是用于细化三维重建的数学优化方法。它调整相机的位置、姿态及可选的内参，以及所观测点的三维位置，以最小化重投影误差，即所观测图像点与从三维模型投影所得点之间的差异。优化通常使用 Gauss-Newton 或 Levenberg-Marquardt 等二阶方法，需要谨慎初始化和鲁棒离群值剔除才能有效收敛。近期研究还探索了无初始化 BA，旨在简化优化过程。

运动恢复结构 (structure from motion, SfM) 是从不同视角拍摄的一组二维图像中重建三维结构的过程。不同于常假定相机位置已知的摄影测量，SfM 同时估计场景的三维结构和相机运动。SfM 通常用于非因果、非实时场景，图像可以来自不同来源（例如互联网照片集合），而非连续视频流。通常，SfM 最终会借助 BA 进行优化。诸如 Building Rome in a Day^[14] 的标志性项目展示了它的可扩展性和大规模应用潜力。

视觉里程计 (visual odometry, VO) 聚焦于通过分析视频中的连续视觉帧估计相机运动。它识别相邻图像之间的视觉对应关系，以测量传感器随时间产生的相对运动。VO 主要处理局部运动估计，在近期观测构成的滑动窗口内运行，并不构建全局地图。不过，VO 常与回环检测和建图组件结合，形成完整的视觉 SLAM 系统。

视觉 SLAM (visual SLAM) 是一种计算技术，使系统能在未知环境中同时进行实时自身定位和环境建图。它融合摄影测量、视觉里程计和运动恢复结构的要素，处理图像数据、跟踪相机运动并构建细致的三维地图。与 VO 相比，它通常显式识别先前访问过的地点、相对于这些地点重新定位，并可选地调整该“环”附近的位姿估计，这一过程称为**回环 (loop closure)**。视觉 SLAM 是机器人、自动驾驶车辆和增强现实的核心技术；在这些领域，精确导航和环境理解至关重要。

7.2 视觉 SLAM 系统的处理流水线

构建完整的视觉 SLAM 系统，需要将多种组件组合为一个能够满足实时运行、可扩展性和鲁棒性要求的统一框架。关键考虑包括决定每个组件何时及如何运行、高效组织计算和数据流，以及确保其对多样环境的适应性。与 LiDAR SLAM 一样（参见第 1 章和第 8 章的引言讨论），视觉 SLAM 问题可以分为多个阶段，即将计算划分为 SLAM 前端和 SLAM 后端。然而，不同于基于 LiDAR 的系统，视觉系统并不直接测得三维几何，而是观测场景辐照度在成像平面上的投影，因此视觉 SLAM 的整体估计更具挑战性。现代完整视觉 SLAM 系统通常包含三个相互补充的核心子功能：里程计前端、建图后端，以及回环与重定位组件。本节主要概述该流水线，更详细的说明留待第 7.3 节。

7.2.1 视觉里程计前端

视觉 SLAM 系统的核心要素是视觉里程计，其目标是估计相邻相机帧之间的相对运动。这一阶段为相机位姿提供初始估计。如前所述，计算视觉里程计有两类方法：(i) 基于特征的方法将问题分为三个阶段：检测和提取特征点、计算图像间的两两对应关系，随后通过相对于相机运动和三维点坐标最小化重投影误差，确定相对相机运动。最后一个阶段与经典 BA 十分相似。(ii) 直接法一步处理该问题，直接相对于相机运动和三维结构优化光度损失函数。因此，它们与光流方法及有时称为光度 BA 的方法密切相关。

至少在朴素的早期表述中，由于具有显式数据关联，基于特征的方法表现出更大的收敛域。为缓解这一问题，直接法通常采用由粗到细的策略，即从对齐降采样图像开始，或者甚至尝试对齐稠密的（学习得到的）特征，而非亮度或颜色。

7.2.2 建图后端

后端使用 BA 或位姿图优化等全局优化技术，对轨迹和地图进行优化。该步骤细化前端给出的估计，并将观测（包括由地点识别得到的观测）整合为一致地图。因此，系统获得更长期的一致性，并减小长距离畸变。

7.2.3 视觉地点识别与重定位

视觉里程计易于漂移，因为相机跟踪误差会随时间累积。在没有 GPS 等绝对定位传感器时，可以通过将当前图像与先前观测图像对齐来消除漂移并强制全局一致性。为此，需要在可能很大的图像集合中计算对应关系。这可以通过高效匹配 SIFT、SURF 或 BRIEF 等经典特征描述子实现，也可以通过合适的已训练神经网络实现，后一方法近年来日益流行。所得组件检测相机何时重新访问先前已建图的区域（回环检测），校正累积漂移，并在跟踪失败时恢复定位。

7.2.4 计算与数据流

高效的数据流是高性能 SLAM 系统的必要条件：

流水线并行。跟踪、建图和优化等不同组件通常并行运行，以最大化效率。

数据共享。关键点或位姿等中间输出在组件间共享，以减少冗余计算。

自适应调度。全局优化等计算密集型任务根据系统需求进行调度，并优先保证实时响应性。

7.3 视觉 SLAM 基础

下面回顾实现视觉 SLAM 前端、后端和视觉地点识别所涉及的要素。

7.3.1 相机模型

对于观测场景成像过程的参数化传感器描述，应包含几何组成部分（也称投影函数），它描述三维点如何映射到二维像素；还应包含光度组成部分，它描述物理光强（辐亮度）如何映射为像素值。



图 7.1: 图 7.1 视觉 SLAM 系统的一项核心要求是选择合适的镜头。图中依次为 BF2M2020S23 (195°)、BF5M13720 (183°)、BM4018S118 (126°)、BM2820 (122°) 和 GoPro 替换镜头 (150°)。鱼眼和广角镜头提供更宽的视场，但需要合适的投影模型。常用选择包括 Brown–Conrady (BC) 模型^[283]、Kannala–Brandt (KB) 模型^[537] 和 Double Sphere (DS) 模型^[1117]。六参数 DS 模型与八参数 KB 模型具有相当的重投影精度，而投影函数计算时间约快五倍。(©2018 IEEE)

7.3.1.1 几何相机模型

透视相机。投影函数一般记为：

$$\mathbf{z} = \pi(\mathbf{x}^c, \boldsymbol{\xi}),$$

其中, $\mathbf{x}^c = [x \ y \ z]^\top \in \mathbb{R}^3$ 是相机坐标系中的三维点, $\mathbf{z} = [u \ v]^\top \in \Omega \subset \mathbb{R}^2$ 是图像域 Ω 中对应投影二维点的坐标, $\boldsymbol{\xi} \in \mathbb{R}^n$ 表示相机内参, 通常已在视觉 SLAM 中预先标定. $\boldsymbol{\xi}$ 的维数取决于所用相机模型.

反投影操作记为:

$$\mathbf{x}^c = \pi^{-1}(\mathbf{z}, \boldsymbol{\xi}),$$

它从二维图像点 $\mathbf{z}$ 在三维中重建一条射线. 由于点的深度未知, 所得 $\mathbf{x}^c$ 仅确定到尺度.

实践中, 存在适用于不同镜头几何和相机类型的众多投影函数 (例如参见 [687]). 直线模型 (也称针孔或透视相机模型) 最为简单, 可用于无畸变的窄视场镜头:

$$\boldsymbol{\xi}_p = \begin{bmatrix} f_u & f_v & u_0 & v_0 \end{bmatrix}^\top, \quad \pi_p(\mathbf{x}^c, \boldsymbol{\xi}) = \begin{bmatrix} f_u \frac{x}{z} + u_0 \\ f_v \frac{y}{z} + v_0 \end{bmatrix}, \quad \pi_p^{-1}(\mathbf{z}, \boldsymbol{\xi}) \sim \begin{bmatrix} \frac{u-u_0}{f_u} \\ \frac{v-v_0}{f_v} \\ 1 \end{bmatrix},$$

其中 f_u 和 f_v 分别表示水平方向和垂直方向的焦距, 单位为像素. u_0 和 v_0 表示主点的二维坐标, 且这里假定像素为矩形. 对于常见的方形像素, 可预期 $f_u \approx f_v$.

为考虑一定程度的镜头畸变, 最常用的是径向-切向模型:

$$\boldsymbol{\xi}_{RT} = \begin{bmatrix} \boldsymbol{\xi}_p^\top & k_1 & k_2 & p_1 & p_2 \end{bmatrix}^\top, \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \frac{x}{z} \\ \frac{y}{z} \end{bmatrix}, \quad r = \sqrt{x'^2 + y'^2},$$

$$\begin{bmatrix} x'' \\ y'' \end{bmatrix} = \begin{bmatrix} x'(1 + k_1 r^2 + k_2 r^4) + 2p_1 x' y' + p_2(r^2 + 2x'^2) \\ y'(1 + k_1 r^2 + k_2 r^4) + p_1(r^2 + 2y'^2) + 2p_2 x' y' \end{bmatrix}, \quad \pi_p(\mathbf{x}^c, \boldsymbol{\xi}) = \begin{bmatrix} f_u x'' + u_0 \\ f_v y'' + v_0 \end{bmatrix}.$$

注意, 反投影模型 $\pi_{RT}^{-1}(\mathbf{z}, \boldsymbol{\xi})$ 无法写出解析表达式, 因为无法从 x'', y'' 得到 x', y' 的解析解 (即去畸变). 不过, 可以采用迭代方法, 例如 Newton-Raphson 方法.

广角与鱼眼相机. 对于视场 (field of view, FOV) 最高约为 180° 的广角镜头 (见图 7.1), 具有少量径向畸变系数的针孔模型通常足够. Brown-Conrady 模型 [283] 等同于上述省略切向系数的径向-切向模型, 即 $\boldsymbol{\xi}_{BC} = [\boldsymbol{\xi}_p^\top \ k_1 \ k_2]^\top$. 注意, 即使相对于径向-切向模型有所简化, 仍不存在解析去畸变.

对于视场超过 180° 的鱼眼镜头, 例如 [142] 使用 Kannala-Brandt (KB) 模型 [537]:

$$\boldsymbol{\xi}_{KB} = \begin{bmatrix} \boldsymbol{\xi}_p^\top & \mathbf{k}_{KB}^\top \end{bmatrix}^\top, \quad \pi_{KB}(\mathbf{x}^c, \boldsymbol{\xi}) = \begin{bmatrix} f_u r(\theta) \cos \psi + u_0 \\ f_v r(\theta) \sin \psi + v_0 \end{bmatrix}^\top.$$

这里, 入射光线由角度 $\theta = \arctan \frac{\sqrt{x^2 + y^2}}{z}$ 和 $\psi = \arctan \frac{y}{x}$ 参数化; 畸变由四个系数 $\mathbf{k}_{KB} = [k_1 \ \dots \ k_4]^\top$ 参数化, 其畸变表达式为 $r(\theta) = \theta + \sum_{n=1}^4 k_n \theta^{2n+1}$.

对于鱼眼和广角镜头, Double Sphere (DS) 模型 [1117] 是一种常用替代方案, 它在精度和速度之间提供良好折中. 在 DS 模型中, 一个点依次投影到球心沿 γ 偏移的两个单位球上, 再使用沿 $\frac{\alpha}{1-\alpha}$ 偏移的针孔模型投影到图像平面. 这得到:

$$\pi_{DS}(\mathbf{x}^c, \boldsymbol{\xi}) = \begin{bmatrix} f_u \frac{x}{\alpha d_2 + (1-\alpha)(\gamma d_1 + z)} + c_u \\ f_v \frac{y}{\alpha d_2 + (1-\alpha)(\gamma d_1 + z)} + c_v \end{bmatrix}, \quad \text{with } \boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{\xi}_p^\top & c_u & c_v & \gamma & \alpha \end{bmatrix}^\top.$$

如 [1117] 所示, 该模型具有闭式反投影解. 因此, 六参数 DS 模型的计算速度约为八参数 KB 模型的五倍, 同时提供可比的重投影误差.

7.3.1.2 光度模型

光度标定将辐照度映射为像素值:

$$I = f(E, T),$$

其中 I 是像素强度, E 是辐照度, T 包含影响此映射的其他相机属性, 例如曝光时间、模拟和数字增益、伽马校正、去拜耳 (de-bayering) 处理和镜头暗角。光度标定通常只在尺度意义上重要, 且主要服务于希望获得稠密、带纹理场景表示, 或依赖图像间光度一致性的方法。

7.3.1.3 时间相关效应

若相机在图像采集期间运动 (这几乎总是 SLAM 或视觉里程计系统的情形), 还必须考虑该运动对成像过程的影响。多数现代消费级相机采用卷帘快门 (rolling shutter), 按时间顺序逐行采集图像。相比之下, 常用于机器感知的全局快门 (global shutter) 相机会同时采集所有图像行。

在实践中, 建议使用全局快门相机, 或者通过针对各图像行改变相机位姿, 将卷帘快门效应纳入相机模型。需要注意的是, 在视觉惯性系统中, 这一处理会显著更实用: IMU 能在相机曝光时间窗内有效测量局部运动, 因此极大简化卷帘快门相机的使用和建模。

7.3.1.4 实际考虑

选择相机模型时, 首要目标是确保参数化模型能有效、准确地逼近传感器和镜头系统的行为。使用鱼眼镜头时, 推荐采用球面模型; 而对直线镜头, 应使用线性基础模型。更一般地说, 选择相机模型需要在计算效率和精度之间权衡: 不适配的相机模型会引入不准确性和系统偏差, 显著降低视觉 SLAM 系统的准确性和鲁棒性。

7.3.2 关键点

SLAM 系统能否完成环境建图并提供准确定位, 取决于其检测、描述和匹配场景关键特征的能力, 这些特征通常称为“关键点” (keypoints) 或“特征” (features)。在视觉 SLAM 中, 关键点检测是识别显著、可区分且可重复的图像区域, 以确保可以从不同视点、多次运行和不同条件下可靠地再次检测到它们。理想的关键点检测器应在光照、视点或遮挡变化下仍保持这些性质。检测到关键点后, 关键点描述子将相应关键点检测的局部外观编码为紧凑且具有区分性的表示。理想情况下, 描述子应唯一地刻画关键点周围区域, 同时对光照变化、旋转或尺度变化等变换保持不变性。这既确保较高召回率, 即在不同条件下匹配相同关键点, 也确保较高精度, 即避免与外观相似但无关的特征产生错误对应。

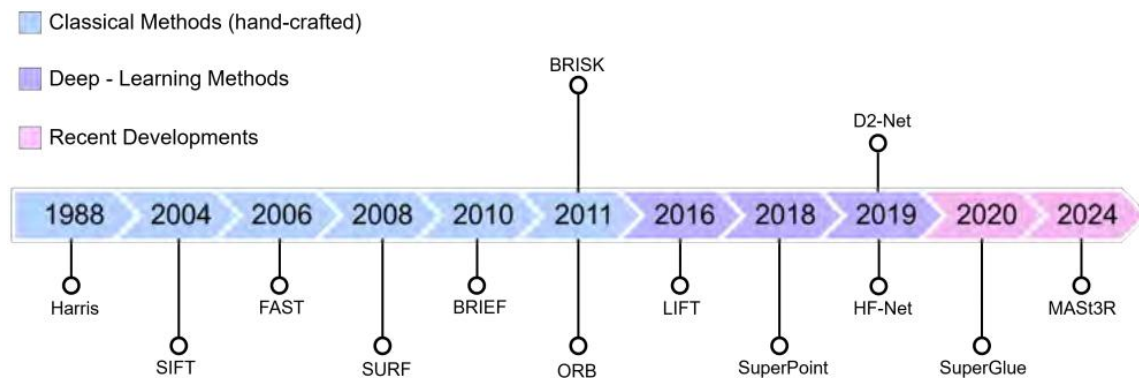


图 7.2: 图 7.2 一些塑造基于视觉的 SLAM 与图像匹配中视觉关键点研究文献的代表性算法时间线。

在实践中, 剧烈光照变化、无纹理表面和动态场景等现实挑战会在检测与匹配中引入误差, 直接影响 SLAM 性能。因此, 研究工作致力于设计具有特定特性的关键点以增强鲁棒性。最理想的特性包括尺度与旋转不变性 (保证视点变化时检测一致)、可重复性与可区分性 (支持可靠重检和唯一识别), 以及效率 (在计算约束下支持实时运行)。为满足这些需求, 关键点检测与描述近年已从经典手工设计技术演变为基于深度学习的方法。下文各小节将结合图 7.2 概述文献中最具代表性的关键点方法及该领域的新兴趋势。

7.3.2.1 经典关键点检测器与描述子

经典的手工设计技术在特征检测和描述的发展中发挥了关键作用。其中，Harris-Stephens 关键点检测器^[432]（广为人知的“Harris 角点检测器”）是最具影响力的方法之一。它通过分析图像块内二阶矩矩阵的特征值识别角点：当两个特征值均较大、表明两个正交方向上都有强烈强度变化时，将该图像块判为关键点。Shi-Tomasi 角点检测器^[1004]建立在同一原理上，但直接使用较小特征值进行角点选择；相较之下，Harris 定义“角点性”（cornerness）响应函数以高效近似这一过程。Harris 检测器鲁棒且计算高效，但缺乏尺度不变性，后续的 SIFT、SURF 等方法致力于解决这一局限。

关键点检测的一个里程碑是尺度不变特征变换（Scale Invariant Feature Transform, SIFT）^[699]的提出，它在具有挑战性的环境中为精度和召回率设立了新标准。SIFT 对尺度和旋转具有高度不变性，并对光照变化具有部分不变性。它遵循结构化的三阶段过程：(1) 在尺度空间高斯差分（Difference of Gaussians, DoG）金字塔中将极值检测为关键点，并通过三维二次拟合细化其位置；(2) 根据局部图像梯度主方向分配方向；(3) 从离散化梯度方向的直方图中计算 128 维描述子。不过，SIFT 出色的鲁棒性伴随较高计算代价，使其在实时应用中开销更大。

为提高效率，研究者提出了加速分割测试特征（Features from Accelerated Segment Test, FAST）^[946]这一高速角点检测器。它评估候选关键点位置周围圆形邻域中的像素强度，并采用提前拒绝策略以减少计算。FAST 还借助决策树和非极大值抑制进一步提升速度。但它不像 SIFT 那样具有尺度不变性，因而对较大变换敏感。随后，快速鲁棒特征（Speeded-Up Robust Features, SURF）^[63]使用积分图像和盒式滤波器替代高斯导数来近似 SIFT，在速度与鲁棒性之间取得更好平衡。

对快速而鲁棒描述子的需求推动了二进制方法的发展。二进制鲁棒独立基本特征（Binary Robust Independent Elementary Features, BRIEF）^[141]采用二进制强度比较构建紧凑描述子，使得可用 Hamming 距离快速匹配描述子。BRIEF 虽缺少尺度和旋转不变性，但证明了可通过简化的二进制测试有效捕获局部梯度，而局部梯度正是 SIFT 鲁棒性的基础。受此启发，ORB^[953]在 FAST 关键点检测和 BRIEF 描述的基础上，使用图像金字塔和强度质心法增加尺度和旋转不变性。与此同时，BRISK^[643]采用尺度空间金字塔上的 FAST 或 Harris 角点，并通过识别类似 SIFT 的主关键点方向，在二进制描述子中纳入旋转变化不变性。ORB、BRISK 等方法增加旋转和尺度不变性会轻微影响输出关键点的可区分性，但已证明在视觉 SLAM 和实时机器人应用中有效。不过，当计算代价不是约束（例如计算机视觉应用）时，SIFT 和 SURF 仍可能更可取，因为它们在有挑战性的光照或视角变化下仍提供更鲁棒性。

随着诸多变体见于文献，经典手工设计的关键点检测和描述方法始终是计算机视觉与机器人学的基础，它们在鲁棒性、效率和不变性之间经过精心权衡。然而，能够适应日益复杂、动态环境的关键点需求仍是一项持续挑战。这推动研究转向基于学习的方法，后者旨在为真实应用自动优化关键点，为下一代特征检测技术奠定基础。

7.3.2.2 基于深度学习的关键点检测与描述

到 2010 年代末，基于深度学习的图像关键点方法开始获得关注；它们利用大规模数据集和卷积神经网络（convolutional neural networks, CNN）直接从数据学习鲁棒特征。不同于人工设计的关键点，这类方法能自动发现并优化特征表示，在此前难以处理的场景中显示出无与伦比的适应性和准确性。例如，它们在极端光照变化下检测稳定关键点已取得显著成功，而这正是经典手工方法的难点。因此，视觉 SLAM 社群日益摆脱传统特征工程，转而采用数据驱动的代表学习进行关键点检测与描述。

学习不变特征变换（Learned Invariant Feature Transform, LIFT）^[1245]是最早将关键点检测、方向估计和描述子计算集成于端到端可训练流水线的深度学习方法之一。LIFT 使用 CNN 从小图像块提取特征，相对于经典方法，对尺度、光照和旋转变化有更强鲁棒性。它采用顺序学习策略：先训练描述子，随后训练方向估计，最后训练关键点检测，以确保稳定有效的特征提取。类似地，SuperPoint^[267]提出自监督框架，在一次前向传播中检测关键点并计算描述子，因而适合实时应用。与基于图像块的网络不同，SuperPoint 处理整幅图像，并利用单应适配（homographic adaptation）这一自监督学习技术生成伪真值关键点。相较 SIFT、ORB 和 FAST 等经典方法，该方法显著改善关键点重复性和描述子质量，尤其是在光照变化下。

在这些进展基础上，HF-Net^[970]提出结合全局图像检索和精确局部特征匹配的层次化定位方法。HF-Net 将关键点检测、局部描述子和全局描述子整合到单一 CNN 中，在维持高鲁棒性的同时提高计算效率。该

架构通过限制用于匹配的图像数量来降低运行时间，因而即使在夜景等极端外观变化下，也特别适合大规模 SLAM 和实时应用。HF-Net 学得特征比 SuperPoint 更稀疏却更具区分性，使其成为 DX-SLAM^[649] 等基于深度学习 SLAM 系统的优选。

值得注意的是，D2-Net^[295] 提出“先描述、后检测”（describe-and-detect）方法，颠倒了传统关键点检测与描述子提取的顺序。它并非先检测关键点，而是使用 CNN 计算稠密特征图，再将这些图中的局部极大值识别为关键点。该方法捕获高层语义信息，因此对极端光照变化和弱纹理环境具有鲁棒性。不同于将检测与描述分离的 SuperPoint，D2-Net 联合优化这两项任务，从而提高描述子一致性。不过，这种稠密方法虽提高鲁棒性，却比经典稀疏方法更耗计算。尽管存在该折中，D2-Net 对视觉定位和 SfM 任务仍十分有效，推动了基于深度学习特征提取的边界。

7.3.3 重投影误差

视觉重投影误差度量观测图像点 $\mathbf{z}_j \in \mathbb{R}^2$ 与重投影点 $\pi(\mathbf{x}_j^c, \boldsymbol{\xi}) \in \mathbb{R}^2$ 之间的差异：

$$\mathbf{e}_{\text{reproj}} = \mathbf{z}_j - \pi(\mathbf{x}_j^c, \boldsymbol{\xi}),$$

其中 $\mathbf{z}_j$ 是观测到的二维图像点， $\mathbf{x}_j^c \in \mathbb{R}^3$ 是相机坐标系中的三维点， $\boldsymbol{\xi}$ 是相机内参标定参数， π 为投影函数。

假设观测图像点 $\mathbf{z}_j$ 受到高斯噪声扰动，则似然函数可写为：

$$p(\mathbf{z}_j | \mathbf{x}_j^c, \boldsymbol{\xi}) \sim \mathcal{N}(\pi(\mathbf{x}_j^c, \boldsymbol{\xi}), \boldsymbol{\Sigma}_j),$$

其中 $\boldsymbol{\Sigma}_j$ 是图像中特征位置高斯噪声的协方差。在最简单情形，假设噪声在图像上各向同性且恒定，即 $\boldsymbol{\Sigma}_j = \sigma^2 \mathbf{I}_2$ 。

最大化一组观测的似然，等价于最小化其负对数似然：

$$\mathcal{L} = -\sum_j \log p(\mathbf{z}_j | \mathbf{x}_j^c, \boldsymbol{\xi}) = \frac{1}{2} \sum_j \|\mathbf{z}_j - \pi(\mathbf{x}_j^c, \boldsymbol{\xi})\|_{\boldsymbol{\Sigma}_j}^2 + \text{const.}$$

去掉常数后，可得到图像中所观测点集的加权平方重投影误差：

$$E_{\text{reproj}} = \frac{1}{2} \sum_j \|\mathbf{z}_j - \pi(\mathbf{x}_j^c, \boldsymbol{\xi})\|_{\boldsymbol{\Sigma}_j}^2. \quad (7.1)$$

为处理离群值，可使用 Huber 或 Tukey 等鲁棒核（参见第 3 章）。例如，鲁棒重投影误差可写为：

$$E_{\text{robust}} = \frac{1}{2} \sum_j \rho(\|\mathbf{z}_j - \pi(\mathbf{x}_j^c, \boldsymbol{\xi})\|_{\boldsymbol{\Sigma}_j}). \quad (7.2)$$

其中 $\rho(\cdot)$ 是降低大残差影响的鲁棒核函数。为简洁起见，下文省略对相机内参 $\boldsymbol{\xi}$ 的依赖。

7.3.4 基于关键点的视觉 SLAM

基于特征的视觉 SLAM 的核心是最小化重投影误差。假设我们有一组环境点 $P_j \in \mathbf{P}$ ，它们在移动相机采集的一组图像 $C_i \in \mathbf{C}$ 中被观测到。为简化记号，以下仅使用点和相机的标识符，写作 $j \in \mathbf{P}$ 和 $i \in \mathbf{C}$ 。视觉 SLAM 的目标是估计世界参考系 $\mathcal{F}^w$ 中的点坐标 $\mathbf{x}_j^w \in \mathbb{R}^3$ ，以及相机位姿 $\mathbf{T}_w^i \in \text{SE}(3)$ 。在单目情形，每个图像中每个点的观测仅为图像坐标 $\mathbf{z}_{ij} \in \Omega_i \subset \mathbb{R}^2$ （图 7.3）。

将重投影误差应用于全部点和相机位姿时的最小化，称为完整 BA：

$$\{\mathbf{T}_w^{i*}, \mathbf{x}_j^{w*} \mid i \in \mathbf{C}, j \in \mathbf{P}\} = \arg \min_{\mathbf{T}_w^i, \mathbf{x}_j^w} \frac{1}{2} \sum_{i,j} \rho\left(\left\|\mathbf{z}_{ij} - \pi\left(\mathbf{R}_w^i \mathbf{x}_j^w + \mathbf{t}_w^i\right)\right\|_{\boldsymbol{\Sigma}_{ij}}\right). \quad (7.3)$$

通常，研究者通过若干先进方法处理重投影误差优化，每种方法在计算复杂度和鲁棒性上具有不同取舍：

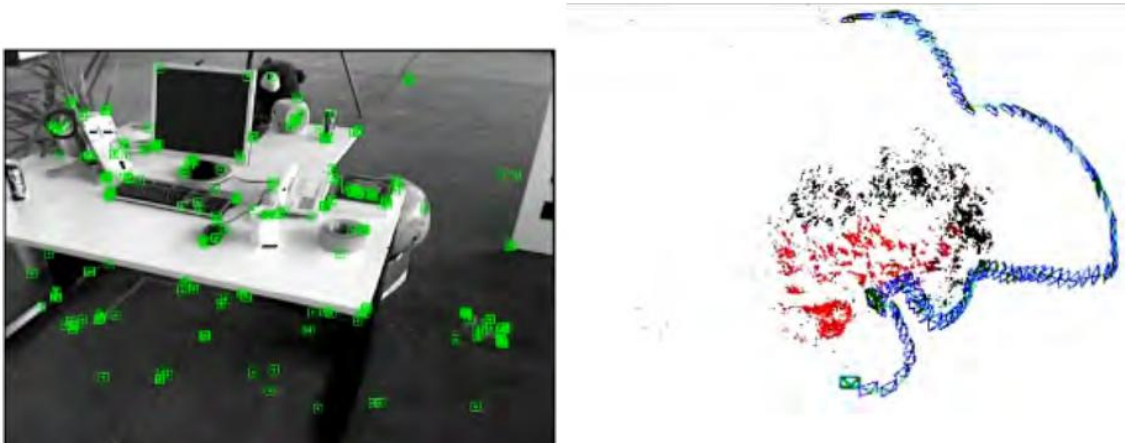


图 7.3: 图 7.3 基于特征的视觉 SLAM 问题: 给定每幅图像中已匹配的一组特征 (左), 估计其对应三维点的位置, 以及每幅图像采集位置处的位姿 (右)。图像由 ORB-SLAM^[787] 生成。

批量优化。相对于全部位姿和路标, 对全部观测迭代最小化总重投影误差, 即求解式 (7.3)。常用的最小化算法为 Gauss-Newton (GN) 和 Levenberg-Marquardt (LM)。这是 SfM 的黄金标准, 但在实时 SLAM 中对每幅图像运行的代价过高。

基于滤波的方法。使用扩展卡尔曼滤波 (extended Kalman filter, EKF) 或信息滤波器等顺序、因果方法进行实时状态估计。这类方法只保留最新相机位姿以简化问题, 但不幸会破坏稀疏性 (图 7.4), 因而只能处理数百个特征。此外, 滤波方法不重新线性化过去观测, 导致精度损失。

基于关键帧的方法。这类方法只保留少数图像, 即关键帧^[583], 以简化问题; 中间图像及其观测被舍弃, 不参与地图估计。在相同计算量下, 它们能构建比滤波方法更长、更准确的地图^[1039]。

因子图。上述全部方法均可用因子图形式化。具体而言, SLAM 问题被表示为图: 节点对应变量 (如位姿和路标), 因子表示重投影误差以及标定和/或相机位姿的先验; 本书第一部分已对此作详细讨论。

7.3.5 光度误差与直接法

光度误差提供了重投影误差的替代方案: 它直接最小化观测图像区域和投影图像区域之间的像素强度差。该方法根植于光度一致性原则, 即假定连续帧中的对应像素在一致光照条件下表示同一场景点。

7.3.6 视觉地点识别与全局定位

视觉地点识别是指: 给定一幅查询图像, 在已登记图像数据库中找到来自同一地点的图像。通常的做法是为每幅图像计算概括其内容的描述子 $\mathbf{d} = f(I) \in \mathbb{R}^d$, 再通过 k-NN 搜索检索该描述子空间中最接近的图像。Galvez-Lopez 和 Tardos^[362] 提出了词袋方法, 其基本思想是将 ORB 或其他描述子量化到视觉聚类或“词”中并聚合。它们在较小时间和空间范围内表现优异, 却受限于手工特征对视觉纹理变化的不变性较低。对于这些情形, 研究者提出并训练深度架构, 以高不变性进行特征提取和聚合, 从而适应视觉外观变化^[35, 508]。

7.3.7 初始化

式 (7.3) 的重投影误差最小化通常由非线性迭代优化完成, 该过程需要足够准确的初始猜测才能收敛。因此, 视频序列最初几帧中视觉 SLAM 状态的初始化对正确跟踪十分重要, 尤其对单目配置而言, 因为无法由单帧观测到完整状态。

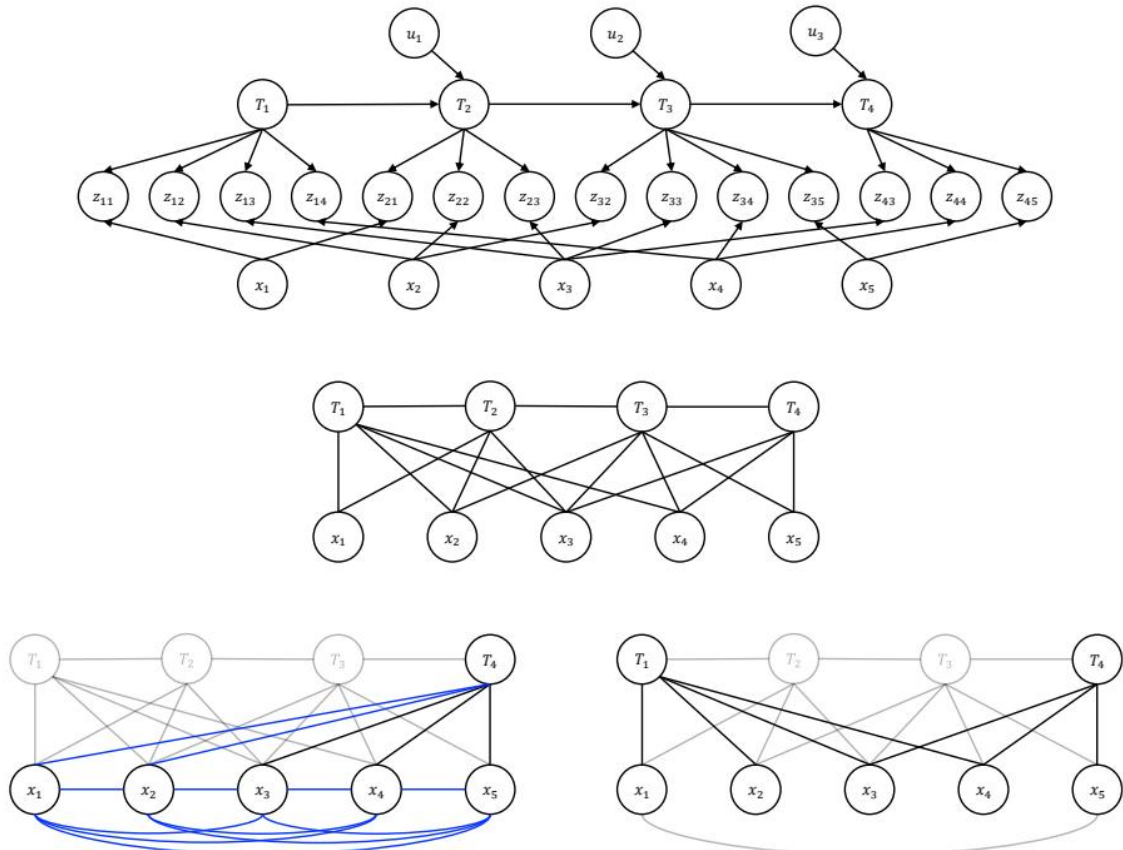


图 7.4: 图 7.4 含四个相机位姿和五个特征的视觉 SLAM 示例：贝叶斯网络（上）及其对应的马尔可夫随机场（中）。EKF SLAM 将过去相机位姿边缘化，得到稠密图（左下）。关键帧 SLAM 仅保留少数相机位姿，并舍弃中间图像的观测，从而保持 SLAM 的稀疏性（右下）[1039]。

7.3.8 地图表示

地图表示是视觉 SLAM 系统的关键方面，因为它定义环境如何建模和存储，以服务导航、建图和定位。一种设计良好的地图表示应在准确性、内存效率和计算代价之间取得平衡。关于稠密地图表示的更广泛讨论，参见第 5 章。

7.4 关于图像对齐和 BA 的进一步考虑

本节进一步介绍图像对齐，即如何由图像估计当前相机位姿（这是里程计和回环检测的关键），并简要说明 BA 的求解技术。

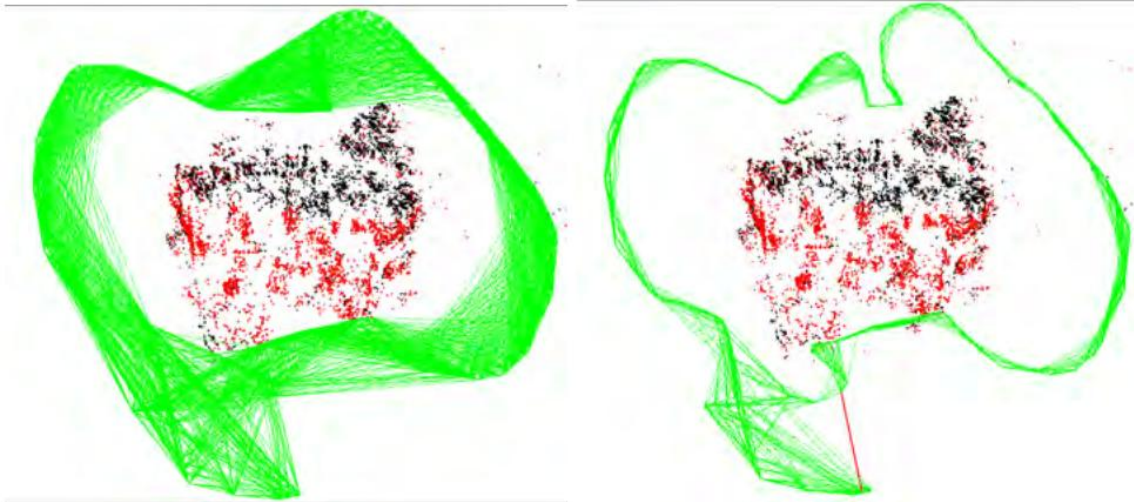


图 7.5: 图 7.5 ORB-SLAM 中的局部性表示^[787]。共视图（左）连接至少共同观测到 θ 个点的关键帧（本例中 $\theta = 15$ ），用于局部 BA。本质图（右）是更稀疏的版本，在本例中连接至少共同观测到 $\theta := 100$ 个点的关键帧，用于回环校正期间的位姿图优化。（©2015 IEEE）

7.4.1 基于关键点图像对齐

尽管完整 BA（式 (7.3)）是 SfM 的黄金标准，但其代价过高，无法以通常为 10–50 Hz 的帧率运行。为实现实时运行，多数基于关键点的视觉 SLAM 流水线采用两个关键思想：

跟踪与建图并行。将 SLAM 过程划分为两个并行运行的线程^[583]：

跟踪线程为当前图像 $i \in \mathcal{C}$ 寻找特征匹配，并在不更新估计地图点的情况下，利用仅位姿 BA 计算其相机位姿：

$$\mathbf{T}_w^{i*} = \arg \min_{\mathbf{T}_w^i} \sum_{j \in \mathcal{P}} \rho \left(\left\| \mathbf{z}_{ij} - \pi \left(\mathbf{R}_w^i \mathbf{x}_j^w + \mathbf{t}_w^i \right) \right\|_{\Sigma_{ij}} \right). \quad (7.4)$$

建图线程仅对一组称为关键帧的图像子集 $\mathcal{K} \subset \mathcal{C}$ 求解 BA，只有这些图像的位姿会被纳入地图。这样，BA 只需以关键帧频率运行，通常为 0.5–5 Hz。关键帧可按固定频率插入；更合理的做法是将含有显著新信息的帧升级为关键帧。

局部性。当相机在大环境中运行时，除回环事件外，其观测对远处地图部分的影响可忽略。通常将回环校正交给一个运行频率很低的第三线程，而在建图线程中，对一个关键帧窗口运行局部 BA。局部窗口可根据时间准则定义为最近 k 帧或关键帧，这是视觉里程计或视觉惯性 SLAM 系统的常见选择。对于视觉 SLAM 系统，更好的选择是基于共视性定义局部窗口，例如将与当前关键帧至少共同观测到 θ 个点的关键帧纳入窗口^[1038, 787]（见图 7.5）。这样，局部 BA 只更新一组相互共视的关键帧及其观测到的点（图 7.6）：

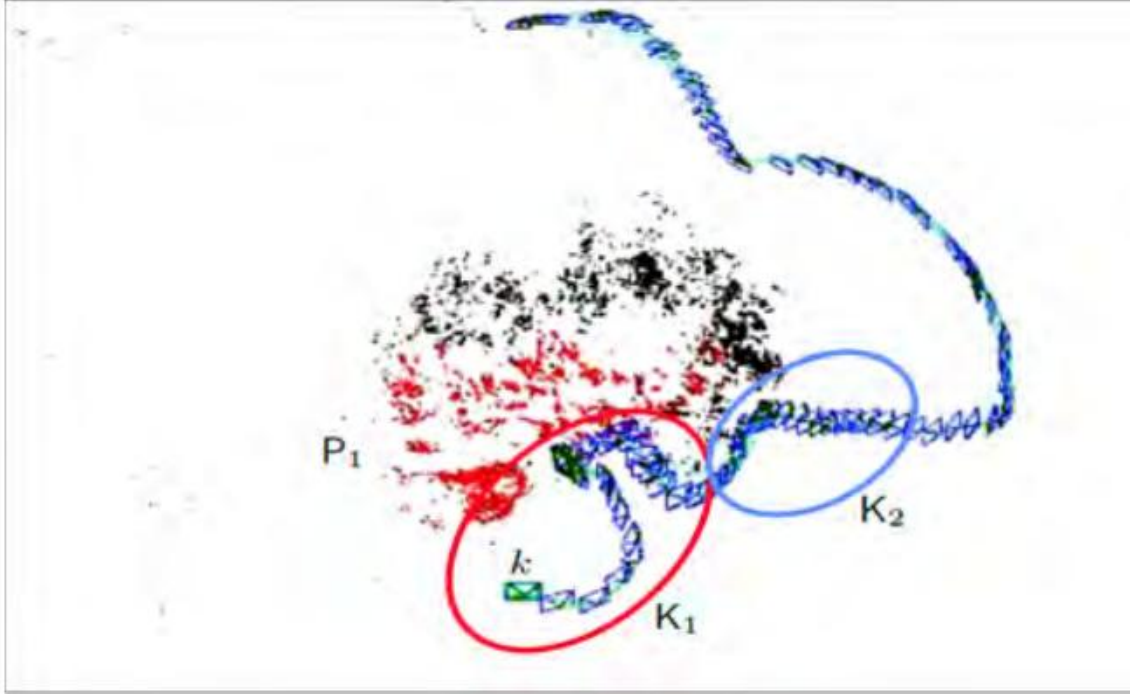


图 7.6: 图 7.6 ORB-SLAM 中局部 BA 的实现^[787]。局部地图由关键帧集合 K_1 定义，该集合包含当前关键帧 k 及其在共视图中的邻居；还包括这些关键帧观测到的点集 P_1 （红色）。 K_2 是地图中能观测到 P_1 某个点的其余关键帧集合。

$$\{\mathbf{T}_w^k, \mathbf{x}_j^w \mid k \in K_1, j \in P_1\} = \arg \min_{\mathbf{T}_w^k, \mathbf{x}_j^w} \frac{1}{2} \sum_{\substack{i \in K_1 \cup K_2, \\ j \in P_1}} \rho \left(\left\| \mathbf{z}_{ij} - \pi \left(\mathbf{R}_w^i \mathbf{x}_j^w + \mathbf{t}_w^i \right) \right\|_{\Sigma_{ij}} \right). \quad (7.5)$$

图 7.7 给出了完整视觉 SLAM 流水线的示例，其中包含四个线程：按帧率运行的跟踪线程、按关键帧频率运行的局部建图线程、尝试为每个关键帧检测回环并在检测到时校正的回环线程，以及可选运行、用于在回环后改进地图的完整 BA 线程。

7.4.2 直接图像对齐

LSD-SLAM^[307] 或 DSO^[308] 等直接法通常遵循相似流水线：先估计帧间跟踪和建图，再保证某种形式的全局一致性。但它们并非先提取、匹配和跟踪点，然后最小化几何重投影误差，而是直接使用传感器亮度信息，目标是在给定原始传感数据的条件下，计算三维结构和相机运动的最大后验估计。这等价于通过最小化下列形式的光度损失，联合求解对应关系估计和 SLAM 问题：

$$E_{\text{photo}} = \sum_{i \in \mathcal{F}} \sum_{\mathbf{z} \in \mathcal{P}_i} \sum_{j \in \text{obs}(\mathbf{z})} \rho \left(I_i(\mathbf{z}) - I_j(\omega(\mathbf{z}, d_{\mathbf{z}}, \mathbf{T}_j^i)) \right), \quad (7.6)$$

其中优化变量为全部相机参数 $\mathbf{T}_j^i \in \text{SE}(3)$ 和所有深度值 $d_{\mathbf{z}} \in \mathbb{R}$ 。该损失确保所有帧对 i, j 中对应点的颜色 I_i 与 I_j 一致。更具体地，先对全部关键帧集合 $\mathcal{F}$ 求和；再对关键帧 i 中每个点 $\mathbf{z} \in \mathcal{P}_i$ ，保证在该点可见的所有帧 $\text{obs}(\mathbf{z})$ 中颜色一致。变形函数 ω 将具有深度值 $d_{\mathbf{z}}$ 的点 $\mathbf{z}$ 以刚体运动 $\mathbf{T}_j^i$ 从帧 i 变换到帧 j ，再进行透视投影回图像 I_j 。

如图 7.8 所示，LSD-SLAM 交替优化深度图和相机运动（跟踪），并执行位姿图优化（pose-graph optimization, PGO），以保证所估计相机运动与计算得到的两两图像对齐之间的全局一致性。

相较之下，DSO^[308] 以光度 BA 的形式联合优化结构和运动。鲁棒损失 ρ 通过小图像块上的加权平方差实现，其中还包含自动曝光时间适配（曝光时间未知时）。相应残差之间的依赖关系采用因子图表示。DSO

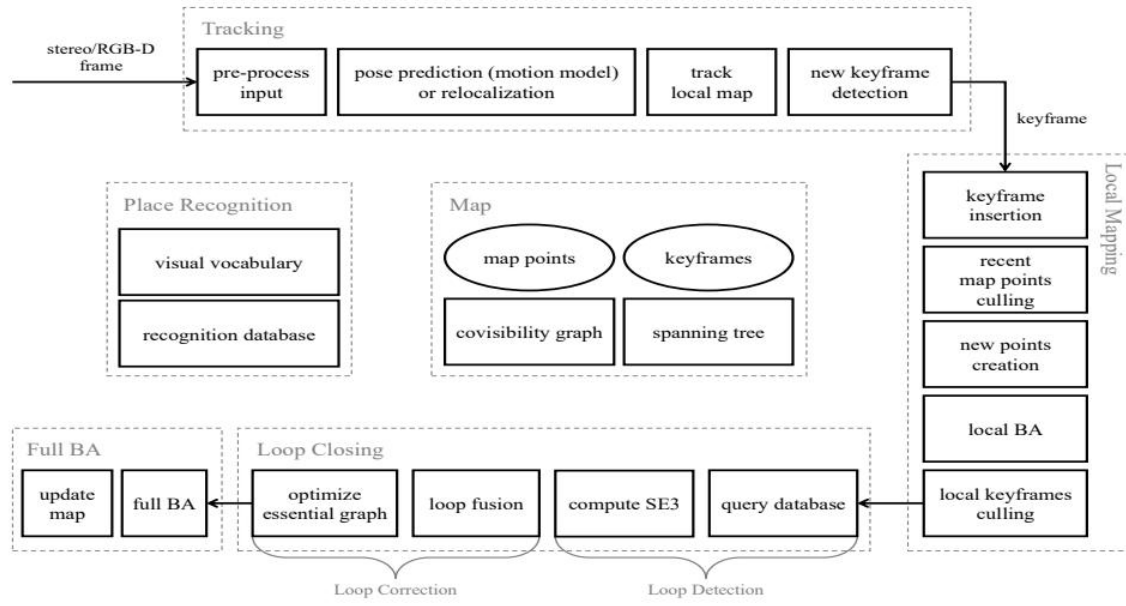


图 7.7: 图 7.7 ORB-SLAM2 系统结构 [785]，展示地图、地点识别数据库以及由四个线程组成的完整处理流水线：跟踪、局部建图、回环和完整 BA。(©2017 IEEE)

通过将优化限制在部分关键帧的滑动窗口中，并边缘化较旧帧的影响，在 CPU 上实现实时性能。由于它利用全部传感器亮度数据给出结构和运动的统计最优估计，精度得到显著提升。

在具有实时能力的 SLAM 方法中，可分几步实现全局一致性。第一，可如 DSO 一样联合优化最近 k 个关键帧，以保证滑动时间窗口上的一致性（滑动窗口光度 BA）。第二，可额外运行 PGO [553, 365]（见图 7.9），重新计算与所有先前估计的两两图像对齐最大程度一致的相机轨迹。第三，可采用称为位姿图束调整（Pose Graph Bundle Adjustment, PGBA）[1135] 的自适应 PGO 版本；它还纳入 BA 的完整光度不确定性，同时保持相同计算效率，因为只更新相机位姿。

7.4.3 求解 BA

虽然可以使用第 2 章讨论的通用变量消去技术求解 BA，但也有能利用问题稀疏结构的专用求解器。图 7.10 给出一个玩具示例，含有四台相机和从中观测到的九个点。观测雅可比矩阵非常稀疏，因为每个观测 z_{ij} 只依赖相机 i 和点 j 。因此，每个观测在 Hessian 矩阵中引入一个相机对角块、一个点对角块和一个非对角的相机-点块。点的数量通常比相机数量大几个数量级，故合理做法是先消去点，再求解相机。

这可通过 Schur 补完成。若有一个 D 可逆的线性系统，可在其左右两侧分别乘以矩阵，将其变换为：

$$\begin{aligned} \begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} &= \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}, \\ \begin{pmatrix} I & -BD^{-1} \\ 0 & I \end{pmatrix} \begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} &= \begin{pmatrix} I & -BD^{-1} \\ 0 & I \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}, \\ \begin{pmatrix} A - BD^{-1}C & 0 \\ C & D \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} &= \begin{pmatrix} b_1 - BD^{-1}b_2 \\ b_2 \end{pmatrix}. \end{aligned}$$

由此得到可以先求解 x_1 、再求解 x_2 的系统：

$$\begin{aligned} (A - BD^{-1}C) x_1 &= b_1 - BD^{-1}b_2, \\ Dx_2 &= b_2 - Cx_1. \end{aligned}$$

BA 问题在每次迭代中都需要求解下列形式的方程：

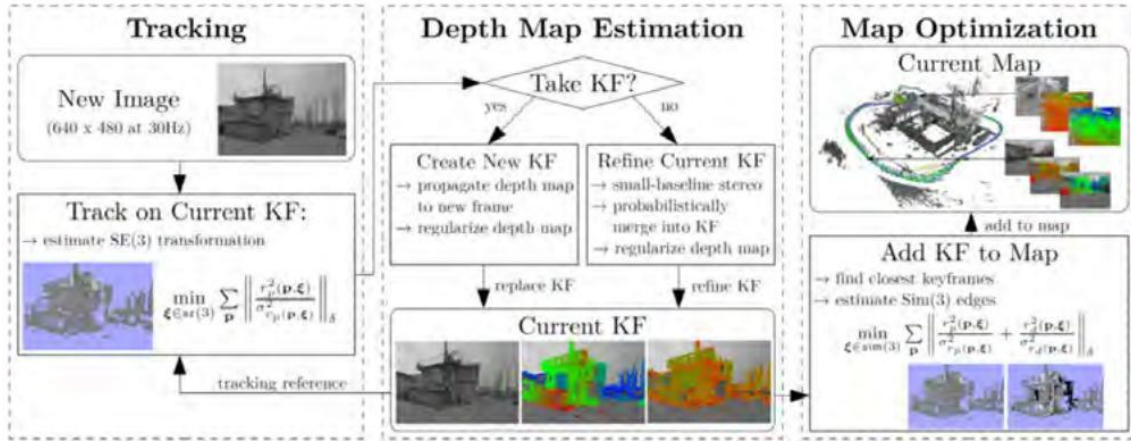


图 7.8: 图 7.8 Large-Scale Direct (LSD) SLAM [307] 示意总览。三个组件交替执行直接相机跟踪、直接建图和用于全局一致性的 PGO。相比之下, Direct Sparse Odometry [308] 在一次 Gauss–Newton 优化中完成结构和运动优化, 以获得更高精度。已证明 DSO 等直接方法比基于关键点的方法精度更高, 因为它们不执行中间抽象, 并可从极其细微的亮度变化中确定相机运动 [308]。(©2017 Springer)

$$\begin{pmatrix} \mathbf{H}_{cc} & \mathbf{H}_{cp} \\ \mathbf{H}_{cp}^\top & \mathbf{H}_{pp} \end{pmatrix} \begin{pmatrix} \mathbf{d}_c \\ \mathbf{d}_p \end{pmatrix} = \begin{pmatrix} \mathbf{b}_c \\ \mathbf{b}_p \end{pmatrix}.$$

这可分三步完成: 计算点的 Schur 补以得到降维相机系统, 求解该系统以获得相机更新量, 最后求解点更新量:

$$\begin{aligned} \mathbf{H}_{cc}^{\text{red}} &= \mathbf{H}_{cc} - \mathbf{H}_{cp} \mathbf{H}_{pp}^{-1} \mathbf{H}_{cp}^\top, \\ \mathbf{H}_{cc}^{\text{red}} \mathbf{d}_c &= \mathbf{b}_c - \mathbf{H}_{cp} \mathbf{H}_{pp}^{-1} \mathbf{b}_p, \\ \mathbf{H}_{pp} \mathbf{d}_p &= \mathbf{b}_p - \mathbf{H}_{cp}^\top \mathbf{d}_c. \end{aligned} \quad (7.7)$$

由于 $\mathbf{H}_{pp}$ 为块对角矩阵, 可逐点极高效地计算 Schur 补和最终点解。如图 7.10 所示, 降维相机系统较不稀疏, 因为其中含有关联共同观测到某点的相机对的块。该示例中, 相机 1 和相机 2 之间存在块, 而相机 1 和相机 3 之间不存在。在局部 BA 中, 降维相机系统几乎是满的, 因而可使用稠密矩阵求解器; 相比之下, 完整 BA 的关键帧数量大得多, 但它们之间共视性较少, 因此降维相机系统可从稀疏求解器中获益。

7.4.4 再议束调整

BA 已研究逾一个世纪, 第 7.4.3 节所详述的经典方法已确立, 并在许多奠基性论文中证明有效 [1107, 14, 978]。但该流水线有两项重要缺陷: 其一, 解需要对路标和相机位姿进行合适初始化; 其二, 对大规模问题, 计算和内存要求可能增长到难以承受的程度。近年来, 一系列论文解决这些缺陷, 并挑战传统计算流水线 [469, 470, 258, 259, 1170, 1171, 1172]。

关键计算瓶颈是求解降维相机系统 (7.7)。Power Bundle Adjustment [1171] 不使用迭代共轭梯度算法直接求解, 而是近似 Schur 矩阵的逆:

$$\mathbf{H}_{cc}^{\text{red}} = \mathbf{H}_{cc} - \mathbf{H}_{cp} \mathbf{H}_{pp}^{-1} \mathbf{H}_{cp}^\top, \quad (7.8)$$

并通过矩阵幂级数 [1171] 得到:

$$(\mathbf{H}_{cc}^{\text{red}})^{-1} \approx \sum_{i=0}^m (\mathbf{H}_{cc}^{-1} \mathbf{H}_{cp} \mathbf{H}_{pp}^{-1} \mathbf{H}_{cp}^\top)^i \mathbf{H}_{cc}^{-1}. \quad (7.9)$$

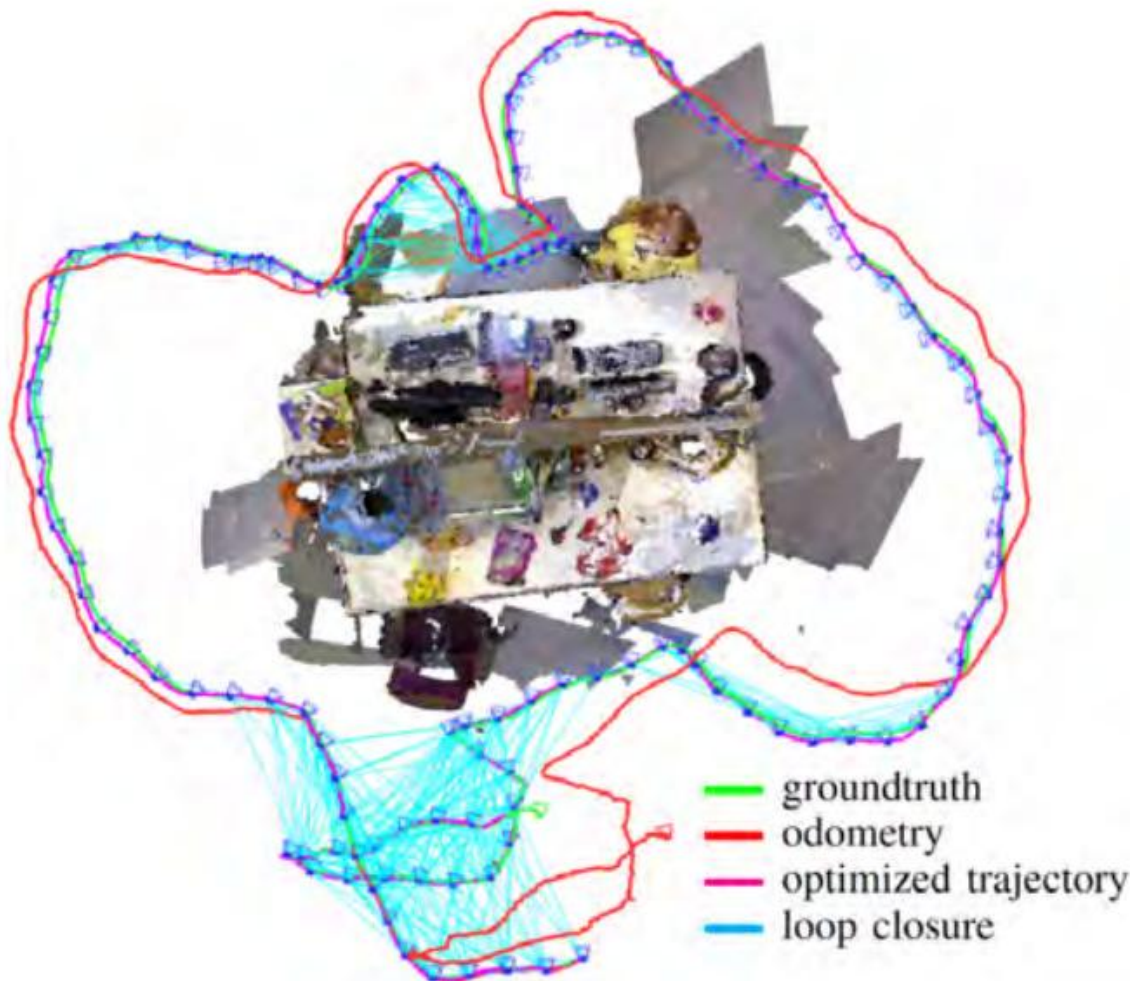
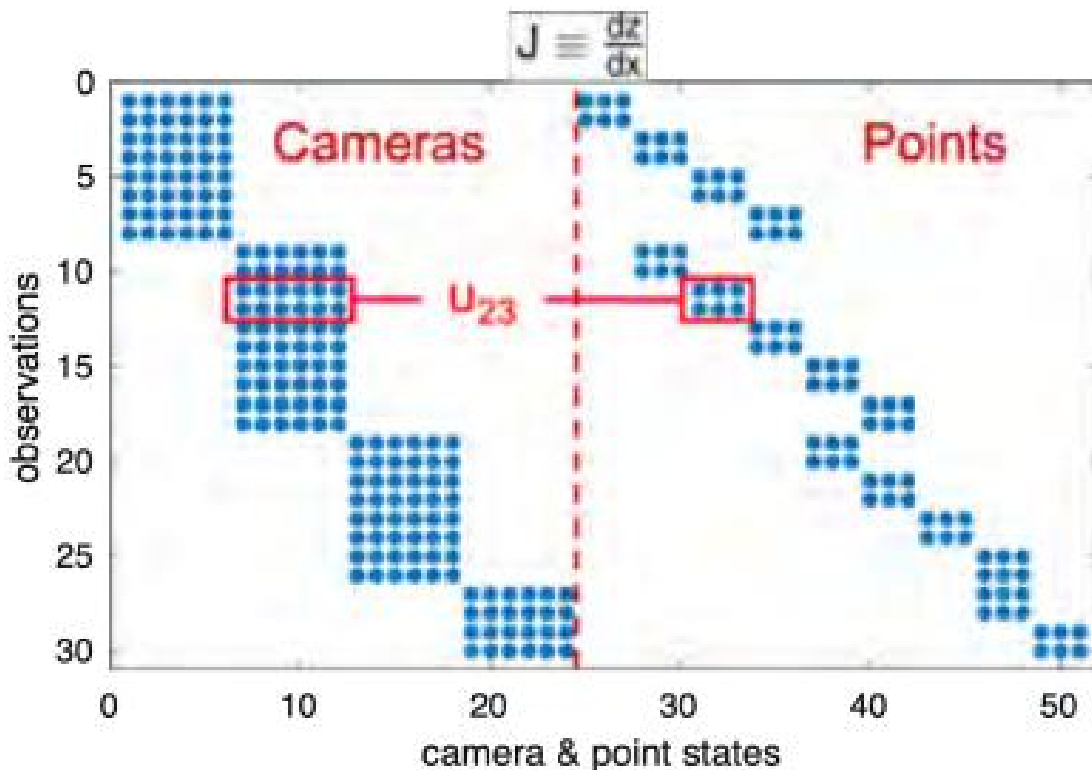


图 7.9: 图 7.9 在直接视觉 SLAM 方法中, 可进行位姿图优化, 以计算与此前估计的所有两两图像对齐全局一致的轨迹^[553]。(©2013 IEEE)

随着截断参数 m 增大, 该幂级数可证明收敛到真实逆矩阵^[1171]。其主要优点是以简单矩阵乘法替代繁重的矩阵求逆, 因而计算更快、内存效率更高。



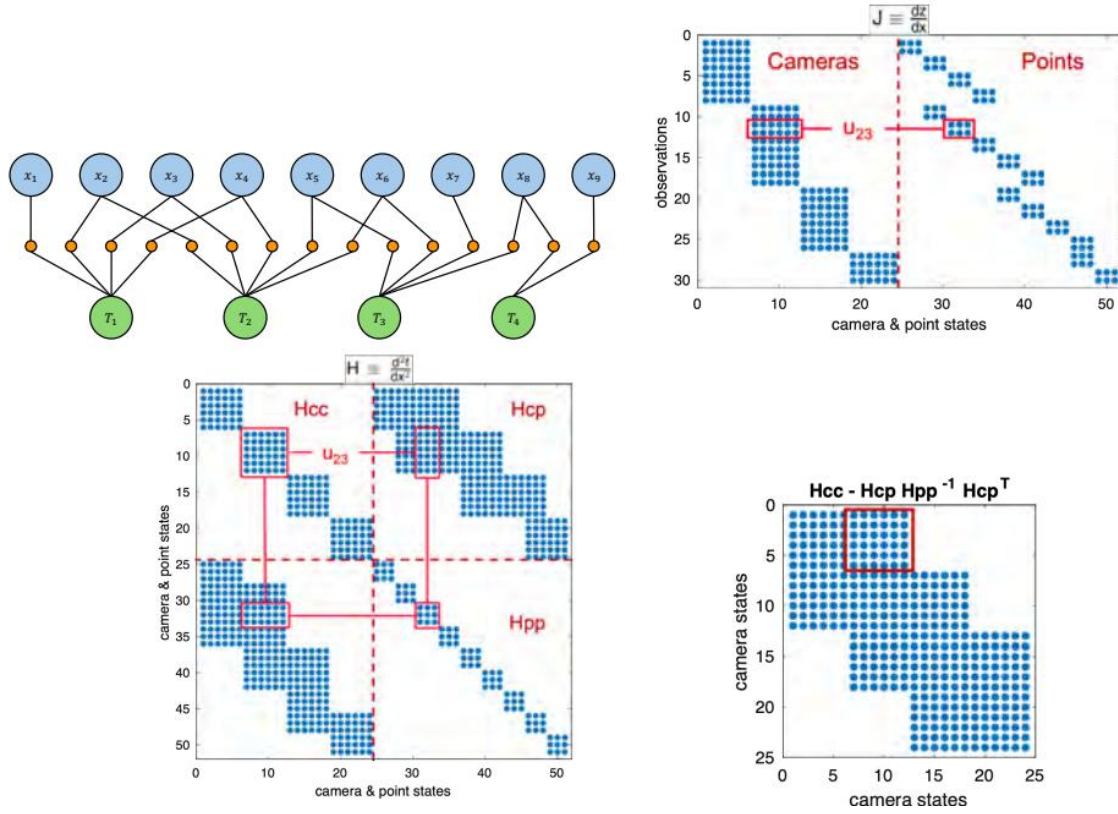
^[469, 470] 通过重新采用变量投影概念来缓解对初始化的依赖 (见图 7.11)。为此, 将 BA 问题分为两个阶段。第一阶段以通用投影矩阵替换复杂的透视投影, 使所得优化可将路标解析地表示为相机参数的函数。这消除了路标与相机位姿之间“先有鸡还是先有蛋”的依赖关系, 使优化相机位姿时具有更大的吸引域。第二阶段为投影细化, 使用所得解作为原始 (透视) 重建的初始化。尽管该策略对于超过 100 台相机时计算代价过高, 但与^[1172]所提幂级数方法结合后, 可为无需初始化的大规模 BA 问题提供可扩展解法。

7.5 完整视觉 SLAM 系统示例

LSD-SLAM (Large-Scale Direct SLAM)^[307] 是一种直接 SLAM 系统, 聚焦于稠密跟踪和半稠密建图。它依赖光度误差最小化而非基于关键点的方法, 因此在低纹理环境中特别有效。该系统可实时运行, 提供场景的半稠密重建, 适合室内和小尺度室外环境中的单目相机。

ORB-SLAM^[787, 785] 因其鲁棒性和灵活性而成为采用最广泛的 SLAM 系统之一。它集成基于 ORB (Oriented FAST and Rotated BRIEF) 描述子的关键点跟踪、有效回环检测和稀疏地图表示。ORB-SLAM 支持单目、双目和 RGB-D 相机, 因此可高度灵活地适配不同设置, 在要求高精度和鲁棒重定位能力的场景中表现出色。ORB-SLAM3^[142] 将其扩展到鱼眼相机、多地图和视觉惯性 SLAM。OKVIS^[645] 最初构想为视觉惯性里程计系统; 其最新版本 OKVIS2^[644] 是视觉惯性 SLAM 系统, 也可在仅视觉 (多相机) 模式下运行。它和各种 ORB-SLAM 版本一样, 使用关键点和描述子 (BRISK)。

Direct Sparse Odometry (DSO)^[308] 是一种用于估计三维点云和相机轨迹的直接方法。不同于 LSD-SLAM, 它在一次 Gauss-Newton 优化中联合估计相机运动和路标点。为获得实时性能, 仅更新最近 k 个关键帧, 因而形成滑动窗口光度 BA。所考虑关键帧数 k 在速度和精度之间提供折中。此外, DSO 使用带相机响应函数和暗角的完整光度标定。因此, 传统亮度恒常假设被辐照度恒常假设取代, 即假定三维世界中的相



应点随时间发出相同辐照度。该方法已扩展到双目系统^[1160]和全向相机^[739]，并已加入回环检测以减少长序列中的漂移^[365]。

在具有实时能力的方法中，已证明 DSO 等直接法比基于关键点的方法更准确、更鲁棒^[308]；但为达到最佳性能，它们通常需要良好光度标定和全局快门相机。卷帘快门效应会引入几何畸变。尽管可在直接 SLAM 方法中建模这类畸变^[980, 981]，所得方法往往不再具备实时能力。因此，通常更适合由基于关键点的方法处理，因为后者在设计上最小化几何畸变。尽管如此，在低分辨率视频中，模糊和降采样会使可靠特征点更难识别，直接法常表现更好^[1227]。此外，特征点对高效重定位与回环很有价值。因此，实用视觉 SLAM 系统常转向结合两类方法优势的混合方法。例如，^[385]将基于特征的重定位信息紧密整合到直接视觉 SLAM 方法中，进一步提高其鲁棒性和精度。

7.6 实时稠密重建

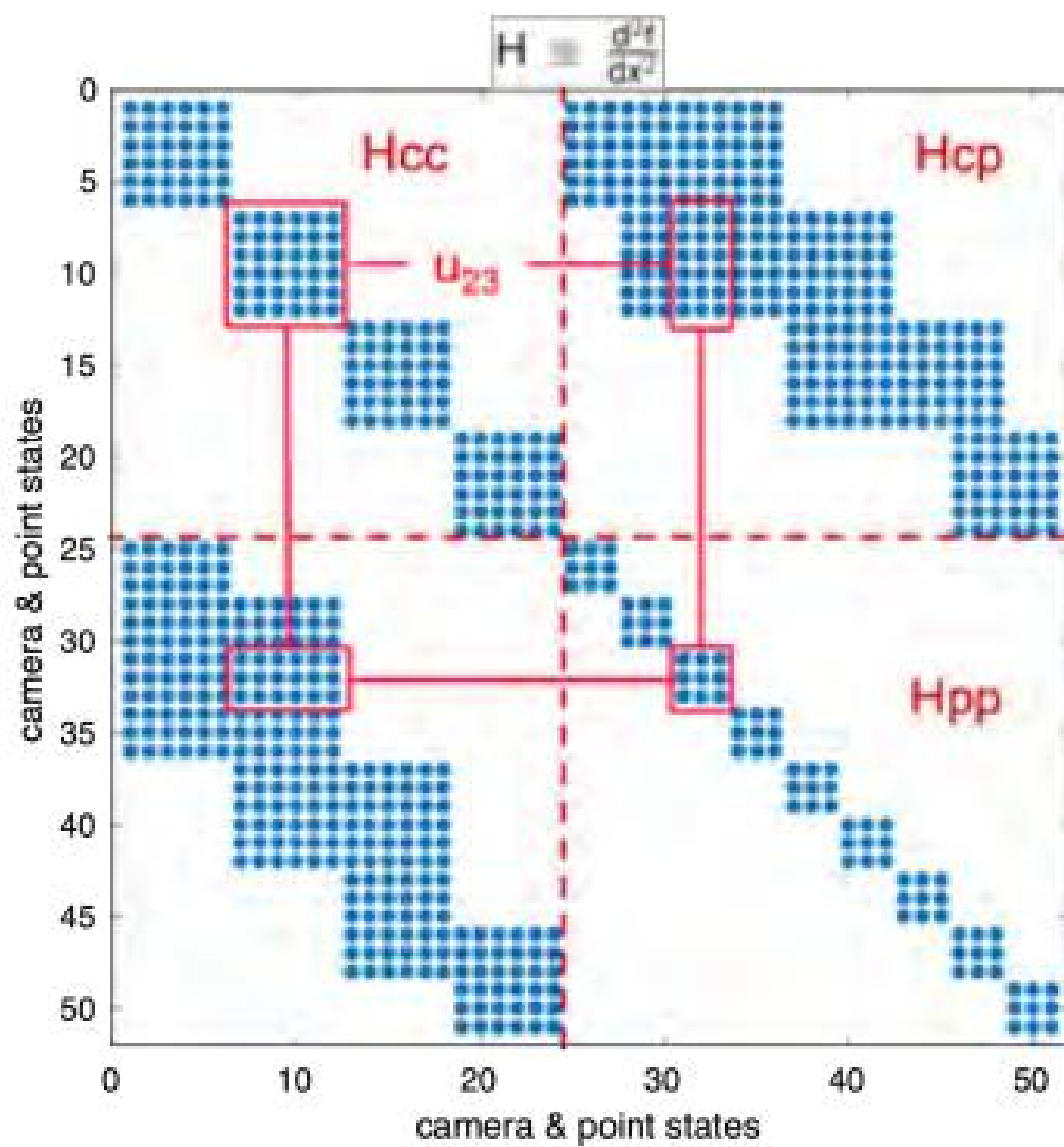
传统 BA 和 SLAM 解决的是重建相机运动和稀疏路标点集的问题；但从增强现实到自主机器人等众多应用，更希望获得所观测世界的稠密重建。为此，多年来出现了从单目相机进行实时稠密重建的多种算法^[1046, 800, 1180, 875]。传统上，它们通过最小化损失函数，借助变分方法估计连续三维结构^[1046]：

$$\min_{h: \Omega \rightarrow \mathbb{R}} \frac{1}{2} \sum_{i \in \mathcal{I}(z)} \int_{\Omega} \rho_i(z, h) dz + \lambda \int_{\Omega} |\nabla h|^2 dz, \quad (7.10)$$

其中优化对象是稠密地图 h ，它为图像平面 $\Omega \subset \mathbb{R}^2$ 中的每个像素 z 赋予深度值。残差

$$\rho_i(z, h) = \left| I_i \left(\pi \left(\mathbf{R}_w^i \mathbf{x}^w(z, h) + \mathbf{t}_w^i \right) \right) - I_0(z) \right|, \quad (7.11)$$

强制参考图像 I_0 中像素 z 的亮度与一组相邻图像 I_i 中相应像素保持一致。相应像素的获得方式是：取三维点 $\mathbf{x}^w(z, h)$ ，利用相应的旋转矩阵 $\mathbf{R}_w^i \in \text{SO}(3)$ 和平移向量 $\mathbf{t}_w^i \in \mathbb{R}^3$ 将其从世界坐标系变换到相机 i ，



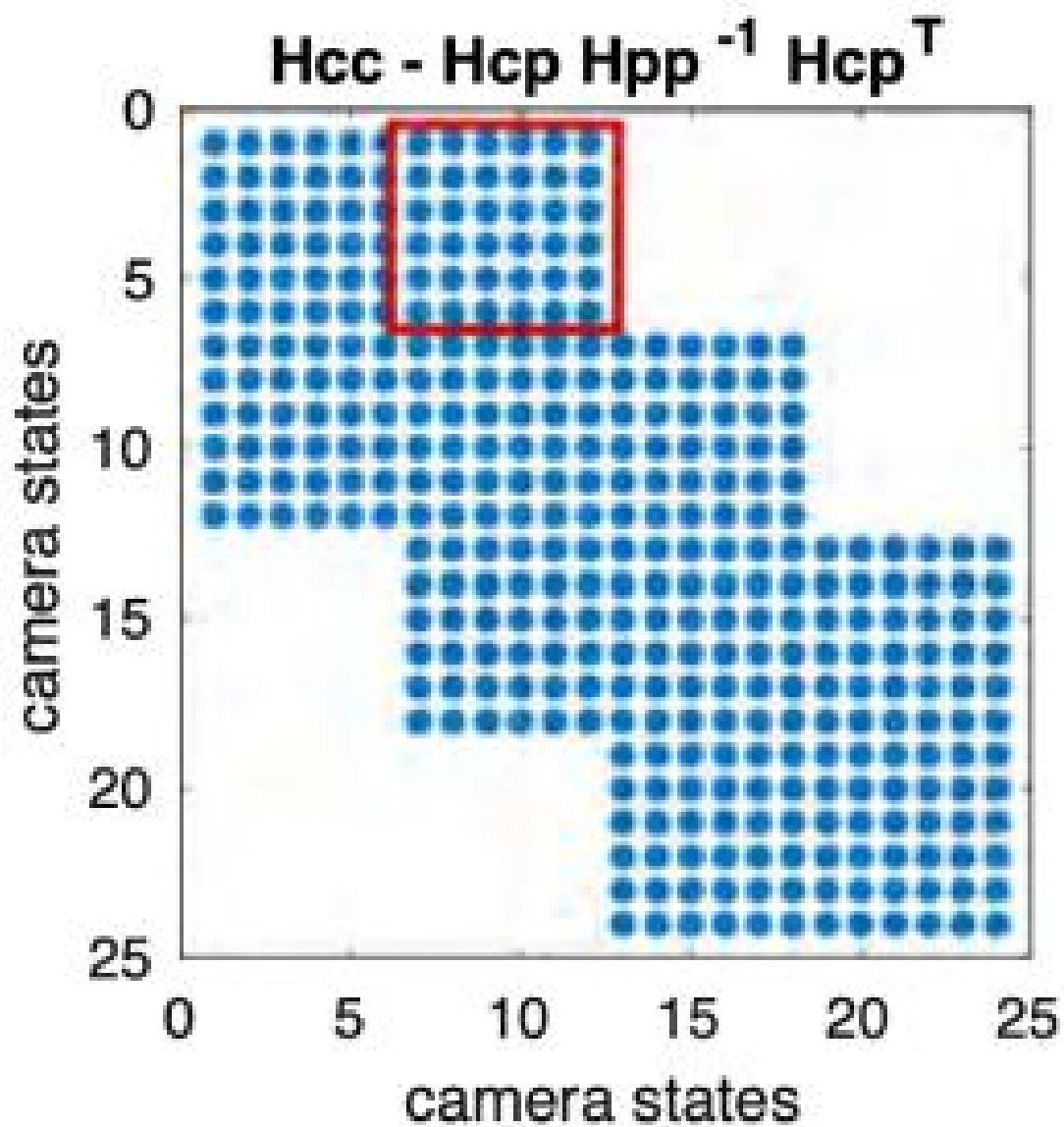


图 7.10: 图 7.10 含四台相机和九个点的玩具示例。第一行给出因子图和点观测的雅可比矩阵；第二行给出完整 Hessian 矩阵和降维相机系统的 Hessian 矩阵。

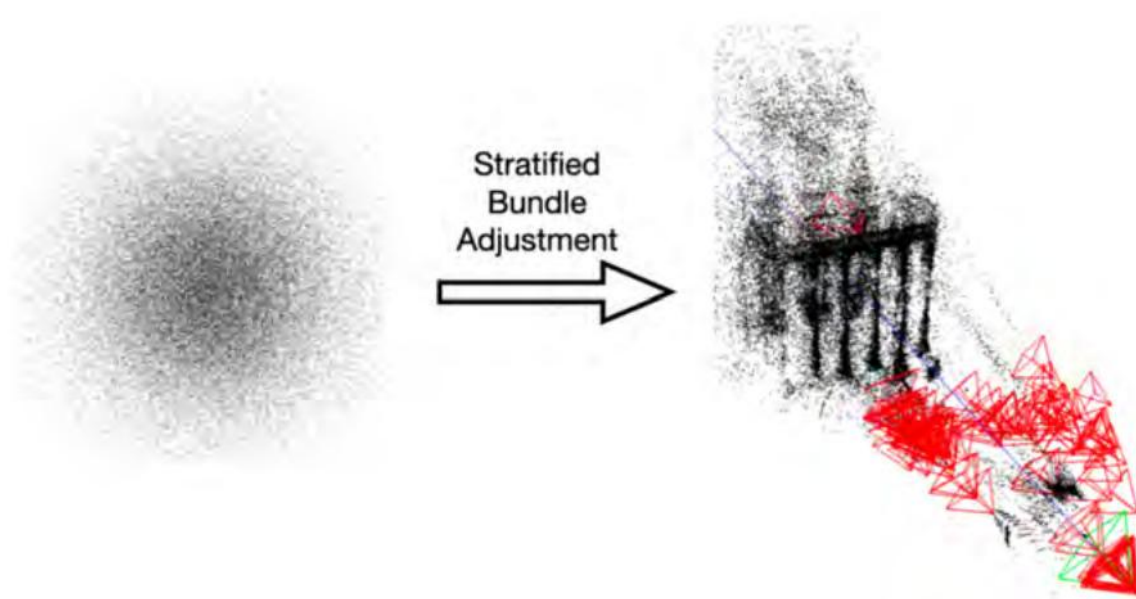


图 7.11: 图 7.11 在一系列论文^[469, 470, 1171, 1172]中, 研究者倡导使用变量投影方法和矩阵幂级数, 以时间和内存高效的方式求解无初始化的大规模束调整问题。如图所示, 可以从随机初始化出发计算相机位姿和路标。(©2024 Springer)



图 7.12: 图 7.12 可使用在 GPU 上高效并行的变分方法^[1046], 由手持单目相机(下)实时计算三维世界的稠密重建(上)。相关方法见^[800, 1180, 875]。(©2010 Springer)

最后通过模型 $\pi(\cdot)$ 将其投影到图像 I_i 中。

总变分正则项（权重为 λ ）强制所计算深度图具有空间平滑性，并如图 7.12 所示，对未观测区域产生类似肥皂膜的填充效果。

7.7 基于深度感知相机的 SLAM

随着 Microsoft Kinect 的推出，深度感知相机成为普及设备。这类所谓 RGB-D 相机通常基于结构光或飞行时间（time-of-flight），提供深度图像流，往往还伴有彩色图像流。因此，从概念上说，它们介于标准相机和 LiDAR 传感器之间。但与 LiDAR 传感器不同，它们立即提供深度值的二维阵列。配合合适算法，RGB-D 相机是强大的三维感知设备；但受限于室内应用（基于红外的感知会受到阳光干扰）和一定测程（通常约 5 米以内）。

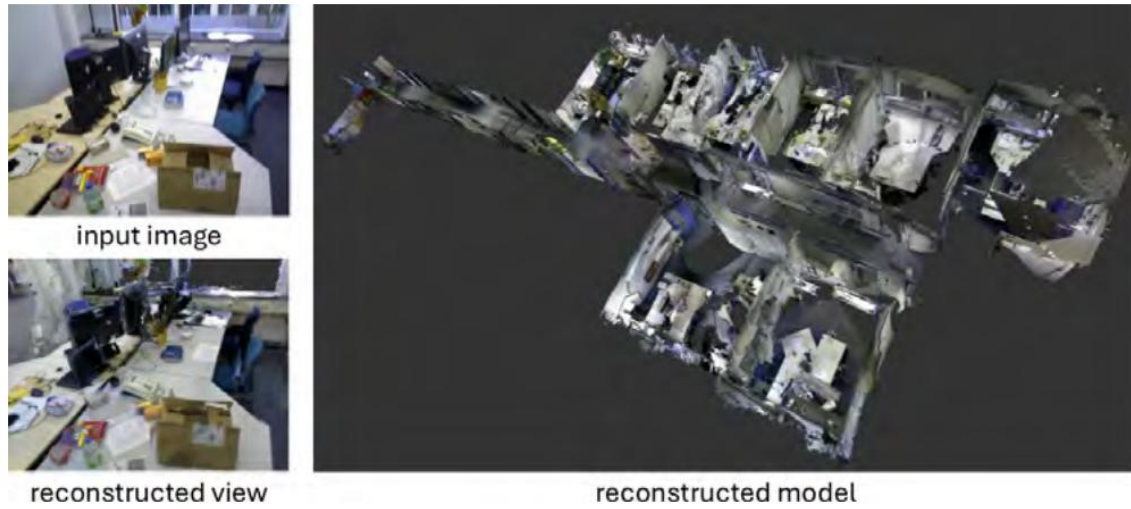


图 7.13: 图 7.13 由移动 RGB-D 相机得到的含多个办公室的大尺度走廊场景稠密重建^[1030]。使用八叉树^[1030]或体素哈希^[808]以及直接相机跟踪^[554]，此类重建已证明可在 GPU^[1030]乃至平板电脑 CPU^[1031]上实时运行。（©IEEE）

KinectFusion^[801]建立在早期范围图像融合工作^[234]的基础上，提出了从移动 RGB-D 相机重建相机运动和三维结构的方法。将各种深度图像融合为一致三维重建的基本思路是：将每幅深度图像编码为（投影）有符号距离函数 $d_i(\mathbf{x})$ ；该函数为每个体素 $\mathbf{x} \in V$ 编码到最近表面点（沿视线）的有符号距离。随后可将各个距离函数加权平均，计算所有体素的聚合距离函数 $D(\mathbf{x})$ ：

$$D(\mathbf{x}) = \frac{\sum_i \omega_i(\mathbf{x}) d_i(\mathbf{x})}{\sum_i \omega_i(\mathbf{x})}. \quad (7.12)$$

在深度方向噪声服从高斯分布的假设下，这个加权平均正是距离函数的最大似然估计。为提高鲁棒性，通常对截断有符号距离函数取平均，使每个感知到的表面点只对重建产生局部影响。权重 $\omega_i(\mathbf{x})$ 编码相应表面测量的置信度（它依赖传感器，且通常随到物体距离增加而衰减）。为填补重建中的孔洞，可以使用后处理技术^[801]，或修改上述加权方案以确保重建的水密性^[1044]。

早期 RGB-D SLAM 方法通过使用 ICP 算法^[78]对齐相应深度点云来跟踪相机；随后工作提出直接最小化度量两个连续 RGB-D 帧 (I_1, d_1) 与 (I_2, d_2) 色彩和深度一致性的残差^[554]：

$$r_I(\boldsymbol{\xi}) = I_2(\tau_g(\mathbf{z})) - I_1(\mathbf{z}), \quad r_d(\boldsymbol{\xi}) = d_2(\tau_g(\mathbf{z})) - [g\pi^{-1}(\mathbf{z}, d_1(\mathbf{z}))]_z, \quad (7.13)$$

其中 $g \in \text{SE}(3)$ 表示待估刚体运动， $\tau_g(\mathbf{z}) = \pi g \pi^{-1}(\mathbf{z}, d_1(\mathbf{z}))$ 表示对应像素间诱导的变形， $[\cdot]_z$ 返回一点的 z 分量。随后可用合适分布拟合所有残差 $\mathbf{r} = (r_I, r_d)$ 的分布，并通过在 $\boldsymbol{\xi} \in \mathfrak{se}(3)$ 上进行由粗到细的

Gauss-Newton 优化, 确定相机变换 $g = \exp(\xi)$ 的最大后验估计^[1029, 553]。与经典 ICP 方法相比, 该直接跟踪方法在既定基准上将均方根跟踪误差降低近一个数量级, 同时显著更快; 详见^[554]。

在更大尺度上建图时, 均匀体素表示占用内存过多。因此, 通常采用体素哈希^[808]或八叉树^[1030]等自适应表示, 见图 7.13 和第 5 章。

后续方法还通过采用如 Kintinuous^[1183]的移动融合窗口, 或如 ElasticFusion^[1184]那样纳入回环并通过形变图变形整个稠密地图表示, 展示了对更大场景的可扩展性。

7.8 将视觉与其他模态相结合

出于多种原因, 建议将视觉与其他模态结合。这既可提供额外鲁棒性和精度, 也可提供绝对尺度信息。具体而言, 惯性传感器提供度量尺度以及高度可靠、鲁棒的局部相对运动测量; GPS/WiFi 则可用于全局(重)定位和地理定位。这些互补输入解决仅视觉系统固有的局限, 在实际应用中不可或缺。

7.8.1 惯性测量单元 (IMU)

IMU 提供角速度和线加速度的高频测量, 从而支持对局部运动的精确估计。融合传感信息的一种简单方式是松耦合: 先独立地将每个传感器的信息汇总为位姿估计, 再使用卡尔曼滤波融合这些估计。该方法在实践中常有良好表现, 例如用于四旋翼^[306]或纳米飞行器^[291]的自主导航, 但不如紧耦合传感信息集成精确。

早期紧耦合方法采用 EKF 方案, 在预测步骤积分 IMU 运动学量, 并以视觉关键点测量作为更新, 例如奠基性的 MSCKF^[777]。另一种方式是采用第 11 章详述的因子图方法, 构建滑动窗口或批量优化器。图 7.14 给出了一个用于双目惯性里程计的因子图^[1116], 它将双目 LSD-SLAM 与 IMU 信息结合。图 7.15 展示紧耦合 IMU 信息如何减小漂移, 从而获得更清晰、更精确的重建。ORB-SLAM3^[142]、OKVIS2^[644]等最先进的基于关键点方法同样采用这种因子图紧耦合。

实践中, 一些最准确的视觉惯性里程计系统以惯性积分为基础, 主要使用视觉来校正由随时间(两次)积分的噪声和偏置引起的快速增长漂移。此外, IMU 使运动的度量尺度和传感器相对于重力的姿态可观测, 从而将系统的规范自由度缩减为仅四个未知量(全局 x, y, z 位置和偏航角); 而仅视觉系统有七个未知量(全局 x, y, z 、滚转角、俯仰角、偏航角和尺度)。从多方面看, IMU 与视觉作为感知模态相互互补, 因此非常适合在视觉惯性 SLAM 或里程计系统中结合。

多年来已提出大量视觉惯性里程计/SLAM 方法, 通常是现有视觉 SLAM 系统的扩展。常用方法包括 ORB-SLAM 的视觉惯性版本^[786]、Direct Sparse Visual-Inertial Odometry^[1136]、VINS-Mono^[896]、BASALT^[1118]和 DM-VIO^[1135]。最后一种是单目惯性表述, 使用延迟边缘化(delayed marginalization)概念, 以更好地刻画相应传感器中运动的可观测性。

7.8.2 用于全局定位的 GPS 和 WiFi

IMU 改善局部运动估计, 而 GPS 和 WiFi 对全局定位至关重要, 尤其在大尺度环境中。室外环境中, GPS 提供绝对位置数据, 使系统可将 SLAM 地图锚定到全局坐标。这对自动驾驶车辆等室外应用至关重要^[1181, 193], 见图 7.16。室内场景中, 在 GPS 信号不可用时, WiFi 信号可实现粗定位, 补充基于视觉地图的定位。

7.9 延伸阅读与近期趋势

视觉 SLAM 是一个极其活跃且不断变化的研究领域。尽管我们尝试全面概述经典视觉 SLAM 方法, 本章不可避免地仅覆盖了相关工作的一个部分; 希望读者查阅众多被引文献, 以获得更深入的了解。

关键点检测与匹配。在特定设置中, 基于学习的关键点相比传统手工关键点展现出前所未有的不变性和适应性, 基于深度学习的关键点提取技术正在改变这一领域。这一转变不仅增强了关键点检测与描述, 还显著推动了关键点匹配技术。不同于经典方法中依赖人工设计启发式规则进行描述子匹配, 基于学习的方法如今融入全局空间感知以更鲁棒地建立对应关系。特别是, SuperGlue^[971]和 MAST3R^[642]等技术利用图神

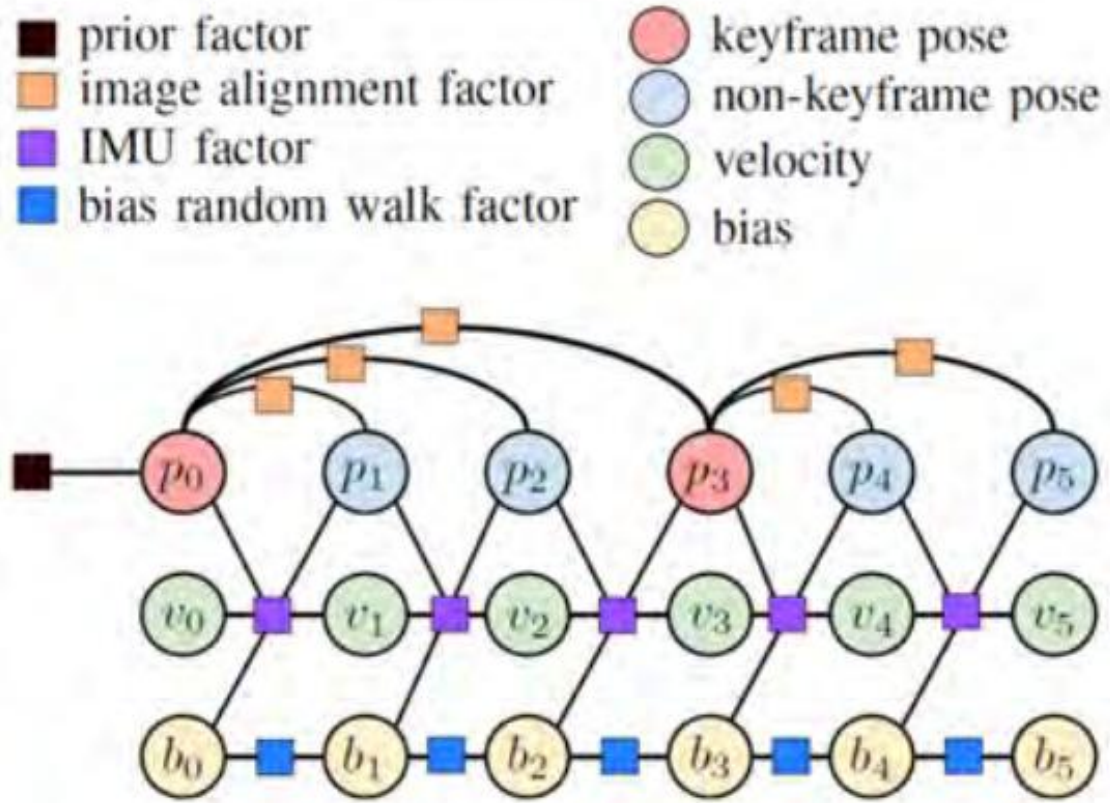
Visual SLAM

图 7.14: 图 7.14 可使用因子图以紧耦合方式优雅地集成多种传感器。这是视觉与 IMU 紧耦合融合的因子图示例^[1116]；它显著提高相机运动与三维结构的精度和鲁棒性，见图 7.15。(©2016 IEEE)



图 7.15: 图 7.15 将 IMU 测量与直接图像对齐紧密融合，相比仅依赖图像对齐的纯视觉里程计系统（右），可得到更准确的位置跟踪（左）。融合通过图 7.14 所示因子图实现。重建点云来自纯里程计，未强制实施回环^[1116]。(©2016 IEEE)

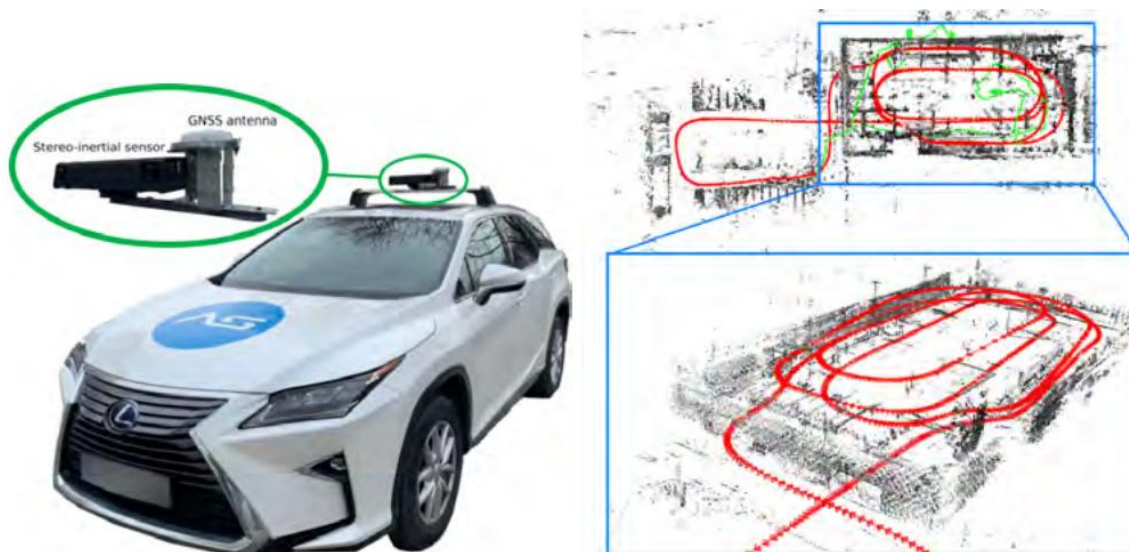


图 7.16: 图 7.16 将包括双目相机、IMU 和 RTK-GPS 信息在内的多种传感器 (左) 通过因子图紧耦合融合, 可得到高度准确、鲁棒的轨迹和点云; 如右图所示, 它适用于室内外环境, 并展示了汽车行驶于多层停车场中的重建^[1181]。(©2020 Springer)

经网络 (graph neural networks, GNN) 和 Transformer, 通过考虑更广泛的图像结构来细化匹配。这些方法可适应极端视点变化、光照变化和遮挡, 解决传统匹配器的根本局限; 在这些情形中, 传统局部描述子比较通常表现不佳。

SuperGlue 通过整合自注意力和交叉注意力机制来增强特征匹配, 从而消解重复纹理和遮挡导致的歧义。不同于仅基于局部描述子运行的传统关键点匹配器, SuperGlue 纳入上下文信息, 显著提升室内外环境中的准确性。其最优匹配层进一步确保稳健建立关键点对应, 并在需要时允许关键点不匹配, 因此对真实场景十分有效。类似地, MAST3R 将这一思路扩展为三维感知特征匹配: 它从两幅图像重建三维场景表示, 以改善无纹理区域和极端视点变化下的性能。在此基础上, MAST3R-SLAM^[789] 将这些进展整合到 SLAM 流水线中, 改进相机位姿估计、全局地图一致性和回环策略。从手工二维特征匹配转向基于学习、具备上下文感知且融入三维信息的方法, 标志着视觉 SLAM 范式转变, 增强复杂环境中的场景理解、跟踪鲁棒性和长期稳定性。

总体而言, 用于图像匹配和视觉 SLAM 的关键点提取, 已从传统手工方法显著演进到有望在挑战条件下提供更强鲁棒性的深度学习方法。经典方法至今仍因高效率和可解释性而十分重要, 但在无纹理表面、极端视点变化和光照变化情形中表现受限。深度学习已解决这些局限, 推动研究转向更具适应性和上下文感知的关键点检测与匹配技术。不过, 在资源受限平台上平衡计算效率和实时性能, 以及确保获得多样、无偏的训练数据集, 仍是挑战。未来研究很可能聚焦于优化这些技术, 同时最大化准确性和效率, 使其更适用于大规模真实世界应用。

其他前沿。需要强调的是, 近年视觉 SLAM 还有许多令人振奋的进展。其中包括将视觉 SLAM 推广到包含移动且可能可变形物体的动态环境。此外, 基于学习的视觉 SLAM 方法日益流行: 在大量发表工作中, 经典视觉 SLAM 流水线越来越多的组件 (特征提取、对应关系估计、图像对齐、相机跟踪、BA、稠密重建等) 正被基于学习的表述增强或替代。本手册第三部分将更详细讨论这些思想。

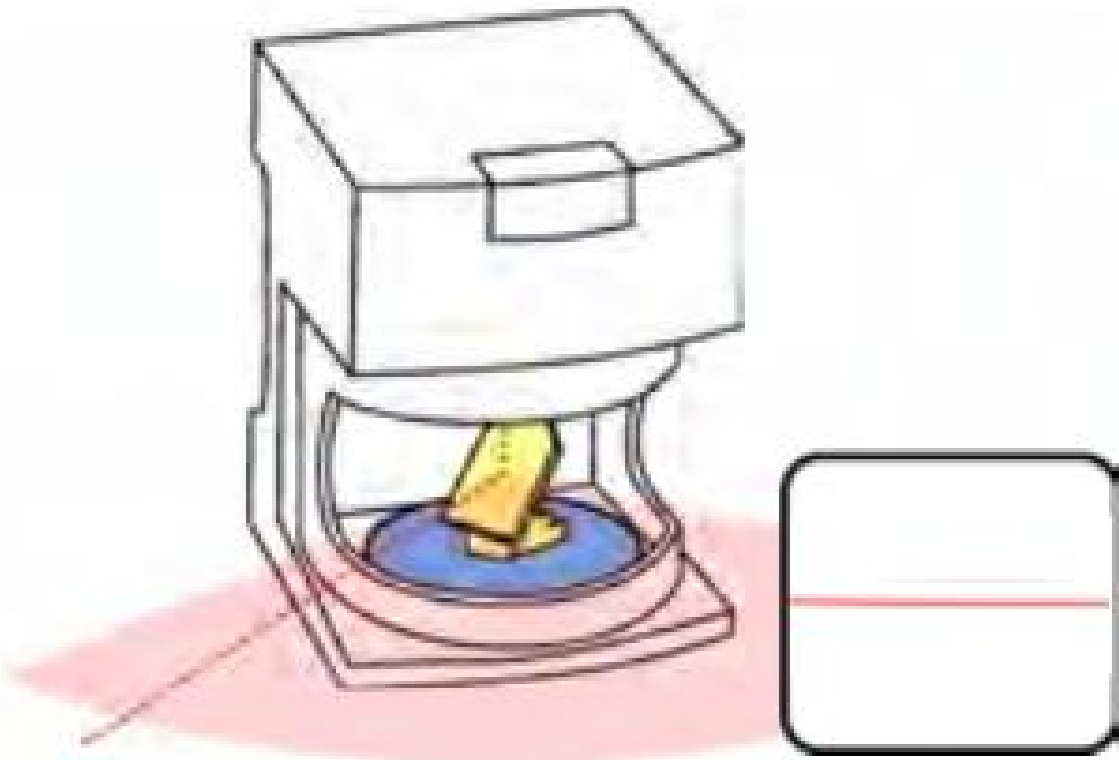
第八章 LiDAR SLAM

作者：Jens Behley、Maurice Fallon、Shibo Zhao、Giseop Kim、Ji Zhang、Fu Zhang 和 Ayoung Kim

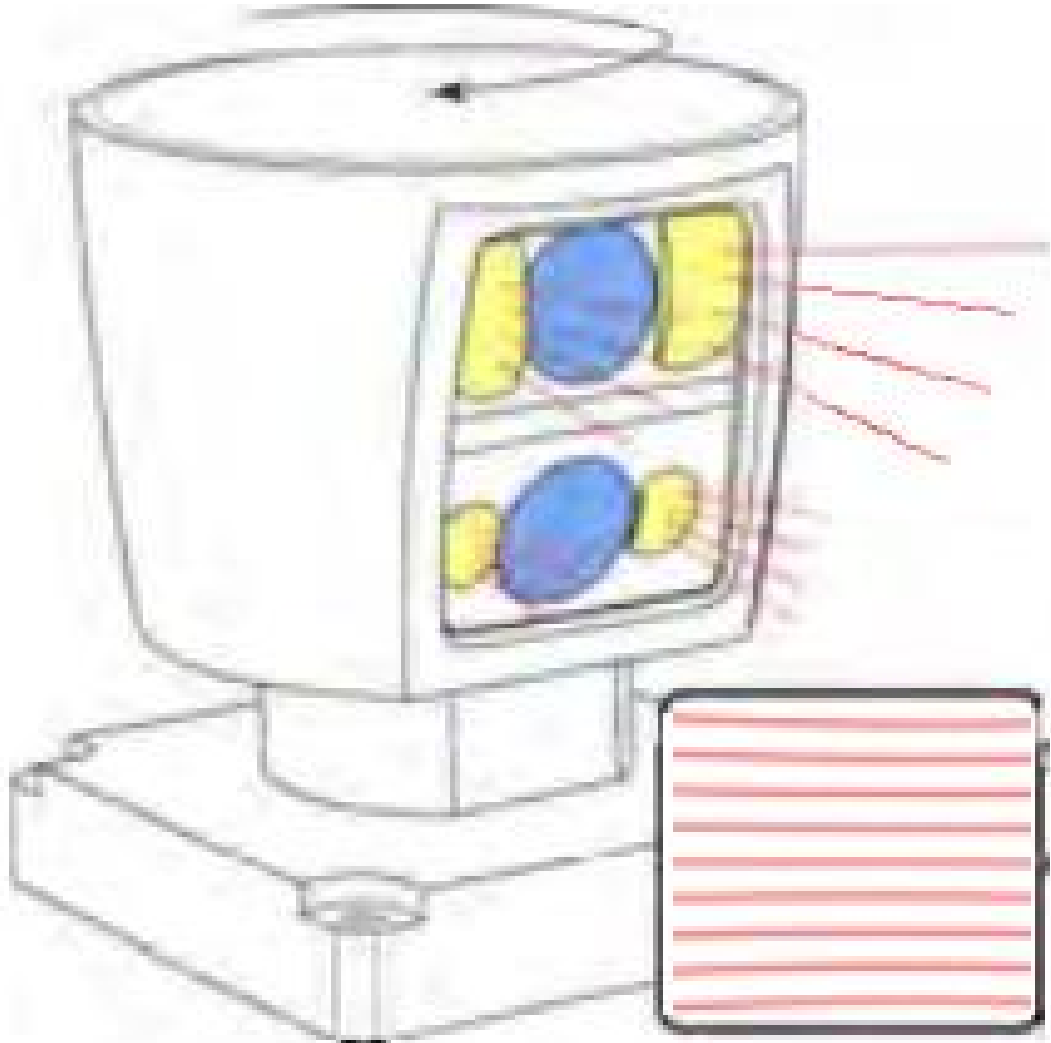
除相机外，LiDAR 传感器是机器人学中最常用的感知模态之一。LiDAR 是一种主动发射激光脉冲、测量脉冲经表面反射并返回探测器所需时间的技术，由此直接测量到表面的距离。LiDAR 传感器可用于感知周围环境结构，也可用于估计传感器的运动或自身位置。

LiDAR 技术的发展始于 20 世纪 60 年代和 70 年代，当时的固定式系统主要用于大气研究、地形测绘和军事应用。这些早期 LiDAR 系统笨重且昂贵，不适合移动应用。20 世纪 80 年代，激光技术和计算能力进步，使 LiDAR 系统更紧凑且价格更低。这些系统仍主要固定使用，用于地形测绘和环境监测等应用^[248, 683]。20 世纪 90 年代，LiDAR 传感器与全球定位系统（GPS）及 IMU 集成，开始支持最早的移动 LiDAR 建图系统。这些系统通常安装在车辆或飞机上，以创建大范围的详细三维地图^[499]。20 世纪 90 年代末，研究者开始探索在机器人中使用 LiDAR 进行实时 SLAM。二维 LiDAR 推动并催化了 21 世纪初概率建图和 SLAM 的发展；随着 2004 年 DARPA Grand Challenge 和 2007 年 DARPA Urban Challenge 的举行，三维 LiDAR 在机器人和自动驾驶车辆中的地位更加突出。

本章首先简要回顾不同类型 LiDAR 传感器的基础技术（第 8.1 节），随后讨论 LiDAR 里程计（第 8.2 节）和基于 LiDAR 的地点识别（第 8.3 节）。此外，还将讨论将里程计和地点识别整合到完整 LiDAR SLAM 系统中的方法（第 8.4 节）。最后给出延伸阅读并讨论新趋势（第 8.5 节）。



(a) 二维 LiDAR

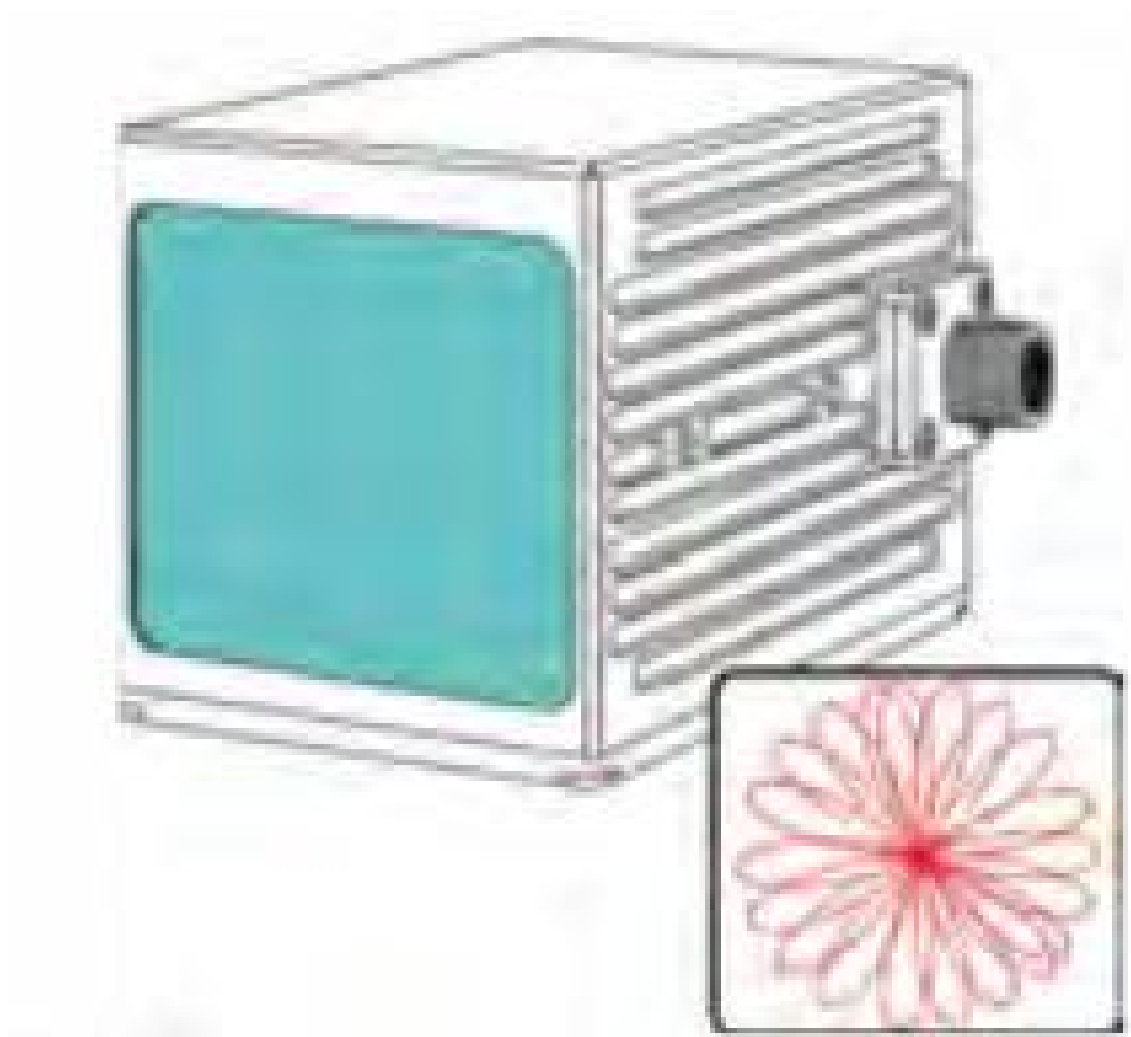


(b) 三维旋转 LiDAR

(c) 螺旋扫描图 8.1 常见 LiDAR 传感器类型及其光束模式。(a) 采用旋转发射镜（黄色）的标准二维 LiDAR 传感器。编码器盘（蓝色）用于测量反射镜的旋转角度。(b) 机械式多光束 LiDAR 示例：多个发射器（黄色）发射激光束，再由探测器（蓝色）检测。整个传感器头旋转以产生 360° 水平视场。(c) 宏观扫描 Risley 棱镜 LiDAR，具有螺旋扫描模式，只在镜头窗口方向（青色）进行测量。由于螺旋模式，它能够随时间产生更稠密的点云。

8.1 LiDAR 感知基础与分类

在 LiDAR 传感器内部沿一个或两个轴旋转激光发射器和探测器，就可以逐步构建传感器周围环境的详细点云。最基本的原理是飞行时间（time-of-flight, TOF）：发射激光脉冲，并根据测得的光脉冲返回探测器所需时间推断距离。TOF LiDAR 传感器能够采集高分辨率测量，但对外部光照敏感；外部光会降低信噪比（signal-to-noise ratio, SNR）^[598]，进而降低测量精度和频率。除 TOF 外，原本为雷达传感器开发的调幅连续波（Amplitude Modulated Continuous Wave, AMCW）和调频连续波（Frequency Modulated Continuous Wave, FMCW）技术也已用于 LiDAR。



LiDAR 传感器可按其感知机制分类^[935]，包括机械式 LiDAR、扫描式固态 LiDAR、闪光 LiDAR，以及使用宏观扫描 Risley 棱镜的传感器。其中，本节考察两类常用类型：二维/三维机械式 LiDAR 和宏观扫描 LiDAR。

机械式 LiDAR 是最常见的 LiDAR 类别。它们使用旋转组件引导激光束，但受到机械磨损和数据采集速率较低的限制。最简单的二维机械式 LiDAR 使用旋转反射镜引导单束激光并测量距离，如图 8.1(a) 所示。借助编码器盘，可以测量 LiDAR 光束的角度方向，并将其与距离测量配对，从而生成二维剖面测量。由于其工作原理，这类 LiDAR 只能在单个二维平面内扫描。

单个传感器进行完整三维扫描的需求，源于 21 世纪初的 DARPA Grand Challenges（早期自动驾驶汽车竞赛），并促成了 Velodyne HDL-64E 这一先驱传感器的开发。挑战赛中大多数队伍都使用了该传感器^[1115, 772, 536]。在三维机械式 LiDAR 中，多个激光发射器安装在同一旋转机构上，以不同俯仰角指向并采集独立的范围测量；整个机构绕方位轴旋转 360°，如图 8.1(b) 所示。所得点云能够详细描绘物体和传感器周围环境的三维形状，如图 8.2 所示，并已在多项研究中讨论^[1178, 557, 935]。随后，随着 LiDAR 传感器尺寸和价格显著下降，该技术不断发展。

近年来出现了更多原型传感器，利用固态 LiDAR、Risley 棱镜和多边形反射镜等多种物理属性。不同于传统扫描激光器，其中许多传感器使用微机电系统（micro-electromechanical systems, MEMS）^[467] 反射镜或光学相控阵（optical phased arrays, OPA）^[446]，以避免或至少减少机械旋转。这一点十分重要，因为主动驱动的机械部件减少后，机械磨损降低，可提高 LiDAR 在环境建图中的使用寿命和可靠性。

Risley 棱镜^[668] 是一项值得注意的进展，它能够以很小的实体运动快速、可控地偏转光束。这一创新得到更紧凑的传感器，但目前视场较为有限。

上述所有传感器都产生一组独立范围测量，每次测量还带有强度（也常称反射率）值，即 LiDAR 光束被反射的程度。将单个光束的范围测量及其关联角度结合后，可以转换为二维或三维点云。然而，近期进展已在默认范围测量之外引入其他测量能力。例如，FMCW LiDAR 以变化频率连续投射射光，并通过检测到的频移测量光束命中物体的相对速度。这类似于 FMCW 在雷达中的使用。该方法对动态环境或具有挑战性的场景很有用^[1201]，但相较其他变体通常更复杂、成本更高。另一种创新感知机制是闪光 LiDAR，它可以提供类似光度测量的环境通道，与相机获得的通道相似。这些技术在功耗、重量和成本上各有不同，为不同应用中的 LiDAR 里程计和 SLAM 提供了多种选择。

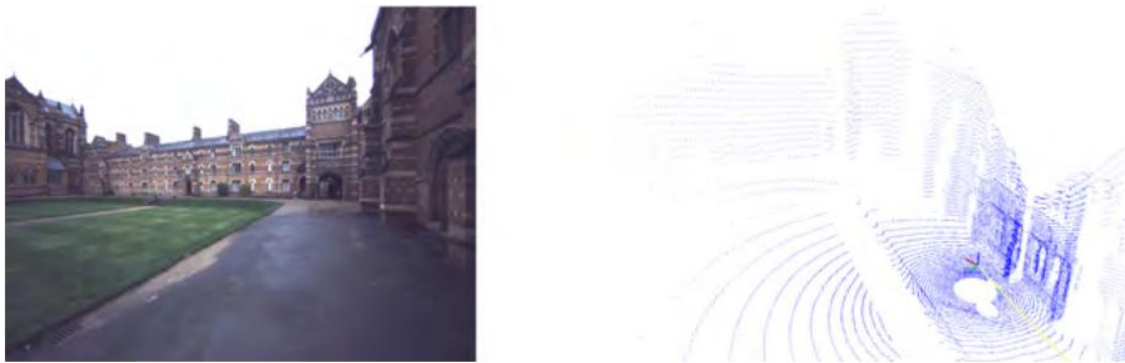


图 8.1: 图 8.2 单次多光束 LiDAR 扫描及对应图像示例。扫描由具有 104° 垂直视场的 64 光束 Hesai QT64 扫描器完成。注意其中独立的扫描线，以及设备附近（右侧三维视图中的彩色坐标轴附近）点云较稠密、远处点云明显更稀疏的现象。

8.2 LiDAR 里程计

LiDAR SLAM 的第一个构件是 LiDAR 里程计。给定一次 LiDAR 扫描和过去的观测（可以是单次扫描，也可以是聚合到局部地图中的多次扫描），LiDAR 里程计的目标是在实时条件下估计机器人或车辆的增量自身运动。这里，扫描（scan）指 LiDAR 传感器采集的一次数据扫描或周期。更具体地说，一次扫描通常

表示传感器的一次完整旋转或完整扫描，为特定时刻的周围环境提供上下文快照。因此，扫描通常带有时间戳，以便按顺序排列并作为连续观测处理。

LiDAR 里程计的核心技术是扫描配准，也称扫描匹配。扫描配准将两次扫描精确对齐，以估计它们之间准确的相对变换。一次扫描实际上是一组点或点云。文献中已经提出多种重要点云配准算法，包括 Horn 方法^[477]和 ICP。Horn 方法可以给出闭式解，但需要预先确定对应关系，限制了其在真实场景中的适用性。相比之下，ICP 算法^[79, 959]不需要预先知道对应关系，因此成为确定点云相对变换的更常用基础技术。下一节将进一步讨论 ICP。

LiDAR SLAM 的早期发展可追溯到 Lu 和 Milios 的奠基工作^[701]。他们引入位姿网络的思想，首次提出全局一致的二维范围扫描配准，这一概念与现代位姿图方法十分接近。该工作还通过定义二维扫描配准基础，为 LiDAR 里程计奠定了基础。^[826, 596]等工作对扫描到扫描匹配的概率建模作出了重要贡献。在二维扫描匹配中，点和线是常用选择；Olson^[825]则引入了利用相关技术进行实时配准的有影响力方法。

三维 LiDAR 的早期扩展建立在这些二维扫描技术之上：通过主动使传感器点头运动^[433]或旋转运动^[100]，或者被动利用人类携带者^[102]或车辆^[100]的运动，积累更稠密的三维点云。这些三维点云支持 LiDAR SLAM 的三维扫描匹配，但由于数据量增加，也带来显著计算挑战。LOAM^[1269]解决了这些挑战，展示了实时扫描匹配能力，并成为后续 LiDAR 里程计和 SLAM 方法的基础。

8.3 扫描配准基础

扫描配准是 LiDAR 里程计和建图系统的基本组件，它通过对齐两次扫描实现准确配准和建图。目标是找到一个变换，即旋转 $\mathbf{R} \in \text{SO}(3)$ 和平移 $\mathbf{t} \in \mathbb{R}^3$ ，将其中一次扫描（例如传感器最近采集的扫描）尽可能对齐到另一次扫描（例如先前扫描或局部地图），同时得到采集两次扫描时的相对位置。研究者已经开发了大量具有高精度、鲁棒性和低计算代价的扫描配准技术与算法。

迭代最近点及其变体。如第 5 章所述，点云是三维坐标系中的点集，可数学表示为 $\mathbf{P} = \{\mathbf{p}_i \in \mathbb{R}^3 \mid i = 1, 2, \dots, N\}$ ，其中 $\mathbf{p}_i = (x_i, y_i, z_i)$ 表示点的三维坐标。对于扫描配准，ICP 算法^[79]最小化两个点云 $\mathbf{P}$ 和 $\mathbf{Q}$ 之间的总配准误差。下面将 $\mathbf{P}$ 记为源点云， $\mathbf{Q}$ 记为目标点云。

ICP 通过迭代确定每次优化迭代 k 的变换 $\mathbf{R}^k, \mathbf{t}^k$ ，以最小化总配准误差。该误差由不同距离度量 d 测量，定义为

$$\mathbf{R}^k, \mathbf{t}^k = \arg \min_{\mathbf{R}, \mathbf{t}} \sum_{(\mathbf{p}, \mathbf{q}) \in \mathbb{C}} d(\mathbf{p}_i, \mathbf{R}\mathbf{q}_i + \mathbf{t}), \quad (8.1)$$

其中，源点云 $\mathbf{P}$ 与目标点云 $\mathbf{Q}$ 之间的对应关系集合 $\mathbb{C}$ 为

$$\mathbb{C} = \{(\mathbf{p}, \mathbf{q}) \mid \mathbf{p} \in \mathbf{P}, \mathbf{q} \in \mathbf{Q}\}. \quad (8.2)$$

在 ICP 算法中，通过在每次迭代中根据上一次迭代 $k-1$ 的变换重新计算对应关系集合 $\mathbb{C}$ ，来迭代确定源点云与目标点云之间的变换；上一次变换由旋转 $\mathbf{R}^{k-1}$ 和平移 $\mathbf{t}^{k-1}$ 给出。

为最小化式 (8.1) 中的总配准误差，需要明确两个方面：

1. 使用何种几何关系定义距离度量 $d(\cdot)$ ？目标是尽可能紧密地对齐两个点云，而不定义距离度量就无法确定这种紧密程度。

2. 如何确定用于最小化的对应关系集合 $\mathbb{C}$ ？这涉及为源点 $\mathbf{p} \in \mathbf{P}$ 找到对应的目标点 $\mathbf{q} \in \mathbf{Q}$ 。

下面讨论解决这两个方面的常见方法。

8.3.1 配准残差中的距离度量

上面的第一个问题是决定使用哪些几何元素定义残差，必要时可超越简单的点到点距离。通常，点、线和平面是式 (8.1) 中用于定义距离的最常见几何元素，如图 8.3 总结所示。

点到点 ICP 是最基本的方法，它最小化两个点云中对应点之间的欧氏距离。Zhang^[1281]以及 Besl 和 McKay^[79]关于 ICP 的奠基工作，将形状（例如曲线和表面）表示为点集，从而把形状匹配表述为点匹配问题。这种点到点代价直接而简单，但可能对噪声、离群值和测量点稀疏性敏感。点到线距离也常作为误差度

8.2 LiDAR Odometry

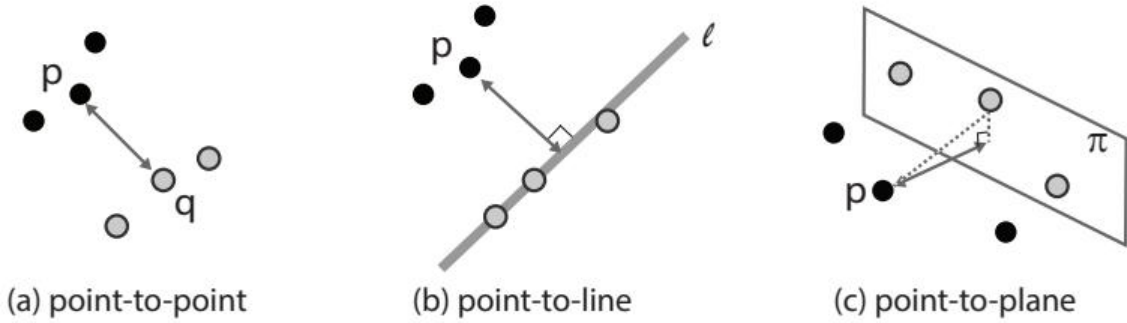


图 8.2: 图 8.3 ICP 中常用的距离度量。(a) 点到点距离很直接, 即两个点之间的欧氏距离。(b)、(c) 点到高层特征 (例如线或平面) 的距离, 是到利用目标点重建的线或平面的最短距离。

量。点到线距离度量一个点云中的点与另一个点云中的线 (由点连接形成) 之间的距离。在具有线性特征的结构化环境中, 它可以取得更好的结果。利用更高层的几何特征还可以进一步改进: 可以度量一个点云中的点与另一个点云中的平面 (局部表面) 之间的距离。这种方法对噪声更鲁棒, 并能在具有平面表面的环境中达到更高精度。

在这些基本几何元素之上, 其他 ICP 变体引入了不同距离度量。采用多距离度量^[839]、连续时间表述^[256]和自适应阈值^[1133]的技术, 是近年来 ICP 的重要进展。另一些方法^[985, 591]不使用欧氏距离, 而是评估局部邻域概率分布之间的差异。

另一种知名的基于分布的匹配方法是正态分布变换 (normal distributions transform, NDT)^[82]。NDT 将输入点云划分为一组体素, 并为每个体素中的点拟合一个正态分布 (详见第 5.3.2.2 节)。它不需要承担确定最近邻关联的代价, 而是利用体素化执行分布到分布的匹配。这一过程可以利用更平滑、更鲁棒的配准代价曲面, 尤其适用于复杂环境。

8.3.2 确定对应关系

常见 ICP 算法的第二个核心设计决策, 是在源点云 P 和目标点云 Q 之间进行数据关联或对应关系搜索。

最基本的形式是: 对每个点 $p \in P$, 在目标点云 Q 中几何寻找其最近邻; 使用迭代更新的变换 (R^{k-1}, t^{k-1}) , 对应关系集合为:

$$\mathbb{C} = \left\{ (p, q) \mid p \in P, q = \arg \min_{q' \in Q} \left\| p - (R^{k-1} q' + t^{k-1}) \right\|_2 \right\}. \quad (8.3)$$

但在包含数千个点的大点云上计算时, 这通常是一项开销很大的操作。

为使 LiDAR 里程计能够实时运行, 通常使用两种策略减少对应关系搜索时间: (1) 减少对应关系的潜在候选集合; 或 (2) 使用不同于精确距离邻域搜索的搜索策略, 以更快地找到候选点。

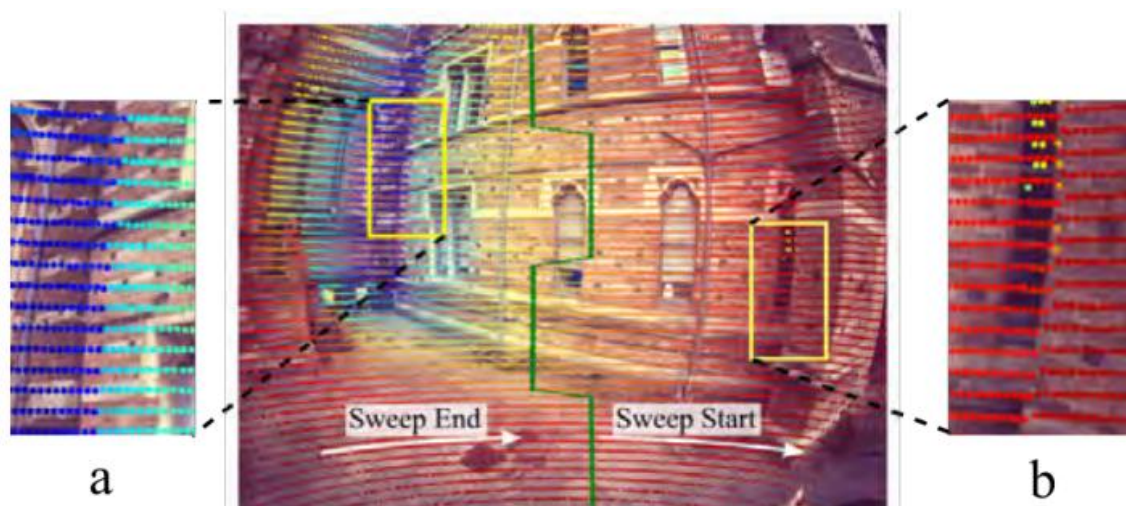
流行 LiDAR 里程计系统中的若干 ICP 变体^[1269, 839, 999]采用第一种策略: 通过构建候选集合较小的地图, 确定满足特定几何准则的点, 从而得到 $P' \subset P$ 和 $Q' \subset Q$ 。所用准则包括确定位于边缘或表面的点^[1269, 839]、移除对应地面的描述性较弱的点^[998], 以及对目标扫描进行下采样^[1133, 256]。这些策略确实加快了对应关系搜索, 但可能会从目标点云 Q 中移除真实对应关系。

相比之下, 第二种策略使用带有高效近似的搜索结构, 即使不总能得到精确最近邻, 也能加快对应关系搜索。常见做法是在范围图像中进行投影邻域搜索^[66], 或利用体素栅格进行近似邻域搜索^[1269, 256, 1133]。此外, 除了欧氏空间中的纯几何对应关系搜索, 还可以使用替代距离度量, 或将点投影到特征空间^[839]中识别对应关系。

到扫描内部运动畸变的影响。

8.4.2 基于特征的 LiDAR 里程计

运动畸变校正后，即可建立点对应关系。与视觉 SLAM（见第 7 章）类似，利用特征进行扫描关联已得到充分研究。基于特征的方法只处理从点云中提取的少量特征，因此能够高效表示扫描。对应关系匹配和残差计算也可在特征层面完成，从而显著降低总体计算代价。



(a) 不进行运动去畸变的 LiDAR-相机叠加



(b) 进行运动去畸变的 LiDAR-相机叠加图 8.5 叠加在相机图像上的 LiDAR 点云。扫描开始处（“b”）的轻微错位，到 LiDAR 扫描结束时（“a”）显著加重。底部显示运动补偿后扫描开始处（“d”）和结束处（“c”）的叠加结果更加一致。引自^[1072]。

低层特征。实践中最常用的特征是线和平面。在这一研究方向中，知名 LOAM 算法^[1269]是 LiDAR SLAM 的重要突破。LOAM 使用低层检测器高效识别可用于扫描配准的中层特征。通过识别高曲率或低曲率点检测边缘和平面。每个点的曲率由分析该点与邻域点之间的差异计算得到。高曲率点标记为边缘特征，低曲率点标记为平面特征。算法并不将所有点作为特征，而是选择最重要的一部分边缘点和平面点，在保持精度的同时降低计算复杂度。

主成分分析还可用于识别点的局部邻域主方向,以有效检测特征^[67, 753]。通过计算点及其邻域点协方差矩阵的特征值和相应特征向量,可以确定变化的主轴,从而辨别点的几何属性。具有一个主导特征值的点可分类为边缘点,表示尖锐过渡或边界;具有两个相近特征值的点可识别为来自平面区域,表示点云中的平坦表面。这样,通过考虑几何属性区分边缘和平面特征,提高 LiDAR 里程计精度。

高层特征。在 LiDAR 里程计中,语义、surfel 和强度等高层特征也可在提高系统精度和鲁棒性方面发挥重要作用。与低层特征相比,这些特征提供更丰富的环境信息,有利于更好的场景理解和更准确的建图。

语义特征。语义特征使用机器学习和深度学习技术,将 LiDAR 点云分类并标记为车辆、行人、建筑物和植被等物体类别,通常用于区分动态物体和静态物体。它可通过聚焦稳定路标提高里程计可靠性^[187]。

Surfel 特征。Surfel 是点云表面几何的小型盘状表示(见第 5.3.2.2 节)。可以为一组点拟合椭圆盘,其主半轴长度由邻近点协方差矩阵的特征值决定。扫描配准期间,这种表面表示可用于计算点到平面的距离^[846, 66, 903]。

强度特征。强度特征^[277, 415]指 LiDAR 回波的反射率或强度值。这些值提供环境中物体材料属性和表面特征的额外信息,为特征匹配提供额外维度,在无结构环境等具有挑战性的场景中尤其重要。

8.4.3 直接逐点 LiDAR 里程计

上文所述基于特征的方法存在一个问题:它往往舍弃孤立点的细微贡献,因为这些点无法明确对应到平面或边缘。在自然环境中对包含灌木或树枝的非结构化环境建图时,这一点尤其突出。此外,当从一种传感器换到另一种传感器时,还需要调节人工设计的特征检测器。后端要处理的点数通常随 LiDAR 扫描规模线性增长;使用现代 64 光束或 128 光束传感器时,该方法的计算代价可能变得无法承受。

另一种选择是直接使用点,而不提取中层特征。与视觉 SLAM 中的直接法类似,可以用类似 ICP 的方式直接对齐点。不过,由于逐点对应关系匹配的计算代价很高,这类直接法在 LiDAR SLAM 发展的早期并未受到青睐。

Zhou 等人^[1295]通过使用 GPU 加速的 KD 树实现加速最近邻搜索,为直接法的实用化铺平了道路。随后,Xu 等人的直接法 Fast-LIO2^[1211]使用一种称为增量 KD 树(incremental KD-tree, iKD-tree)的扩展,在地图规模增大时避免对应关系搜索瓶颈,展示了高度准确的帧率里程计。iKD-tree 能够适应点的分布,并通过偶尔重新平衡自身来高效执行添加、删除和查询操作,从而避免每加入一次扫描就重建整棵树。

8.4.4 局部建图与位姿估计

获得可靠对应关系后,下一步是对连续扫描进行一致配准。如前所述, LiDAR 里程计中的增量位姿估计通常通过 ICP 的某种变体进行扫描配准(见第 8.2.1 节)。

早期方法聚焦于扫描到扫描里程计,力求以完整频率(10 Hz)输出,同时将每次连续扫描配准视为独立操作。然而,单次 LiDAR 扫描可能相对稀疏;若参考扫描仅为前一次扫描,许多点会找不到合适对应关系,导致配准误差累积以及整体地图不一致。

更现代的方法是在传感器周围构建详细、准确的局部地图,这称为扫描到地图里程计。该范式已成功用于二维 LiDAR SLAM^[458]、三维 LiDAR 里程计^[1211, 256, 1133]和三维 LiDAR SLAM 系统^[839, 66, 846],并被证明能够降低总体漂移率。

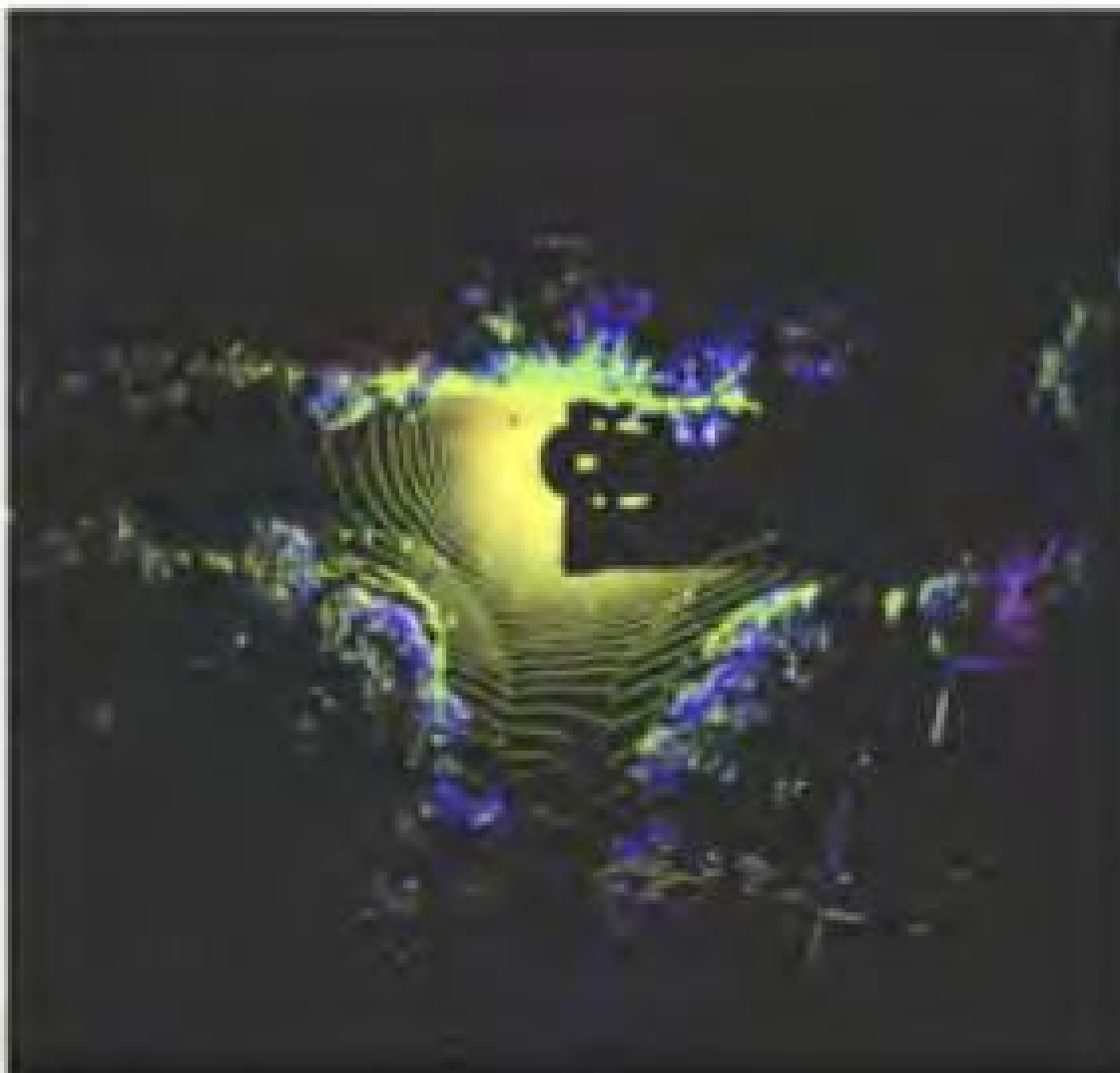
可以使用扫描到扫描里程计或 IMU 预积分得到的运动预测,对输入扫描预对齐,然后再与持久局部地图进行精细配准。局部地图比单次扫描稠密得多,因此能产生更合适的内点关联。配准后,将输入扫描加入局部地图。

受计算限制,早期 LiDAR 里程计方法^[1269]需要交替进行高频扫描到扫描里程计和低频扫描到局部地图里程计。较新的 LiDAR 里程计方法^[256, 1133]则利用体素化局部地图表示提供的直接逐点对应关系,采用单阶段扫描到地图对齐。

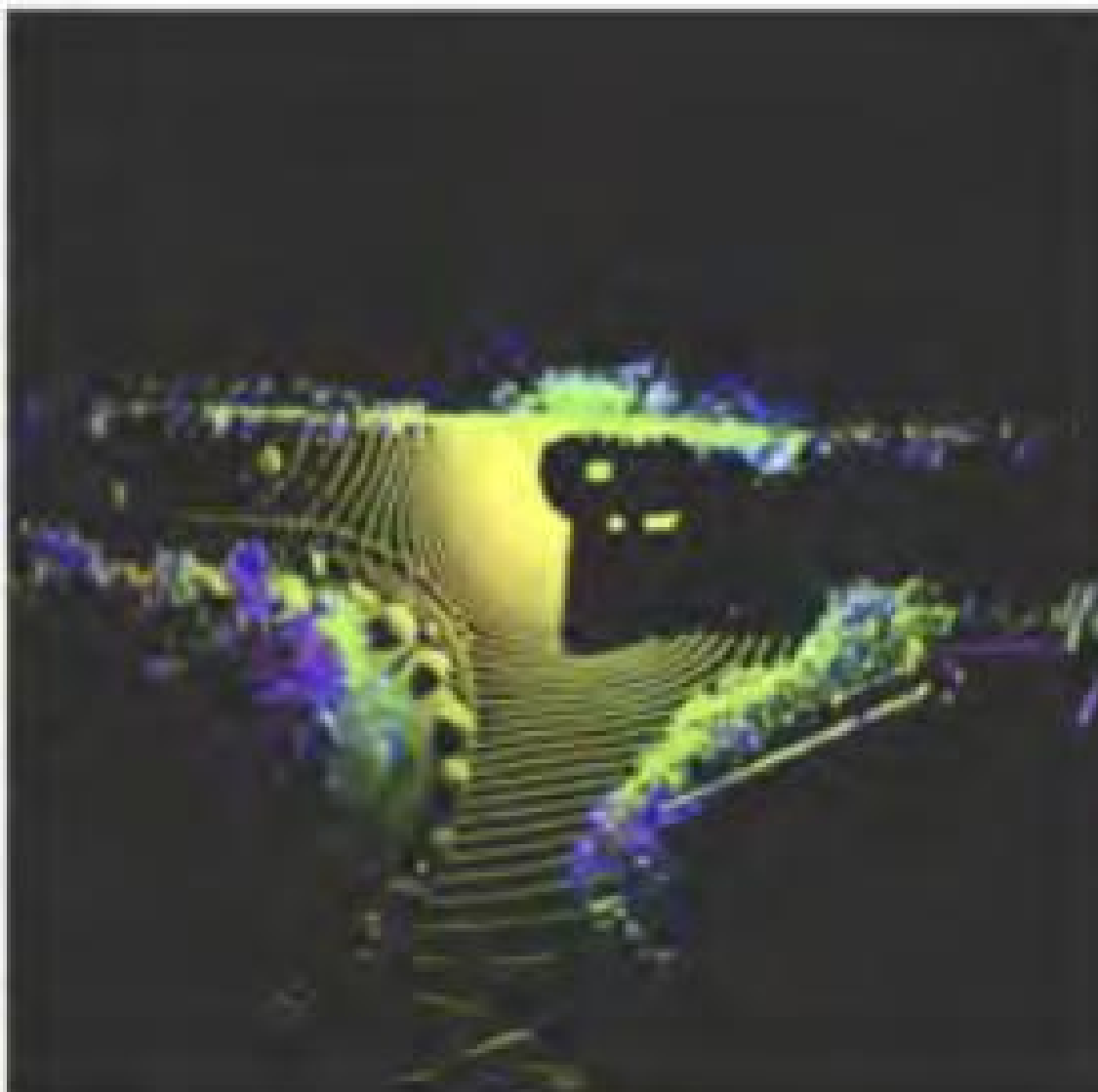
最后,使用深度学习是 LiDAR 里程计的一种完全不同方法。直接从 LiDAR 测量中学习自身运动的工作已取得一些有希望的成果。早期研究使用带真值标签的有监督学习^[655],之后扩展到无监督学习方法^[210]。尽管深度 LiDAR 方法的性能总体上很有前景,但其泛化能力仍受到质疑。

(a) 地点 A









(b) 地点 B 图 8.6 尽管地点 A 和地点 B 的 RGB 图像在视觉上差异很大（分别为航拍视图和机器人前视图），但其对应 LiDAR 扫描呈现结构相似的模式。这会造成结构感知混叠：由于共享道路拓扑和周围结构，不同地点在基于 LiDAR 的地点识别算法中显得相似。本例取自 STheReO 数据集的 SNU Afternoon 序列 [1262]。

8.4.5 LiDAR 里程计小结

总之，常见三维 LiDAR 里程计算法通过将输入扫描迭代对齐到连续更新的局部地图，可以产生高度准确、鲁棒的运动估计；过程中通常还借助 IMU 测量或运动模型校正扫描运动畸变。所得系统的漂移率可达到每行驶 1000 m 约 1 m，但性能高度依赖机器人周围环境、场景中的动态程度以及传感器自身的运动。地点识别是 LiDAR SLAM 的关键组件，而处理剩余的小量漂移是 LiDAR SLAM 的重要方面。

8.5 LiDAR 地点识别

地点识别系统旨在识别机器人或传感器先前访问过的地点。这是 LiDAR SLAM 的关键能力，也支持多会话 SLAM 和在先验地图中的全局定位等相关问题。不同于视觉数据，LiDAR 数据能够使机器人获得周围环境一致的度量三维信息，因此相较传统相机，受光照变化影响更小。然而，尽管具有这一相对于视觉定位的优势，LiDAR 感知的性质也给 LiDAR 地点识别带来了独特挑战。例如：

数据稀疏。视觉测量中的像素通常既稠密（例如百万像素相机）又有组织（即组织成规则二维网格），而传统 LiDAR 传感器捕获点的间距和局部密度会随传感器类型（例如光束数量）及感知范围变化（例如较远处点更稀疏）。点云分辨率通常远低于相机图像。由于这些限制，LiDAR 地点识别通常不依赖局部关键点描述子，即不为每个点单独计算特征描述子。为解决结构不足，方法会识别具有语义意义的点云片段 [285, 1274]，或计算全局描述 [568, 1212]（即扫描的单一代表描述子）。近年来，随着深度学习发展，学习确定鲁棒局部关键点描述子的研究也十分活跃 [156]。第 8.3.2 节将更详细回顾 LiDAR 地点识别领域的奠基论文和不同范式。

结构混叠。基于 LiDAR 的地点识别系统面临的第二个困难，是人造环境中的结构重复，例如长走廊或高速公路上难以区分的结构。考虑一栋规则的现代办公楼中每层的走廊：使用相机时，视觉地点识别或许能识别相同走廊墙面上独特或具有描述性的视觉纹理（如图片、海报或装饰），但仅凭在走廊中获得的一次扫描，很难区分具体楼层。室外环境也有类似挑战：不同地点可能具有视觉差异，但 LiDAR 扫描中的结构布局相似，从而产生感知混叠，如图 8.6 所示。ScanContext [568] 等全局 LiDAR 描述子在这类情况下通常失败。SegMap [285] 和 InstaLoc [1274] 等使用物体级聚类的方法，利用更高层语义特征，在这些情形中可能取得更好效果。

综上，开发 LiDAR 地点识别方法时，需要牢记这些特定挑战。

8.5.1 问题定义

本节关注地点识别任务，即回环候选检测问题。给定一个查询（即用点云表示的一次扫描），目标是从数据库中检索与查询相似的对条目。数据库（即先前访问过的地点）是在探索区域中按时空顺序获取的一组互不相交的地点描述子。

LiDAR 地点识别的一个关键考虑，是检索方法对传感器类型、采集时间和机器人位姿变化的鲁棒性。例如，如果使用了不同 LiDAR 设备，查询所用 LiDAR 类型可能不同于创建数据库所用的类型。此外，建图（即建立访问地点数据库）与稍后重新访问地点之间不可避免存在时间间隔。这一时间间隔可能导致环境结构变化，也可能因移动物体或人员产生动态差异。但最显著的变化来自机器人位姿变化：数据库和查询之间的平移和/或旋转偏移，会使同一环境所采集传感器数据的外观不同。因此，LiDAR 地点识别方法必须同时对位姿变化和环境变化具有鲁棒性。

如果一种方法无法确定位姿变化，但仍能正确识别候选地点，则称其具有**不变性**。如果它还能够估计位姿变化，则称其对重访位姿变化具有**感知能力**。研究者特别关注这种感知能力，主要有两个原因。第一，它可作为精细配准的良好初始猜测，这对建立估计精确回环所需的 $SE(2)$ 或 $SE(3)$ 约束至关重要（见第 8.4.1 节）。第二，追求更复杂的感知能力自然可以增强不变性，例如估计航向变化 [563] 或推断重叠程度 [188]。

8.5.2 LiDAR 地点识别方法

除实现不变性和感知能力外，还应注意：研究者提出了不同粒度的点云表示，以应对原始 LiDAR 测量的无结构性质，并确保大规模机器人自主运行中的实时地点检索性能。基于描述子的 LiDAR 地点识别方法通常可按使用局部描述子或全局描述子进行检索和匹配来分类。不过，也有一些变体不仅使用描述子距离进行地点识别，还直接学习地点相似度函数。下面详细讨论这些不同范式。

8.5.2.1 局部描述子

在 LiDAR 地点识别研究早期，随着视觉地点识别方法（例如 SIFT^[699]、ORB^[953]、DBoW2^[362]）的发展，计算局部关键点描述子自然成为二维^[1096]和三维 LiDAR 传感器^[101]的方法选择。然而，专门为稠密 RGB-D 点云配准或物体识别开发的三维局部描述子^[960, 961, 1103]，通常难以适应 LiDAR 感知的稀疏性和无结构性质，尤其是在室外场景中。为减轻这种敏感性，研究者提出使用局部关键点的统计分布（例如直方图）^[462]。但这些描述子仍局限于局部邻域；由于缺乏来自整次扫描的度量结构上下文，其描述能力较弱。

8.5.2.2 全局描述子

与局部方法相反，全局描述子利用整次扫描中的高层模式，而不是聚焦低层局部关键点。这些方法通过构建更简单、更粗粒度的表示来解决结构不足问题，因而通常产生计算效率更高的匹配方法。广泛用于生成全局描述子的两种粗粒度表示如下。

鸟瞰图 (bird's-eye view, BEV)。BEV 表示将三维点云转换为结构化、粗粒度的俯视图，可使用极坐标表示或稀疏栅格表示。Scan Context++^[563, 568]和 RING++^[703, 1212]是使用这类表示的方法示例。³（例如，RING++^[1212]使用在 $[-80, 80]$ m 栅格上均匀分布的 120×120 像素表示。）前者提出航向对齐匹配算法以实现方向不变性；后者利用 Radon 变换从理论上证明其不变性和感知能力。

范围图像。作为三维点云的替代，范围图像（见第 5.1 节）提供结构化且对齐良好的扫描表示。近年来稠密多光束 LiDAR 技术的发展（见第 8.1 节），使得将 LiDAR 数据表示为稠密范围图像更为合适。其优势在于可以直接借用计算机视觉领域成熟的工具，例如卷积神经网络 (CNN)^[608]或视觉 Transformer^[280]，来提取细致的特征表示。Overlap-Net^[188, 720]证明可以通过对范围图像应用重叠损失实现航向不变性；近期 FRAME^[1028]则展示了使用范围图像进行矿井 LiDAR 地点识别。

8.5.2.3 高层或组合描述子

为扫描（局部或全局）构造描述子时，一些方法采用混合策略，甚至直接学习地点识别的相似度函数。

基于分割的方法。如前所述，用单个描述子表示扫描容易受到结构混叠影响。因此，有研究尝试用一组有意义的物体或片段描述地点，以增加唯一性和描述能力并避免感知混叠。这些方法包括 SegMap^[285]和 InstaLoc^[1274]。

连接局部和全局的描述子。前面介绍的全局描述子通常只有在点云投影沿特定方向（即顶视图或球面视图）一致对齐时才有效。这一要求可能将方法的应用限制在沿道路以可预测方向行驶的车辆。为解决这一问题，研究者提出 BTC^[1258, 1259]，同时利用局部和全局描述子。该方法旨在保持局部几何和扫描整体结构，有效结合两类描述子的优势。

直接三维数据处理。近期数据驱动方法可以不依赖手工规则，直接利用原始无结构点进行检索。特别是，研究者提出基于深度学习的方法^[1119, 156]，提取能够适应不同传感器稀疏性和局部表面分布的鲁棒逐点特征。Cattaneo 等人^[156]展示了一种流水线：使用地点判别的三元组损失和可微相对位姿估计，可改善配准并整体受益于 LiDAR SLAM。也可以将此理解为：聚焦感知能力能够同时增强不变性和区分能力。

8.5.3 LiDAR 地点识别小结

总之，LiDAR 地点识别与视觉地点识别具有相同作用和共同属性，其性能也由相同指标定义。

传感器模态和相关技术通常相互补充：在许多 LiDAR 地点识别失败的位置，视觉地点识别能够成功，反之亦然。移动机器人导航系统通常同时使用两种传感器模态，以确保结果鲁棒可靠。

下一节将介绍如何将 LiDAR 地点识别与位姿图优化结合，校正 LiDAR 里程计中不可避免的漂移，从而形成一致、准确且可扩展的 LiDAR SLAM 系统。

8.6 LiDAR SLAM

第 8.2 节所述 LiDAR 里程计系统旨在估计传感器在环境中运动时的局部一致运动。然而，设备行驶较长距离后，这一运动估计必然会累积漂移。

为抵消漂移，LiDAR SLAM 系统可以通过识别传感器何时返回环境中先前访问过的部分，维护覆盖整个建图历史的全局一致估计。这类识别事件称为回环，它们可用于校正系统当前位姿估计，也可用于修正此前所有位姿估计组成的完整轨迹及相应地图表示。

LiDAR SLAM 系统所追求的一个关键性质是维护全局一致地图。这要求系统在单一地图表示中实现当前观测与过去观测之间的一致性。在大尺度且可能动态的环境中，这项任务尤其具有挑战性。

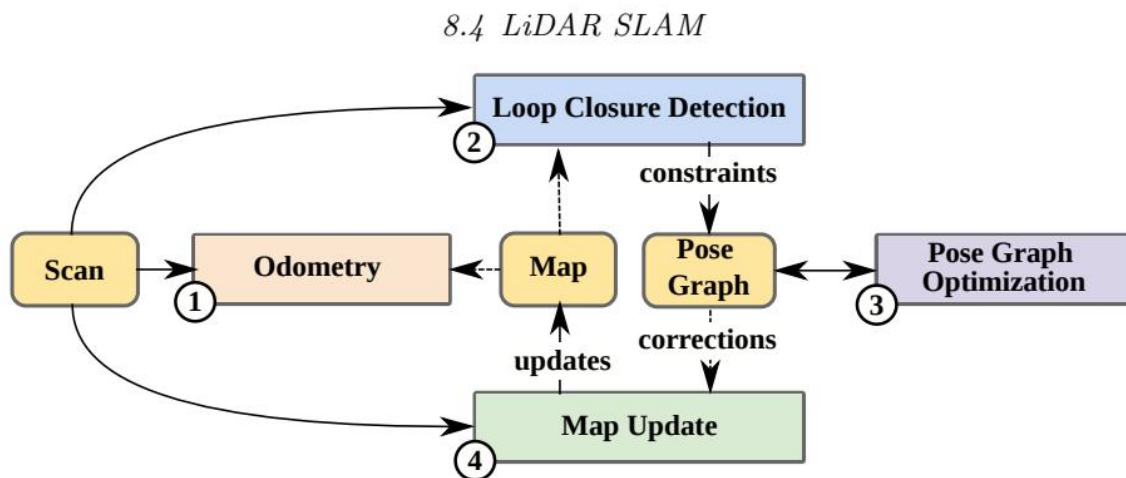


图 8.4: 图 8.7 典型 LiDAR SLAM 系统的概念结构，由多个组件组成：(1) 里程计估计机器人/传感器位姿；(2) 回环检测确定是否重新访问某地点；(3) 位姿图优化使用回环约束，通过因子图优化校正位姿轨迹；(4) 地图更新使用最新位姿轨迹修正地图表示。

第 8.2 节讨论了 LiDAR 里程计方法的发展，其漂移率可低至每行驶一公里一米；第 8.3 节回顾了通过 LiDAR 地点识别确定回环的不同方法。LiDAR SLAM 包含将这些组件结合起来、维护一致轨迹和地图表示所需的技术，并且要求它们能在机器人、车辆或感知系统上实时、机载运行。

目前大多数 LiDAR SLAM 系统^[66, 846, 282, 265, 1253]都由图 8.7 系统图所示组件构成。下一节将更详细讨论这一结构；第 8.4.2 节聚焦后端优化和地图更新的关键步骤，并介绍后端优化期间整合回环以校正机器人轨迹的常见技术。

LiDAR SLAM 的进阶主题包括多会话建图和多机器人建图。这些主题关注如何将多个建图会话融合到共同的全局参考系中：可以是并行的（多台机器人实时运行并实时求解），也可以是长期的（随时间对齐多个地图，以推断环境变化）。第 8.4.3 节将介绍多会话和多机器人 SLAM，并概述常见解决方案和挑战。

最后，随着 LiDAR SLAM 日趋成熟，其他进阶主题也得到关注。自动驾驶车辆等安全关键系统依赖 SLAM，因此必须考虑 LiDAR SLAM 系统的鲁棒性和可扩展性。本章最后的第 8.5 节将讨论这些主题。

8.6.1 LiDAR SLAM 系统的结构

实现 LiDAR SLAM 系统^[66, 846, 282, 265, 1253]时，通常将其分解为若干模块：一个模块通过里程计估计器维护相对位姿估计，其他模块识别和利用回环，重新估计位姿轨迹，再根据修正后的位姿轨迹更新关联地

图表示。LiDAR SLAM 系统通常包含一个以 LiDAR 传感器帧率（例如 10 Hz）运行的里程计估计模块，而其他模块以较低频率（例如 1 Hz）运行。

具体看图 8.7：(1) 里程计组件使用当前活动局部地图，以帧到地图的方式估计位姿。对里程计模块而言，最常见方法是将输入激光扫描配准到滚动/活动局部地图（如第 8.2 节所述）。里程计模块通常只能访问传感器直接邻域的数据，这些数据通常称为活动局部地图。

接着，(2) 基于 LiDAR 的地点识别方法确定潜在回环候选，如第 8.3 节所述。地点识别只能确定两个地点（更确切地说，是在这两个地点采集的观测）相似。要确定两地点之间精确的相对变换估计，还需要对相应 LiDAR 扫描进行精细配准（通常使用 ICP）。

为初始化配准，需要足够好的相对变换初始猜测。对小型位姿图，可以使用几何先验（取自已有位姿图）。在无法使用几何先验时，研究者已经提出不依赖初始猜测、但能够鲁棒估计相对变换的现代全局配准方法^[1223, 665]。当两次扫描重叠较少时，这些方法仍能良好工作。

可以使用回环候选之间的行驶距离或时间差，或使用基于 RANSAC 的几何验证对齐的置信度等启发式规则，确定回环候选的有效性，避免向位姿图加入错误回环。

模块 (3) 是后端位姿图优化（PGO）步骤。它使用完整 SLAM 约束集求解优化后的位姿图，并更新位姿轨迹。下一节将更详细讨论 PGO。

最后，(4) 地图更新机制根据校正后的轨迹，将传感器测量整合到组合地图表示中。对此有多种方法。最常见的方法是直接使用点^[256]；而 surfel^[282, 846, 66]和隐式表示^[1253, 265]等方法则尝试提高底层地图质量，或为地图提供更强的概率基础。关于不同稠密地图表示的技术细节和讨论，参见第 5 章。

下一节将详细讨论后端位姿图优化，以及通常如何更新完整地图表示。

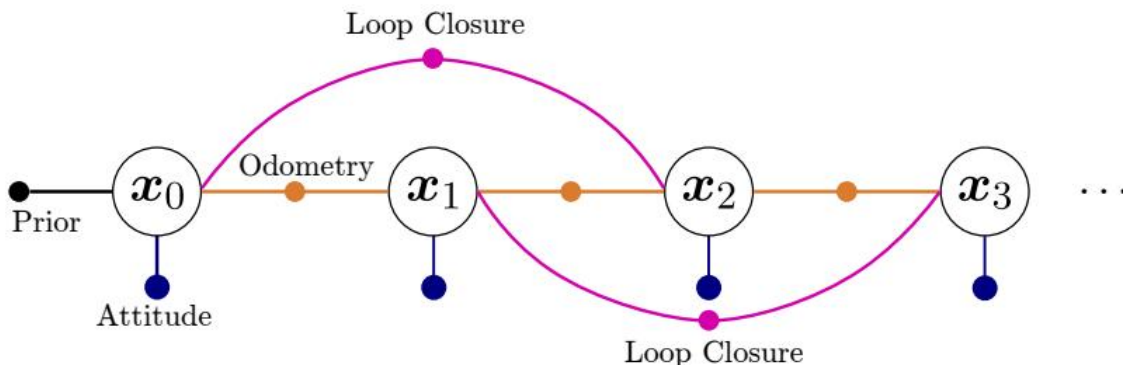


图 8.5: 图 8.8 用位姿图表示的 SLAM 问题。每个节点表示传感器位姿，边表示来自里程计（橙色）和回环（洋红色）的约束。先验因子固定图的原点；有惯性感知时，可选的姿态因子用于约束俯仰角和滚转角。引自^[887]。

8.6.2 位姿图优化与地图更新

LiDAR SLAM 系统的关键部分，是在提出并验证回环后更新位姿轨迹和地图表示。由于局部位姿估计包含漂移，必须在更新后的位姿轨迹中考虑局部位姿估计误差。此外，整合过去测量的已有地图表示也需要更新。

SLAM 后端通常使用位姿图优化（PGO）。PGO 校正估计轨迹，使其同时满足里程计约束以及在重新访问已观测地点时识别的回环约束。如第一部分（见第 1 章）所述，可以用因子图表示这些约束，如图 8.8 所示。由于图仅由相对位姿约束组成，通常称为位姿图。图 8.9 展示了回环检测和位姿图优化前后的点云地图示例。

可以使用 g2o^[614]和 GTSAM^[253]等通用求解器优化约束集。为在位姿图不断增大时实现实时性能，需要迭代求解持续增长的优化问题。不过，约束集通常是稀疏的，只有少量互联边。对底层方程系统进行重排

序和重新线性化的稀疏矩阵分解方法，可以在不到一秒的时间内更新包含 1000 个以上节点的图。更多内容参见第 1 章，以及 iSAM2^[534] 和 HOG-Man^[401]。

需要注意，虽然 1000 个节点对应大型位姿图，但仍必须考虑可扩展性。通常只相对稀疏地添加里程计约束，而不是以传感器频率（例如 10 Hz）添加；可以改为每行驶数米添加一次。另一种方法是将已建图环境划分为固定物理尺寸的子地图（例如每行驶 30 m 一个子地图），每个子地图对应一个位姿图节点。假定子地图内部里程计局部准确，无需重新调整子地图内的轨迹。这种方法使位姿图 SLAM 能够扩展到城市规模地图。



图 8.6: 图 8.9 通过纳入回环约束，SLAM 系统可以创建重新访问地点的全局一致地图。左侧显示重新访问地点的纯里程计轨迹，在该路口原始访问（蓝色）与当前访问（紫色）之间存在明显错位。右侧显示整合回环后的位姿图优化结果，得到一致的道路路口地图。

在后端中，位姿图优化必须同时考虑里程计的位姿估计误差和回环约束误差。为正确建模里程计位姿的位姿估计不确定性，还可以估计数据驱动的协方差，以合理分配位姿图优化中的误差^[622, 159]，例如对漂移率可能较高的边约束使用较大的协方差。最后，为处理错误回环约束和位姿图的不良配置，研究者已开展大量鲁棒位姿图优化研究^[12, 153, 1059, 1060, 1221]，可以降低权重、禁用或忽略否则会导致地图退化或发散的位姿图约束（详见第 3 章）。

位姿图优化后，完整机器人轨迹已实现全局一致，但这一更新的影响也需要反映到地图本身。常见方法是利用过去观测简单地重建地图。这要求系统无限期存储过去观测，在大尺度环境中很快变得不适用。替代方法是使地图表示发生形变^[846]，或将地图元素（例如 surfel 或子地图）直接连接到位姿，以更可扩展地实现地图变形。

8.6.3 多机器人与多会话 LiDAR SLAM

随着单机器人、单会话 LiDAR SLAM 系统日趋成熟，研究者开始关注将其扩展到多机器人和多会话应用。这很有用：它可以将不完整地图扩展到新扫描区域，或使多台现场机器人利用共同地图表示协调活动。另一个用途是对同一区域在不同时间采集的地图进行共同配准，以推断长期环境变化，例如安全或监测应用中的变化。

首先需要指出，现代 LiDAR SLAM 系统越来越准确，在开阔空间中典型漂移为每千米一米，但 SLAM 地图中总会残留一些小误差。直接将两次独立 SLAM 任务的最终点云地图共同配准，会在某些位置产生不一致的点云对齐，如图 8.10 所示。

该领域的早期工作聚焦于如何对多个建图会话进行联合后端优化。一种方法是将多个 SLAM 实例的约束直接从各自约束集合转移到单一全局地图表示中。另一种方法是分别构建每个地图，每张地图有自己的坐标系、节点和边。为将它们彼此连接，Kim 等人^[562]引入称为锚节点的辅助变量，以处理不同 SLAM 任务

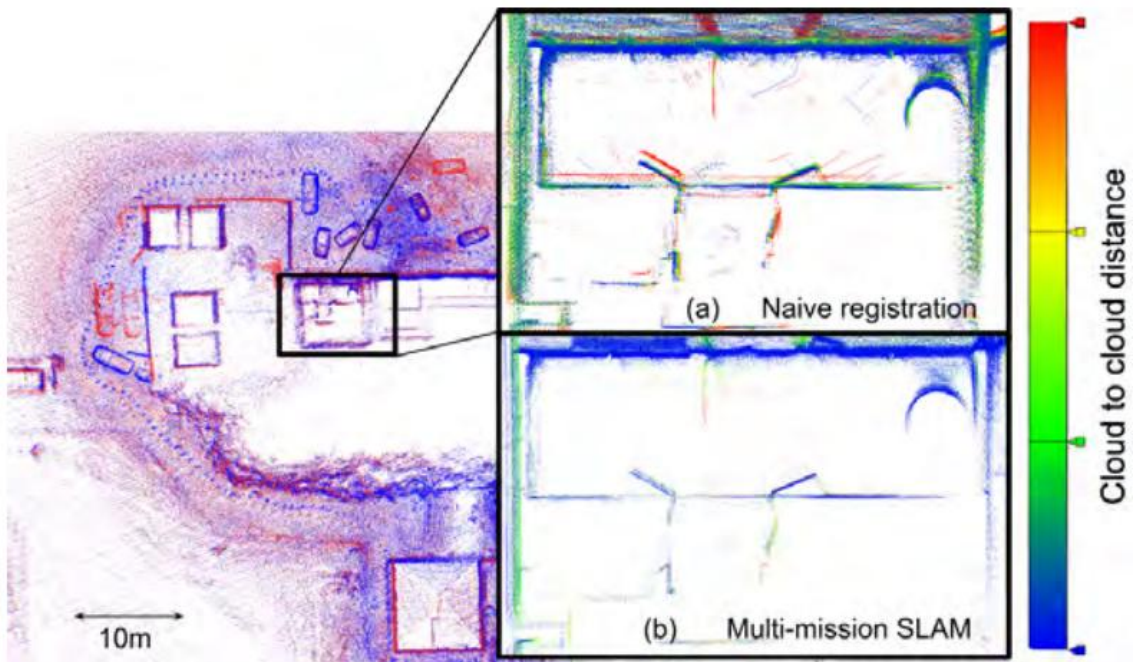


图 8.7: 图 8.10 (a) 两个全局点云的朴素直接对齐与 (b) 多会话位姿图松弛的比较。(a) 两个全局点云之间的点到点距离产生“双墙”现象，虚构出环境变化。(b) 多会话松弛降低点到点距离，使结构得到更清晰重建。引自^[951]。(©2024 IEEE)

的地图坐标差异。该节点能够轻松实现每张地图的全局对齐，已用于多会话视觉 SLAM 和 LiDAR SLAM^[749, 565]。

如^[284]所述，除后端优化外，还必须确定如何在完全断开的 SLAM 任务之间建立回环约束。不同于单会话 SLAM，最初没有几何先验可用于形成首个会话间约束；多会话 SLAM 完全依赖地点识别将地图相互关联。

8.6.3.1 多机器人 SLAM

实时多机器人 SLAM 更进一步：它解决多会话建图问题，但使用的是现场运行机器人实时采集的数据。从多个平台实时估计组合地图，可以使机器人团队协调任务规划、最佳选择探索前沿，并避免重复返回另一台机器人已建图的位置。该能力使机器人团队能够协同工作，高效探索区域、识别无障碍路线，并可能识别感兴趣的人员或物体。这对搜救和军事应用都很重要。

可以区分集中式系统和分散式系统。集中式系统可将传感器测量传输到基站，并在该处计算组合地图。这可以让现场机器人的计算和感知尽可能简单，例如 Amazon 和 Ocado 仓库作业中使用的移动机器人。另一方面，分散式（或分布式）系统在每台机器人上独立构建 SLAM 地图，再由基站合并机器人地图集合。

为描述最新技术的发展，我们参考两个重要的国际多机器人探索挑战赛。第一个是 2010 年在澳大利亚布里斯班举行的 Multi Autonomous Ground-robotic International Challenge (MAGIC)。该挑战要求轮式机器人队伍在 $500\text{ m} \times 500\text{ m}$ 的区域执行侦察任务，正确定位并分类模拟威胁。获胜的密歇根队部署了 14 台三维打印机器人，每台配备二维 LiDAR 扫描器^[828]和用于识别回环的相机。每台机器人执行二维 LiDAR 里程计，并将位姿图约束传输到基站，由基站组装全局二维多机器人地图。

过去十年的研究进展由 2021 年在肯塔基州路易斯维尔举行的 DARPA Subterranean Challenge^[600] 体现。与 MAGIC 类似，该挑战要求参赛队伍探索未知环境，但机器人需要在更复杂的三维地下环境中运行，其中包含楼梯、路缘和坡道。这里广泛使用三维多光束 LiDAR，同时辅以视觉里程计、轮式/足式里程计，有时还使用热成像里程计，以克服 LiDAR 里程计可能失败的退化情形，例如地下隧道中的狭窄环境。

SubT 队伍发表的综述文章比较了参赛系统^[299]。每个队伍都采用半分散式方法：每台机器人在机载设

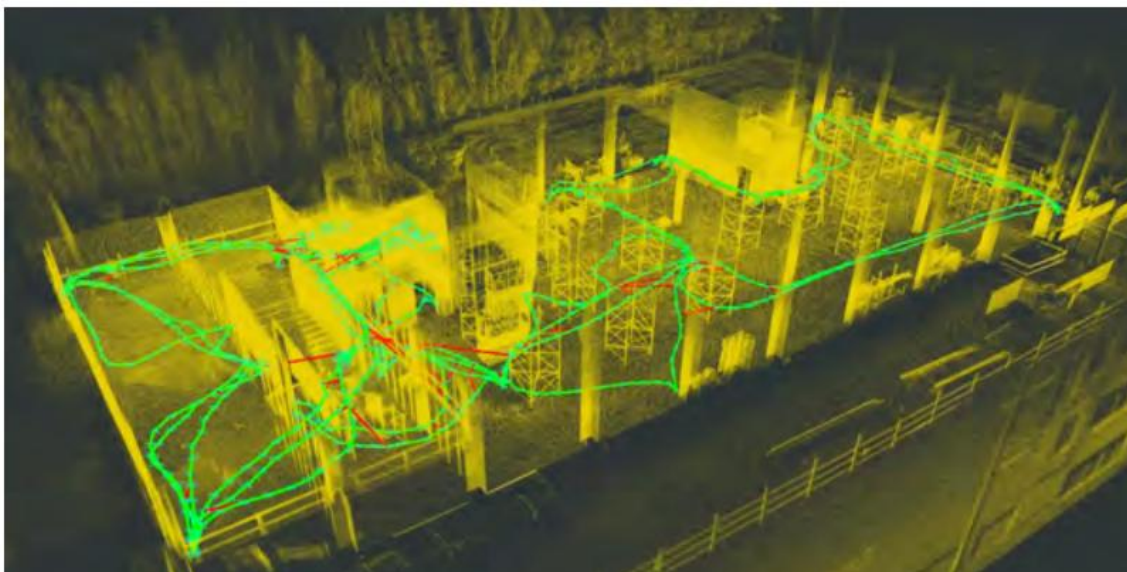


图 8.8: 图 8.11 建设工地的多会话 SLAM 地图。通过建立会话间回环约束（红色）并联合优化五个建图会话的轨迹（绿色），将五个不同建图会话合并。



图 8.9: 图 8.12 多机器人 SLAM 从 2010 年 MAGIC 挑战赛的二维系统发展到 2021 年 DARPA SubT 挑战赛的完整三维系统。图片展示两次挑战赛的获胜队伍：密歇根大学队和 Cerberus 队。图片由 Edwin Olson 和 Cerberus 队提供。

备上构建基于位姿图的 SLAM 地图，再由单一中央基站创建多机器人 SLAM 地图。关键挑战包括压缩和通信，因为机器人队伍需要维护动态无线网络，将数据传回基站。例如，CSIRO 的 WildCat SLAM 系统^[601]采用压缩 surfel 表示来表示局部子地图。这些 surfel 地图比原始点云占用空间小得多，大幅降低了向基站计算机传输地图和位姿图约束所需的带宽。在决赛期间，每台机器人完整地图的大小仅为 21.5 MB。

如前所述，最复杂的问题是完全分布式 SLAM 系统：每台机器人平台都需要在通信和可扩展性约束下，构建和维护整体组合地图表示。一些现有方法^[493, 1095]已探索如何实现这一目标，重点是逐步在每台机器人之间共享约束集合和局部子地图。该主题的核心问题是保持一致性。

8.7 延伸阅读与近期趋势

过去几十年，尤其是 LOAM^[1269]提出以来，LiDAR SLAM 取得了重大进展。研究者还开发了高精度里程计^[1211, 999, 1133, 256]和高效位姿图 SLAM 系统^[66, 265, 846, 903]。然而，尽管取得这些进展，仍有许多未解决问题和挑战。

鲁棒且韧性的感知。Zhao 等人近期的鲁棒性评估^[1288]发现，LiDAR SLAM 系统在杂乱、无结构环境中难以有效运行。无结构走廊、地下矿井以及雪、雾、尘等极端天气条件，也是 DARPA Subterranean Challenge 参赛者综述^[299]指出的挑战场景。

此外，当前 LiDAR SLAM 系统的性能通常通过实验展示，缺乏正式鲁棒性评估。最佳性能依赖特征工程和手工参数调节；这些参数或许应像 KISS-ICP^[1133]那样，根据运行场景动态调整。未来改进应考虑主动调整算法行为，通过内省适应环境变化。

关于地点识别，研究方向十分广泛。^[1007, 1249]等重要综述论文为该主题提供了全面基础。当前研究包括能够跨不同 LiDAR 传感器类别泛化的鲁棒检索方法^[529]。另一些研究致力于实现 LiDAR 与雷达^[1248]、OpenStreetMap^[211]等其他模态之间的异构地点识别。研究者还成功利用 LiDAR 强度信息^[1000, 1151]，结合传统 XYZ 数据改善性能。跨多个建图会话的长期地点识别^[566]，以及变化检测和终身地图管理^[565, 1251]，也是有前景的研究主题。

多传感器融合。融合具有互补特性的多种传感器，是构建更鲁棒、更具韧性机器人系统的关键途径。通过融合不同的互补传感器，可以缓解某一传感器感知退化，例如雷达在雨天或烟雾中表现良好，或在 LiDAR 失效的隧道中使用视觉特征跟踪^[1191, 1287]。不过，将更多传感器接入 LiDAR 后，必然会获得大量传感器数据，导致冗余。如何在冗余和轻量计算之间取得平衡，仍是开放问题；融合多个传感器估计时，还必须高效选择最可靠的信息。解决方案包括早期融合（使用单一紧耦合估计器）和后期融合（组合各个独立传感器的位姿估计）。

最后，多传感器系统可能缺乏准确标定，且各传感器不一定精确同步。这会给底层估计系统带来负担，使完整紧耦合传感器融合难以在长时间间隔内实际使用。这些有趣的研究问题仍有很大探索空间。

不确定性与贝叶斯估计。与传感器如何可靠融合密切相关的另一个问题，是 LiDAR SLAM 中的不确定性估计。若要以概率方式融合多个位姿估计，需要经过适当标定的传感器不确定性度量。然而，大多数成功的基于 LiDAR 的方法依赖 ICP，而 ICP 并不能提供经过标定且鲁棒的位姿不确定性估计。

尽管已有一些用于近似协方差的早期方法^[622, 159]，LiDAR SLAM 系统中使用的估计不确定性通常并不可靠。因此，后端位姿图优化常使用固定的里程计协方差。对里程计过程中的不确定性进行更具内省性的处理，有望在更早阶段处理退化的位姿估计。关于不确定性估计仍有许多开放问题；鲁棒解决方案将支持更具韧性的 SLAM 系统和多模态 SLAM 的发展。

闭环自主系统中的部署。最后需要考虑 LiDAR SLAM 在目标最终应用中的表现。这些应用十分多样，包括穿越森林的轻型空中飞行器、扫描建筑工地的手持设备，以及在恶劣天气中运行的自动驾驶汽车。传统的尺寸、重量和功耗（Size, Weight, and Power, SWaP）电子系统要求，还要与准确性、鲁棒性、计算量和延迟等参数共同考虑。例如，自动驾驶汽车尽可能低延迟地响应即将发生的碰撞的能力，会受到 LiDAR 系统计算延迟的影响。因此，在某些应用中，最准确且计算最复杂的系统不一定总是首选方案。

第九章 雷达 SLAM

作者: Martin Magnusson、Christoffer Heckman、Henrik Andreasson、Ayoung Kim、Timothy Barfoot、Michael Kaess 和 Paul Newman

本章探讨雷达 (RAdio Detection and Ranging) 在 SLAM 中的应用。与相机和 LiDAR 相比, 雷达在 SLAM 中的应用相对不足; 但由于它能够在低能见度、远距离条件下工作, 并且能够原生产生速度信息, 其受欢迎程度正在上升。我们首先讨论机器人中通常使用的雷达传感器类型、其独特的感知原理, 以及雷达应用面临的一些挑战 (第 9.1 节)。随后讨论雷达滤波、雷达里程计 (第 9.2 节)、地点识别 (第 9.3 节) 以及 SLAM (第 9.4 节); 图 9.1 展示这些部分如何衔接。最后讨论雷达 SLAM 数据集 (第 9.5 节), 并展望雷达的未来应用 (第 9.6 节)。

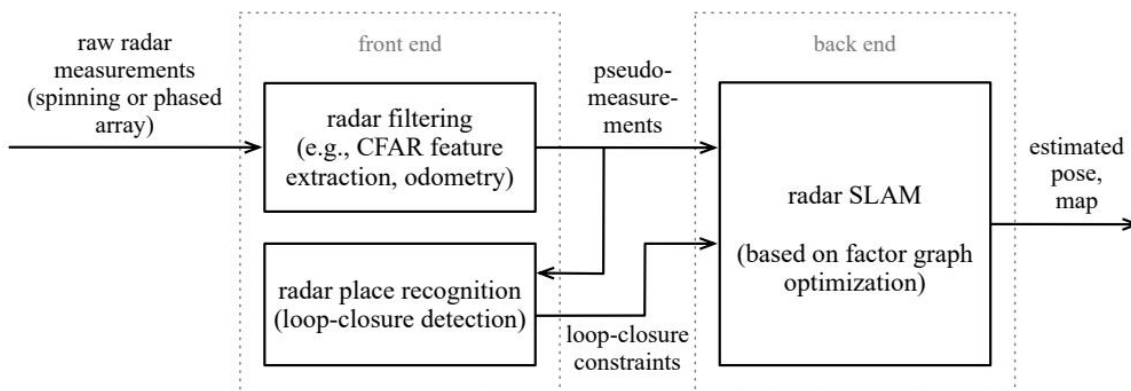


图 9.1: 图 9.1 基于雷达的 SLAM 信息流与其他 SLAM 系统遵循相同模式, 但各过程的细节会针对雷达特性进行调整。

9.1 雷达简介

9.1.1 传感器类型

下面介绍机器人中最常见的两类雷达: 旋转雷达和片上系统 (system-on-a-chip, SoC) 雷达 (见图 9.2)。二者各有优缺点, 主要区别在于如何进行“驱动”: 旋转雷达机械旋转单个天线, 而 SoC 雷达使用多个天线, 通过组合各天线信号推断反射发射信号的物体的角度和距离。

9.1.1.1 旋转雷达

旋转雷达有时也称扫描雷达或成像雷达。它利用旋转雷达传感器构建周围环境的精确极坐标表示, 如图 9.2 所示。其主要数据产品是极坐标雷达图。成像雷达的特点是能够检测距离超过 100 m 的物体。在某些模式下, 还可以利用多普勒效应确定这些物体的速度。

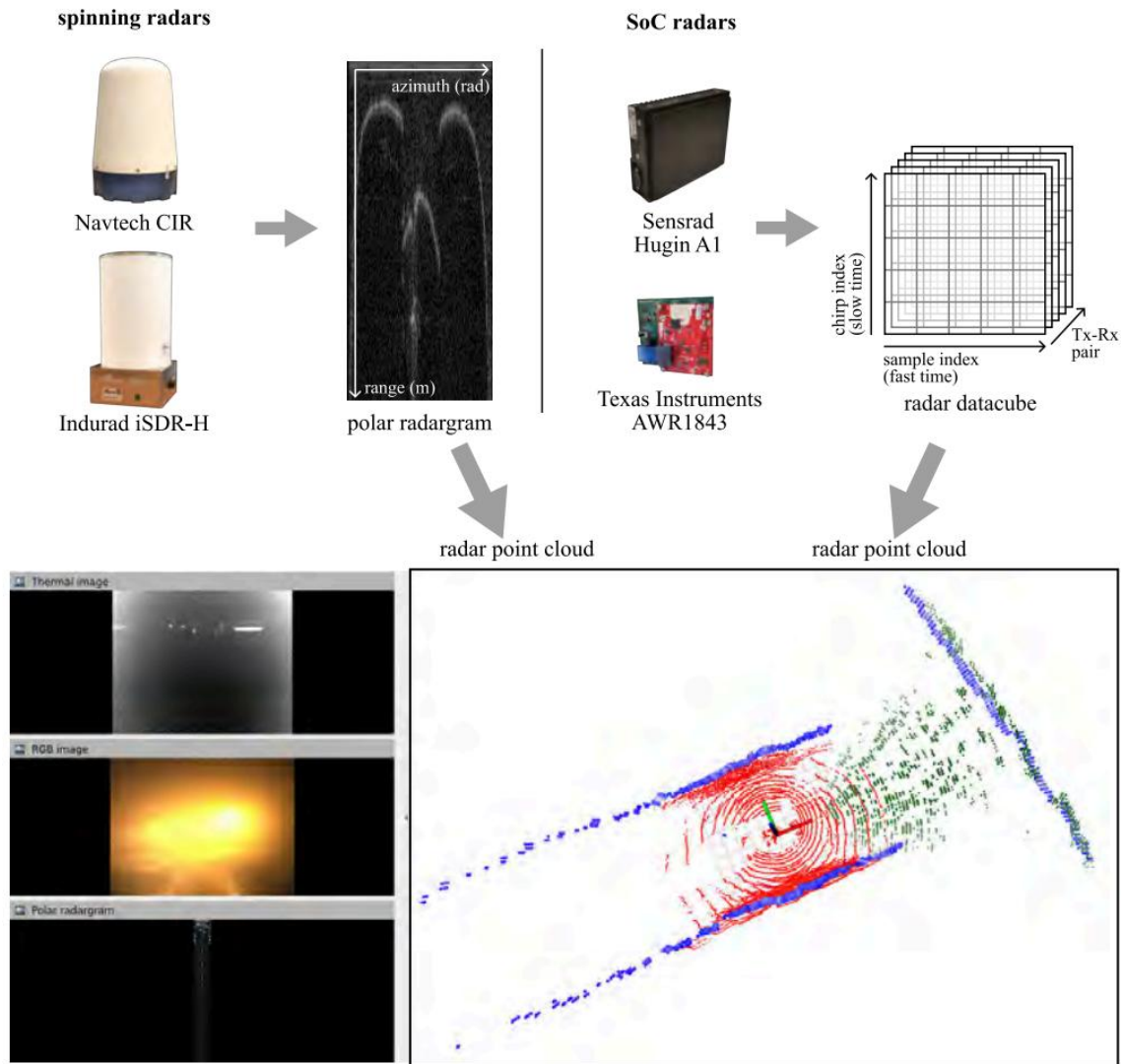


图 9.2: 图 9.2 机器人中使用的两类主要雷达：旋转雷达（左上）和相控阵 SoC 雷达（右上）。图中给出每类传感器的示例及其产生的主要数据产品。原始数据通常随后通过雷达滤波转换为稀疏点云（下方）。右下为带烟雾隧道中的三个点云：红色为受烟雾影响、测程显著受限的 LiDAR 点云；蓝色为二维旋转雷达点云，清晰显示各个方向的隧道壁，且可见较远距离；绿色为 SoC 雷达三维点云，可见车辆前方的墙壁和地面。左下为同一场景的热成像与视觉/RGB 图像数据，以及从中提取蓝色点云的极坐标雷达图。

不同于 LiDAR 系统，旋转雷达只能提供二维平面数据，无法测量所检测物体的高度。即使反射可能来自天线垂直波束宽度内的不同高度，这一限制仍然存在。此外，由于其工作机制是在每个天线角度发射和接收单个脉冲，这些雷达有时无法提供速度数据，因为速度计算需要多个脉冲。较新的工作利用多个相邻方位角的信号恢复速度。

与目前普遍使用的多垂直光束三维 LiDAR 传感器类似，构造机械旋转三维雷达在实践中不可行，因为与激光二极管相比，天线及其聚焦机构尺寸大得多。

9.1.1.2 SoC 雷达

片上系统 (SoC) 雷达将处理单元集成到数量极少的芯片中；这些芯片直接安装在贴片天线上（发射器/接收器阵列），或集成到印刷电路板中。由于集成架构和没有运动部件，SoC 雷达相较旋转雷达更轻、更省电。SoC 雷达的精度和分辨率取决于天线阵列设计，以及制造商用于整合多个天线测量的专有处理技术。其主要输出是雷达数据立方体 (radar datacube)。

SoC 回波通常按方位角、俯仰角和距离映射到球面坐标。由于径向速度增加了一个维度，这类雷达通常称为 3+1D 或 4D。相反，由于垂直排列天线较少而垂直分辨率有限或缺失的系统，称为二维阵列雷达，产生 2+1D 数据产品。

9.1.2 雷达感知原理

本节讨论与机器人应用相关的基本操作、天线和数据类型，以及应用挑战。下面各小节将分别概述它们，并说明雷达与其他测距传感器的差异。

毫米波 (millimeter-wave, mmWave) 雷达指电磁波长在 1–10 mm、频率范围为 30–300 GHz 的雷达系统。在这一范围内，多数雷达传感器工作于 76–81 GHz 频段，因为高级驾驶辅助系统 (ADAS) 等汽车应用已在该频段获得专用频谱。

9.1.2.1 雷达散射截面

毫米波雷达通过发射电磁脉冲实现感知；脉冲传播直到遇到物体，再反射回雷达。雷达散射截面 (radar cross section, RCS) 受物体材料组成、大小和形状影响，在确定每个物体对电磁脉冲的反射强度方面起着关键作用。从本质上说，RCS 表示一个假想球体的大小，该球体反射的能量与目标物体相同。因此，相较于小物体或行人 (RCS 较低)，车辆和厚混凝土墙等更大、更坚固的结构具有更高 RCS。

雷达强度指物体回波的强度，是雷达发射功率和物体 RCS 的函数。也就是说，该强度主要由雷达发射功率和所遇目标的 RCS 决定。文献表明，这种强度度量对于提取物体语义细节或辅助导航至关重要，因为较强回波往往对应环境中独特且易于识别的特征^[1240]。雷达信号强度还受到多个因素影响，包括所用天线类型、发射电磁脉冲的特性以及天线检测物体回波的能力。

9.1.2.2 FMCW 测距

雷达至少包含一个发射天线 (TX) 和一个接收天线 (RX)。¹ 在 FMCW 雷达中，TX 天线发射频率稳步增加的射频脉冲，称为 chirp。该过程通常称为锯齿调制；后文还将讨论另一种基于 chirp 的技术，即三角调制。chirp 在环境物体上反射，RX 天线捕获回波（见图 9.3）。接收到 chirp 后，信号先被放大，再与原始 TX chirp 混合或相减，生成中频 (intermediate frequency, IF) 信号。由于发射和接收信号的斜率相同，混合过程产生频率恒定为 f_0 的正弦 IF 信号。雷达系统的性能，包括测距和速度检测能力，会受到多种 chirp 参数影响，例如带宽（初始频率与终止频率之差）、chirp 斜率以及 chirp 之间的持续时间。

这种频率调制的核心思想，是将时间信息编码到连续波上，从而计算距离。与之相反，调幅方法中，返回信号的精度完全取决于信号带宽。频率调制对信噪比和目标 RCS 等问题更具韧性，而这些因素可能影响单次回波的清晰度。在 FMCW 中，脉冲序列中的波以独特时间间隔交错，信号保持相干。这种相干性使每个 chirp 能够准确关联到其发射-接收天线对及其在序列中的位置，从而按幅度和相位对信号进行相关。这相较更早的不相干脉冲式飞行时间雷达系统有显著进步，后者依赖不相干幅度测量，需要熟练操作从杂波中筛选并分离重要信号。

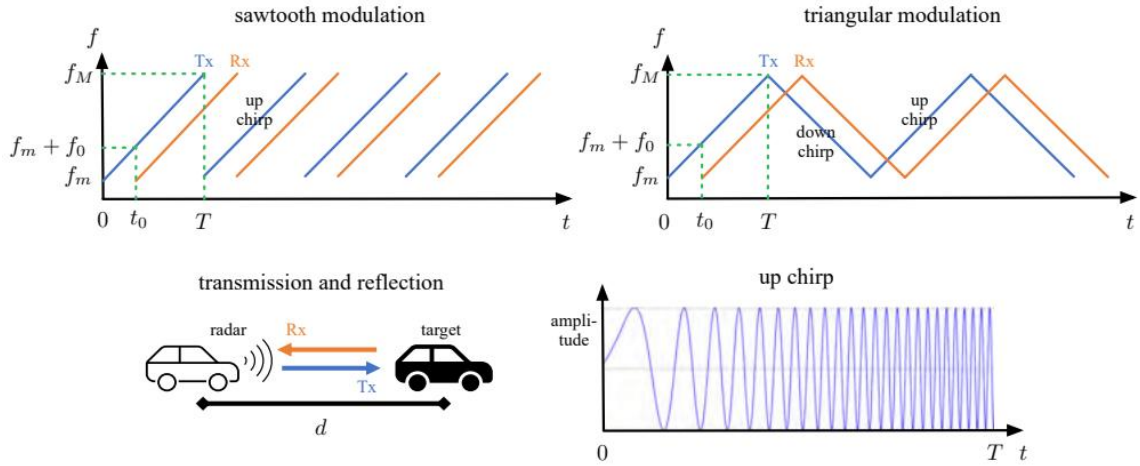


图 9.3: 图 9.3 调频连续波 (FMCW) 雷达通过发射频率递增或递减的 chirp (波), 这些波从目标反射后被接收。通过分析反射信号相对于发射信号的频率和相位偏移, 可以确定距离 (及速度)。两种常用频率调制策略是锯齿调制和三角调制; 后者的优势是能够将多普勒频移与距离频移分离。

在 FMCW 中, 距离计算依赖信号发射与接收回波之间的时间间隔。利用光速 c 和初始到达时间 t_0 , 可按下式计算物体距离 d :

$$d = \frac{c}{2} t_0. \quad (9.1)$$

图 9.3 左上部分的绿色虚线显示了生成 IF 信号的区域。发射信号与接收信号之差就是 IF 信号。IF 信号是正弦波 $\sin(2\pi f_0 t + \phi_0)$, 其频率与常数 f_0 成正比; 该常数从 t_0 延续到 T , 只取决于目标距离, 并有相位偏移 ϕ_0 。

频率 f_0 由目标距离、发射 chirp 的持续时间 T 以及发射信号带宽 $B = f_M - f_m$ 决定。带宽和发射时间通过 chirp 斜率 $S = B/T$ 关联:

$$f_0 = \frac{2Bd}{Tc} = \frac{2Sd}{c} \Rightarrow d = \frac{cf_0}{2S}, \quad (9.2)$$

此时距离 d 表示为所测中频 f_0 的函数。

虽然本节在频域中给出公式, 但现实世界中的信号采集主要通过数字采样完成。采样由高频模数转换器 (analog-to-digital converter, ADC) 以很高频率进行, 通常每 100 ns 或更短时间采样一次。随后, 采样数据依次进行快速傅里叶变换 (fast Fourier transform, FFT), 生成频率-幅度图, 从中可以辨识信号峰值。距离频率通过对单个 chirp 及其对应回波信号数据进行 FFT 得到; 速度频率则通过对多个 chirp 和回波序列执行一系列 FFT 计算。

9.1.2.3 用锯齿调制确定距离和速度

锯齿频率调制 (见图 9.3) 通常用于 SoC 雷达和一些旋转雷达。IF 信号的相位 ϕ_0 可表示为信号波长 λ 和距离 d 的函数:

$$\phi_0 = 2\pi f_m t_0 = \frac{4\pi d}{\lambda}. \quad (9.3)$$

尽管基频 f_0 和相位 ϕ_0 都是距离 d 的函数, 但相位仅在足够小的距离值下有效, 且存在相位缠绕。因此, 它通常不用于测距, 而用于测量小的距离变化 Δd ; 在速度估计中, 相位会对这类变化产生线性响应。

为估计径向速度 (见图 9.4), 如图 9.3 左上所示, 至少使用两个连续上行 chirp。序列中每个 chirp 的时间间隔很小, 量级约为 40 微秒, 因此两次采样的距离测量以及相应 IF f_i 和 f_{i+1} 几乎相同。然而, IF 信号

的相位不同。这一相位差 $\Delta\phi$ 对应物体运动；估计速度 v 由相位差确定：

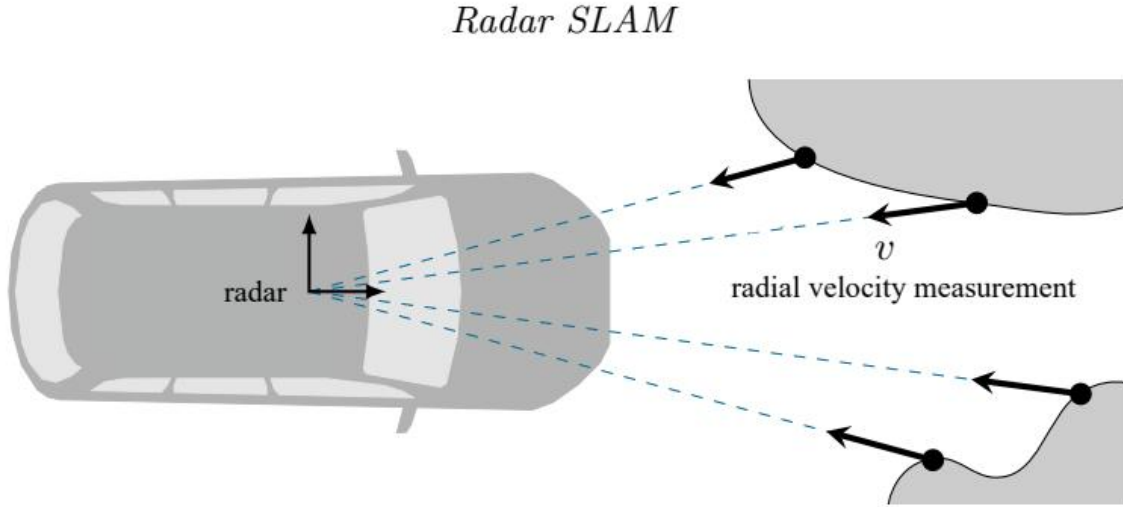


图 9.4: 图 9.4 雷达可利用多普勒效应测量设备与其成像场景之间的径向速度。

$$\Delta\phi = \frac{4\pi\Delta d}{\lambda} = \frac{4\pi vT}{\lambda}, \quad (9.4)$$

化简为

$$v = \frac{\lambda\Delta\phi}{4\pi T}. \quad (9.5)$$

由于速度是相位的函数，在 $|\Delta\phi| < \pi$ 时可获得无歧义的最大可检测速度。因此， $v_{\max} = \lambda/(4T)$ 是信号波长和 chirp 间时间间隔的函数。

9.1.2.4 用三角调制确定距离和速度

也可以使用三角频率调制（见图 9.3）；例如，它有时用于旋转雷达。实际上，从上行 chirp 得到中频时，它包含两个分量：(i) 由信号飞行时间造成的频移（前文已讨论） $f_{0,t}$ ；(ii) 若雷达和目标存在相对速度，则由多普勒效应造成的表观频移 $f_{0,d}$ ：

$$f_{0,\text{up}} = f_{0,t} + f_{0,d}. \quad (9.6)$$

但是，若在上行 chirp 后紧接着使用一个从相同目标反射的下行 chirp，时间频移的符号会翻转，而多普勒频移的符号不会：

$$f_{0,\text{down}} = -f_{0,t} + f_{0,d}. \quad (9.7)$$

由这两个方程可求出 $f_{0,t}$ 和 $f_{0,d}$ ：

$$f_{0,t} = \frac{f_{0,\text{up}} - f_{0,\text{down}}}{2}, \quad f_{0,d} = \frac{f_{0,\text{up}} + f_{0,\text{down}}}{2}. \quad (9.8)$$

最后，根据这两个分量可计算距离和速度：

$$d = \frac{cf_{0,t}}{2S}, \quad v = \frac{cf_{0,d}}{2ST}. \quad (9.9)$$

值得注意的是，先前式 (9.2) 中的距离计算未校正多普勒效应，而这里的计算已校正。使用三角调制的缺点是增加了延迟，因为计算距离和速度时需要稍早的数据；但不再需要处理信号相位。

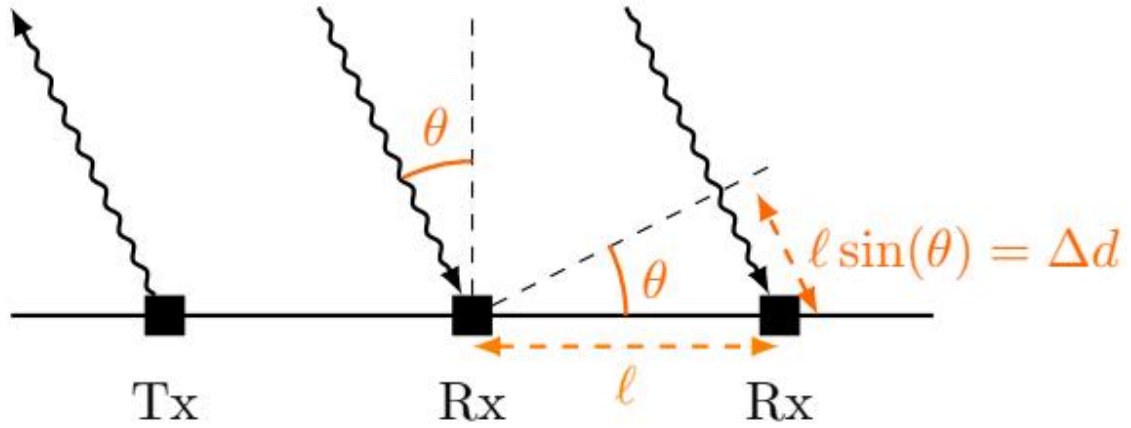


图 9.5: 图 9.5 使用相控阵雷达估计到达角。TX 发射天线发射的信号，由相距 ℓ 的两个 RX 接收天线接收时所测得的相位差，对应于目标的角度 θ 。

9.1.2.5 确定 SoC 雷达的角度

对于旋转雷达，在每一时刻波束集中于单一方位方向，因此确定目标角度十分简单。SoC 雷达的角度估计稍复杂，但可用与上述类似的相位差计算实现。

给定多个间距为 ℓ 的接收单元，反射中会产生距离差 Δd 。利用几何关系 $\Delta d = \ell \sin \theta$ ，可由式 (9.3) 的变体推导到达角 θ 。如图 9.5 所示，接收信号到达第二个接收天线必须多走 $\ell \sin \theta$ 的距离。这对应于两个 RX 天线接收信号之间 $\Delta \phi = (2\pi/\lambda)\ell \sin(\theta)$ 的相位差。给定所测相位差 $\Delta \phi$ ，可计算到达角 θ ：

$$\theta = \arcsin \frac{\lambda \Delta \phi}{2\pi \ell}. \quad (9.10)$$

原则上，一个 TX 天线和两个 RX 天线已足以确定目标角度；但多于两个 RX 天线可提高分辨率，因此可以区分相邻的多个目标。返回信号在每个 RX 天线处都会额外偏移 $\Delta \phi$ 。对 RX 天线上的信号采样并对此信号序列执行 FFT，可以可靠地估计 $\Delta \phi$ 。

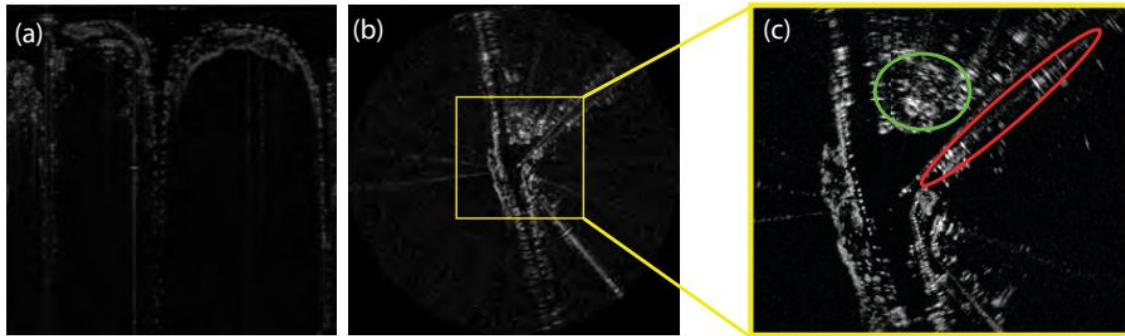


图 9.6: 图 9.6 (a) 中的极坐标雷达图转换为 (b) 中的欧氏坐标后，可从 (c) 的放大图观察到几类区别于其他传感器的雷达噪声。最常见的是散斑噪声回波，模糊杂波用绿色圈出。多径反射发生在回波接触天线前先从附近墙壁或地面反弹时，从而产生真实目标的反射；一串重复回波用红色圈出。原始图像取自 Mulran 数据集 [567]。

上述示例说明了“SIMO”相控阵雷达系统 (single input, multiple output, 单输入多输出)。多数用于 SLAM 的雷达传感器是具有多个 TX 和 RX 天线的 MIMO (multiple input, multiple output, 多输入多输出) 系统。只要 RX 天线能够区分多个 TX 天线的信号，与其将 RX 天线数量加倍，不如增加一个 TX 天线也可以达到相同分辨率。可以采用不同技术确保 TX 信号不相关 (正交)，例如频分 (每个发射器使用不

同频段)、码分(每个发射器发送由唯一代码序列调制的信号),或时分多址(time-division multiple access, TDMA),即每个发射器使用不同时间槽。同样原理也可应用于能够同时测量方位角和俯仰角的二维 TX-RX 阵列,产生 3+1D 数据。

9.1.3 雷达应用的挑战

与任何传感器系统一样,雷达技术也带来一系列需要在开发中审慎考虑的挑战。这些挑战包括多种在雷达中尤其突出的噪声类型,例如多径反射、因波束宽度较大造成的有偏且稀疏的距离读数、接收机饱和和散斑噪声。图 9.6 给出了部分噪声现象的示意,它显示了转换为欧氏坐标的极坐标雷达图。第 9.1.4 节将讨论一些相关雷达滤波技术。

9.1.3.1 散斑噪声

雷达测量噪声来自多个来源,包括热噪声、电子缺陷以及目标 RCS 的变化。雷达发射电磁脉冲时,捕获天线视场内所有物体反射回来的能量。脉冲与物体的相互作用会散射雷达波,导致相长和相消干涉。这类相互作用无论信号来源如何,都可能产生假信号,或抵消天线接收到的合法回波。这些因素导致频域信号变化,其中最显著峰值混合了真实目标和虚警。缺乏区分真实回波与虚假回波的机制时,传感器会生成散布点的图案,通常称为散斑噪声。为了准确识别对位姿估计和特征匹配至关重要的路标,必须估计这些反射周围的不确定性,可能还需跨多次扫描进行估计。

9.1.3.2 多径

除散斑噪声外,多径回波也是另一种测量误差,由与单一物体相关的不同探测路径产生。设想传感器前方有一个路标:部分发射射线可能直接到达该路标,而其他射线可能先从地面或墙壁反射,之后才到达天线。雷达会将其解释为路标位于道路下方或墙壁后方,从而感知到所谓“鬼影物体”(ghost objects)或静态离群值。消除这些离群值对于确保点云建图或定位的可靠性至关重要。

9.1.3.3 运动引起的畸变

包括 LiDAR 和雷达在内的扫描传感器天然存在运动畸变。旋转雷达尤其如此,它通过单次旋转构建每幅极坐标图像。因此,若传感器正在运动,同一旋转开始和结束时捕获的单个物体位置不同。对低频运行的传感器或车辆快速运动时,这一差异会很显著。例如,常用成像雷达 Navtech CIR 304 的工作频率为 4 Hz,使得原始帧难以准确对齐。还有更快的旋转雷达,如以 10 Hz 旋转、与许多 LiDAR 相近的新型 Navtech RAS3,以及可达 50 Hz 的 Indurad iSDR。即便如此,当传感器安装在汽车等快速移动平台上时,运动引起的畸变仍可能显著。

9.1.4 雷达滤波

假目标的出现和分布(第 9.1.3 节)会随时间变化,且具有不可预测的参数;因此,简单阈值等静态滤波方法不足以应对,因为它们可能让虚警通过。为此,研究者设计了能够动态估计虚警分布的方法。这类滤波方法通常考虑一个方位方向上的测量,试图估计哪些距离单元包含真实目标,哪些属于虚警。

恒虚警率(constant false alarm rate, CFAR)滤波器^[811]是一种广泛实现的方法,旨在在动态变化且不均匀的干扰中维持指定虚警概率。该过程先将信号(例如在雷达 ADC 样本经 FFT 后得到的频域表示)划分为称为单元的离散分段,再使用滑动窗口评估这些单元。窗口核心是一组待测单元(cells under test, CUT)。通过比较 CUT 的强度与其前后相邻的训练单元强度进行评估。在一些实现中,训练单元和 CUT 之间还会设置保护单元,以防 CUT 的局部影响影响训练单元幅值。根据 CUT 集的强度是否超过计算阈值,决定接受或拒绝;该阈值由周围训练单元(不包括保护单元)的比较强度导出。CFAR 有多种变体,经典版本为单元平均 CFAR(cell-averaging CFAR, CA-CFAR):阈值相对于训练单元的平均功率计算,并按根据目标虚警概率选择的阈值乘数缩放。常见替代方案为有序统计 CFAR(ordered-statistic CFAR, OS-CFAR),它使用训练单元的中位数等更鲁棒的统计量,而非均值。

图 9.7 展示了 CA-CFAR 阈值如何适应雷达信号，以及与其他两种滤波方法相比，哪些距离单元被选为目标或噪声。

专门针对雷达里程计设计和测试的 CFAR 变体是 BFAR (bounded false alarm rate) [25]。它只需对 CFAR 检测器计算的输出 Z 应用仿射变换 $T = aZ + b$ ；其中 b 是可学习参数，用于缩放输出，使其在 CFAR 输出与固定阈值之间混合。

尽管 CFAR 及其变体用于许多雷达应用，若干雷达里程计和 SLAM 流水线采用更简单的滤波策略。CFAR 是为目标检测开发的，在该任务中不遗漏弱检测可能很重要；而为里程计过滤雷达时，关注点是仅保留能够随时间和从不同视点可靠检测到的点。一种常用技术是在每个方位角上，选择高于静态阈值的 k 个最强回波，其中 k 从 1 起。也可使用统计阈值，选择强度高于均值一个标准差的所有点。

一些近期方法改用机器学习技术提高雷达输出的准确性和分辨率，通常使用 LiDAR 数据作为训练真值。Cheng 等人 [195] 使用生成对抗网络 (generative adversarial network, GAN) 根据距离-多普勒速度矩阵生成点云。Xu 等人 [1210] 训练回归器和分类器：回归器输出改进的高分辨率深度读数，分类器估计数据是否超出测程。这些方法旨在学习一种模型，只保留雷达图中对应于 LiDAR 可以检测到的表面的回波。

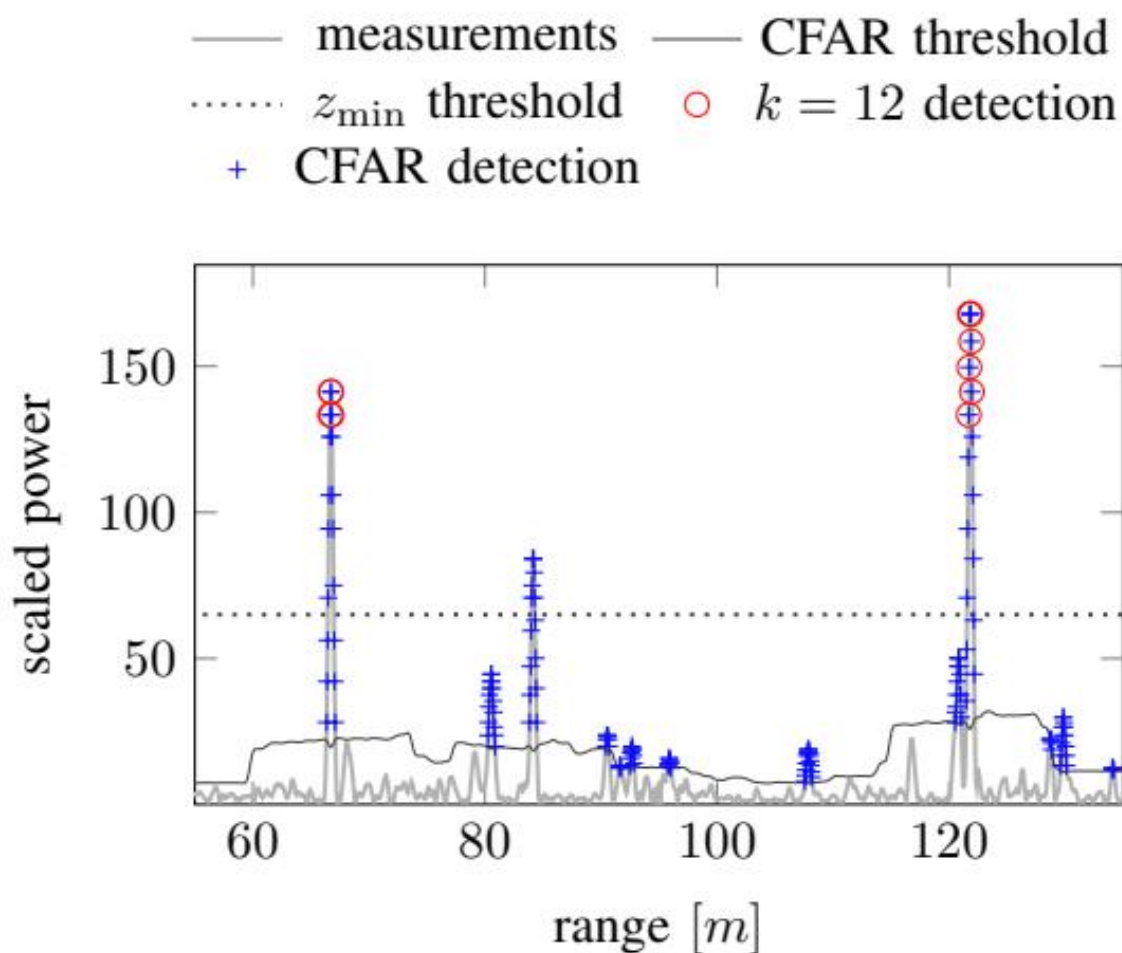


图 9.7: 图 9.7 雷达滤波示例，比较 CFAR 滤波器、恒定功率阈值和 k 最强策略。一个方位方向上的功率/距离曲线以灰色绘制。真实目标的回波在图中呈现为尖峰，但也存在若干模糊峰值。CFAR 产生自适应阈值，以黑线绘制；CFAR 报告的检测以蓝色绘制，在本例中包含多个“虚警”。 k 最强滤波器（本例 $k = 12$ ）更保守，仅返回主要目标附近的点。图引自 Adolfsson 等人 [6] (©2021 IEEE)。

9.2 雷达里程计

雷达里程计的目标是：给定随时间排序的一组雷达读数，估计传感器的自身运动。雷达里程计方法通常处理某种中间表示，例如最近 N 次雷达读数，或持续裁剪的局部地图。其重点是获得局部尺度上的准确位姿估计。即使里程计方法性能良好，如果不显式考虑回环，误差最终也会无界累积。使用旋转雷达的方法每行驶 100 m 可能累积约 1–2% 的平移漂移。

许多雷达传感器的一个特征，是能够逐点提供“多普勒”速度估计。第 9.2.1 节将说明如何在无对应关系的情况下利用它估计里程计。除基于多普勒的方法外，还经常通过空间对应关系估计两次相邻雷达扫描之间的相对变换，以确定一次扫描的哪些部分也存在于另一次扫描中。给定一组对应关系，可以计算并优化距离度量。根据扫描器类型，获得对应关系的方式不同：旋转雷达通常产生原始信号，可以像第 9.2.2 节那样直接使用，也可以从原始信号提取包含信息的高层特征（第 9.2.3 节），或提取可用于配准的距离点，类似 LiDAR 里程计（第 9.2.4 节）。



图 9.8: 图 9.8 Oxford Radar Robotcar 数据集^[127]上的开环雷达里程计示例。使用 CFEAR 方法^[8]处理 Navtech 二维旋转雷达数据。真值轨迹以蓝色绘制，里程计估计以橙色绘制；从雷达图提取的点目标以灰色绘制。

图 9.8 展示了使用二维扫描雷达进行开环雷达里程计的示例。

9.2.1 多普勒里程计

由多普勒测量获得的径向速度可用于直接估计传感器的线速度。Doer 和 Trommer^[272] 将多普勒信息与三点 RANSAC 和最小二乘问题结合，用于估计线速度。

然而，单个雷达数据中的逐点速度测量无法直接观测旋转速度分量，除非能对雷达系统运动学模型作出假设，例如已知雷达相对于旋转中心的安装位置并假设不存在侧滑^[551, 354]。因此，仅依赖单个雷达多普勒信息的雷达里程计系统通常还使用 IMU 数据（更具体地说是陀螺仪）。Huang 等人^[490] 将消费级 IMU 与级联 SoC 雷达结合，在多样三维室内空间中实现低漂移。Kubelka 等人^[609] 比较了多种基于配准的 3+1D 雷达方法与无配准的多普勒 + IMU 里程计^[272]，发现无配准方法产生的误差最低，尤其在特征稀疏环境中；据报道，其在 4.5 km 轨迹上的漂移低至 0.3%。

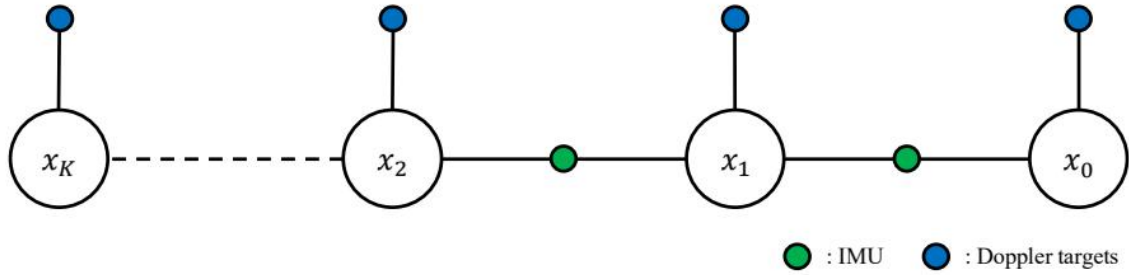


图 9.9: 图 9.9 雷达惯性速度估计系统的因子图表示。利用多普勒目标和多组 IMU 测量作为约束，联合估计过去 K 个时间步的状态。图改编自 Kramer 等人^[606]。

Kellner 等人^[551] 展示了：当雷达安装在具有 Ackermann 转向的车辆上，且已知雷达传感器相对于车辆旋转中心的安装位置时，可以从 2+1D 雷达数据估计线速度和旋转速度。Galeote-Luque 等人^[354] 将其扩展到 3+1D 情形，估计五个自由度（三维线运动以及偏航和俯仰旋转，但不包括滚转）。

由于基于多普勒的里程计模态相当专属于雷达方法，下面以 Kramer 等人^[606] 的工作为例说明。

9.2.2 示例：多普勒里程计因子表述

从雷达整合多普勒因子的一种直接方式，是在包含过去 K 次雷达测量的滑动窗口中估计传感器平台的机体坐标系速度。这些速度通过 IMU 的加速度计测量积分相互连接，从而形成准确速度估计的完整系统。系统结构可以用因子图表示：利用多普勒目标和多组 IMU 测量作为约束，联合估计过去 N 个时间步的状态。

IMU 测量（加速度计和陀螺仪）通常同时受到偏置 $\mathbf{b}$ 和重力 $\mathbf{g}^w$ 的影响。由雷达数据得到的速度估计不含偏置，因此可将加速度计偏置纳入状态向量进行补偿。但要补偿重力影响，需要估计 IMU 姿态，具体是由方向四元数 $\mathbf{q}_s^w$ 表示的俯仰和滚转。于是，完整状态向量可表示为

$$\mathbf{x} = \left[\mathbf{v}^{s\top} \quad \mathbf{q}_s^{w\top} \quad \mathbf{b}^\top \right]^\top.$$

雷达惯性自身速度估计被表述为一个优化问题，其代价函数整合约束和测量，以准确估计传感器平台速度：

$$J(\mathbf{x}) = \underbrace{\sum_{k=1}^K \sum_{d \in \mathcal{D}_k} w_{d,k} e_{d,k}}_{\text{多普勒项}} + \underbrace{\sum_{k=1}^{K-1} \|\mathbf{e}_k\|_{\mathbf{W}_k}^2}_{\text{惯性项}}. \quad (9.11)$$

其中， K 是估计状态的过去雷达测量数， $\mathcal{D}_k$ 是时刻 k 雷达测量返回的目标集合， $e_{d,k}$ 是多普勒速度误差， $\mathbf{e}_k$ 是 IMU 误差。对 IMU 误差，误差项由协方差矩阵 $\mathbf{W}_k$ 加权；对多普勒速度测量，则由相应雷达目标的归一化强度加权：

$$w_{d,k} = \frac{i_{d,k}}{\sum_{j \in \mathcal{D}_k} i_{j,k}}, \quad (9.12)$$

其中 $w_{d,k}$ 是扫描 $\mathcal{D}_k$ 中目标 d 的权重, $i_{d,k}$ 是目标 d 的强度。

本例中的雷达测量由目标集合组成。对每个 $d \in \mathcal{D}$, 测量 $[r \ v \ \theta \ \phi]^\top$, 分别表示目标 d 的距离、多普勒(径向)速度、方位角和俯仰角。多普勒速度测量 v 等于目标与传感器之间相对速度向量 $\mathbf{v}^s$ 在传感器坐标系中, 沿传感器原点到目标的射线 $\mathbf{r}^s$ 的投影模长, 即目标在传感器坐标系中的速度与从传感器指向目标的单位向量的点积:

$$\mathbf{v}^{s\top} \left(\frac{\mathbf{r}^s}{\|\mathbf{r}^s\|} \right). \quad (9.13)$$

该方法假设场景目标静止, 仅传感器平台运动。此时, 每个雷达目标都可为机体坐标系中传感器组速度估计提供约束。每个雷达目标的速度误差为

$$e_{d,k} = v_{d,k} - \mathbf{v}_k^{s\top} \left(\frac{\mathbf{r}_{d,k}^s}{\|\mathbf{r}_{d,k}^s\|} \right), \quad (9.14)$$

其中 $v_{d,k}$ 是时刻 k 目标 d 的径向速度测量, $\mathbf{v}_k^s$ 是时刻 k 的传感器速度, $\mathbf{r}_{d,k}^s$ 是时刻 k 从传感器指向目标 d 的向量。如前所述, 雷达测量受到非高斯噪声影响, 雷达扫描还常包含虚假目标数据。可以对多普勒残差使用 Cauchy 鲁棒范数应对这些挑战。此外, 本例假定雷达传感器与 IMU 位置和姿态相同; 一般并非如此, ^[605] 给出了一般推导, 阐明机体角速度与雷达传感器坐标系速度之间的耦合。

9.2.3 直接里程计

直接处理原始雷达图(如图 9.6 和图 9.2 所示), 而不是处理从雷达图中过滤得到的点的方法, 称为直接法。

直接方法使用相位相关和 Fourier–Mellin 变换^[176, 848]等经典信号处理技术。给定两个连续极坐标雷达图, 可在极坐标系中通过平移找到相对旋转, 其中垂直平移对应方位角变化。利用相位相关, 可从使两幅极坐标图一致性最大的像素平移选择相对方向。随后, 在笛卡尔坐标系中类似地计算图像相关性, 细化平移(图 9.6)。这些直接相关方法假设特定位置的功率回波随时间保持静止, 从而能够进行有意义的相关。但这一假设常常失败, 尤其是在存在动态物体或噪声伪影常见的雷达数据中。

Barnes 等人^[53]的“通过运动进行掩膜”(Masking by Moving)方法也使用相关性。它在规则的位姿候选网格上, 计算当前扫描与上一扫描旋转副本之间的二维相关。Barnes 等人通过训练卷积神经网络(CNN)避免纳入由噪声造成的虚假特征, 以处理功率回波非静止问题。CNN 训练目标是预测一个掩膜, 仅保留静止、因而更适合相关扫描匹配的雷达图部分。

与使用全部或几乎全部雷达图直接里程计方法相反, 本章其余方法都是间接方法: 它们先选择特定特征或关键点, 再在更稀疏的点上运行, 以估计里程计。

9.2.4 基于特征的里程计

二维旋转 FMCW 雷达的雷达图本质上是鸟瞰图像, 因此自然有许多工作使用计算机视觉领域的基于图像的特征提取和匹配技术(例如 SIFT^[140, 650]、SURF^[473]和 ORB^[546]特征)寻找对应关系并估计里程计。Callmer 等人^[140]匹配群岛中的大尺度岛屿特征, Li 等人^[650]从卫星雷达中提取特征。与相机图像相比, 雷达数据噪声更高且高度依赖环境, 因此提取可用于特征匹配的描述子更困难^[474]。与其他传感器相比, 雷达特征描述子还可能具有更强地点依赖性, 使传感器位姿不同时数据关联更困难(另见第 9.3 节)。FSCD 和 BASD^[908, 983]是专为雷达图设计的关键点提取器和特征描述子示例。

上述技术提取一组显著关键点(可在后续帧中可靠检测), 并计算描述关键点周围区域的特征描述子。给定提取的特征集合, 通常通过特征描述子匹配计算对应关系, 并结合 RANSAC 等鲁棒估计器。Lim 等人^[663]给出了基于鲁棒估计器的抗离群基于特征雷达里程计示例。给定对应关系后, 最小化特征之间的空间距离, 通常使用奇异值分解(singular value decomposition, SVD)求最小二乘解。一般而言, 相比下一节讨论的基于配准的方法, 基于特征的方法对较大初始误差更稳定, 因为特征描述子能够鲁棒关联, 而配准方法主要依赖点邻近性进行数据关联。

除上述手工设计的特征描述子外，也可通过深度神经网络生成关键点和特征描述子，从而自动生成特征^[52]。其缺点是需要包含雷达图和相似环境真值位姿的训练数据。

9.2.5 基于配准的里程计

LiDAR 测距传感器的点云里程计估计已有成熟方法，相似技术也可用于从雷达图或雷达数据立方体提取的点云。本节说明如何计算这类雷达点云并用于里程计估计。

对图 9.6 所示能够提供原始信号数据的旋转雷达，首先需要通过滤除不对应相关峰值的信号回波，选择要使用的点。² 如第 9.1.4 节所述，许多方法在每个方位角上提取一组距离读数，例如使用 CFAR (图 9.7、第 9.1.4 节)、使用噪声统计量去除冗余或噪声读数^[158]、BFAR^[25]，或简单选择每个方位角上最强的 k 个回波。Kellner 等人^[550]是例外，他们使用 DBSCAN 聚类，同时考虑相邻方位角，而非每次限制在一个方位维度上搜索目标。近期方法还常使用机器学习训练模型，从雷达图估计类似 LiDAR 的点云^[195, 1210]。关键挑战是提取适当数量的读数：太少会丢失相关信息，太多会包含噪声^[158]。例如，研究发现 CFAR 在这方面难以调节^[127]。

使用上述方法之一提取点云后，可以直接使用，也可以通过分析局部邻域估计法向量、平面和点分布等附加信息。雷达数据的配准方法通常类似 LiDAR 扫描配准 (见第 8.2 节)；但雷达数据的噪声水平和稀疏性使两两配准更具挑战。

基于配准的雷达里程计常见策略是将新扫描配准到多次过去扫描，过去扫描可以聚合为子地图，也可以作为独立点云集合。与多次扫描配准能够补偿前述雷达数据的若干挑战。加入更多关键帧后，里程计估计对稀疏且含噪的雷达点云不那么敏感。更多对应关系增加了约束，能够减少特征贫乏环境中的漂移；同时也提供时间冗余，即使用多个关键帧时，突然遮挡或移动物体产生的伪对应关系对里程计估计影响较小。

基于配准的雷达里程计方法示例包括连续时间 ICP^[127]、功率平移 NDT^[615] 和 CFEAR^[8] (不要与第 9.1.4 节讨论的 CFAR 滤波方法混淆)，它们都使用子地图或多个关键帧。在 CFEAR 中，通过选择每个方位角上最强的 k 个回波，从雷达图提取点云。每个新点云使用点到点、点到线或点到分布误差度量，联合配准到最近的 s 个关键帧，这类似 NDT 扫描配准 (见第一部分)。对每个点，在一定半径内邻近点的协方差矩阵估计法向量。根据法向量一致性、平面性 (协方差矩阵的条件数) 和邻域点数对点对应关系加权。Kung 等人^[615]使用固定阈值从雷达图提取点云，并利用 NDT 表示将多个点云聚合到雷达子地图中，其中每个点的贡献由其回波信号强度加权。Burnett 等人^[127]使用 BFAR^[25]从雷达图提取点云并聚合到局部子地图，随后使用连续时间 ICP 表述优化轨迹估计，其中点关联到高斯过程运动先验。

上述方法均基于二维雷达数据。近期基于配准的 3+1D 雷达里程计流水线往往采用相似策略，但点云提取在传感器上完成，因此选择哪些信号峰值作为有效目标，取决于传感器特定阈值，而非显式特征提取。由于 3+1D 点云提供逐点多普勒速度信息，这些方法通常先根据多普勒数据采用最小二乘估计自身速度 (见第 9.2.1 节)，再进行点云配准予以细化。4DRadarSLAM^[1271]中的基于配准里程计组件使用适配雷达点云的 GICP^[985]变体，以协方差矩阵对点加权；由于方位角和俯仰角精度有限，距离传感器越远的点不确定性越高。4D iRIOM^[1310]采用一对多的分布到分布匹配，以缓解雷达点云噪声和稀疏性：不将每个点匹配到局部子地图中最近的对应分布，而是匹配到最近分布的加权集合。^[1310, 1271]中给出的完整 SLAM 流水线将在第 9.4.2 节进一步介绍。EFEAR-4D 方法^[1199]将 CFEAR 二维里程计^[8]扩展到 3+1D 雷达点云。它先计算基于多普勒的自身速度估计，去除与该最小二乘估计不一致、因而可假定来自移动障碍物的离群点；其余配准方案与 CFEAR 类似，即将扫描配准到一系列先前关键帧，并利用法向量一致性和平面性关联和加权单个点匹配。

9.2.6 运动补偿

如第 9.1.3.3 节所述，里程计估计中必须补偿运动畸变。对频率为 4 Hz 的低速旋转雷达，据报道使用恒速度模型进行自身运动补偿可将绝对轨迹误差 (absolute trajectory error, ATE) 降低 29%^[8]。给定两次连续雷达扫描的时间戳及使用上述方法计算的相对位姿，可以计算速度；由于逐点时间信息也可获得，便可相应移动每个雷达点。以这种方式偏移单次雷达扫描中的点，能够补偿扫描雷达缓慢扫描期间产生的显著运动。同一模型还用于为刚性扫描配准提供初始估计；每次将运动补偿后的扫描配准到上一扫描后，将其作为节点

加入 SLAM 位姿图。

不过，该方法仍在离散位姿图设置中运行。连续时间轨迹表示有利于获得平滑且准确的轨迹，因为可以查询每次传感器读数时刻用于去畸变的位姿。Ng 等人^[803]在使用汽车 SoC 雷达的流水线中实现了样条表示。Burnett 等人^[129]使用高斯过程构造因子图，将 IMU 传感器与旋转雷达结合表示轨迹。

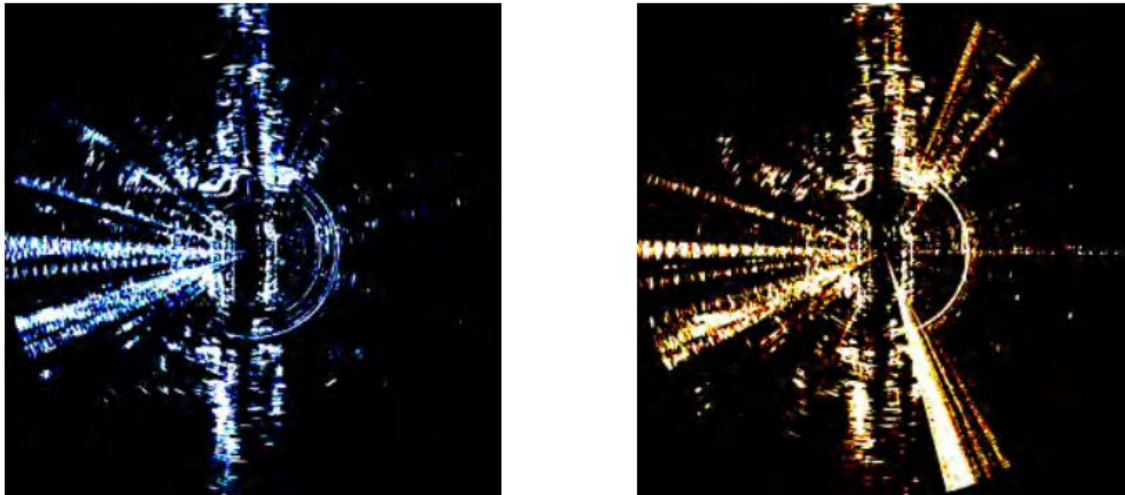


图 9.10: 图 9.10 在同一地点获得的两次雷达扫描。噪声模式造成视觉混叠，使地点识别过程复杂化。

9.3 雷达地点识别

与其他传感器模态一样，地点识别 (place recognition, PR) 是雷达 SLAM 的必要模块。关于一般问题的良好概述可参见第 8 章。如视觉和 LiDAR 地点识别章节所述，在平移和旋转变化下保持不变性至关重要，例如在不同车道行驶，或从另一条街道到达同一交叉路口时。

9.3.1 雷达地点识别的独特挑战

原则上，前面章节讨论的若干地点识别方法，在适当转换数据格式后也可用于雷达数据；但雷达仍具有一些独特特征和挑战（另见第 9.1.3 节）。

雷达地点识别的主要挑战由多个因素造成。首先，雷达数据分辨率低，细节较少，因而可用于识别的可区分特征更少。雷达数据的宽波束导致角度歧义，使远处物体呈现为宽斑块，而从近处观察时其形状会明显不同。此外，较低的信噪比也会给地点识别带来挑战，除非进行充分滤波，如图 9.6 所示。接收机饱和可能产生强径向特征，该特征随目标观测角度显著变化，如图 9.10 样例所示。因此，由于这些挑战，地点的外观可能从附近不同位置观察时发生显著变化。

由于雷达类型不同，地点识别也需要根据传感器类型区别处理。长距离 360° 旋转雷达与测程和视场有限的 SoC 雷达，对地点的感知和描述有显著差异。例如，旋转雷达产生二维雷达图，可将其视为图像，自然可以将受视觉地点识别启发的方法应用于旋转雷达。对雷达图应用 CFAR 等目标检测算法会得到点云，之后可以改用二维 LiDAR 地点识别方法。旋转雷达往往测程很长，对地点识别十分有利^[567]；但其扫描速率较慢，即使车辆低速行驶，也可能产生运动畸变。另一方面，SoC 雷达无需机械旋转天线即可覆盖视场，因此更新速率更快，较少受运动畸变影响。多数使用 SoC 雷达数据进行地点识别的方法处理二维/三维点云数据，而非完整数据立方体。因此，SoC 雷达地点识别通常建立在现有 LiDAR 地点识别方法之上；但 SoC 雷达测量通常受限于较小视场，并且稀疏性和噪声更高。

目前，使用旋转雷达的地点识别文献多于使用 SoC 雷达的文献。部分原因是旋转雷达在更宽视场内捕获更丰富信息，能够更好地捕获周围结构，从而实现鲁棒地点识别，尤其适合室外应用；另一原因是大规模数

据集更易获得。但这一优势并不妨碍 SoC 雷达用于地点识别。旋转雷达提供更大视场，但其投影模型仅提供二维信息，丢失高度细节。对于室内地点识别，较短测程和较小视场通常已足够，此时 SoC 雷达的 3+1D 测量仍具有优势。

9.3.2 基于学习的雷达地点识别

对于旋转雷达，可将二维雷达图视为图像，利用二维图像检索进行地点识别。³

Saftescu 等人^[963]的工作是二维 FMCW 雷达地点识别的早期方法之一。它使用 CNN 表示雷达图，并针对其极坐标性质使用柱面卷积，学习旋转不变表示。随后，可将查询雷达图与参考图像数据库匹配，得到仅基于外观的拓扑地点识别系统。De Martini 等人^[246]加入位姿细化，得到拓扑度量建图和定位系统。

地点识别方法可使用输入数据的增强实例训练，以提高对小变化的鲁棒性。地点识别数据通常在驾驶沿途按扫描连续记录，因此可以通过采样时间上相邻的帧获得“增强”实例，并通过平移极坐标雷达图人为加入旋转。与这些增强实例形成对比，网络训练不仅要识别相似实例，还要区分时间上相距较远的实例（及其增强版本）。这样，无需知道雷达数据的真实度量位置，也可通过无监督方式获得训练地点识别算法的数据^[352]。

上述方法专为提供 360° 长距离覆盖的旋转雷达设计。SoC 雷达没有运动部件、体积更小，对汽车应用更有吸引力；但其测程较短、视场较小，因此需要略有不同的处理。

Cai 等人^[139]在将点云投影到二维图像平面后，使用深度时空编码器生成特征向量，作为拓扑地点识别的地点描述子。根据 RCS 对这些向量重新排序后，将其传入 NetVLAD^[35]层，以滤除无关静态特征。在该工作中，车辆周围安装多个雷达传感器，以克服视场有限问题。

Herraez 等人^[454]展示了单次扫描雷达地点识别流水线：通过学习关注对地点识别重要的显著点，处理数据稀疏和噪声问题。与 Cai 等人^[139]类似，Herraez 等人也利用 NetVLAD 生成特征编码。但他们使用刚性核点卷积捕获三维上下文信息，而非将三维点云投影到二维图像。进一步地，“点重要性估计器”输出点对地点识别重要的概率。该估计器使用已知对应点云集合训练，在另一扫描中小半径内存在对应关系的查询点被标为重要。RCS 信息通过独立网络纳入，该网络将点的 RCS 数据编码为更紧凑的特征表示。Peng 等人^[858]以类似第 9.2.1 节多普勒里程计的方法过滤噪声点：使用 RANSAC 估计自身速度，得到可能静止的内点集合，再将剩余离群点作为噪声滤除。Peng 等人没有使用为视觉地点识别设计的 NetVLAD，而是展示了名为 MinkLoc4D 的雷达专用特征提取骨干网络，该网络借鉴 LiDAR 地点识别架构^[1320]。

9.3.3 基于描述子的雷达地点识别

使用手工设计的描述子而非学习得到的嵌入通常同样有效。雷达地点识别的一种常见做法，是直接采用 LiDAR 描述子，例如 Scan Context^[568]、RING^[1212]或 M2DP^[445]；不过，由于传感器数据格式不同，需要进行适当调整。例如，旋转雷达数据缺少三维结构信息（如高度），但可以用 RCS 代替。此外，还应针对雷达传感器数据的固有挑战采取额外措施，通常需要仔细进行噪声滤波、稀疏性处理和运动补偿。

Hong 等人^[474]通过将原本为三维点云设计的 M2DP 描述子^[445]适配到从雷达图提取的二维点云，实现地点识别。此外，他们研究二维平面上的点分布，以判断点云是否可能具有区分性。对二维点执行 PCA 可得到两个特征值，它们描述点沿各特征向量方向的展开程度。当两个特征值差异显著时，说明扫描在某一方向上缺乏特征（例如高速公路行驶数据）；Hong 等人不会将此类扫描用于地点识别。

Jang 等人^[510]修改了 RING 描述子^[1212]以适用于雷达。RING 生成的描述子为正弦图形式，具有旋转和平移不变性。两个正弦图之间的相关性应能给出地点识别问题的正确匹配；但雷达图像中的高水平噪声可能使直接比较失效。研究表明，进一步加入自相关能够提高雷达图像的地点识别性能。

Adolfsson 等人^[9]以多种方式调整由二维雷达图构成的 Scan Context 描述子。首先，他们计算 Scan Context 网格单元中所有点的强度和，以同时编码强度和点密度。其次，每个描述子都由一组经过噪声滤波和运动补偿后聚合的极坐标图像生成，从而减轻第 9.3.1 节所列的一些挑战。仅保留每个方位方向上最强的 k 个回波是一种保守滤波器，往往可以去除雷达点云中原本存在的大部分噪声。在多个已配准点云上生成 Scan Context 描述子，可以进一步弥补这种保守滤波留下的稀疏数据。在行驶过程中使用旋转雷达时，通过恒加速度模型对点云去斜，对于生成可比较的描述子十分重要。

面向 3+1D SoC 雷达的地点识别方法也通常实现 Scan Context 描述子的变体^[1271, 1310, 658]。然而，虚

假雷达点很容易扭曲高度测量，因此，对于雷达点云而言，使用每个径向网格单元的最大高度（原始 Scan Context 的做法）并不总是像 LiDAR 那样有效。Intensity Scan Context 描述子^[1151]是一种替代方案：它在 Scan Context 网格单元中存储点的最大测量强度，而不是高度。另一种方法是在网格单元内使用强度和，从而同时利用强度和点密度；这样既能在描述子中编码垂直结构，又不易受到噪声点位置的影响。使用 3+1D 雷达时，另一个重要因素是：与 Scan Context 所针对的 360° LiDAR 相比，修改后的雷达描述子必须适应窄得多的视场。由于同一地点从两个不同视点观察时可能呈现完全不同的外观，当传感器只覆盖小视场时，很难实现旋转不变性。一种解决办法是根据当前里程计估计进行回环预过滤，只尝试将当前描述子与偏航角处于某一范围（例如 20°）内的帧匹配^[1271]。

9.4 雷达 SLAM

雷达 SLAM 系统一般采用与 LiDAR 或相机 SLAM 解决方案相同的总体结构。本节简要介绍过去二十年中的若干代表性系统，重点关注针对雷达数据进行 SLAM 定制时的特有方面。第 9.4.3 节还将介绍与其他外感知传感器组合的多模态系统。

9.4.1 地图表示

许多雷达 SLAM 系统使用第 5 章讨论过的类似地图表示。然而，雷达测量稀疏且常包含虚假数据，给建图过程带来了独特挑战。一些较早的雷达建图工作直接基于现有目标检测模型建立地图表示，通常使用路标地图，其中每个检测到的目标构成地图的一部分^[224, 271, 140, 983]。这种目标检测也可用于占据栅格地图，例如 Mullane 等人^[781]的系列工作；他们在建图阶段利用了检测概率^[782]。

由信号中的单个峰表示的检测目标，也可以像 LiDAR 扫描中的点一样处理。例如，这些点可用于创建二维占据栅格地图^[730, 762]或点云地图^[474]。提取的点云还可以加入额外信息，例如^[9]计算周围点的分布以估计方向和权重，其方式类似 surfel 或 NDT 单元。直接点云表示在新兴的高分辨率 SoC 雷达中也很常见；这类雷达具有更大的垂直视场（3+1D）和更多 TX/RX 天线（例如 Sensrad Hugin 雷达具有 48 个 TX 和 48 个 RX 天线）^[1271, 1310]。

虽然不太常见，但完整的雷达热图（峰值检测前、每个方向和距离网格单元的强度样本）也可用于提供稠密栅格地图^[950]。此时，地图表示环境中每个位置的反射功率全局热图，而非占据证据。随后可通过相关性实现这些热图之间的二维对齐。

Kramer 和 Heckman^[604]在生成里程计的同时，提出了一种用于雷达数据体素建图的新型传感器模型，即使存在视觉遮挡也能创建稀疏地图。该传感器模型采用 Octomap^[478]中用于估计占据单元与自由单元的对数几率方法，但替换了其中的射线投射模型。Octomap 射线投射模型假定传感器首次接触的位置是唯一相关的占据点，而其推广模型利用雷达能够穿透某些材料的能力，在传感器视场内更新体素概率：有雷达回波的单元概率增加，未检测到的扫描单元概率降低，同时不假设射线上的信息。

Nuss 等人^[815]提出了一种专门考虑雷达的相关栅格地图表示。他们设计了用于处理栅格地图中动态障碍物的状态估计滤波器，称为概率假设密度多实例 Bernoulli 滤波器。该滤波器将栅格单元视为有限随机集合，并动态融合雷达和 LiDAR 数据。

最后，可以使用机器学习技术处理雷达数据密度低、噪声高的问题^[1210, 773]，以 LiDAR 数据作为真值，提高雷达输出的分辨率并降低噪声。Mopidevi 等人^[773]构建全局雷达地图，并使用预测网络对地图块进行上采样：网络过滤自由空间区域中的噪声，填充稀疏和空缺区域，生成更接近 LiDAR 结果的地图。

近年来，最初为 RGB 数据开发^[765]、后来应用于 LiDAR 数据^[1293, 1056]的神经场也被用于二维雷达数据^[98]。神经表示的关键特征是对环境进行隐式表示，使神经网络能够接收空间中的一点并返回距离最近表面的距离、颜色和不透明度等量。在 Borts 等人^[98]的雷达场中，使用物理先验的雷达传感器模型创建隐式神经几何和反射率模型，随后可利用它从未观测视点合成雷达测量。雷达传感器处接收的功率取决于已知的发射功率和天线增益，也取决于由物体大小、雷达反射率和方向性共同决定的 RCS。神经表示学习将测得的 RCS 分解为大小（面积）与反射率和方向性的乘积两部分。

9.4.2 雷达 SLAM 流水线

前几节介绍了基于图的雷达 SLAM 框架的主要组成部分：开环里程计估计、用于检测回环的地点识别以及地图表示，此外还介绍了与雷达 SLAM 相关的物理和技术原理。

本节回顾若干仅使用雷达作为外感知传感器的完整 SLAM 流水线，并讨论它们如何实现这些组成部分。这些流水线通常遵循图 9.1 所示的架构：前端先将原始雷达数据滤波为点云，并通过地点识别模块确定回环、向底层位姿图添加相应约束；后端地图模块执行逐帧里程计，SLAM 模块则在检测到回环后对地图进行全局优化。



图 9.11: 图 9.11 使用 3+1D 雷达 (Sensrad Hugin) 和 IMU 输入通过雷达 SLAM 生成的三维地图示例。颜色表示高度。左：回环闭合前；注意左侧三维地图中明显累积的水平和垂直漂移。右：回环闭合后。

对于旋转雷达产生的二维雷达图，TBV-SLAM (“trust but verify”，信任但验证) 流水线^[9]建立在第 9.2 节介绍的 CFEAR 二维雷达里程计方法之上。系统按顺序使用每次雷达扫描跟踪传感器位姿，但出于效率考虑，只将较稀疏的关键帧集合加入 SLAM 位姿图。当估计行驶距离超过某个阈值（例如 1.5 m）时，向位姿图添加新的关键帧，并根据其与最新关键帧的对齐创建里程计约束。与通常情况一样，图中的约束需要两个节点之间的相对位姿偏移，以及用协方差矩阵表示的相关不确定性。有研究表明^[9]，使用预先定义且元素较小的对角协方差矩阵，效果优于根据配准代价函数 Hessian 估计协方差。后者原本应能更好地捕获输入点云形状导致的不确定性，例如隧道中的一对点云应在隧道方向上具有更大的不确定性。然而，使用 Hessian 往往会低估或高估不确定性，可能使后端优化轻微错置关键帧。TBV-SLAM 流水线的核心是地点识别模块：它检索多个候选回环（“信任”），随后逐一测试并选出经过验证的最佳候选。Scan Context 描述子^[563]（见第 9.3.3 节）经过调整，同时考虑点密度和信号强度，以替代二维雷达中不可用的高度数据。此外，系统还实现了若干检索和验证回环候选的技术。对于每个关键帧，系统通过对点云施加横向平移偏移，创建多个增强描述子。搜索回环时，将查询描述子与数据库中的所有增强描述子匹配，以处理车辆在不同车道行驶时形成的回环。通过这种方式找到的候选回环随后依据里程计估计进行滤波，而这依赖于长距离内足够准确的里程计。传播查询帧和候选帧之间里程计约束的不确定性后，可以丢弃估计位置相距较远的候选回环。不过，同时考虑描述子相似度和里程计不确定性可以进一步提高回环检索的鲁棒性。也就是说，不是先寻找与查询描述子 q 最相似的候选 c ，满足 $c = \arg \min_c d_{\text{descriptor}}(q, c)$ ，再依据里程计滤波，而是通过

$$c = \arg \min_c [d_{\text{descriptor}}(q, c) + d_{\text{odometry}}(q, c)],$$

直接选择候选。其中， d_{odometry} 计算的是：在考虑累积里程计不确定性的情况下，帧 q 与 c 位于同一地点的可能性。按此方式找到的雷达扫描对先由 CFEAR 配准模块对齐，最后通过对齐验证模块进行验证；该模块包含重叠度量以及用于检测轻微错位的 CorAl^[7] 度量。总体而言，该流水线展示了如何将基于描述子的地点识别适配到雷达数据（考虑二维雷达图中的多重信号回波，并补偿相对稀疏且缓慢的扫描），以及如何筛选多个回环候选，根据几何对齐和前端位姿估计验证最佳候选。

Hong 等人^[474] 的 RadarSLAM 流水线是二维旋转雷达 SLAM 的另一个代表性例子。该流水线直接跟踪雷达图中检测到的关键点来实现里程计（开环位姿跟踪）。系统先用斑点检测器生成关键点，再用 Lucas-Kanade 跟踪器^[707] 在连续帧之间跟踪这些点。根据跟踪器计算的行驶距离，采用恒速度模型补偿扫描器

一次旋转期间的运动，并将变换后的关键点与关键帧位姿一起存储到因子图中。和前述方法一样，关键帧根据行驶距离选取。位姿跟踪使用稀疏关键点，而回环检测则从雷达图中提取更稠密的点云。点云提取方式与 TBV-SLAM^[9] 类似；但 RadarSLAM 不选择每个方位角上最强的 k 个点，而是选择强度高于均值一个标准差的所有点。随后从关键帧点云创建 M2DP 描述子^[445]，并通过匹配描述子相似的帧检测回环。为避免匹配缺乏描述性的点云，RadarSLAM 不从过于细长的关键帧中选择回环；也就是说，当 PCA 计算的两个特征值差异明显时，不选择该关键帧，因为这类点云很可能来自高速公路路段等不具唯一性的地点。系统不再执行其他回环验证。

Schuster 等人^[983] 给出了一个基于二维 SoC 雷达的 SLAM 流水线示例。与上述方法不同，该系统维护的图不仅包含传感器位姿，还包含单独的雷达特征节点；TBV-SLAM 和 RadarSLAM 只优化位姿图（尽管会将雷达点关联到位姿节点，以便地点识别和地图渲染）。由于二维 SoC 雷达通常比 360° 旋转雷达或 3+1D SoC 雷达提供少得多的检测结果，因此在可优化图中维护全部检测结果更为可行。图中添加边以表示所有同时观测到的特征之间的观测关系。系统使用二值环状统计描述子 (BASD^[908]) 提取路标。BASD 是紧凑的二值描述子，可以直接比较局部区域内所有特征点的描述子，从而将近期特征与地图中已有特征关联起来。通过 RANSAC 离群点剔除阶段的特征会与带有轮速编码器里程计信息的位姿节点一起作为顶点加入图中。假设开环跟踪的漂移适中，系统不包含显式地点识别步骤；但回环闭合后，可以用 BASD 描述子匹配点特征，随后在后端优化 SLAM 图。

近期两个基于 3+1D SoC 雷达的六自由度雷达 SLAM 方法是 4DRadarSLAM^[1271] 和 4D iRIOM^[1310]。二者都以三维雷达点云为输入，点检测（对数据立方体进行滤波）由传感器自身在板上完成，因此 SLAM 流水线不会显式包含 CFAR 或其他峰值检测步骤。不过，输入点云仍可能包含大量噪声点。两种方法都利用 3+1D 点云提供的多普勒径向速度信息过滤运动物体的点。车辆自身速度可通过对测得的多普勒点速度执行线性最小二乘估计；对于速度模型不一致的离群点则将其移除。iRIOM 进一步对点云去噪：仅保留满足以下条件的内点，即在固定半径内具有足够多的邻点，且这些邻点的空间分布较为紧凑（根据其空间分布的协方差矩阵判断）。多普勒自身速度估计被用作扫描到子地图配准的先验。4DRadarSLAM 使用一种称为 APDGICP 的 GICP 变体^[985]，通过协方差矩阵为点加权；由于方位角和俯仰角精度有限，距离传感器越远的点具有越高的不确定性。子地图（关键帧）被插入图中。系统通过匹配 Scan Context 检测回环；与上述二维方法不同，它可以使用包含点高度的原始 Scan Context。4D iRIOM 使用原始 Scan Context，而 4DRadarSLAM 使用 Intensity Scan Context^[1151]，以避免高度测量噪声导致描述子缺少信息。4DRadarSLAM 还包含验证步骤：若两帧之间累积的里程计距离超过某个阈值，则拒绝 Scan Context 返回的候选匹配。

9.4.3 雷达 SLAM 中的多模态

到目前为止，我们主要讨论只使用雷达数据（有时还使用 IMU 或轮速里程计等本体感知传感器）的雷达 SLAM 方法。本节讨论将毫米波雷达与其他外感知传感器（如 LiDAR、相机）或外部数据（如卫星图像或先验地图）组合的系统。

一些雷达建图工作采用雷达-LiDAR 融合，以根据当前可见性条件生成尽可能好的点集（在清晰条件下更信任 LiDAR，在低能见度条件下更信任雷达）。Fritsche 等人^[342, 343] 根据估计距离融合两种传感器，从而决定在特定情况下信任哪一种。雷达和 LiDAR 也被组合用于改善地点识别：通过将雷达配准到 LiDAR 地图，克服不同传感器模态之间的差异^[1247, 1248]。

Doer 和 Trommer^[273] 扩展了 ROVIO^[90]（一种基于滤波器的视觉-惯性里程计方法），使用第 9.2.1 节所述的多普勒里程计方法^[272] 集成雷达自身速度估计。类似地，也可以将热成像相机数据与雷达融合，构建多模态雷达-热成像估计流水线^[273]。Zhang 等人^[1272] 将热成像相机数据与 3+1D 雷达点云结合，在低能见度环境中实现鲁棒的帧间里程计。基于 Transformer 的特征匹配器在连续热成像帧中检测对应点，雷达点云则用于改善深度估计。

关于利用多模态数据进行基于雷达的导航，还应提及使用俯视图和道路地图的方法；不过，文献中的大多数工作专门将这类数据用于定位，而非完整 SLAM。Hong 等人^[475] 展示了如何利用二维扫描雷达数据在 OpenStreetMap 等先验公共地图中定位。其系统使用 RadarSLAM^[474] 运行里程计，并将传感器估计位姿表示为高斯混合模型。从 OpenStreetMap 提取代表建筑物墙体的线段作为先验地图特征，再从雷达图提取的定向点与先验特征匹配。然而，先验地图信息可能不确定、不完整或过时，因此点到特征的数据关联

具有挑战性。系统从当前位姿估计的高斯混合模型中采样多个位姿，并在每个位姿下将当前雷达扫描的定向点与先验地图的线特征匹配，从而将扫描定位到先验地图。另一种利用先验地图数据（此处为卫星图像）定位二维雷达图的方法是 RSL-Net^[1069]，它由一组深度神经网络组成。第一个网络从输入的俯视照片生成合成图像，展示该地点可能产生的雷达图；另一个网络利用前一网络生成的合成雷达图，估计真实雷达图与俯视图象之间的相对旋转；最后一个网络估计雷达图与俯视图象之间的相对平移偏移。

一些系统使用所谓的超宽带（ultra-wideband, UWB）无线电感知。虽然 UWB 同样使用无线电频谱内的电磁波，有时也被称为 UWB 雷达，但 UWB 的测距能力与毫米波雷达大不相同。在雷达 SLAM 框架中，UWB 雷达主要用于检测不同地点传感器读数之间的相似性。发送宽带、宽波束信号后得到的频率响应可以提供当前位置的特征。度量信息则由轮速编码器或 IMU 数据提供。Schouten 和 Steckel^[979] 以及 Takeuchi 等人^[1065] 使用 UWB 波形特征数据库检测重访地点：对于每个地点（节点），存储雷达回波的特征及估计位姿，然后利用里程计和回环约束创建并优化图。两种方法都依靠里程计获得节点之间的距离（即图中的边）来实现度量 SLAM，特征则由脉冲回波 UWB 传感器生成。相反，Premachandra 等人^[886] 使用 UWB 雷达检测点特征，将其用于基于路标的 SLAM 框架。他们在机器人两侧分别安装多个雷达模块，并对两侧传感器信号中匹配峰值进行三边测量，以检测路标。此外，一些基于 UWB 的定位方法采用锚点-标签传感器配置：锚点固定在已知位置，电池供电的 UWB 标签安装在待定位机器人或资产上。但由于这种方法需要预先安装基础设施，它与 SLAM 并不直接相关。

9.5 雷达数据集

本节简要总结雷达 SLAM 文献中具有代表性的数据集，数据集也列于表 9.1。

旋转雷达。表 9.1 中使用旋转雷达的数据集均采用 Navtech 二维雷达传感器。Oxford Radar RobotCar 数据集^[54] 和 MulRan^[567] 是最早用于里程计和 SLAM 的两个大规模雷达数据集，均已被文献广泛使用。Oxford 数据集覆盖城市驾驶路线的多次行驶，总里程达 280 km，包含多种天气条件。MulRan 数据集覆盖更多样的环境，包括密集城市和较偏远的道路行驶，并且不同会话之间的时间跨度更长，但总行驶里程较少。MulRan 主要用于促进地点识别研究，但也广泛用于里程计和 SLAM 方法基准测试。Boreas 数据集^[128] 包含一年内反复行驶同一路线的数据（总计 385 km），尤其包括雪、雨等恶劣天气。除 SLAM 相关基准外，该数据集还包含目标检测基准（汽车、行人和骑行者）。与表 9.1 中其他数据集不同，Oxford Offroad Radar Dataset^[353] 关注非城市驾驶，覆盖约 154 km 的非铺装道路和无人居住区域的山间小道。

SoC 雷达。目前也有若干 SoC 雷达数据集，同时提供 2+1D 和 3+1D 数据。其中一些面向自动驾驶的数据集主要关注目标检测，但也被用于开发和测试 SLAM 方法。nuScenes^[137] 将采集车辆上安装五个 Continental ARS408-21 雷达与一个 LiDAR 和六个相机结合，重点覆盖四座城市的城市驾驶，并且为 23 个目标类别提供了目标检测标注。RadarScenes^[982] 是一个包含四个 77 GHz 汽车 2+1D 雷达（未命名）和一个相机的数据集，侧重语义感知，含有 11 类目标的标注，但缺乏准确真值以及 IMU 和 LiDAR 数据。ColoRadar^[607] 是一个雷达 SLAM 数据集，包含 3+1D Texas Instruments MMWCAS-RF-EVM 开发板和 2+1D Texas Instruments 模块的数据，以及 IMU 和三维 LiDAR 数据。值得注意的是，该数据集除雷达传感器的原始模数转换器（analog-to-digital converter, ADC）数值、三维“热图”（数据立方体）和单个目标点外，还覆盖室内、室外及地下矿井环境，并包含用于位姿估计的六自由度（6-DoF）真值跟踪。NTU4DRadLM^[1273] 和 MSC-RAD4R^[212] 均包含 Oculii Eagle 传感器的高分辨率 3+1D 雷达数据。NTU4DRadLM 覆盖结构化（大学校园）和非结构化（公园）环境；MSC-RAD4R 覆盖城市和乡村道路驾驶。Snail-Radar 数据集^[482] 具有两个高分辨率 3+1D 雷达，即 Oculii Eagle 和 Continental ARS548，并包含手持采集和城市道路驾驶数据。

数据集	LiDAR	相机	真值	环境	恶劣天气
Oxford Radar RobotCar ^[54]	有	双目/单目	GPS/IMU VO	+ 密集城市	雨
Boreas ^[128]	有	单目	GPS/IMU RTK	+ 稀疏城市	雨、雪、雾
MuRan ^[567]	有	无	SLAM	混合城市	—
RADIATE ^[1001]	有	双目	GPS/IMU	混合城市	雨、雪
OORD ^[353]	有	单目	GPS	城市与越野	雪、夜间
nuScenes ^[137]	有	双目	GPS/IMU	混合城市与自然 环境	雨
RadarScenes ^[982]	无	单目	无	混合城市道路	雨、雾
ColoRadar ^[607]	有	无	SLAM	多种环境	—
NTU4DRadLM ^[1273]	有	单目	SLAM	混合城市	—
MSC-RAD4R ^[212]	有	双目	GPS + RTK	城市与乡村道路	雪、夜间
Snail ^[482]	有	双目	TLS	道路与隧道	雨、夜间
K-Radar ^[836]	有	双目	GPS/IMU RTK	+ 道路	雨、雪、夜间、雾
TruckScene ^[324]	有	双目	GPS/IMU RTK	+ 道路	雨、雪、夜间、雾
HeRCULES ^[569]	有	双目	GPS/IMU RTK	+ 混合城市与自然 环境	雨、雪、夜间

表 9.1 公开雷达相关数据集概览。“真值”列中，VO 表示视觉里程计，TLS 表示测量级地面激光扫描，RTK 表示带实时动态校正的 GPS。

9.6 延伸阅读与近期趋势

目前能够处理三维数据的雷达 SLAM 流水线仍然很少，但随着高分辨率 SoC 传感器的发展，可以预期出现更多基于雷达的全三维里程计和地点识别研究。例如，对无人机而言，这一点尤其重要，因为雷达目前在无人机上的应用仍较少。要实现三维大视场雷达单元，需要具有创造性的设计。同时，SLAM 对多个小视场雷达组成的传感器阵列的处理能力也很可能取得显著进展；这自然会伴随标定及其他问题。

研究重点还可能转向更充分地利用包含频谱数据的原始雷达图和数据立方体。传统雷达 SLAM 系统通常通过生成点云来解释环境。然而，未来系统预计会更充分地利用雷达数据中丰富的频谱信息，提供更细致、更有层次的地图，并改进目标检测和分类。其中一个挑战是说服制造商开放其产品原始输出的访问权限，供研究使用。

雷达语义分割也可能在地点识别中发挥更重要的作用。借助分割技术，SLAM 系统能够更有效地区分环境中的不同目标类型，从而实现更智能的导航和决策。这还有助于减少雷达回波中的歧义，使建图更加可靠。

最后，多模态数据方法将受到更多关注。这类方法把雷达数据与其他信息源（包括其他传感器和地理先验，例如 OpenStreetMap）结合起来。例如，将雷达观测与这些先验结合，雷达 SLAM 系统可能能够在大规模室外环境中更准确地定位，进一步提高自主系统的鲁棒性和可靠性。

¹ 尽管 TX 和 RX 天线有时可以是同一个单元，但多数 SoC 传感器选择将二者分开，从而能够集成多个 TX 和 RX 天线。² 这些滤波技术可以视为与第 9.2.3 节中的关键点提取类似，但不提取描述子。³ 更全面的概述可参见文献^[121]。

第十章 基于事件的 SLAM

作者：Guillermo Gallego、Javier Hidalgo-Carri6 和 Davide Scaramuzza

读者不难注意到，在需要解释空间关系并与环境交互的现实应用中，SLAM 至关重要。SLAM 所使用的主要传感器决定了系统的成功与适应能力。视觉 SLAM 是最普及的 SLAM 方法类别之一，因为相机易于获得、价格低廉，并能产生直观且信息丰富的信号，使机器人能够在广泛场景中感知世界（例如构建无需 GNSS 等额外基础设施的轻量系统）。尽管相关技术已经取得进展，当前最先进的人工视觉系统在现实任务中的鲁棒性和效率仍不及生物视觉系统。标准相机以与场景动态无关的固定帧率感知世界，因此在两帧之间的时间段内处于“失明”状态，引入延迟，可能丢失跟踪；当场景中没有运动时，还会产生大量冗余数据。本章探索一个具有前瞻性的挑战：借助受生物启发的硅视网膜或“事件相机”技术，理解并构建快速（不受帧率限制）、低功耗且能适应宽范围照明条件的视觉 SLAM 系统。事件相机克服了标准相机的若干局限（见图 10.1）。

我们首先介绍事件相机的工作原理（第 10.1 节）以及相应的挑战与应用（第 10.2 节），随后聚焦于事件相机数据处理方法（第 10.3 节）以及相应的前端（第 10.4 节）和后端处理（第 10.5 节）。最后讨论最先进的系统（第 10.6 节）、数据集、模拟器与基准（第 10.7 节），并在第 10.8 节以新趋势和延伸阅读作结。

10.1 传感器描述

10.1.1 工作原理

与由外部时钟（例如 30 Hz）决定采集整幅图像速率的传统相机不同，事件相机（例如 Dynamic Vision Sensor, DVS）

的像素彼此独立工作，异步地响应场景中发生的亮度变化（图 10.2b）。逐像素变化由场景照明（例如闪烁的灯光）和/或相机与场景（包括运动物体）之间的相对运动引起。因此，事件相机的输出是一系列数字“事件”（或“脉冲”），每个事件表示亮度（对数强度）的变化。这种编码受生物视觉通路的脉冲特性启发（图 10.2a）。

具体而言，每个像素在发送事件时记忆对数强度 L ，并持续监测相对于该记忆值、幅度足够大的变化 ΔL （图 10.2）。当变化达到阈值 C 时，

$$\Delta L \doteq L(\mathbf{x}_k, t_k) - L(\mathbf{x}_k, t_k - \Delta t_k) = p_k C, \quad (10.1)$$

相机便发送事件 $e_k \doteq (\mathbf{x}_k, t_k, p_k)$ 。该事件从芯片传出，其中包含 x, y 像素位置 x_k 、时间 t_k 以及变化的 1 位极性 $p_k \in \{+1, -1\}$ （即亮度增加或减小）。 Δt_k 是同一像素上前一事件以来经过的时间。

事件相机是数据驱动传感器：其输出取决于场景中的运动量或照明变化量。运动越快，每秒产生的事件越多，因为每个像素会根据所监测强度信号的变化速率自适应其采样率。

生物启发：瞬态通路。事件相机的设计受到生物视觉通路工作方式的启发；视觉通路是动物和人类处理信息的路径。根据双流假说，背侧通路（也称“瞬态”或“where”通路）专门处理动态视觉信息（例如场景中的运动），而腹侧通路（称为“持续”或“what”通路）专门负责物体和视觉目标的识别。DVS^[661] 对应从光感受器到神经节细胞的瞬态通路部分，采用简化的三层像素设计，在生物逼真度与电路稳定性之间取得平衡（图 10.2）。三层分别实现光转换、增量调制和比较功能。Asynchronous time-based image sensor (ATIS)^[883] 或 Dynamic and Active-Pixel Vision Sensor (DAVIS)^[111] 等相机同时建模两条视觉通路，因此输出两类信号：DVS 事件和灰度信息（例如图像）。关于主要事件相机类型的更多细节见^[884, 361]。

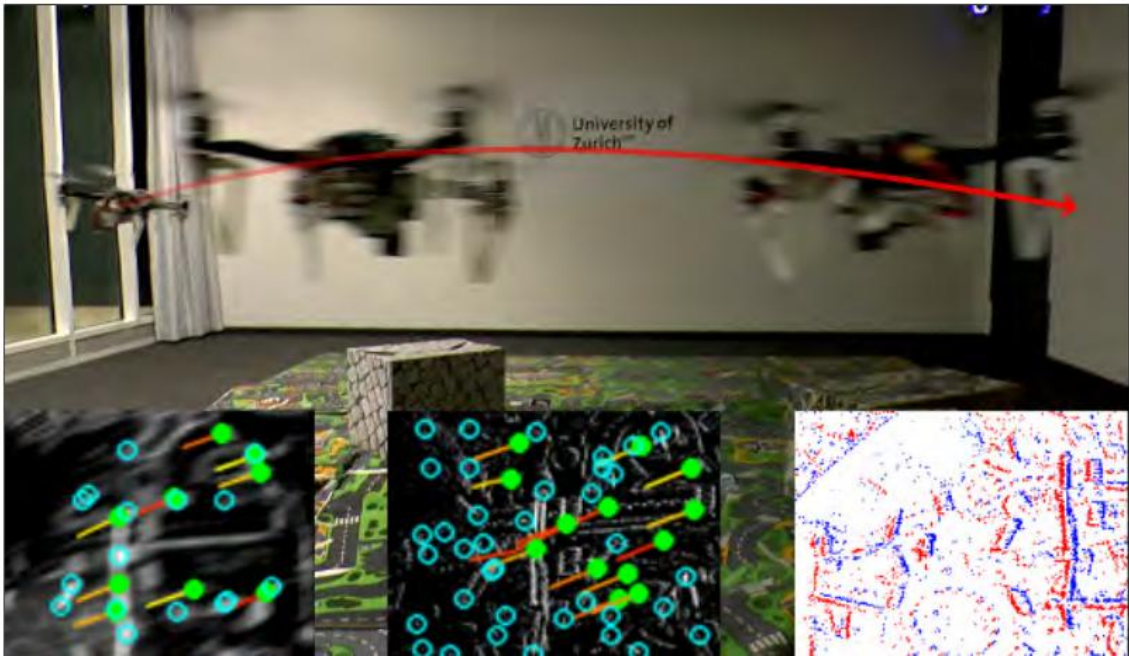


图 10.1: 图 10.1 配备下视 DAVIS 相机^[111] (240×180 像素) 的无人机, 使用视觉惯性里程计 (VIO) 算法^[944] 进行状态估计并完成自主飞行。事件相机数据的高速和高动态范围特性使系统能够在困难照明条件下运行。插图展示了在灰度帧 (左, 存在运动模糊) 和事件扭曲后经运动补偿的图像 (中, 清晰) 中检测并跟踪的特征 (即关键点)。事件数据根据极性以红/蓝色显示, 清楚地响应场景轮廓。同一 VIO 算法^[944] 还在高速场景中得到演示, 例如将事件相机系在绳子上旋转。图像来自^[361] (©2020 IEEE)。

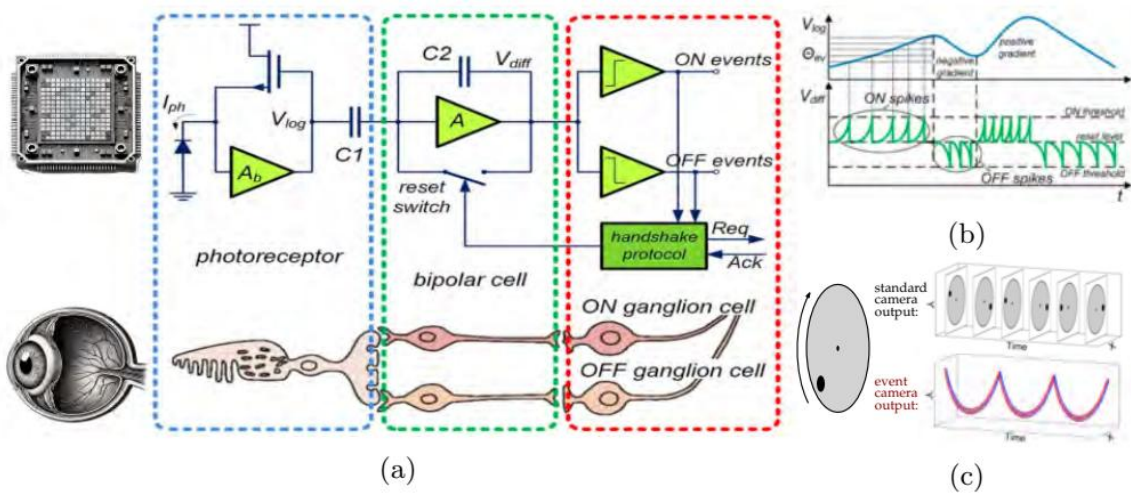


图 10.2: 图 10.2 事件相机 (例如 DVS) 的工作原理: (a) 人类视网膜三层模型及相应的 DVS 像素电路; (b) DVS 像素的工作示意图, 将光转换为事件 (脉冲), 信号颜色与 (a) 中各层相匹配; (c) 标准相机和事件相机对旋转圆盘上的黑点这一视觉刺激的响应对比。事件相机连续传输亮度变化, 在时空中形成事件螺旋。红色表示正事件 (ON 脉冲), 蓝色表示负事件 (OFF 脉冲)。图像改编自^[884] (©2014 IEEE)。

10.1.2 事件相机的优势

事件相机的感知原理与过去七十年主导计算机视觉和机器人视觉的标准（基于曝光）相机截然不同，并带来诸多优势：

高时间分辨率：事件以微秒级分辨率检测并加盖时间戳，因此能够捕获极快运动，而不会受到帧式相机常见的运动模糊影响。事件几乎连续地产生，从而避免标准相机中可能造成帧间大位移并破坏数据关联的盲时间。

低延迟：每个像素独立工作，无需等待全局曝光时间；一旦检测到亮度变化，事件便立即传输，延迟低于 1 ms。

低功耗和低带宽：事件表示非冗余的时间数据，功耗仅用于有用信息。与以相同速率运行的传统相机相比，带宽也得到降低。在芯片层面，相机功耗低于 10 mW，使嵌入式系统的功耗可低至 100 mW 或更低^[31]。

高动态范围 (HDR)：事件相机可感知的光值范围非常大（通常超过 120 dB，而标准相机约为 60 dB），能够同时感知极暗（月光）和极亮（日光）区域。因此不会出现帧式相机常见的欠曝光或过曝光。这一特性源于两个事实：每个像素独立工作，并以对数尺度将光转换为电压。

10.1.3 当前器件与趋势

“我应该购买或使用哪种事件相机来解决 SLAM 问题？”刚进入这一新兴领域的人经常提出这个问题。事件相机的特性通常通过表格进行比较 [361, 表 1]、[162, 表 1-2]。虽然存在多种事件相机设计，但大多数仍是实验室原型；只有少数实现了商业化，从而支持针对经典问题和新问题（如基于事件的 SLAM）探索新方案。在主要制造商（SONY、Samsung、iniVation / SynSense、Prophesee、Omnivision）商业化的器件中，有几项趋势值得关注：

像素尺寸：随着传统相机的百万像素竞赛和工业需求压力，像素间距（即尺寸）已从 40 μm (DVS128^[661]) 显著减小到小于 5 μm ^[326]。DVS 像素比传统像素执行更多操作（调制、比较等），因而需要更多晶体管，难以在相同传感器面积内封装。为最大化像素受光面积（即填充因子）并缩小像素感光部分之间的间隙，已经采用堆叠技术和背照式技术^[326]。

灰度输出：DAVIS 或 ATIS 等早期器件同时输出灰度数据（例如图像^[111]），这对固定相机应用尤其有用（尽管这并非 SLAM 的通常场景）。受百万像素竞赛驱动，HD 事件相机^[326]等新型号停止灰度输出，以便为事件输出留出更多面积。

颜色对许多运动相关任务并非必需，因此只有少数事件相机型号提供彩色滤光片，用于检测各颜色通道（红、绿、蓝，即 RGB）的变化^[770]。

惯性数据：一些相机还提供集成在同一器件中的惯性测量单元（IMU）数据。IMU 是相机有价值的互补本体传感器，可实现视觉惯性里程计（VIO）和 SLAM，并比单传感器系统具有更高的鲁棒性和精度。

不能想当然地认为高空间分辨率事件相机必然优于低分辨率相机。虽然捕获精细空间细节很重要，但噪声和带宽同样影响目标应用的需求。在 SLAM 及相关任务中，事件相机可能快速运动和/或经过纹理丰富的场景；HD（1 百万像素）事件相机每秒可产生数亿个事件。这会导致相机输出传输总线饱和以及高处理需求；目前不存在能实时处理如此事件速率的算法与硬件组合（除非转为阵列表示和/或进行子采样/降采样）。人们正在开发新的混合传感器，例如^[1238]（事件空间分辨率低于强度输出），以及模仿生物视觉以降低带宽的中央凹传感器^[317]，它们可能为上述问题提供替代方案。在 SLAM 中，低像素分辨率（例如 QVGA）更适合算法原型开发和计算受限机器人的实时运行。通常，视场（光学系统）的选择与像素数量同样重要。

10.2 挑战与应用

事件相机代表了视觉数据采集领域的革命性技术。因此，需要设计新的方法（算法和硬件）处理所采集的数据并提取有价值的信息，以发挥传感器优势。主要挑战包括：

处理时空输出：事件相机的输出与标准相机根本不同：事件是异步且空间稀疏的，而图像是同步且稠密的。因此，为图像序列设计的视觉 SLAM 算法不能直接应用于事件数据。

处理运动相关数据：与图像不同，每个事件仅包含二值（增加/减小）亮度变化信息，该信息不仅取决于场景纹理，还取决于场景与相机之间的相对运动。

处理噪声和动态效应：事件相机存在噪声，来源包括固有的光子散粒噪声、晶体管电路噪声、对入射光量的依赖、非理想因素以及低功耗（亚阈值）运行。

这些挑战要求重新思考事件数据的时空、光度和随机特性。在 SLAM 中，这引出了如下问题：从事件中提取位姿或深度信息的最佳方式是什么？应采用何种地图和相机轨迹表示，以考虑事件数据近似连续的时间粒度和稀疏性？如何在运动相关数据下建立对应关系（数据关联）？如何建模问题（及其解），而不引入帧式技术的典型瓶颈？

上述问题推动了基于事件的 SLAM 研究（图 10.3）。这一主题既独立发展，也与其他任务结合；例如，基于事件的 SLAM 促进了其他事件视觉任务的研究。早期工作^[229, 570]（旋转运动 SLAM）和^[571]（6 自由度 SLAM）已经利用 SLAM 与图像重建（从事件恢复绝对强度）之间的协同关系。基于事件的 SLAM 与光流估计也在^[229, 1243, 1010, 585]中联合处理。

10.3 基于事件的 SLAM 方法概述与分类

根据同时处理的事件数量，基于事件的 SLAM 方法大致分为两类：(i) 逐事件运行的方法，系统状态（例如场景地图和相机轨迹）可在单个事件到达时改变，从而实现最低延迟；(ii) 对事件组/批次/切片/数据包运行的方法，会引入一定延迟。后一类的关键设计选择是数据包大小；已有许多方案，例如固定事件数量、固定时间长度和混合判据。

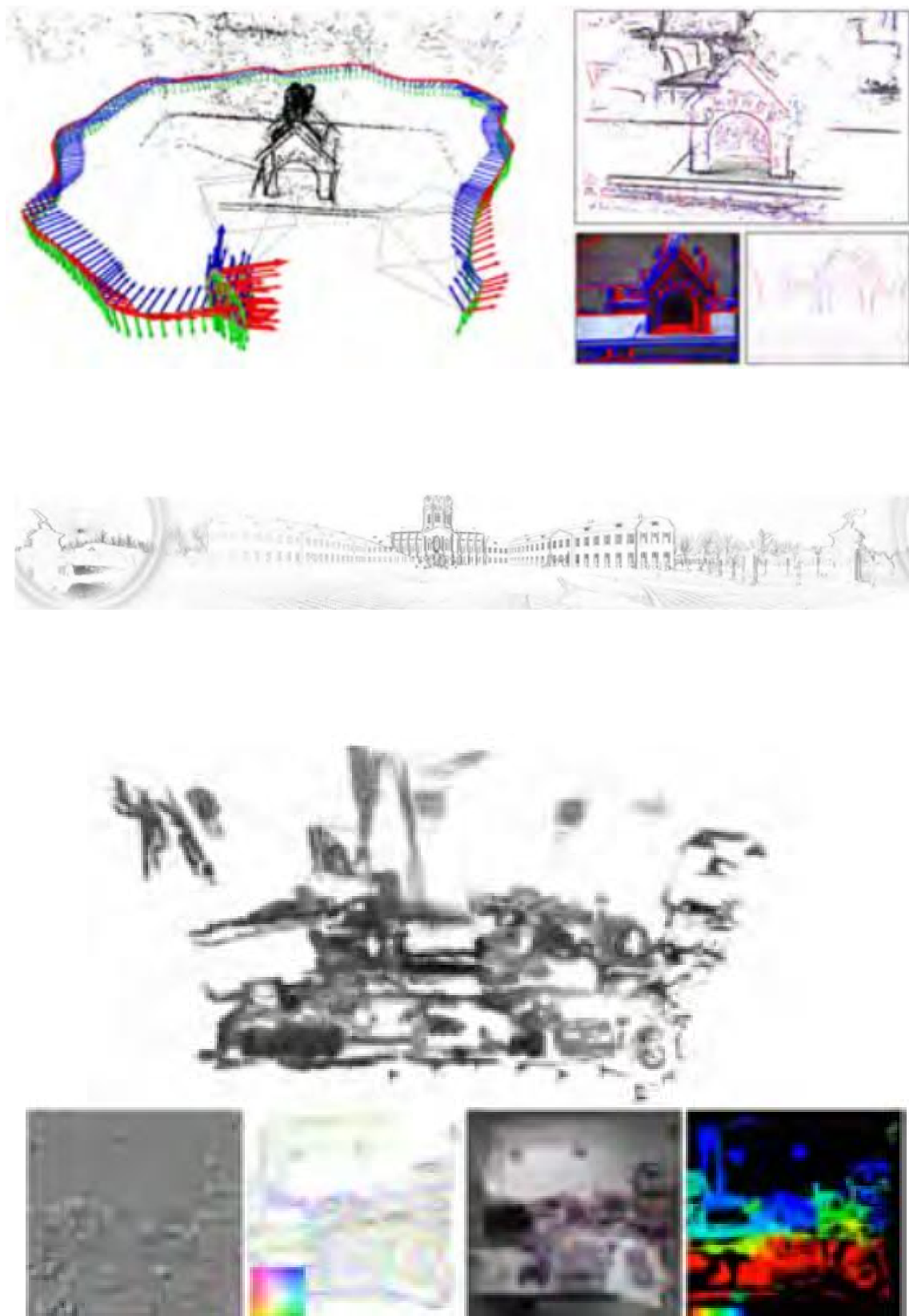
正交地，根据事件的处理方式，还可区分基于模型的方法和数据驱动（即机器学习）方法。仿照帧式 SLAM 的分类，基于事件的 SLAM 可分为间接法（基于特征，使用事件角点、直线、法向流等）和直接法（使用全部事件）。这一分类与所用目标函数或损失函数的类型有关：几何型与光度型（例如事件极性或事件速率/活动度的函数）；也与总体思想有关：间接法通常包含两个步骤（特征提取，将事件“转换”为几何基元；随后运行几何 SLAM 流水线），直接法通常只有一步，将事件数据映射为运动和场景参数。对于直接法，事件生成模型（10.1）（或其线性化版本^[361]）是设计估计方法的基石。事件间数据关联是事件视觉、尤其是 SLAM 的核心问题。由于事件相机具有高时间分辨率，数据关联通常通过时间和空间邻域处理；硬关联和软关联策略均已得到研究。



EVO：实时事件 6 自由度并行跟踪与建图的几何方法

EDS：事件辅助直接稀疏里程计

CMax-SLAM：使用对比度最大化的基于事件的旋转运动束调整与 SLAM



事件相机实时三维重建与 6 自由度跟踪 (ETAM)



DEVO: 深度事件视觉里程计

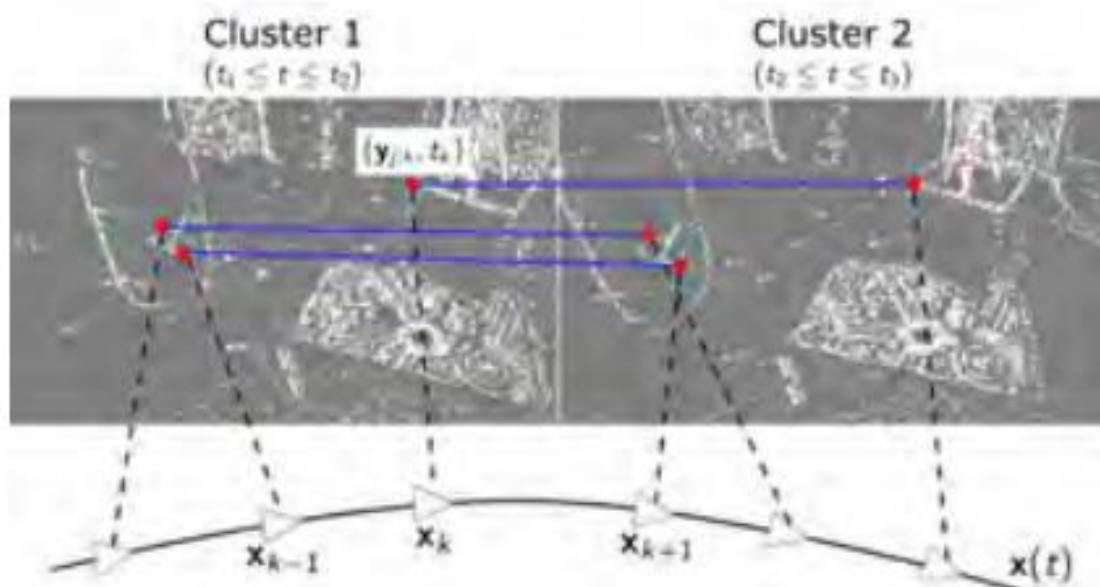
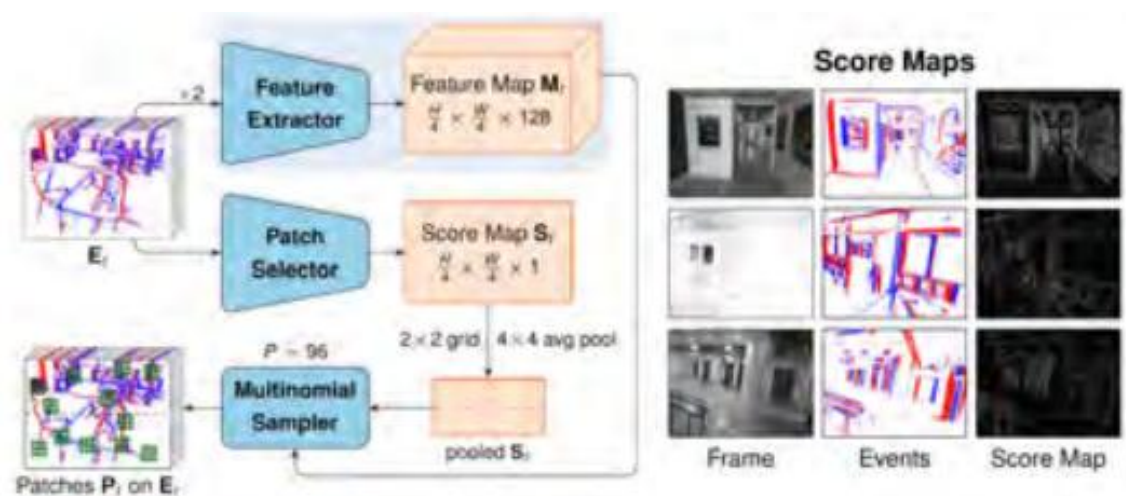
ESVO2: 使用双目事件相机的直接视觉惯性里程计

基于高斯过程连续时间方法的事件双目视觉里程计 (GPCT) 图 10.3 基于事件的 SLAM 正在积极研究中, 系统涵盖多种方法, 包括经典方法和较新的深度学习方案。由于事件由图像平面中运动的边缘触发, 因此以边缘形式 (例如稀疏或半稠密三维地图) 恢复场景地图是自然选择。图像改编自 EVO^[914] (©2017 IEEE)、EDS^[461] (©2022 IEEE)、CMax-SLAM^[416] (©2024 IEEE)、Kim 等^[571] (©2016 Springer)、ESVO2^[812] (©2025 IEEE)、DEVO^[585] (©2024 IEEE) 和 Wang 等^[1154] (©2023 IEEE)。

上述各类别都有优点和缺点。使用事件相机求解 SLAM 很有挑战性, 历史上通常沿多个维度逐步增加复杂度: 未知量数量 (自由度, 即 DoF)、运动类型 (从旋转或二维场景到 6 自由度运动)、场景复杂度 (纹理) 及其运动状态 (静态与动态, 即独立运动物体, IMO)。基于事件的 SLAM 并非孤立问题: 如第 10.2 节所述, 它与光流、跟踪、分割等问题存在联系, 联系强弱取决于所作假设或具体场景。除上述趋势外, 还可观察到早期研究主要采用基于模型的方法, 而近期论文开始探索深度学习方法的可能性。

10.4 基于事件的 SLAM 系统前端

基于事件的 SLAM 系统通常由多个模块组成, 分别处理特征提取、数据关联、引导初始化、位姿估计、深度估计等较小子问题。系统的一项基本划分是前端与后端。从输入输出角度看, 前端接收原始传感器数据 (以及相机标定等可能的辅助信息), 输出一组事件相机位姿和场景地图 (见图 10.4)。后端细化这些变量 (即 SLAM 问题的未知量), 改善它们与传感器数据之间的拟合。后端在前端之后运行, 速率较低 (取决于变量数量), 并可将其输出反馈给前端, 以减少漂移和校正错误。



因此，前端将传感器信息（例如光子）转换为几何基元（例如相机位姿），同时也提取光度信息（例如地图外观）。这通常包含“特征”或“信息”提取步骤。首要挑战是理解事件流包含的信息，并使用保留数据特性（低延迟、稀疏性、HDR 等）的方法提取这些信息。在照明恒定的假设下，事件由运动轮廓（边缘）产生。因此，可以把运动中的事件相机视为异步边缘检测器，并以场景轮廓的形式表述 SLAM 问题（图 10.3）。这在先验上是合理的，因为轮廓是图像平面中信息量最大的区域，可据此估计视网膜运动，进而推断三维信息。每个事件只包含一个四元组并带有噪声，携带的信息很少；因此需要大量事件（例如数千或数百万个）才能可靠估计相机位姿和场景地图。事件流中的信息提取取决于任务和诸多设计选择，例如 SLAM 变量（场景地图、相机轨迹）的表示形式、用于处理数据的硬件以及输出速率等。

基于事件的 SLAM

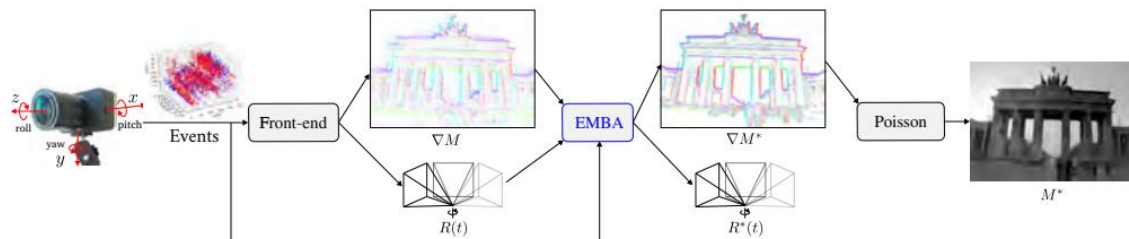


图 10.3: 图 10.4 基于事件的 SLAM 流水线：前端计算地图和相机位姿，后端细化地图与位姿。由于事件响应运动边缘，恢复的地图通常是边缘（梯度）地图。示例展示了直接旋转 SLAM 流水线（位姿仅由旋转组成，地图简化为全景地图）^[417]。还可通过泊松积分恢复绝对强度地图。

10.4.1 预处理与事件表示

在 SLAM 问题中，事件相机在穿过场景时连续输出数据。从相机视角看，所有场景边缘似乎都在运动，因此事件会在图像平面“各处”触发。由于事件稀疏且具有微秒级时间分辨率，每个事件都对应不同的相机位姿。这与传统（帧式）相机截然不同：一幅图像中的所有像素测量具有相同时间戳，因而共享同一相机位姿（传统多视图几何^[437]正是建立在这一范式上）。许多 SLAM 方法将事件数据转换为其他表示（事件图像、时间图或“时间表面”、体素网格等）^[361]，原因包括兼容传统计算机视觉方法和便于解释等。这一步通常意味着信息量化（例如将时间戳相近的事件分组）和/或稀疏性损失（例如在无事件的位置用零填充数组）。

因此，事件表示研究^[361, 370]受到越来越多关注。它通常是前端的第一阶段，并强烈影响后续处理：事件被转换为更熟悉、易处理的表示（例如图像），从而可以输入为传统图像设计的成熟 SLAM 方法，或设计基于图像的学习方法。之所以进行这种转换，部分原因是研究界仍在探索从事件流提取信息的最佳方式，并试图复用成熟的图像方法。前端可以使用不同事件表示；例如 EVO^[914]在建图模块（EMVS^[916]）中使用原始事件，在相机跟踪模块中使用事件（类似边缘）图像。理想情况下，应设计使用事件表示的 SLAM 方法，保留事件相机的高速和稀疏特性，避免传统相机的量化时间、延迟和非稀疏性问题。实践中，这是一个需要异步地重新思考视觉处理的新兴研究主题，仍有大量基础问题有待改进和研究。

10.4.2 间接法

前端的设计选择会很大程度影响系统的其余部分。一项主要选择是采用间接法还是直接法。间接法大致包含两步：首先从事件中提取并跟踪基于点、基于直线或其他类型的特征；然后利用经典 SLAM 的结果，根据这些几何基元估计相机运动和三维场景结构。特征将事件数据压缩为少量信息丰富的基元，使计算资源得到更有效和高效的利用。核心问题是建立并维护事件特征（以及地图地标）之间的对应关系，即数据关联。由于每个事件携带的信息很少且与运动有关，无法仅凭单个事件无歧义地确定关联。得益于事件相机的高时间分辨率，可在像素空间中依据时空邻近性建立关联。因此，跟踪特征比匹配特征更自然。

给定场景地图，相机位姿估计或相机跟踪通常表述为特征配准/对齐问题，通过最小化几何目标（例如以像素空间欧氏距离度量的重投影误差）求解。场景三维结构通常通过三角化（即反投影）对应特征位置得到，例如使用给定的相机位姿获得三维点和直线。可以利用成熟的大量几何方法（多视图几何^[437]）。

与常规视觉 SLAM 类似，间接事件法高度依赖鲁棒的特征提取和跟踪。然而，由于要处理大噪声、稀疏性、异步性和运动依赖等独特挑战，这些组件尚不如帧式方法成熟，限制了系统精度和适用性。为应对这些问题，一些系统采用传感器融合（灰度图像和/或 IMU 数据）。

10.4.3 直接法

直接法使用所有可用数据（而非仅使用符合特征定义的事件数据）估计相机运动和三维场景结构。它不显式提取特征，而是直接将事件数据与地图、图像或其他事件对齐。当事件速率相对于系统处理能力过高时，会采用去噪或子采样等数据削减机制，以减少需要处理的事件数量^[416, 572]。

由于直接法只有一个步骤，运动（相机跟踪）或场景参数（建图）通过优化某个目标函数（例如光度误差或空间事件速率误差）获得。基于光度的目标会诱导几何配准目标；问题未知量由事件传达的类边缘亮度模式与对应图像或地图像素的对齐确定。直接法依赖事件数据的近似连续性，例如可以从上一次估计的位姿计算增量相机位姿：事件连续触发，没有间隙或盲时间，因此增量很小。

直接法中，一个因其最先进的精度而受到重视的子类，是通过事件对齐估计运动或场景参数的方法，其结果表现为清晰的事件扭曲图像（IWE）。其思想是通过“抵消运动”来估计运动，即寻找能够对事件数据进行运动补偿的参数。事件对齐可用方差、梯度幅值、离散度等不同目标度量；这类方法也称聚焦最大化或对比度最大化（CMax）^[360, 1010]。在事件可被扭曲到少数像素或一条直线的问题中，这些方法可能出现不希望的全局最优解^[1009]。直接法中的数据关联通常由像素网格距离隐式、软性地处理；不过，在某些情况下，使用最近邻值的硬关联同样可行且有效。

10.4.4 基于模型与基于学习的方法

迄今为止，大多数基于事件的 SLAM 方法都是人工设计的，依据事件相机和 SLAM 问题的工作原理及人类直觉构建。相比之下，深度学习方法利用人工神经网络（ANN）对事件数据建模：可以先将事件转换为类似图像的表示，也可以直接用脉冲神经网络（SNN）处理事件。根据监督信号类型，这些方法通常分为有监督和自我监督方法。自我监督方法利用事件或其他传感器（例如共址灰度相机）中的时间动态或光度一致性损失，估计深度和相机位姿^[1243]；有监督方法需要地面真值数据训练^[585]，而在现实世界中获取此类数据通常很困难。近年来，人们已在汽车、无人机等平台上采集许多多模态数据集，为 ANN 训练提供了所需数据。

基于学习的方案可以替代 SLAM 流水线的一部分，例如特征提取与跟踪^[760]，也可以尝试替代整个系统（端到端）。这类方法能够隐式处理复杂数据表示和噪声，但需要大量训练数据；当应用于与训练集显著不同的场景（即发生“域偏移”）时，还可能出现泛化问题。

10.5 基于事件的 SLAM 系统后端

SLAM 后端等细化模块^[136]的目标是改善 SLAM 问题变量与传感器数据之间的一致性，从而提高拟合的精度和鲁棒性，减少系统跟踪与建图模块之间的误差传播。如前面章节所述，束调整（BA）通常用于视觉 SLAM 后端。

基于事件的 BA 仍处于起步阶段，因为多数基于事件的 SLAM 系统缺少细化步骤。相反，它们以并行跟踪和建图方式运行^[571, 914, 1302]，每个模块同时依赖另一个模块的输出作为输入才能正常工作。当前事件 SLAM 系统优先考虑简洁性以及事件相机的低延迟优势，而非精度和鲁棒性。除第 10.2 节所述挑战（噪声、运动相关外观等）外，事件后端还需要联合估计相互关联的变量，这会产生高维搜索空间，使优化在复杂度和延迟方面代价高昂，并容易陷入局部极小值。

直到最近，BA 才开始用于包含事件相机的系统。由于 SLAM 后端的设计高度取决于前端输出（要获得最佳性能，两个模块必须紧密集成），因此我们将事件后端分为间接后端（基于特征）和直接后端（基于光度）。

间接后端继承自经典的间接帧式方法^[1107, 645, 787]。它们在事件流中检测到的几何基元（角点、直线等）上运行，也可能先将事件转换为图像^[204, 944]，以复用帧式检测器。其目标通常是最小化重投影误差，即图像平面中的欧氏距离^[437]。这种方法的优点是能够复用经典 SLAM 中成熟且鲁棒的技术；但它丢弃了事件中包含的大量信息（图像重建方法^[918, 1280]已揭示这些信息），且往往难以达到期望性能：受噪声和事件运动依赖性的影响，当前事件角点不如帧式角点精确稳定，因此在 SLAM 中的应用较少^[611]。间接后端的例子包括^[944, 204, 1154]。

直接后端处理传感器数据（而非几何基元），目标通常是最小化某种形式的光度误差，因此比间接后端更适配事件数据。^[461]等利用共址帧的灰度信息，借用帧式系统的后端^[308, 26]。然而，灰度帧可能出现运动模糊且动态范围较低。仅事件后端不受这些限制；由于发展较晚，目前主要针对受限运动（平面或旋转）。其目标可以是最大化事件对齐（也称运动补偿或 CMax）^[359, 416]，或最小化每个事件传达的光度误差（即时间对比度）^[417, 418]。这些方法依据事件生成模型（10.1）设计。由于单个事件携带的信息很少，而 SLAM 中问题未知量通常很多，精确 BA 需要大量事件，对计算资源、功耗和延迟提出要求。在自然场景和 6 自由度运动下，研究高效的直接、纯事件 BA 仍有很大空间。

10.6 最先进的系统

表 10.1 汇总了基于事件的 VO/SLAM 具体系统，并按照前文分类描述其特性（直接、间接等）。本章不可能也无意详细介绍所有系统，但有几项趋势值得关注。

文献仍由基于模型的系统主导，数据驱动方法尚未取而代之（尽管未来可能发生这种转变，正如其他计算机视觉任务一样）。从研究伊始，事件相机 SLAM 就在不同假设下处理问题，并逐步提高以下方面的复杂度：(i) 相机运动；(ii) 场景类型；(iii) 额外传感器（或信息，例如场景地图），以简化问题。例如，连接到事件相机的深度传感器可减轻仅从事件估计深度的负担，IMU 则可提供准确的角速度信息。

当某种事件方法表现良好后，通常沿着近乎标准的“利用”路线逐步改进（类似帧式 SLAM）：例如，将单目方法^[914]扩展到双目或多相机场景^[381]；将^[1302]（以及^[585]）等纯事件方法通过融合惯性数据^[685, 812]（以及^[408]）增强鲁棒性；还可扩展基础系统以处理全向镜头等。尽管遵循这种利用路线，基于事件的 SLAM 仍是新兴领域，处于探索不同技术的阶段。分析表 10.1 中的方法即可看出：研究者利用多样的思想和原理，采用不同地图表示、事件表示和损失函数，设计支撑这些系统的估计算法。尤其需要继续研究能够发挥传感器本质特性的状态估计方法。

10.7 数据集、模拟器与基准

基于事件的视觉系统在原型开发、训练和基准测试中，对高质量、多样且信息丰富的真实与合成数据有很高需求。开发模拟器、数据集和排行榜对于推动领域发展、建立共同且坚实的科学技术基础至关重要。下面介绍事件相机领域具有代表性的 SLAM 数据集、基准和模拟器。

（表 10.1 的方法清单沿用原始出版物中的系统名称、缩写、输入类型和运动类别；各列含义见下方表注。）

系统	方法类型	直接/间接	事件表示	全局细化(BA)	运动	场景	输入	备注
Cook ^[229]	基于模型	直接	Event Frame	否	旋转	自然场景	E	使用光流的交互网络
Weikersdorfer ^[1175]	基于模型	间接	Individual Event	否	平面	二维黑白	E	首个基于滤波器的 Ev-SLAM
PF-SMT ^[570]	基于模型	直接	Individual Event	否	旋转	自然场景	E	两个交错的贝叶斯滤波器
Censi ^[160]	基于模型	直接	Event Packet	否	6 自由度	黑白	E+F+D	基于图像梯度的滤波式 VO
EB-SLAM-3D ^[1176]	基于模型	直接	Individual Event	否	6 自由度	自然场景	E+D	使用深度传感器增强事件
Yuan ^[1260]	基于模型	间接	Event Frame	否	6 自由度	黑白	E+I+M	基于竖直线的相机跟踪

系统	方法类型	直接/间接	事件表示	全局细化(BA)	运动	场景	输入	备注
Kueng ^[611]	基于模型	间接	Local Point Set	否	6 自由度	自然场景	E+F	基于事件特征跟踪的 VO
ETAM ^[571]	基于模型	直接	Individual Events	否	6 自由度	自然场景	E	三个交错的滤波器
CMax- ω ^[356]	基于模型	直接	Individual Events	否	旋转	自然场景	E	对比度最大化
EVO ^[914]	基于模型	直接	Edge Map	否	6 自由度	自然场景	E	事件-事件几何对齐
EVIO ^[1303]	基于模型	间接	Point sets	否	6 自由度	自然场景	E+I	基于滤波器和运动补偿特征
Rebecq ^[915]	基于模型	间接	MC Event Images	是	6 自由度	自然场景	E+I	基于特征,滑动窗口后端
RTPT ^[925]	基于模型	直接	Individual Events	否	旋转	自然场景	E	全景跟踪与建图
Gallego ^[358]	基于模型	直接	Individual Events	否	6 自由度	自然场景	E+M	鲁棒传感器模型
Mueggler ^[779]	基于模型	直接	Individual Events	是	6 自由度	自然场景	E+I+M	连续时间位姿估计器
USLAM ^[944]	基于模型	间接	MC Event Images	是	6 自由度	自然场景	E+F+I	传感器融合与滑动窗口后端
Chin ^[204]	基于模型	间接	Event Frames	是	旋转	星场景	E	针对星跟踪设计
ESVO ^[1302]	基于模型	直接	Time sur-faces (TS)	否	6 自由度	自然场景	2E	基于 TS 图像块的双目匹配
Hadviger ^[425]	基于模型	间接	Corners on TS	否	6 自由度	自然场景	2E	交叉相关特征描述子
CMax-GAE ^[572]	基于模型	直接	Individual Events	否	旋转	自然场景	E	对比度最大化
EKLT-VIO ^[725]	基于模型	间接	Individual events	是	6 自由度	自然场景	E+F+I	EKLT 跟踪器和 VIO 后端
EDS ^[461]	基于模型	直接	Event images	是	6 自由度	自然场景	E+F	帧式后端 (DSO)
CB-VIO ^[684]	基于模型	间接	Individual events	是	6 自由度	自然场景	E+F+I	特征跟踪器和 VIO 后端
Wang ^[1154]	基于模型	间接	Binary images	是	6 自由度	自然场景	2E	特征匹配
El Moudni ^[303]	基于模型	直接	Time Sur-faces (TS)	否	6 自由度	自然场景	2E	使用 ESVO 跟踪器和 EMVS 建图器
ESVIO ^[183]	基于模型	间接	Time sur-faces (TS)	是	6 自由度	自然场景	2E+2F+I	基于 TS 的特征跟踪
ESVIO-direct ^[685]	基于模型	直接	Time sur-faces (TS)	是	6 自由度	自然场景	2E+I	ESVO 的扩展
PL-EVIO ^[410]	基于模型	间接	Time sur-faces (TS)	是	6 自由度	自然场景	E+F+I	点与线特征、滑动窗口 BA
CMax-SLAM ^[416]	基于模型	直接	Individual Events	是	旋转	自然场景	E	对比度最大化
EVI-SAM ^[409]	基于模型	直接、间接	Individual Events	是	6 自由度	自然场景	E+F+I	稠密建图
Zuo ^[1317]	基于模型	直接	Individual Event	否	6 自由度	自然场景	E+D	使用深度传感器增强事件
DEVO ^[585]	基于深度学习	间接	Event voxel grids	是	6 自由度	自然场景	E	DPVO ^[1084] 的事件版本
EMBA ^[417]	基于模型	直接	Individual Events	是	旋转	自然场景	E	细化运动和梯度地图
EPBA ^[418]	基于模型	直接	Individual Events	是	旋转	自然场景	E	细化运动和强度地图
ES-PTAM ^[381]	基于模型	直接	Events (也作为帧)	否	6 自由度	自然场景	2E	使用 EVO 跟踪器和 EMVS 建图器
ESVO2 ^[812]	基于模型	直接	Time sur-faces (TS)	是	6 自由度	自然场景	2E+I	ESVO 的扩展

系统	方法类型	直接/间接	事件表示	全局细化(BA) 运动	场景	输入	备注
DEIO ^[408]	基于深度学习	间接	Event voxel grids	是 6 自由度	自然场景	E+I	DEVO 和 DPVO 的扩展

表 10.1 基于事件的视觉 SLAM 方法汇总, 按时间顺序排列。列分别表示方法类型 (基于模型或基于深度学习、直接或间接)、是否具有全局细化模块 (即后端/BA)、可处理的相机运动类型 (旋转、平面、6 自由度) 和场景类型 (高对比度黑白场景等), 以及输入数据类型 (事件相机、帧式相机、深度传感器、IMU 和地图); 其中 “2E” 表示双目事件 (两个传感器)。

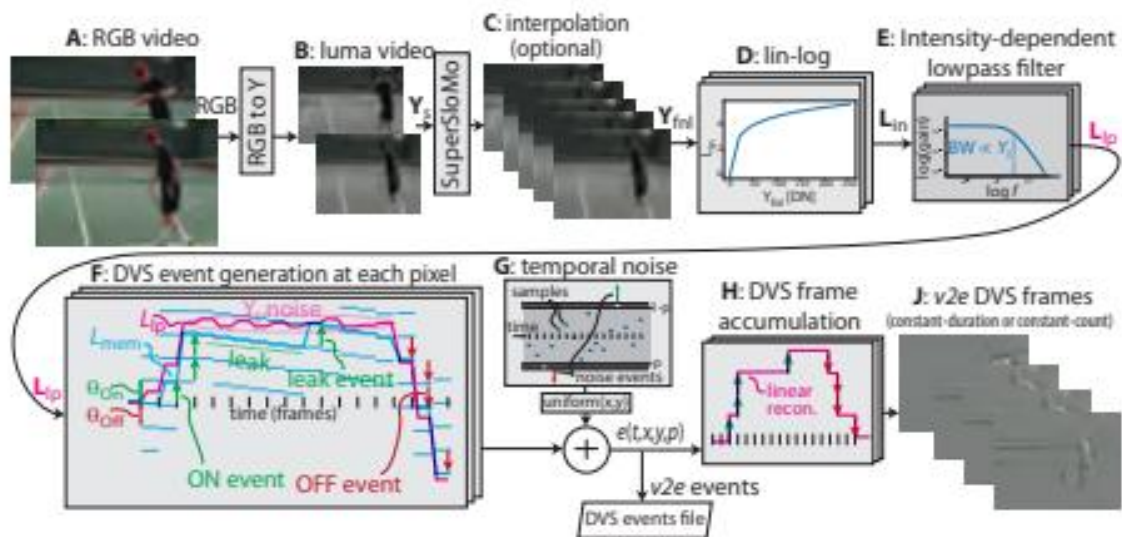
10.7.1 模拟器

目前有多种公开工具可生成高质量的合成事件相机数据。ESIM^[917] 是^[778]的演进版本;^[778]是最早模拟事件相机工作原理的模拟器之一。更早的工作 (如^[535]) 只是对连续两帧之间的差异进行阈值化, 以生成类似事件相机输出的边缘图像。ESIM 将渲染引擎与事件模拟器紧密耦合, 使后者能够根据视觉信号的动态自适应地渲染帧。

CARLA^[279] 中的事件相机模拟器将 ESIM 扩展到更多样、丰富且复杂的自动驾驶场景。在基于事件学习单目深度^[460]的背景下, CARLA 开发的事件传感器从模拟器渲染图像中计算逐像素亮度变化, 以与 ESIM 相同的方式模拟事件相机。图 10.5 展示了 CARLA 模拟器生成的 RGB 图像和事件。



(a) CARLA 模拟器



(b) V2E 模拟器图 10.5 事件相机模拟器: (a) 在 CARLA 中使用 ESIM 模拟器生成的 RGB 图像和事件^[279]; (b) V2E 工具的详细数据处理步骤^[481]。

受基于学习的方法需要大量事件数据训练、而事件数据又因传感器新颖而难以获得这一事实的推动，研究者开发了将传统相机录制的任意视频转换为合成事件数据的工具 Video to Events (Vid2E) [371]。Vid2E 旨在缩小传统视觉与事件视觉公开数据集之间的差距，使面向真实事件数据训练的网络可以使用几乎无限的现有视频数据集。Vid2E 将 ESIM 与自适应视频帧插值相结合，解决事件数据稀缺瓶颈。ESIM 能够以任意时间分辨率自适应渲染虚拟场景，但视频序列通常只提供固定且较低时间分辨率的强度测量。Super SloMo [521] 可重建任意时间分辨率的中间帧，然后应用事件相机模拟器 ESIM。中间帧数量需要谨慎选择：数量过少会导致亮度信号混叠，数量过多则增加计算负担。

事件相机模拟器的一个重要方面是准确建模噪声，以缩小算法迁移时的仿真到真实差距。例如，ESIM 及其衍生模拟器根据经验观测 [661] 实现了简单噪声模型：事件相机的对比度阈值 ((10.1) 中的 C) 并非固定，而是服从正态分布。为模拟这一点，ESIM 在每个模拟步骤从正态分布 $\mathcal{N}(C, \sigma_C^2)$ 采样，其中噪声水平 σ_C 可调。此外，ESIM 允许分别设置正、负对比度阈值 (C^+ 和 C^-)，以更准确地模拟真实事件相机。模拟器还应考虑其他噪声效应，例如电子噪声或事件像素带宽有限导致的对比度阈值空间和时间变化。

Vid2E [371] 建模理想事件相机的照明。V2E [481] 则基于 DVS 电路提出更真实的事件相机噪声模拟器。V2E 是首个包含时间噪声、泄漏事件和有限强度相关带宽的事件相机模拟器，并采用与 Vid2E 相同的高斯阈值分布。V2E 朝着更真实的模拟器迈进一步，可生成覆盖多种照明条件的合成数据集，这对事件应用十分重要。与 Vid2E 类似，V2E 使用 Super SloMo [521] 提高输入视频的时间分辨率。图 10.5 详细展示了 V2E 的架构。

模拟事件相机噪声是实现真实合成事件生成的挑战。EventGAN [1306] 提出一种端到端深度学习方案模拟事件相机数据：利用已有的图像标注数据，根据一对时间相邻的图像帧模拟事件，并使用 U-Net [934] 编码器-解码器网络。该方法在图像和事件对上训练神经网络，不采用直接数值误差损失，而使用对抗判别器损失和一对循环一致性损失。EventGAN 为每种极性生成三维时空体素网格，而不是单个事件集合；这种表示常作为人工神经网络的输入。

VISTA 2.0 [30] 是集成多种传感器类型（包括 RGB 相机、LiDAR 和事件相机）的模拟器，用于自动驾驶车辆的策略学习。它使用高保真的真实世界数据模拟多样场景，包括不同天气、照明和道路条件。事件相机模拟器与带自适应采样的 ESIM 类似，使用人工神经网络估计连续两帧之间的双向光流。VISTA 2.0 面向感知到控制策略的训练；与仅使用真实世界训练数据相比，它表现出更强的鲁棒性和仿真到真实迁移能力，从而改善安全关键场景中的车辆控制。

Video to Continuous Events (V2CE) [1282] 旨在生成比既有模拟器更真实的事件时间戳。Vid2E 和 V2E 在离散时间戳生成事件，而真实事件以连续时间方式产生。对于对时间戳分布敏感的任务，这一差异不可忽略；它会造成显著域偏移，从而限制合成事件的使用。V2CE 分两阶段运行：第一阶段使用有监督的三维 U-Net 编码器-解码器 ANN，从视频预测两个体素网格（每种极性一个，与 EventGAN 类似）；第二阶段从体素网格恢复精确事件时间戳。该方法迭代推断每个体素中的事件数量及其相对位置。V2CE 还表明，它能够在饱和光照区域以及理想传感器事件生成模型不成立的边缘处准确生成事件。

10.7.2 数据集与基准

自 ECDS [778] 发布以来，专用于视觉里程计和 SLAM 的事件数据集数量显著增加（见表 10.2）。ECDS 是首个同时提供同步事件、IMU 和 6 自由度相机真值位姿的数据集。更早的数据集 [955] 包含合成和真实事件，场景简单、视觉对比度高，且仅有纯旋转运动（3 自由度）；真值由 IMU 获得。另一项工作 [55] 支持事件相机与帧式相机运动的 5 自由度比较，其真值来自云台编码器和地面机器人的轮式里程计，因而容易漂移。ECDS 包含手持相机在多种场景中的 6 自由度运动（低速和高速），并由运动捕获系统提供精确的相机真值位姿。数据集包含 11 个真实事件场景和另外 2 个合成事件场景；合成数据由后来发展为 ESIM [917] 的第一版模拟器生成。

RPG 双目数据集 [1301] 包含在办公室环境中使用双目 DAVIS [111] 录制的 8 段手持序列，以及由模拟器 [778] 生成的一段合成序列（包含处于不同深度的三个正视平面）。该数据集不提供深度真值，但具有运动捕获系统提供的精确位姿真值，是事件双目 SLAM 方法原型开发和评估的良好起点。

Multi Vehicle Stereo Event Camera Dataset (MVSEC) [1304] 是首个在多种平台上提供深度真值的数据集。它采集室内和室外场景，覆盖不同照明水平和运动速度。平台包括手持装置、六旋翼、汽车和摩托车，均配备经过标定的 3D LiDAR、IMU 和标准帧式相机。MVSEC 可用于位姿估计、建图、障碍物规避和三维重建等任务，并通过集成 LiDAR 提供准确的深度和位姿真值。数据集包含长序列，可全面评估基于事件的算法。

UZH-FPV (第一人称视角) 数据集 [257] 专为推动自主无人机竞速研究而设计。它采用定制四旋翼、Qualcomm Flight Board 和安装在 Lumenier QAV-R 碳纤维机架上的 mDAVIS346 [1074] 事件相机。记录覆盖不同速度和轨迹的室内外场景，对导航和状态估计提出挑战。该数据集支持 VIO、事件数据处理和高速场景下的实时无人机应用研究，已成为开发高速相机运动算法的重要资源，并用于会议和研讨会竞赛。

Event Camera Motion Segmentation Dataset (EV-IMO) [769] 是首个专门用于室内独立运动物体 (IMO) 分割的事件数据集。它包含 32 分钟记录，使用运动捕获系统跟踪最多三个快速运动的 IMO，并提供逐像素运动掩膜以及自运动和深度真值。该数据集适用于机器人学，尤其是需要精确运动检测的受限场景（如目标跟踪和自主导航）。EV-IMO2 [126] 在其基础上增加了更多序列、三种更高质量的事件相机和更复杂的场景，既是当前算法的挑战性基准，也是开发新方法（包括单目和双目事件 SLAM）的丰富训练集。

表 10.2 事件 SLAM 数据集汇总，按时间顺序排列。传感器数据采用与表 10.1 相同的记号。双目和多传感器数据集的进一步介绍见综述 [380]。

数据集	平台	像素分辨率	传感器
ECDS [778]	手持	240×180	E、F、I
RPG-stereo [1301]	手持	240×180	2E
MVSEC [1304]	手持、无人机、汽车、摩托车	346×240	2E、2F、I、LiDAR、GPS
UZH-FPV [257]	无人机	346×260	E、F、I
EV-IMO [769]	手持	346×260	E、F、I、深度
EV-IMO2 [126]	手持	640×480	3E、F、I、深度
DSEC [372]	汽车	640×480	2E、2F、LiDAR、GPS
TUM-VIE [584]	手持	1280×720	2E、2F、I
EDS [461]	手持	640×480	E、F (RGB)、I

数据集	平台	像素分辨率	传感器
VECtor ^[363]	手持	640×480	2E、2F、RGB-D、I、LiDAR
M2DGR ^[1250]	地面机器人	640×480	E、F、I、LiDAR、GPS、热成像
ViViD++ ^[635]	手持、汽车	240×180 、 640×480	E、F、RGB-D、热成像、LiDAR、GPS
FusionPortable ^[523]	手持、四足机器人	346×240	2E、2F、I、LiDAR、GPS
Stereo HKU-VIO ^[183]	手持	346×260	2E、2F、I
M3ED ^[163]	无人机、汽车、四足机器人	1280×720	2E、2F、I、LiDAR、GPS
CoSEC ^[861]	汽车	1280×720	2E、2F、I、LiDAR、GPS

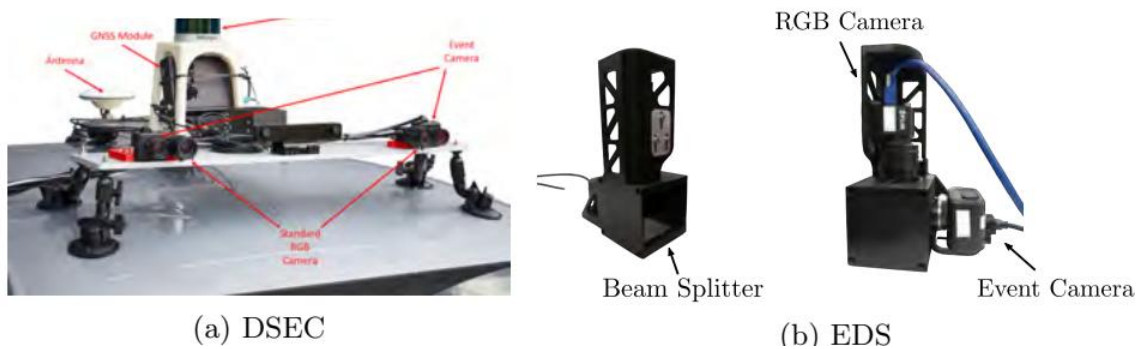


图 10.4: 图 10.6 部分事件 SLAM 数据集的细节: (a) DSEC 数据集^[372] 中安装在汽车顶部的传感器套件 (©2021 IEEE); (b) EDS 数据集^[461] 中使传感器共享空间对齐视场的分光器细节 (©2022 IEEE)。

Stereo Event Camera Dataset for Driving Scenarios (DSEC)^[372] 是面向自动驾驶的大规模数据集, 尤其支持通过融合事件和帧开发能够应对不利照明条件的鲁棒感知系统。平台采用多相机配置, 包括两台 VGA 分辨率 Prophesee Gen 3.1 事件相机 (基线 60 cm) 和两台 RGB 相机 (FLIR Blackfly S, 基线 51 cm, 见图 10.6), 同时配备 Velodyne VLP-16 LiDAR 和 RTK GPS 以实现精确定位。数据在瑞士不同城乡环境采集, 涵盖白天、夜间和直射阳光等多种照明条件, 并提供用于双目匹配的深度真值图。DSEC 还提供光流和视差, 用于在挑战性驾驶条件下基准测试算法; 基准指标包括 N 像素视差误差、平均绝对误差 (MAE) 和均方根误差 (RMSE), 以评估融合高分辨率事件相机数据与 RGB 帧的算法。

TUM Stereo Visual-Inertial Event Dataset (TUM-VIE)^[584] 使用双目 Prophesee Gen4 事件相机 (1 百万像素) 以及同步的 200 Hz IMU 数据和 20 Hz 双目灰度帧。它包含手持和头戴装置在多种室内外环境中的序列, 涵盖体育活动、HDR 场景和低照度条件等。TUM-VIE 用于促进 VIO、SLAM、三维重建和传感器融合研究, 尤其关注传统方法可能失效的挑战条件, 推动高分辨率事件感知算法的发展。

Event-Aided Direct Sparse Odometry (EDS) 数据集^[461] 包含高质量事件、彩色帧和 IMU 数据, 用于单目 VIO 研究。数据使用定制分光器装置采集 (见图 10.6), 使 RGB 帧和事件在同一光轴上精确对齐, 这在以往数据集中并不常见。记录场景包括自然室内环境, 提供高分辨率、良好标定的数据, 可用于光流估计、深度估计, 以及各种运动和照明条件下的鲁棒视觉里程计。

Versatile Event-Centric (VECtor) 基准数据集^[363] 同样用于评估事件 SLAM 算法。采集平台搭载多样传感器套件, 包括双目相机 (事件式和帧式)、RGB-D 传感器、128 通道 LiDAR 和九轴 IMU, 均安装在灵活的 3D 打印支架上。数据集包含小规模室内环境 (如运动捕获场) 和具有不同复杂度及照明条件的大规模室内环境。据称, 该数据集提供 VGA 分辨率、同步的多环境数据, 支持在静态和动态、低照度及 HDR 场景中可靠评估 SLAM 算法, 是推进多传感器 SLAM 研究的综合资源。

Vision for Visibility Dataset (ViViD++)^[635] 由包含热成像相机的多传感器平台采集, 支持研究能够处理低能见度、运动干扰和外观变化的 SLAM 算法, 利用不同传感器的互补优势。FusionPortable 数据集^[523] 包含一台四足机器人, 在走廊、食堂、道路和花园等不同场景中运动, 覆盖不同照明条件。

最后, Multi-robot, Multi-Sensor, Multi-Environment Event Dataset (M3ED)^[163] (非正式名称为 MVSEC 2.0) 聚焦机器人应用中的高速动态运动。它组合 1 百万像素双目事件相机、灰度和彩色相机、64 线 LiDAR 和高质量 IMU, 全部与 RTK 定位同步。不同于早期数据集, M3ED 提供来自多种平台、结构化和非结构化环境的异构数据, 并带有位姿、深度和语义标签真值, 是开发面向传统驾驶或室内应用之外动态环境的鲁棒事件感知算法的重要资源。

10.7.3 指标

理想情况下, SLAM 系统应分别刻画定位模块和建图模块的质量。然而, (i) 两个模块以交织方式运行 (深度误差会影响相机位姿误差, 反之亦然), (ii) 定位真值信息 (6 自由度) 比精确深度更紧凑且更容易获取, 因此深度估计误差通常被包含在相机轨迹误差的评估中。

由于经典 SLAM 和事件 SLAM 都输出相机轨迹, 事件 SLAM 沿用经典 SLAM 的性能评估协议。常用的两个指标是绝对轨迹误差 (ATE) 和相对位姿误差 (RPE)^[1045]。ATE 评估相机位姿相对于固定世界坐标系的精度, 可全面反映 VO 系统的长期性能。RPE 评估连续 (即相邻) 时间步之间的相对位姿, 关注 VO 系统的局部一致性。ATE 中的平移误差 (也称位置误差) 是估计相机位置与真值位置之间的欧氏距离; 旋转误差 (或方向误差) 由 $SO(3)$ 中的测地距离确定。类似地, RPE 的平移和旋转部分在时间间隔内的相机位姿对之间计算。有些研究还计算相对于场景平均深度或总行程的位置误差, 以确保误差对场景或轨迹尺度不变。

还可使用其他误差指标评估估计的平移向量和旋转矩阵^[1305], 包括平均 RPE (ARPE)、平均相对旋转误差 (ARRE) 和平均端点误差 (AEE)。具体而言, ARPE 和 AEE 测量两个平移向量之间的位置和方向差异, ARRE 计算两个旋转矩阵之间的测地距离。

除上述指标外, 平均线速度和角速度误差也可用于评估相机位姿估计, 尤其是事件相机能够借助事件估计突发、快速相机运动的场景。相机位姿随时间变化, 并由相应的线速度和角速度决定。已有多个工具箱^[405, 1283] 公开发布, 用于促进研究可复现性并降低 SLAM 轨迹评估的复杂性。

如果单独评估深度估计, 通常使用直到固定深度值的多个截断处的平均深度误差, 以比较不同三维地图尺度下的方法。然而, 包含深度真值的数据集并不多 (见第 10.7 节)。首选指标是估计三维点到真值地图最近表面之间欧氏距离的均方根误差 (RMSE)。文献还使用相对误差 (REL) 和完整度 (恢复点的数量) 等指标^[302, 460, 381]。

10.8 延伸阅读与近期趋势

尽管基于事件的 SLAM 研究已取得长足进展, 但由于该技术仍较新, 许多开放问题和挑战依然存在。这些问题涉及: 对于给定任务 (如 SLAM), 如何采集和处理视觉信息 (硬件和软件), 使系统在鲁棒性、延迟、效率和精度等方面达到或超过生物系统的表现。

传感器是异步的, 但由于神经形态计算的进入门槛, 文献中的大多数系统仍设计在串行 (即冯·诺依曼) 处理器上。与预期的全神经形态系统^[845] 相比, 这在效率 (功耗)、延迟等方面并非最优; 全神经形态系统将事件相机与异步 (受大脑启发、基于脉冲) 的处理器、控制器和执行器等配对。构建能够模仿动物高效处理方式、在环境中自主建图和定位的机器人, 一直是研究界的长期愿望 (可能应用于 AR/VR 中“始终开启”的 inside-out 跟踪等)。实现这一愿望需要以神经形态工程方式重新思考并协同设计传感器、处理器和算法^[245], 这极具挑战性, 需要广博而深入的专业知识以及多个学科的协调。

在不久的将来, 人们将开发受两条视觉通路启发的新型混合传感器^[1238], 提供空间和时间对齐、低延迟、HDR 及精细节 (高像素数) 的数据。研究者也在探索模仿生物学的中央凹传感器^[317], 以降低带宽需求。事件相机及其演进 (例如像素处理器阵列、Aeveon 传感器等近传感器处理器), 以及与其他传感器 (帧式相机、结构光、LiDAR、雷达等) 的组合和数据融合, 仍有广阔的研究空间, 可进一步提升 SLAM 性能。

10.9 致谢

作者感谢 Giovanni Ciofi 在本章准备工作中提供的支持。

第十一章 面向 SLAM 的惯性里程计

作者：Guoquan (Paul) Huang、Cédric Le Gentil、Teresa Vidal-Calleja、Davide Scaramuzza、Frank Dellaert 和 Luca Carlone

惯性测量单元 (IMU) 已成为机器人同步定位与建图中最普遍的里程计来源之一。IMU 测量其所附着刚体的线加速度和转速。IMU 的外形、成本和性能覆盖很宽范围：从飞机使用的大型高精度光学传感器，到智能手机及其他消费设备使用的体积小但噪声较大的微机电系统 (MEMS)。MEMS IMU 的低尺寸、重量和功耗 (SWaP) 及低成本特性，使其成为机器人学的理想传感器；面向 SLAM 的相关研究已持续二十多年。

本章先介绍 IMU 基础知识及其测量模型 (第 11.1 节)，再介绍 IMU 预积分 (第 11.2 节)，该技术可将高频 IMU 数据加入因子图优化框架。随后说明使用 IMU 数据会在优化中引入额外变量 (例如传感器偏置)，并讨论 IMU 与相机、LiDAR 等外感传感器组合系统的可观测性¹ (第 11.3 节)。最后给出现代 IMU 中心 SLAM 系统能够实现的实例 (第 11.4 节) 并回顾近期趋势 (第 11.5 节)。

11.1 惯性传感与导航基础

六轴惯性测量单元 (IMU) 由加速度计和陀螺仪组成：前者测量传感器相对于惯性坐标系的线加速度，后者测量传感器的角速度 (或转速)²。惯性导航系统 (INS) 传统上属于航空航天工程研究范畴，其目标是根据初始状态和 IMU 测量历史，估计 IMU 所安装平台的当前状态 (如位姿和速度)^[171, 1099]。INS 可分为捷联式系统和稳定式系统：捷联系统将 IMU 固连于平台机架；稳定式系统将 IMU 安装在内万向架、多万向架结构或浮球上，使其方向相对于惯性坐标系保持恒定。机器人中的多数 INS 属于前一类，即使用刚性连接于感知平台、测量局部线加速度和角速度的 IMU。在机器人学中，惯性里程计常作为惯性导航的同义词，以强调估计的里程计性质。

INS 产生的里程计估计显然会随时间漂移，因此大多数应用还依赖 GPS、相机或 LiDAR 等其他传感器；此时称为辅助惯性导航系统 (AINS)。机器人学通常直接说明与 IMU 组合使用的传感器。例如，融合相机和 IMU 来提供三维运动跟踪的系统称为视觉惯性里程计；若还包含回环，则称为视觉惯性 SLAM 系统。

11.1.1 感知原理与测量模型

IMU 通常包含三轴加速度计和三轴陀螺仪，分别测量传感器平台的线加速度和角速度。陀螺仪设计的基本原理是角动量守恒；加速度计则利用质量块的惯性，测量相对于惯性坐标系的运动学加速度与重力加速度之差。加速度计可以基于多种原理设计，例如采用安装成摆锤质量块的速率陀螺仪、低摩擦外壳中检验质量块的惯性，或利用两条悬挂金属带振动差异、在其间布置检验质量块的结构。

测量模型。下面给出 IMU 测量模型，它将 IMU 测量与机器人状态及待估计的其他量 (例如偏置) 联系起来。为简化叙述，假设传感器坐标系与机器人机体坐标系 $\mathcal{F}^b$ 重合，世界坐标系 $\mathcal{F}^w$ 是惯性坐标系³。时刻 t 的 IMU 测量 $\mathbf{a}(t)$ 与 $\boldsymbol{\omega}(t)$ 通常认为受到加性白高斯噪声 $\boldsymbol{\eta}$ 和缓慢变化的传感器偏置 $\mathbf{b}$ 污染：

$$\mathbf{a}(t) = \mathbf{R}_w^b(t) (\mathbf{a}^w(t) - \mathbf{g}^w) + \mathbf{b}^a(t) + \boldsymbol{\eta}^a(t), \quad (11.1)$$

$$\boldsymbol{\omega}(t) = \boldsymbol{\omega}_b^b(t) + \mathbf{b}^g(t) + \boldsymbol{\eta}^g(t). \quad (11.2)$$

如通常约定，上标 $\mathcal{F}^b$ 表示相应物理量在机体（IMU）坐标系 $\mathcal{F}^b$ 中表达。时刻 t 的 IMU 位姿由变换 $\{R_b^w(t), p^w(t)\}$ 描述，它将传感器坐标系 $\mathcal{F}^b$ 中的点映射到 $\mathcal{F}^w$ （注意 $R_b^w(t) = (R_b^w(t))^\top$ ）。 $\mathbf{a}^w(t) \in \mathbb{R}^3$ 是传感器在世界坐标系中的加速度， $\mathbf{g}^w$ 是世界坐标系中的重力向量；因此 $R_b^w(t)(\mathbf{a}^w(t) - \mathbf{g}^w)$ 是 IMU 在机体/IMU 坐标系中感受到的加速度。 $\boldsymbol{\omega}_b^b(t) \in \mathbb{R}^3$ 是 $\mathcal{F}^b$ 相对 $\mathcal{F}^w$ 的瞬时角速度，并在 $\mathcal{F}^b$ 中表达。噪声项 $\boldsymbol{\eta}^g(t)$ 和 $\boldsymbol{\eta}^a(t)$ 假设为零均值高斯随机变量，待估偏置 $\mathbf{b}^a(t)$ 与 $\mathbf{b}^g(t)$ 假设服从随机游走。这里噪声和偏置向量的上标表示传感器（加速度计与陀螺仪），而非坐标系；例如 $\mathbf{b}^a(t)$ 是加速度计偏置。

扩展模型。虽然测量模型 (11.1)–(11.2) 在机器人应用中通常足够，但在重新标定传感器平台等情况下，可能需要更精细的模型以准确刻画感知过程。受制造误差影响，加速度计可能存在失准和尺度误差，模型可扩展为：

$$\mathbf{a}(t) = \mathbf{T}_a \mathbf{R}_w^b(t)(\mathbf{a}^w(t) - \mathbf{g}^w) + \mathbf{b}^a(t) + \boldsymbol{\eta}^a(t), \quad (11.3)$$

其中 $\mathbf{T}_a$ 为同时建模加速度计测量失准和尺度误差的形状矩阵。尺度误差可包含静态成分或温度相关成分，并可在传感器内参标定时确定。类似地，陀螺仪测量模型可扩展以刻画失准和尺度误差：

$$\boldsymbol{\omega}(t) = \mathbf{T}_g \boldsymbol{\omega}_b^b(t) + \mathbf{b}^g(t) + \boldsymbol{\eta}^g(t), \quad (11.4)$$

其中 $\mathbf{T}_g$ 是同时建模陀螺仪失准和尺度误差的形状矩阵。陀螺仪测量还常受加速度影响，这一现象称为 g 敏感性。若影响量处在加性白噪声 $\boldsymbol{\eta}^g(t)$ 的范围内，通常可忽略；但某些 MEMS 硬件中该影响更显著，可建模为：

$$\boldsymbol{\omega}(t) = \mathbf{T}_g \boldsymbol{\omega}_b^b(t) + \mathbf{T}_s \mathbf{R}_w^b(t)(\mathbf{a}^w(t) - \mathbf{g}^w) + \mathbf{b}^g(t) + \boldsymbol{\eta}^g(t), \quad (11.5)$$

其中 $\mathbf{T}_s$ 为 g 敏感性矩阵，可在标定期间估计。

11.1.2 初始对准

在 SLAM 中，通常把全局坐标系设为轨迹起始位姿，即把初始位姿 $\{R_b^w(0), p^w(0)\}$ 设为单位位姿。但 INS 的 IMU 测量包含重力项（参见 (11.1)），世界坐标系通常选为与重力对齐，因此初始位姿也必须与重力方向对齐。换言之，机器人方向不再可以任意选择，必须与重力方向一致。具体地，需要计算把机体（IMU）坐标系对齐到世界坐标系的旋转 $R_b^w(0)$ 。为简化起见，假设机器人初始静止，即部署开始时未施加比力，常用低成本 MEMS IMU 只测得重力。仅给定局部重力测量 $\mathbf{g}^b$ 时，无法恢复绕重力方向的旋转（偏航），它可依应用自由选择；但可通过以下静态初始化确定滚转和俯仰对应的旋转：

$$\begin{cases} \mathbf{z}_w^b = \frac{\mathbf{g}^b}{\|\mathbf{g}^b\|} \\ \mathbf{x}_w^b = \frac{\mathbf{e}_1 - \mathbf{z}_w^b \mathbf{e}_1^\top \mathbf{z}_w^b}{\|\mathbf{e}_1 - \mathbf{z}_w^b \mathbf{e}_1^\top \mathbf{z}_w^b\|} \\ \mathbf{y}_w^b = \mathbf{z}_w^b \times \mathbf{x}_w^b \end{cases} \Rightarrow \mathbf{R}_w^b = \begin{bmatrix} \mathbf{x}_w^b & \mathbf{y}_w^b & \mathbf{z}_w^b \end{bmatrix} \quad (11.6)$$

这里对 $\mathbf{e}_1 = [1 \ 0 \ 0]^\top$ 与 $\mathbf{g}^b$ 作 Gram-Schmidt 正交归一化， $\times$ 表示叉乘。直观地说，旋转矩阵 $\mathbf{R}_w^b$ 的最后一列 $\mathbf{z}_w^b$ 是世界坐标系 z 轴相对于机体坐标系的方向。世界 z 轴与重力对齐，因此 (11.6) 由机体坐标系的重力测量计算 $\mathbf{z}_w^b$ ；随后正交归一化过程计算 $\mathbf{x}_w^b$ 和 $\mathbf{y}_w^b$ ，以在任意偏航选择下补全 $\mathbf{R}_w^b$ 的列。

使用高端 IMU 的对准。高端 IMU 的陀螺仪足够灵敏，可测量地球自转速率 $\boldsymbol{\omega}_{ie}$ 。若所选世界坐标系是惯性坐标系（例如地心惯性坐标系 ECI^[318]），则可使用机体坐标系中的重力向量 $\mathbf{g}^b$ 与地球自转速率 $\boldsymbol{\omega}_{ie}$ 测量进行解析对准：

$$\begin{cases} \mathbf{g}^b = \mathbf{R}_w^b \mathbf{g}^w \\ \boldsymbol{\omega}_{ie}^b = \mathbf{R}_w^b \boldsymbol{\omega}_{ie}^w \\ \mathbf{g}^b \times \boldsymbol{\omega}_{ie}^b = \mathbf{R}_w^b (\mathbf{g}^w \times \boldsymbol{\omega}_{ie}^w) \end{cases} \Rightarrow \mathbf{R}_w^b = \begin{bmatrix} \mathbf{g}^w^\top \\ \boldsymbol{\omega}_{ie}^w^\top \\ (\mathbf{g}^w \times \boldsymbol{\omega}_{ie}^w)^\top \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{g}^b^\top \\ \boldsymbol{\omega}_{ie}^b^\top \\ (\mathbf{g}^b \times \boldsymbol{\omega}_{ie}^b)^\top \end{bmatrix} \quad (11.7)$$

所得旋转矩阵 $\mathbf{R}_w^b$ 通常需投影到 $\text{SO}(3)$ ，以减弱测量噪声影响。

11.2 IMU 预积分与因子图

上一节的 IMU 测量模型 (11.1)–(11.2) 将测量与机器人状态，尤其是位姿、速度及传感器偏置联系起来。原则上可按第 1 章从这些模型导出最大后验估计器，但这会产生不切实际的大因子图：典型 IMU 以高频率（例如 200–1000 Hz）测量，测量模型要求在每一个 IMU 采样时刻向因子图添加状态，所得因子图很快无法求解。连续时间表述虽可避免以 IMU 速率添加变量，却仍须为每次测量添加一个高频因子，因子图依旧臃肿。

IMU 预积分的核心思想是避免以 IMU 速率向因子图添加状态或测量：将一段时间内的 IMU 测量积分为相对运动测量，再将较少的运动测量加入因子图。然而，朴素积分（第 11.2.1 节）仍要求在因子图求解器每次迭代时重新积分，因为积分初始条件可能改变。预积分通过分离依赖状态变量的项和依赖测量的项来避免该问题。预积分思想可追溯至 [709]，并由 [334, 335] 扩展到流形；第 11.2.2 节紧随 [334, 335] 的表述，第 11.2.3 节讨论更高级技术，近期工作留待第 11.5 节介绍。

11.2.1 运动积分

本节从 IMU 测量推断机器人运动。为此，引入如下运动学模型 [790, 318]：

$$\dot{\mathbf{R}}_b^w = \mathbf{R}_b^w (\boldsymbol{\omega}_b^b)^\wedge, \quad \dot{\mathbf{v}}^w = \mathbf{a}^w, \quad \dot{\mathbf{p}}^w = \mathbf{v}^w, \quad (11.8)$$

它描述机体坐标系 $\mathcal{F}^b$ 相对于世界坐标系 $\mathcal{F}^w$ 的旋转 $\mathbf{R}_b^w$ 、平移 $\mathbf{p}^w$ 和速度 $\mathbf{v}^w$ 的演化。设 Δt 为 IMU 采样周期，时刻 $t + \Delta t$ 的状态由对 (11.8) 积分得到：

$$\begin{aligned} \mathbf{R}_b^w(t + \Delta t) &= \mathbf{R}_b^w(t) \text{Exp} \left(\int_t^{t+\Delta t} \boldsymbol{\omega}_b^b(\tau) d\tau \right) \\ \mathbf{v}^w(t + \Delta t) &= \mathbf{v}^w(t) + \int_t^{t+\Delta t} \mathbf{a}^w(\tau) d\tau \end{aligned} \quad (11.9)$$

$$\mathbf{p}^w(t + \Delta t) = \mathbf{p}^w(t) + \int_t^{t+\Delta t} \mathbf{v}^w(\tau) d\tau, \quad (11.10)$$

上式第一行假设角速度 $\boldsymbol{\omega}_b^b$ 的方向在区间 $[t, t + \Delta t]$ 内不变⁴。进一步假设 $\mathbf{a}^w$ 与 $\boldsymbol{\omega}_b^b$ 在该时间区间内恒定，可写为：

$$\begin{aligned} \mathbf{R}_b^w(t + \Delta t) &= \mathbf{R}_b^w(t) \text{Exp}(\boldsymbol{\omega}_b^b(t) \Delta t) \\ \mathbf{v}^w(t + \Delta t) &= \mathbf{v}^w(t) + \mathbf{a}^w(t) \Delta t \\ \mathbf{p}^w(t + \Delta t) &= \mathbf{p}^w(t) + \mathbf{v}^w(t) \Delta t + \frac{1}{2} \mathbf{a}^w(t) \Delta t^2. \end{aligned} \quad (11.11)$$

更一般地，(11.11) 可理解为使用 Euler 积分数值求解 (11.9) 中的积分。利用 (11.1)–(11.2)，可将 $\mathbf{a}^w$ 与 $\boldsymbol{\omega}_b^b$ 写成 IMU 测量的函数，得到：

$$\begin{aligned} \mathbf{R}(t + \Delta t) &= \mathbf{R}(t) \text{Exp} \left((\tilde{\boldsymbol{\omega}}(t) - \mathbf{b}^g(t) - \boldsymbol{\eta}^{gd}(t)) \Delta t \right) \\ \mathbf{v}(t + \Delta t) &= \mathbf{v}(t) + \mathbf{g} \Delta t + \mathbf{R}(t) (\tilde{\mathbf{a}}(t) - \mathbf{b}^a(t) - \boldsymbol{\eta}^{ad}(t)) \Delta t \\ \mathbf{p}(t + \Delta t) &= \mathbf{p}(t) + \mathbf{v}(t) \Delta t + \frac{1}{2} \mathbf{g} \Delta t^2 + \frac{1}{2} \mathbf{R}(t) (\tilde{\mathbf{a}}(t) - \mathbf{b}^a(t) - \boldsymbol{\eta}^{ad}(t)) \Delta t^2, \end{aligned} \quad (11.12)$$

为便于阅读，以下省略坐标系下标。该速度和位置数值积分在两次测量之间的积分时间内假设方向 $\mathbf{R}(t)$ 恒定；对于非零转速测量，它并非微分方程 (11.8) 的精确解。实践中，高频 IMU 可减轻此近似的影响。这里采用 (11.12)，因为它简单且便于建模与不确定性传播；第 11.2.3 节再讨论更高级的积分技术。离散时间噪声 $\boldsymbol{\eta}^{gd}$ 的协方差依赖采样率，并与连续时间噪声 $\boldsymbol{\eta}^g$ 满足 $\text{Cov}(\boldsymbol{\eta}^{gd}(t)) = \frac{1}{\Delta t} \text{Cov}(\boldsymbol{\eta}^g(t))$ ； $\boldsymbol{\eta}^{ad}$ 同理（参见 [232, Appendix]）。

尽管 (11.12) 可直接视为因子图中的概率约束，但它要求以高频将状态加入因子图：它联系时刻 t 和 $t + \Delta t$ 的状态，而 Δt 正是 IMU 采样周期，因此每到一条 IMU 测量都要增加一个状态 [503]。

可以通过在更长时间区间上积分来避免这一问题。若因子图已经建模其他传感器测量（例如第 7 章的视觉测量），即可在因子图中两个时间相邻状态之间积分 IMU 测量，称这些状态为“关键帧状态”⁵。对两个连续关键帧间的所有 Δt 区间迭代 (11.12)，得到⁶：

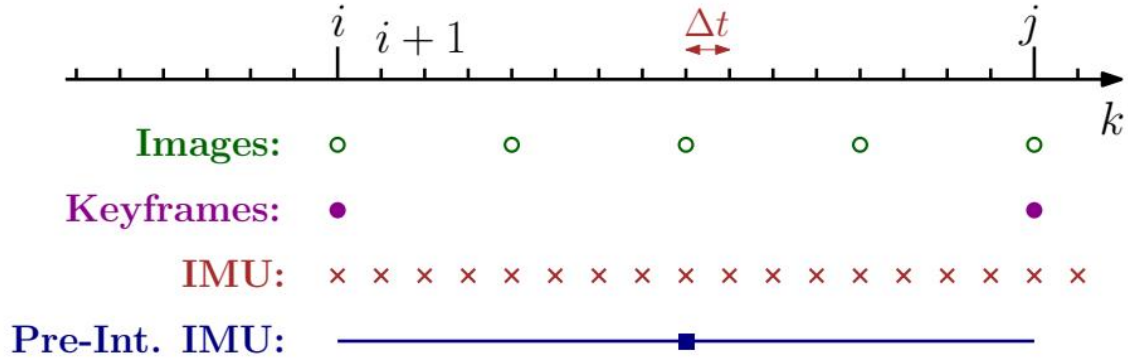


图 11.1: 图 11.1 IMU 与相机具有不同采样速率。引自 [335] (©2016 IEEE)。

$$\begin{aligned}
 \mathbf{R}_j &= \mathbf{R}_i \prod_{k=i}^{j-1} \text{Exp} \left((\tilde{\omega}_k - \mathbf{b}_k^g - \boldsymbol{\eta}_k^{gd}) \Delta t \right), \\
 \mathbf{v}_j &= \mathbf{v}_i + \mathbf{g} \Delta t_{ij} + \sum_{k=i}^{j-1} \mathbf{R}_k (\tilde{\mathbf{a}}_k - \mathbf{b}_k^a - \boldsymbol{\eta}_k^{ad}) \Delta t \\
 \mathbf{p}_j &= \mathbf{p}_i + \sum_{k=i}^{j-1} \left[\mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{g} \Delta t^2 + \frac{1}{2} \mathbf{R}_k (\tilde{\mathbf{a}}_k - \mathbf{b}_k^a - \boldsymbol{\eta}_k^{ad}) \Delta t^2 \right]
 \end{aligned} \tag{11.13}$$

这让因子图以较低频率连接关键帧状态；但若在优化过程中关键帧起始状态或偏置变化，必须重新进行积分 [645]，仍会造成很大计算负担。这正是预积分要解决的问题。

11.2.2 流形上的 IMU 预积分

预积分把关键帧间相对旋转、速度和位置变化表达在第一个关键帧坐标系中，从而把依赖起始状态的项移到因子一侧。下面使用流形上的旋转表示，以保证优化中始终满足旋转矩阵的正交约束。

11.2.2.1 预积分 IMU 测量

定义关键帧 i 至 j 的预积分测量为：

$$\begin{aligned}
 \Delta \mathbf{R}_{ij} &\doteq \mathbf{R}_i^\top \mathbf{R}_j = \prod_{k=i}^{j-1} \text{Exp} \left((\tilde{\omega}_k - \mathbf{b}_k^g - \boldsymbol{\eta}_k^{gd}) \Delta t \right) \\
 \Delta \mathbf{v}_{ij} &\doteq \mathbf{R}_i^\top (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g} \Delta t_{ij}) = \sum_{k=i}^{j-1} \Delta \mathbf{R}_{ik} (\tilde{\mathbf{a}}_k - \mathbf{b}_k^a - \boldsymbol{\eta}_k^{ad}) \Delta t \\
 \Delta \mathbf{p}_{ij} &\doteq \mathbf{R}_i^\top (\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{ij} - \frac{1}{2} \mathbf{g} \Delta t_{ij}^2) \\
 &= \sum_{k=i}^{j-1} \left[\Delta \mathbf{v}_{ik} \Delta t + \frac{1}{2} \Delta \mathbf{R}_{ik} (\tilde{\mathbf{a}}_k - \mathbf{b}_k^a - \boldsymbol{\eta}_k^{ad}) \Delta t^2 \right],
 \end{aligned} \tag{11.14}$$

为了使这些量仅依赖于 IMU 测量，通常在区间内把偏置近似为常数：

$$\mathbf{b}_i^g = \mathbf{b}_{i+1}^g = \dots = \mathbf{b}_{j-1}^g, \quad \mathbf{b}_i^a = \mathbf{b}_{i+1}^a = \dots = \mathbf{b}_{j-1}^a. \tag{11.15}$$

令 $\Delta \tilde{\mathbf{R}}_{ij}$ 、 $\Delta \tilde{\mathbf{v}}_{ij}$ 和 $\Delta \tilde{\mathbf{p}}_{ij}$ 分别为无噪声积分值。使用指数映射的一阶近似及旋转交换关系：

$$\text{Exp}(\phi + \delta\phi) \approx \text{Exp}(\phi) \text{Exp}(\mathbf{J}_r(\phi) \delta\phi), \tag{11.16}$$

$$\text{Exp}(\phi) \mathbf{R} = \mathbf{R} \text{Exp}(\mathbf{R}^\top \phi). \tag{11.17}$$

可将预积分旋转写为名义预积分和小旋转噪声的乘积：

$$\begin{aligned}
 \Delta \mathbf{R}_{ij} &\stackrel{\text{eq. (11.16)}}{\simeq} \prod_{k=i}^{j-1} \left[\text{Exp}((\tilde{\omega}_k - \mathbf{b}_i^g) \Delta t) \text{Exp}(-\mathbf{J}_r^k \boldsymbol{\eta}_k^{gd} \Delta t) \right] \\
 &\stackrel{\text{eq. (11.17)}}{=} \Delta \tilde{\mathbf{R}}_{ij} \prod_{k=i}^{j-1} \text{Exp}(-\Delta \tilde{\mathbf{R}}_{k+1,j}^\top \mathbf{J}_r^k \boldsymbol{\eta}_k^{gd} \Delta t) \\
 &\doteq \Delta \tilde{\mathbf{R}}_{ij} \text{Exp}(-\delta\phi_{ij})
 \end{aligned} \tag{11.18}$$

进一步使用小角度近似及叉乘矩阵恒等式：

$$\exp(\phi^\wedge) \approx \mathbf{I} + \phi^\wedge, \quad (11.19)$$

$$\mathbf{a}^\wedge \mathbf{b} = -\mathbf{b}^\wedge \mathbf{a}, \quad \forall \mathbf{a}, \mathbf{b} \in \mathbb{R}^3, \quad (11.20)$$

即可得到速度和位置预积分误差：

$$\begin{aligned} \Delta \mathbf{v}_{ij} &\stackrel{\text{eq. (11.19)}}{\simeq} \sum_{k=i}^{j-1} \Delta \tilde{\mathbf{R}}_{ik} (\mathbf{I} - \delta \phi_{ik}^\wedge) (\tilde{\mathbf{a}}_k - \mathbf{b}_i^a) \Delta t - \Delta \tilde{\mathbf{R}}_{ik} \boldsymbol{\eta}_k^{ad} \Delta t \\ &\stackrel{\text{eq. (11.20)}}{=} \Delta \tilde{\mathbf{v}}_{ij} + \sum_{k=i}^{j-1} \left[\Delta \tilde{\mathbf{R}}_{ik} (\tilde{\mathbf{a}}_k - \mathbf{b}_i^a)^\wedge \delta \phi_{ik} \Delta t - \Delta \tilde{\mathbf{R}}_{ik} \boldsymbol{\eta}_k^{ad} \Delta t \right] \\ &\doteq \Delta \tilde{\mathbf{v}}_{ij} - \delta \mathbf{v}_{ij} \end{aligned} \quad (11.21)$$

$$\begin{aligned} \Delta \mathbf{p}_{ij} &\stackrel{\text{eq. (11.19)}}{\simeq} \sum_{k=i}^{j-1} \left[(\Delta \tilde{\mathbf{v}}_{ik} - \delta \mathbf{v}_{ik}) \Delta t + \frac{1}{2} \Delta \tilde{\mathbf{R}}_{ik} (\mathbf{I} - \delta \phi_{ik}^\wedge) (\tilde{\mathbf{a}}_k - \mathbf{b}_i^a) \Delta t^2 \right. \\ &\quad \left. - \frac{1}{2} \Delta \tilde{\mathbf{R}}_{ik} \boldsymbol{\eta}_k^{ad} \Delta t^2 \right] \\ &\stackrel{\text{eq. (11.20)}}{=} \Delta \tilde{\mathbf{p}}_{ij} + \sum_{k=i}^{j-1} \left[-\delta \mathbf{v}_{ik} \Delta t + \frac{1}{2} \Delta \tilde{\mathbf{R}}_{ik} (\tilde{\mathbf{a}}_k - \mathbf{b}_i^a)^\wedge \delta \phi_{ik} \Delta t^2 \right. \\ &\quad \left. - \frac{1}{2} \Delta \tilde{\mathbf{R}}_{ik} \boldsymbol{\eta}_k^{ad} \Delta t^2 \right] \\ &\doteq \Delta \tilde{\mathbf{p}}_{ij} - \delta \mathbf{p}_{ij}, \end{aligned} \quad (11.22)$$

因此，带噪预积分测量可重写为：

$$\begin{aligned} \Delta \tilde{\mathbf{R}}_{ij} &= \mathbf{R}_i^\top \mathbf{R}_j \text{Exp}(\delta \phi_{ij}) \\ \Delta \tilde{\mathbf{v}}_{ij} &= \mathbf{R}_i^\top (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g} \Delta t_{ij}) + \delta \mathbf{v}_{ij} \\ \Delta \tilde{\mathbf{p}}_{ij} &= \mathbf{R}_i^\top (\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{ij} - \frac{1}{2} \mathbf{g} \Delta t_{ij}^2) + \delta \mathbf{p}_{ij} \end{aligned} \quad (11.23)$$

这一表示将预积分量的名义值与由陀螺仪、加速度计噪声引起的小误差显式分开。

11.2.2.2 噪声传播

预积分噪声向量由旋转、速度和位置误差构成，并假设为零均值高斯变量：

$$\boldsymbol{\eta}_{ij}^\Delta \doteq [\delta \phi_{ij}^\top, \delta \mathbf{v}_{ij}^\top, \delta \mathbf{p}_{ij}^\top]^\top \sim \mathcal{N}(\mathbf{0}_{9 \times 1}, \boldsymbol{\Sigma}_{ij}). \quad (11.24)$$

由旋转误差乘积表达式、对数映射的一阶近似，可导出旋转误差及其线性化递推：

$$\text{Exp}(-\delta \phi_{ij}) \doteq \prod_{k=i}^{j-1} \text{Exp} \left(-\Delta \tilde{\mathbf{R}}_{k+1j}^\top \mathbb{J}_r^k \boldsymbol{\eta}_k^{gd} \Delta t \right). \quad (11.25)$$

$$\delta \phi_{ij} = -\text{Log} \left(\prod_{k=i}^{j-1} \text{Exp} \left(-\Delta \tilde{\mathbf{R}}_{k+1j}^\top \mathbb{J}_r^k \boldsymbol{\eta}_k^{gd} \Delta t \right) \right). \quad (11.26)$$

$$\text{Log}(\text{Exp}(\phi) \text{Exp}(\delta \phi)) \approx \phi + \mathbf{J}_r^{-1}(\phi) \delta \phi. \quad (11.27)$$

$$\delta \phi_{ij} \simeq \sum_{k=i}^{j-1} \Delta \tilde{\mathbf{R}}_{k+1j}^\top \mathbf{J}_r^k \boldsymbol{\eta}_k^{gd} \Delta t \quad (11.28)$$

$$\delta \mathbf{v}_{ij} \simeq \sum_{k=i}^{j-1} \left[-\Delta \tilde{\mathbf{R}}_{ik} (\tilde{\mathbf{a}}_k - \mathbf{b}_i^a)^\wedge \delta \phi_{ik} \Delta t + \Delta \tilde{\mathbf{R}}_{ik} \boldsymbol{\eta}_k^{ad} \Delta t \right] \quad (11.29)$$

$$\delta \mathbf{p}_{ij} \simeq \sum_{k=i}^{j-1} \left[\delta \mathbf{v}_{ik} \Delta t - \frac{1}{2} \Delta \tilde{\mathbf{R}}_{ik} (\tilde{\mathbf{a}}_k - \mathbf{b}_i^a)^\wedge \delta \phi_{ik} \Delta t^2 + \frac{1}{2} \Delta \tilde{\mathbf{R}}_{ik} \boldsymbol{\eta}_k^{ad} \Delta t^2 \right]$$

这些递推关系可在预积分过程中累积协方差 $\boldsymbol{\Sigma}_{ij}$ ，从而将 IMU 噪声正确地传递到关键帧之间的九维预积分测量。

11.2.2.3 纳入偏置更新

前述预积分是在某个线性化偏置 $\bar{\mathbf{b}}_i$ 下计算的。优化后偏置发生小变化时，不必重新积分原始 IMU 数据；可用一阶雅可比修正预积分旋转、速度和位置：

$$\begin{aligned}\Delta \tilde{\mathbf{R}}_{ij}(\mathbf{b}_i^g) &\simeq \Delta \tilde{\mathbf{R}}_{ij}(\bar{\mathbf{b}}_i^g) \text{Exp}\left(\frac{\partial \Delta \tilde{\mathbf{R}}_{ij}}{\partial \mathbf{b}^g} \delta \mathbf{b}^g\right) \\ \Delta \tilde{\mathbf{v}}_{ij}(\mathbf{b}_i^g, \mathbf{b}_i^a) &\simeq \Delta \tilde{\mathbf{v}}_{ij}(\bar{\mathbf{b}}_i^g, \bar{\mathbf{b}}_i^a) + \frac{\partial \Delta \tilde{\mathbf{v}}_{ij}}{\partial \mathbf{b}^g} \delta \mathbf{b}^g + \frac{\partial \Delta \tilde{\mathbf{v}}_{ij}}{\partial \mathbf{b}^a} \delta \mathbf{b}^a \\ \Delta \tilde{\mathbf{p}}_{ij}(\mathbf{b}_i^g, \mathbf{b}_i^a) &\simeq \Delta \tilde{\mathbf{p}}_{ij}(\bar{\mathbf{b}}_i^g, \bar{\mathbf{b}}_i^a) + \frac{\partial \Delta \tilde{\mathbf{p}}_{ij}}{\partial \mathbf{b}^g} \delta \mathbf{b}^g + \frac{\partial \Delta \tilde{\mathbf{p}}_{ij}}{\partial \mathbf{b}^a} \delta \mathbf{b}^a\end{aligned}\quad (11.30)$$

这里 $\delta \mathbf{b}_i^g$ 与 $\delta \mathbf{b}_i^a$ 分别是陀螺仪和加速度计偏置的更新量。这一近似正是预积分在迭代优化中高效的关键。

11.2.2.4 预积分 IMU 因子与偏置模型

预积分因子将第 i, j 个关键帧的位姿、速度和偏置关联起来；其旋转、速度、位置残差为：

$$\begin{aligned}\mathbf{r}_{\Delta \mathbf{R}_{ij}} &\doteq \text{Log}\left(\left(\Delta \tilde{\mathbf{R}}_{ij}(\bar{\mathbf{b}}_i^g) \text{Exp}\left(\frac{\partial \Delta \tilde{\mathbf{R}}_{ij}}{\partial \mathbf{b}^g} \delta \mathbf{b}^g\right)\right)^\top \mathbf{R}_i^\top \mathbf{R}_j\right) \\ \mathbf{r}_{\Delta \mathbf{v}_{ij}} &\doteq \mathbf{R}_i^\top (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g} \Delta t_{ij}) \\ &\quad - \left[\Delta \tilde{\mathbf{v}}_{ij}(\bar{\mathbf{b}}_i^g, \bar{\mathbf{b}}_i^a) + \frac{\partial \Delta \tilde{\mathbf{v}}_{ij}}{\partial \mathbf{b}^g} \delta \mathbf{b}^g + \frac{\partial \Delta \tilde{\mathbf{v}}_{ij}}{\partial \mathbf{b}^a} \delta \mathbf{b}^a\right] \\ \mathbf{r}_{\Delta \mathbf{p}_{ij}} &\doteq \mathbf{R}_i^\top (\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{ij} - \frac{1}{2} \mathbf{g} \Delta t_{ij}^2) \\ &\quad - \left[\Delta \tilde{\mathbf{p}}_{ij}(\bar{\mathbf{b}}_i^g, \bar{\mathbf{b}}_i^a) + \frac{\partial \Delta \tilde{\mathbf{p}}_{ij}}{\partial \mathbf{b}^g} \delta \mathbf{b}^g + \frac{\partial \Delta \tilde{\mathbf{p}}_{ij}}{\partial \mathbf{b}^a} \delta \mathbf{b}^a\right],\end{aligned}\quad (11.31)$$

常把 IMU 偏置建模为连续时间随机游走：

$$\dot{\mathbf{b}}^g(t) = \boldsymbol{\eta}^{bg}, \quad \dot{\mathbf{b}}^a(t) = \boldsymbol{\eta}^{ba}. \quad (11.32)$$

$$\mathbf{b}_j^g = \mathbf{b}_i^g + \boldsymbol{\eta}^{bgd}, \quad \mathbf{b}_j^a = \mathbf{b}_i^a + \boldsymbol{\eta}^{bad}, \quad (11.33)$$

由此得到连接相邻关键帧偏置的偏置因子残差：

$$\|\mathbf{r}_{\mathbf{b}_{ij}}\|^2 \doteq \|\mathbf{b}_j^g - \mathbf{b}_i^g\|_{\boldsymbol{\Sigma}^{bgd}}^2 + \|\mathbf{b}_j^a - \mathbf{b}_i^a\|_{\boldsymbol{\Sigma}^{bad}}^2 \quad (11.34)$$

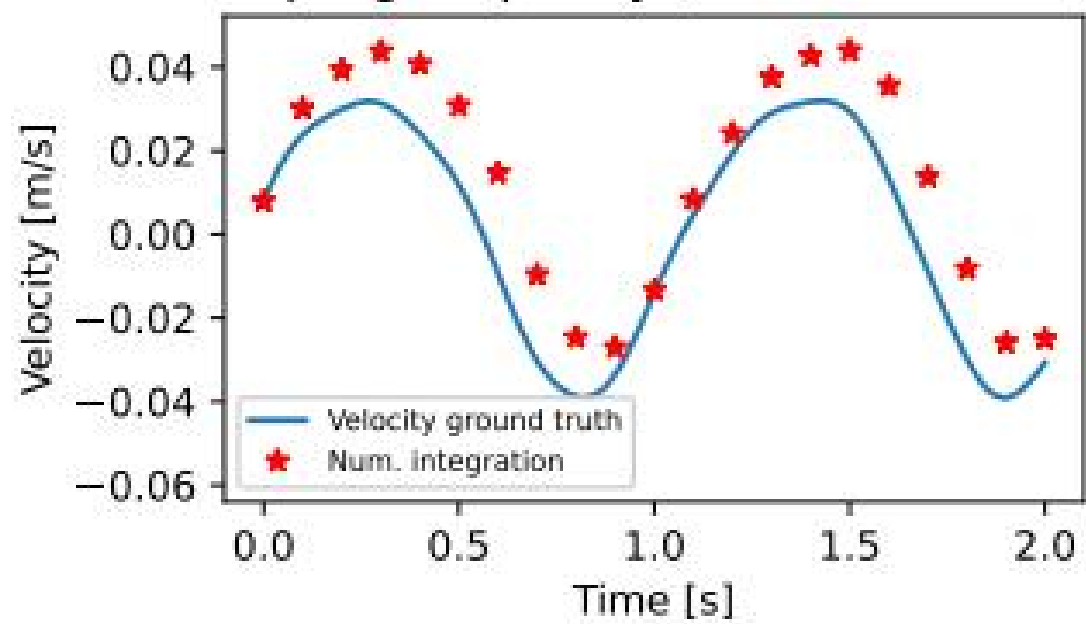
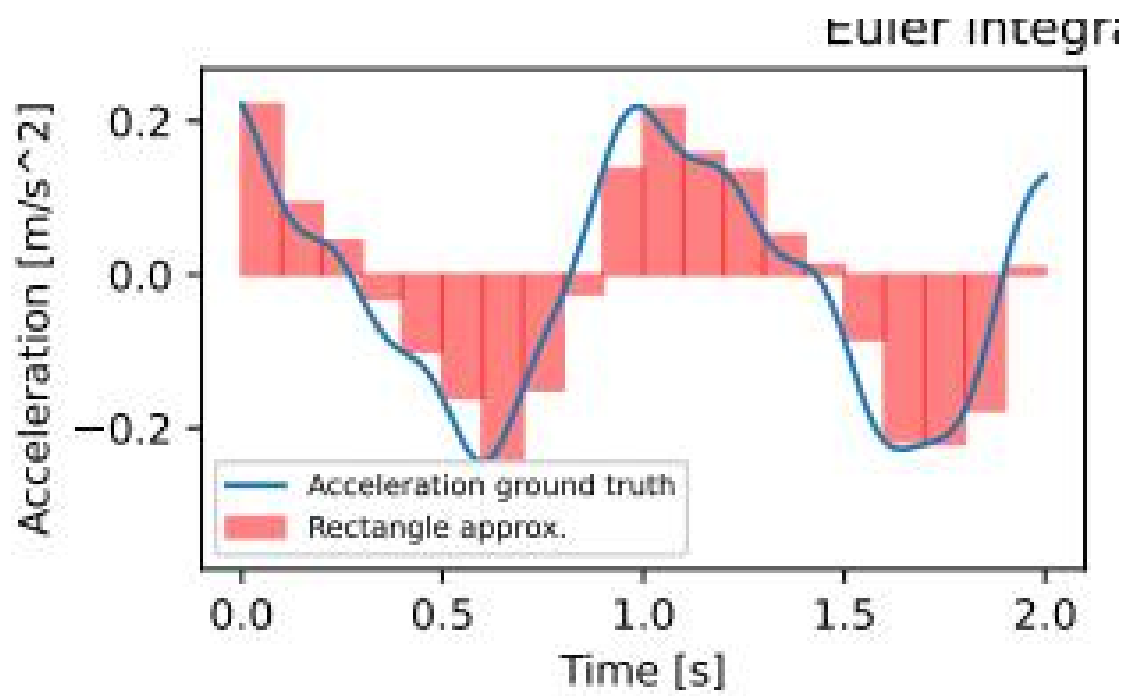
11.2.3 高级预积分技术

本节考察标准预积分方法的若干局限及较新的替代方案。首先分析 (11.14) 隐含的信号与运动假设，随后介绍放宽这些假设的代表性工作。更准确的预积分测量能够在辅助惯性导航中改善定位与建图精度；这里侧重说明方法的基本思想，完整推导请参阅相应论文。

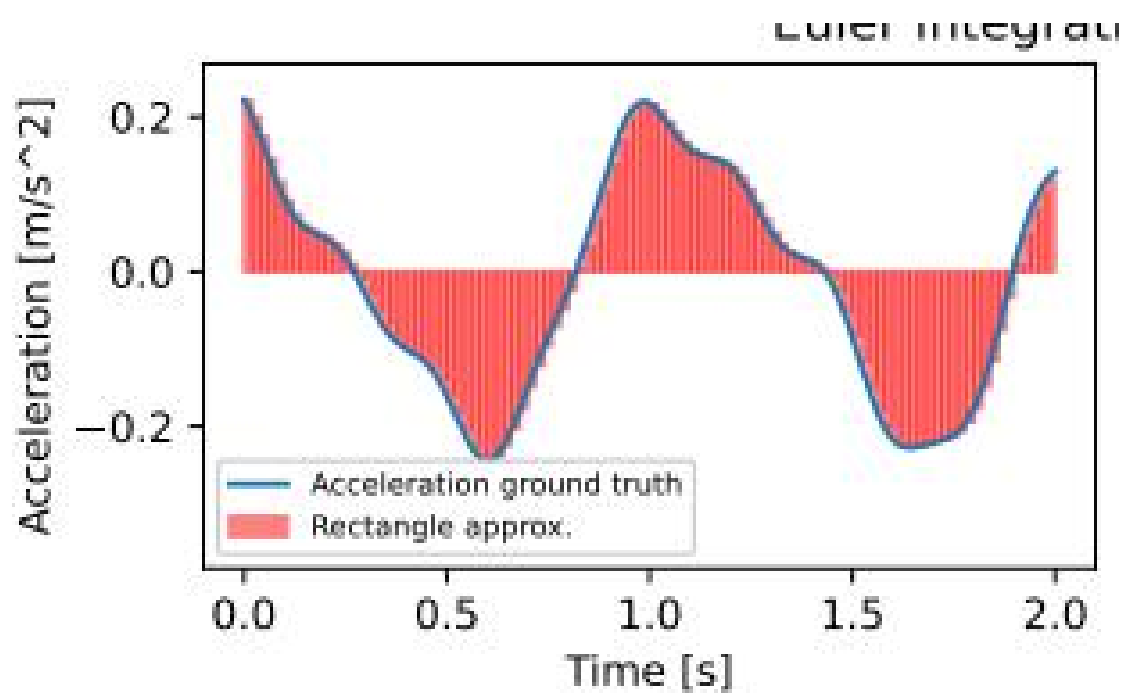
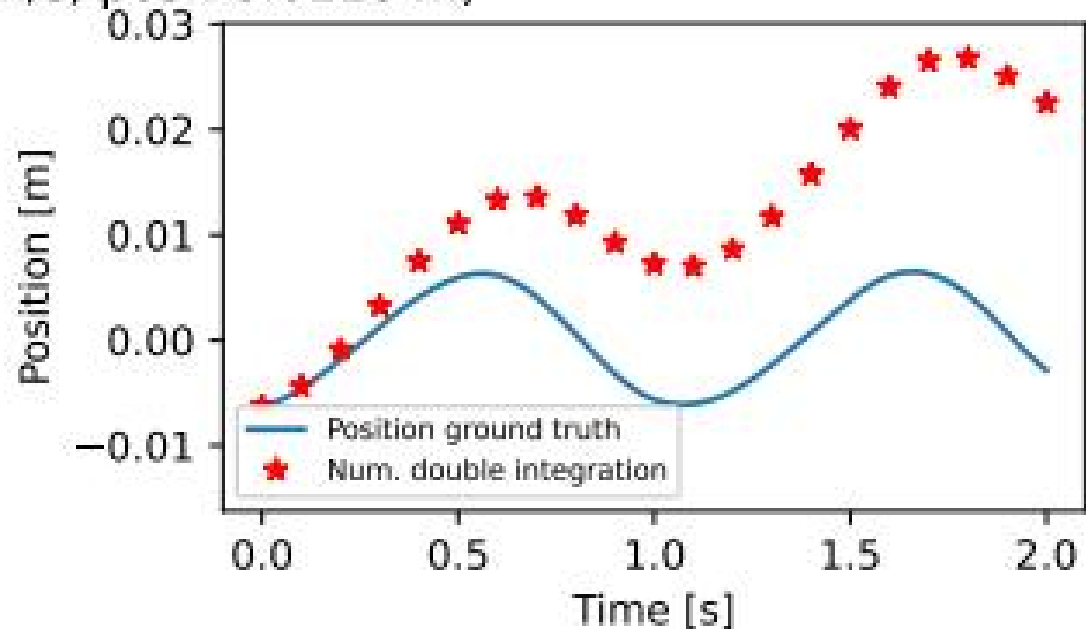
11.2.3.1 数值积分精度

标准预积分使用 Euler 方法把惯性信号积分为离散时刻的旋转、速度和位置伪测量。该方法快速高效，但会引入积分误差并造成漂移。Euler 方法实质上对信号应用矩形求积法，以给定频率采样的分段常值片段近似原信号；在惯性系统中，这些样本就是加速度计或陀螺仪测量。

采样频率较低时，分段常值假设不能准确表示输入信号，二重积分因而会迅速累积误差，如图 11.2 上行所示。提高采样频率可减小误差（图 11.2 下行），但真实惯性导航系统的采样频率受传感器硬件限制。^[631] 使用高斯过程（Gaussian Process, GP）回归⁷，在任意选定时间戳对陀螺仪与加速度计信号进行虚拟上采样。该方案优于标准预积分，但仍依据分段常值假设实施数值积分，未充分利用 GP 模型的连续性。下面介绍更精细的积分方法。



m/s, pos.: 0.0118 m)



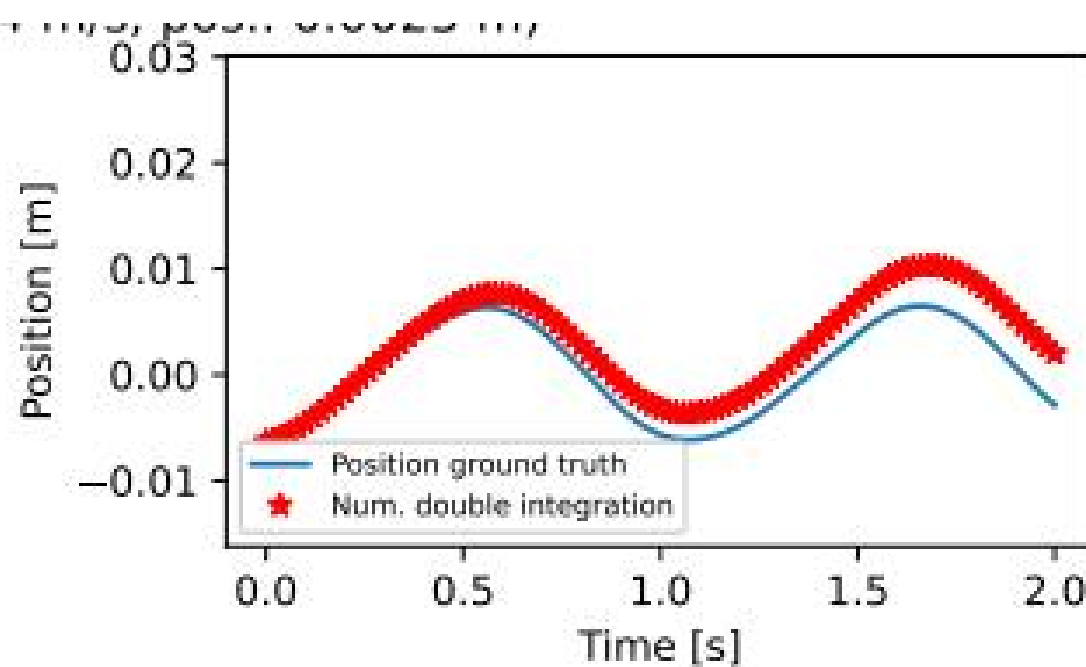
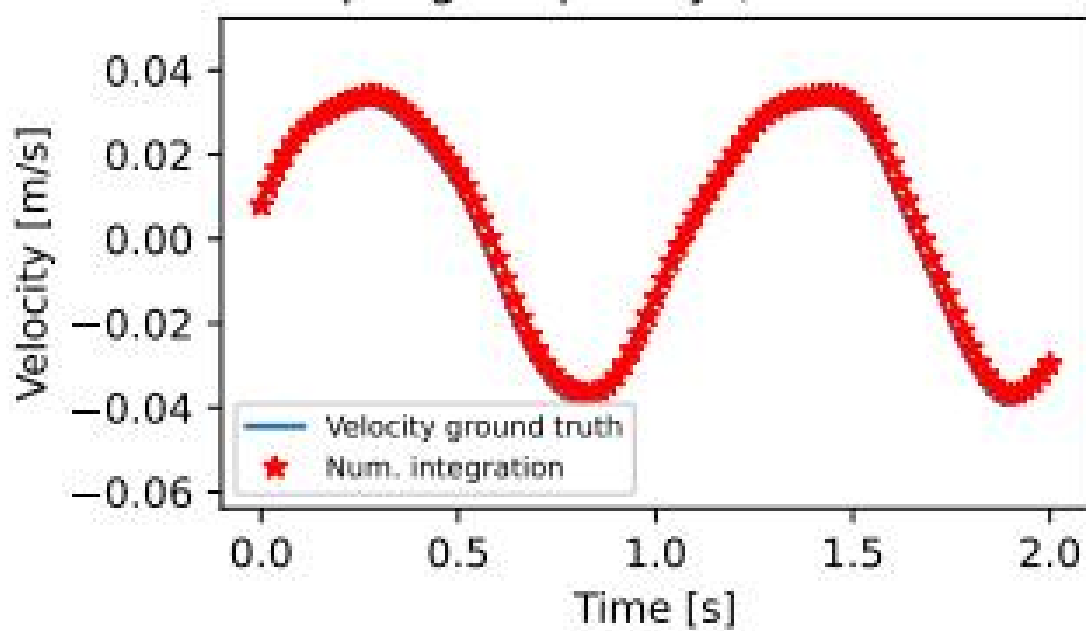


图 11.2: 图 11.2 已知初始条件下的 Euler 积分示例：上行采用低采样率，下行采用高采样率。随积分步长变小，离散积分更接近连续时间真值。

11.2.3.2 连续加速度预积分

另一种减小积分误差的方法，是采用不局限于离散时间截的连续时间表示，更准确地逼近真实惯性信号并进行解析积分。除提高精度外，连续时间表示还能异步查询预积分测量；当辅助传感器没有硬件同步，或具有完全异步的采样过程（例如事件相机）时，这一点尤其有用。

预积分的一项困难在于处理旋转空间：旋转运算不满足交换律，经典 Riemann 积分中的许多工具不能直接使用。因此，一些工作将预积分的旋转部分和平移部分分开。本节先假定旋转积分已经求得，讨论平移分量的连续时间积分；旋转空间中的连续积分留待下一小节。

^[300] 先用零阶积分器 ^[1105] 对陀螺仪测量积分，再通过求解连续时间线性时变微分方程，给出速度和位置预积分测量的连续表述。该工作讨论了常加速度计测量和局部坐标系恒加速度两种模型。与假设全局坐标系恒加速度的标准预积分 ^[335] 相比，局部坐标系恒加速度更符合真实场景；在 EuRoC 数据集 ^[131] 上，它相对于标准预积分和常加速度计测量模型均使整体 VIO 精度提高约 5%。

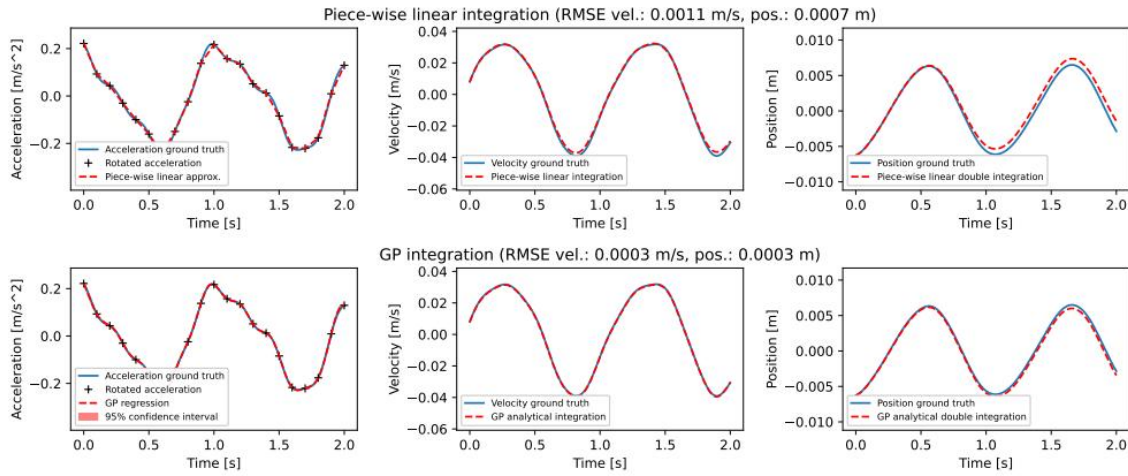


图 11.3: 图 11.3 上行：采用分段线性近似（对应恒加加速度运动假设）的连续积分；下行：采用高斯过程回归的无模型积分。

为进一步放宽恒加速度运动模型假设，可用能够解析积分的函数近似输入数据。在假定旋转预积分已求解的条件下，^[632] 连续表示旋转校正后的加速度计测量 $\hat{\mathbf{a}}_k = \Delta \mathbf{R}_{ik} \tilde{\mathbf{a}}_k$ 。图 11.3 展示了分段线性表示和基于 GP 的连续表示相对于图 11.2 中 Euler 方法的精度提升。分段线性近似从 $\hat{\mathbf{a}}_k$ 到 $\Delta \mathbf{v}_{ik}$ 的第一次积分就是经典梯形求积法；它可解释为恒加加速度运动模型，已经能显著提高积分精度。

若进一步采用 $\hat{\mathbf{a}} \sim \mathcal{GP}(0, k_a(t, t') \mathbf{I})$ ，其中 $k_a(t, t')$ 为平方指数协方差核，则可通过在线性算子作用下对 GP 进行推断，直接解析求得 $\hat{\mathbf{a}}$ 的积分和二重积分 ^[969]。平方指数核无限可微，因此该方法不依赖显式运动模型。图 11.3 下行表明，非参数 GP 模型比分段线性方法具有更高的积分精度。核函数超参数控制信号平滑度，可由数据学习或依据先验经验设定。

11.2.3.3 连续旋转预积分

连续表示提高了平移和速度预积分精度，自然也希望把这一思想扩展到旋转部分。然而，旋转 $\mathbf{R}$ 属于非欧氏的李群 $\text{SO}(3)$ ，群运算不满足交换律，不能像欧氏向量一样直接积分。求解运动学模型 (11.8) 的乘积积分为：

$$\mathbf{R}_b^w(t + \Delta t) = \mathbf{R}_b^w(t) \prod_t^{t+\Delta t} \text{Exp}(\omega_b^b(\tau))^{d\tau} \quad (11.35)$$

$$\dot{\mathbf{r}} = (\mathbf{J}_r(\mathbf{r}))^{-1} \omega_b^b, \quad (11.36)$$

式 (11.35) 不存在已知的通用解析解^[109]，因此需要新的方法在旋转空间中完成连续、无显式运动模型的积分。^[630] 将旋转写为 $\mathbf{R}(t) = \text{Exp}(\mathbf{r}(t))$ ，利用李代数中的旋转向量 $\mathbf{r}(t)$ 构成线性向量空间，并在其中使用线性工具进行连续积分。其动力学由 (11.36) 给出，其中 $\mathbf{J}_r(\mathbf{r})$ 是在 $\mathbf{r}$ 处计算的 $\text{SO}(3)$ 右雅可比。

IMU 并不直接观测 $\mathbf{r}$ 或 $\dot{\mathbf{r}}$ 。^[630] 的核心思想是用 GP 和一组虚拟观测 $\hat{\mathbf{r}}_t$ 对 $\dot{\mathbf{r}}$ 建模，再借助 GP 上的线性算子表示连续旋转向量函数。虚拟观测可理解为连续旋转动力学的控制点，通过非线性最小二乘问题估计；该问题采用基于 (11.36) 的残差，并把陀螺仪测量作为 ω_b^b 的观测。所得方法实现了无显式运动模型的连续旋转预积分，相比标准离散预积分至少提高一个数量级的精度。

这一连续方法与^[47]中的 STEAM 连续时间状态估计十分相似：二者均在李代数中进行基于 GP 的插值。主要区别是这里采用平方指数核，产生稠密线性系统，而 STEAM 使用稀疏的 Markov 表述。不过，IMU 预积分窗口通常足够短，求解稠密系统并非问题。^[376] 进一步扩展优化诱导值的思想，同时估计旋转向量与旋转校正后的加速度，从而能够刻画预积分测量协方差矩阵中旋转和平移部分的相关性。

11.3 辅助惯性导航的可观测性

受测量噪声、传感器偏置和数值积分误差影响，纯惯性里程计会迅速漂移，使用低精度惯性传感器时尤其如此。常见的抑制方法是把 IMU 与相机或 LiDAR 等外感传感器组合，形成辅助惯性导航系统 (AINS)。引入外感传感器通常还会扩大待估状态，例如增加外部路标变量，因此需要判断传感器数据是否足以无歧义地估计系统的 SLAM 状态。可观测性分析正是用于判定现有测量包含的信息能否无歧义地估计状态或参数^[117, 453]。

可观测性分析通常通过推导线性化测量模型并计算可观测性矩阵完成；该矩阵与状态估计的 Fisher 信息矩阵（以及协方差矩阵）密切相关^[487, 485]（见第 6 章）。系统可观测时，可观测性矩阵满秩；否则，研究其零空间即可确定估计器信息不足的状态空间方向。分析结果可用于提高估计一致性^[1234, 456, 653]、确定初始化估计器所需的最少测量^[456, 737]，以及识别会产生额外不可观测方向、实践中应避免或告警的退化运动^[1235]。因此，AINS^[1234]、特别是视觉惯性系统^[457, 654, 1235]的可观测性受到广泛研究。

本节讨论辅助 IMU 的传感器产生点、线和平面等几何特征时的可观测性。这一统一处理适用于相机和 LiDAR 等多种传感器，也有助于理解退化构型。为便于讨论，以包含 IMU、外感测量和一个或多个环境特征的 AINS 状态为例：

$$\mathbf{x} = \{\mathbf{R}_b^w, \mathbf{b}^g, \mathbf{v}^w, \mathbf{b}^a, \mathbf{p}^w, \mathbf{x}_f^w\} \quad (11.37)$$

11.3.1 线性化测量模型

假定辅助 IMU 的传感器前端输出基于路标的点、线或平面几何特征。大多数 AINS，特别是相机辅助系统，使用点特征^[456, 653, 645, 896, 375, 335]；条件允许时也可使用线和平面特征^[599, 455, 414, 1236]。此时需要把这些几何特征加入待估状态。式 (11.37) 中， $\mathbf{R}_b^w$ 表示机体坐标系相对于世界坐标系的旋转， $\mathbf{p}^w$ 和 $\mathbf{v}^w$ 分别是世界坐标系中的机器人位置与速度， $\mathbf{b}^g$ 和 $\mathbf{b}^a$ 是机体坐标系中的陀螺仪和加速度计偏置； $\mathbf{x}_f^w$ 可以是点、线、平面或它们的组合，并在世界坐标系中表达。

可观测性分析既需要由 IMU 加速度和角速度测量决定的系统动力学模型，也需要外感测量模型。下面先在线性化点附近线性化 IMU 运动学，再给出几何特征的外感测量方程。该分析既揭示理论上不可观测的自由度，也能指出由运动条件引起的退化。

11.3.1.1 线性化 IMU 运动学模型

由 IMU 运动学及偏置随机游走模型得到：

$$\dot{\mathbf{R}}_b^w = \mathbf{R}_b^w (\omega_b^b)^\wedge, \quad \dot{\mathbf{v}}^w = \mathbf{a}^w, \quad \dot{\mathbf{p}}^w = \mathbf{v}^w, \quad (11.38)$$

$$\dot{\mathbf{b}}^g(t) = \boldsymbol{\eta}^{bg}, \quad \dot{\mathbf{b}}^a(t) = \boldsymbol{\eta}^{ba} \quad (11.39)$$

在标称状态附近对误差状态一阶线性化，可写成连续时间系统：

$$\dot{\tilde{\mathbf{x}}}(t) \simeq \begin{bmatrix} \mathbf{F}_c(t) & \mathbf{0}_{15 \times n_f} \\ \mathbf{0}_{n_f \times 15} & \mathbf{0}_{n_f} \end{bmatrix} \tilde{\mathbf{x}}(t) + \begin{bmatrix} \mathbf{G}_c(t) \\ \mathbf{0}_{n_f \times 12} \end{bmatrix} \boldsymbol{\eta}(t) =: \mathbf{F}(t)\tilde{\mathbf{x}}(t) + \mathbf{G}(t)\boldsymbol{\eta}(t) \quad (11.40)$$

其中，误差状态向量 $\tilde{\mathbf{x}} = \{\tilde{\boldsymbol{\theta}}, \tilde{\mathbf{b}}^g, \tilde{\mathbf{v}}^w, \tilde{\mathbf{b}}^a, \tilde{\mathbf{p}}^w, \tilde{\mathbf{x}}_f^w\}$ （按列向量表示）描述状态相对于线性化点的偏差，例如 $\tilde{\mathbf{b}}^g$ 表示相对于线性化点的陀螺仪偏置变化；旋转部分在该线性化点使用切空间表示 $\tilde{\boldsymbol{\theta}}^8$ 。其中 n_f 是 $\tilde{\mathbf{x}}_f^w$ 的维数， $\mathbf{F}_c(t)$ 和 $\mathbf{G}_c(t)$ 分别是 IMU 状态的连续时间线性化转移矩阵与噪声雅可比矩阵， $\boldsymbol{\eta}(t)$ 是堆叠噪声，包含驱动陀螺仪偏置和加速度计偏置的噪声，以及用加速度计和陀螺仪测量替代式（11.38）中的实际加速度和角速度时产生的 IMU 噪声（推导见第 11.2.1 节）。

离散化后，相邻时刻误差状态转移矩阵具有分块结构：

$$\Phi_{(k+1,k)} = \begin{bmatrix} \Phi_{11} & \Phi_{12} & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_{n_f \times 3} \\ \mathbf{0}_3 & \mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_{n_f \times 3} \\ \Phi_{31} & \Phi_{32} & \mathbf{I}_3 & \Phi_{34} & \mathbf{0}_3 & \mathbf{0}_{n_f \times 3} \\ \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{0}_3 & \mathbf{I}_3 & \mathbf{0}_3 & \mathbf{0}_{n_f \times 3} \\ \Phi_{51} & \Phi_{52} & \Phi_{53} & \Phi_{54} & \mathbf{I}_3 & \mathbf{0}_{n_f \times 3} \\ \mathbf{0}_{3 \times n_f} & \mathbf{0}_{3 \times n_f} & \mathbf{0}_{3 \times n_f} & \mathbf{0}_{3 \times n_f} & \mathbf{0}_{3 \times n_f} & \mathbf{I}_{n_f} \end{bmatrix} \quad (11.41)$$

11.3.1.2 外感测量模型

外感传感器可测量点、线和平面等几何特征。以单目/双目相机、声呐或 LiDAR 提供的点特征为例，可将测量统一建模为传感器坐标系中路标相对位置的距离和/或方位观测：

$$\mathbf{z}_p = \underbrace{\begin{bmatrix} \lambda_r & \mathbf{0}_{1 \times 2} \\ \mathbf{0}_{2 \times 1} & \lambda_b \mathbf{I}_2 \end{bmatrix}}_{\Lambda} \begin{bmatrix} z_r \\ \mathbf{z}_b \end{bmatrix} = \Lambda \begin{bmatrix} \|\mathbf{p}_f^c\| + \eta^r \\ h_b(\mathbf{p}_f^c) + \eta^b \end{bmatrix} \quad (11.42)$$

$$\tilde{\mathbf{z}}_p = \mathbf{z}_p - \hat{\mathbf{z}}_p \simeq \Lambda \begin{bmatrix} \frac{\partial z_r}{\partial \mathbf{p}_f^c} & \frac{\partial p_f^c}{\partial \mathbf{x}} \\ \frac{\partial \mathbf{z}_b}{\partial \mathbf{p}_f^c} & \frac{\partial \mathbf{p}_f^c}{\partial \mathbf{x}} \end{bmatrix}_{\tilde{\mathbf{x}}} \tilde{\mathbf{x}} + \boldsymbol{\eta}^r =: \Lambda \begin{bmatrix} \mathbf{H}_r \\ \mathbf{H}_b \end{bmatrix} \mathbf{H}_f \tilde{\mathbf{x}} + \Lambda \begin{bmatrix} \eta^r \\ \eta^b \end{bmatrix} =: \mathbf{H}_x \tilde{\mathbf{x}} + \boldsymbol{\eta}^p \quad (11.43)$$

三维直线可使用 Plücker 坐标表示。其到原点的距离为 $d_\ell^w = \|\mathbf{n}_\ell^w\|/\|\mathbf{v}_\ell^w\|$ ，并可依据相机位姿从世界坐标系变换到相机坐标系^[1021]：

$$\mathbf{l}^w = \begin{bmatrix} \mathbf{n}_\ell^w \\ \mathbf{v}_\ell^w \end{bmatrix} = \begin{bmatrix} \mathbf{p}_1^w \times \mathbf{p}_2^w \\ \mathbf{p}_2^w - \mathbf{p}_1^w \end{bmatrix} \quad (11.44)$$

$$\begin{bmatrix} \mathbf{n}_\ell^c \\ \mathbf{v}_\ell^c \end{bmatrix} = \begin{bmatrix} \mathbf{R}_c^{w^\top} & -\mathbf{R}_c^{w^\top}(\mathbf{p}_c^w)^\wedge \\ \mathbf{0} & \mathbf{R}_c^{w^\top} \end{bmatrix} \begin{bmatrix} \mathbf{n}_\ell^w \\ \mathbf{v}_\ell^w \end{bmatrix} \quad (11.45)$$

给定图像中线段的两个端点 $\mathbf{q}_1 = [u_1, v_1, 1]^\top$ 和 $\mathbf{q}_2 = [u_2, v_2, 1]^\top$ ，二维直线测量可表示为这两个端点到投影直线的距离^[1315]。使用已知相机内参将三维 Plücker 直线投影到图像平面^[1021]：

$$\boldsymbol{\ell} = \underbrace{\begin{bmatrix} f_2 & 0 & 0 \\ 0 & f_1 & 0 \\ -f_2 c_1 & -f_1 c_2 & f_1 f_2 \end{bmatrix}}_{\mathbf{K}} \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \end{bmatrix} \begin{bmatrix} \mathbf{n}_\ell^c \\ \mathbf{v}_\ell^c \end{bmatrix} =: \begin{bmatrix} \ell_1 \\ \ell_2 \\ \ell_3 \end{bmatrix} \quad (11.46)$$

$$\mathbf{z}_\ell = \begin{bmatrix} \frac{\mathbf{q}_1^\top \boldsymbol{\ell}}{\sqrt{\ell_1^2 + \ell_2^2}} \\ \frac{\mathbf{q}_2^\top \boldsymbol{\ell}}{\sqrt{\ell_1^2 + \ell_2^2}} \end{bmatrix} + \boldsymbol{\eta}^\ell \quad (11.47)$$

平面特征可用世界坐标系中的单位法向量及到原点的有符号距离表示，并转换到局部传感器坐标系。若平面由 LiDAR 或深度传感器等点云中提取，可采用从平面到原点的最近点 $\mathbf{p}_\pi^c = d_\pi^c \mathbf{n}_\pi^c$ 作为 AINS 状态中的平面表示^[374]：

$$\begin{bmatrix} \mathbf{n}_\pi^c \\ d_\pi^c \end{bmatrix} = \begin{bmatrix} \mathbf{R}_w^c & \mathbf{0}_{3 \times 1} \\ -(\mathbf{p}_\pi^c)^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{n}_\pi^w \\ d_\pi^w \end{bmatrix} \quad (11.48)$$

$$\mathbf{z}_\pi = d_\pi^c \mathbf{n}_\pi^c + \boldsymbol{\eta}^\pi = \mathbf{p}_\pi^c + \boldsymbol{\eta}^\pi \quad (11.49)$$

11.3.2 可观测性分析

根据前述线性化系统与测量模型，可以构造可观测性矩阵以分析系统^[486]。该矩阵堆叠沿轨迹各离散时刻的测量雅可比与状态转移矩阵：

$$\mathbf{M}(\hat{\mathbf{x}}) = \begin{bmatrix} \mathbf{H}_{x_1} \boldsymbol{\Phi}_{(1,1)} \\ \mathbf{H}_{x_2} \boldsymbol{\Phi}_{(2,1)} \\ \vdots \\ \mathbf{H}_{x_k} \boldsymbol{\Phi}_{(k,1)} \end{bmatrix} \quad (11.50)$$

对于一般运动和典型点、线、平面特征，AINS 至少具有四个固有不可观测自由度：全局平移三维和绕重力方向的全局偏航。它们对应可观测性矩阵的零空间：

$$\text{null}(\mathbf{M}) = \text{span}[\mathbf{u}_1 \mathbf{u}_{2:4}] = \text{span} \begin{bmatrix} \mathbf{u}_g & \mathbf{0}_{12 \times 3} \\ -\mathbf{p}_1^w \times \mathbf{g}^w & \mathbf{I}_3 \\ -\mathbf{p}_f^w \times \mathbf{g}^w & \mathbf{I}_3 \\ -\mathbf{g}^w & \frac{\mathbf{v}_\ell^w}{d_\ell^w \|\mathbf{v}_\ell^w\|} (\mathbf{R}_\ell^w \mathbf{e}_1)^\top \\ 0 & -(\mathbf{R}_\ell^w \mathbf{e}_3)^\top \\ -d_\pi^w \mathbf{n}_\pi^w \times \mathbf{g}^w & \mathbf{n}_\pi^w (\mathbf{R}_\pi^w \mathbf{e}_3)^\top \end{bmatrix} \quad (11.51)$$

其中， $\mathbf{u}_g$ 表示绕重力方向的不可观测旋转，其构造使用时刻 $k = 1$ 的传感器位置 $\mathbf{p}_1^w$ 、从传感器坐标系 C_1 到世界坐标系 W 的旋转 $\mathbf{R}_{C_1}^w$ 以及重力和速度向量。 $\mathbf{R}_\pi^w$ 是利用平面法向量 $\mathbf{n}_\pi^w$ 通过 Gram-Schmidt 正交归一化（参见式 (11.6)）构造的旋转矩阵； $\mathbf{R}_\ell^w$ 则由直线法向量 $\mathbf{n}_\ell^w$ 和直线方向 $\mathbf{v}_\ell^w$ 构造，其三列分别为归一化法向量、归一化方向向量及二者的叉积。可以看出，第一个零向量 $\mathbf{u}_1$ 与绕重力方向的旋转（因而与偏航）相关，而 $\mathbf{u}_{2:4}$ 与机器人的运动相关；更详细的分析见^[1234, 1233]。

总之，可观测性矩阵具有四维零空间（参见式 (11.51)），准确表达了系统全局位置和偏航不可观测这一事实。直观地说，IMU 数据以及未知点、线、平面路标的测量都不包含全局坐标系信息；唯一例外是由加速度计测得的重力方向，它使滚转和俯仰可观测。这种不可观测性在 SLAM 中很常见⁹，并非病态现象：它只表示由于这些变量只有相对测量，我们可以任意设置世界坐标系的偏航和三维原点。加入提供绝对测量的传感器（例如 GPS）后，这种不可观测性会消失。更值得关注的是，对于某些运动（和线性化点），可观测性矩阵的零空间可能扩大，从而产生额外的不可观测维度；下一节将讨论这种现象。

这些不可观测方向不是算法缺陷，而是由相对传感器测量的物理对称性决定的。即使使用不同的优化器或滤波器，也无法仅凭此类测量消除全局位置和航向基准的不确定性。

11.3.3 退化运动

某些运动会给 AINS 引入额外不可观测方向，除前述预期的四维自由度外，还会使状态空间某些方向产生大误差并导致导航失败。表 11.1 总结了 AINS 的退化运动（完整推导见^[1234]）。纯平移对所有特征类型都退化，使完整全局方向不可观测；直观而言，系统不旋转时，重力测量与加速度计偏置会相互混淆，滚转和俯仰也不再可观测。恒加速度（包括零加速度的匀速运动）、纯旋转和朝向单点特征运动，在单目相机（仅方位测量）情况下使尺度不可观测。其中恒加速度使整个系统（位置、速度、加速度偏置和特征）的尺度不可

观测；纯旋转和朝向特征运动则只使特征尺度不可观测。这后三种退化仅在传感器到特征距离显著大于传感器与机器人机体间外参平移时成立，即 $\|p_f^c\| \gg \|p_e\|$ ，实践中通常满足。

表 11.1 AINS 的退化运动。

运动	传感器	不可观测量
1. 纯平移	通用	全局方向
2. 恒加速度	单目相机	系统尺度
3. 纯旋转	单目相机	特征尺度
4. 朝向点特征运动	单目相机	特征尺度

11.4 视觉惯性里程计与实际考虑

如上所述，惯性测量通常与其他传感器数据融合，以抑制里程计漂移。本节特别讨论使用因子图融合相机视觉测量与 IMU 测量的情形¹⁰。相机和 IMU 都价格低、重量轻、功耗低，而且具有互补性：IMU 能捕获快速加速度和转动，相机则能提供丰富的环境观测。一方面，相机能相较纯惯性里程计显著减少漂移；另一方面，IMU 可使原本无法估计的量变得可观测。特别是在单目相机 SLAM 中，若无先验信息就无法估计场景尺度（尺度不可观测）；只要机器人运动不退化（第 11.3.3 节），加入 IMU 即可恢复尺度。由一个或多个相机及 IMU 组成的系统通常称为视觉惯性里程计（VIO）系统；纳入回环后则成为视觉惯性 SLAM 系统。

11.4.1 视觉惯性里程计

VIO 系统常作为里程计来源，并常用于轨迹跟踪与控制的闭环。在虚拟现实等应用中，VIO 用于补偿用户在虚拟环境中的运动。两种情形都要求极低延迟的估计，通常约为 10–50 ms。例如 Meta Quest 3 的刷新率介于 72 Hz 和 120 Hz^[761]，VIO 延迟直接影响 VR 体验质量，并是缓解晕动症的关键。对于轨迹跟踪，保持低延迟同样重要，因为较大延迟可能引起跟踪控制器不稳定和发散。

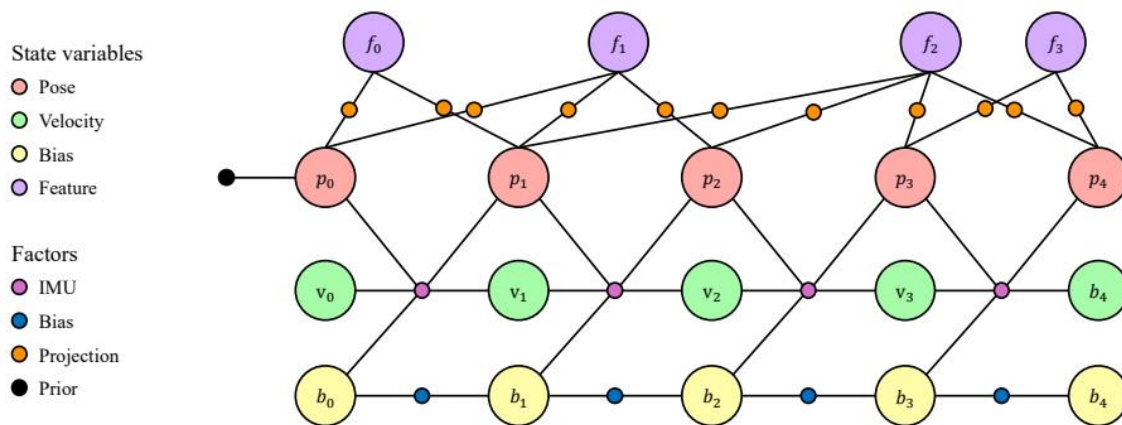


图 11.4: 图 11.4 使用预积分 IMU 因子的视觉惯性里程计因子图示例^[335]。紫色为预积分 IMU 因子，约束连续位姿、速度和偏置；蓝色为偏置因子，约束 IMU 偏置随时间的演化；橙色为视觉因子，关联相机位姿与外部地标位置；黑色为先验。

基于这些考虑，因子图 VIO 系统通常实现固定滞后平滑器（也称滑动窗口优化），只在滚动时间窗内估计状态（例如最近 5–10 秒）。图 11.4 展示了相应因子图。时间窗长度需要在计算量和精度之间权衡：窗口越长，待估状态空间越大。随着时间推进，超出滚动窗口的因子和变量被逐步边缘化。很多优化实现还通过 Schur 补消去视觉地标，以进一步缩小状态空间，例如^[335]。固定滞后平滑器的替代方案是 iSAM2 等增量求解器（第 1.7 节），它在计算当前时刻估计时复用先前优化结果。该方法实践中可获得很高精度^[335]，但无法保证系统延迟，运行时间可能突增，对某些应用不利。

VIO 系统与性能。过去十年中，视觉惯性里程计/SLAM 系统大量涌现，其中许多拥有开源实现。常用方案包括 ORB-SLAM 的视觉惯性版本^[786]、Direct Sparse Visual-Inertial Odometry^[1116]、VINS-Mono^[896]、OpenVINS^[375]、Kimera^[3, 942]、BASALT^[1118] 和 DM-VIO^[1043]。优秀 VIO 系统的漂移低于行驶距离的 1%（例如行驶 100 m 后累计误差小于 1 m），在一些情况下可低至 0.1%。

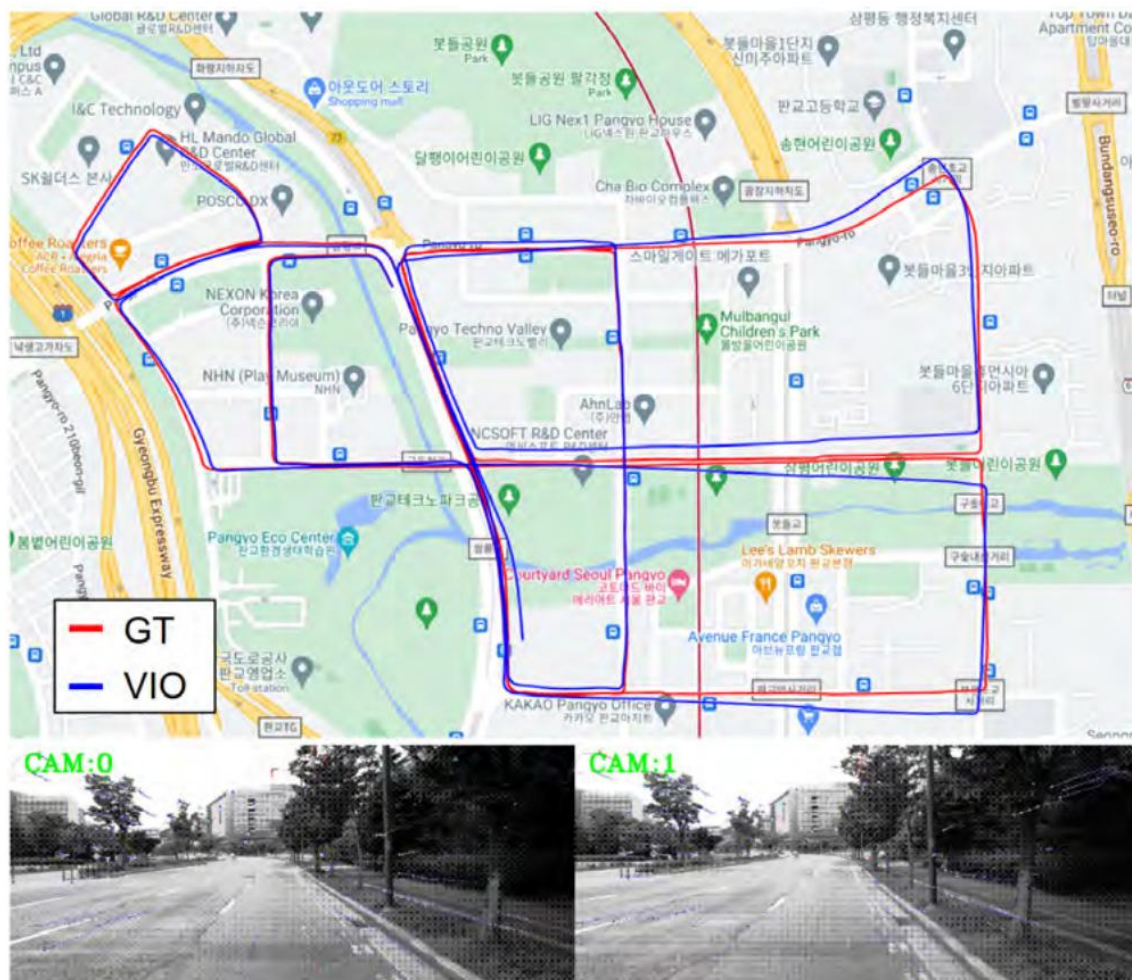


图 11.5: 图 11.5 近期基于滑动窗口优化的 VIO 算法^[180] 在 KAIST 城市自动驾驶数据集第 38 序列上的示意图。该序列持续 36 分钟，长度为 11.42 km；VIO 估计和真值叠加在 Google 地图上，底部为两幅样例图像。VIO 的最终 ATE（不含回环）为 2.05 度和 21.2 m（0.18%）。

VINS-Mono^[896] 等滑动窗口优化方法已在实践中取得巨大成功。以 First-Estimate Jacobian (FEJ) - based Window Bundle Adjustment (WBA) - VINS^[180, 181] 为例，它在 KAIST Urban Dataset^[515] 上运行。该数据集面向挑战性复杂城市环境下的自动驾驶与定位，由装有双目相机、二维/三维 LiDAR、Xsens IMU、光纤陀螺 (FoG)、轮编码器和 RTK GPS 的车辆在韩国采集。相机以 10 Hz 运行，IMU 感知速率为 100 Hz；真值轨迹由 FoG、RTK GPS 和轮编码器融合获得。图 11.5 将第 38 序列的 FEJ-WBA-VINS^[180, 181] (VIO) 估计轨迹与真值轨迹叠加于 Google 地图。对 11.42 km 路径，最终绝对轨迹误差约为 2.05 度和 21.2 m（所行轨迹的 0.18%）；值得注意的是，这些结果来自不使用回环的纯在线 VIO。

^[3, 4] 还讨论了 VIO 在自动驾驶汽车及其他自主系统中的应用，以及特征跟踪、关键帧选择和不同传感器模式融合（包括单目、双目与 RGB-D 相机图像，以及轮式里程计）相关挑战。

11.4.2 外参标定

为实现准确的辅助惯性导航，必须进行传感器外参标定。外参标定是估计不同传感器之间的相对位姿，例如相机相对于 IMU 的位姿。相关方法大致分为离线与在线标定。离线方法要求在系统部署前执行标定流程，常使用标定靶^[350]、已知运动模式^[686]或环境先验^[631, 714]；流程可能耗时，且需要专用设备与受训操作人员。因此，尽管通常更准确，离线标定对希望由非专业人员大规模使用的系统可能繁琐且不受欢迎。在线方法则无需专门流程^[301, 633, 1211, 1237]，而是在状态估计问题中把外参标定参数一并估计。它能适应传感器位移等系统变化，无需重新标定；但在线法精度可能低于离线法，并可能使状态估计问题更复杂甚至病态^[1235]。

11.4.3 时间同步

惯性辅助系统的另一关键问题是传感器数据的时间同步。未被建模的错误同步会导致轨迹估计出现显著误差和/或在基准指标中引入偏差。同步可通过硬件或软件完成。底层硬件方法通常依赖专用硬件，通过专用同步输入引脚以共同的时钟信号触发多个传感器的数据采集；当传感器经不同通信协议连接到计算机时，这并非总能实现。一些传感器自带同步机制，可无需专用硬件输入便同步不同传感器的时钟。PTP (Precision Time Protocol) 是在以太网上进行软件同步的一个例子，许多 LiDAR、雷达和 INS 方案均可通过该协议同步。另一种方案是在传感器层级给数据加时间戳，再在后处理中对齐时间戳；它通常不如前述方案准确和鲁棒。若系统不能同步且无法后处理（例如在线应用），一些状态估计算法会把时间偏移量作为状态变量纳入估计^[301, 376, 1237]。

11.5 延伸阅读与近期趋势

惯性里程计的进展正稳步进入工业产品，但辅助惯性导航仍是活跃研究领域。

扩展位姿预积分。面向 SLAM 的惯性里程计近期趋势包括使用扩展位姿流形和高阶噪声传播^[120]，改善 IMU 预积分的不确定性建模。Brossard 等^[120]扩展预积分理论以考虑地球自转及科里奥利力、离心力。Vial 等^[1126]给出结合线速度传感器与导航级 IMU 的扩展位姿预积分实例；在 1.8 km 海上航迹航行一小时后，报告的平移误差约为 5 m。

连续时间状态表示。本章主要从预积分角度使用 IMU，以减少因子图中离散状态变量数目。但连续时间状态表示也能在不增加估计状态维度的情况下处理大量 IMU 测量。例如，^[349]使用 B 样条基函数，^[47]使用高斯过程先验；两种表述都可在基于固定状态变量集之间插值动力学的残差中，以高频使用 IMU 测量。^[130]比较了把 IMU 测量作为连续时间 GP 先验的直接输入，与直接在残差中使用 IMU 测量两种方式，并得出在 LiDAR-惯性传感器组合中把惯性信息作为状态测量可获得更好的里程计精度。^[659]比较了^[47]的 GP 状态表示和^[376]的连续 GP 预积分；在事件 VIO 情形中，后者在精度和计算效率上都略有优势。

仅本体感知的里程计。近期工作利用本体传感器进行辅助惯性导航。用于里程计时，^[441]的足式机器人和^[813]的轮载 IMU 表明，可利用系统运动学知识提供具有亚百分比位置误差的有竞争力的 IMU 里程计估计。在^[441]中，关键信息是机器人足端与地面的接触；在^[813]中，单平面旋转运动用于约束 IMU 偏置，从而限制航位推算漂移。^[813]还被扩展为完整 SLAM 系统^[1200]，通过道路横坡角随时间的模式识别检测回环，展示了能进行回环检测和校正的基于 IMU 本体感知系统。需注意，惯性传感器通常能提高性能和鲁棒性，但 IMU 丢失数据或饱和会对整个系统造成灾难性影响。^[266]研究了陀螺仪饱和时使用加速度计数据估计角速度的方法，从而提高下游 SLAM 算法的鲁棒性。

纯惯性里程计 (IOO)。对 IMU 测量进行朴素积分，而没有视觉等辅助信息，通常会使得里程计估计迅速发散；即使在辅助惯性里程计中，辅助来源不可用时也存在此问题。例如，在移动 AR/VR 的手部跟踪中，高动态手部很容易离开跟踪相机视场，只有 IMU 数据可维持运动跟踪；无纹理场景也会妨碍特征检测和跟踪，使 VIO 只能依赖 IMU。因此，近期工作研究利用学习和神经网络降低纯惯性里程计漂移^[1215, 179, 1055, 451, 452, 220, 898]，包括用神经网络以数据驱动方式建模 IMU 偏置^[225]，或从带噪 IMU 测量序列直接预测位移^[682]。例如，可借助可微积分模块积分去除预测偏置的 IMU 读数^[1276, 898]，使用真值偏置监督^[123]，或使用条件扩散模型近似作为概率分布建模的偏置^[1298]。这些方法表明可大幅降低纯惯性里程计漂移，但目前泛化能力有限，例如难以泛化至不同传感器或训练时未见的运动。

边缘端超高效且鲁棒的 VIO。尽管 SLAM 已取得进展，嵌入式机器人系统的计算约束仍是关键挑战。由于严格的尺寸、重量和功耗 (SWaP) 限制，在小尺寸平台构建鲁棒 VIO 很困难，主要难点往往是数据管理而非计算。例如，在 Meta XR 可穿戴设备的 SLAM 和手部跟踪模块中，主要能耗来自 RAM 数据访问 [623]。为减少数据传输，[390] 提出了传感器上计算架构，[865, 867] 开发了量化视觉惯性里程计 (QVIO) 算法。对于计算单元仅支持单精度浮点运算、或需要借此加速并实现实时性能的低 SWaP 平台，研究者提出了新的平方根（信息或协方差）滤波器 [866, 1194]，以在保持数值稳定性的同时提高效率。[1279, 1048] 给出了片上视觉惯性里程计系统的 ASIC 设计与实现。

¹ 可观测性说明在何种条件下估计问题是适定的，即给定测量时，是否可能计算出接近真值的估计。² IMU 通常还包括测量磁北方向的罗盘。由于室内和城市等许多机器人应用中存在大型金属结构和电子设备造成的局部磁扰动，该传感器会呈现较大偏置，因此在 SLAM 中较少使用。³ 航空航天通常区分非惯性导航坐标系（如 Earth-Centered Earth-Fixed, ECEF 和 Local Geodetic Vertical, LGV）与惯性坐标系（如 Earth-Centered Inertial, ECI）[318]。机器人近地小尺度应用常使用噪声较大的低成本 IMU，地球自转影响相对测量噪声可以忽略，因此通常将固定在地球某处的世界坐标系 $\mathcal{F}^w$ 近似视为惯性坐标系。⁴ 更严格地说，此处隐含假设为角速度方向在区间内不变；否则旋转积分一般需使用时间有序指数。⁵ 此处“关键帧状态”不必等同于视觉前端检测到的图像关键帧，而是指因子图中选取的离散状态节点。⁶ 为简化起见，假设 IMU 与其他传感器已同步，且 IMU 测量恰在时刻 t_i 和 t_j 采样。实际中可对测量进行插值，以近似这一条件；时间同步的进一步讨论见第 11.4.3 节。⁷ 高斯过程回归是一种用于插值的非参数概率方法；关于高斯过程回归的深入介绍，参见 [909]。⁸ 直观地说，为线性化旋转变量，可将任意旋转写成线性化点处旋转的扰动： $\mathbf{R}_b^w = \hat{\mathbf{R}}_b^w \text{Exp}(\tilde{\boldsymbol{\theta}})$ ，其中 $\tilde{\boldsymbol{\theta}}$ 是适当的切空间向量；再使用小角度近似 $\text{Exp}(\tilde{\boldsymbol{\theta}}) \simeq \mathbf{I} + \tilde{\boldsymbol{\theta}}^\wedge$ 。⁹ 不使用 IMU 时，基于路标的 SLAM 问题的零空间至少为六维，因为除三维位置外，系统的完整三维旋转也不可观测。¹⁰ LiDAR-惯性等其他辅助惯性组合也遵循相同基本原则。

第十二章 面向 SLAM 的腿部里程计

作者：Marco Camurri 和 Matías Mattamala

足式机器人凭借穿越高度非结构化地形的能力正日益普及。其主要优势在于，腿部提供主动悬架，使机器人机体运动与地形轮廓解耦^[988]。因此，足式机器人能够通过楼梯、不平整地面和其他轮式平台难以应对的地面障碍^[218, 500]。虽然其他章节回顾的 SLAM 算法同样适用于足式机器人，但腿部带来的额外感知是可用于里程计的有价值新信息源。对于足式运动控制与规划，这一点尤其重要：为避免跌倒和失效，需要高频、低漂移的实时位姿和速度估计，而这本身颇具挑战。

本章讨论配备机载 IMU 和关节传感器（位置与力矩）的足式机器人实时位姿与速度估计基础，重点关注腿部里程计，即根据腿部感知确定机器人机体的相对运动。首先给出历史背景和预备知识（第 12.1 节），然后介绍估计腿部里程计所需的关键理论工具（第 12.2 和 12.3 节）。在已知支撑腿的前提下，解释如何由腿部关节测量获得相对运动（第 12.2 节），从实践上可将腿部里程计作为 SLAM 问题中的一项测量，类似轮式里程计或惯性预积分。随后描述接触状态的估计及实践中采用的主要技术（第 12.3 节），讨论如何使用因子图将腿部里程计与不同传感器模态融合（第 12.4 节）。最后回顾领域中的开放问题（第 12.5 节），并讨论应对这些问题的新趋势（第 12.6 节）。

12.1 历史背景与预备知识

尽管现代多数足式平台都搭载其他章节研究过的传感器，如相机（第 7 章）、LiDAR（第 8 章）和惯性测量单元（第 11 章），足式平台还可以利用机器人腿部的运动学和动力学信息，获取机体相对运动测量，这一技术称为腿部里程计。该术语类比轮式车辆的里程计：轮式车辆通过测量车轮随时间的转动量推断行驶距离。

与轮式车辆不同，足式机器人通过腿部与地面不断建立和解除接触来运动。每个腿步周期包含一个腿暂时离开地面的阶段（腾空或摆动阶段），随后进入与地面无滑动接触的阶段（支撑阶段）。支撑阶段的腿负责推动机器人前进，因此腿部里程计可分解为两个子问题：

1. **接触估计：**确定给定时间段内哪些腿处于支撑阶段。2. **运动估计：**根据这些腿在支撑阶段的测量确定增量运动。

在深入腿部里程计之前，本节简要回顾历史背景和预备概念。

12.1.1 历史背景

腿部里程计与行走机器的发展直接相关。足式机器人起源于 20 世纪 50、60 年代；当时人们首次尝试制造能够克服崎岖地形中轮式车辆局限的运输系统^[674]。通用电气的 Walking Truck^[776] 重 1400 kg，是这一时期设计的人操作机器的典型例子。随着 20 世纪 60 年代新控制策略的发展，研究转向能够自动行走的小型平台^[750, 751, 1100]。20 世纪 80 年代末 Raibert 开发的单足、双足和四足机器人展现了卓越的运动能力^[902]，推动足式平台用于现实任务。实现现实世界自主性成为发展腿部里程计和状态估计系统的主要驱动力。

Roston 和 Krotkov^[947] 最早利用腿部运动学估计足式平台运动之一，研究对象是用于太空探索、重 2.5 吨的 Ambler 六足机器人。随后，研究者将类似方法用于具有不同腿部形态的其他六足机器人，例如 RHex^[669]。后续工作通过粒子滤波器^[208] 或卡尔曼滤波器^[202, 926] 将腿部里程计与 IMU、视觉等其他传感器模态结合。Bloesch 等^[88] 的里程碑工作为四足机器人建立了基于滤波、适用于全地形的运动学-惯性里程计估计器基础^[89]，并为扩展到双足平台^[948, 874] 铺平道路。

2012–2015 年开展的 DARPA Robotics Challenge (DRC) 推动了人形机器人全身状态估计系统的发展, 目标不仅是估计 6 自由度位姿, 还要估计机器人身体的完整状态。为此, 各团队将运动学和动力学信息^[1203]与惯性和关节感知相结合, 并融合 LiDAR^[315]、双目视觉^[316] 等外感模态。当时多数方案依赖卡尔曼滤波, 但也有少量工作探索基于优化的方法^[1204] 和因子图^[336]。

近年来, 特别是四足平台的兴起和商业化, 使发展更具原理性、鲁棒性更强的腿部里程计和状态估计算法受到关注。四足^[1191] 和双足^[439] 平台的因子图估计方案不断发展, 支持与其他传感器模态进行有原则的融合。另一些方向研究建模方面更基础的挑战^[19] 及不变估计框架^[441, 438]。近期 DARPA Subterranean (SubT) 挑战赛 (2018–2021) 还在极端场景下为足式状态估计提出新问题, 不同足式、轮式和空中平台利用互补传感器模态构建联合估计方案^[558, 1287, 838]。第 12.6 节将进一步讨论这些趋势及其对足式平台状态估计研究的重塑作用。

12.1.2 参考坐标系

图 12.1 展示与估计问题相关的参考坐标系。惯性坐标系 $\mathcal{F}^w$ 和基座坐标系 $\mathcal{F}^b$ 分别刚性连接到地面和机器人浮动基座。无损失地, 假设 IMU 与 $\mathcal{F}^b$ 重合。每个末端执行器都附着有一个坐标系, 对应各足端 (图 12.1 示例中的 $\mathcal{F}^{f1}$ 和 $\mathcal{F}^{f2}$)。

此外, 当足端与地面接触时, 会创建一个或多个临时惯性坐标系 $\mathcal{F}^k$ 。对人形机器人, 该坐标系在触地时与足端坐标系重合; 对点足四足机器人, 它与足端具有相同笛卡尔位置。

12.1.3 状态定义

足式机器人的状态由基座位姿和速度以及关节状态定义。本章假设关节状态由专用传感器直接测量 (第 12.1.6 节), 因此估计目标仅为机器人基座位姿和速度。于是, “状态估计” 和 “里程计” 可互换使用, 后者等价于不含回环的 SLAM^[136]。

更形式化地, 机器人状态由位置、方向、线速度和角速度组成:

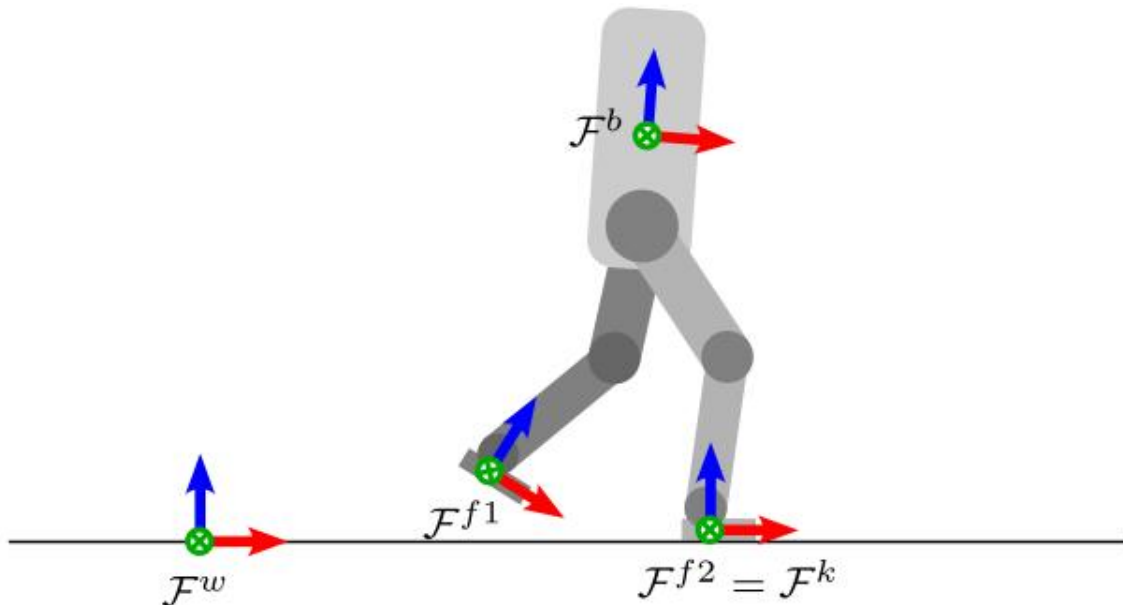


图 12.1: 图 12.1 足式机器人的参考坐标系约定。世界坐标系 $\mathcal{F}^w$ 固定于地球, 基座坐标系 $\mathcal{F}^b$ 附着在主机架上。无损失地, 图中未显示 IMU 坐标系, 因为可认为其与 $\mathcal{F}^b$ 重合。足端接触地面时定义接触坐标系 $\mathcal{F}^k$; 它刚性固定于地面、垂直于地面, 并与足端坐标系 $\mathcal{F}^{f2}$ 重合。

$$\mathbf{x}_k = \begin{bmatrix} t_k & \mathbf{R}_k & \mathbf{v}_k & \boldsymbol{\omega}_k \end{bmatrix}^\top \quad (12.1)$$

约定机器人位置 $t = t_b^w \in \mathbb{R}^3$ 和方向 $R = R_b^w \in \text{SO}(3)$ 表示世界坐标中的基座位姿；机器人速度 $v = v_b^b$ 、 $\omega = \omega_b^b \in \mathbb{R}^3$ 在基座坐标中表示基座的 twist。

12.1.4 足式机器人运动学

足式机器人由承载身体的主连杆（也称躯干）和连接在其上的一条或多条运动链（腿）组成。本章考虑实践中最常见的配置：每条腿具有 6 个驱动自由度且足端为平足的双足机器人，以及每条腿具有 3 个自由度且足端为点足的四足机器人（图 12.2）。假设机器人为刚体（例如没有可动脊柱），忽略其他上肢（例如手臂）。

腿部里程计关注建模平足或点足相对于机器人身体的相对位姿（或位置）。令 $q \in \mathbb{R}^N = [q_1, \dots, q_N]^T$ 为具有 N 个主动自由度的关节型机器人关节位置集合，对应腿部转动关节的角位置。本章图 12.1 及后文中，四足和双足机器人的主动自由度均取 $N = 12$ ，其他平台可以不同。关节位置 q 通常由安装在每个关节上的旋转编码器直接测量（第 12.1.6.1 节）；也可利用机器人运动学模型和电机与关节之间传动机构上的编码器间接测量，例如通过线性编码器测量液压活塞位移，再计算活塞驱动的转动关节角度。

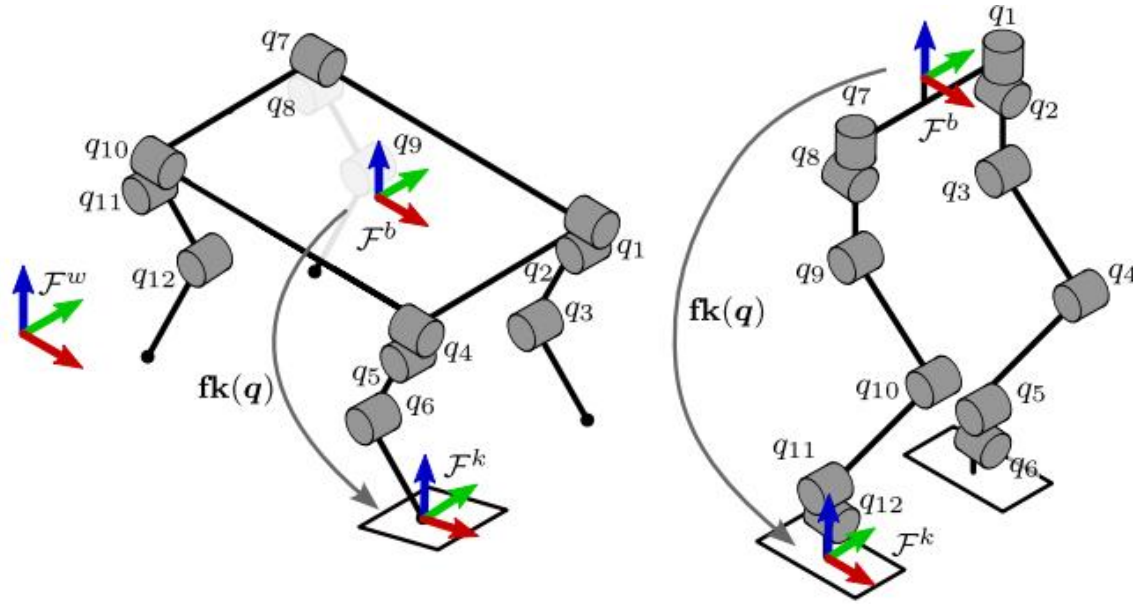


图 12.2: 图 12.2 典型足式机器人的运动链：四足机器人和人形机器人。

关节位置的时间导数是关节速度 $\dot{q} = [\dot{q}_1, \dots, \dot{q}_N]^T$ ，通常通过对编码器读数数值微分估计。当读数噪声很大时，也可利用安装在连杆上的 IMU 等额外传感器估计关节速度^[1205]。

给定 q ，对每只足端 f 定义正向运动学函数 $\text{fk}(q) : \mathbb{R}^{12} \rightarrow \text{SE}(3)$ ^[717]，将关节位置映射为足端相对于机器人基座的位姿：

$$T_f^b = \text{fk}(q) = \begin{bmatrix} f_R(q) & f_p(q) \\ \mathbf{0} & 1 \end{bmatrix} \quad (12.2)$$

其中 $f_R : \mathbb{R}^{12} \rightarrow \text{SO}(3)$ 和 $f_p : \mathbb{R}^{12} \rightarrow \mathbb{R}^3$ 分别表示足端在基座坐标中的方向和笛卡尔位置。对点足四足机器人，腿部里程计只使用 $f_p(q)$ ，因为足端可绕接触点转动而关节位置不变。

足端 f 的正向运动学函数 (12.2) 的时间导数是雅可比矩阵 $J(q)$ ，可用于计算单个足端相对于基座的速度^[718]：

$$\begin{bmatrix} v_f^b \\ \omega_f^b \end{bmatrix} = J(q) \dot{q} = \begin{bmatrix} J_v(q) \\ J_\omega(q) \end{bmatrix} \dot{q} \quad (12.3)$$

其中 $J_v(q) \in \mathbb{R}^{3 \times 12}$ 和 $J_\omega(q) \in \mathbb{R}^{3 \times 12}$ 分别是雅可比矩阵的线速度和角速度部分（一些参考文献^[319, 441]先定义角速度块）。由于 q 表示全部 $N = 12$ 个关节的位置，而每条腿在运动学上相互独立，因此 $J(q)$ 是稀

疏分块矩阵，非零块记为 $\bar{J}(q)$ 。这些块可将某条腿的关节角速度子集映射为对应足端的线速度和角速度。例如，人形机器人第二条腿的雅可比矩阵为：

$$J_2(q) = \begin{bmatrix} \mathbf{0}_6 & \bar{J}_2(q) \end{bmatrix} = \begin{bmatrix} J_{2,v}(q) \\ J_{2,\omega}(q) \end{bmatrix} = \begin{bmatrix} \mathbf{0}_{3 \times 6} & \bar{J}_{2,v}(q) \\ \mathbf{0}_{3 \times 6} & \bar{J}_{2,\omega}(q) \end{bmatrix} \quad (12.4)$$

其中 $\bar{J}_2(q) \in \mathbb{R}^{6 \times 6}$ 表示雅可比矩阵的非零块。式 (12.2)–(12.3) 是设计足式平台状态估计器的基础，它们建立了关节状态与机器人基座运动的关系。本章假设关节状态可以通过编码器直接测量，详见第 12.1.6 节。

12.1.5 足式机器人动力学

浮动基座刚体系统的动力学可表示为两条耦合动力学方程，并使用递归 Newton–Euler 算法计算^[319]。第一条描述浮动基座刚体（欠驱动的 6 自由度）的动力学，第二条描述通过主动关节连接在其上的 $N = 12$ 个刚体（主动自由度）的动力学。两条运动方程可写为矩阵形式：

$$M(q) \begin{bmatrix} \dot{v} \\ \dot{\omega} \\ \ddot{q} \end{bmatrix} + h(q, \dot{q}) = \begin{bmatrix} J_b^T \\ J_q^T \end{bmatrix} f + \begin{bmatrix} \mathbf{0}_6 \\ \tau \end{bmatrix} \quad (12.5)$$

第一项 $M(q)$ 是质量矩阵，与 $\dot{v} \in \mathbb{R}^3$ 、 $\dot{\omega} \in \mathbb{R}^3$ 和 $\ddot{q} \in \mathbb{R}^{12}$ 的堆叠相乘，分别表示浮动基座线加速度、角加速度和主动关节加速度。第二项 $h \in \mathbb{R}^{18}$ 是偏置项，包含科里奥利、离心和重力效应。右侧最后一项描述基座力矩和主动关节力矩 $\tau \in \mathbb{R}^{12}$ ；由于基座不驱动，其力矩为零。

(12.5) 中倒数第二项与腿部里程计最相关，其维度取决于机器人构型（人形或四足）和接触腿数。令 c 为接触腿数量， d 为单腿主动关节数（四足 $d = 3$ ，人形 $d = 6$ ）。 $J_b \in \mathbb{R}^{dc \times 6}$ 是将基座 twist 映射到足端速度的雅可比矩阵，取决于机器人正向运动学及其在惯性坐标系 $\mathcal{F}^w$ 中的绝对身体方向； $J_q \in \mathbb{R}^{dc \times 12}$ 是 (12.3) 中雅可比矩阵的堆叠，其排列取决于机器人类型及接触腿数量。双腿站立的人形机器人有：

$$J_q = \begin{bmatrix} J_1(q) \\ J_2(q) \end{bmatrix} \quad (12.6)$$

对于四足机器人，腿可绕接触点转动，因此 J_q 只使用线速度雅可比块 J_v 。例如，第一、第三和第四条腿站立时：

$$J_q = \begin{bmatrix} J_{1,v}(q) \\ J_{3,v}(q) \\ J_{4,v}(q) \end{bmatrix} \quad (12.7)$$

最后， $f \in \mathbb{R}^{dc}$ 是所有接触足端作用于地面的力和/或力矩的集合。四足机器人四足着地时， f 是四个线力的堆叠：

$$f = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix}. \quad (12.8)$$

人形机器人双足着地时，还包含接触点三个轴上的线力和力矩：

$$f = \begin{bmatrix} f_1 \\ \tau_1 \\ f_2 \\ \tau_2 \end{bmatrix}. \quad (12.9)$$

如第 12.3.4 节将进一步说明，可根据上述力和力矩推断腿部接触状态。

12.1.6 关节感知

与固定基座机械臂类似，足式机器人的关节和末端执行器配备多种传感器，主要用于规划和控制^[1012]。这里简述也用于腿部里程计的三类重要传感器：编码器、力/力矩传感器和接触传感器。

12.1.6.1 旋转编码器

旋转编码器是将旋转轴角位置转换为模拟或数字信号的机电装置。在足式机器人中，它们用于测量关节角并确定机器人运动学，也可安装在其他部件上。例如，机械式 LiDAR 利用编码器测量光束阵列的方位角（见第 8 章）。编码器可按工作原理（光学或磁性）、读数类型（绝对式或增量式）和输出类型（模拟或数字）分类。足式机器人最常采用绝对式和相对式光学数字编码器；相关文献^[719]还介绍了其他编码器。

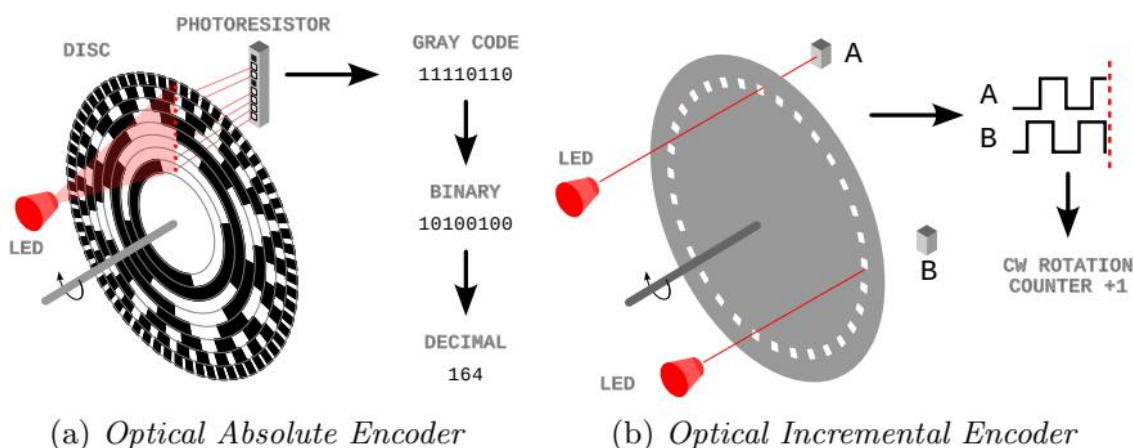


图 12.3: 图 12.3 (a) 8 位光学绝对编码器的工作原理：红外光束照射旋转圆盘，圆盘按照编码旋转角度的特定图案遮挡光线；光敏电阻阵列将各扇区有无光照转换为 Gray 码二进制数，再译为表示绝对旋转角的十进制数。示例编码器分辨率为 $360/256 = 1.41^\circ$ 。(b) 光学增量编码器原理：光敏电阻 A、B 相差 90° ；A 上升沿随后 B 下降沿表示顺时针旋转，AB = 11 到 AB = 10 的变化使计数器增加。

绝对式光学数字编码器（图 12.3a）以绝对方式测量关节角，同一关节构型会得到相同读数。其工作原理是：红外光源（如 LED）照射静止部分径向排列的光敏元件；光源与光敏元件之间有一块与轴一起旋转的圆盘，圆盘划分为不透明或透明的同心扇区，并按特定图案将角度范围编码为二进制数。二进制数按 Gray 码排序，连续自然数对应的二进制数仅相差一位，从而降低读数错误概率。编码角度的二进制字所用位数（即同心扇区数）决定角分辨率；8 位传感器有 256 个可能值，角分辨率为 $360/256 = 1.41^\circ$ 。

相反，增量式光学编码器（图 12.3b）测量相对于初始构型的角度变化：上电时角度为零，随后测量相对于该参考点的角度。根据转动方向加减小角增量即可得到角度。它不使用 Gray 码，而是采用带规则径向槽的不透明码盘，使单个光敏电阻 A 在恒速转动时输出方波；第二个光敏电阻 B 与 A 相差 90° 。两路信号组成的 2 位字 AB 随时可取四种值，状态转换用于判断旋转方向及计数增减^[719]。由于结构简单、成本更低，高分辨率增量编码器曾与低分辨率绝对编码器配合，用于在后者测得初始角后计算关节角^[987]。随着绝对编码器分辨率提高、成本降低，增量编码器正被快速替代。

12.1.6.2 力与力矩传感器

力和力矩传感器分别将施加在表面某点的线力或施加在轴上的机械力矩转换为电信号，主要用于执行器力矩控制或感知末端执行器与环境的交互。足式运动的每一步都会对地面施加力，需要直接或间接测量这些力以识别支撑腿。

两类传感器的工作原理通常相同而几何形状不同：传感器内部表面设计为在待测力方向的应力作用下轻微变形；表面粘贴柔性的可变电阻元件，即应变计。应变计电阻随变形量成比例变化，压缩会降低电阻率，拉伸会提高电阻率。图 12.4 展示将应变计贴在称重元件上测量线力的示例^[719]；测量力矩时，可将一组应变计贴在与电机轴相连的轮状柔性辐条上。

为测量末端执行器承受的全部力和力矩，可使用六轴传感器，在足够多的构型上布置应变计，以测量各方向的力和力矩。这类传感器常用于操作任务，也可安装在人形机器人的足部，直接测量其与地面的交互。

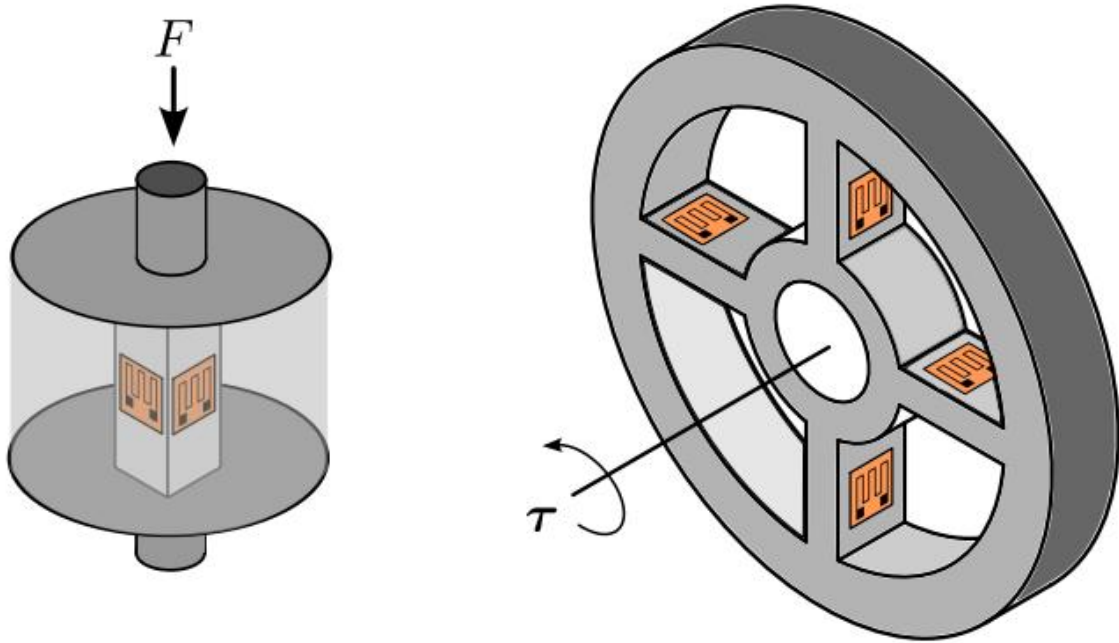


图 12.4: 图 12.4 力和力矩传感器的工作原理示例。应变计粘贴在可压缩的称重元件上，通过元件的微小变形测量线力。

可回驱电机设计^[544]使经济型动态足式机器人得以出现后，还可以由电机电流估计力矩，因为电流与力矩成比例。

12.1.6.3 接触传感器

力和力矩传感器在腿部里程计中的主要用途是确定支撑腿；另一种成本更低的方案是接触传感器，其输出为表示某只足是否接触地面的二值信号。这类传感器主要为小型和中型四足机器人开发，这些机器人的足端通常采用球形或圆形橡胶足垫。

接触传感器主要分为光学式^[397]和机械式^[794]。光学接触传感器在概念上类似编码器：LED 与光电二极管对通过小孔感测光线。足端接触地面后，足垫表面发生足够变形，带动遮光片移动并遮住小孔，从而检测接触。机械式接触传感器则在足垫内部设置简单的按钮开关；足端承受足够大的力时，开关被压下。

与价格较高的力/力矩传感器相比，接触传感器的主要缺点是响应较慢。此外，它们具有与力/力矩传感器相同的局限：需要把电缆布设到足端，而且足端承受的频繁冲击容易损坏传感器。因此，这类传感器主要见于面向室内运行的小型四足机器人。

12.2 运动估计

已知关节状态和机器人运动学后，便可计算基座的增量运动。主要有两种方式：利用正向运动学获得两个时刻间的相对位姿，或利用微分运动学瞬时估计机器人速度。二者都假设新建立的接触坐标系在一段时间内保持静止。

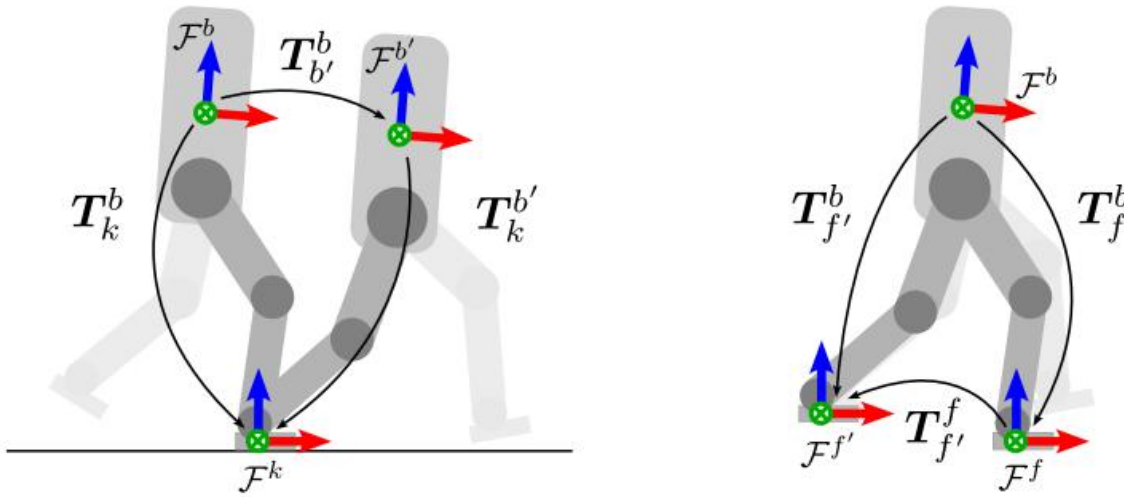


图 12.5: 图 12.5 理想接触下腿部里程计的工作方式。左：假设接触坐标系 $\mathcal{F}^k$ 刚性固定在地面（黄色），即可确定机器人身体的相对运动。右：也可表示连续两个时刻腿相对于身体坐标系（黄色）的运动。

12.2.1 相对位姿估计

图 12.5 展示人形机器人沿 zx 平面行走的简化示例。两个连续时刻的机器人基座分别用 $\mathcal{F}^b$ 和 $\mathcal{F}^{b'}$ 表示；同两时刻的足端坐标系和关节位置以类似方式定义。接触坐标系保持静止，因此当足端坐标系与接触坐标系重合时，机器人身体向前运动的位移等于足端相对身体向后运动的位移：

$$\mathbf{T}_{b'}^b = \mathbf{T}_k^b (\mathbf{T}_k^{b'})^{-1} = (\mathbf{T}_{f'}^f)^{-1} = (\mathbf{T}_f^b)^{-1} \mathbf{T}_f^b = \mathbf{fk}(\mathbf{q}')^{-1} \mathbf{fk}(\mathbf{q}) \quad (12.10)$$

(12.10) 建立了关节状态与机器人相对位姿之间的映射。连接这些相对位姿即可仅凭关节感知通过航位推算获得有效运动估计^[947]，但只在支撑阶段成立。对于点足机器人，还要求始终至少有三只足接触地面，这对四足平台并不实用。

四足机器人的标准方案^[88]是在 (12.1) 状态中加入与各腿关联、用世界坐标表达的接触坐标系位置 $\mathbf{c}_i = \mathbf{t}_k^w \in \mathbb{R}^3$ 。人形机器人具有脚踝，因此还可将接触点方向 $\mathbf{B}_i = \mathbf{R}_k^w \in \text{SO}(3)$ 加入状态^[948]。

任意时刻不一定有足够数量的腿接触地面（例如疾驰步态可能存在所有腿离地的阶段），因此通常还假设存在 IMU，忽略 (12.1) 中的角速度 ω ，并将 IMU 偏置纳入状态（见第 11 章）。四足和双足机器人的相应状态分别为：

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{t}_k & \mathbf{R}_k & \mathbf{v}_k & \mathbf{b}_k^a & \mathbf{b}_k^\omega & \mathbf{c}_1 & \mathbf{c}_2 & \mathbf{c}_3 & \mathbf{c}_4 \end{bmatrix}^\top \quad (12.11)$$

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{t}_k & \mathbf{R}_k & \mathbf{v}_k & \mathbf{b}_k^a & \mathbf{b}_k^\omega & \mathbf{c}_1 & \mathbf{c}_2 & \mathbf{B}_1 & \mathbf{B}_2 \end{bmatrix}^\top \quad (12.12)$$

(12.11) 表示四足机器人状态，(12.12) 表示人形机器人状态。对任意第 i 条腿，运动估计关系可精确写成：

$$\mathbf{T}_k^b = \mathbf{fk}(\mathbf{q}) = (\mathbf{T})^{-1} \mathbf{C}_i \quad (12.13)$$

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} \quad (12.14)$$

$$\mathbf{C}_i = \begin{bmatrix} \mathbf{B}_i & \mathbf{c}_i \\ \mathbf{0} & 1 \end{bmatrix} \quad (12.15)$$

$$(T)^{-1}C_i = \begin{bmatrix} R^\top & -R^\top t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} B_i & c_i \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} R^\top B_i & R^\top c_i - R^\top t \\ 0 & 1 \end{bmatrix} \quad (12.16)$$

$$\mathbf{f}_p(\mathbf{q}) = \mathbf{R}^\top (c_i - t) \quad (12.17)$$

$$\mathbf{f}_R(\mathbf{q}) = \mathbf{R}^\top \mathbf{B}_i \quad (12.18)$$

其中 $f_p(q)$ 表示足端相对于固定坐标系的位置变化, $f_R(q)$ 表示方向变化, 二者均由关节角决定。对于四足点足, 只使用 (12.17), 因为点足不能提供方向估计。式 (12.17) 和 (12.18) 是滤波器或因因子图等估计框架中的基本腿部里程计测量, 详见第 12.4 节。

12.2.2 速度估计

(12.3) 的微分运动学函数可用于从每条支撑腿直接得到速度测量。该方法广泛用于四足机器人 [89, 143, 578], 较少用于双足机器人 [1085]。优点是速度测量可方便地 (预) 积分到滤波器或因因子中; 它不保留历史, 避免位置误差累积 [315], 也不需维护额外状态 (接触位姿或位置)。但关节速度通常由关节位置数值微分得到, 舍入误差可能降低估计性能。

类似于相对位姿测量, 考虑刚性接触的腿运动关系。支撑腿的接触点 k 必须静止, 因此其在固定坐标系中观察到的速度为零:

$$\mathbf{v}_k^w = \mathbf{0}. \quad (12.19)$$

$$\mathbf{v}_k^b = \mathbf{v}_f^b = \mathbf{J}_v(\mathbf{q})\dot{\mathbf{q}}. \quad (12.20)$$

$$\begin{aligned} \mathbf{v}_k^w &= \mathbf{v}_b^w + \boldsymbol{\omega}_b^w \times \mathbf{t}_k^b + \mathbf{v}_k^b \\ \mathbf{0} &= \mathbf{v}_b^w + \boldsymbol{\omega}_b^w \times \mathbf{f}_p(\mathbf{q}) + \mathbf{J}_v(\mathbf{q})\dot{\mathbf{q}} \\ \mathbf{v}_b^w &= -\boldsymbol{\omega}_b^w \times \mathbf{f}_p(\mathbf{q}) - \mathbf{J}_v(\mathbf{q})\dot{\mathbf{q}} \end{aligned} \quad (12.21)$$

上式考虑了基座与足端之间杠杆臂造成的附加线速度 [270], 将机器人绝对速度映射到正向和微分运动学函数, 可作为图中的测量更新或因因子。由于约定在机体坐标中表达速度, 可使用机器人方向 $R = R_b^w$ 将其转换。到此为止假设支撑腿已知, 下一节介绍识别支撑腿的不同方法。

12.3 接触估计

接触估计的定义取决于应用。在协作机器人中, 末端执行器一旦“接触”某物 (即受到不可忽略的外力) 就可能被视为接触; 但在腿部里程计中, 只有接触点随时间保持静止时足端才算接触, 对于点足机器人还必须确保没有滑动。

技术上, 当地面反作用力 (GRF) 的垂直分量 f_z 满足摩擦锥条件时, 足端不会滑动 [949]:

$$\sqrt{f_x^2 + f_y^2} \leq \mu_{x,y} f_z \quad (12.22)$$

$$\begin{bmatrix} -\tau_y/f_z \\ \tau_x/f_z \end{bmatrix} \leq \begin{bmatrix} CoP_x \\ CoP_y \end{bmatrix} \quad (12.23)$$

$$|\tau_z| \leq \mu_z f_z \quad (12.24)$$

其中 f_x 、 f_y 是相对于接触平面的切向 GRF 分量, 取决于地形局部形态; $\mu_{x,y}$ 是摩擦系数, 取决于地面和接触足端的机械性质。平足人形机器人还可向地面施加力矩, 这给不滑动条件 (12.22) 增加了足端不旋转

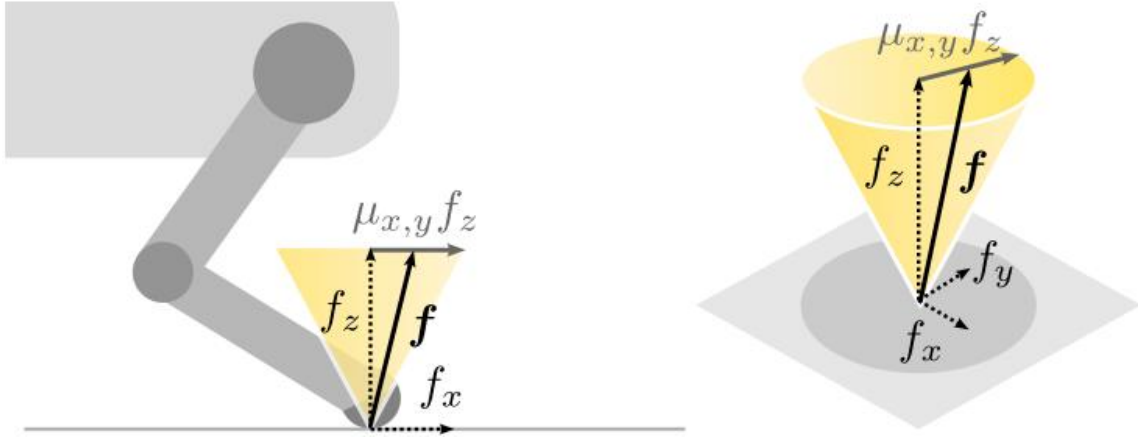


图 12.6: 图 12.6 四足机器人腿部的接触点。当足端施加的力 $\bar{f} = [f_x, \bar{f}_y, f_z]^\top$ 位于摩擦锥内时，可将该腿视为支撑腿。

的约束； τ 是接触力矩， μ_z 是旋转摩擦系数， CoP_x 、 CoP_y 是压力中心分量上界，它们由接触表面几何定义支撑多边形边界。

只要法向力 f_z 足够大，无论其他接触扳手（力和力矩）分量如何，(12.22)–(12.24) 都能满足。因此最常见的接触估计方法是直接对 f_z 阈值化。不同实现的区别在于如何测量或估计该力（接触传感器、F/T 传感器、关节感知或 IMU）以及机器人的具体特性。

12.3.1 使用接触传感器

接触传感器在硬件中隐式地对 f_z 阈值化：只有当测得的力超过标称 f_z 时，调好的二值信号才会激活。这是最简单的情况，腿部里程计可直接依赖传感器提供的二值状态。

12.3.2 使用力/力矩传感器

若足端安装 F/T 传感器，可直接随时间测量 f_z ，并将特定力模式与非二值事件关联。例如，持续时间超过阈值、幅值虽小但不断上升的力，表示足端正撞地但尚未进入支撑；相反，力降到某值以下（例如单腿预期负载的一半）表示足端即将解除接触，此时该腿的信息应丢弃或增大其不确定性^[315]。

12.3.3 使用 IMU

如第 11 章所述，IMU 能提供加速度和角速度测量。通常我们用它们测量相对于机器人身体的量，也可将 IMU 安装在腿部或足端。作用于足端的任何力（例如触地时的冲击）都会导致其加速度变化，因此一些工作使用腿部 IMU 隐式检测支撑腿^[1230, 949, 729]。该方法的主要优点是传感器不直接承受冲击，不易损坏；代价是需要额外信号处理才能有效检测加速度变化。

12.3.4 根据关节力矩感知

在足端增加接触传感器看似直接，但不同机器人形态、设计和集成难题使其并非总能实现。此时可利用机器人动力学（第 12.1.5 节）由关节力矩估计 GRF。以四足平台为例，可利用 J_q 的分块结构，由 (12.5) 计算末端力：

$$\mathbf{f}_i = -(\bar{\mathbf{J}}_{i,v}^\top)^{-1}(\boldsymbol{\tau}_i - \mathbf{h}_{q,i} - \mathbf{F}^\top \dot{\mathbf{v}}) \quad (12.25)$$

其中 $f_i \in \mathbb{R}^3$ 和 $\tau_i \in \mathbb{R}^3$ 分别是第 i 条腿的 GRF 和力矩； $\bar{J}_{i,v}$ 是第 i 个足端雅可比 J_v 的非零块（对四足机器人方阵）； $F \in \mathbb{R}^{3 \times 3}$ 是质量矩阵的一个分块； $h_{i,q} \in \mathbb{R}^3$ 是第 i 条腿的离心/科里奥利/重力力矩向量。

f_i 的估计只能在机体坐标中给出。要恢复真实 GRF，还缺少两类信息：其一是地形局部倾角，接触力方向取决于此；人形机器人的脚踝可给出较好近似，而四足机器人无法在没有外感知的情况下确定接触坐标系方向，只能由其他接触足（例如拟合平面）启发式推断^[329]。其二是摩擦系数，力的水平分量取决于它；摩擦系数由机器人踩踏材料决定，只能先验给定或根据滑移量推断^[513]。

总体而言，从关节感知确定接触状态仍是开放问题。研究者已发展概率式接触检测（例如结合机器人运动学和动力学^[497]）及学习方法（见第 12.6 节）。

12.4 使用腿部里程计进行状态估计

明确获得腿部里程计测量的步骤后，本节说明如何将其纳入估计框架以求解状态估计问题。主流方案基于滤波，在闭环控制所需的高频下融合 IMU 与腿部里程计。前几章介绍的因子图平滑方案直到近几年才用于足式平台，且重点通常不是为控制提供估计，而是提供较低频、用于建图的估计，以便融合 LiDAR 或相机等慢速传感器。

无论采用哪种方法，要将腿部里程计与其他传感器最优融合，都必须量化其不确定性。下面先讨论这一问题，再介绍滤波和平滑方法。

12.4.1 编码器噪声传播

腿部里程计的不确定性主要来自测量关节位置且受噪声影响的关节编码器。可将该噪声建模为加性零均值高斯项 $\eta_q \in \mathcal{N}(0, \Sigma_q)$ ，真实值 q 与测量值 $\tilde{q}$ 的关系为：

$$\tilde{q} = q + \eta_q. \quad (12.26)$$

$$\mathbf{f}_p(q + \eta_q) \approx \mathbf{f}_p(q) + J_c(q)\eta_q \quad (12.27)$$

$$J(q + \eta_q)(\dot{q} + \dot{\eta}_q) \approx J(q)\dot{q} + \frac{\partial}{\partial q}(J(q)\dot{q})\eta_q + J(q)\dot{\eta}_q. \quad (12.28)$$

由于正向和微分运动学函数包含旋转，是非线性的，不会保持编码器噪声的高斯性质。不过，可以像前几章一样采用一阶近似；该近似局部线性并保持高斯性， $J_c(q)$ 是身体机械臂雅可比，即以接触坐标表达的机械臂雅可比^[439]。速度测量同样可对编码器位置噪声和速度噪声作高斯近似；与给定编码器测量对应的多个噪声线性组合可合并为一个噪声项。

12.4.2 因子图平滑

为生成运动行为并避免跌倒等灾难性失效，足式机器人要求严格的实时控制和高频状态估计。历史上通常使用扩展卡尔曼滤波（EKF）^[88, 143]、无迹卡尔曼滤波（UKF）^[89] 或不变 EKF ^[441, 1255, 670] 等非线性卡尔曼滤波变体，将惯性和运动学等高频传感器数据融合并馈送给控制器。控制回路中加入外感传感器更新也已得到演示^[144]，但通常用于建图和规划。

基于卡尔曼滤波的方法必须同时指定过程模型和测量模型；过程模型不可用时，通常以恒速模型或更常见的 IMU 传播替代。因子图方法更为一般，它把过程模型和测量模型都统一表示为状态与测量之间的关系。

足式系统的因子图方法主要区别在于估计状态的数量与时间范围。只考虑连续两个状态时，因子图类似滤波器，Two-State Implicit Filter (TSIF)^[92] 就属于这一情形。增大窗口后，因子图不再只估计最新状态，而能校正窗口内的一段历史状态。状态频率是一项设计选择：以 IMU 等高频率加入更多状态可简化估计器设计，但必须缩短时间窗口以限制计算量；固定状态数量时，则可通过预积分测量获得更长的时间范围，如第 11 章的 IMU 预积分。

下面用两个代表性方法说明如何在因子图估计框架中结合预积分理论和腿部里程计：面向双足机器人的接触预积分^[440]，以及面向四足机器人的速度预积分^[1191]。二者都把第 12.2 节的测量重新表述为图中因子的残差与协方差。

12.4.2.1 接触预积分

接触预积分把人形机器人的运动学相对运动增量进行积分，形成连接 (12.12) 所定义的两个机器人状态的因子^[440]。图 12.7a 中，黑色先验因子固定图的规范自由度，橙色预积分 IMU 因子提供运动先验，绿色正向运动学因子约束接触坐标系位姿，蓝色接触预积分因子则编码两腿接触状态之间的相对运动。

正向运动学因子依据支撑腿的正向运动学，把惯性坐标系中的足端接触坐标系位姿与机器人位姿关联起来。将 (12.27) 的编码器噪声代入 (12.17)–(12.18) 的相对位姿测量，可得相应残差与协方差：

$$\mathbf{r}_{\mathcal{F}} = \text{Log}(\mathbf{C}_i^{-1} \mathbf{T} \mathbf{f}_k(\tilde{\mathbf{q}})) \quad (12.29)$$

$$\Sigma_{\mathcal{F}} = \mathbf{J}_c(\tilde{\mathbf{q}}) \Sigma_q \mathbf{J}_c^{\top}(\tilde{\mathbf{q}}) \quad (12.30)$$

$$\Delta \tilde{\mathbf{B}}_{ij} = \mathbf{B}_i^{\top} \mathbf{B}_j \text{Exp}(\delta \boldsymbol{\theta}_{ij}) = \mathbf{I} \quad (12.31)$$

$$\Delta \tilde{\mathbf{c}}_{ij} = \mathbf{B}_i^{\top}(\mathbf{c}_j - \mathbf{c}_i) + \delta \mathbf{d}_{ij} = \mathbf{0} \quad (12.32)$$

$$\mathbf{r}_{\mathcal{C}} = \begin{bmatrix} \text{Log}(\mathbf{B}_i^{\top} \mathbf{B}_j) \\ \mathbf{B}_i^{\top}(\mathbf{c}_j - \mathbf{c}_i) \end{bmatrix} \quad (12.33)$$

$$\Sigma_{\mathcal{C}} = \begin{bmatrix} \Sigma_w & \mathbf{0} \\ \mathbf{0} & \Sigma_v \end{bmatrix} \Delta t_{ij} \quad (12.34)$$

12.4.2.2 速度预积分

点足平台（例如四足机器人）的运动学不能约束两个状态之间完整的 6 自由度相对运动，因此不能直接使用前述正向运动学因子和接触预积分因子。另一种方案是利用第 12.2.2 节的腿部里程计线速度测量，将其预积分为约束机器人位姿相对变化的因子^[1191, 578]。该因子假设可由 (12.28) 确定机体瞬时线速度，并受高斯噪声 $\boldsymbol{\eta}_v$ 影响：

$$\tilde{\mathbf{v}} = -\mathbf{J}_v(\mathbf{q})\dot{\mathbf{q}} - \boldsymbol{\omega} \times \mathbf{f}_p(\mathbf{q}) + \boldsymbol{\eta}_v. \quad (12.35)$$

$$\Delta \tilde{\mathbf{t}}_{ij} = \Delta \mathbf{t}_{ij} + \delta \mathbf{p}_{ij} = \sum_{k=i}^{j-1} \left[\Delta \tilde{\mathbf{R}}_{ik} \tilde{\mathbf{v}}_k \Delta t \right] + \delta \mathbf{p}_{ij}, \quad (12.36)$$

其中，与接触预积分因子类似， $\delta \mathbf{p}_{ij}$ 是预积分速度噪声项^[1190, 578]。据此可得到速度预积分因子及其协方差：

$$\mathbf{r}_{\mathcal{V}} = \mathbf{R}_i^{\top}(\mathbf{t}_j - \mathbf{t}_i) - \Delta \mathbf{t}_{ij} \quad (12.37)$$

$$\Sigma_{\mathcal{V}, ij} = \sum_{k=i}^{j-1} \Sigma_{\mathcal{V}, ik} + \mathbf{A} \Sigma_v \mathbf{A}^{\top}, \quad (12.38)$$

速度预积分不需要显式保存接触位置状态，但需处理速度测量噪声及各时刻机器人方向变化。

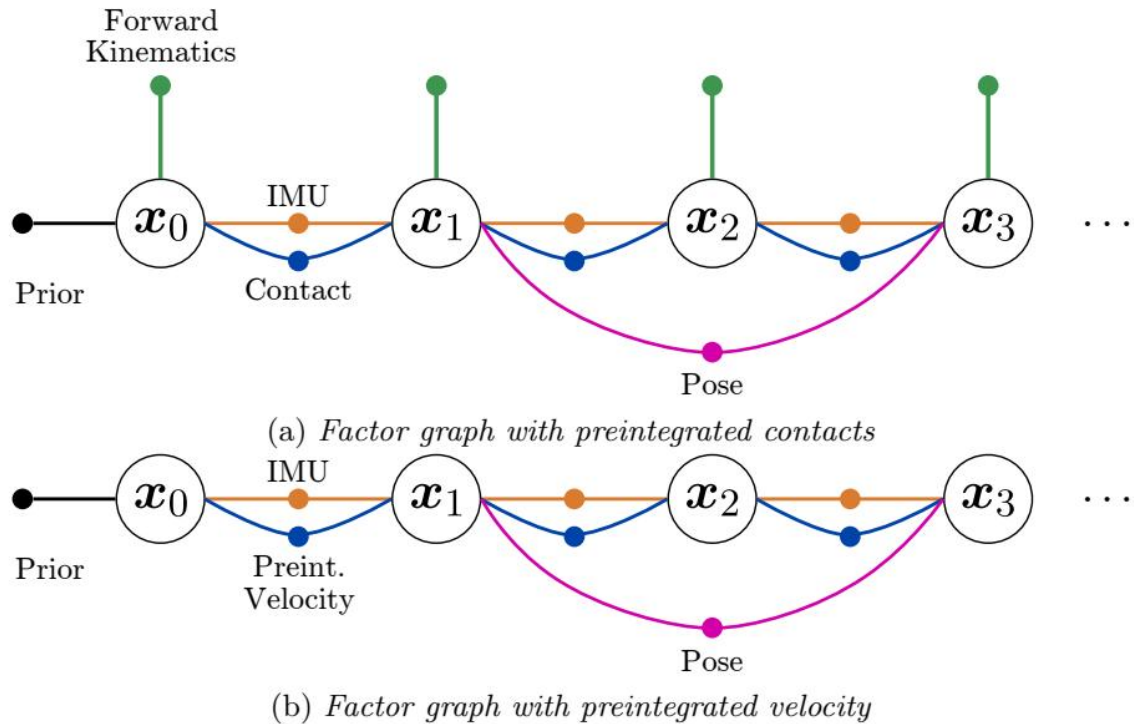


图 12.7: 图 12.7 预积分接触 (上) 和速度 (下) 的因子图表述。还可加入 IMU、视觉或 LiDAR 因子, 以构建完整状态估计系统。

12.4.2.3 多测量处理

当多条腿同时处于支撑阶段时, 每条腿都产生一项测量。可以把各腿 (及其测量) 独立处理; 当接触位姿或位置明确作为状态时, 这种做法较容易^[440, 578]。否则, 虽然同时采集的各腿速度测量仍可视作独立, 但通常更适合将其平均为一项测量, 以降低滤波器或因子图的计算负担^[1191]。

12.4.3 与外感传感器融合用于 SLAM

腿部里程计为计算连续状态之间的增量运动提供了额外途径。其主要用途是改善里程计估计, 使建立在其上的 SLAM 系统获得低漂移的节点间边, 从而在回环时减少突兀校正。

LiDAR 和相机等额外传感器可自然地与滤波和平滑方法集成: 前者增加测量, 后者增加因子。但不同独立来源以不同频率、噪声水平和失效率运行, 多源融合并非简单。如果某种传感器不满足零均值高斯噪声假设或完全失效, 滤波器 (或因子图) 状态可能受到破坏。因此受 DARPA SubT 挑战推动, 松耦合方法日益增多: 不同子系统 (视觉惯性、腿部惯性、LiDAR 惯性) 并行运行, 再对输出进行分诊, 从各子系统选择最佳估计^[299, 558]。

松耦合的替代方案是紧耦合方法。^[1191] 使用固定滞后平滑器, 在同一因子图中融合腿部里程计、IMU、相机和 LiDAR。为解决不同因子之间不一致或传感器失效问题, 分诊直接发生在因子层面: 某传感器失效时, 不向图中添加相应因子; 对于非零均值高斯噪声, 则可在因子内使用鲁棒代价函数。

12.5 开放挑战

前文介绍了如何计算腿部里程计并将其与其他传感器融合, 同时作出了一些实践中往往不成立的假设。放宽这些假设仍是开放问题, 下面简要介绍由此产生的挑战。

12.5.1 腿部变形

本章腿部里程计方程均假设机器人是完美刚体。该假设不成立时，腿部里程计测量会产生偏差，因为正向运动学函数计算的是足端理想位置而非真实位置（见图 12.8）。同样，当接触点不动但所受力使腿弯曲时，关节角也会变化。若问题只持续很短时间，过去的策略是分析力曲线检测冲击，并在冲击期间拒绝相应测量^[143]。

腿部里程计

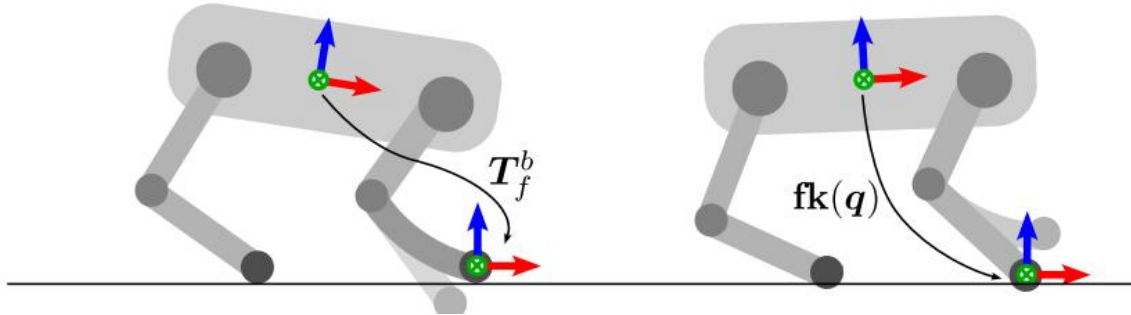


图 12.8: 图 12.8 四足机器人腿部变形示例。左侧为基座与接触点之间的真实变换；由于正向运动学假设腿刚性，右侧错误地估计出向上运动。

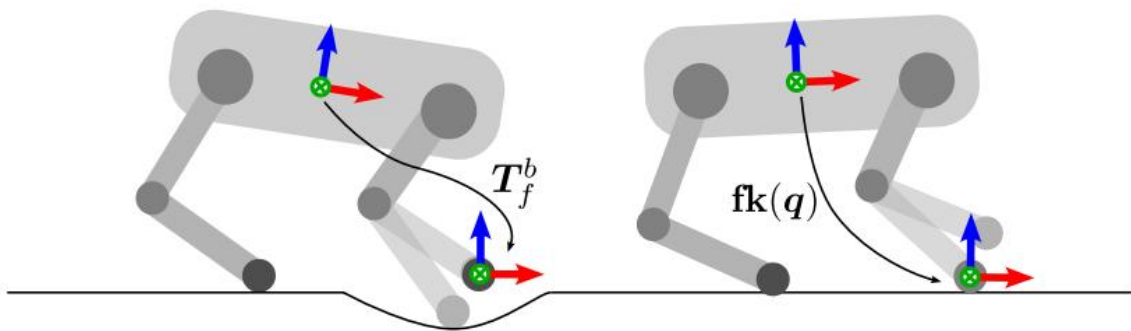


图 12.9: 图 12.9 四足机器人地面变形示例。机器人最初接触平坦地面；保持接触状态时地面发生变形，足端陷入地面（左）。足端下沉使关节角发生变化，该运动被解释为相对于初始触地点的向上运动（右）。

双足机器人往往具有更长的腿，因此腿部柔性问题可能更加严重。一种方法是利用腿部负载与柔性之间的相关性（直观地说，负载越大，腿部弯曲越明显），结合机器人结构几何和材料属性对弯曲特性进行精细建模；但这种方法复杂，且通常泛化能力较差。更常用的方案是在连杆上安装额外 IMU，估计连杆方向并与关节读数比较^[1128]。

12.5.2 非刚性接触与滑移

若机器人在松软或可塌陷地面上行走，腿会伸展并穿入地形（图 12.9 左）。若在地面开始变形前检测到接触，并假设接触点静止，则正向运动学会将其解释为向上运动（图 12.9 右）。其效果虽类似腿部变形，但此时破坏的是接触点零速度假设^[314]：地面变形使接触点向下移动。足端被视为接触但施加在地面上的力违反 (12.22) 和/或 (12.24) 时，就会发生类似滑移的问题。

此时需要相机等外传感器，在非退化运动和机器人构型下使接触点速度可观测^[1085]。可以在^[1191]中将接触速度显式作为附加状态，也可以在^[578]中将其作为足端位置的导数。

12.6 延伸阅读与近期趋势

本节聚焦状态估计和足式机器人学近期趋势。其中一些趋势针对接触估计等开放挑战，另一些则代表当前技术（特别是学习算法辅助技术）的重要范式转变。

基于学习的接触估计。接触估计假设刚性接触，但滑移、松软地形或腿部变形都容易破坏该假设。鉴于准确建模这些问题存在挑战，研究者提出数据驱动的接触估计方法，通常学习一个决定何时建立接触的二值信号。既有工作以有监督方式学习接触分类器^[143]，也有工作通过聚类以无监督方式学习^[948]，其中关节和惯性等本体感知是模型的主要信号。

随着深度学习方法兴起，人们开始使用神经网络架构确定足端接触状态，并在更广泛的结构化和非结构化环境中取得更好泛化^[670]。视觉触觉传感器利用机器学习和计算机视觉进展^[641]，也是改善力和接触估计的希望方向^[1003]。

端到端学习。高频腿部里程计和本体状态估计最初是为闭环、基于模型的运动控制而发展；近期强化学习（RL）进展对其必要性提出挑战。RL 运动控制器表明，运动只需基座速度和方向即可，这些量可由标准本体状态估计器显式提供^[498]，甚至可直接由关节与惯性感知原始数据提供^[638, 763]。

后者似乎使人质疑状态估计的必要性，但这与特定 RL 控制器的训练方式有关。训练目标是跟踪速度命令，模拟人类操作员或规划系统控制机器人的方式。为实现该目标，运动控制器只需知道基座相对于重力向量的方向以及瞬时机体速度。如第 11 章所示，这些量可由惯性数据完全观测，因此能在训练期间隐式估计。

另一方面，运动学习的另一个趋势旨在获得能够探索世界的高级机动技能：学习可穿越不同障碍物并相对于起始位置到达目标的运动控制器，机器人跑酷就是例子^[1311, 464]。这种导航行为必须有里程计估计，因为机器人需要在惯性坐标系中跟踪朝向目标的进展。这说明复杂运动和导航任务仍需要状态估计。

已有少量工作将状态估计作为运动策略学习的一部分^[516]，显式估计机体速度、足端高度和接触状态。虽然目前仅用于运动控制，但它有望为本体状态估计或 SLAM 中的里程计因子提供更准确的腿部里程计。

人形机器人。人形机器人承载着机器人学发展的部分愿景，即创造能够承担人类不愿从事的枯燥、危险和肮脏任务的人工智能体。原则上，类人的身体应使机器人能够在以人为中心的环境中无缝工作，使用为人设计的工具、设备乃至车辆。

第 12.1.1 节简要介绍的 DARPA Robotics Challenge 是实现这一目标的主要努力之一。其任务既包括崎岖地形运动，也包括操作工具和驾驶车辆，带来诸多挑战。挑战赛于 2015 年结束后，足式机器人研究大多转向四足平台，因为四足在控制和鲁棒性方面具有明显优势，并推动其用于工业巡检和监测。直到 2021 年，受人工智能（AI）的乐观预期、快速进展以及四足机器人成功的推动，人形平台才重新引起商业兴趣。

近期多家公司致力于开发新的人形平台，将 AI 系统具身化到现实世界。人形机器人目标是解决配送、仓储和制造中的复杂任务，在高要求环境中与人并肩工作。这为本章主题提出了尚未充分探索的新挑战：人形机器人需要长期、准确且可靠的全身估计，才能完成最后一公里配送；在仓库搬运包裹或其他物体时，足端以及躯干、手臂和手部都可能产生间歇接触；在人附近作业还要求具备柔顺性，这也需要放宽本章采用的刚性接触假设。

12.7 致谢

作者感谢 Michele Focchi（特伦托大学）在本章部分内容准备过程中提供的建议。

第十三章 通过深度学习增强 SLAM

作者: Zachary Teed, Jia Deng, Boris Chidlovskii, Jérôme Revaud, Felix Wimbauer 和 Daniel Cremers

随着大规模、高质量标注数据集的出现^[608],研究者已将深度神经网络应用于众多挑战,从图像分割^[690]、光流估计^[278]到蛋白质预测^[388, 528]。这些系统能够利用大规模标注数据,并借助具有强大内部表示的高度过参数化神经网络,学习从输入集到标签空间的映射。一个自然的问题是:如何将深度学习的成功拓展到 SLAM?

如前几章所述,现代 SLAM 系统包含许多相互连接的组件,包括特征检测、特征匹配、离群值剔除和重定位。一条有前景的研究路线是用学习系统替代其中的单个组件。例如,特征检测网络^[267]、描述子网络^[214]、匹配网络^[1052, 1137, 971, 1138, 671]以及离群值剔除网络^[971]可以在大规模数据集上训练,且通常能够超过经典方法的性能。训练完成后,这些组件可以重新接入完整 SLAM 流水线,从而得到更准确、更鲁棒的系统。还可以训练深度神经网络,直接从输入图像预测有用的三维属性,例如单目深度^[91, 1228, 302]或图像帧对之间的相对位姿^[1229]。随后,深度或相对位姿可以自然地集成到现有 SLAM 系统中,例如作为因子图中的附加损失项^[1229]。

模块化方法的一个问题是,各组件的误差可能相互传递并不断累积。越来越多的研究表明,深度网络能够将流水线中越来越多的部分组合为端到端可训练的模块,包括将 BA 等优化问题集成为神经网络层^[1082, 1068, 1155](另见第 4 章)。DuSt3R^[1164]系列工作更进一步:它用一个网络替代完整技术栈,直接从未标定图像预测点图(pointmap,即结构化三维点云的一种参数化表示)。

本章将讨论把神经网络能力引入 SLAM 流水线的若干有前景途径。图 13.1 概览了本章介绍的一些工作。从许多方面看,这些技术都试图结合两类方法的优点:有些方法将神经网络预测的深度、位姿和不确定性输入基于优化的 SLAM 流水线;有些方法将端到端学习组件与因子图结合;还有一些方法利用经过适当训练的网络,以新颖方式估计帧间运动和三维结构。由此得到的方法可以显著增强视觉 SLAM,在提高精度和鲁棒性的同时,通常还能很好地泛化到训练域之外。具体而言,第 13.1 节回顾使用深度学习预测深度和相机位姿的方法;第 13.2 节讨论特征匹配和跟踪应用;第 13.3 节聚焦可微优化,并介绍 DROID-SLAM,可将其视为展示这种集成方式的首个成功系统。随后各节讨论从未标定图像生成点图的基于 Transformer 的架构,包括 DuSt3R(第 13.4 节)、MASt3R(第 13.5 节),以及它们与 SLAM 和 SFM 流水线的集成(第 13.6 节)。本章最后综述开放性挑战(第 13.7 节)。

(a) D3VO^[1229] (©2020 IEEE) (b) MonoRec^[1187] (©2021 IEEE)

(c) DROID-SLAM^[1082]

(e) DuSt3R^[1164] (©2024 IEEE)

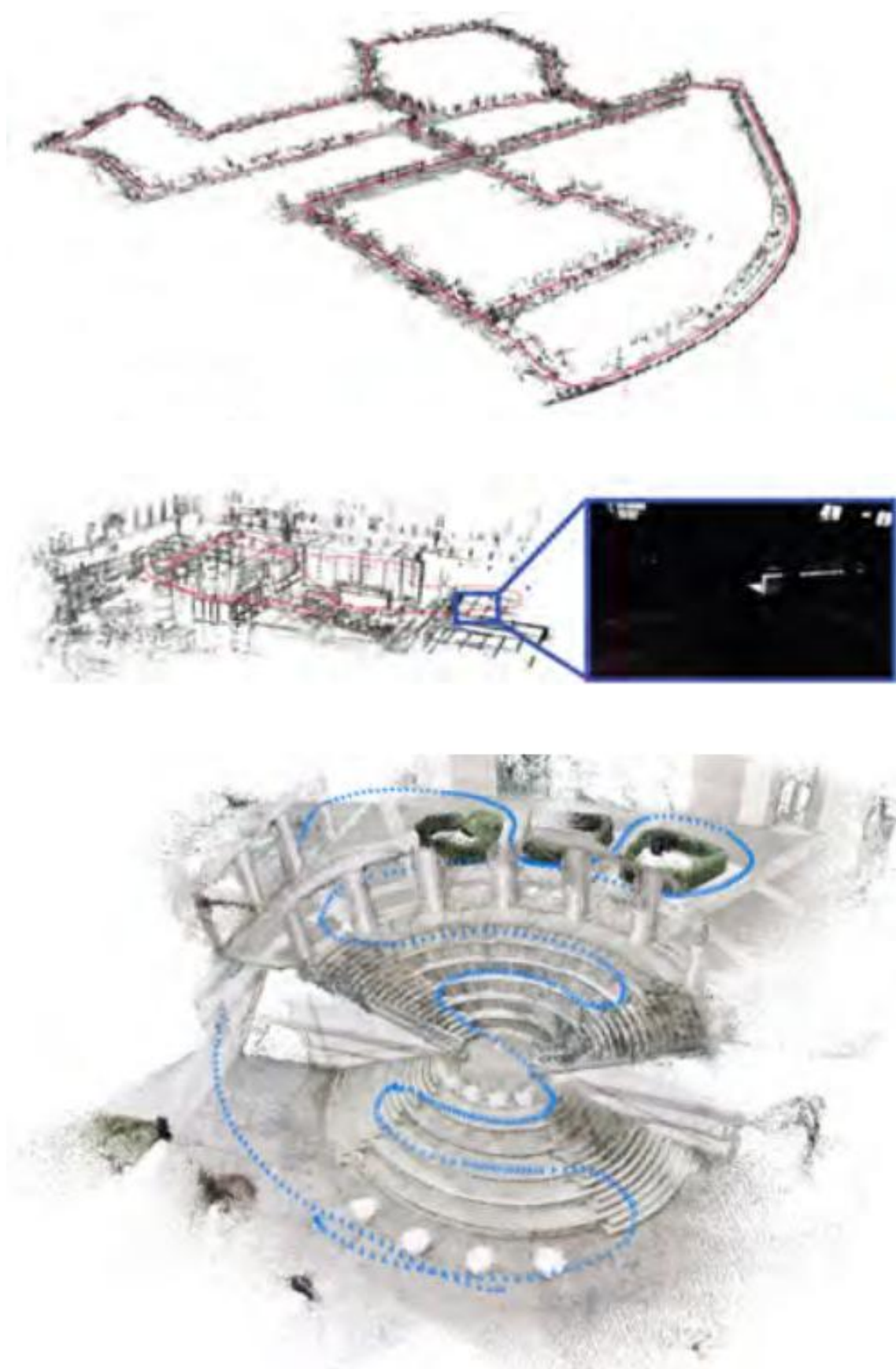
(d) DPVO^[1084]

(f) MUST3R^[135] (©2025 IEEE) 图 13.1 过去数年中,人们开展了大量工作,将深度网络的预测能力引入视觉 SLAM。本章将回顾其中的一些方法。

13.1 深度和相机位姿的深度学习

可以训练深度神经网络,学习从输入空间(例如图像)到标签空间(例如深度)的映射。现代神经网络由大量可微层组成,这些层可以组合为端到端可训练系统。网络参数通过损失函数上的梯度下降变体进行优化,该损失函数衡量网络预测与真实标签的匹配程度。







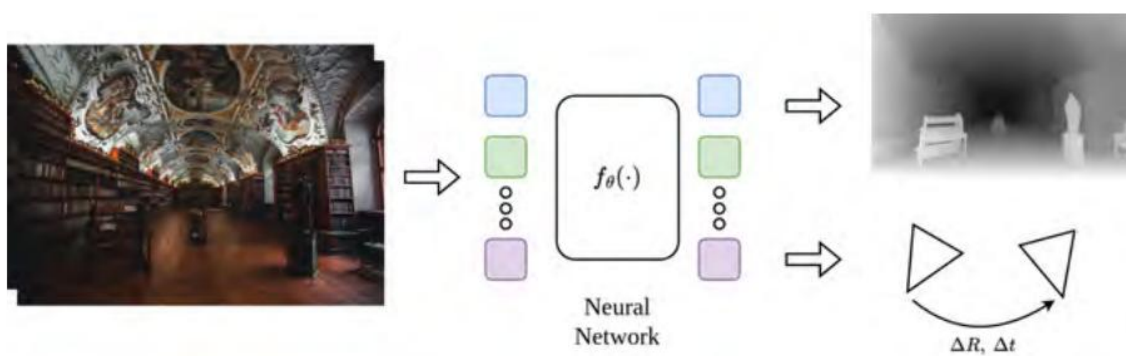


图 13.1: 图 13.2 可以训练神经网络，从输入图像或图像集合预测深度或相对位姿等三维属性。这些三维属性可用于增强 SLAM 系统。

借助现代深度传感器和激光扫描仪，如今可以采集大量三维数据。例如，可以使用深度传感器或 LiDAR 采集与深度图配对的图像；此外，还可以使用渲染引擎生成大规模合成三维数据集。如图 13.2 所示，可以在这些数据上训练神经网络，直接从输入图像预测深度或相对相机位姿等三维属性。本节将说明如何利用这些网络的预测来增强 SLAM 系统。

13.1.1 深度预测的深度学习

给定大量图像-深度对，可以训练神经网络，直接从单幅输入图像预测深度（或深度的某种参数化）。Eigen 等人^[302]首次展示了利用卷积神经网络从单幅图像预测深度。多年来，得益于网络架构的改进和三维数据集的扩充，单图像深度网络不断发展。现代深度网络通常使用预训练视觉 Transformer (ViT) 骨干网络^[280]，并配合卷积解码器对低分辨率 ViT 特征进行上采样，生成最终深度估计^[1162, 480]。为提高泛化能力，这些网络往往在包含合成数据和真实数据的二十多个数据集组合上训练。

深度预测在 SLAM 中有许多直接应用，尤其适用于单目场景。单目 SLAM 系统通常只能生成带尺度因子的重建结果，并且会受到尺度漂移影响。Yang 等人^[1228]和 Hu 等人^[480]表明，度量深度预测可以用于抑制单目 SLAM 的尺度漂移。在纯相机旋转、相机参数变化和动态物体等具有挑战性的情况下，深度预测还可以增强系统鲁棒性并避免失败^[660]。

此外，视觉 SLAM 系统往往只跟踪一组稀疏特征点，因而得到场景的稀疏重建。例如，纹理较少的区域几乎不会出现在最终点云中。虽然稀疏点云已经很有用，但机器人导航或自动驾驶等许多下游应用需要稠密重建。下面给出一些旨在获得更稠密三维重建的方法实例。

MonoRec: 单目稠密重建^[1187]。将稀疏 SLAM 系统输出稠密化的一种方法，是为视频的每一帧预测稠密深度图。可以利用 SLAM 系统产生的位姿，将所有像素提升到三维并累积到全局参考坐标系，从而获得如图 13.1(b) 所示的稠密重建。然而，直接使用深度网络生成的深度图并非最优，原因有二。第一，单图像深度网络在相邻帧之间往往产生几何不一致的输出。第二，许多场景包含动态物体，会在静态重建中引入拖影伪影。MonoRec^[1187]在深度网络中加入代价体积，通过预测运动物体掩膜来过滤动态物体，从而改善几何一致性。

为提高几何一致性，MonoRec^[1187]针对每一帧 I_t 构造代价体积 $C_t(d)$ ，该体积由连续帧 $\{I_{t_1}, \dots, I_{t_n}\}$ 聚合得到。具体而言，在若干离散深度步长 d 上计算每个像素的光度一致性，并将结果聚合为三维体积。MonoRec 使用同时接收输入图像 I_t 和代价体积 $C_t(d)$ 的深度网络来预测深度。代价体积为网络提供额外上下文，以生成几何上一致的深度。

代价体积假设世界是静态的；如果场景中有物体运动，就会提供错误信号。因此，MonoRec 依赖额外的掩膜网络，从代价体积预测运动物体分割掩膜 M_t 。将预测深度提升至三维时，利用该运动物体分割掩膜过滤动态点。图 13.1(b) 表明，掩膜模块可以在稠密重建中滤除运动物体。

MonoRec 使用 SLAM 系统的稀疏三维点预测进行训练。掩膜模块可从准确深度中获益，而深度模块可从准确的动态掩膜中获益，因此联合训练两个组件可以进一步提升性能，得到准确且一致的深度图。

Behind the Scenes (BTS)^[1188]。MonoRec^[1187]生成的稠密点云能够准确表示视频中观测到的三维世界。然而，被遮挡的区域不会成为重建三维场景的一部分。Behind the Scenes (BTS) 通过预测体积重建而不是深度图来解决这一不足。在该表示中，会重建相机视锥内的全部内容，包括当前帧中被遮挡的区域。BTS 将场景表示为体积密度场 $\phi(\mathbf{x}) : \mathbb{R}^3 \rightarrow \mathbb{R}_+$ ，它将相机视锥中的任意点 x 映射到体积密度 σ 。通过成熟的体积渲染技术，可以从该场景表示合成任意新视图。总体而言，体积重建为许多下游应用提供了有用的附加信息，尤其是自动驾驶和机器人领域。

13.1.2 相机位姿预测的深度学习

与训练神经网络预测深度类似，也可以训练网络预测帧对或视频序列之间的相机位姿。设 I_1 和 I_2 是一对输入图像，可以设计神经网络 f_θ ，以两幅图像为输入，预测从 I_1 到 I_2 的相对相机位姿 $f_\theta(I_1, I_2) = T_1^2 \in \text{SE}(3)$ 。给定真实位姿 $T_1^{2*} \in \text{SE}(3)$ ，可定义损失

$$\mathcal{L}_{pose} = \left\| (T_1^2)^{-1} \cdot T_1^{2*} \right\|, \quad \text{where } T_1^2 = f_\theta(I_1, I_2) \quad (13.1)$$



without mask module



with mask module

图 13.2: 图 13.3 MonoRec: 网络架构中的掩膜模块可依据跨时间的亮度不一致过滤运动物体^[1187] (©2021 IEEE)。

该损失衡量预测位姿与真实位姿之间的相对变换。还可以定义其他损失，例如最小化旋转角，或计算预测相对旋转与真实相对旋转的四元数表示之间的距离。

早期相机位姿估计工作将两帧拼接为输入，并回归轴角^[1114]或欧拉角^[1299]等相机位姿参数。给定帧对之间的相对位姿，可以通过复合这些位姿来串联恢复相机的完整轨迹。TartanVO^[1167]在输入图像中加入预测光流和相机内参，提高了相对位姿估计及其在下游数据集上的泛化能力。

两视图位姿预测的一个问题是，在较长视频序列中，相对位姿误差会不断累积。DeepVO^[1163]提出使用循环神经网络，以帧序列为输入并预测相机位姿序列。DeepVO 从输入视频序列提取特征以生成特征图序列，再通过 LSTM 处理这些特征图，预测一系列相机位姿估计。

尽管这些工作展示了有前景的结果，但直接预测相机位姿通常不如经典的 SfM 或 SLAM 技术准确，因为后者在系统中更深入地嵌入了几何约束^[1082]。此外，这类技术容易过拟合训练分布，需要在大量数据集组合上训练，以促进更好的泛化。

13.1.3 深度和相机位姿的无监督学习

深度和位姿的监督学习需要真实三维数据，而这些数据可能并不总是可用，或者难以获取。事实证明，无需显式三维监督，也可以训练位姿网络和深度网络。一种常见技术是使用视图合成作为监督信号^[387, 1299]。给定具有相对位姿 T_1^2 的两幅图像 I_1 和 I_2 ，可以使用从 I_2 采样的像素，生成从 I_1 视点观察到的新图像。

具体而言，设相机投影 $\pi_c: \mathbb{R}^3 \mapsto \mathbb{R}^2$ 将三维点映射到二维像素坐标，逆投影函数 $\pi_c^{-1}: \mathbb{R}^2 \times \mathbb{R}^+ \mapsto \mathbb{R}^3$ 将带深度 d 的二维像素映射到其三维点。给定这些函数和相对位姿 T_1^2 ，可以将 I_1 中的 (x^1, d^1) 对映射到其在 I_2 中的位置：

$$x^2 = \pi_c(T_1^2 \pi_c^{-1}(x^1, d^1)) = \omega_{1 \rightarrow 2}(x^1, d^1) \quad (13.2)$$



图 13.3: 图 13.4 DVSO: 将深度神经网络的单目深度预测输入 DSO^[308] 等经典 SLAM 方法后，单目方法“Deep Virtual Stereo Odometry” (DVSO)^[1228] 获得了与当前最先进双目方法相当的高精度相机轨迹。该深度网络使用双目视频以自监督方式训练，显著提高了精度，并使单个运动相机能够恢复带尺度的重建。

忽略遮挡，并简记为 $\omega_{1 \rightarrow 2}$ 。给定深度图 d ，式 (13.2) 可用于将 I_2 变形到 I_1 的视点，定义变形图像

$$\tilde{I}_2(x) = I_1(\omega_{1 \rightarrow 2}(x, d(x))) \quad (13.3)$$

其中 $I_1(\cdot)$ 表示双线性插值。由于双线性插值相对于像素坐标是局部可微的，因此可以计算合成图像 $\tilde{I}_2$ 对深度和相机位姿的导数。Godard 等人^[387]表明，可以利用视图合成损失在双目图像上训练深度网络。后续工作进一步表明，可以利用视图合成监督联合训练深度网络和位姿网络^[1299, 1129]。

Deep Virtual Stereo Odometry (DVSO)^[1228]。单目 SLAM 系统的一个局限是鲁棒性通常低于双目系统，且更容易发生漂移。DVSO 表明，可以利用在双目帧对之间通过视图合成监督训练的神经网络深度预测^[387]，增强经典视觉里程计系统 DSO^[308]。如图 13.4 所示，DVSO 获取神经网络的深度预测，并加入一个附加因子，鼓励预测深度图与 DSO 计算所得深度保持一致。

神经网络预测的深度提高了重建质量，也提高了 VO 系统预测位姿的精度和鲁棒性。此外，由于深度预测采用度量单位，该方法甚至可以从单个相机恢复尺度准确的重建结果。这项工作被称为“虚拟立体里程计”，因为在单目应用中，深度网络实际上虚构了第二个相机的作用。

D3VO: 用于单目视觉里程计的深度、位姿和不确定性预测 [1229]。除单目深度预测外，还可以将相对相机运动的预测输入经典 SLAM 流水线，以确保恢复的轨迹与连续帧相机运动预测保持一致。DSO [308] 等直接视觉 SLAM 方法依赖跨帧颜色一致性来推断定位和建图。在真实世界中，这种颜色一致性并不总能保证，尤其对于有光泽、金属或玻璃等非 Lambert 表面，对应点很可能具有不同颜色。虽然可以显式建模世界的反射属性，但这会带来很高的计算负担，在实时场景中往往不可行。

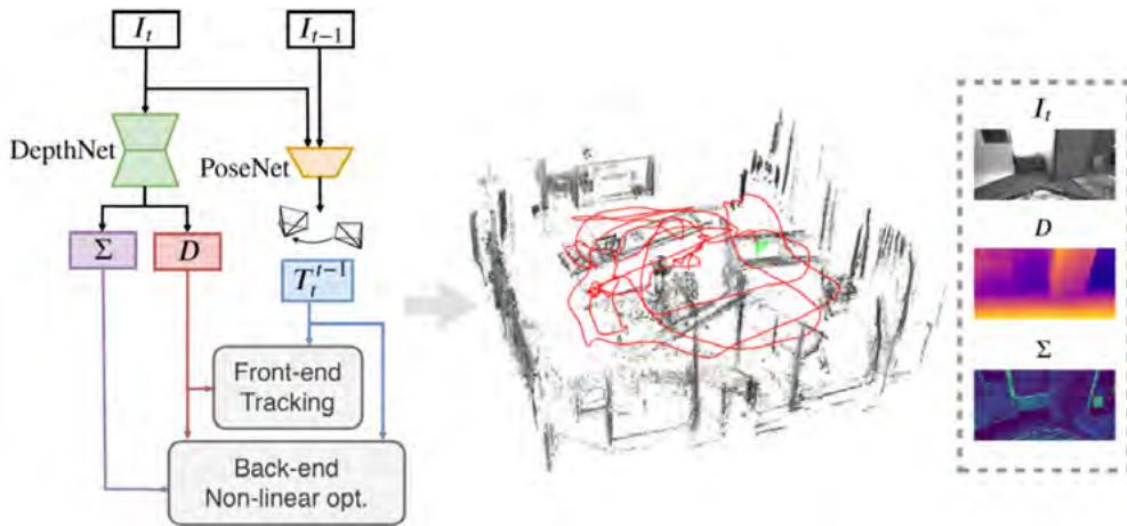


图 13.4: 图 13.5 D3VO [1229] 将用于位姿、深度和不确定性预测的自监督神经网络与直接 SLAM 系统 [308] 结合。网络的位姿和深度预测作为因子图中的附加因子集成到 SLAM 系统中，促使 SLAM 状态变量与神经网络预测对齐。通过结合深度学习与因子图，D3VO 同时受益于神经网络的鲁棒性和直接 SLAM 的精度。图像来自 [1229] (©2020 IEEE)。

事实证明，可以训练神经网络预测跨帧颜色很可能保持不变的位置。这些预测同样可以输入直接 SLAM 流水线，在预期颜色不一致的位置降低残差权重。D3VO [1229] 以这种方式将神经网络对单目深度、帧间相对相机运动和不确定性的预测输入 SLAM 流水线。如图 13.5 所示，D3VO 使用通过视图合成损失以自监督方式训练的深度网络和位姿网络。预测结果以前端因子图项和后端相应损失项的形式进入系统。通过将神经网络的深度、位姿预测与直接视觉里程计 [308] 结合，D3VO 能够获得高精度相机轨迹，并对单目 VO 常见的失败情况具有鲁棒性。基于学习的深度细化模块也已用于改进基于特征的 SLAM [514]。

13.2 特征匹配和光流的深度学习

前几节展示了如何使用神经网络直接从输入图像预测深度和相机位姿等三维信息。这种前馈方法不同于典型 SLAM 流水线，后者通常将推理写成优化问题：给定特征匹配等带噪测量 z ，求解 x^{MAP} 。

预测相机位姿或深度并非易事。网络需要从头学习多视图几何，例如深度、相机位姿和相机内参之间的关系。另一种思路是预测测量量（如特征匹配），而不是直接预测状态变量。

13.2.1 特征检测器和描述子的学习

间接式 SLAM 方法依靠特征检测器和描述子，从输入图像提取一组稀疏视觉测量。首先识别图像中的显著区域，通常称为关键点；随后利用每个关键点的局部邻域计算向量描述子，用于在图像之间匹配关键点。

可以训练神经网络，替代 SLAM 和 SfM 流水线中常用的手工设计特征检测器与描述子。例如，SuperPoint [267] 提出使用卷积神经网络同时完成：(1) 检测显著图像区域；(2) 为检测到的关键点分配特征描述子。类似地，Universal Correspondence Network [214] 表明，神经网络生成的特征描述子可以提高下游特征匹配精度。这类网络通常使用对比损失训练，使匹配点的特征描述子相似，并使不匹配关键点的描述子距离尽可能大。

13.2.2 特征匹配

给定从一对图像中提取的关键点集合，也可以使用神经网络改善图像间关键点匹配。SuperGlue^[971] 首次提出利用神经网络引导特征匹配。SuperGlue 在一对输入图像提取的关键点上运行，通过自注意力层和交叉注意力层迭代更新与每个关键点关联的特征向量，如图 13.6 所示。网络输出一个分配矩阵，给出第一幅图像中的每个关键点与第二幅图像中每个关键点匹配的可能性（另有一列表示未匹配关键点）。SuperGlue 预测的关键点匹配可以提高包括位姿估计在内的多个下游任务的精度。LightGlue^[671] 通过一系列架构和训练改进，提高了 SuperGlue 的精度和速度。

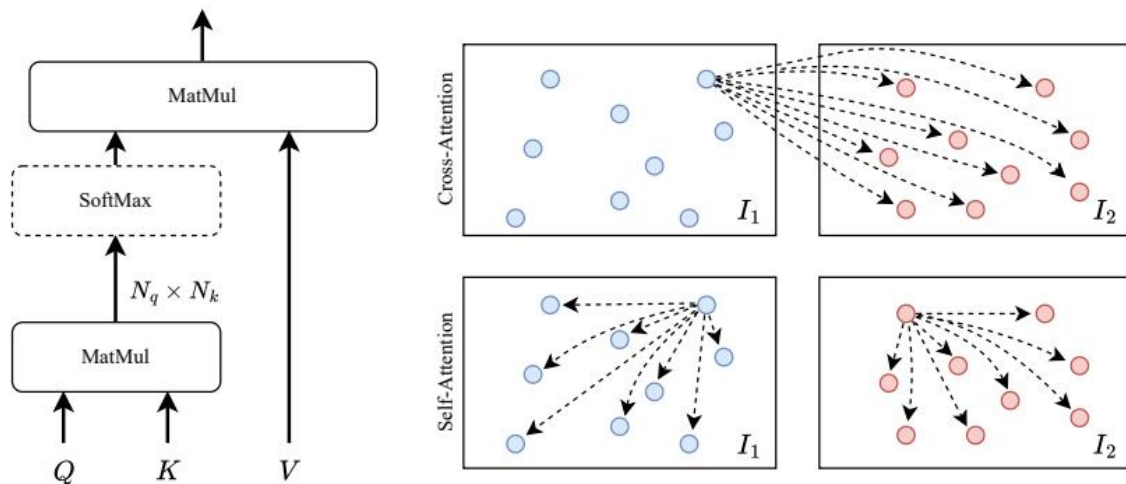


图 13.5: 图 13.6 SuperGlue^[971]、LightGlue^[671] 和 LoFTR^[1052] 等特征匹配网络中常用的多头自注意力与交叉注意力示意图。SuperGlue 和 LightGlue 在与图像关键点关联的特征向量上执行自注意力和交叉注意力；LoFTR 不依赖关键点检测，而是在稠密特征网络上执行自注意力和交叉注意力。

SuperGlue 和 LightGlue 的一个潜在局限是，它们只处理一组稀疏图像关键点。这样虽然可以减少计算量，却无法利用完整图像；在纹理有限等困难样例上，可用的高质量匹配数量可能因此受限。LoFTR^[1052] 提出了一种稠密像素匹配网络。与 SuperGlue 和 LightGlue 类似，LoFTR 使用自注意力和交叉注意力（图 13.6），并首先在低分辨率特征网络上进行稠密匹配，再在更高分辨率上细化预测，以降低计算量。

这些系统产生的特征匹配可以作为视觉测量及其对应因子，直接集成到间接式 SLAM 系统中。在传统特征描述子不可靠的情况下，深度特征匹配可以提高 SLAM 系统的鲁棒性。

13.2.3 光流与 RAFT

光流与特征匹配问题密切相关。光流的任务是估计一对图像帧之间逐像素的运动。这种逐像素运动（即光流场）可以作为基于因子图的视觉 SLAM 系统中的测量。

最初，光流被表述为一个优化问题，其代价函数平衡两个项：数据项和正则化项。数据项鼓励视觉上相似的图像区域对齐，正则化项则鼓励物理上合理的运动场。近年来，深度学习成为一种有前景的替代方法^[278, 1051, 502, 907]。本节介绍其中一种方法 RAFT^[1081]（图 13.7），因为它是 DROID-SLAM^[1082]（第 13.3 节）的基础。较新的光流网络通过加入图 13.6 所示的多头注意力和 Transformer 模块^[494]，进一步改进了 RAFT。

图 13.7 展示了 RAFT 的整体设计。RAFT 同时结合深度学习和传统一阶优化的要素。首先从输入图像提取特征，再据此构造四维相关体积。将光流场初始化为零，并用它在相关金字塔中执行查询。相关特征与当前光流场估计一起输入卷积 GRU，由 GRU 生成光流场更新以及更新后的隐状态。RAFT 在带真实光流的合成数据集组合上训练^[743, 278]，但对真实数据也表现出良好泛化能力^[757]。

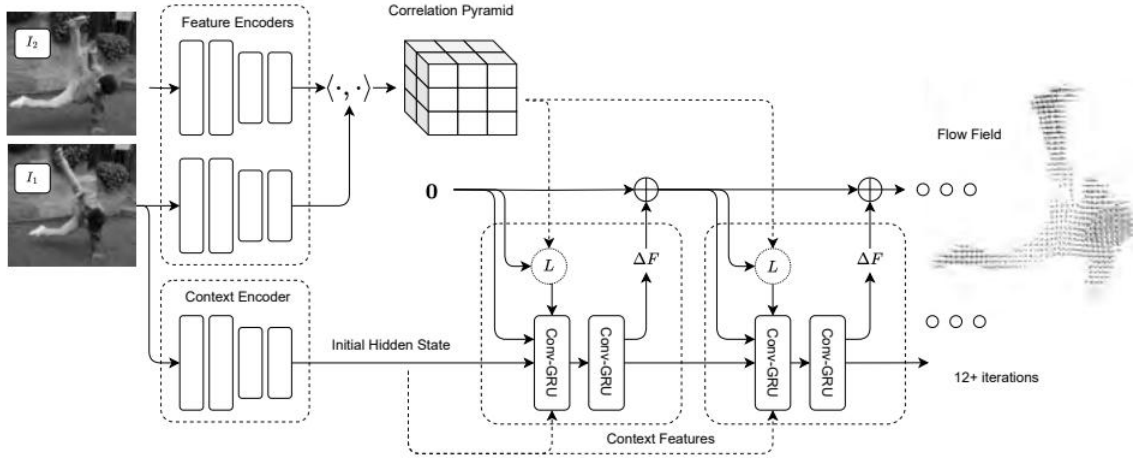


图 13.6: 图 13.7 RAFT (Recurrent All Pairs Field Transforms) 光流架构。使用一对残差网络从输入图像提取特征；随后，RAFT 对 I_1 和 I_2 的特征执行全对内积，构造相关体积。从初始化为 0 的光流场开始，RAFT 反复利用当前光流估计在相关金字塔中查询，并通过循环卷积 GRU 更新光流估计。

特征提取器。特征提取器将输入图像编码为较低分辨率的稠密特征图。RAFT 使用一对特征提取器：(1) 外观编码器 g_θ ；(2) 上下文编码器 c_θ 。二者将尺寸为 $(h_{in}, w_{in}, 3)$ 的输入图像映射为形状为 (h, w, c) 的低分辨率特征图，其中 c 是特征通道数。默认情况下，RAFT 以 $1/8$ 分辨率提取特征，因此 $(h, w) = (h_{in}/8, w_{in}/8)$ ；不过，其他网络也会使用更高分辨率。

相关金字塔。利用外观特征构建多分辨率相关金字塔。相关体积是一个四维张量，形状为 $h \times w \times h \times w$ ，其中 h 和 w 是上述外观特征图的尺寸。相关体积 C 通过计算 $g_\theta(I_1)$ 中所有像素与 $g_\theta(I_2)$ 中所有像素之间的内积构造：

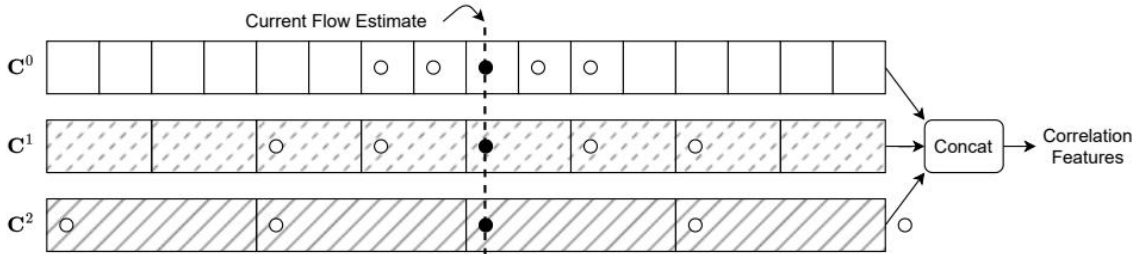


图 13.7: 图 13.8 RAFT 相关查询算子。该图展示了查询操作的一个一维切片。对于当前光流估计，RAFT 构造一个局部邻域，并从相关金字塔的每一层提取特征，再将这些特征展平并拼接。

$$C_{ijkl}(I_1, I_2) = \langle g_\theta(I_1)_{ij}, g_\theta(I_2)_{kl} \rangle = \sum_m g_\theta(I_1)_{ijm} \cdot g_\theta(I_2)_{klm}. \quad (13.4)$$

该操作可以通过一次矩阵乘法实现（展平空间维度并对第一个参数转置），因此可在 GPU 上高度优化。RAFT 不只使用单个相关体积，而是构造多分辨率相关金字塔。具体做法是对相关体积的最后两个维度迭代执行平均池化，得到四层金字塔 $\{C^0, C^1, C^2, C^3\}$ ，其中第 l 层体积 C^l 的形状为 $h \times w \times h/2^l \times w/2^l$ 。

相关查询。RAFT 定义了查询算子，利用当前光流 f 在相关金字塔中建立索引（图 13.8）。对于 $g_\theta(I_1)$ 中的像素 (u, v) ，光流场通过光流向量 $(u', v') = f(u, v) + (u, v)$ 计算其在 I_2 中的估计对应位置。

RAFT 随后在 (u', v') 周围构造 $(2r + 1) \times (2r + 1)$ 的邻域网格

$$N(u', v') = \{(u' + \Delta u, v' + \Delta v) \mid \Delta u, \Delta v \in \{-r, \dots, r\}\} \quad (13.5)$$

其中 r 是定义查询半径的超参数。利用双线性插值, 该邻域网格用于从 $\mathbf{C}^0$ 建立索引。对于低分辨率 $\mathbf{C}^l$, 在缩放后的对应坐标 $N(u'/2^l, v'/2^l)$ 周围构造邻域网格并采样特征。最终特征向量如图 13.8 所示, 由各层特征拼接而成, 同时包含小位移的细分分辨率特征和大位移的粗分辨率信息。

更新算子。RAFT 的权重绑定更新算子从零场开始, 估计光流场 $(f^1, \dots, f^N)$ 。在每次更新迭代中, 算子产生一个更新方向, 并将其应用于当前估计, 得到下一次估计 $f^{k+1} = \delta f^k + f^k$ 。

更新算子使用基于 GRU^[217] 的门控激活单元, 其中线性层由卷积替代。卷积允许信息在图像空间传播; 权重绑定和有界激活则有助于收敛到不动点。每次迭代接收四个输入 (每个输入都是二维特征图): (1) 上一次迭代的隐状态; (2) 上下文特征; (3) 当前光流估计; (4) 查询算子输出的相关特征。这些特征输入 Conv-GRU 后, 生成更新后的隐状态和光流更新 δf^k 。

13.2.4 作为视觉测量的光流

在静态场景中, 两帧之间的光流是深度和相机位姿的函数。设 I_i 和 I_j 是两幅图像, 其相机位姿 (相机到世界变换) 为 T_i^w 和 T_j^w 。给定深度为 d 的像素 (u, v) , 首先将该点反投影到深度 d , 再把三维点从帧 i 变换到帧 j , 最后投影到相机 j , 即可计算诱导光流:

$$\hat{f}(u, v) = \pi(T_i^j \cdot \pi^{-1}(u, v, d)) - (u, v) \quad T_i^j = (T_j^w)^{-1} \cdot T_i^w. \quad (13.6)$$

其中, $\pi: \mathbb{R}^3 \mapsto \mathbb{R}^2$ 将三维点投影到像素坐标, 逆投影函数 $\pi^{-1}: \mathbb{R} \times \mathbb{R}^2 \mapsto \mathbb{R}^3$ 则将带深度 d 的像素坐标 (u, v) 反投影到其三维坐标。

这里假设使用针孔相机; 如果已知畸变模型, 也可以在预处理阶段先去畸变, 从而使用包含径向或切向畸变的复杂相机模型。为方便记号, 将 π 和 π^{-1} 重载为作用于尺寸为 $h \times w$ 的稠密像素网格 $(\mathbf{u}, \mathbf{v})$ 的向量化算子, 使光流场可写为

$$h_{ij}(T_i^w, T_j^w, d_i) = \pi(T_i^j \cdot \pi^{-1}(\mathbf{u}, \mathbf{v}, d_i)) - \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} \quad (13.7)$$

其中 d 是一个 $h \times w$ 的稠密深度图。

13.2.5 根据光流估计位姿和深度

可以利用因子图反向求解: 使用光流估计来求解深度和相机位姿。考虑图像集合 $\{I_1, \dots, I_m\}$, 构建一个图, 其边连接具有光流估计的图像 (图 13.9(a)); $(i, j) \in \mathcal{G}$ 表示图像 I_i 和 I_j 之间有光流估计。将帧 I_i 与 I_j 之间的光流估计记为测量 z_{ij} (帧图中的每条边对应一个因子, 为便于记号, 使用元组 (i, j) 对因子索引), 状态变量 x 则由位姿 $(T_1^w, \dots, T_n^w)$ 和稠密深度图 $(d_1, \dots, d_n)$ 组成。可以构造一个描述给定状态变量时应观测到的光流的代价函数:

$$J(x) \triangleq \sum_{(i,j) \in \mathcal{G}} \|z_{ij} - h_{ij}(T_i^w, T_j^w, d_i)\|_{\Sigma_{ij}}^2 \quad (13.8)$$

其中测量预测函数 h_{ij} 在式 (13.7) 中定义。

该代价函数对应图 13.9(b) 中的因子图, 其中每个因子涉及三个状态变量: 源位姿、目标位姿和源深度。参数化有几点需要注意: 实践中最好使用逆深度参数化深度变量, 因为这样可得到更好的代价函数局部近似。由于位姿是流形状态变量, 还应使用局部参数化 $\exp(\xi_i^\wedge)T_i^w$ 。

13.3 可微束调整与 DROID-SLAM

可以使用 RAFT (或其他现成方法) 计算光流, 再最小化最小二乘优化问题求解位姿和深度。单独这样做的效果并不理想。光流网络针对光流任务训练, 而不是三维重建或位姿估计; 其损失函数未必能很好地刻

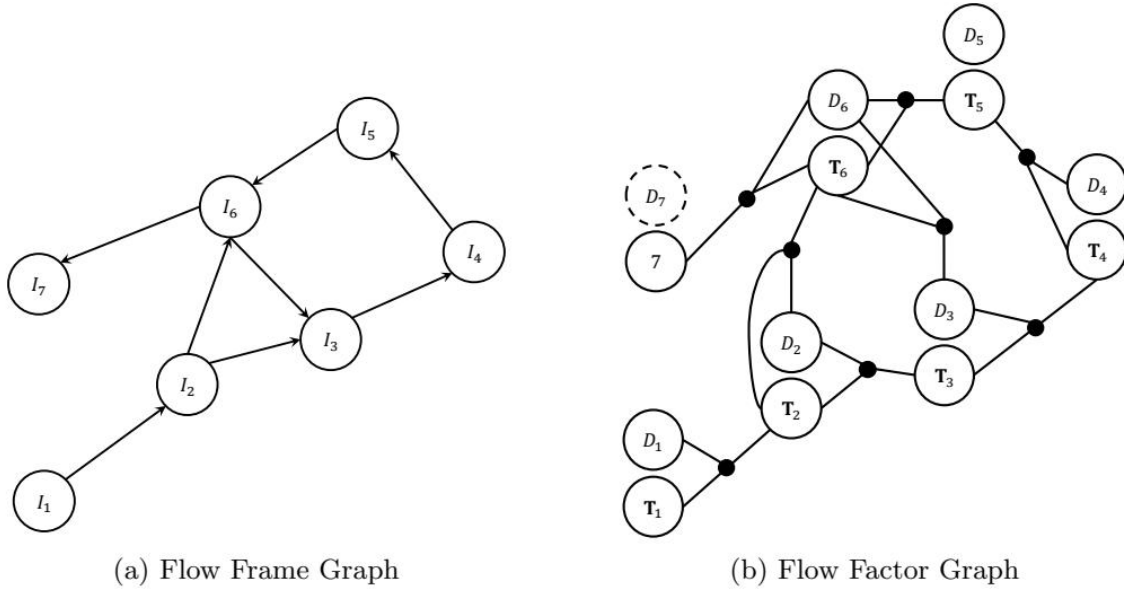


图 13.8: 图 13.9 光流帧图和光流因子图。

画 SLAM 真正关心的目标，尤其是相机位姿的精度。第二个问题是，光流网络的输出包含离群值，且代价函数对所有像素赋予相同权重。

一种解决方案是将光流等中间表示作为可微优化层的输入。利用第 4 章介绍的技术，可以通过优化层进行反向传播，对该中间表示实施监督，并进一步训练网络权重。此外，还可以让网络输出相应置信度权重，以进一步塑造优化景观。为具体说明这一思想，回顾第 1 章：因子图产生如下形式的代价函数

$$J(\mathbf{x}) \triangleq \sum_i \|\mathbf{e}_i(\mathbf{x}_i)\|_{\Sigma_i}^2 = \|\mathbf{e}(\mathbf{x})\|_{\Sigma}^2 \quad \mathbf{e}_i(\mathbf{x}) = \mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i) \quad (13.9)$$

其中 $\mathbf{e}_i$ 是误差向量，表示世界模型与观测测量之间的差异。误差向量 $\mathbf{e}_i$ 通常表示重投影误差（间接法）或光度误差（直接法）。

与其直接建模误差，不如使用神经网络生成误差及其置信度。定义神经网络函数 e_θ ，将当前状态 $\mathbf{x}$ （例如位姿）和测量 $\mathbf{z}$ （例如图像）映射为误差向量 $\mathbf{e} \in \mathbb{R}^m$ ，并以类似方式参数化协方差。代入学习得到的函数 e_θ 和 Σ_θ ，代价函数可改写为

$$J(\mathbf{x}) \triangleq \|\mathbf{e}_\theta(\mathbf{z}, \mathbf{x})\|_{\Sigma_\theta(\mathbf{z}, \mathbf{x})}^2 \quad (13.10)$$

从而允许神经网络塑造代价函数。

BANet^[1068] 是最早将 BA 集成到端到端可训练架构的工作之一，它在神经网络提取的特征图上定义光度误差。本节概述 DROID-SLAM^[1082] 和 DPVO^[1084]，这两个系统都将光流作为可微 BA 的中间表示来构建 SLAM 和 VO 系统。

13.3.1 DROID-SLAM 架构

DROID-SLAM 的架构（图 13.10）复用了 RAFT 的许多组件。它接收两个输入：输入图像序列 $(I_1, \dots, I_n)$ ，以及连接共视图像帧对的帧图；输出位姿和深度估计序列。

特征提取与视觉相似性。网络输入是帧序列 $(I_1, \dots, I_n)$ 和帧图 $\mathcal{G}$ 。与 RAFT 一样，DROID-SLAM 使用两个特征提取器：上下文编码器 c_θ 和外观编码器 g_θ ；二者都是残差网络，输出分辨率为输入的 1/8 的特征。

外观特征按照 RAFT 的方式构造一组相关金字塔。对帧图中的每条边 (i, j) ，利用 I_i 与 I_j 的外观特征相关性构造一个相关金字塔。如果帧图有 E 条边，总共会构造 E 个相关金字塔。

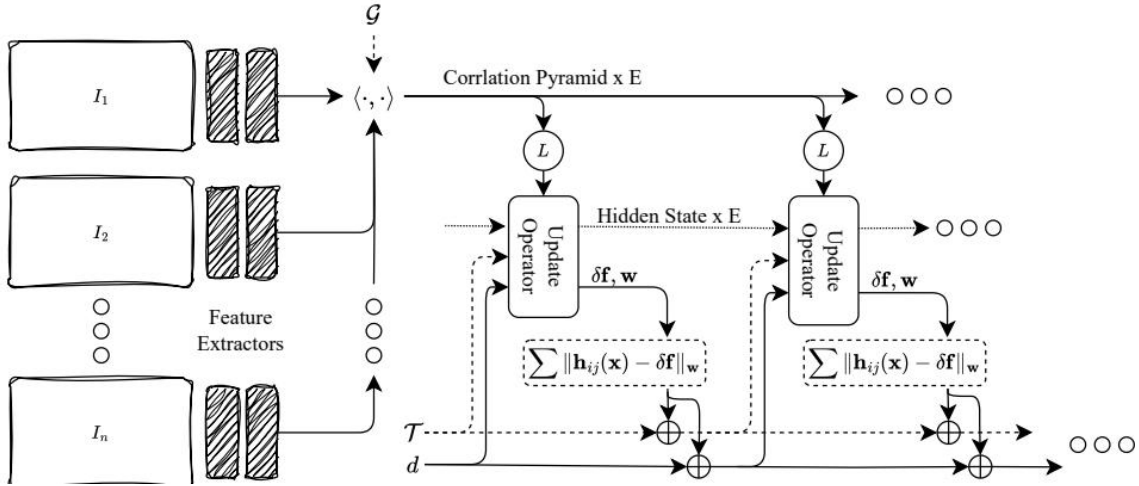


图 13.9: 图 13.10 DROID-SLAM 架构：从 N 幅输入图像提取特征，并构造 E 个相关体积（帧图每条边对应一个）。更新算子在帧图的每条边上对称运行。在每次迭代中，算子利用当前位姿估计 $\mathcal{T}_1, \dots, \mathcal{T}_n$ 和深度 $d_1, \dots, d_n$ 在相关金字塔中查询。相关特征和隐状态用于生成光流修正 δf 及置信度权重 w ，进而参数化代价函数。随后执行若干次 Gauss-Newton 更新，产生位姿和深度更新。

更新算子。更新算子作用于帧图，产生全局位姿和深度更新。帧图中的每条边 (i, j) 都有关联的相关金字塔 C_{ij} 和隐状态 $\mathbf{q}_{ij}$ 。隐状态 $\mathbf{q}_{ij}$ 从源帧 I_i 的上下文特征初始化。

更新算子的每次迭代使用当前位姿和深度估计，根据式 (13.7) 定义的测量函数计算帧图每条边的诱导光流。与 RAFT 类似，更新算子是一个卷积 GRU，但在此会针对帧图中的每条边复制一个实例。每次迭代中，更新算子使用：(1) 隐状态；(2) 相关特征；(3) 光流特征；(4) 上一次迭代的残差，生成更新后的隐状态。随后由更新后的隐状态预测测量误差 δf_{ij} （例如光流场更新）及关联置信度图 $\mathbf{w}_{ij}$ 。

可微束调整。利用误差估计和置信度图，可以构造如下形式的非线性最小二乘（NLS）BA 代价函数：

$$J(\mathbf{x}) \triangleq \sum_{(i,j) \in \mathcal{G}} \|\delta \mathbf{f}_{ij} - [\mathbf{h}_{ij}(\mathbf{x}_{ij}) - \mathbf{h}_{ij}(\mathbf{x}_{ij}^0)]\|_{\Sigma_{ij}}^2 \quad (13.11)$$

其中，先将 $\mathbf{w}_{ij}$ 转换为块对角矩阵 $\Sigma_{ij} = \text{diag } \mathbf{w}_{ij}$ ，以构造协方差； $\mathbf{x}_{ij}^0$ 是该迭代开始时的初始位姿和深度估计。该代价函数要求更新状态，使测量函数的变化 $\mathbf{h}_{ij}(\mathbf{x}_{ij}) - \mathbf{h}_{ij}(\mathbf{x}_{ij}^0)$ 与网络预测的误差 δf_{ij} 一致。权重 $\mathbf{w}_{ij}$ 使网络能够进一步控制优化景观：对网络有信心的图像区域赋予更大权重。通过展开 Gauss-Newton 迭代来更新深度和相机位姿。利用第 4 章介绍的技术，可以对每一个 Gauss-Newton 步骤执行反向传播，从而端到端训练网络。

训练。DROID-SLAM 在合成 TartanAir 数据集^[1166]上训练，该数据集包含真实深度和相机位姿。训练使用位姿损失和光流损失的加权组合。位姿损失惩罚网络预测位姿与真实位姿之间的差异；类似地，光流损失惩罚由预测位姿和深度通过式 (13.6) 诱导的光流与由真实位姿和深度诱导的光流之间的差异。

13.3.2 DROID-SLAM 推理

网络在固定规模设置（in-place setting）中训练，即接收固定数量的帧作为输入。若要在帧图动态变化、帧不断加入和移除的在线设置中运行，则需要若干修改。

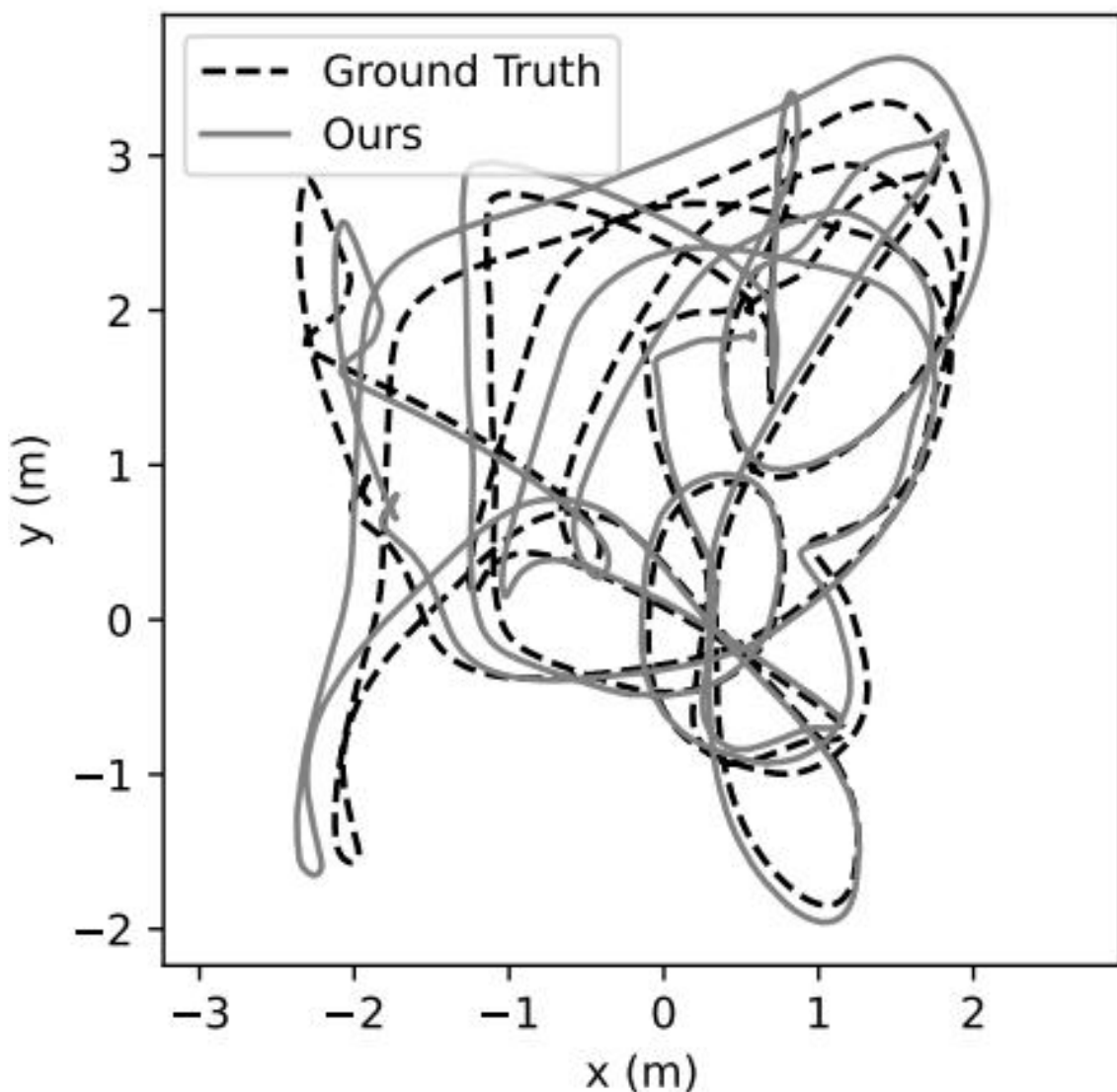
运动过滤。初始化时，选择运动足够的帧很重要。DROID-SLAM 根据相对于上一关键帧的相对运动，使用一个简单探测器过滤帧。设 I_{t-1} 是上一关键帧、 I_t 是输入帧，算法构造只有一条边 $I_{t-1} \rightarrow I_t$ 的帧图，然后运行更新算子计算光流修正 δf 。若 δf 的幅度低于阈值，则丢弃该帧；否则保留。

初始化。累积足够数量的帧后，DROID-SLAM 尝试初始化。它构造一个将每帧与时间邻居连接的帧图，

然后运行若干轮更新算子。尽管初始化策略很简单，但在所有基准数据集上都表现良好。当然，在纯旋转情况下该方法会失败。

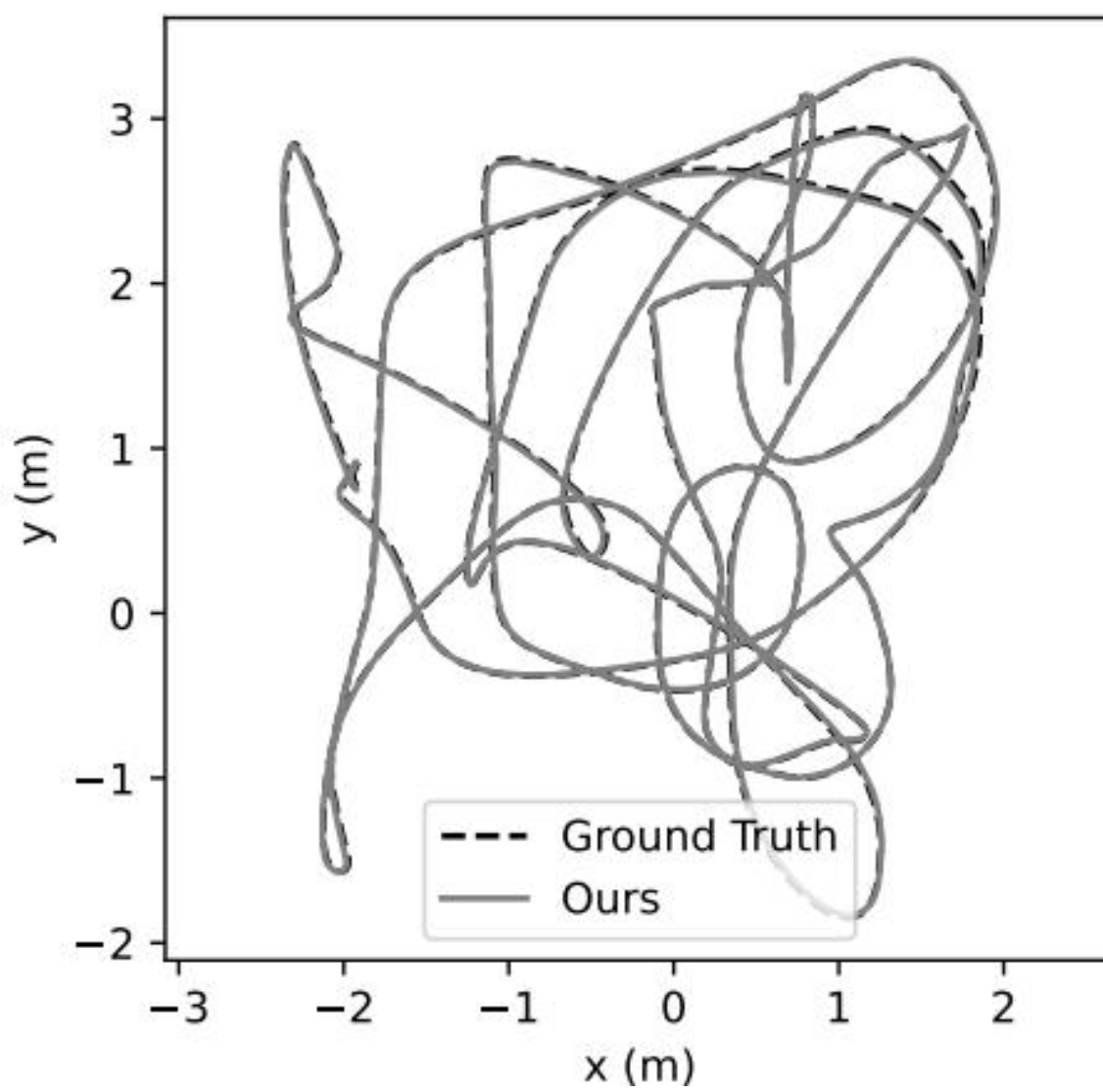
前端。加入新帧后，将相应边加入帧图。关键帧与前两帧之间总是双向加入时间边。系统还利用诱导光流计算所有关键帧对之间的距离，并在共视度高的帧之间添加新边。DROID-SLAM 通过对邻近边执行非极大值抑制等方式，避免加入过多边。

全局优化。DROID-SLAM 还使用后端过程联合优化完整帧图。它首先在时间相邻帧之间添加边，再利用当前位姿和深度估计计算所有帧对之间的光流。如果光流低于某一阈值，则将长距离边加入图中。这些长距离边可以在漂移不太严重的前提下实现中距离回环；但 DROID-SLAM 不包含重定位模块，因此无法闭合具有大量漂移的大回环。如图 13.11 所示，带中距离回环的全局优化能够显著提高估计轨迹的精度。



(a) 仅前端

(b) 全局优化后图 13.11 EuRoC 基准 V1 02 medium 场景中估计轨迹与真实轨迹的比较。全局优化将 RMSE ATE 从 16.5 cm 降至 1.2 cm。



13.3.3 泛化到其他模态

因子图表述使 DROID-SLAM 很容易推广到 RGB-D 和双目视频，而无需重新训练网络。除 RGB-D 和双目外，该方法还已推广到视觉惯性 SLAM^[864] 和事件相机^[760]。

RGB-D 视频。对于深度视频，只需加入一个附加因子，鼓励预测深度接近传感器深度即可。传感器深度估计通常是稀疏的，因此只在传感器读数有效的像素上应用该因子，得到更新后的代价函数

$$J(\mathbf{x}) \triangleq \sum_{(i,j) \in \mathcal{G}} \|\mathbf{z}_{ij} - \mathbf{h}_{ij}(\mathbf{x})\|^2 + \sum_i \|\bar{\mathbf{d}}_i - \mathbf{d}_i\|_{\Sigma_d}^2 \quad (13.12)$$

其中 $\bar{\mathbf{d}}_i$ 是传感器深度观测， Σ_d 是传感器协方差，用对缺失传感器值置零的对角矩阵表示。

双目视频。推广到双目视频并不困难。在双目情况下，每个输入由双目装置同步采集的左帧和右帧组成，并假设左右帧之间的相对位姿已知。DROID-SLAM 只需将左右帧都加入帧图；对于每个左帧，再加入连接其右侧对应帧的边。优化期间固定左右帧的相对位姿。图 13.12 表明，双目帧对能够改善性能；此外，全局优化还会带来额外收益。

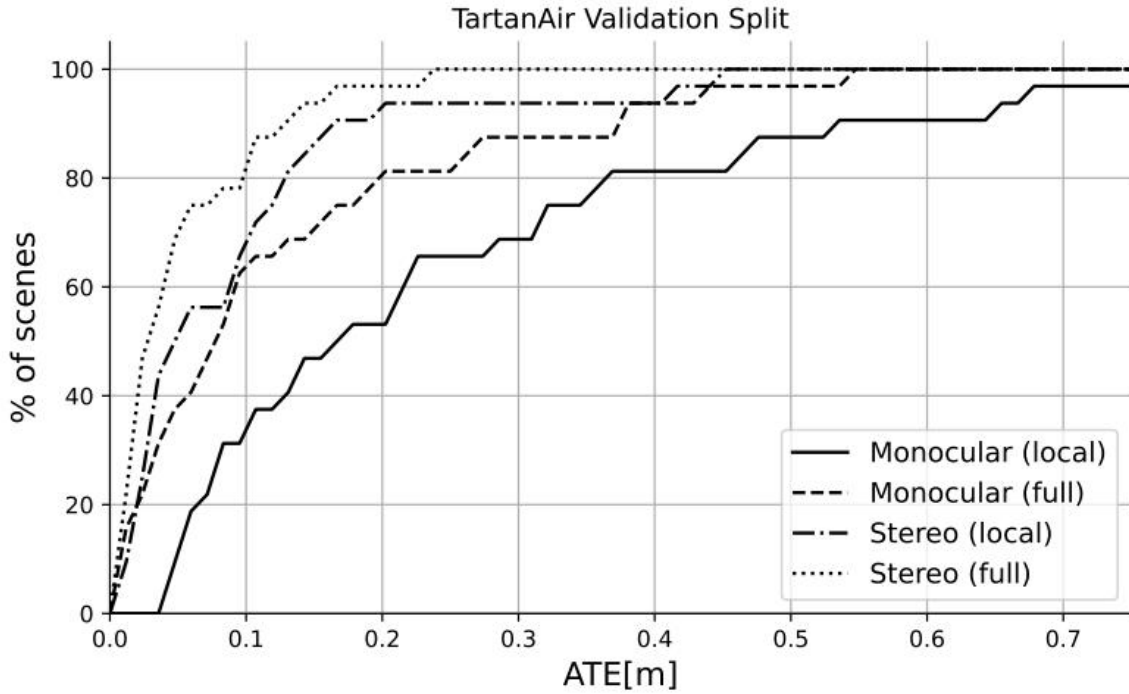


图 13.10: 图 13.12 DROID-SLAM 单目和双目版本在有、无全局优化时的比较。DROID-SLAM 在 TartanAir 验证集的 32 个序列上运行，并绘制绝对轨迹误差 (ATE) 低于各阈值的轨迹百分比。双目帧显著改善了性能，部分原因是解决了尺度漂移问题。单目和双目系统都受益于全局优化。

13.3.4 Deep Patch Visual Odometry (DPVO)

Deep Patch Visual Odometry^[1084] 与 DROID-SLAM 密切相关，但它处理的是稀疏图像块而非稠密帧。这不仅提高了计算效率，也带来精度方面的若干优势。

图像块图。DPVO 不处理完整帧，而是处理图像块，并使用图像块图数据结构存储图像块与图像之间的关联。图像块图是表示图像块生命周期的二部图，图中的边连接图像块与帧。

每个图像块被表示为其提取图像中的前视平面。因此，每个图像块只用一个（逆）深度参数化。为在帧之间重新投影图像块，需要更新测量函数：

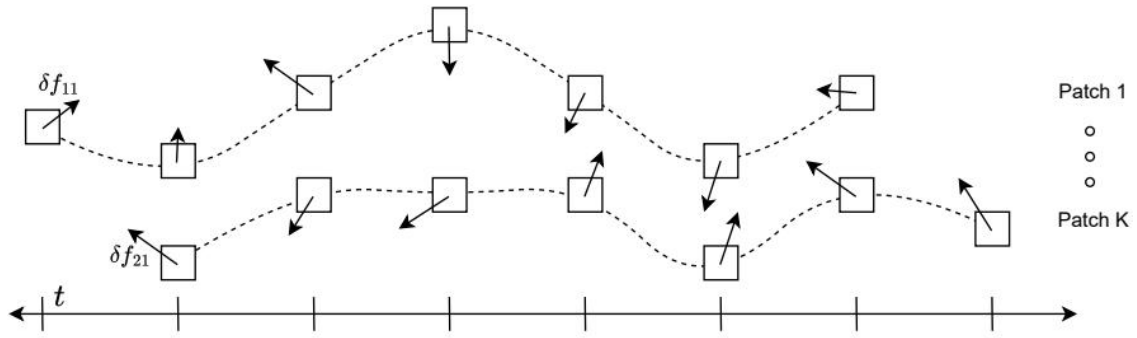


图 13.11: 图 13.13 DPVO 图像块图：每个图像块都随时间跟踪。更新算子为图中每条边输出光流修正 δf ，图中用误差向量表示。

为在帧之间重新投影图像块，需要更新测量函数

$$h_{ij}(\mathbf{T}_{s(i)}^w, \mathbf{T}_j^w, d_i) = \pi \left(\mathbf{T}_{s(i)}^j \cdot \pi^{-1}(\mathbf{u}, \mathbf{v}, d_i) \right) - \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} \quad (13.13)$$

其中 i 是图像块索引， $s(i)$ 是源索引，即提取图像块 i 的帧索引。 u 和 v 是图像块的像素坐标， d_i 是在整个图像块上共享的单个标量深度。

图像块提取。每幅输入图像都由特征编码器和上下文编码器处理，输出 1/4 分辨率的特征。随机采样图像块中心，并以此构造带一个像素偏移的 3×3 图像块（这里的 1 px 是提取特征图分辨率下的像素，因此对应原图中的 4 px）。DPVO 使用非常简单的方法构造图像块图：每个图像块与时间半径为 r 的所有帧相连。通过选择更多图像块并延长其生命周期，可以在计算量和精度之间进行权衡。

更新算子。更新算子作用于图像块图的边。与 RAFT 和 DROID-SLAM 一样，DPVO 的更新算子是一个循环式、权重绑定的单元，但加入了若干新组件。它在图像块生命周期上执行时间卷积，并使用聚合层在帧和图像块之间池化特征。更新算子为每条边产生光流更新 δf 及其置信度权重 w 。利用式 (13.7) 作为测量函数，可以据此构造代价函数，并通过展开两次 GN 迭代产生位姿和深度更新。

推理。DPVO 首先累积帧以完成初始化。累积到指定数量的帧后，通过运行若干轮更新算子进行初始化。之后，每加入一帧新图像，就从新帧提取图像块，并将边加入图像块图：包括连接先前图像块与新帧的前向边，以及连接新图像块与先前帧的后向边。随后运行更新算子细化深度和位姿估计。与 DROID-SLAM 一样，DPVO 还使用关键帧机制，确保优化窗口足够大。

回环与全局优化。DPVO 纯粹是视觉里程计系统，无法在优化窗口之外执行回环。Deep Patch Visual SLAM (DPVS) ^[672] 在 DPVO 基础上加入全局优化和回环。DPVS 借助重定位模块执行长距离回环，并根据邻近性检测执行短距离回环。与 DROID-SLAM 不同，DPVS 可以在单个 GPU 上保持实时性能。

13.4 DuSt3R

如前几节所示，从高层次看，现代 SLAM 系统可以描述为一系列步骤组成的流水线：首先获取帧之间的像素对应关系，包括关键点、图像块或稠密光流；然后对状态变量 x （例如相机参数和深度图）与观测对应关系之间的因子图建模；最后求解使整体代价函数 $J(x)$ 最小的状态变量更新。需要注意，在更广义的 SfM 场景中，待估计的相机参数还包括每台相机的内参标定，而在 SLAM 场景中通常假设内参已知。前几章介绍了如何在每个步骤中插入基于学习的方法（例如用于估计光流的神经网络），但总体而言，SLAM 流水线仍然高度依赖手工设计且相当复杂。

近来出现了一类新型的基于学习的方法。DuSt3R ^[1164] 不再将 SLAM 问题拆分为一系列原子步骤，而是让一个神经网络一次性解决所有步骤：仅给定图像，便可直接输出相机位姿、稠密深度图甚至内参，无需任何显式求解器或迭代算法。此类网络的应用远不止 SLAM，因为它不对输入图像之间的时间关系作任何假设，因此更一般地适用于 SfM 和稠密三维重建。

然而，要求网络从可能数千帧图像重建大型场景目前仍然难以实现。实践中，DUST3R 只解决包含两幅输入图像的局部三维重建问题。后文将看到，在给定一组局部三维重建、执行 SLAM（或大规模三维重建）时，最终仍需要某种形式的因子图。与以往方法相比，DUST3R 的输出更丰富，因此简化并拓展了 SLAM 系统的可能性，使系统更加鲁棒，并支持处理变化的相机内参（例如动态缩放）。

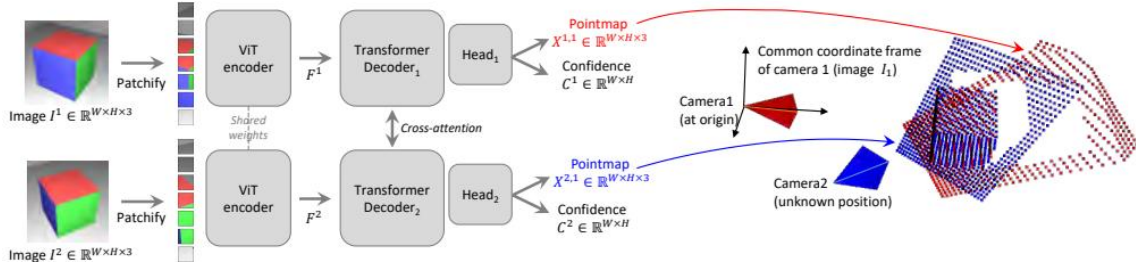


图 13.12: 图 13.14 网络架构。首先使用共享的 ViT 编码器，以 Siamese 方式对场景的两个视图 (I^1, I^2) 编码。得到的 token 表示 F^1 和 F^2 随后输入两个 Transformer 解码器，解码器通过交叉注意力不断交换信息。最后，两个回归头输出对应的两个点图及其置信度图。需要强调的是，两个点图均表示在第一幅图像 I^1 的同一坐标系中。网络使用简单的回归损失（式 (13.17)）训练。图像来自 ^[1164] (©2024 IEEE)。

概述。简言之，DUST3R 能够从未标定、未定姿的相机执行稠密、无约束立体三维重建。其核心组件是一个网络，仅从一对图像回归稠密且准确的场景表示，无需关于场景或相机的先验信息（甚至不需要内参）。所得场景表示基于具有丰富属性的三维点图，同时封装：(a) 场景几何；(b) 像素与场景点之间的关系；(c) 两个视点之间的关系。仅凭该输出，几乎所有场景参数（即相机和场景几何）都可以直接提取。这是因为网络联合处理输入图像和生成的三维点图，从而学习到强大的几何与形状先验，类似于多视图立体中常用的纹理形状、阴影形状或轮廓形状等先验 ^[997]。

点图。将稠密二维三维点场表示为点图 $X \in \mathbb{R}^{W \times H \times 3}$ 。与分辨率为 $W \times H$ 的对应 RGB 图像 I 关联后， X 在图像像素与三维场景点之间建立一一映射，即对所有像素坐标 $(i, j) \in \{1 \dots W\} \times \{1 \dots H\}$ ，都有 $I_{i,j} \leftrightarrow X_{i,j}$ 。这里假设每条相机射线只与一个三维点相交，即忽略半透明表面情况。

给定相机内参 $K \in \mathbb{R}^{3 \times 3}$ ，根据真实深度图 $D \in \mathbb{R}^{W \times H}$ 可以直接得到观测场景的点图 X ： $X_{i,j} = K^{-1} D_{i,j} [i, j, 1]^T$ 。这里 X 表示在相机坐标系中。下文用 $X^{n,m}$ 表示相机 n 的点图 X^n 在相机 m 坐标系中的表达：

$$X^{n,m} = h^{-1}(P_m P_n^{-1} h(X^n)) \quad (13.14)$$

其中 $P_m, P_n \in \text{SE}(3)$ 是图像 m 和 n 的世界到相机位姿；函数 $h : (x, y, z) \rightarrow (x, y, z, 1)$ 是齐次映射， $h^{-1} : (x, y, z, 1) \rightarrow (x, y, z)$ 是其逆映射。

13.4.1 广义立体重建网络

问题定义。构建一个通过直接回归解决广义立体情形三维重建任务的网络。网络 f 接收两幅 RGB 图像 $I^1, I^2 \in \mathbb{R}^{W \times H \times 3}$ ，输出两个对应的点图 $X^{1,1}, X^{2,1} \in \mathbb{R}^{W \times H \times 3}$ 及其置信度图 $C^{1,1}, C^{2,1} \in \mathbb{R}^{W \times H}$ 。需要注意，两个点图都表示在 I^1 的同一坐标系中；这与现有方法有根本不同，但带来了多项关键优势（见第 13.4.3 节）。

网络架构。如图 13.14 所示，网络由两个相同分支组成，每个分支包含图像编码器、解码器和回归头。两幅输入图像首先通过同一个共享权重的 ViT 编码器 ^[280] 以 Siamese 方式编码，得到两个 token 表示 F^1 和 F^2 ：

$$F^1 = \text{Encoder}(I^1), \quad F^2 = \text{Encoder}(I^2).$$

随后，网络在解码器中联合处理这两个表示。解码器是配备交叉注意力的通用 Transformer 网络。每个解码器块依次执行自注意力（一个视图的每个 token 关注同一视图的 token）、交叉注意力（一个视图的每个 token 关注另一视图的所有 token），最后将 token 输入 MLP。重要的是，在解码过程中两个分支不断共享信息，这对于输出正确对齐的点图至关重要。具体而言，每个解码器块都会关注另一个分支的 token：

$$\begin{aligned} G_i^1 &= \text{DecoderBlock}_i^1(G_{i-1}^1, G_{i-1}^2), \\ G_i^2 &= \text{DecoderBlock}_i^2(G_{i-1}^2, G_{i-1}^1), \end{aligned}$$

其中 $i = 1, \dots, B$ ，解码器包含 B 个块，并以编码器 token $G_0^1 := F^1$ 和 $G_0^2 := F^2$ 初始化。这里， $\text{DecoderBlock}_i^v(G^1, G^2)$ 表示分支 $v \in \{1, 2\}$ 中的第 i 个块， G^1 和 G^2 是输入 token，其中 G^2 是另一个分支的 token。最后，在每个分支中，独立的回归头接收解码器 token 集合，并输出点图及关联置信度图：

$$\begin{aligned} \mathbf{X}^{1,1}, \mathbf{C}^{1,1} &= \text{Head}^1(G_0^1, \dots, G_B^1), \\ \mathbf{X}^{2,1}, \mathbf{C}^{2,1} &= \text{Head}^2(G_0^2, \dots, G_B^2). \end{aligned}$$

需要注意，这种通用架构从未显式施加几何约束。因此，点图不一定对应任何物理上合理的相机模型（但在实践中能够很好地拟合）。相反，我们让网络学习训练集中的所有相关先验，而训练集只包含几何一致的点图。通用架构可以利用强大的预训练技术，最终超过现有特定任务架构所能达到的性能。

13.4.2 回归损失与置信度感知损失

三维回归损失。唯一的训练目标基于三维空间中的回归。记由式 (13.14) 得到的真实点图为 $\hat{\mathbf{X}}^{1,1}$ 和 $\hat{\mathbf{X}}^{2,1}$ ，并令 $\mathcal{D}^1, \mathcal{D}^2 \subseteq \{1 \dots W\} \times \{1 \dots H\}$ 表示真实值有定义的两组有效像素。视图 $v \in \{1, 2\}$ 中有效像素 $i \in \mathcal{D}^v$ 的回归损失定义为欧氏距离：

$$\ell_{\text{regr}}(v, i) = \left\| \frac{1}{z} \mathbf{X}_i^{v,1} - \frac{1}{\bar{z}} \hat{\mathbf{X}}_i^{v,1} \right\|. \quad (13.15)$$

为处理预测与真实值之间的尺度歧义，分别用缩放因子 $z = \text{norm}(\mathbf{X}^{1,1}, \mathbf{X}^{2,1})$ 和 $\bar{z} = \text{norm}(\hat{\mathbf{X}}^{1,1}, \hat{\mathbf{X}}^{2,1})$ 对预测点图和真实点图归一化。这两个因子表示所有有效点到原点的平均距离：

$$\text{norm}(\mathbf{X}^1, \mathbf{X}^2) = \frac{1}{|\mathcal{D}^1| + |\mathcal{D}^2|} \sum_{v \in \{1,2\}} \sum_{i \in \mathcal{D}^v} \|\mathbf{X}_i^v\|. \quad (13.16)$$

置信度感知损失。实际上，有些三维点定义不明确，例如天空或半透明物体上的点。更一般地说，图像中的某些区域通常比其他区域更难预测。因此，我们联合学习为每个像素预测一个分数，表示网络对该像素的置信程度。最终训练目标是在所有有效像素上计算式 (13.15) 的置信度加权回归损失：

$$\mathcal{L}_{\text{conf}} = \sum_{v \in \{1,2\}} \sum_{i \in \mathcal{D}^v} C_i^{v,1} \ell_{\text{regr}}(v, i) - \alpha \log C_i^{v,1}, \quad (13.17)$$

其中 $C_i^{v,1}$ 是像素 i 的置信度分数， α 是控制正则项的超参数^[221]。为确保置信度严格为正，通常定义 $C_i^{v,1} = 1 + \exp c_i^{v,1} \gg 0$ ，其中 $c_i^{v,1} \in \mathbb{R}$ 。这样会迫使网络在较困难区域（例如仅被单个视图覆盖的区域）进行外推。以该目标训练网络 f ，无需显式监督即可估计置信度分数。图 13.15 展示了输入图像对及其对应输出的实例。

13.4.3 下游应用

输出点图的丰富属性使我们能够恢复下面介绍的所有场景参数。

恢复内参。根据定义，点图 $\mathbf{X}^{1,1}$ 表示在 I^1 的相机坐标系中。因此，可以通过求解一个简单优化问题估计相机内参。假设主点近似位于图像中心且像素为正方形，此时只需估计焦距 f_1^* ：

$$f_1^* = \arg \min_{f_1} \sum_{i=0}^W \sum_{j=0}^H C_{i,j}^{1,1} \left\| (i', j') - f_1 \frac{(X_{i,j,0}^{1,1}, X_{i,j,1}^{1,1})}{X_{i,j,2}^{1,1}} \right\|,$$

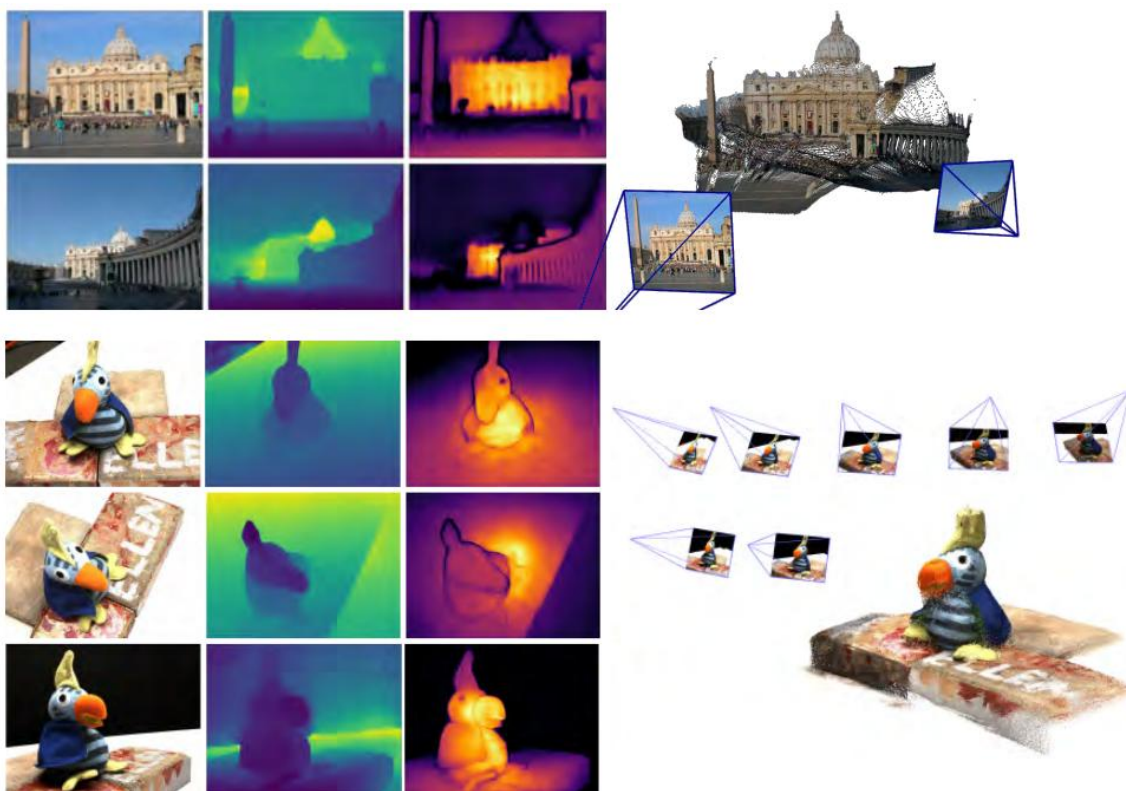


图 13.13: 图 13.15 两个训练期间未见过的场景的重建示例。从左到右依次为 RGB 图像、深度图、置信度图和重建结果。上方场景显示 $f(I^1, I^2)$ 的原始输出；下方场景显示全局对齐后的结果（第 13.4.4 节）。图像来自 ^[1164] (©2024 IEEE)。

其中 $i' = i - W/2$, $j' = j - H/2$ 。快速迭代求解器（例如基于 Weiszfeld 算法^[878]的求解器）可以在几次迭代内找到最优 f_1^* 。对于第二台相机的焦距 f_2^* ，最简单的选择是对图像对 (I^2, I^1) 执行推理，并使用上述公式，将 $X^{1,1}$ 换成 $X^{2,2}$ 。

相对位姿估计。一种方法是执行二维匹配，按上文方法恢复内参，然后估计本质矩阵并恢复相对位姿^[437]。另一种更直接的方法，是使用 Procrustes 对齐^[708]比较点图 $X^{1,1}$ 和 $X^{1,2}$ （或者等价地比较 $X^{2,2}$ 和 $X^{1,2}$ ），得到带尺度的相对位姿 $P^* = \sigma^*[R^*|t^*]$ ：

$$P^* = \arg \min_{\sigma, R, t} \sum_i C_i^{1,1} C_i^{1,2} \|\sigma(RX_i^{1,1} + t) - X_i^{1,2}\|^2,$$

该问题可以闭式求解。不过，Procrustes 对齐对噪声和离群值较敏感。更鲁棒的方案是结合 RANSAC^[328]与 PnP^[437, 640]。

13.4.4 全局对齐

上一节介绍的网络 f 一次只能处理一对图像。DUST3R^[1164]还提出了一种针对大场景的简单后处理优化，将多幅图像预测的点图对齐到联合三维空间。这得益于点图包含的丰富内容：它们按设计包含两幅已对齐的点云，以及像素到三维点的对应映射。

成对图。给定场景图像集合 $\{I^1, I^2, \dots, I^N\}$ ，首先构造连通图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ ，其中 N 幅图像构成顶点集合 $\mathcal{V}$ ，边 $e = (n, m) \in \mathcal{E}$ 表示图像 I^n 与 I^m 共享一些视觉内容。为此，可以使用现成的图像检索方法，也可以将所有图像对输入网络 f （在 H100 GPU 上一次推理耗时约 25 ms），根据每一对图像的平均置信度测量重叠程度，再过滤低置信度图像对。

全局优化。利用连通图 $\mathcal{G}$ ，为所有相机 $n = 1 \dots N$ 恢复全局对齐的点图 $\{\chi^n \in \mathbb{R}^{W \times H \times 3}\}$ 。对于每个图像对 $e = (n, m) \in \mathcal{E}$ ，首先预测成对点图 $X^{n,n}, X^{m,n}$ 及其置信度图 $C^{n,n}, C^{m,n}$ 。为清晰起见，定义 $X^{n,e} := X^{n,n}$ 和 $X^{m,e} := X^{m,n}$ 。由于目标是将所有成对预测表示在同一坐标系中，因此为每条边 $e \in \mathcal{E}$ 引入成对位姿 $P_e \in \mathbb{R}^{3 \times 4}$ 和尺度 $\sigma_e > 0$ ，并构造如下优化问题：

$$\chi^* = \arg \min_{\chi, P, \sigma} \sum_{e \in \mathcal{E}} \sum_{v \in e} \sum_{i=1}^{HW} C_i^{v,e} \|\chi_i^v - \sigma_e P_e X_i^{v,e}\|. \quad (13.18)$$

这里略去了一点记号细节：若 $e = (n, m)$ ，则 $v \in e$ 表示 $v \in \{n, m\}$ 。对于给定图像对 e ，同一个刚体变换 P_e 应将两个点图 $X^{n,e}$ 和 $X^{m,e}$ 分别与世界坐标点图 χ^n 和 χ^m 对齐，因为按定义它们已经表示在同一坐标系中。为避免 $\sigma_e = 0, \forall e \in \mathcal{E}$ 的平凡最优解，约束 $\prod_e \sigma_e = 1$ 。

恢复相机参数。对该框架进行直接扩展即可恢复所有相机参数。只需将

$$\chi_{i,j}^n := P_n^{-1} h(K_n^{-1} D_{i,j}^n [i, j, 1]^\top)$$

替代点图（即像式 (13.14) 一样强制采用标准针孔相机模型），便可以估计所有相机位姿 $\{P_n\}$ 、对应内参 $\{K_n\}$ 和深度图 $\{D^n\}$ ，其中 $n = 1 \dots N$ 。

13.5 MAST3R

DUST3R^[1164]首创的双视图三维重建先验，利用精心整理的三维数据集，给运动恢复结构 (SfM) 带来了范式转变。除了内参和相对位姿外，DUST3R 还可以从点图中获取可靠的像素对应关系：方法是在某个不变特征空间中寻找互为匹配的特征^[1164, 295, 990, 1193]。即使存在极端视点变化，这种方案也能良好工作；但所得对应关系相当不精确，导致次优精度。这是自然结果，因为：(i) 回归本身会受到噪声影响；(ii) DUST3R 从未针对匹配任务进行显式训练。

13.5.1 匹配头

因此，MASt3R 在 DUST3R 基础上增加第二个头，输出两个稠密特征图 D^1 和 $D^2 \in \mathbb{R}^{H \times W \times d}$ ，其中 d 是特征维数：

$$\mathbf{D}^1 = \text{Head}_{\text{desc}}^1([H^1, H'^1]), \quad (13.19)$$

$$\mathbf{D}^2 = \text{Head}_{\text{desc}}^2([H^2, H'^2]). \quad (13.20)$$

该头由两个线性层组成，中间交错使用非线性 GELU 激活函数^[447]。每个局部特征都归一化为单位范数。

13.5.2 匹配目标

MASt3R 的匹配目标是鼓励一幅图像中的每个局部描述子，至多与另一幅图像中表示同一三维场景点的一个描述子匹配。为此，MASt3R 在真实对应关系集合 $\hat{\mathcal{M}} = \{(i, j) \mid \hat{X}_i^{1,1} = \hat{X}_j^{2,1}\}$ 上使用 InfoNCE^[1120] 损失：

$$\mathcal{L}_{\text{match}} = - \sum_{(i,j) \in \hat{\mathcal{M}}} \left(\log \frac{s_{\tau}(i, j)}{\sum_{k \in \mathcal{P}^1} s_{\tau}(k, j)} + \log \frac{s_{\tau}(i, j)}{\sum_{k \in \mathcal{P}^2} s_{\tau}(i, k)} \right). \quad (13.21)$$

其中相似度定义为

$$s_{\tau}(i, j) = \exp \left[-\tau \mathbf{D}_i^{1\top} \mathbf{D}_j^2 \right]. \quad (13.22)$$

这里， $\mathcal{P}^1 = \{i \mid (i, j) \in \hat{\mathcal{M}}\}$ 和 $\mathcal{P}^2 = \{j \mid (i, j) \in \hat{\mathcal{M}}\}$ 分别表示两幅图像中参与计算的像素子集， τ 是温度超参数。需要注意，该匹配目标本质上是交叉熵分类损失：不同于 DUS3R 的回归损失，只有预测出正确像素而不是邻近像素时网络才能获得奖励。因此，它强烈鼓励网络实现高精度匹配。最后，将回归损失和匹配损失组合，得到最终训练目标：

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{conf}} + \beta \mathcal{L}_{\text{match}}. \quad (13.23)$$

13.6 将 MASt3R 扩展到 SfM 和 SLAM

DUS3R 和 MASt3R 旨在估计图像对之间的点图。如前所述，全局优化可以将 MASt3R 和 DUS3R 扩展为多帧重建。本节介绍将 MASt3R 集成到完整 SfM 和 SLAM 系统的工作，这些系统能够重建完整视频序列和大规模帧集合。

13.6.1 MASt3R-SfM

MASt3R^[642] 能够在一次前向传播中执行局部三维重建和匹配。MASt3R-SfM^[289] 因此将其扩展为完整集成的 SfM 流水线，处理完全无约束的输入图像集合，从单视图到大规模场景均可适用，甚至允许相机不发生运动。由于 MASt3R 的基本单元是图像对，对大规模图像集合的扩展性较差。为解决这一问题，MASt3R-SfM 修改冻结的编码器，执行快速图像检索而几乎不增加计算开销，得到关于图像数量近似线性复杂度的可扩展 SfM 方法。得益于 MASt3R 对离群值的鲁棒性，该方法完全不需要 RANSAC。SfM 优化分为两次连续梯度下降，均基于 MASt3R 输出的冻结局部重建：首先使用三维空间中的匹配损失，再使用二维重投影损失细化上一步估计。

13.6.2 MUST3R

MASt3R-SfM 可以无缝处理几十幅图像；然而，当输入更多图像时，DUS3R 和 MASt3R 的成对性质反而成为缺点。由于预测点图表示在每一对图像第一幅图像定义的局部坐标系中，所有预测都位于不同坐标系。因而，若采用朴素方法，需要全局后处理步骤将所有预测对齐到一个全局坐标系；对于大规模集合，这很快变得难以处理。这引出了成对方法的二次复杂度问题，以及实时场景下鲁棒快速优化的问题。MASt3R-SfM^[289] 部分解决了这些问题，而 MUST3R^[135] 采取不同思路，设计了一种可扩展到任意规模大图像集合的新

架构，并能够以高帧率在同一坐标系中推断相应点图。为实现这一点，MUST3R 在 DUST3R 架构上进行若干关键修改，包括使架构对称化并加入工作记忆机制，同时只增加有限复杂度。除处理 SfM 场景中无序图像集合的离线重建外，该模型还可处理稠密视觉里程计和 SLAM，即在线预测运动相机记录的视频流中的相机位姿和三维结构。MUST3R 可以无缝利用记忆机制覆盖这两种场景，无需改变架构，同一网络即可以任务无关的方式执行两种任务（见图 13.16）。



图 13.14: 图 13.16 MUST3R 在 Aachen Day-Night^[1284] nexus4 序列 5（离线，顶部）和 TUM-^[1045] RGBD room 序列（在线，底部）上的重建定性示例。图像来自^[135]（©2025 IEEE）。

13.6.3 MAST3R-SLAM

最后介绍一种使 MAST3R 具备可扩展性的替代方案。MASt3R-SLAM^[789] 是一个实时 SLAM 框架，将 MAST3R 的双视图三维重建先验作为跟踪、建图和重定位的统一基础，如图 13.17 所示。MASt3R-SfM 在无序图像集合的离线设置中将这先验应用于 SfM^[289]；而 SLAM 以增量方式接收数据，并必须保持实时运行，因此需要低延迟匹配、谨慎的地图维护以及高效的大规模优化方法。一方面，MASt3R-SLAM 借鉴 SLAM 系统的滤波和优化技术^[1080, 1081]，在前端对点图执行局部滤波，以支持后端的大规模全局优化。另一方面，与 MAST3R 类似，它不对每幅图像的相机模型作假设，只要求存在一个所有射线通过的唯一相机中心。由此得到一个实时稠密单目 SLAM 系统，能够使用通用的、随时间变化的相机模型重建场景。在已知标定参数时，MASt3R-SLAM 在轨迹精度和稠密几何估计方面展示了当前最先进的性能。

13.7 延伸阅读与近期趋势

基于深度学习的 SLAM 系统正沿多条不同路线发展。研究者一方面继续提升其精度和鲁棒性，另一方面也在努力改善这类 SLAM 系统的可扩展性。

一条研究路线继续将 DUST3R 等前馈网络扩展到数百幅图像^[1156, 1225]。VGGT^[1156] 可以在数秒量级重建数百幅图像，并直接预测相机位姿、内参、深度、点图和特征轨迹，无需三维优化和后处理。不过，VGGT 仍可利用预测的特征轨迹，通过束调整进一步细化相机位姿和内参；因此，在需要高精度预测时，因子图仍发挥关键作用。为了将 VGGT 用于更长图像序列，VGGT-SLAM^[724] 构建了一个 SLAM 框架：使用 VGGT 重建局部子地图，再借助因子图将其对齐到全局坐标系。

另一条研究路线持续改进因子图与深度学习模型的组合。DROID-SLAM 要求输入时已知相机标定，并且在中等动态环境中会失败；MegaSAM^[660] 和 Video Pose Engine (ViPE)^[489] 则可在高度动态环境中运行，且无需已知标定。相反，它们在推理期间联合优化相机标定。这些方法利用最新的深度估计和语义分割模型，并依靠因子图融合多个模型的预测。

SLAM 和三维重建模型的一个近期趋势，是建立在预训练视觉 Transformer 之上。利用自监督学习技术训练的视觉基础模型^[1177, 831]（另见第 17 章）已被证明能为下游三维重建任务提供出色的初始化。随着预训练方法改进，这些改进也可能转化为下游三维任务的性能提升。

这些发展的最终成果，是能够无缝处理包含动态物体、纯旋转和未知相机标定等挑战的视频的系统。近期利用深度学习的方法已足够鲁棒，可以大规模重建互联网视频，例如 10 万个 YouTube 视频^[489, 931]。

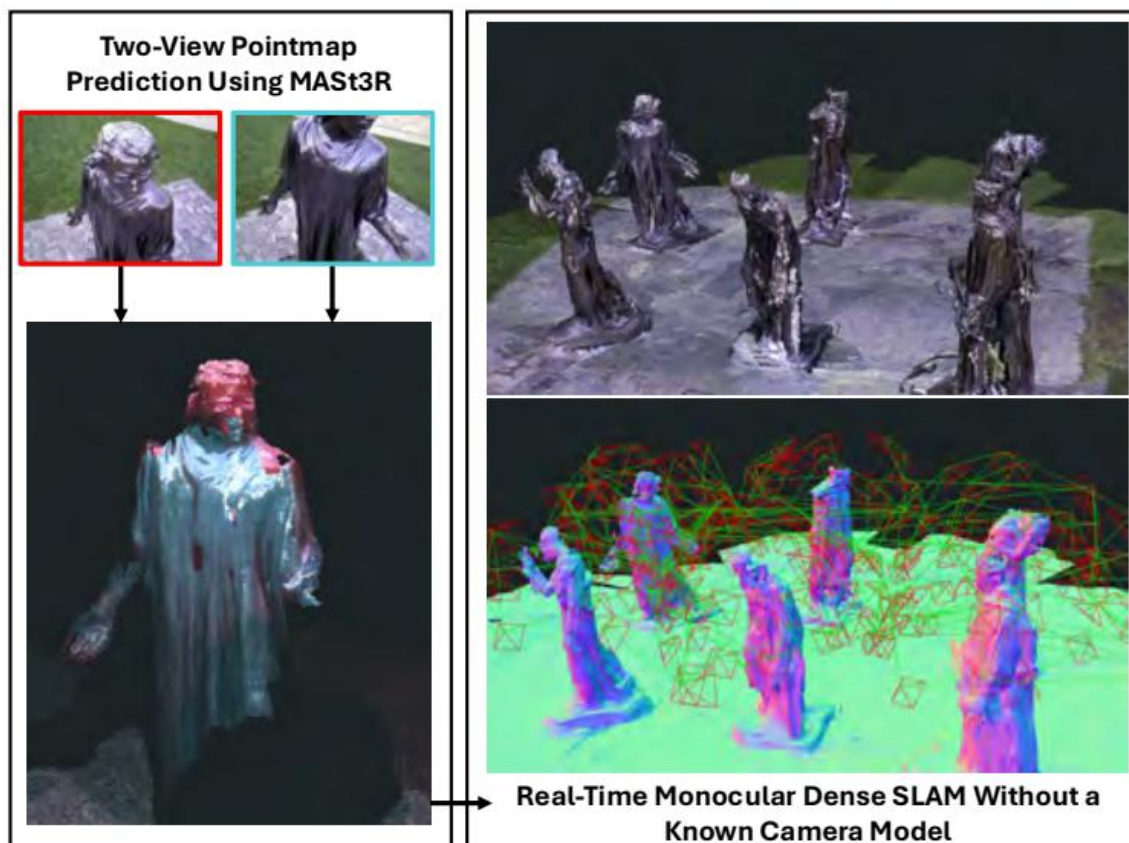


图 13.15: 图 13.17 Burghers 序列^[1296]上稠密单目 SLAM 系统的重建结果。利用左侧所示的 MAST3R 双视图预测, 该系统无需已知相机模型即可实时实现全局重建^[789] (©2025 IEEE)。

第十四章 可微体渲染的地图表示

作者: Hidenobu Matsuki 和 Andrew J. Davison

底层场景表示的选择会显著影响 SLAM 系统的能力和效率。如第 5 章所述, SLAM 中每种场景或地图表示都各有优缺点, 具体取决于相机自运动跟踪、稠密场景重建或对象级分解等下游任务。近年来, 得益于图形硬件和自动微分框架的进步, 可通过可微渲染优化的三维场景表示, 例如 neural radiance field (NeRF)^[764] 和 3D Gaussian Splatting (3DGS)^[552], 已成为计算机图形学界进行高质量逆渲染和新视图合成的重要选择。

这些表示最初面向已知相机位姿条件下的离线新视图合成, 后来逐渐被用于三维重建和场景理解的多种其他任务^[1202], 包括大规模场景的准确稠密几何重建^[819, 1241, 1159]、语义融合^[1291, 1292, 586]等。早期实现需要数天优化; 通过利用体素或点云等显式数据结构实现快速数据访问, 优化时间已降至接近实时。近来, 3D Gaussian Splatting 提供了极快且高质量的渲染能力, 已经在许多应用中有效替代 Neural Radiance Fields。这些新的图形基元也正在被纳入 SLAM 研究。尽管将可微渲染与三维占据地图结合的思想可追溯至 2000 年代初 Thrun 等人提出的“前向传感器模型”^[1089], 现代硬件和算法正在开启全新的可能性。

本章概述 NeRF 和 3DGS 作为 SLAM 场景表示的方法, 讨论其优缺点并提出未来研究方向。我们先回顾三维场景表示的历史(第 14.1 节), 再详细介绍 NeRF(第 14.2 节)和 3DGS(第 14.3 节), 包括它们与 SLAM 的集成, 最后讨论潜在未来方向(第 14.4 节)。

14.1 三维场景表示与可微渲染

14.1.1 可学习三维表示

随着深度学习在二维图像领域取得成功^[260, 443, 444], 研究界逐步探索用于三维任务的学习方法。早期工作直接使用点^[890]、占据栅格^[929]等经典数据结构表示的三维数据, 并将其用于下游任务。然而, 这些经典表示是离散的, 通常不适合深度学习: 它们难以以紧凑方式表示拓扑灵活的水密连续表面, 从而限制了神经网络训练所必需的基于梯度的优化。为构建更利于优化的三维表示, 一些早期工作使用神经网络作为标量函数近似器来定义占据函数或有符号距离函数^[759, 847, 192], 并通过直接三维监督训练。举例来说, DeepSDF^[847]如图 14.1 所示, 采用八层多层感知机 (MLP), 直接从三维坐标和可选潜变量回归有符号距离函数 (SDF)。这些方法展示了不受拓扑和离散化误差约束的连续、紧凑三维形状重建。但三维数据的监督训练需要昂贵的真实标注, 难以扩展数据集采集。这一限制促使研究者利用更易获得的二维图像信息和不同程度的监督进行三维场景理解, 进而发展出通过可微渲染实现的三维重建。

(a) 体素

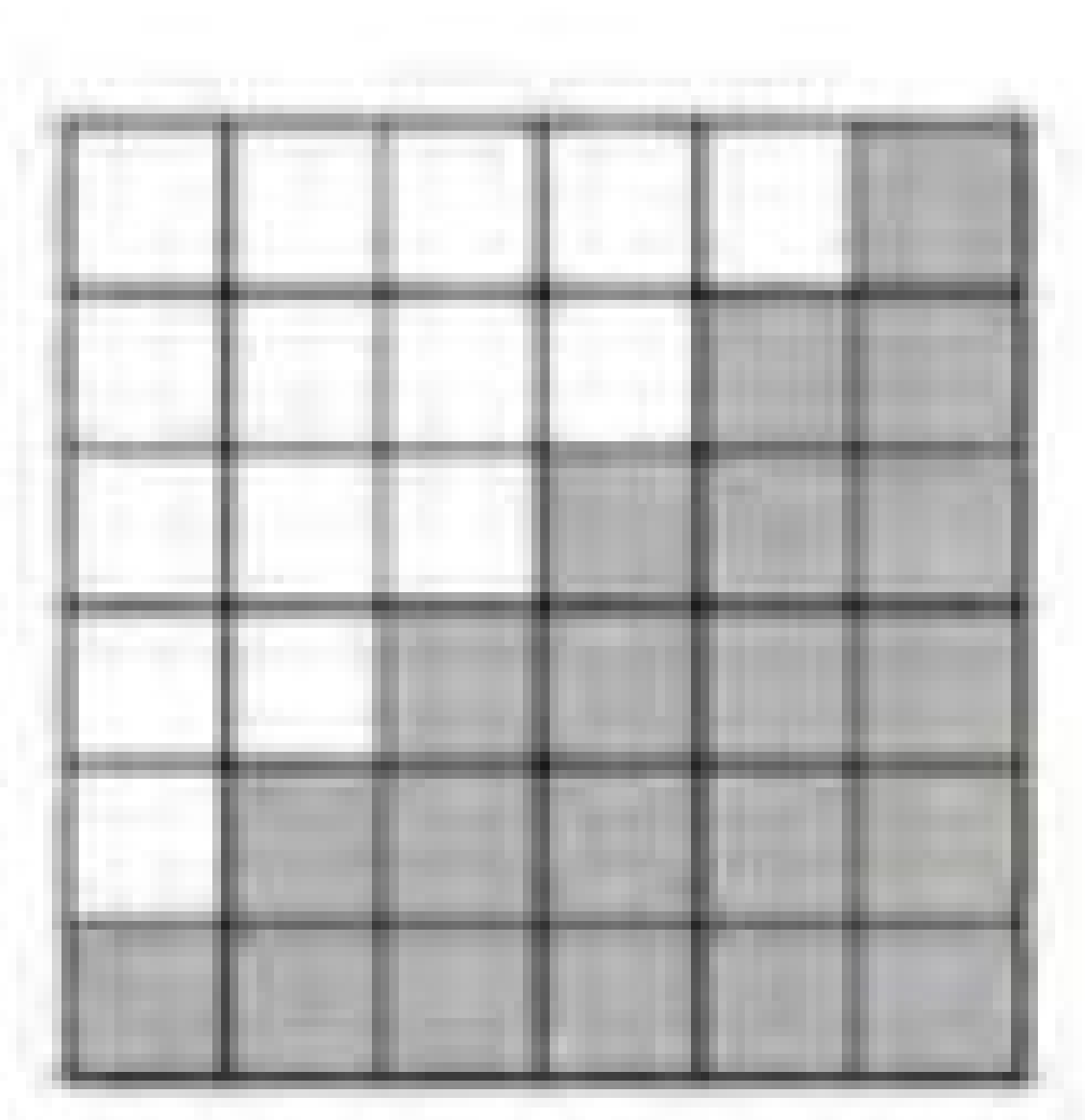
(b) 点

(c) 网格

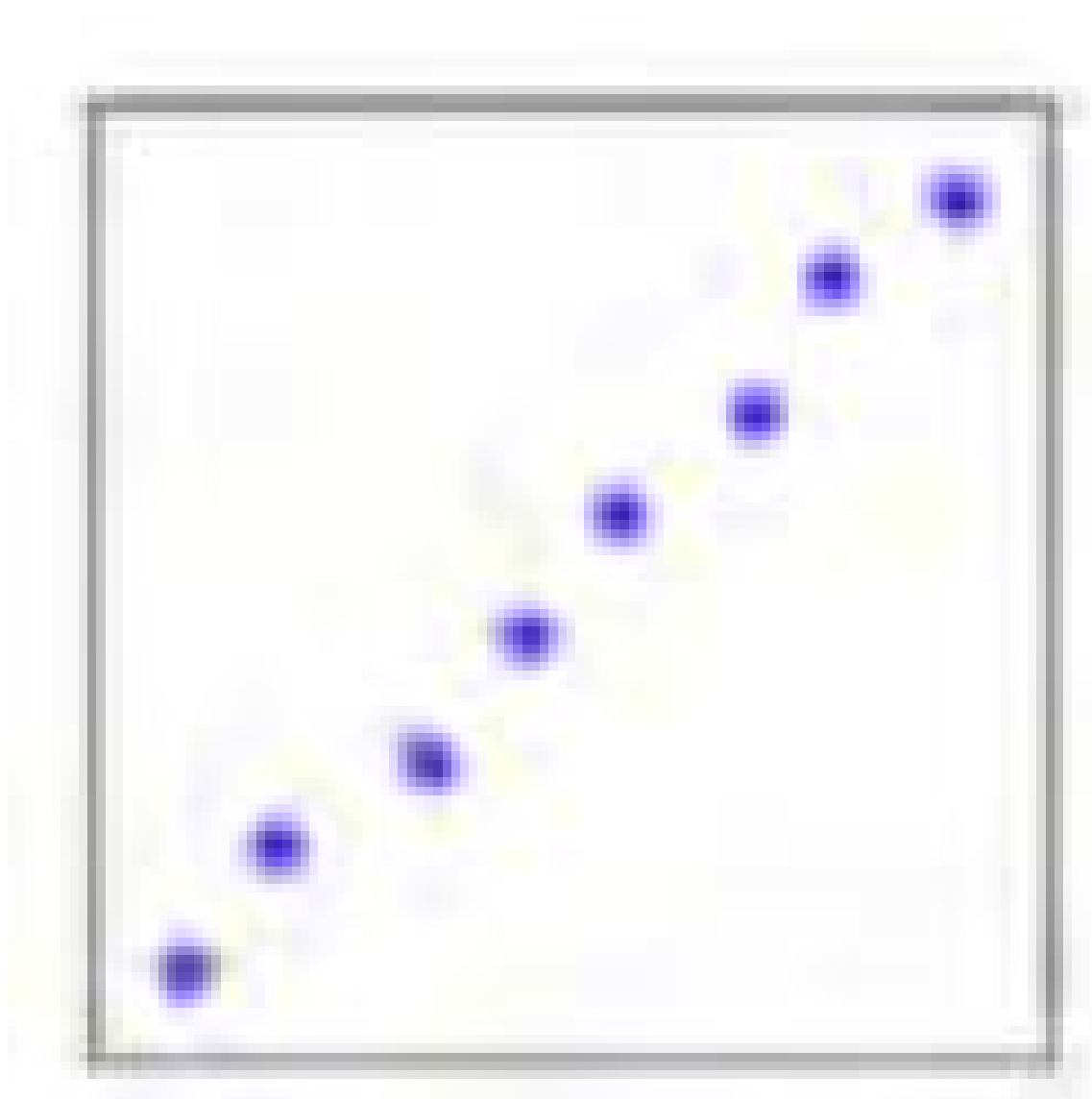
(a) Occupancy Networks^[759] (d) 本文方法

(b) DeepSDF^[847] 图 14.1 可学习三维表示。ONet^[759] (©2019 IEEE) 和 DeepSDF^[847] (©2019 IEEE)

都使用神经网络回归连续隐式表面函数。



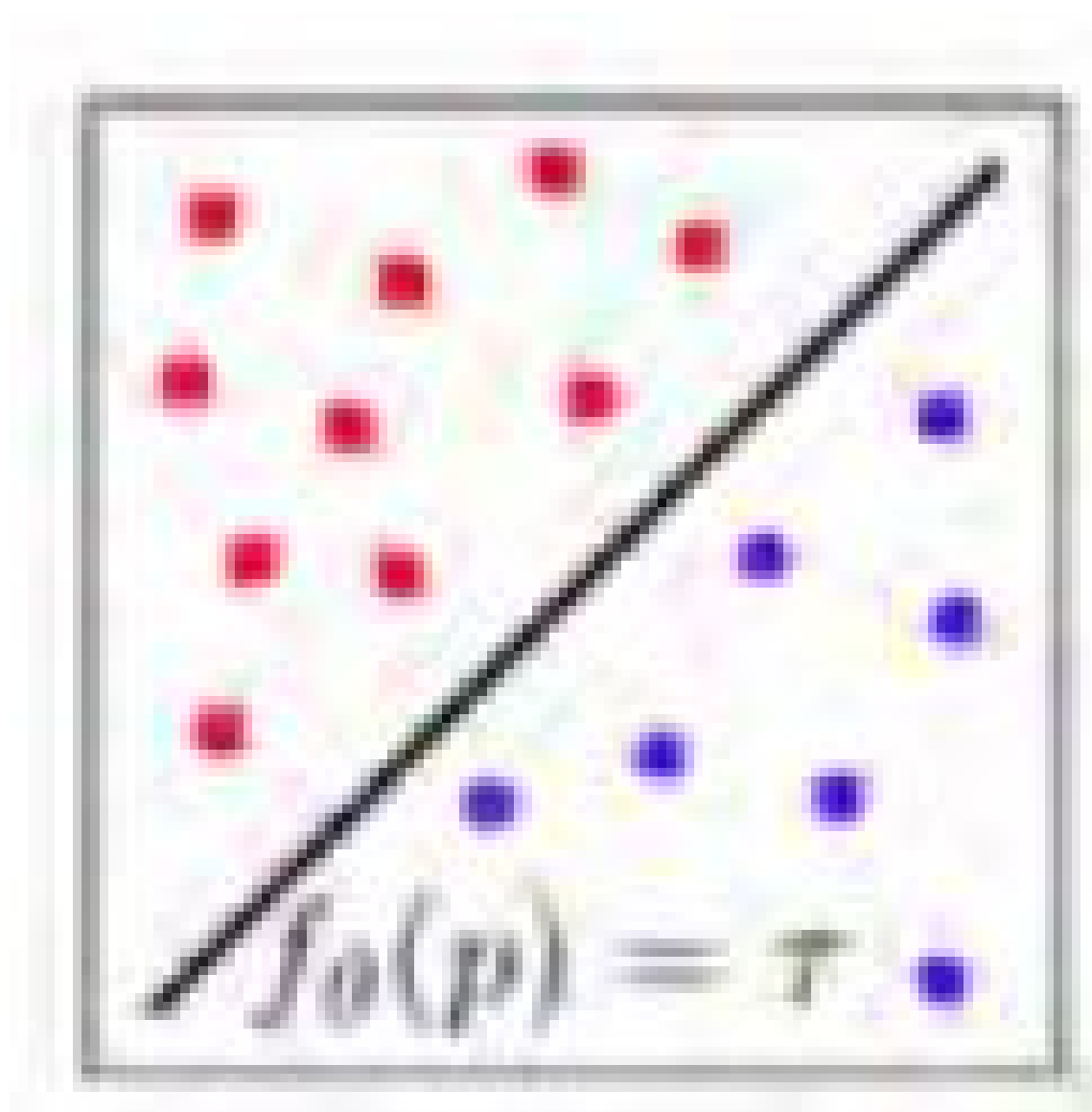




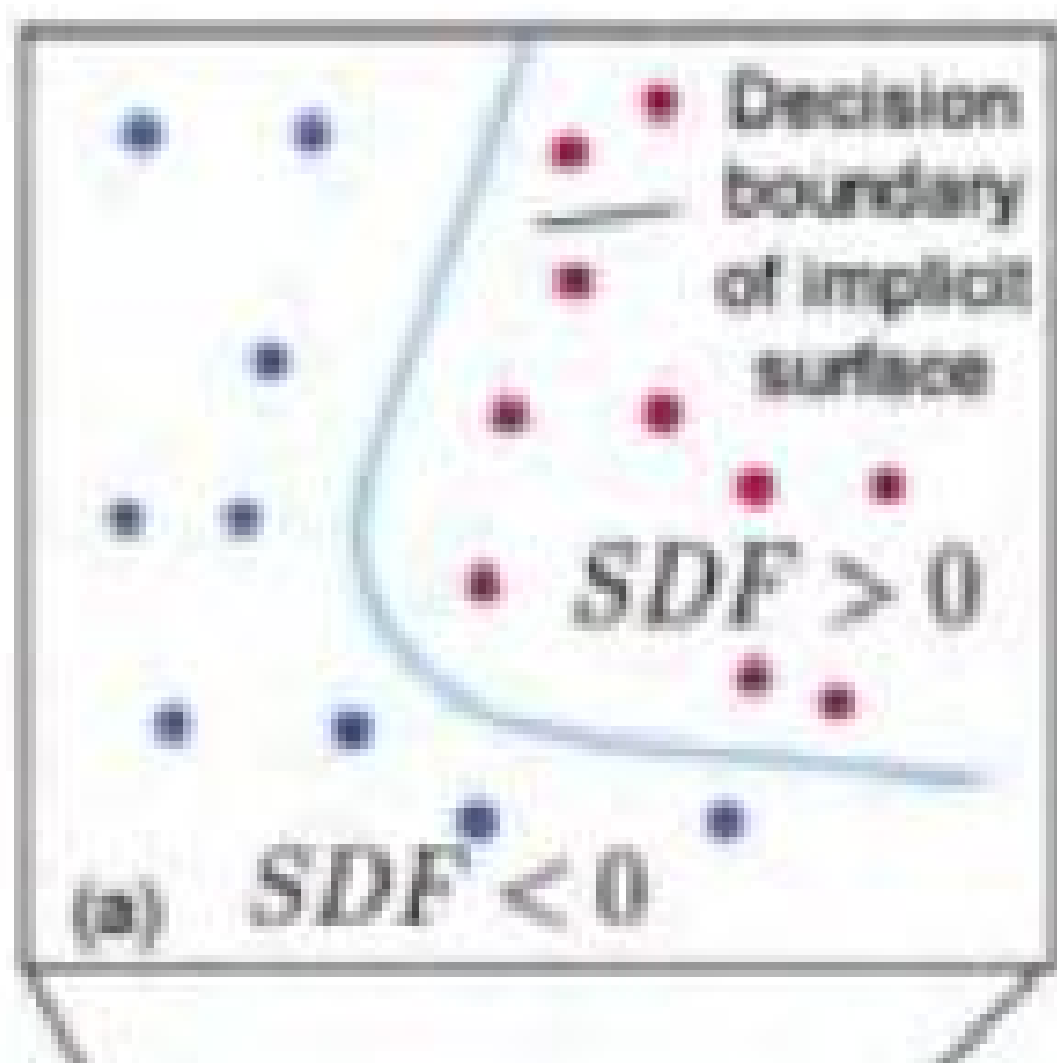


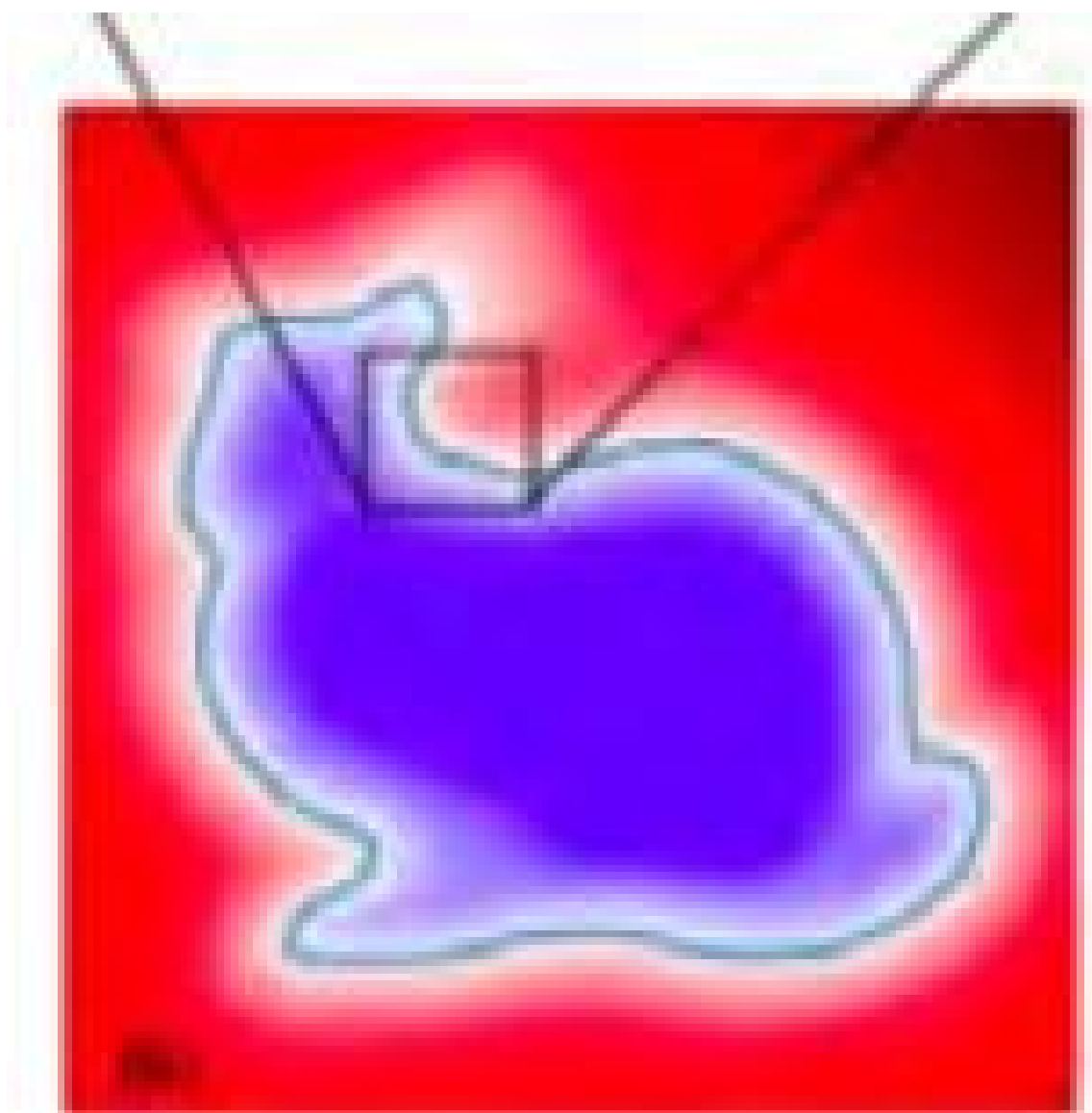














14.1.2 可微渲染

三维数据的监督学习面临大规模真实标注难以获取且成本高昂的挑战。为克服这些问题，研究者将图形渲染流水线纳入神经网络训练，从而推动了可微渲染（differentiable rendering, DR）的发展。DR 允许梯度从二维图像反向传播到三维场景参数（图 14.2）。在图形流水线中，三维模型可以采用两种表示：

- **表面数据**：显式编码对象边界，例如多边形或隐式表面。- **体积数据**：描述密度或颜色等连续场，表示对象的内部属性和材料分布。

这两类表示对应的渲染过程分别称为表面渲染和体渲染。因此，各自采用不同的渲染方法（图 14.3）：

- **表面渲染**通常采用光栅化，将表面几何投影到二维平面。它可利用硬件加速，因此对显式表面非常高效。- **体渲染**通常通过光线步进实现：在体积中投射光线，并沿每条光线路径采样数值。它适于连续场，但计算代价较高。

尽管早期 DR 方法对网格使用软化光栅化^[695, 541, 680]，体渲染已逐渐成为更强大的基于梯度优化方法。体渲染在整个三维空间提供梯度，甚至在确定精确表面位置之前也是如此，从而使神经网络训练更容易，因此被广泛作为 DR 的骨干。继 NeuralVolume^[688] 之后，可微体渲染^[807] 和 Scene Representation Networks^[1015] 等工作使用可微光线步进表示隐式表面。尽管最初仅限于简单形状，这些方法为更复杂、更真实的三维重建奠定了基础。

14.1.3 使用可微渲染的三维场景表示

2020 年，Mildenhall 等人^[764] 提出 NeRF，成为一个重要突破。该方法从带位姿的彩色图像出发，使用体积光线步进优化 MLP 以重建场景。NeRF 使用单个 MLP 作为场景表示，沿像素光线步进并查询 MLP 的不透明度和颜色来执行体渲染。由于体渲染天然可微，MLP 能够利用多视图信息最小化渲染损失，从而实现高质量新视图合成。NeRF 的 MLP 输入包括空间位置和视角方向，并配合位置编码捕获场景高频细节。虽然 NeRF 获得了高质量新视图合成，但训练速度是其主要瓶颈。为此，后续工作引入体素栅格^[1050, 341] 或哈希函数^[784] 等显式体积结构来提高性能。这些进展表明，高质量新视图合成的关键在于可微体渲染，而非神经网络本身；不依赖 MLP 也可以达到与 NeRF 相当的渲染质量。不过，即使在改进系统中，逐像素光线步进仍是渲染速度的重要瓶颈。

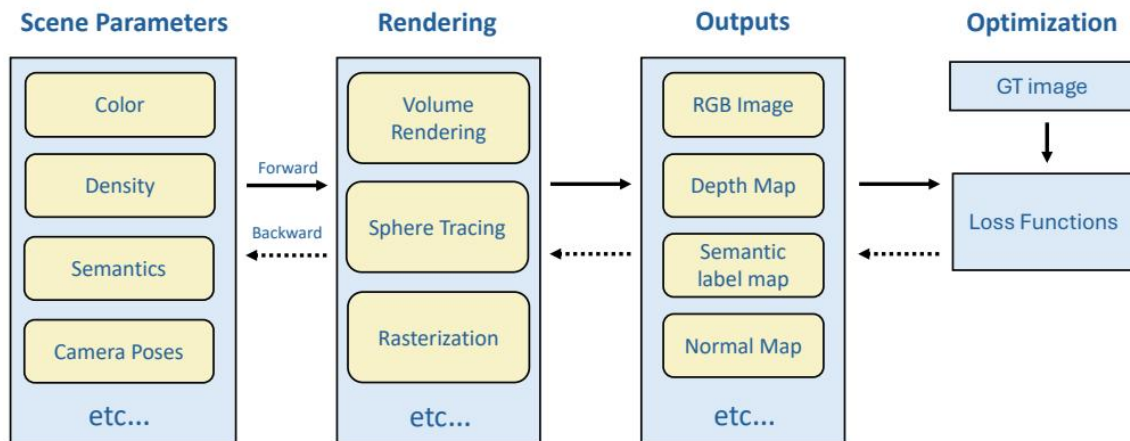
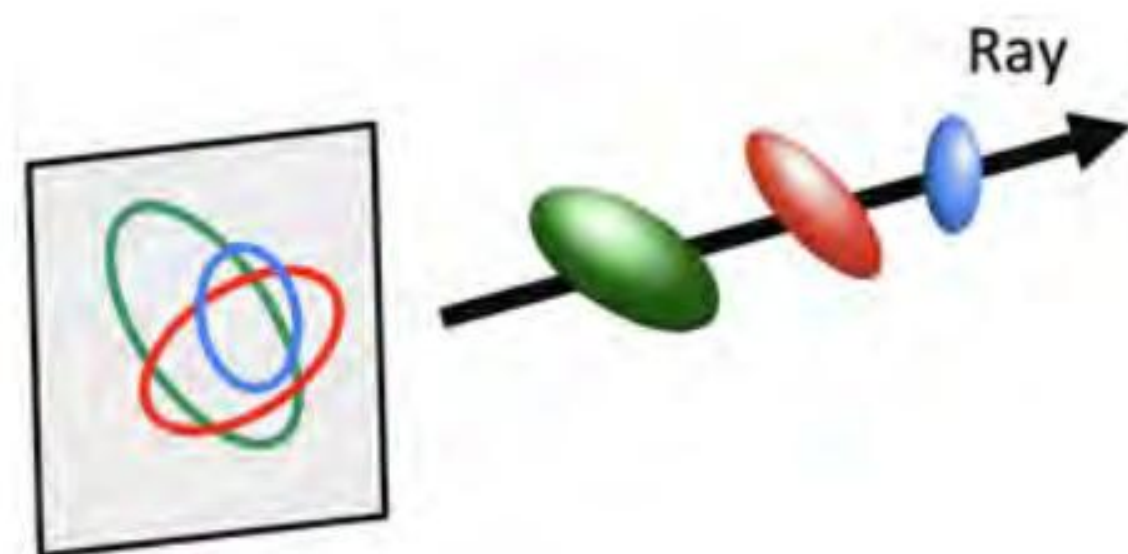
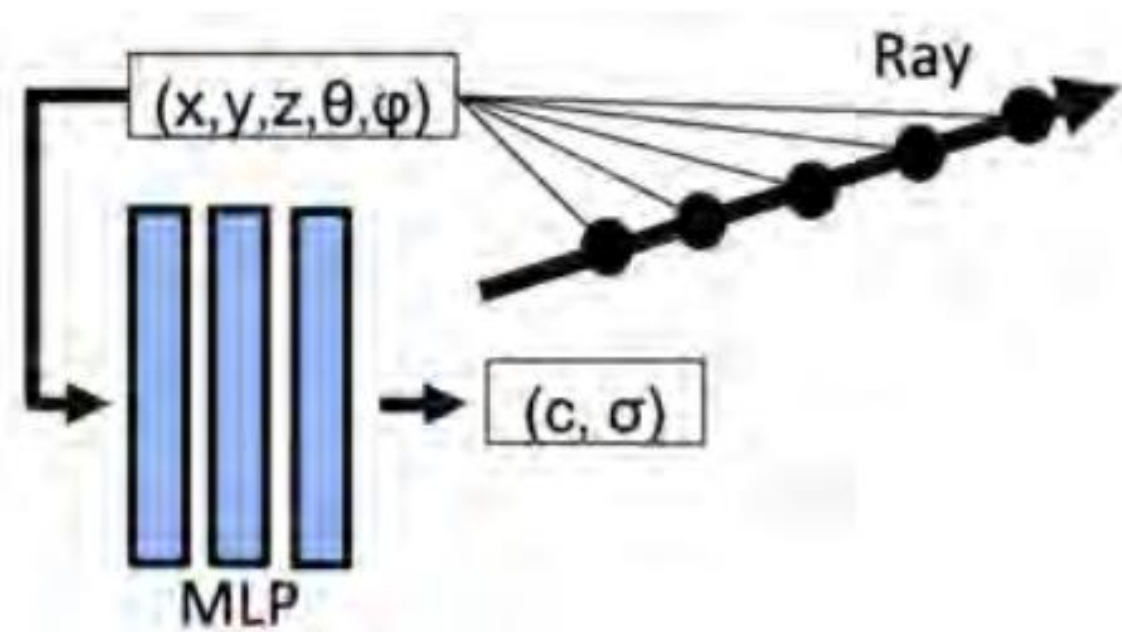


图 14.1: 图 14.2 可微渲染流水线概览。由于梯度可一致传播，DR 能够以端到端方式从多视图二维图像优化三维场景参数。

(a) 光线步进 (NeRF)

(b) 光栅化/Splatting (3DGS) 图 14.3 NeRF 和 3D Gaussian Splatting 渲染方法的比较。基于光线步进的方法混合沿射线固定间隔采样点的推理结果；光栅化则将射线上的每个图元投影到图像平面后再混合。

在 NeRF 和体渲染快速发展的同时，研究者也探索了可微的基于点渲染技术^[597, 954, 556]。这些方法采用点云^[597, 954] 或 metaball^[556] 等基于点的表示，并通过基于 CNN 的光栅化器或近似渲染器定义可微渲



渲染流水线。2023 年, Zwicker 等人^[1319]提出了一种基于 3D Gaussian Splatting^[552] (3DGS) 的截然不同的点渲染方法, 用于辐射场捕获。3DGS 使用大量半透明高斯团表示场景, 并通过 alpha 混合渲染。不同于 NeRF, 3DGS 使用可微光栅化 (图 14.3)。它和传统图形光栅化一样, 遍历待光栅化的图元而非沿射线步进, 因而利用三维场景的天然稀疏性, 既能表达高保真三维场景, 又能在保持与最高效体积 NeRF 相近训练速度的同时显著加快渲染。

下文将探讨 NeRF 和 3DGS 的技术细节 (图 14.4)。尽管二者共同目标是辐射场捕获, 但场景表示和渲染流水线存在显著差异。我们先概述 NeRF, 重点介绍其基于光线步进的可微渲染表述及其使用不同数据结构、发展为通用 neural field 表示的扩展; 随后考察 3DGS 及其基于光栅化的渲染流水线, 并强调其与 NeRF 的关键区别。



14.2 Neural Radiance Fields (NeRF)

本节说明 neural radiance field (NeRF) 的技术细节、衍生方法及其在视觉 SLAM 中的应用。这些技术背景为理解下一节中与 3D Gaussian Splatting 的动机和差异提供基础。

14.2.1 方法概述

NeRF 模型表示为函数 $f_{\Theta}(\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$, 其中 Θ 是 MLP 权重, 场景中的三维坐标为 $\mathbf{x} = (x, y, z)$, 视角方向为 $\mathbf{d} = (\theta, \phi)$, 颜色为 $\mathbf{c} = (r, g, b)$, 体积密度为 σ 。为合成新视图, NeRF 向场景投射相机光线, 为每个像素生成采样点。使用 NeRF MLP 在每个采样点计算局部颜色和密度, 再通过体渲染合成二维图像 (图 14.5)。相机光线 $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ 的期望颜色 $C(\mathbf{r})$ 按下列积分计算:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt. \quad (14.1)$$

其中 dt 表示每个积分步中光线行进的微分距离, t_n 和 t_f 分别表示近、远边界。 $\sigma(\mathbf{r}(t))$ 和 $\mathbf{c}(\mathbf{r}(t), \mathbf{d})$ 分别表示具有视角方向 $\mathbf{d}$ 的相机光线上点 $\mathbf{r}(t)$ 的体积密度和颜色。从 t_n 到 t 的累积透射率为 $T(t) =$



图 14.2: 图 14.4 MipNeRF360^[58] (左, ©2019 IEEE) 和 3DGS^[552] (右) 的新视图合成示例。两种方法都可从真实世界中带位姿的 RGB 图像捕获高保真辐射场。图像来自^[58, 552]。

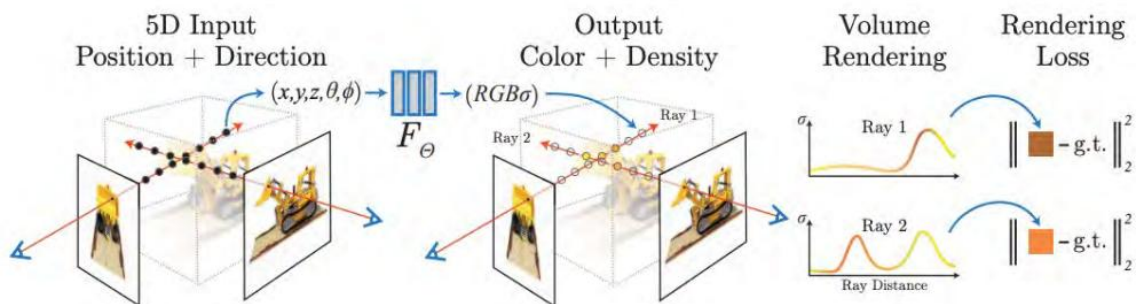


图 14.3: 图 14.5 NeRF 优化流水线概览。辐射场由 MLP 表示，通过可微体渲染生成二维图像。优化 MLP 参数以最小化渲染彩色图像与真实彩色图像之间的光度误差。图像来自^[764] (©2020 Springer)。

$\exp(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds)$ 。连续积分通过数值求积进行估计：将光线划分为 N 个等间隔区间，在区间之间均匀采样并在每个区间内随机采样。利用求积规则，体渲染方程写作：

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N \alpha_i T_i \mathbf{c}_i, \quad \text{where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right). \quad (14.2)$$

$\delta_i = t_{i+1} - t_i$ 表示连续采样点 t_i 与 t_{i+1} 之间的间隔， σ_i 和 $\mathbf{c}_i$ 分别表示在光线上第 i 个采样点求得的密度和颜色；此外， $\alpha_i = 1 - \exp(-\sigma_i \delta_i)$ 表示第 i 个采样点通过 alpha 合成得到的不透明度。

为优化模型，使用渲染图像与真实彩色图像之间的平方光度误差：

$$L = \sum_{\mathbf{r} \in R} \|\hat{C}(\mathbf{r}) - C_{gt}(\mathbf{r})\|_2^2, \quad (14.3)$$

其中 $C_{gt}(\mathbf{r})$ 表示训练图像中与 $\mathbf{r}$ 关联像素的真实颜色， R 表示用于合成目标图像的一批光线。在 SLAM 中，常利用不透明度的 alpha 混合渲染深度，从而额外最小化期望深度与传感器深度或预测深度之间的差异。

14.2.2 NeRF 中的数据结构

原始 NeRF 论文^[764]使用单个 MLP 表示整个辐射场；在特定三维位置评估颜色和密度时，需要一次前向推理。这意味着所有网络参数都参与评估，导致前向推理和反向训练都非常慢。为加快推理和训练，多个方法部分或完全引入显式数据结构，实现快速数据访问。

体素/哈希栅格。早期工作引入体素栅格以加速前向推理。随后，Plenoxel^[341]和 DirectVoxGo^[1050]将体素特征栅格作为可训练属性，并将辐射场编码其中。与基于 MLP 的 NeRF 相比，特定位置的颜色和密度只需简单索引查询和三线性插值，显著加速整个流水线（从数天降至 10 分钟）。Instant-NGP^[784]（图 14.6）进一步引入多分辨率哈希编码，在哈希表中高效编码体积特征；结合高度优化的 CUDA 实现，可在数秒内完成 NeRF 训练。

点和 surfel。基于点的表示也可与 NeRF 结合使用（图 14.7）。Point-NeRF^[1209]优化带特征的点云和微型 MLP 解码器，并通过体渲染框架渲染图像。它表明，相比规则体素栅格，点表示具有空间分配灵活性，能够跳过空白区域，从而比原始 NeRF 快三倍。类似点云，也可以用 surfel 表示辐射场^[366]。

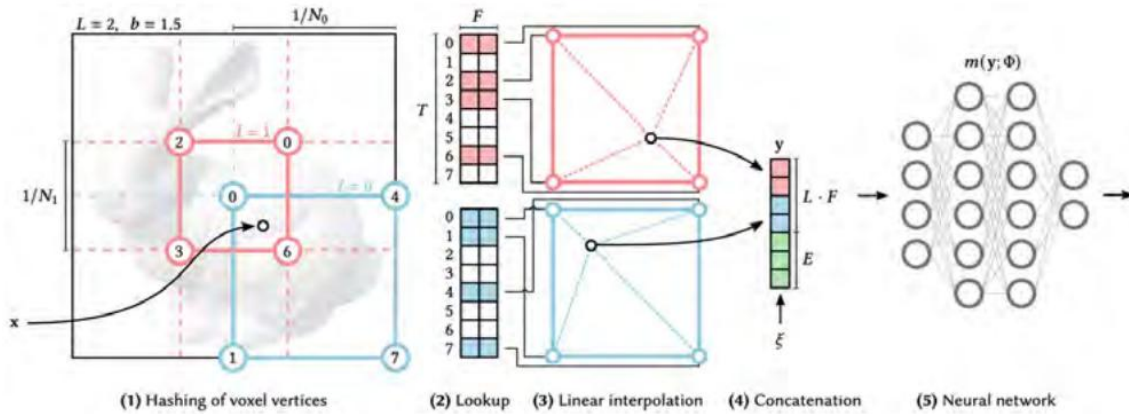


图 14.4: 图 14.6 基于栅格的 NeRF 示例 (Instant-NGP^[784])。辐射场由多分辨率哈希栅格和微型 MLP 解码器的组合建模。图像来自^[784]。

14.2.3 Neural Fields

早期 NeRF 工作主要关注将三维位置映射到颜色和密度的辐射场，以实现照片级真实感场景捕获；但根据下游任务不同，这一基本思想也可扩展到各种其他输出。这类通用场表示称为 Neural Fields。下面介绍几个主要例子。

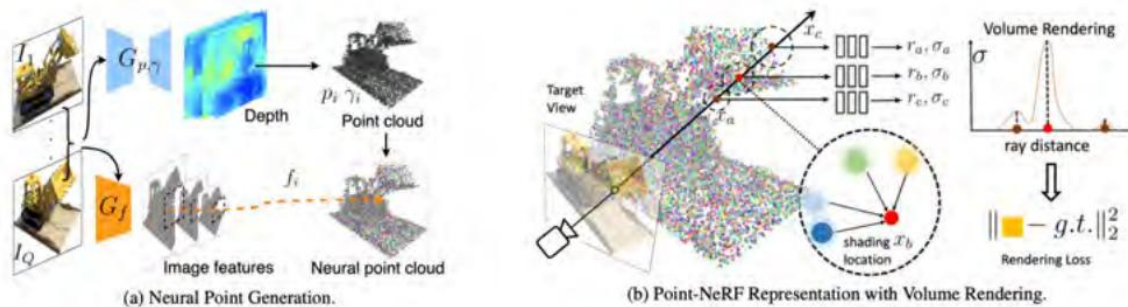


图 14.5: 图 14.7 基于点的 NeRF 表示示例 (Point-NeRF^[1209])。辐射场由带可训练特征的点与微型 MLP 解码器的组合建模。图像来自^[1209] (©2022 IEEE)。

表面表示 (SDF 与占据场)。为从 RGB 图像获得几何上更准确的三维重建, Neural Fields 可以使用有符号距离函数 (SDF) 和占据场等表示 (图 14.8)。NeuS^[1159] 和 VolSDF^[1241] 等技术利用 SDF 表示表面。SDF 编码点到最近表面的距离, 符号表示点位于对象内部还是外部。将 SDF 集成到 NeRF 中可以实现精确的表面建模。UNISURF^[819] 等技术使用占据场表示三维空间中表面的存在与否: 占据场将空间划分为栅格, 每个单元表示是否被表面占据。该方法可以与 NeRF 结合, 有效处理复杂场景。为从这些表示中提取网格, 可以使用 marching cubes 等技术, 将连续表面表示转换为离散网格格式。

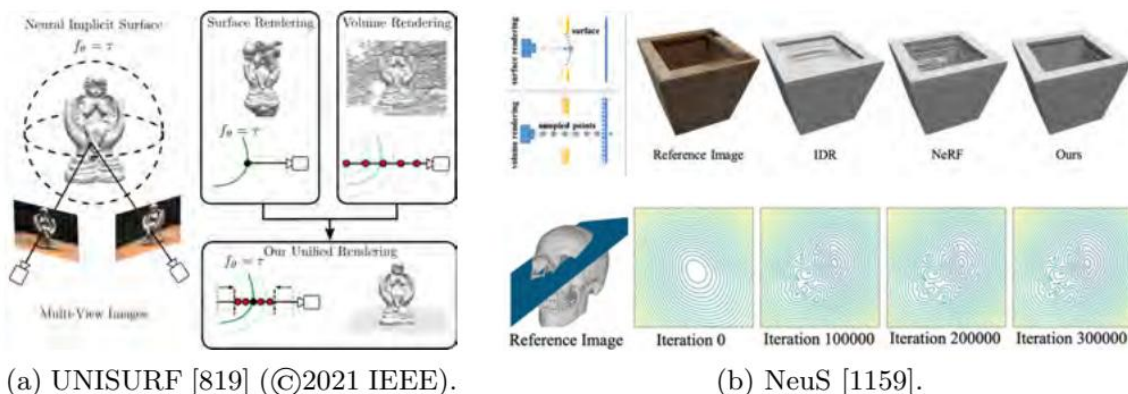


图 14.6: 图 14.8 用于表面重建的 Neural Field。UNISURF 使用占据场, NeuS 使用由 MLP 建模的连续有符号距离函数; 二者都可通过体渲染生成图像。图像来自^[819, 1159]。

语义。将语义信息纳入 Neural Fields, 可以把场景分解为有意义的组件, 增强场景理解和交互能力 (图 14.9)。早期工作^[1291, 1292] 扩展 NeRF, 使其除颜色和密度外还输出语义标签: 利用多视图语义图像训练模型, 使其能够预测场景每一部分的语义类别。这些场将多视图的高维特征融合为三维一致的语义特征场。更一般地, NeRF 可以融合预训练基础网络的二维特征, 通过语言提示或点击执行场景分解和交互式编辑等更高级的语义任务^[586, 1111, 745]。

14.2.4 用于视觉 SLAM 的 NeRF/Neural Fields

Neural Fields 最初设计用于利用带位姿的彩色图像进行离线新视图合成。然而, 它们生成高保真重建的能力引发了将其用于视觉 SLAM 的兴趣, 后者需要实时联合估计相机位姿和场景表示。

与经典稠密 SLAM 方法相比, 基于 Neural Field 的三维表示具有多项优势。第一, 通过可微渲染进行端到端优化, 简化了二维到三维的数据融合, 无需像基于 TSDF 的 SLAM 那样进行多阶段手工设计的融合步骤, 而这些步骤的一致性并不总能保证。第二, Neural Fields 可以建模复杂光场, 更灵活地处理透明物体和反射, 从而最小化光度误差。第三, 根据数据结构不同, 它们能够借助神经表示中固有的先验完成场景补

全和压缩。

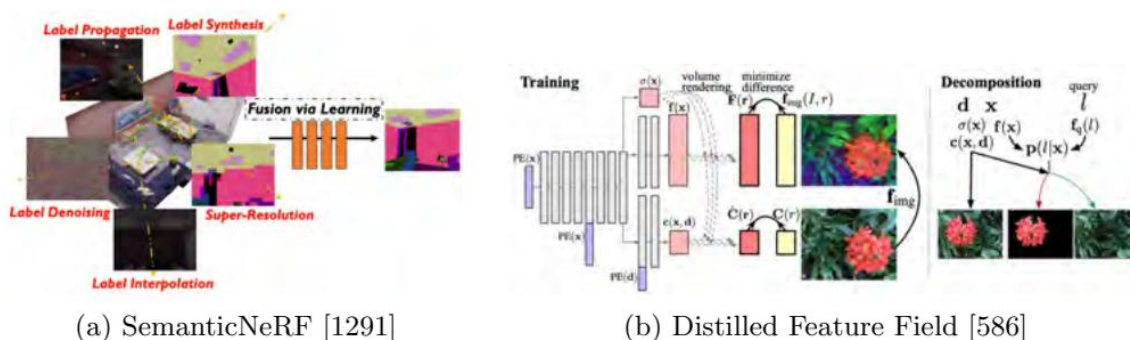


图 14.7: 图 14.9 用于语义场景重建的 Neural Field。通过为 MLP 增加语义头, NeRF 可以输出多视图一致的语义标签/特征, 并通过逐视图语义预测进行训练。图像来自 [1291, 586]。

iMAP [1047] (图 14.10) 是这一方向的首项工作, 使用类似原始 NeRF 的纯 MLP 地图表示。iMAP 的关键洞见是, 得益于 MLP 固有的平滑先验, 基于 MLP 的方法可以实现地图压缩和场景补全。与 NeRF 研究的发展类似, 研究者也在 SLAM 中探索经典表示和神经表示的结合。NICE-SLAM [1309] 首次引入混合表示, 将多分辨率体素特征栅格与微型 MLP 结合。其他工作也采用了类似的栅格表示 [1232, 525, 1153]。NeRF-SLAM [940] 利用 DROID-SLAM 的深度信息 (参见第 13 章) 加速 NeRF 重建。最近, Point-NeRF 风格的点基元也被应用于 SLAM [968]。在诸多技术组件中, 底层数据结构的选择对 SLAM 的重建质量、计算效率和实时性能影响最大。因此, 下面考察 Neural-Field SLAM 方法中使用的各种数据结构, 并详细说明其主要特征与权衡。

14.2.4.1 Neural Field SLAM 中数据结构的特征与权衡

底层数据结构的选择会显著影响基于 Neural Field 的 SLAM 属性。表 14.1 概览了各种 Neural-Field SLAM 方法。下面详细介绍这些表示相关的主要特征和权衡。

压缩与推理速度之间的权衡。早期 Neural Field SLAM 使用单个 MLP, 较新的方法则部分或完全使用类似经典图形基元的显式数据结构。MLP 用网络参数近似整个辐射场函数, 在压缩方面具有巨大优势, 但参数的物理含义难以解释。另一方面, 体素栅格等显式数据结构更易解释, 能够快速访问数据并局部更新地图, 但需要更多内存。因此, 地图表示体现了典型的时间-空间复杂度权衡。对于 SLAM 任务, 实时运行尤其重要, 实践中通常更偏好显式数据结构。



图 14.8: 图 14.10 iMAP [1047] 的定性结果。该方法首次展示了可以使用 MLP 和可微渲染框架, 从 RGB-D 视频流实时联合估计相机位姿和场景几何。

底层数据结构	MLP	体素/哈希	点/surfel
压缩	好	中等	中等
推理速度	受限	好	中等
场景补全	好	好	中等
动态分配	受限	受限	好
遗忘问题	受限	中等	中等

表 14.1 Neural Field 的底层数据结构比较。

场景补全。 iMAP 表明，由于函数的连续性，基于 MLP 的三维重建能够填补未观测区域的孔洞。多分辨率栅格也具有类似属性（图 14.11），因为位置编码或多分辨率栅格会在频率或栅格分辨率内平滑信号。基于点的表示则没有这种效果，因为其基元是离散的，不具备全局平滑能力。

图形基元的动态分配。 与离线场景重建任务不同，SLAM 需要根据运行时观测在线更新状态变量，以适应场景边界变化和回环等动态变化。在 Neural Fields 中，MLP 和体素类型表示由于边界和位置编码/体素分辨率固定，难以进行这些更新，通常需要动态分配多个局部 NeRF 模型^[1070]（图 14.12）。相比之下，点表示在基元分配方面更灵活，能够更有效地处理回环^[673]。

(a) iMAP^[1047]

(b) NICE-SLAM^[1309] 图 14.11 基于 NeRF 的 SLAM 的孔洞填充效果。得益于多分辨率频率或栅格编码以及连续 MLP 函数，该表示能够平滑局部几何，并为小范围未观测区域提供合理的孔洞填充效果。图像来自^[1047]和^[1309]（©2022 IEEE）。

(a) Loopy-SLAM^[673]

(b) MIPS-Fusion^[1070] 图 14.12 基于 NeRF 的 SLAM 回环。基于体素的方法需要多个局部 Neural Field 子地图来处理回环导致的相机位姿更新，而离散点表示可以更无缝地改变基元分配。图像来自^[673]（©2024 IEEE）和^[1070]（©2023 ACM）。

遗忘问题。 在纯 MLP 表示和混合表示中，全局函数的存在意味着任何局部参数更新都会影响整体结果。因此，地图必须在优化窗口中保留过去信息作为训练信号，并需要显式关键帧数据库。这个通常称为“遗忘问题”的现象，可以通过减少对全局函数（通常由 MLP 表示）的依赖，或完全移除全局函数来缓解。

14.3 3D Gaussian Splatting

NeRF 执行可微光线步进，而 3D Gaussian Splatting (3DGS) 对体数据执行可微光栅化。类似常规图形光栅化，3DGS 遍历待光栅化的图元，而不是沿光线步进，利用三维场景的天然稀疏性，获得足以捕获高保真三维场景的表达能力，同时显著提高渲染速度（图 14.13）。输入是一组带有 SfM 估计位姿的彩色图像；此外，3DGS 使用 SfM 过程中生成的稀疏点云构造一组初始三维高斯图形基元。

14.3.1 方法概述

每个三维高斯由三维均值位置 $\boldsymbol{\mu} \in \mathbb{R}^3$ 、协方差矩阵 $\boldsymbol{\Sigma} \in \mathbb{R}^{3 \times 3}$ 、不透明度 o 和由球谐 (SH) 表示的方向相关颜色 c 参数化。给定缩放矩阵 $\boldsymbol{S} \in \mathbb{R}^{3 \times 3}$ 和旋转矩阵 $\boldsymbol{R} \in \text{SO}(3)$ ，协方差矩阵 $\boldsymbol{\Sigma}$ 表示为：

$$\boldsymbol{\Sigma} = \boldsymbol{R} \boldsymbol{S} \boldsymbol{S}^\top \boldsymbol{R}^\top. \quad (14.4)$$

为实现可微光栅化，使用近似投影函数将这些三维高斯投影到图像平面 ($z = 1$ 平面)。投影高斯在二维图像空间中的协方差矩阵 $\boldsymbol{\Sigma}'$ 为：

$$\boldsymbol{\Sigma}' = \boldsymbol{J} \boldsymbol{W} \boldsymbol{\Sigma} \boldsymbol{W}^\top \boldsymbol{J}^\top, \quad (14.5)$$

其中 $\boldsymbol{J}$ 表示投影变换的雅可比矩阵， $\boldsymbol{W}$ 是视图变换矩阵。将 $\boldsymbol{\Sigma}'$ 左上角的 2×2 块作为二维协方差矩阵。通过查询像素 $\mathbf{x}_{2D} = [u, v]^\top$ ，可以得到二维高斯函数的值。

像素 x_{2D} 的最终颜色 C 由三维高斯的 alpha 混合确定：



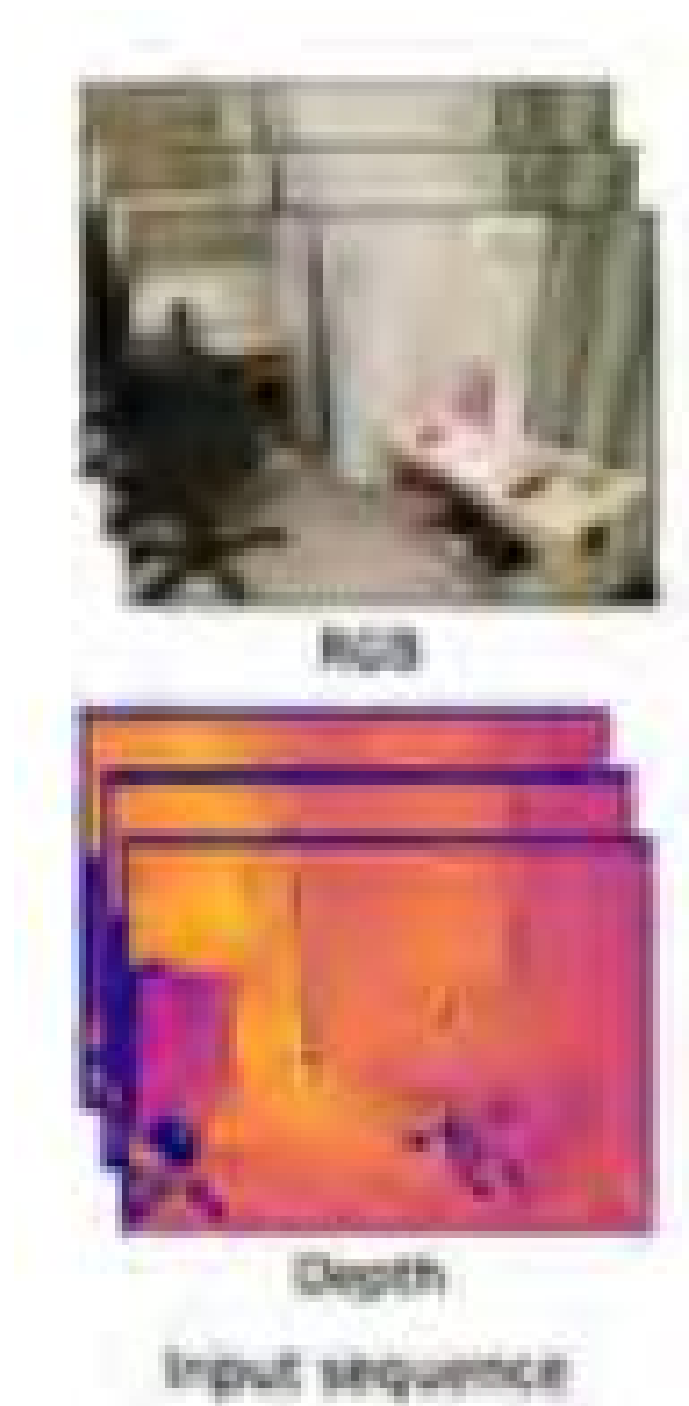


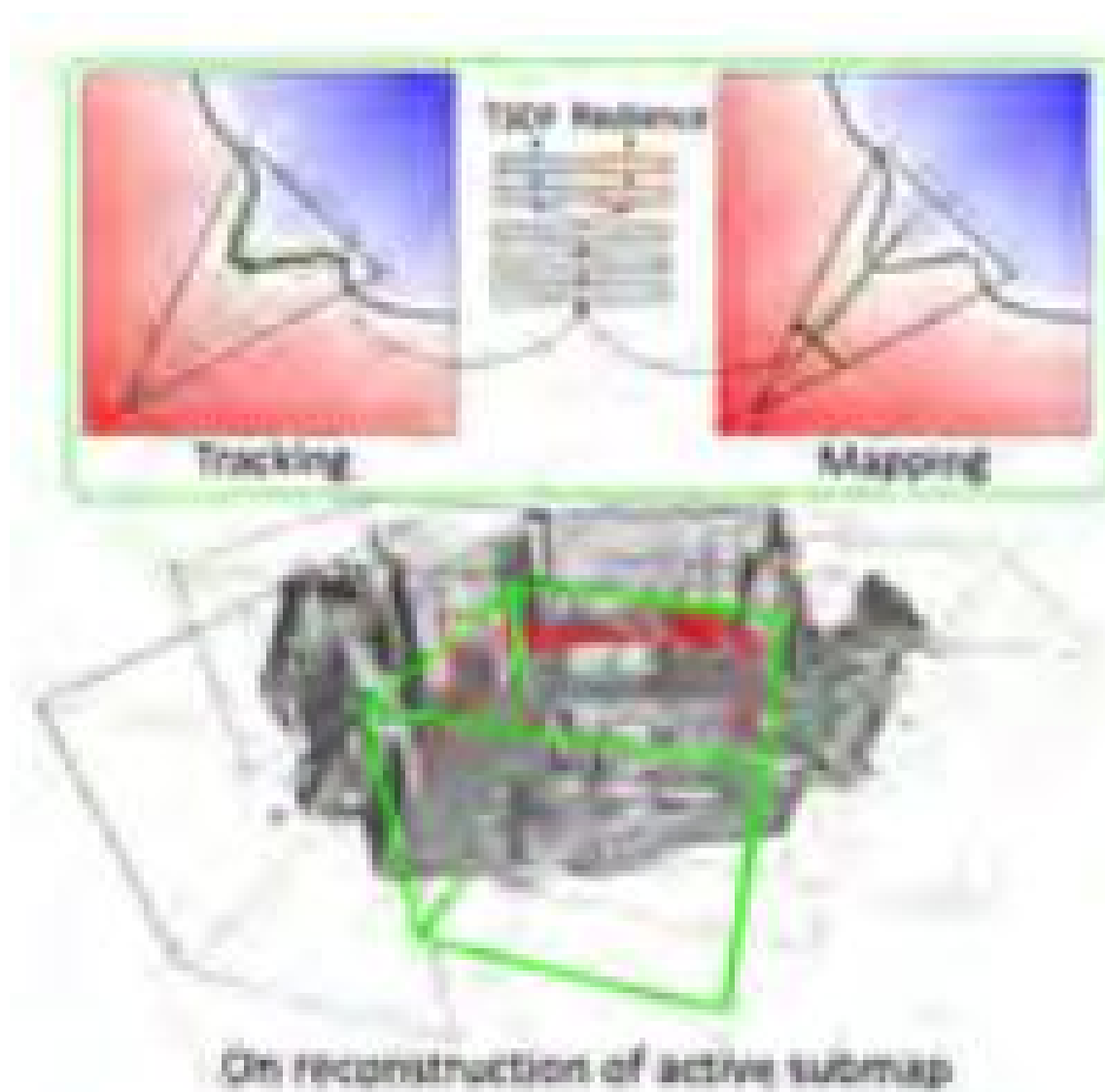












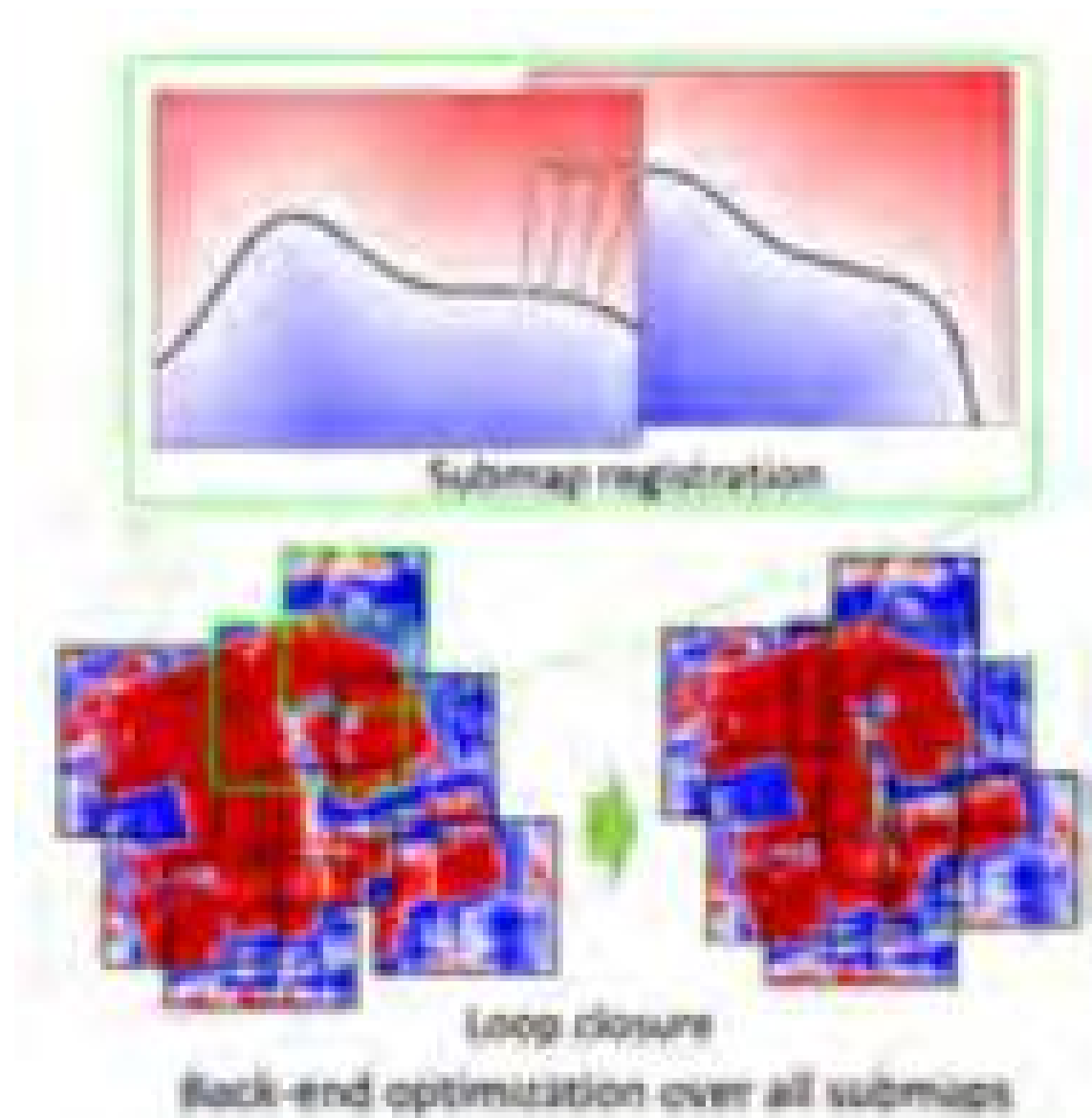




图 14.9: 图 14.13 3DGS 新视图渲染结果。(左) RGB 渲染；(右) 高斯体着色。3DGS 通过分配大量半透明三维高斯体表示场景。

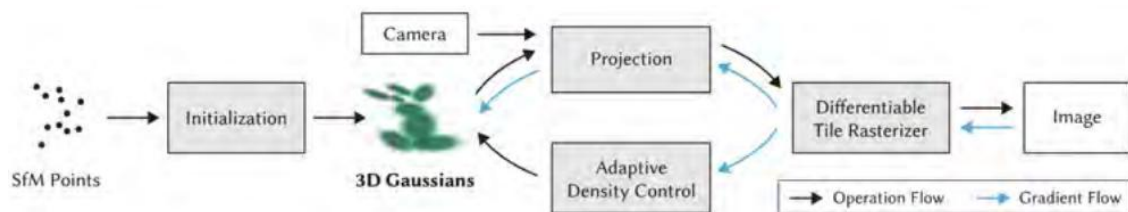


图 14.10: 图 14.14 3DGS 优化过程概览 [552]。

$$C = \sum_{i \in \mathcal{N}} \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (14.6)$$

其中 $\mathcal{N}$ 是沿像素光线的高斯集合, α_i 是不透明度 o_i 与像素空间高斯函数的乘积:

$$\alpha_i = o_i G_{2D}(\mathbf{x}_{2D}). \quad (14.7)$$

在 alpha 混合中, 按照相对于渲染视点的深度顺序对三维高斯排序。输入图像被划分为 16×16 的图块, 并使用 GPU 优化的基数排序对每个图块分别执行该排序。此外, 为提高效率, 只考虑位于当前视锥内的高斯 (视锥剔除)。通过最小化渲染图像与真实图像之间 L1 损失和 SSIM 损失的加权和, 估计三维高斯参数, 权重为 λ :

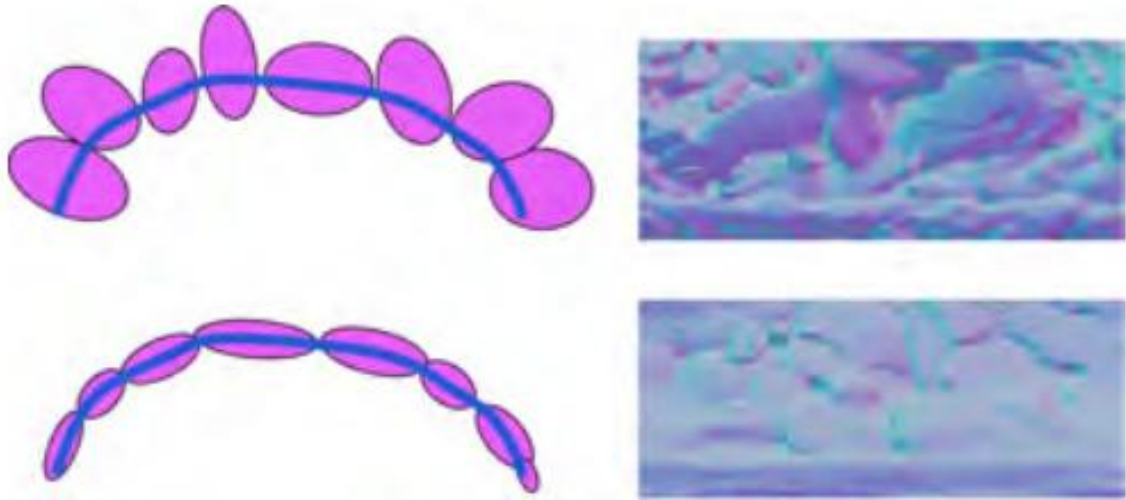
$$\mathcal{L} = (1 - \lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{SSIM}}. \quad (14.8)$$

此外, 在优化过程中会根据高斯的梯度和不透明度执行自适应稠密化与剪枝, 以灵活调整高斯数量 (图 14.14)。

14.3.2 3D Gaussian Splatting 的应用

类似 NeRF 演变为通用 Neural Field, 3DGS 也被应用于各种三维视觉任务。鉴于该领域研究数量庞大, 这里重点介绍几篇早期工作。

表面重建。3DGS 主要设计用于照片级真实感新视图合成, 并不天然保证几何精度。若干工作针对这一局限进行了改进 (图 14.15)。SuGAR^[411] 为 3DGS 引入几何正则化, 实现几何准确的三维网格提取。该方法为每个三维高斯定义表面法向, 并通过鼓励相邻高斯尽可能平坦来强制局部表面平滑。2D Gaussian Splatting (2DGS)^[483] 将三维高斯替换为二维圆盘, 在图形基元中显式定义表面法向。通过加入法向和深度一致性损失, 2DGS 进一步提高了表面重建精度。



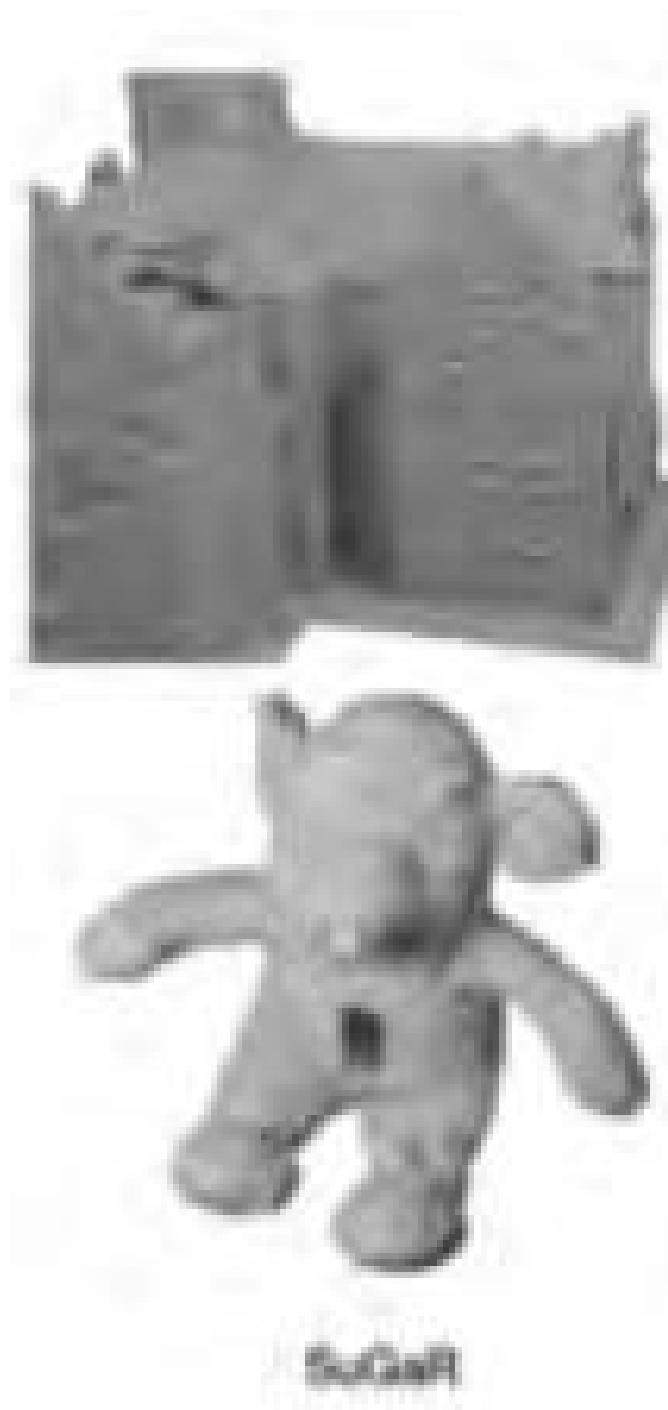
(a) SuGAR^[411] (©2024 IEEE)

(b) 2DGS^[483]。

语义场景理解。3DGS 也可以扩展到语义场景理解。与 NeRF 类似, 最常见的做法是通过可微渲染支持的端到端训练, 将二维语义分割传播到带语义特征的三维高斯中。例如, LangSplat^[895] 和 FMGS^[1316] 在 3DGS 训练过程中, 将 CLIP、DINO 等二维基础模型集成到多视图一致的三维高斯中, 实现开放词汇语义分割 (图 14.16)。由于三维高斯是显式离散表示, 编辑重建场景 (例如操纵或删除) 比连续 NeRF 表示更直接。







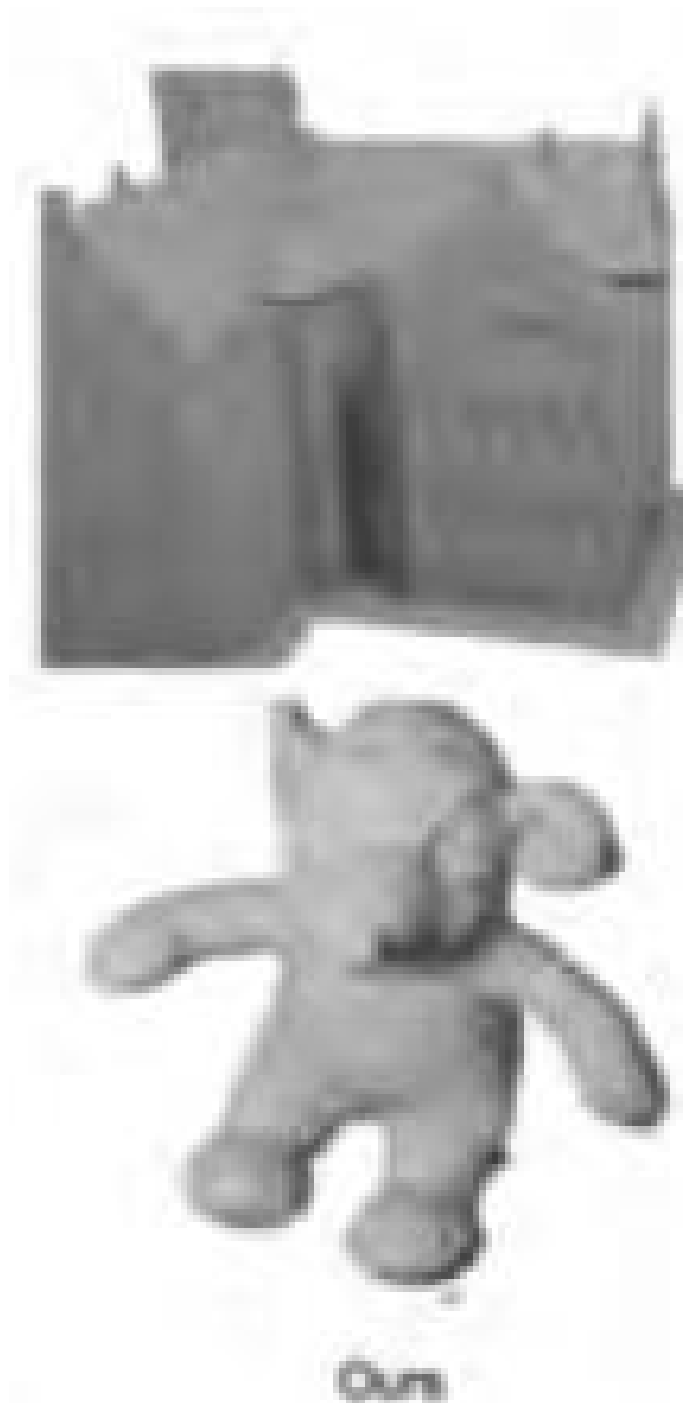


图 14.11: 图 14.15 面向表面的 Gaussian Splatting 方法示例。通过定义表面法向方向并强制相邻高斯之间保持平滑, 这些方法能够实现准确的表面重建。图像来自^[411] (©2024 IEEE) 和^[483]。

14.3.3 用于 SLAM 的 3D Gaussian Splatting

3DGS 在实时新视图合成方面取得了显著成功，其用于 SLAM 的潜力目前正在积极探索。与 NeRF 类似，3DGS 通过可微渲染的端到端优化，将定位和建图无缝统一。这种集成不再需要多阶段手工设计的算法，并能够建模复杂光场，包括透明物体产生的光场，而这些光场很难用经典三维表示捕获。

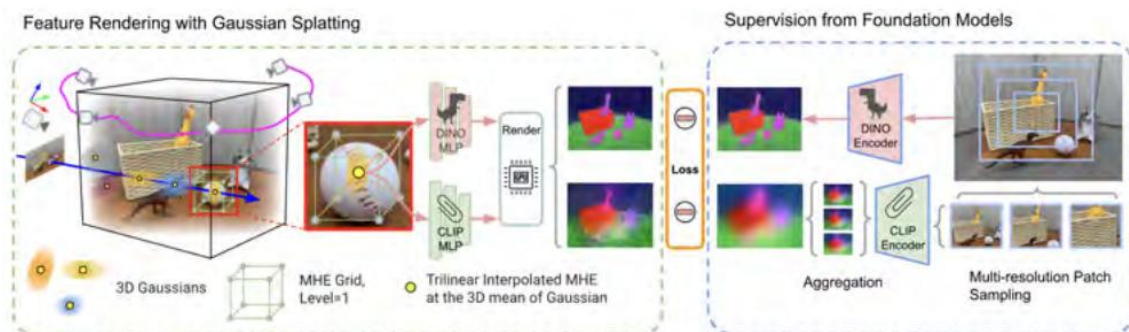


图 14.12: 图 14.16 Gaussian Splatting 用于语义场景重建的示例。多视图二维图像特征通过可微渲染融合到三维高斯中。图像来自^[1316] (©2024 Springer)。

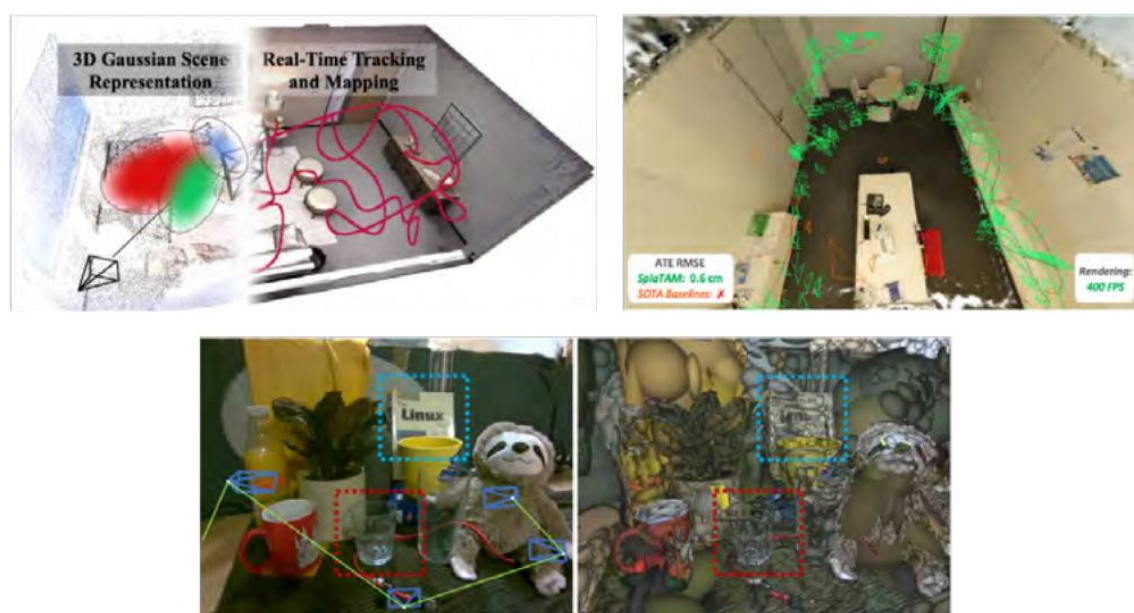
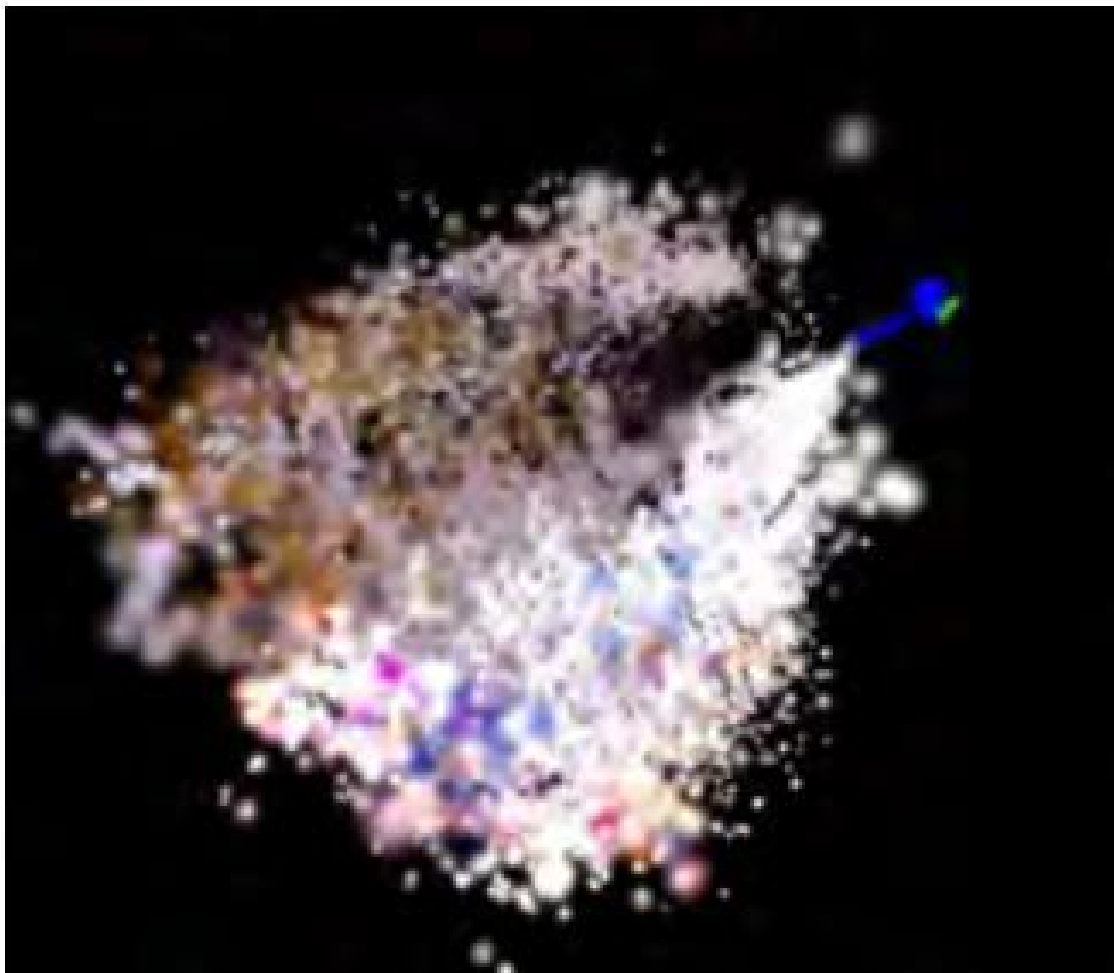


图 14.13: 图 14.17 基于 3D Gaussian Splatting 的视觉 SLAM 方法示例^[1214, 548, 740]。图像来自^[1214] (©2024 IEEE) 和^[548] (©2024 IEEE)。

除这些相似性外，3DGS 相比 NeRF 还具有若干独特优势。第一，其高效的基于光栅化流水线显著加快图像渲染，直接惠及需要实时地图交互的下游应用（例如机器人导航）。第二，其离散、点状的高斯基元表示允许灵活分配和更新场景元素。这种灵活性尤其适合 SLAM：状态估计需要根据运行时观测动态适应，例如更新场景边界和闭合回路。

利用这些属性，3DGS 可以作为视觉 SLAM 系统的核心场景表示（图 14.17）。若干早期工作探索了其同步定位与建图能力^[1214, 548, 740]。一种直接应用是 RGB-D SLAM，其中外部深度传感器（例如飞行时间（ToF）深度相机）提供准确、稠密的深度数据，可作为高斯位置的先验或真实值信号。加入该深度信息后，可以准确初始化三维高斯的位置，并通过最小化深度渲染损失进一步正则化，从而实现准确的相机跟踪以及高保真、快速的地图渲染。

然而，这些方法需要额外硬件才能重建三维高斯，而原始 3DGS 只需要普通相机采集 RGB 图像。理想情况下，方法应完全利用三维高斯灵活且可优化的特性，仅在基于视觉的单目 SLAM 设置中运行。在这一背景下，MonoGS^[740]等工作展示了仅使用彩色图像执行单目 SLAM 的可行性（图 14.18）。在 MonoGS 中，虽然三维高斯的初始位置未知，但可以随机初始化高斯，再通过多视图优化逐步估计和细化其位置。



三维高斯的离散特性对全局地图更新具有显著优势，尤其是在回环场景中。由于表示由一组独立且定义明确的基元组成，即使环境随时间变化，也更容易匹配和对齐场景的不同片段。例如，LoopSplat^[1308]展示了这一思想的实际应用：离散三维高斯元素充当鲁棒特征，可以在不同时间和视点之间可靠配准。3DGS 作为 SLAM 表示的另一项显著优势，是能够捕获复杂光场。通过有效捕获环境中复杂的光分布，3DGS 不仅提高了视觉保真度，还提供了可改善建图和定位性能的附加线索。I²-SLAM^[42]充分利用这一属性，将运动模糊、色调映射等更精确的基于物理的渲染过程集成到现有 3DGS 框架中，实现 HDR 地图重建与准确相机位姿跟踪的同步进行（图 14.19）。

尽管 3D Gaussian Splatting (3DGS) 作为统一 SLAM 表示的全部潜力仍在积极研究中，另一个有前景的方向是将 3DGS 集成到现有点云处理方法中。只需忽略 3DGS 的附加特征，就可以将其视为点云，因此能够直接结合稳定且在实际应用中性能良好的成熟点算法（图 14.20）。例如，Photo-SLAM^[488]将稀疏跟踪系统 (ORB-SLAM) 与 3DGS 结合；GS-ICP SLAM^[424]和 RTG-SLAM^[868]则利用 ICP 进行跟踪。在 RTG-SLAM 中，系统进一步加入基于 ORB 关键点后端优化的跟踪，取得了快速、准确且具有较高实用性的性能。

此外，3DGS 的固有通用性也延伸到与 LiDAR 测量的融合。近期若干方法提出了 LiDAR+3DGS 混合框架^[471, 1054, 624]，表明 3DGS 可以作为跨多种传感器模态的统一地图表示。从 RGB SLAM 到 LiDAR 增

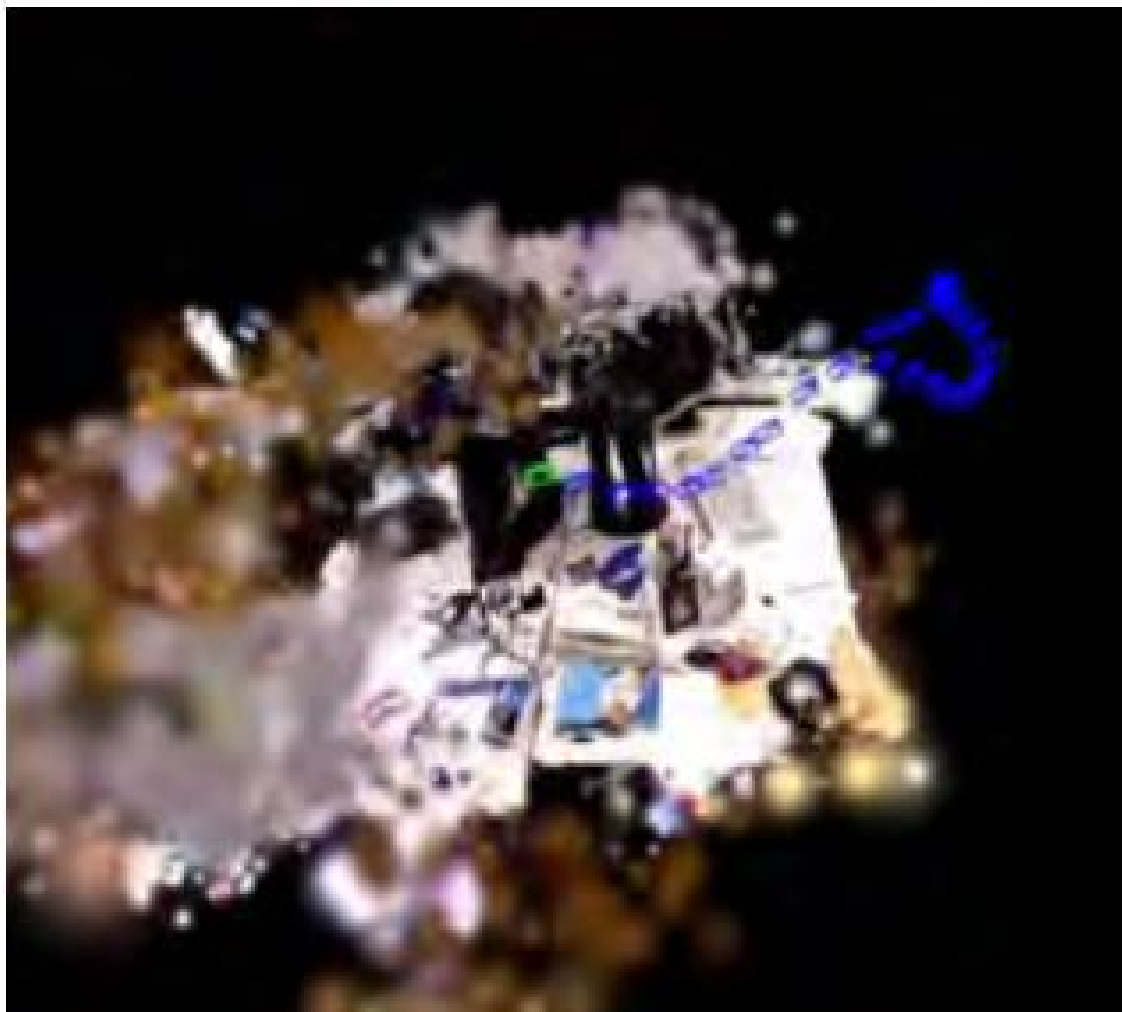




图 14.14: 图 14.18 MonoGS 的单目重建结果^[740]。得益于三维高斯空间分配的灵活性，该方法无需任何深度监督，仅从彩色图像联合估计相机位姿和场景重建。

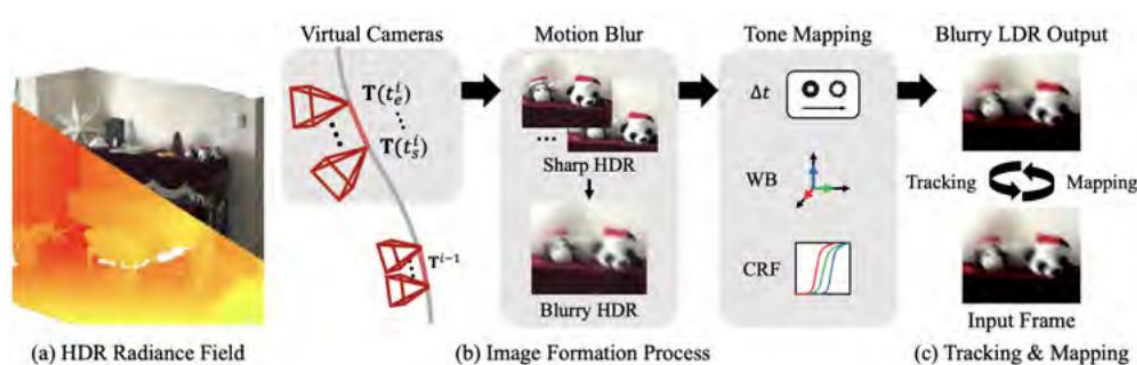


图 14.15: 图 14.19 I²-SLAM 的系统概览^[42] (©2024 Springer)。该方法将物理成像过程集成到 SLAM 中，以捕获 HDR 辐射场。

强重建的广泛应用体现了 3DGS 的关键优势：灵活性和适应性使其适用于多样的 SLAM 场景。

14.4 延伸阅读与近期趋势

基于 NeRF/3DGS 的 SLAM 具有端到端优化、复杂光场捕获、实时渲染和动态基元分配等优势，是有前景的研究方向。然而，仍有许多重要技术挑战。本节讨论这些局限，并概述新的趋势和潜在未来研究方向。



(a) PhotoSLAM ^[488]



(b) RTG-SLAM ^[868] 图 14.20 将 3DGS 与基于关键点的 SLAM 结合的方法示例。图像来自 ^[488] (©2024 IEEE) 和 ^[868] (©2024 ACM)。

实时性能。基于 NeRF/3DGS 的端到端 SLAM 面临的主要挑战之一是处理速度极慢。2015 年，ElasticFusion ^[1184] 等经典稠密 SLAM 方法已能在 GTX 780Ti 上实现包含跟踪、建图和渲染的实时性能；但大多数基于 NeRF/3DGS 的 SLAM 方法即使在 FLOPs 和显存明显更高的现代 GPU（例如 NVIDIA RTX 4090）上，也难以超过 5 fps。这一限制使当前基于 NeRF/3DGS 的 SLAM 难以用于实时机器人应用，只

能依赖离线服务器 GPU。由于 3DGS 支持实时渲染，如何高效优化它仍是关键挑战。提高效率的一条潜在路线是采用紧凑表示，减少高斯数量，从而降低计算开销；另一种方法是使用高阶优化器替代标准一阶梯度下降，以加速收敛。还可以训练网络，通过一次前向传播同时预测相机位姿和三维高斯，从而消除迭代优化。PixelSplat^[169] 和 MVSplat^[191] 已证明，前馈网络能够从带位姿图像预测三维高斯参数，无需测试时优化。尽管这些早期工作需要外部模块估计的相机位姿，随着 DUST3R^[1164] 的成功（详见第 13 章），不依赖位姿输入、能鲁棒预测点图的无位姿三维重建方法开始受到关注。在此基础上，No-PoSplat^[1242]（图 14.21）进一步实现了无位姿 3DGS 预测，展示出有前景的结果。将这些技术用于多视图增量式 SLAM 设置，是值得探索的未来方向。

相机跟踪精度。基于 NeRF/3DGS 的 SLAM 相机跟踪精度往往落后于经典稀疏 SLAM 方法。这些方法依赖多帧之间的光度一致性，在 Replica^[1040] 等具有真实深度和彩色图像的合成数据集上表现良好。然而，在真实场景中，由于传感器噪声、卷帘快门等未建模效应，该假设常常失效。提高精度可能需要更精确地建模场景光场和渲染过程，或探索彩色图像域之外的替代匹配范式。正如上一段所述，引入更鲁棒的前馈预测模型也可能解决这一挑战。

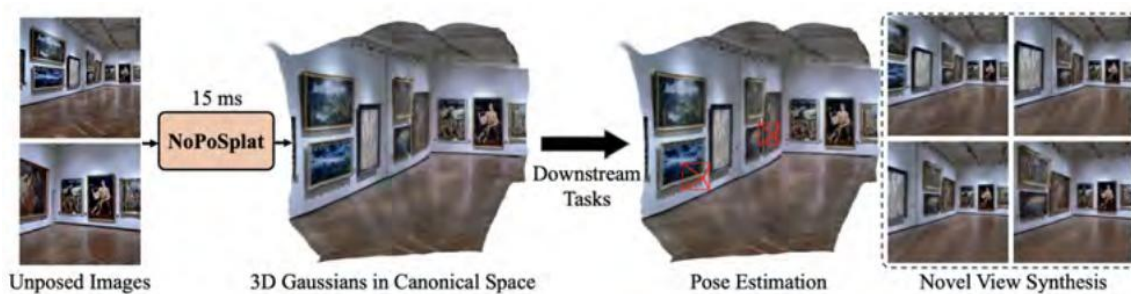


图 14.16: 图 14.21 NoPoSplat 的系统概览^[1242]。该方法直接从两个输入视图预测三维高斯，可用于位姿估计和新视图合成。图像来自^[1242]。

四维场景重建。当前大多数 NeRF/3DGS SLAM 方法假设环境静态，但真实场景往往存在显著动态变化。一种常见做法是屏蔽潜在动态物体，只重建静态背景，以获得鲁棒里程计估计。不过，考虑到 3DGS 在非刚性/四维场景重建和点跟踪方面取得的显著成功，探索完整的 4D-SLAM 框架似乎是一条有前景的路线^[138, 986, 1289, 741]。

大规模/室外场景重建。当前大多数基于 NeRF/3DGS 的 SLAM 方法局限于房间尺度序列，因此大规模和室外场景重建仍是重大挑战。扩展规模需要高效地图更新，例如采用自适应分辨率管理计算负载，并进行漂移校正以维持大范围区域的精度。室外环境还引入额外复杂性，包括天气和一天中不同时间导致的光照变化，这会使光场建模更加困难。

第十五章 动态与可变形 SLAM

作者：Lukas Schmid、Jose Maria Martinez Montiel、Shoudong Huang、Daniel Cremers、Jose Neira 和 Javier Civera

长期以来，SLAM 系统一直面临这样的环境挑战：不仅传感器在运动，场景本身也在运动。为应对这一挑战，数据关联结合 RANSAC 等离群值剔除方法，试图区分两类传感器数据：一类对应相对于传感器运动呈现一致位姿变化的实体，另一类描绘独立运动的实体。这类似于星星在天空中看似协同移动，而行星却四处游走，因而得名。¹

本手册前面介绍的方法，均围绕根据传感器观测估计机器人位姿和环境表示展开。但这些方法有一个重要假设，即环境是静态的：场景保持不变，只有机器人运动。这一假设带来多项重要影响。例如，第 1 章说明，如果检测到的特征保持在原处，重新检测它们就可以直接更好地约束机器人位姿。相反，第 5 章展示了只要底层环境保持不变，就可以通过融合多个带位姿观测高效地重建稠密场景模型。

然而，真实世界场景往往高度动态，违反了“环境中的一切都刚性连接且不可移动”的静态假设。例如，设想自动驾驶车辆要在城市中安全行驶，需要在地图表示中记录哪些信息：环境不仅有静态背景，还有其他车辆、骑行者和行人等各种运动元素。再考虑家庭或服务机器人：人类经常在机器人周围移动，修改物体或重新布置房间。更极端的例子是手术机器人在人体内导航，那里没有静态背景，整个场景都在持续变形。这些因素使动态和可变形场景中的机器人状态估计与环境表示都成为困难问题。

为应对这些局限，本章分析动态和可变形 SLAM 的挑战。第 15.1 节首先定义并分类不同类型的动态效应和动态 SLAM 目标；第 15.2 节和第 15.3 节分别介绍短期和长期动态 SLAM 中的主要问题及当前解决方法；第 15.4 节讨论可变形 SLAM；最后，第 15.5 节回顾尚存挑战，并展望这一快速发展领域的未来研究方向。

¹ 希腊语 $\pi\lambda\alpha\nu\eta\tau\eta\varsigma$ (planētēs) 意为“漫游者”。

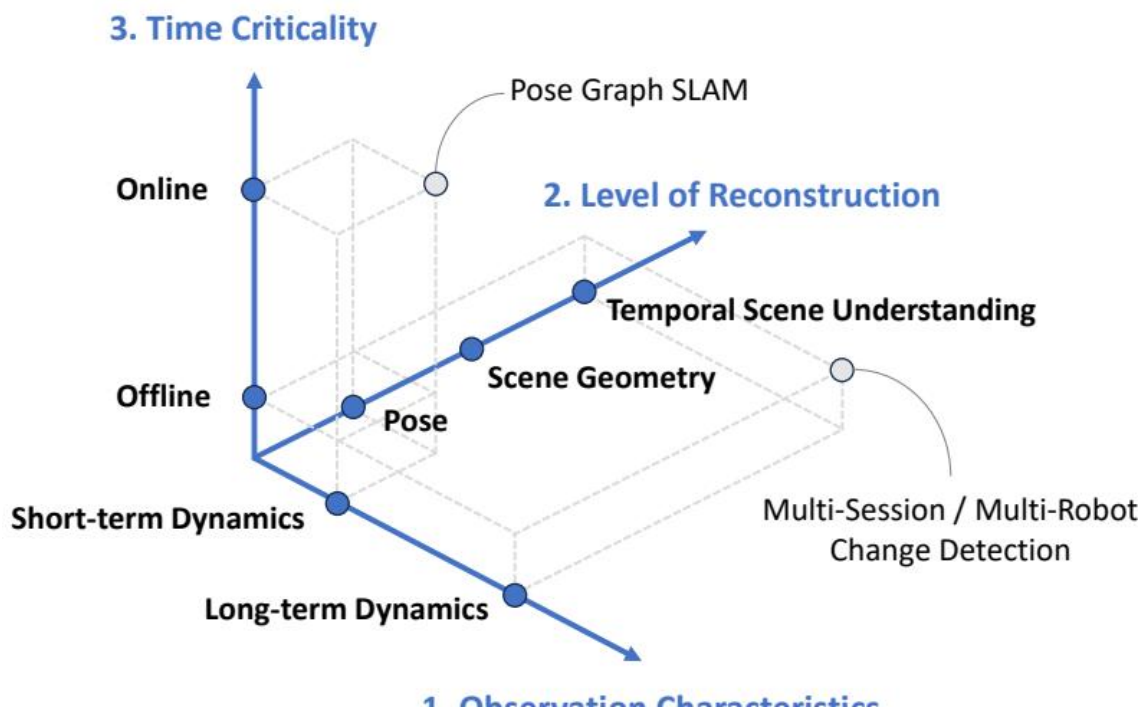
1. 观测特性图 15.1 动态 SLAM 的主要考量概览：(1) 机器人观测中存在哪些动态效应；(2) 为完成任务，机器人需要以何种细节层次理解环境；(3) 机器人何时以及以多快的速度处理这些信息。例如，典型位姿图 SLAM 可以通过将动态部分作为离群值剔除，实时估计机器人位姿。另一方面，多会话变化检测旨在从不频繁的观测（会话）中详细理解场景演化。需要注意，有些解决方案可以在这些坐标轴张成的空间中清晰描述；另一些方法则覆盖一个“区域”或“体积”，同时处理短期和长期动态、联合状态估计与场景表示、在线与离线组件，以及这些维度的组合。

15.1 动态 SLAM 问题的刻画

建模动态环境时需要重点考虑：(1) 机器人观测中存在哪些动态效应；(2) 为完成任务，机器人需要以何种细节层次理解环境；(3) 机器人何时以及以多快的速度处理这些信息。理想情况下，通用机器人可以实时构建任何动态场景的丰富描述，但在许多情况下，只需处理与具体应用相关的问题子集就足够且更高效。图 15.1 概览了这些维度，下面分别讨论各个坐标轴。

15.1.1 动态特性的刻画

我们根据动态效应在机器人观测中的表现来刻画动态。遵循^[693]的定义，将动态区分为短期动态和长期动态，分别指机器人观测中更瞬态或更突发的特征。关键思想是：从足够短的时间尺度看，几乎所有物理过



程在根本上都是连续的。因此，机器人观测有多大差异、变化属于短期还是长期动态，并不取决于现象的绝对持续时间，而取决于场景变化速率与机器人观测速率之间的关系。图 15.2 展示了这种关系，并给出多个例子说明它适用于广泛时间尺度。

虽然上述定义具有一般性并适用于很大的时间尺度，但需要指出，在大多数机器人应用中，相关变化速率（人或物体通常以约 10^{-1} 到 10^2 m/s 的速度运动）与观测速率（大多数机器人传感器的频率约为 10^0 到 10^3 Hz）处于同一数量级。在这种情况下，区分往往简化为机器人当前是否正在观测运动。于是，短期动态指机器人视野内发生的所有运动；对动态实体的连续观测会表现为瞬态变化。例如，连续观测到的滚动球的平滑轨迹，或人在相机前行走时的协调运动。第 15.2 节将讨论应对短期动态的技术。

另一方面，长期动态指发生在机器人视野之外的动态。虽然运动在局部仍是连续的，但机器人只能观测测量间隔内累积的运动结果，因此通常表现得更突发。例如，一个人拿起瓶子，再将它放回初始位置旁边。如果机器人只在交互前后观察桌面上的瓶子，从机器人角度看，瓶子似乎在两次观测之间瞬移。多会话或多机器人建图是更极端的例子：单次访问期间场景可能静态，但机器人不在场时的多次访问之间会累积变化。

因此，这一观点的重要含义是：短期与长期动态的区别取决于机器人的观测，而非环境或运动物体的固有属性，如图 15.3 所示。不过，长期动态发生的概率通常与观测之间的时间间隔相关。第 15.3 节将概述长期动态方法。

最后，上述情况通常假设场景有静态背景及多个动态实体。但在内窥镜手术和医疗机器人等相关应用中，场景可能完全变形并在机器人周围运动，通常属于上文所述的短期动态。没有任何静态场景部分的设置通常称为可变形 SLAM，第 15.4 节将讨论这一问题。

15.1.1.1 更多术语

需要指出，在相关研究社群中，还使用许多术语描述与上述概念相关的内容。除短期动态外，常用术语包括 *dynamic* 和 *high-dynamic*，通常指当前正在运动的物体（可能位于机器人视野内）。

另一方面，描述长期动态效应时，会使用 *semistatic*、*quasi-static* 或 *low-dynamic* 等术语。这些术语通常指可能移动但在被观测时静止的物体，或多次访问之间发生的变化，后者是长期动态的特殊情形。

除处理三维 SLAM 中的动态效应外，有些方法还旨在重建场景的历史、演化或更高层理解。这通常称为时空场景模型，或四维场景模型（即三维空间加时间）。²

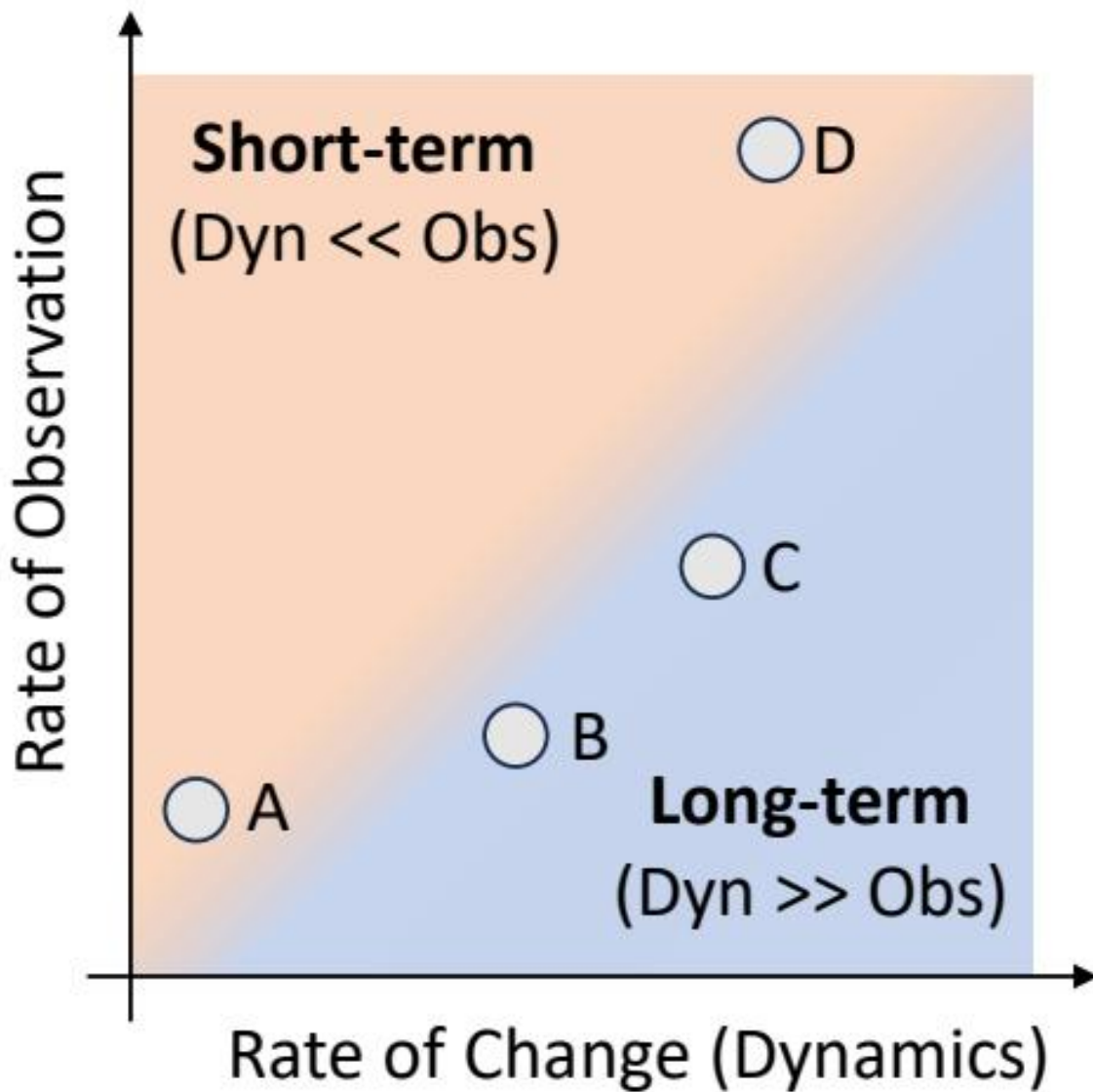


图 15.1: 图 15.2 短期与长期动态的定义：机器人表现出的动态特征由场景变化速率与机器人观测速率的关系决定，而非由绝对时间跨度决定。注意，在多数机器人领域，传感速率足以捕获视野内的连续运动，因此可简化为：机器人视野内运动的物体（短期动态）与在机器人视野外发生变化的物体（长期动态）。示例：A) 每天测量植物生长。虽然观测频率很低，植物也只生长很少，因此跟踪植物的方法类似于跟踪人^[175]。B) 每天巡逻建筑。观测频率相当，但场景部分区域可能已完全变化，适合采用多会话变化检测等长期方法^[565]。C) 机器人背后物体的重新布置。机器人可能只离开一分钟，但两次观测间物体配置可能已显著改变^[976]。D) Hawk-Eye 系统以 340 Hz 观测网球，可跟踪速度高达 250 km/h 的网球，得到短期动态观测和毫米级跟踪精度^[675]。

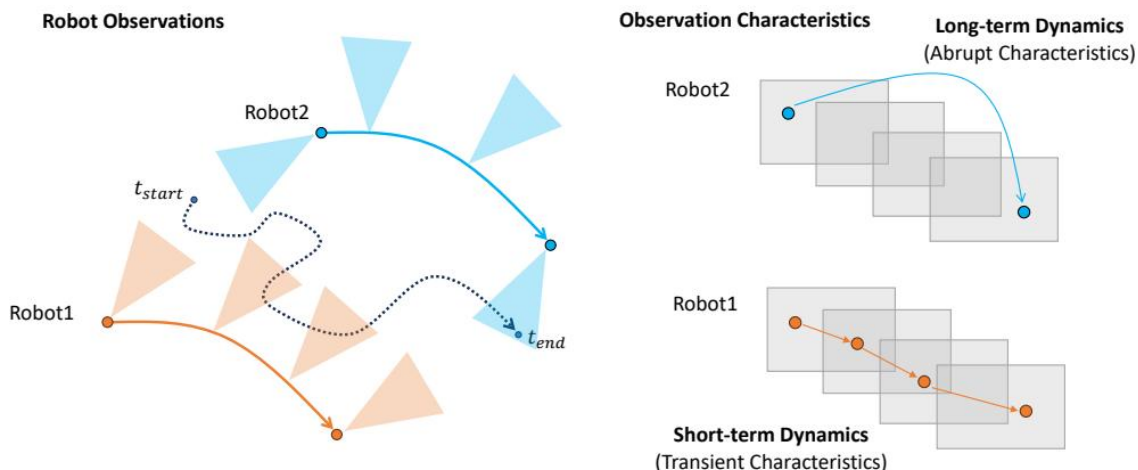


图 15.2: 图 15.3 机器人作为观测者的作用。同一物体的物理运动（虚线）对于 Robot1 可以表现为短期动态，对于 Robot2 则表现为长期动态。类似地，多会话或多机器人建图可能包含物体在观测时运动（短期动态），也可能包含物体在机器人视野外、通常是在会话或访问之间发生变化（长期动态）。

² 注：这不同于雷达 SLAM（第 9 章）中使用的 4D 或 3+1D，其中第四维指雷达测量的速度。

此外，lifelong、persistent 和 continuous 等术语有时用于描述能够应对变化环境的 SLAM、建图或定位系统，这些系统往往在很长时间跨度内运行。最后需要注意，不仅几何动态会影响传感器读数，纹理、光照和昼夜或季节变化等其他变化也会影响传感器读数，尤其是在使用视觉传感器时^[1169, 1008]。

15.1.1.2 动态程度

上述刻画似乎将问题明确分为三种情况：静态背景、短期动态和长期动态。但在实践中，边界往往模糊，不同效应可能组合出现，且每种效应具有不同强度。例如，场景可能包含单个刚性运动物体（如汽车或箱子^[691]）、单个运动且变形的物体（如人^[449]）、只有长期变化（如建筑的定期巡逻^[321]）、同时运动和变化的多个实体（如家庭中的机器人和人^[976]），甚至完全没有静态实体（如内窥手术^[893]）。理想情况下，通用机器人能够联合估计所有这些效应，但考虑的效应越多，问题复杂度呈指数增长。因此，预先知道可能出现或与任务相关的动态效应，可以针对具体场景设计更专业、性能更好且更易处理的解决方案。

15.1.2 状态估计与场景表示

第 5 章介绍过，SLAM 是同时估计机器人位姿和环境表示的双重过程。动态环境中的 SLAM 也适用同样的考量。如果应用只需估计机器人状态，通常可以将动态观测视为离群值或噪声。此时，类似静态场景中的 SLAM，可以忽略或拒绝对应特征或测量，并相对于静态背景跟踪或定位，从而获得准确结果。第 15.2.1 节和第 15.3.1 节将分别讨论短期和长期动态下的相关问题。可变形 SLAM 是一种特殊情况，因为不存在可用于定位的静态部分，第 15.4 节将讨论这一设置。

相反，如果需要稠密地图，重建运动和变化的场景就要求机器人检测并表示场景中的部分或全部运动与变化。第 15.2.3 节讨论短期动态表示和技术，第 15.3.2 节讨论长期动态，第 15.3.3 节介绍首批统一动态 SLAM 方法。此外，第 15.3.4 节还将简要介绍用于更高层理解动态和变化场景的方法。

15.1.3 在线与离线方法

我们考虑的最后一个坐标轴是感知系统的时间紧迫性，即必须实时处理动态，还是可以离线处理数据。由于大多数机器人应用要求机器人立即与环境交互，尤其是在动态场景中，本章主要关注实时方法。但在建筑

定期监测等应用中,也可以先采集传感器数据再离线处理。类似于(离线)运动恢复结构与(在线)SLAM的区别,离线处理可以利用更多计算资源,以更高分辨率处理数据,通常能得到更准确结果。在线和离线方法的另一个重要区别是,离线方法已经拥有全部数据。具体而言,要检测时刻 t 的帧 F_t 中的运动或变化物体,未来测量 $F_{t+1}, \dots, T$ 都已可用,它们能提供在线方法无法获得的重要信息。本章主要关注在线方法,同时,在第15.3.2节简要介绍地图清理和变化检测等离线问题,在第15.4.1节介绍非刚性运动恢复结构(NRSfM)。

最后,虽然本手册第二部分按感知模态组织,但动态场景对SLAM的影响在多种感知模态之间往往相似。因此,本章按不同动态特性组织。不过,各节仍会讨论不同感知模态的考量;迄今为止,视觉、LiDAR和本体传感器在动态与可变形SLAM中的应用最为显著。

15.2 短期动态与动态 SLAM

下面各节进一步讨论单个短期动态物体对SLAM的影响。早期方法使用数据关联识别可由传感器运动解释的测量,并将它们与对应物体独立改变位姿的测量区分开。初始技术包括统计检验^[797]、RANSAC^[222]和Hough变换^[1073]等经典算法。这些方法可以简单地将动态物体的测量丢弃为离群值。之后,研究者不再满足于忽略动态物体,还尝试重建被其遮挡的环境区域,例如插值缺失数据^[1071]、从未发生遮挡的替代传感器位姿恢复数据^[75],或使用机器学习进行图像“修复”^[77]。这样得到的地图对定位更加有用,因为渲染地图中不再存在“孔洞”。第15.2.1节将讨论把动态物体视为离群值的处理。

相关研究还探索了另一种方案:在SLAM框架中跟踪动态物体。在计算机视觉中,这一问题称为多体运动恢复结构,是经典SfM的扩展。其数学基础和实用算法与SLAM视角有许多共同之处^[835]。由于计算成本高昂,该方案最初被认为不可行^[1149];近年来已成为可能^[76],并且为了避免机器人与动态物体碰撞,可能必须采用该方案。第15.2.2节将讨论动态物体跟踪。

为更详细地理解动态场景,还可以进一步以稠密方式重建场景,包括动态物体。获得详细的运动物体模型有助于操作等应用,因为机器人需要理解物体形状和运动才能与之交互。第15.2.3节将讨论稠密动态SLAM。

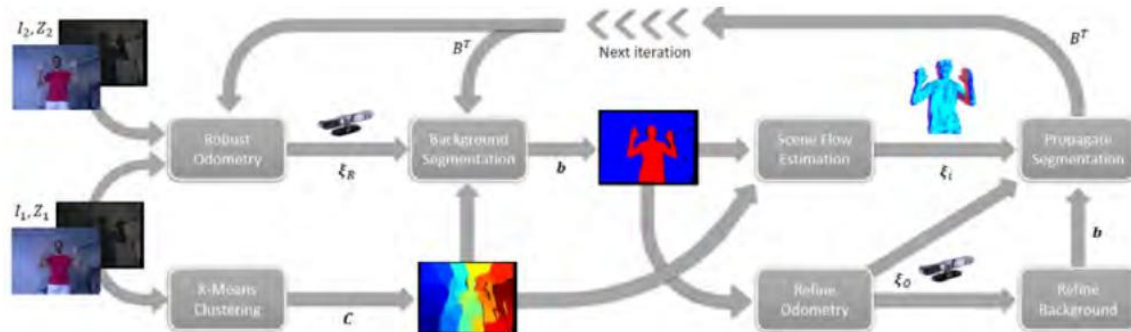


图 15.3: 图 15.4 文献^[509]介绍了一种从 RGB-D 视频序列联合估计相机运动和运动物体三维场景流的方法。首先使用鲁棒里程计,根据各聚类的残差将聚类后的帧划分为动态区域和静态(背景)区域;随后利用该分割结果进一步细化里程计。(©2017 IEEE)

15.2.1 动态物体移除

在部分动态场景中,最简单但仍有效的SLAM方法之一,是丢弃场景动态部分的传感器测量,并从地图状态中移除这些部分。其依据有两点。第一,持久物体和地标通常是建图的主要关注对象,而动态实体很多时候兴趣有限(例如经过的行人)。第二,大多数几何估计技术假设场景刚性,因此可以将动态部分检测为刚性模型中的伪测量。围绕这一目标,研究者提出了大量方法。下面总结主要方法族。需要注意,现代动态SLAM往往组合使用这些原则,以取得最佳性能。

本体传感。本体传感的一个关键属性是,顾名思义,它不依赖机器人外部的观测。因此也不会受到场景运动等外部效应影响。惯性和内部里程计测量携带机器人状态信息,有助于根据相对测量区分机器人自身运

动和场景动态部分的运动^[1024]。关于如何融合惯性或运动学测量与外感知测量，详见第 11 章和第 12 章。

鲁棒估计。大多数场景和传感器提供的测量数量远多于几何估计的最小需求。在动态内容占比较低时，可以利用冗余，将动态物体的测量识别为刚性几何模型的离群值。



无掩膜模块

带掩膜模块图 15.5 MonoRec^[1187] 训练深度网络，从单个运动相机恢复稠密重建。掩膜模块利用亮度一致性代价体积，识别与主导自运动不相容的结构（连续帧之间的颜色），并过滤运动物体。（©2021 IEEE）

主要使用两种方法。第一，通常使用 RANSAC^[328] 初始化状态估计：从完整测量集合中选择具有最大一致性的最小样本集。但在遮挡严重的情况下，最大一致性集可能来自运动物体，导致失败。第二，在初始化之后和迭代优化期间，使用鲁棒损失函数替代典型的 ℓ_2 范数。对于被视为离群值的大残差，鲁棒损失的增长低于二次增长，因此可以降低其对状态的影响。关于 RANSAC 和鲁棒损失的详细信息，参见第 3 章。



多视图运动线索。除了在初始化阶段丢弃动态内容，并通过鲁棒损失降低甚至消除其影响外，另一种常见方法是在测量提供足够非刚性证据后移除动态状态。证据通常实现为高残差测量累积数量的阈值。图 15.4 展示了一个例子：将 RGB-D 图像聚类为多个区域，再根据区域残差将每个区域分类为动态或静态（背景）。

语义信息。语义分割深度网络可以预先分割可能对应动态物体的图像区域。这当然有帮助，但封闭集词汇可能无法分割图像中的所有物体。此外，物体具有潜在动态性并不意味着它此刻正在运动，例如停放的汽车。因此，将潜在动态物体的语义分割与多视图几何检查结合起来可能更合适^[44]。例如，MonoRec^[1187] 提出使用深度网络从单个运动相机进行稠密重建，将亮度代价体积（由一组连续重映射帧计算）与掩膜模块结合，过滤运动物体。这些物体既根据训练数据确定，也根据重映射帧之间的颜色不一致确定，如图 15.5 所示。

领域知识。如果可用，可以集成额外先验和领域知识，以区分静态场景特征和动态离群值。自动驾驶车辆的无侧滑约束就是常见例子。由于知道汽车不能横向移动，因此可以将所有暗示侧向运动的特征作为动态离群值移除。

运动感知传感器。视觉和 LiDAR 传感器通常只在给定时刻采集场景，因此不能直接测量动态物体的运动。事件相机等其他传感器能够以高时间分辨率提供连续测量；此外，雷达和某些 LiDAR 利用多普勒效应直接测量每个观测点的速度。虽然使用这些传感器的动态 SLAM 仍在积极研究，但此类信息可以成为识别动态物体及其运动的重要线索。事件相机和雷达的更多细节参见第 10 章和第 9 章。

15.2.2 动态物体跟踪

如果目标只是估计机器人状态，上一节介绍的动态物体移除可能很方便。但在许多其他情况下，例如为了避免碰撞导航，可能需要跟踪这些动态实体的三维运动。该问题与多目标跟踪（MOT）或多实例动态 SLAM（MID）高度相关，有时也使用这些名称。计算机视觉和机器人领域对二维、三维物体跟踪投入了大量研究^[1246]，相关文献非常庞大，本章无法穷尽；这里聚焦与 SLAM 最相关的部分技术。

同步定位、建图和动态物体跟踪的联合问题可以如下表述。核心思想是将场景中的每个动态物体建模为刚性运动体。第 7 章介绍的静态 SLAM 因子图表示可以扩展到单独运动的物体，如图 15.6 所示。与之前一样，大多数方法采用跟踪与建图并行的方案。对于每个输入帧，跟踪线程提取显著特征（例如 ORB^[953]）；为将点分配给单个物体，通常还会提取语义分割掩膜。

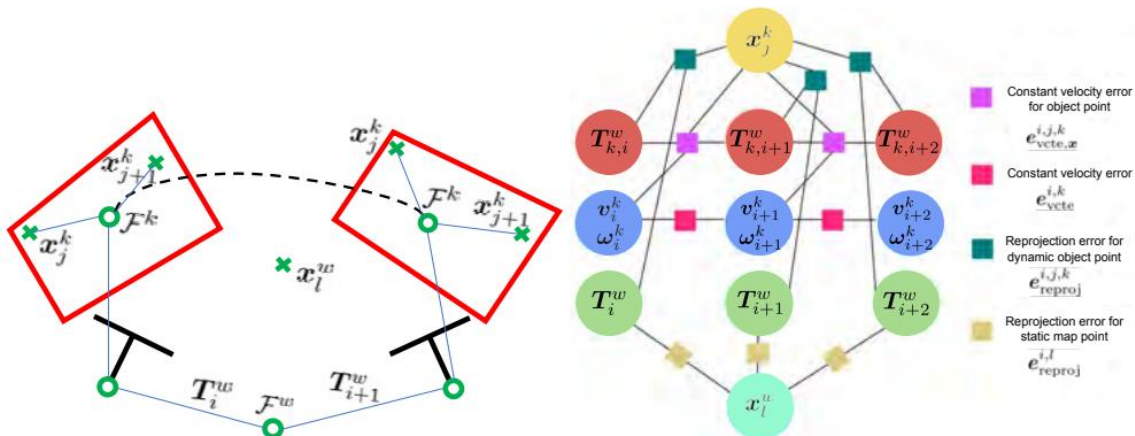


图 15.4: 图 15.6 带动态物体跟踪的 SLAM 概览^[76] (©2021 IEEE)。左：刚体运动假设。相机位姿 T_i^w 相对于静态背景估计；动态物体 k 相对于世界坐标系 $\mathcal{F}^w$ 建模为刚体。注意，每个物体 k 上的所有特征点 x_j^k 都遵循其刚体运动 $\Delta T_{k,i+1}^{k,i}$ 。右：问题对应的因子图表示。相机位姿通过里程计因子连接，动态物体位姿通过恒速度因子连接。

令 $T_i^w \in \text{SE}(3)$ 表示时刻 t_i 相机在世界参考坐标系 $\mathcal{F}^w$ 中位姿的刚体变换， $x_l^w \in \mathbb{R}^3$ 表示以世界坐标系表达的刚性地图点 l 的坐标。每个动态物体 k 在时刻 t_i 由六自由度变换 $T_{k,i}^w \in \text{SE}(3)$ 表示，该变换将时刻 t_i 运动物体坐标系 $\mathcal{F}^k$ 中的点变换到世界坐标系；同时还用物体坐标系中的线速度和角速度 $v_i^k, \omega_i^k \in \mathbb{R}^3$

表示其运动。动态物体 k 上的地图点 j 在局部物体坐标系中表示为 $\mathbf{x}_j^k \in \mathbb{R}^3$ 。对于刚体运动学，局部坐标系 $\mathcal{F}^k$ 中物体点的坐标不随时间变化，因此不依赖 i 。于是，对应 $\mathbf{x}_j^k$ 的重投影误差变为

$$\mathbf{e}_{\text{reproj}}^{i,j,k} = \mathbf{z}_j^i - \pi \left((\mathbf{T}_i^w)^{-1} \mathbf{T}_{k,i}^w \tilde{\mathbf{x}}_j^k \right), \quad (15.1)$$

其中 $\tilde{\mathbf{x}}_j^k \in \mathbb{R}^4$ 是每个物体点 $\mathbf{x}_j^k$ 的齐次坐标， $\mathbf{z}_j^i$ 是与 $\mathbf{x}_j^k$ 对应的图像点， $\pi(\cdot)$ 表示投影函数，此处将其重载为作用于齐次坐标点的函数。

为了进一步约束每个动态物体 k 随时间的运动，通常在动态物体连续观测之间假设线速度和角速度恒定。需要最小化的相应残差为

$$\mathbf{e}_{\text{vcte}}^{i,k} = \begin{bmatrix} \mathbf{v}_{i+1}^k - \mathbf{v}_i^k \\ \boldsymbol{\omega}_{i+1}^k - \boldsymbol{\omega}_i^k \end{bmatrix}. \quad (15.2)$$

为耦合物体速度和位姿，还需加入误差项 $\mathbf{e}_{\text{vcte},\mathbf{x}}^{i,j,k}$ ，最小化运动模型计算的 $\mathcal{F}^w$ 中物体点 j 位置与根据位姿估计得到的同一位置之间的不匹配：

$$\mathbf{e}_{\text{vcte},\mathbf{x}}^{i,j,k} = h^{-1} \left(\left(\mathbf{T}_{k,i+1}^w - \mathbf{T}_{k,i}^w \Delta \mathbf{T}_{k,i+1}^{k,i} \right) \tilde{\mathbf{x}}_j^k \right), \quad (15.3)$$

其中使用 $h^{-1} : (x, y, z, \lambda) \rightarrow (x/\lambda, y/\lambda, z/\lambda)$ 将齐次坐标变换为欧氏坐标。根据恒速度模型，时刻 t_i 到 t_{i+1} 的位姿增量计算为

$$\Delta \mathbf{T}_{k,i+1}^{k,i} = \begin{bmatrix} \text{Exp}(\boldsymbol{\omega}_i^k (t_{i+1} - t_i)) & \mathbf{v}_i^k (t_{i+1} - t_i) \\ \mathbf{0} & 1 \end{bmatrix}. \quad (15.4)$$

图 15.6 (左) 展示了这些建模假设，(右) 展示了待优化的因子图。

这一通用表述已在多项工作中取得显著成功。例如，早期的 Multimotion Visual Odometry (MVO) [527] 检测轨迹片段，并将其分割为满足每个聚类刚体运动约束的簇。Visual Dynamic Object-aware SLAM (VDO-SLAM) [1270]、Dynamic SLAM [448] 和 DynaSLAM II [76] 引入基于因子图的方法，并在观测之间加入运动一致性因子。AirDOS [897] 又将其推广到由多个相连刚体组成的关节物体，例如人体骨架。

15.2.3 稠密动态 SLAM

与第 5 章讨论的静态稠密 SLAM 类似，稠密动态 SLAM 还要估计包含动态实体的场景稠密重建。为此，大多数方法将过程分为定位和建图两步。

定位步骤旨在相对于静态背景跟踪机器人位姿，可以使用动态物体移除 (第 15.2.1 节) 和 MOT (第 15.2.2 节) 中介绍的任意稀疏动态 SLAM 方法。也可以先从输入中移除动态点，再对背景执行稠密跟踪。

建图步骤旨在同时表示静态背景和动态实体。表示的选择是关键设计决策，因为它决定了连续更新表示时需要估计哪些量。背景重建自然与静态稠密建图相同，但也可以使用相同或相似的代表建模动态实体。背景和动态实体不必采用相同表示，但为便于解释和处理机器人地图，实践中通常如此。本节讨论几类常见方法：第 15.2.3.1 节从运动物体分割 (MOS) 和传感器级表示开始，第 15.2.3.2 节讨论目标物体或类别已知时的参数化表示，第 15.2.3.3 节讨论任意运动物体的重建。

15.2.3.1 运动物体分割

处理稠密动态 SLAM 的最小方案，是检测输入数据中的所有动态部分，例如所有点或像素。这一问题通常称为运动物体分割 (MOS)。与动态物体移除 (第 15.2.1 节) 类似，检测到这些点后，可以忽略它们以避免静态背景重建中的不一致和伪影；也可以将其作为场景中动态实体的低层表示保存。检测动态或不一致点时，可以采用前文类似的思想。但关键区别是，稠密 SLAM 还需要稠密的动态点检测，即必须对传感器数据中的每个点进行分类，而非只分类选定特征，如图 15.7 所示。因此，大多数方法依赖几何一致性和/或语义信息作为稠密分割的主要运动线索。前一种方法在相机到模型跟踪期间稠密计算配准残差，将输入图像中高残差区域分类为动态 [984]；或者利用单次测量之间 [1254]、或融合到地图中的测量之间 [975] 的一致性，识别不一致、因而动态的点。通常还会加入聚类步骤，以处理传感数据中的噪声 [984, 1254, 975]。后一种方法通过稠密

语义分割判断输入数据哪些部分反映可能动态的类别或实例（如人），并将其屏蔽^[941]。相机图像和 LiDAR 点云的语义分割已得到广泛研究，可直接重新用于屏蔽动态类别；而利用深度学习方法直接在单次或连续测量上分类动态像素或点，也成为不断增长的研究领域^[871, 758, 664]。基于学习的方法能够纳入典型运动物体的形状、运动模式和传感器特性等先验，可提升目标域性能，但未必能泛化到域外环境。

15.2.3.2 参数化模型

对某些应用而言，丢弃动态测量并重建静态背景可能已经足够；但对于操作或人机交互等应用，机器人可能还需要估计场景中动态实体的详细表示。在这些情况下，第 15.2.3.1 节的基于检测的跟踪方法表达能力有限，因为它们只在每一帧动态区域的噪声、局部测量层面运行。

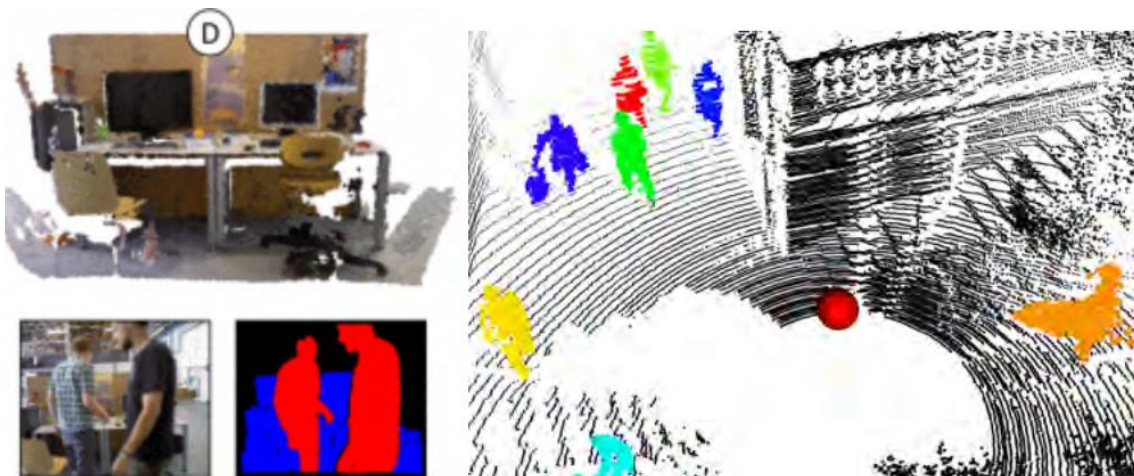


图 15.5: 图 15.7 稠密运动物体分割 (MOS)。(左) StaticFusion^[984] 根据跟踪残差，将每帧 RGB-D 图像（下）分割为动态（红色）和静态（蓝色）部分，并重建静态背景（上）。（右）Dynablox^[975] 利用体积地图一致性，将 LiDAR 扫描稠密分割为动态（彩色）和静态（黑色）点。（^[984] ©2018 IEEE, ^[975] ©2023 IEEE）

如果已知感兴趣物体的类型，其形状通常可以用参数化模型表示。给定这种结构，每个测量都可以视为真实底层参数的观测，并用于优化这些参数。类似 MOT（第 15.2.2 节），可以将这些参数作为变量加入因子图，与模型特定的观测因子、传感器运动和背景一起估计。这一方法尤其受到人体运动捕获和跟踪领域关注：人体可以表示为骨架模型^[897] 或稠密 SMPL^[694] 网格^[943, 449, 450] 等。

15.2.3.3 同时跟踪与重建

另一条路线是同时跟踪与重建，旨在重建任意运动物体。多数情况下，可以简化假设每个运动物体都是刚体，目标则是跟踪和重建一个或多个刚性运动物体。核心思想是为每个物体及背景分别建立稠密表示。大多数方法遵循四个主要步骤^[962, 958, 1206, 927]：第一，在输入图像中进行物体检测，通常将语义或实例分割与几何细化和/或几何运动线索结合；第二，相对于背景跟踪相机运动；第三，相对于当前图像跟踪每个运动物体。第二、三步可以采用多种技术，通常使用最小化每个物体投影深度残差和光度残差的直接法，因为这类跟踪方法可以直接作用于物体或背景的稠密表示。最后，将各物体和背景的测量分别融合到对应模型中，增量估计每个物体的稠密重建。最常见的是将物体表示为截断有符号距离函数 (TSDF) 体积 (TSDF 详情见第 5.2.2 节)，因为它能够融合带传感噪声的测量^[962, 958, 1206, 927]。图 15.8 给出一个例子。另一种方案 DirectTracker^[386] 使用更稀疏的基于点云的重建，联合估计物体分组及各物体的刚体运动。

同时跟踪与重建的主要优点是通用性更强，因为不需要预先给定物体模型。但所需步骤较多，计算成本可能很高，特别是在大场景和动态物体数量较多的情况下。另一个局限是，物体跟踪误差可能导致错位测量被融

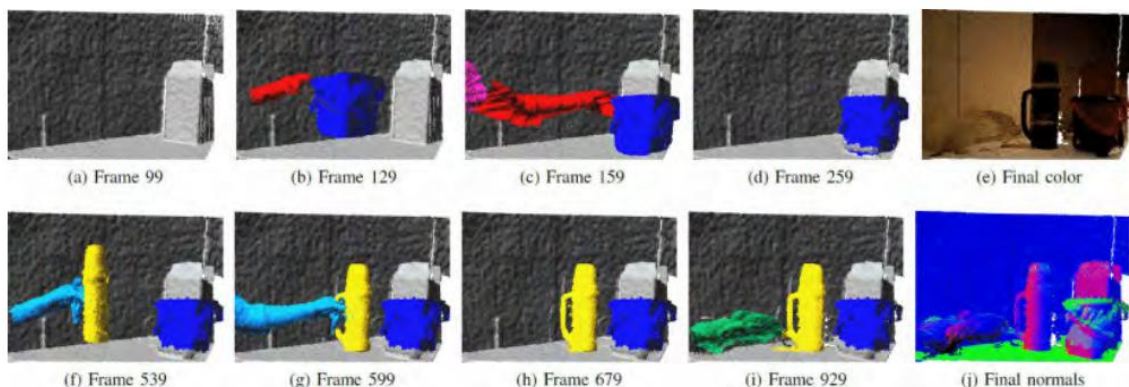


图 15.6: 图 15.8 Co-Fusion^[962] 的同时跟踪与重建。三个物体依次放置在桌上：小箱子（蓝色标签）、烧瓶（黄色）和泰迪熊（绿色）。结果表明，所有物体都被成功分割、跟踪和建模。（©2017 IEEE）

合到物体表示中，降低重建质量，并进一步妨碍未来的准确跟踪；例如，基于关键点的跟踪方法 BundleTrack^[1179] 可以更加鲁棒。

通过估计连接到完整物体的每个刚性部分的变换，可以将刚性物体假设放宽到关节物体^[977]，也可以推广到一般可变形物体。后者包括 DynamicFusion^[802] 和 KillingFusion^[1016]，第 15.4 节将进一步介绍。图 15.9 所示的 AnyCam^[1189] 是一种基于学习的方法，它使用基于 Transformer 的神经网络，从普通单目视频联合估计相机运动以及动态场景中非刚性运动物体的四维重建。

最后，额外重建的物体也可用于改善机器人自运动跟踪。不过，只有在一些特殊情况下，重建物体才会明显改善机器人状态估计，例如机器人视野的大部分被遮挡（超过 70%），且有良好的运动物体模型^[691]，导致相对于背景跟踪不足。

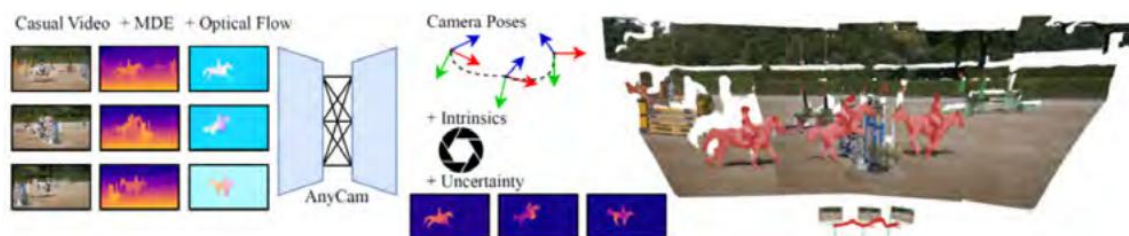


图 15.7: 图 15.9 AnyCam^[1189] 是一种基于 Transformer 的神经网络，从普通输入视频联合估计相机运动和四维非刚性重建。（©2025 IEEE）

15.3 长期动态与终身 SLAM

上一节讨论了处理短期动态观测的考量。本节讨论长期动态对 SLAM 的挑战和影响。

总体而言，长期动态 SLAM 存在若干显著挑战。第一，与短期动态相比，观测到的变化更加突发，原则上可以任意增大（机器人越长时间没有观察某个空间，该空间与上次观测的差异就越大）。这使数据关联显著更加困难。例如，短期运动按定义在观测之间很小，可以通过跟踪处理数据关联；而长期动态下的数据关联可能需要求解潜在的全局重新识别和匹配问题。此外，感知混叠问题会加剧：一个独特地标（或同一物体上的多个特征点）可能因移动而在多个位置被观测到。

第二，由于变化幅度更大，许多在短期情况下可忽略的效应在长期情况下变得重要。这包括从运动引起的几何变化（例如物体被加入、移动或移出场景^[974]）到结构变化（例如建筑工地每次访问时都可能完全不

同^[1057]), 以及从光照变化(不同天气、时间或季节^[1169])到表面纹理或反射率变化等外观变化。

最后, 长期动态观测通常伴随着更长的时间间隔和更远的机器人运动, 因此观测之间可能出现较大的视点变化以及传感器退化或更换。

为应对这些挑战, 第 15.3.1 节先简要考虑其对状态估计的影响, 第 15.3.2 节介绍持久场景表示的离线方法; 第 15.3.3 节讨论在线 SLAM 的扩展以及统一短期、长期动态 SLAM; 第 15.3.4 节介绍时间场景理解模型, 其目标超越空间变化捕获, 重建时间模式。

15.3.1 用于定位的终身 SLAM

终身 SLAM 系统的核心能力, 是尽管存在上述挑战, 仍能准确定位机器人。这一问题在概念上与地点识别或回环检测非常相似, 后者要解决的是在存在变化时判断两个视图是否来自同一地点。本节将这些思想扩展到连续的长期定位。虽然已有多种方法, 这里只总结其中少数最相关的方法。

15.3.1.1 并行地图与摘要地图

第一类方法类似第 15.2.1 节的动态物体移除, 利用这样一个事实: 通常可以从匹配到的内点子集估计机器人位姿。但与将点作为离群值移除不同, 这里当特征外观变化到无法与先前观测匹配时, 会继续向地图添加额外特征点。通常将对场景的一次访问称为一次经历或会话; 在每次经历内部, 变化足够小, 可以跟踪机器人位姿^[219, 852]。如果当前经历能够相对于之前经历定位, 说明地图具有足够表达能力, 无需添加更多数据; 如果定位较弱或失败, 则将新数据加入地图。这样只有在需要时才添加新点, 地图复杂度也因此反映场景外观的变化^[219]。

通过加入跨经历约束, 还可以进一步改善相对于单一参考帧的定位性能。例如, 经历 $\mathcal{E}_A$ 可能无法充分相对于 $\mathcal{E}_C$ 定位, 但若二者都能定位到中间经历 $\mathcal{E}_B$, 则约束 $\mathcal{E}_C \rightarrow \mathcal{E}_B \rightarrow \mathcal{E}_A$ 可以提高地图一致性和定位精度^[296, 780, 852]。为解决经历过多导致地图过大, 以及移动物体产生的误导性特征仍留在地图中的问题, 有时会离线优化并摘要化地图: 在所有经历之间执行完整 BA, 只保留最可能有助于未来定位的特征^[780, 296]。

15.3.1.2 外观不变表示

近期工作不再针对每种外观分别建图, 而是关注特征检测、描述和匹配算法, 使同一特征能够在变化的外观之间匹配。这主要得益于深度学习进步: (i) 可以在覆盖多种条件的大规模数据集上训练这些方法; (ii) 可以考虑更多上下文, 例如整个图像, 而不是经典方法的像素邻域^[971, 35, 267]。

值得特别强调的一类外观不变特征是语义信息。例如, 如果能在不同视觉环境下识别某个物体是椅子, 那么“它是椅子”这一事实不会随时间变化。因此, 特定物体及其配置可以用于在巨大视觉变化(例如空中视点和地面视点)之间定位^[369]。

提取出这类外观不变特征和描述子后, 可以将其作为地标集成到 SLAM 流水线, 以支持地图优化和长期定位。虽然基于深度学习的方法性能出色, 但通常需要 GPU, 而并非所有机器人都配备 GPU; 其计算成本也可能高于经典特征。最后, 尽管这可以解决部分外观相关挑战, 物体移动或变化造成的离群匹配仍然需要识别和剔除。

15.3.1.3 内存、扩展与边缘化

对于长时间甚至无限期运行的机器人, 一个根本问题是机器人不断采集信息并加入地图, 最终导致计算机内存和处理时间受限。

遗忘。部分缓解方法是引入遗忘机制。通常为地图点设置随时间衰减的权重或持久概率, 使长时间未观测的点较不可能在当前定位中作为离群值造成破坏, 并最终从地图中剪枝^[936, 262]。另一种方案是运行地图摘要^[296, 780](通常作为离线过程), 仅保留最可能支持未来定位的特征。这种方法的优点是提供重要性排序, 可以在内存预算约束下只保留前 N 个点。机器人因而可以尽量在内存中保存信息, 只有内存不足时才开始遗忘。

内存管理。不进行遗忘的另一种选择，是动态地将地图特征从机器人内存中保存和加载。RTAB-Map^[616]就采用了这种方法：节点可以在短期内存、工作内存和长期内存之间移动，确保当前处理的节点数量足够少以满足实时计算。这避免了完全删除节点的需要，但会增加内存管理的计算开销，并且最终可能耗尽容量。

边缘化。除地图点数量外，基于位姿图的 SLAM 方法还要优化机器人位姿历史，其规模通常随经过时间线性增长。为避免节点和因子无限增长，边缘化过程旨在删除旧节点而不丢失信息。核心思想是用一个节点替换一组节点（或更一般的信息），该节点的属性反映或概括被替换节点中存储的信息。在概率模型中，这相当于被移除节点的边缘分布（更多细节见^[255]第 5.3 节）。这可以有效压缩图并减小问题规模，而不必完全丢弃节点。然而，实践中边缘化节点时仍会损失部分信息，最重要的是，边缘化节点的拓扑不能再改变。例如，如果某些因子（尤其是观测或回环检测）究竟是内点还是噪声存在不确定性，边缘化会将当前归类冻结到单个节点中；即使未来测量与之矛盾或能够消除歧义，也无法撤销。总之，边缘化过去测量可以在保留信息的同时有效减小问题规模，但会通过“固化”难以事后纠正的错误，降低优化灵活性。因此，何时执行边缘化仍是重要开放问题。

15.3.2 地图清理与变化检测

即使最终会遗忘点或物体，第 15.3.1 节介绍的基于检测方法仍往往会累积过时测量，例如已移动物体的旧观测。具体而言，检测器触发时，只能证明某个物体或特征存在，可以将其加入地图；但反过来并不成立。这体现为“无证据不等于证据表明不存在”的问题。设机器人访问房间，在角落观察到一把椅子（正检测）。之后机器人再次访问却未观察到椅子，这可能是由于机器人没有看向同一角落，椅子仍然存在（无证据）；也可能是机器人确实观察了角落并确定椅子消失（不存在的证据，或负检测）。另外，由于检测器并不完美，机器人即使看向角落也可能因光照变化等原因未识别椅子，此时判断椅子已不存在同样错误。

建立这种负观测或变化，正是变化检测的目标。历史上，该问题主要在离线或后处理设置中解决，因为目标通常是机器人采集数据后保留静态地图。本节将详细介绍这一设置，并在第 15.3.3 节讨论在线 SLAM 的近期扩展。

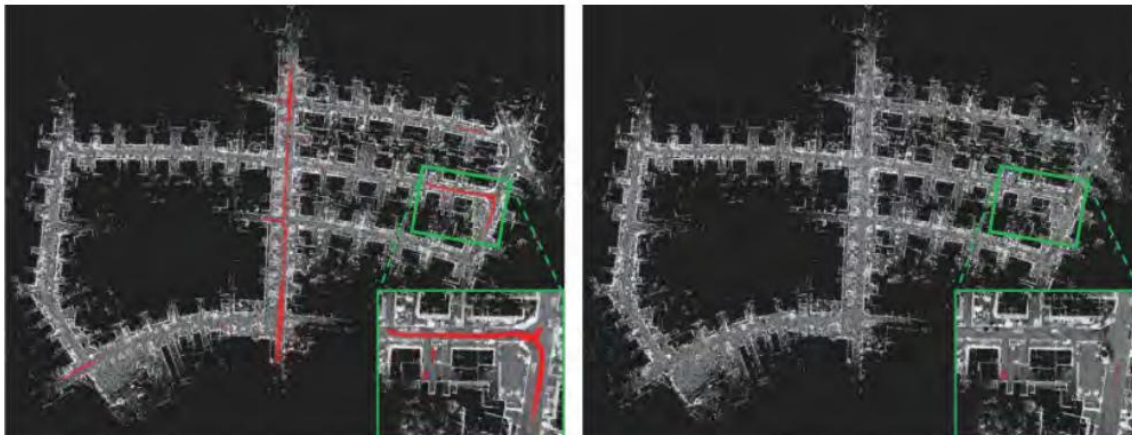


图 15.8: 图 15.10 ERASOR^[662] 的地图清理示例。动态物体测量产生的路径伪影（红色）大多被检测并从静态地图中移除。(©2021 IEEE)

15.3.2.1 地图清理

地图清理的目标是从地图中移除动态和伪测量，只保留场景高质量、静态且持久的特征。最常见的用途是在一次建图会话中滤除短期动态物体产生的伪点和观测^[564, 662]；同样技术也适用于长期和多会话建图^[882]。

从概念上看，其底层检测机制和原则与第 15.2.3.1 节讨论的稠密动态点检测非常相似。但离线处理可以利用更多资源。第一步通过鲁棒 BA 估计全局一致的机器人位姿，将动态点和伪点忽略为离群值^[564, 662]。固定传感器位姿后，重新检查所有测量以确保一致性并移除伪数据。与第 15.2.3.1 节类似，主要方法包括将所

有测量融合到场景体积地图中（通常通过向占据或 TSDF 体素栅格执行光线投射），在每个地图单元检查测量一致性^[973]。虽然该方法考虑全部数据，但可能消耗大量内存并需要很长处理时间。为避免这些问题，基于可见性的方法不构建全局融合地图，而是选择附近测量的子集，计算其中点的一致性，通常直接在传感数据上完成^[564, 662, 882]。

离线处理的显著优点是所有测量都可用于地图清理，还可以执行地面平面分割^[662]、逐步加入高置信度检测的迭代算法^[564]等额外步骤。图 15.10 给出了一个例子。

15.3.2.2 变化检测

历史上，变化检测一词在多个研究社群中使用。最突出的是，二维或基于图像的变化检测在计算机视觉中得到广泛研究，目标是识别同一场景甚至同一视点的两幅图像之间的差异，应用包括背景减除、监控和医学成像等^[900]。此外，基于卫星数据的图像变化检测也是遥感领域的活跃方向^[39]。

在机器人学中，检测图像之间的变化同样很有用。但在建图语境下，目标是根据全部观测估计场景底层三维结构，因此可以直接在子地图或会话层面于三维空间执行变化检测。当测量最终被边缘化或融合到地图中、可能不再可用时，这尤其有用。虽然随着高质量新视图合成发展，机器人图像变化检测重新受到关注^[706]，本节主要讨论三维和基于地图的变化检测方法。

需要注意，地图清理和变化检测的区别可能模糊，因为二者处理相关问题。最典型的情况是多会话建图：机器人早上扫描房间，晚上再次扫描；每次会话都能重建内部一致（可能已清理）的地图，目标是在场景层面识别和记录两次访问之间的变化。为此，通常需要详细场景表示并使用稠密重建方法。此外，有意义的变化表示往往需要对对象级推理，把对象视为一致变化的单元。本节重点关注物体（例如杯子或椅子）被加入、移除或移动的情况；但原则上也可以包含其他几何或非几何变化，例如坐垫变形、墙面重新粉刷或按钮被打开/关闭。

几何变化检测。和本章前文一样，可以使用多种线索对比测量，并将其用于变化检测^[565]。但要直接对比地图，占据图或 TSDF 图等体积表示已被证明最有用^[321, 951]。主要原因是，相比单纯的稠密表面表示，它们显式区分未观测空间和已观测为空闲空间，这对于消除“无证据”歧义至关重要。变化检测时，可以通过体积差分、地图差分或场景差分比较两个体积地图：原先观测为空闲的表面若重新出现，说明出现了新表面；原先存在、现在观测为空闲的表面则说明消失。最后，可以加入只在一个会话中观测到的区域以补全地图，并融合两次观测到的表面以提高重建精度。

为创建一致地图并避免误报，首先需要将所有子地图或会话准确配准到公共参考坐标系。由于每个会话内部可能存在里程计漂移或状态估计不准，仅刚性对齐各会话通常不够；最佳结果通常通过会话间约束和 BA 联合优化会话对齐获得^[321, 951, 565]。

语义信息。变化检测经常纳入语义信息，原因有多项。第一，物体语义标签具有外观不变性，因此可以作为强大抽象，对比不同条件下创建的子地图。语义并不能消除外观变化问题，只是将问题交给可在大规模数据集上训练的检测网络，或针对不同传感器和条件使用专用检测器。第二，语义可以提供有价值的非几何信息，例如从桌面移除一张纸不会改变几何表面，但在语义上可能容易检测。最后，变化检测的目标往往是将物体作为一致变化的整体。由于协调运动也是人类场景理解的重要方面，物体的这一概念与大多数封闭集语义标签高度重叠（杯子、椅子等对象往往作为整体移动）。为获得最佳性能，许多系统结合几何和语义信息^[625, 974]。

语义一致性。场景通常以协调单元发生变化，但机器人通常只能获得场景局部测量，这两点共同导致语义一致性问题。设机器人观察一张桌子；当天晚些时候返回时发现桌子消失，但视野的一半被遮挡。如果在该场景的观测或地图上执行变化检测，只会移除观测到的半张桌子，留下半张悬空的桌子；对人类而言，这显然不合理，因为我们强烈先验认为桌子通常作为整体移动。图 15.11 展示了这一问题。为避免这类伪影，可以遗忘旧地图、每次从头建图，这能保证语义一致性，但也阻止机器人逐渐构建更完整的场景模型。更常见的方法是在对象或语义类别层面执行变化检测，以防止形成此类伪影。最后，未来观测在发现变化后也可能移除伪影，但地图中存在伪影会降低整体保真度，妨碍规划和自主运行。



图 15.9: 图 15.11 语义一致性问题示意图。两次机器人观测之间, 场景中的沙发和灯被调换, 桌面上的物体也发生变化。左图显示融合所有测量得到的重建; 右图显示执行语义感知变化检测后的结果^[974]。由于第二次访问只观测到沙发的一部分, 左图仍有部分沙发残留, 而右图会移除整张沙发 (红色阴影)。此外, 左图融合了自由空间和沙发的冲突测量, 导致新位置的沙发只有部分重建。最后, 桌上的杂志等几何变化较小的物体未被识别, 而是与其他观测合并。(©2022 IEEE)

15.3.2.3 通过变化理解对象

从变化检测对象。变化与语义物体之间的内在联系可以双向推理。前文已说明语义信息可以改进变化检测; 反过来, 发生变化这一事实也可以作为物体检测线索。为此, 通常先在稠密重建中检测变化, 识别所有运动表面, 再将这些表面进行空间聚类, 以识别潜在物体^[321, 28, 327]。几何与语义的互补性在没有语义物体检测器, 或需要通过检测未正确分割的物体来补充语义分割时尤其有用。但与之前一样, 单独的几何物体分割容易受局部观测影响。例如, 两个相互接触的物体同时被移除时, 可能被欠分割为一个物体; 通过遮挡只观测到部分物体时, 则可能被过分割为多个部分。

物体实例重定位。为构建随时间变化、以物体为中心的场理解, 而不仅是估计当前或静态状态, 物体实例重定位的目标是通过长期动态观测跟踪物体。从概念上说, 该问题及其大多数解决方案与地点识别和回环检测类似, 但还有若干额外因素使其更具挑战。

第一, 长期动态具有突发性, 物体原则上可以在两次观测之间任意改变或移动很远。当然, 根据经过时间和假设的物体运动模型, 物体上次观测状态可能为当前观测关联提供有用线索^[951, 96], 但一般并不总是如此。

第二, 单个物体远小于场景, 其几何和纹理变化往往较少, 因此能唯一描述它的特征也少得多。对于小物体, 如果没有近距离观测, 可提取的测量会更少且分辨率更低, 从而进一步加剧这一挑战。

第三, 一个环境中可能存在多个相同或相似类型的物体实例, 因此感知混叠是重大挑战。这还引出一个更具哲学意味的问题: 什么才算物体的唯一实例, 是否应该单独跟踪它们。例如, 在有几十或几百把相同椅子的会议室或研讨室中, 很难判断哪把椅子是哪把, 以及它们随时间如何移动。不过, 对于大多数任务, 这种信息可能并不重要; 更好的做法可能是将所有椅子的观测融合为一个代表性模型, 再复制多次。另一方面, 办公室或建筑中可能有许多相同咖啡杯, 但人们可能非常在意追踪自己的杯子。应对感知混叠的一种方案是考虑解的唯一性: 高度独特物体的唯一匹配可以直接接受; 对于具有多个相似匹配候选的物体, 则采用多假设方法或适当的不确定性度量。

最后, 可以在封闭集或开放集设置下考虑物体实例重定位。前者假设空间中的物体集合固定, 重定位基本上成为最优匹配问题; 后者更一般, 允许物体被加入或移除场景, 同时引出“物体观测相似到什么程度才应视为同一物体”的问题, 因为总有一个新出现但相似物体的替代解释。

针对这些挑战, 主流方案主要关注描述子提取与匹配。一类方法在每个物体上提取关键点集合和描述子, 直接用于识别对应关系并求解六自由度变换。自然的第一步是直接使用 SLAM 前端的视觉特征^[748]。另一类

方法使用三维关键点和描述子，例如快速点特征直方图 (FPFH) [960] 和方向直方图签名 (SHOT) [965]，这些方法已广泛用于变化物体的重定位。其优点是可以从某会话的稠密三维重建中提取，并且对外观变化不敏感；但它们只表示局部几何块，描述能力较弱，通常需要进一步几何验证 [351]。近年来，基于深度学习的局部形状（有时也包含外观）描述子性能最佳，因为可以训练网络强调形状或物体中最具信息的部分 [1140]。图 15.12 给出了一个例子。

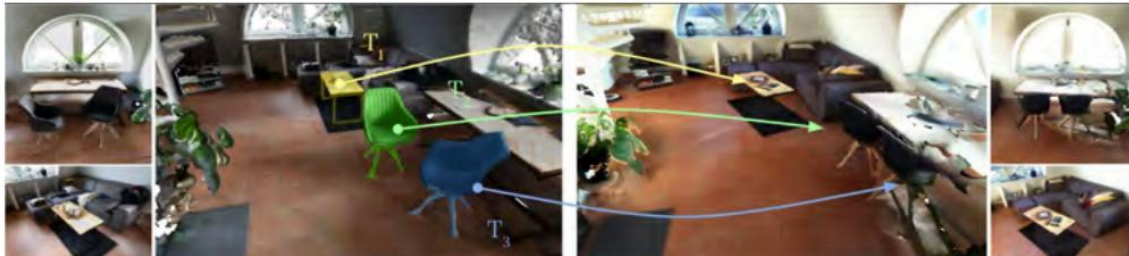


图 15.10: 图 15.12 RIO 数据集 [1140] 中的物体实例重定位任务。目标是确定初始扫描（左）和后续重新扫描（右）之间哪些物体实例相互对应，并估计每个物体与其新位置之间的六自由度位姿 T_1 、 T_2 和 T_3 。(©2019 IEEE)

这也催生了第二类方法：为每个物体计算一个描述子，而非多个关键点 [951, 1307]。当需要维护大型关键点数据库并穷举匹配时，这种方法更有优势，因为后者计算成本高昂。此类方法的一个重要方面，是构造或训练网络，使其对 $SE(3)$ 不变或对 $SE(3)$ 等变，以分别支持跨不同视点匹配和对匹配物体进行配准。

当变化物体的多个局部观测都完成重定位和配准后，不仅可以获得每个物体历史的有价值信息，还能随着时间推移重建更完整的物体模型和更丰富的场景描述 [351, 1307]。

15.3.3 变化感知 SLAM

多会话建图是长期动态观测的典型示例，但绝不是唯一设置。实践中，变化可能在更短时间尺度内发生。例如，机器人从客厅走到厨房取东西；如果客厅在此期间发生变化，机器人一周后返回还是几分钟后返回并无区别。极端情况下，机器人或传感器只需转过视线，再转回来时场景就可能已经改变。变化感知 SLAM 的目标，就是在机器人或系统在线运行时检测并表示这类长期变化。

挑战。与假设所有变化发生在会话之间的多会话建图相比，变化感知 SLAM 允许变化在任意两次观测之间发生。

因此，只能在短窗口内假设场景平稳，可用于融合和消除观测歧义的数据也更少。此外，不准确的相对定位与观测场景的变化在感知上可能相似，使两个问题难以区分。例如，如果机器人位姿估计偏移一米，先前地图物体的表面在当前偏移视图中可能看似缺失，尽管实际并非如此。变化检测与定位之间的相互作用（即为每个定位约束分类内点和离群值）还会因一个问题的解影响另一个问题而加剧。考虑两个极端解可清楚看出潜在问题：如果无视场景可能存在的大幅变化，将所有测量都视为未变的内点，则所得配准可能不准确，表示真实和错误测量的“平均值”；相反，如果将整个先前场景视为已变化且现在不存在，则当前开环估计会平凡地达到最优，但所得位姿估计和场景重建可能不完整且不一致。最后，变化检测必须连续运行，并基于机器人截至当前收集的局部数据。这要求变化检测以较高频率运行，以便机器人及时理解环境；理想情况下，随着更多数据收集，还应能纠正错误的内点/离群值变化决策。

对象级变化感知 SLAM。由于直接在低层传感器数据上全局推理变化场景很快变得难以处理，绝大多数变化感知 SLAM 方法首先在局部对测量进行摘要。由于大多数变化发生在刚性语义物体层面，对象级表示是直观选择 [974, 891, 345, 892, 976]。在这种设置中，提取对象级场景表示，并在每个物体层面进行定位和变化推理；相比对单个传感器测量进行推理，这一问题更易处理，而且从构造上保证了场景表示的语义一致性。

局部一致性。变化感知 SLAM 的一个重要考量，是区分噪声测量和场景真实变化，并据此决定如何将测量融合到地图中。例如，如果对每一帧原始数据运行变化检测，某些噪声点可能穿透已有物体并错误标记为变化；而若采用更保守的变化检测规则，则很容易产生假阴性，将实际上不属于同一物体的测量融合在一起，

形成损坏且不一致的重建（见图 15.11）。解决这一问题的核心思想，是使用局部一致性建立可保证无变化的测量序列。至少，当测量在帧间被跟踪时可以做到这一点^[974, 1130]，直观上对应于机器人观测期间场景不会在无人察觉时变化。实践中，基于对场景变化速率的合理假设以及短时间内较低的里程计误差，还可以将这一思想扩展到滑动窗口^[345]或活动窗口^[976]。

变化感知建图。结合上述思想，Panoptic Multi-TSDFs^[974]提出了首个实时变化感知建图系统。在给定全局一致位姿的假设下，Panoptic Multi-TSDFs 前端只更新能够逐帧跟踪、且测量语义与目标子地图匹配的子地图（称为活动子地图），从而估计一组语义一致且局部一致的子地图。通过对比活动子地图与重叠的非活动子地图，可以在这一更高层次的子地图抽象上高效检测变化；如果先前子地图分别与当前子地图不一致或一致，则可以丢弃或合并。这样既保留了与当前地图匹配的历史子地图信息，又通过移除完整子地图从构造上保证语义一致性。随后不久提出的 POCD^[891]采用另一种方法：也将输入帧分割为物体，但随后相对于当前地图进行跟踪和配准。该方法能够容忍并校正一定程度的里程计噪声。为处理噪声变化观测，它为每个物体建立概率持久模型，并在观测到物体或未能配准本应观测到的物体时更新该模型。

全局一致的变化感知 SLAM。在大规模场景且没有全局位姿时，需要与场景变化联合估计位姿。对于对象级表示，可以直接使用基于因子图的方法，将局部一致的物体观测作为地标。为处理变化检测中的数据关联问题，NeuSE^[345]学习高度描述性且对 SE(3) 等变的神经物体表示。这些潜在描述子从局部观测捕获完整形状，允许直接关联和配准物体实例。因此，这类关联可以作为回环约束，空间不一致或缺失的关联则可作为变化信号。POV-SLAM^[892]采用期望最大化（EM）算法，在物体观测关联条件下优化机器人和物体位姿，并在给定场景空间布局的情况下优化物体观测和持久概率；这样可以持续细化定位和变化检测，尤其是在未来获得新观测时。Khronos^[976]则在相近物体之间创建候选关联因子，并使用基于逐步非凸性的鲁棒因子图优化^[1222]确定内点关联。此外，将（假设静态的）背景变形纳入额外因子以辅助空间优化，并在回环后执行可变形几何变化检测，以解决“证据表明不存在”与“无证据”之间的歧义。

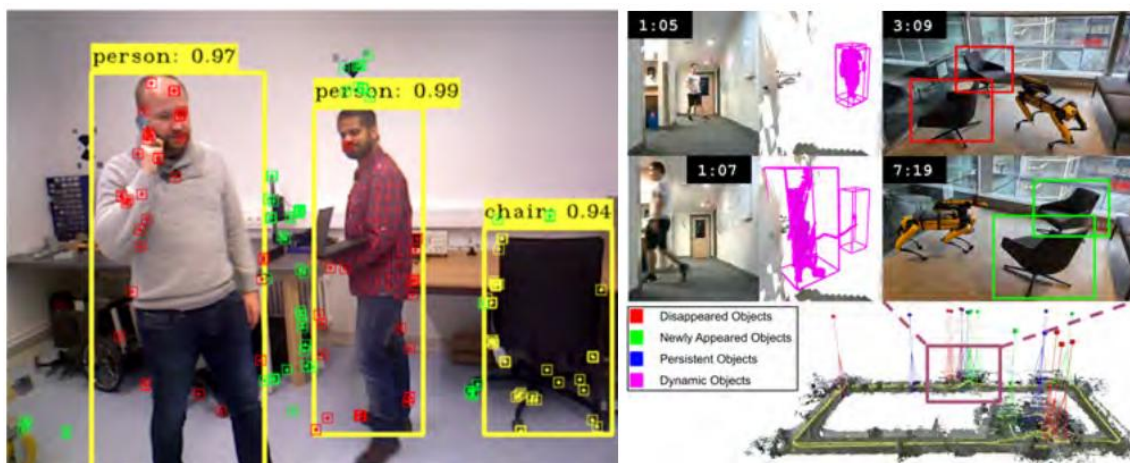


图 15.11: 图 15.13 统一短期和长期动态 SLAM。Changing-SLAM^[1130]（左，©2023 Springer）跟踪短期动态物体的关键点（红色）和背景关键点（绿色），并关联潜在长期动态物体（黄色）。Khronos^[976]（右）则在在线运行中稠密检测和重建短期运动（紫色）及长期变化（移除和添加分别以红色/绿色表示）。

统一短期和长期动态 SLAM。最后，可以利用局部一致性，将短期动态物体跟踪与变化感知方法结合，同时捕获多种动态模式，如图 15.13 所示。Changing-SLAM^[1130]给出了首个此类表述。Changing-SLAM 构建在 ORB-SLAM^[787]之上，采用稀疏 SLAM 表述；每帧中，利用语义掩膜将特征点分组到物体。随后使用 Kalman 滤波器逐帧跟踪这些点，以捕获短期运动。未被跟踪的物体点（长期变化）则在全局范围内与附近、同语义类别的物体关联。最后，为每个物体估计持久分数，使长时间未检测到的物体被遗忘并从地图中移除。

Khronos^[976]提出了首个稠密时空度量-语义 SLAM 方法。Khronos 采用类似思想，使用体积局部地图

重建背景以及跟踪到的静态、运动物体。在回环后优化全局一致位姿，再通过额外变化检测步骤估计每个物体发生变化的时间，从而恢复场景状态的历史和演化。

15.3.4 时间场景理解

建图时，目标是构建场景的数字表示和理解。如前文所述，建模动态环境时，所需理解层次可能从构建场景持久部分的模型（不包含运动和变化），到构建详细的四维重建，并理解何时、如何发生了哪些运动和变化。然而，这种理解还可以超越捕获单个运动和变化实体，进一步估计底层时间变化模式并预测场景未来演化。本节简要概述三类关键时间模型。

动态规律地图。动态地图关注检测和跟踪运动物体；动态规律地图（Maps of Dynamics, MoD）则将这些轨迹或其他运动信息作为输入，估计场景中的典型运动模式^[610]。模式可以是恒定的（例如单向道路上的主导运动方向），也可以依赖时间（例如人们早晨进入办公室、晚上离开）。理解这些模式可以为动态场景导航或人类运动预测提供有价值信息。

周期事件。周期或循环事件是一类特殊但常见的动态模式，例如场景每日或每年的变化。如果假设场景状态由某个周期性隐藏过程控制，就可以将场景观测视为该周期过程的观测，并用于估计其属性。Krajník 等人^[603] 首创了频率地图：在时间域分析观测的不同周期过程，在频域通过傅里叶变换进行谱分析。只存储频谱中的主导模式，就能得到紧凑的地图表示，捕获频率各异（周期可以任意长）的周期事件。由于场景的完整四维模型由这些模式定义，一旦从历史观测中估计出模式，就可以预测场景所有未来状态，从而改善未来定位和路径规划^[602]。

基于学习的方法。除结构化或手工设计的动态统计模型外，还可以使用基于学习的方法预测场景未来变化。通常利用场景图作为物体及其与周围环境关系的紧凑表示，预测对象级变化或运动^[851, 693, 393]。基于学习的方法具有多项优势，包括更灵活地表示复杂动态模式，并能将语义先验（例如“午餐时间前后餐桌容易发生变化”）与特定环境观测相结合。相比更静态的共现先验或纯频率模型^[851, 393]，这可以带来更好的预测性能，改善机器人未来的主动和前瞻性行为^[851, 693]。缺点是，复杂模型可能需要更多训练数据，而变化数据相对于场景静态观测可能稀少且分布不均。

15.4 可变形 SLAM

除刚性运动和变化物体外，机器人经常运行在不仅具有动态性、还包含可变形实体的环境中。如果场景只有部分区域发生变形且需要建模，通常相对于第 15.2.3 节介绍的静态背景估计机器人运动，将问题简化为已知传感器位姿条件下的可变形重建。该问题可作为非刚性运动恢复结构（NRSfM）处理，第 15.4.1 节将介绍。

更一般地，可变形 SLAM（或可变形环境中的 SLAM）指在没有刚性背景的环境中执行 SLAM，即场景中没有任何可见的刚性或静态部分。一个特别相关的案例是微创机器人手术：唯一可用传感器是单目相机，只能观测人体内部的可变形器官^[893]。此时需要估计相机相对于持续变形场景的自运动。由于没有静态部分，可变形 SLAM 问题严重欠约束，比静态和动态 SLAM 更具挑战。

下文首先在第 15.4.1 节讨论可变形 SLAM 与 NRSfM 的差异；然后在第 15.4.2 节介绍可变形 SLAM 中存在深度信息（例如来自 RGB-D 或双目相机）的较简单情况；第 15.4.3 节介绍单目相机这一更具挑战的情况下的可变形 SLAM 流水线；最后第 15.4.4 节介绍地图初始化和扩展。

15.4.1 NRSfM 与可变形 SLAM

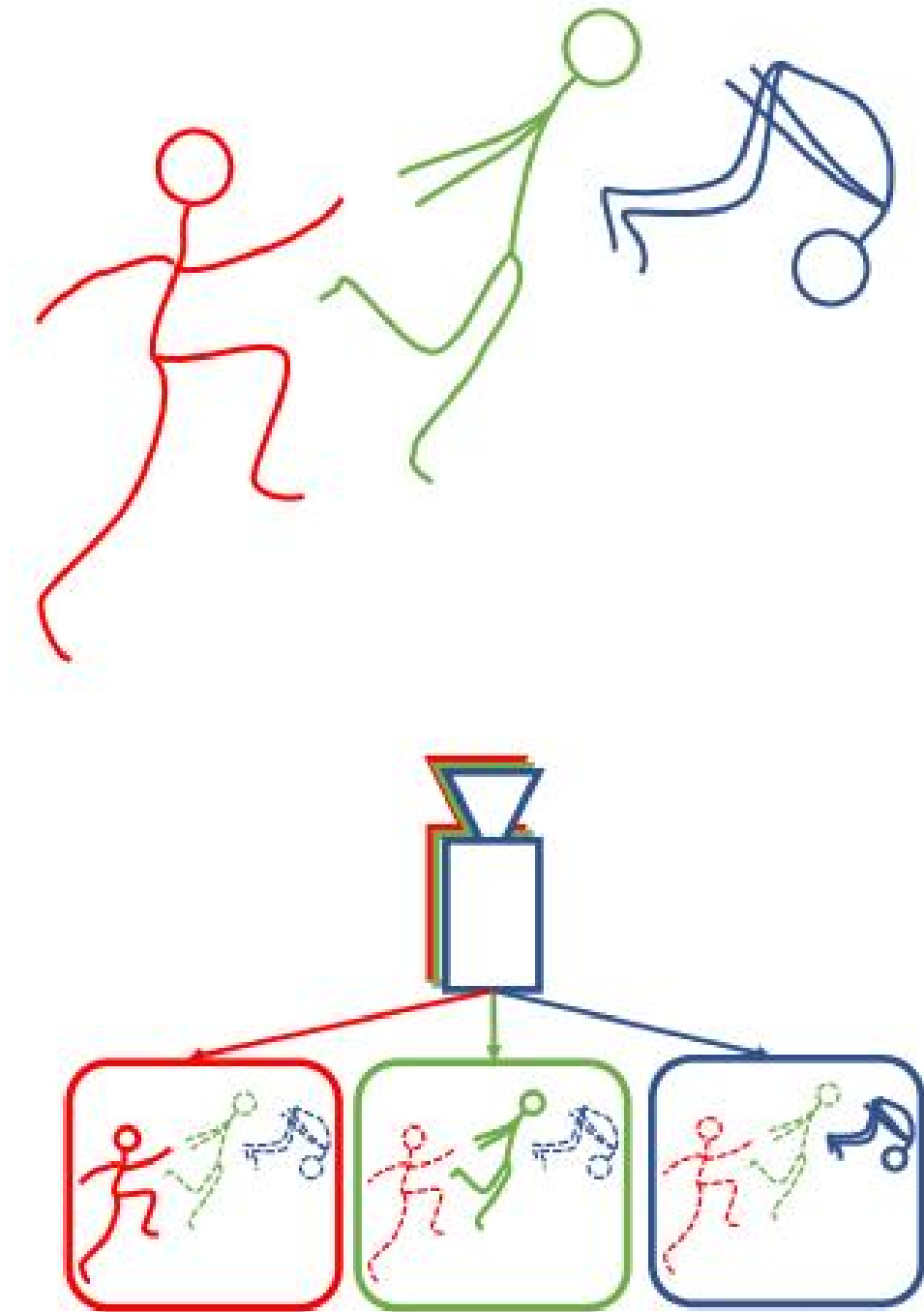
正如刚性视觉 SLAM 与 SfM 相联系，非刚性 SLAM 也与其非刚性对应问题 NRSfM 有密切联系。

15.4.1.1 非刚性运动恢复结构（NRSfM）

NRSfM 是计算机视觉中的经典主题，研究如何使用单目视觉建模变形三维环境，为从视频序列进行变形场景三维建模提供理论基础。场景中的每个三维点在每一帧都有不同的三维位置，因此地图表示的不是单个

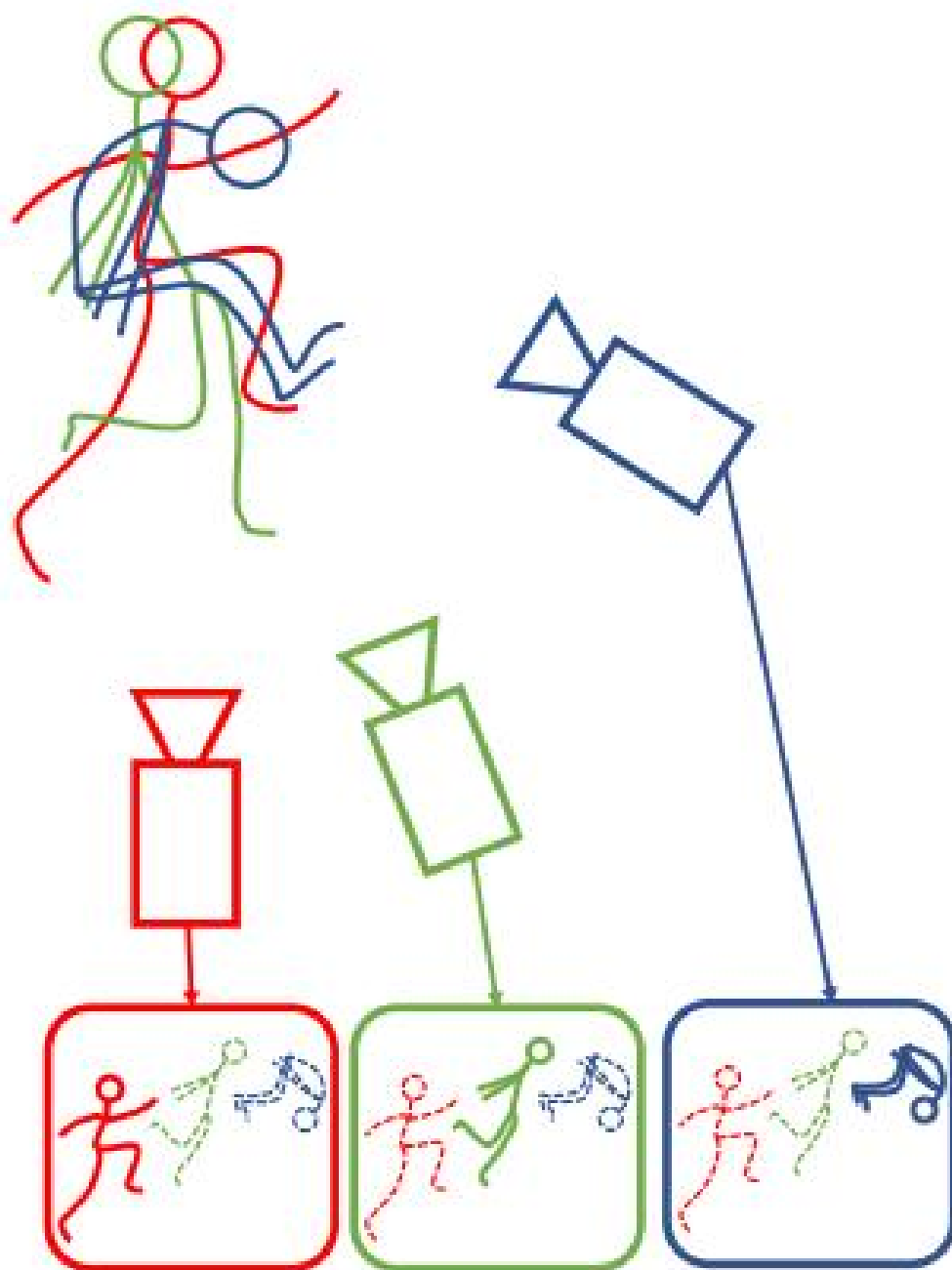
位置，而是每个地图点的一条三维轨迹。结果是，单目问题严重欠约束，需要额外先验约束解。常用选择包括时间平滑先验（假设三维点轨迹随时间平滑）和空间先验（假设一个点的变形与邻域点相似）。

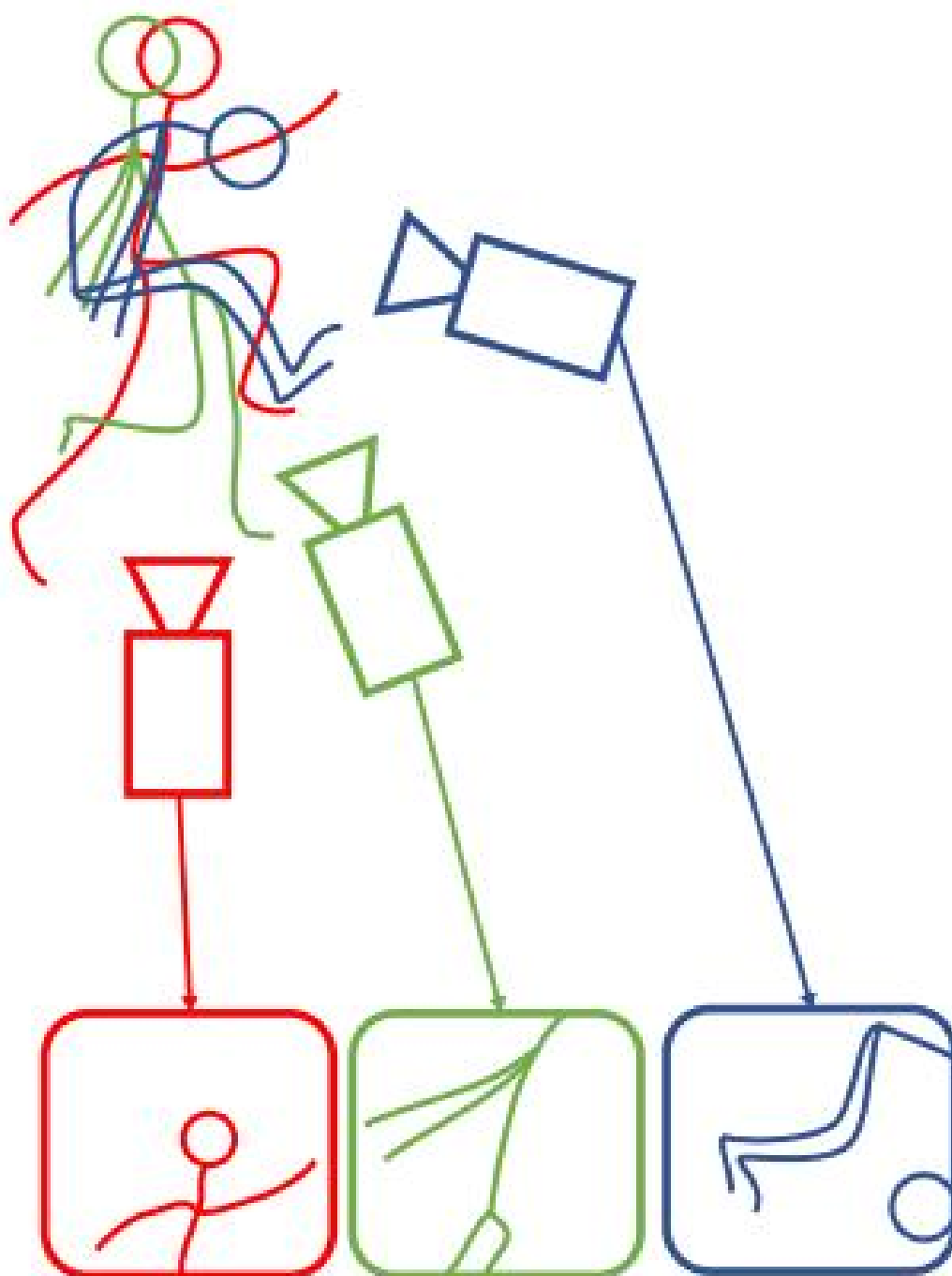
经典 NRSfM 表述假设静态单目相机观测没有静态背景的运动物体。大多数视频帧观测到完整物体范围，即大部分图像在观测同一变形物体时相互重叠。典型例子包括相机前运动的手、飘动的旗帜或跳跃的人。需要注意，即使相机固定，每个点的运动通常也包括物体的显著刚体变换及其局部变形。刚体运动不会被显式计算，而是作为每个点相对于固定相机的非刚性场景运动的一部分（图 15.14(a)）。尽管不计算刚体运动，它会产生显著视差，使估计问题条件良好。



(a)

(b)





(c) 图 15.14 三帧中 NRSfM 与可变形 SLAM 的示意图。场景没有刚性背景。(a) 经典 NRSfM 假设单目相机固定，而三维物体发生包含刚体变换和局部变形的非刚性运动。(b) 可变形 SLAM 假设场景物体主要发生非刚性运动，而运动相机在没有静态背景的情况下关注刚性运动。(c) 可变形 SLAM 还通常假设相机处于近距离视点，只能观测变形场景的一部分，即相机沿探索轨迹运动。

从模板恢复形状。第一类 NRSfM 方法是 Shape from Template (SfT)。它们假设有一个作为先验的、带纹理的观测物体静止三维形状，然后利用该先验恢复变形，从而显著简化问题。这一带纹理的物体静止三维形状称为模板。方法依赖模板的变形模型。一方面，解析等距变形模型假设表面点之间的测地距离保持不变；这一假设适用，并迅速发展出稳定、实时的 SfT 方法 [227, 59, 198]。另一方面，能量方法 [966, 21, 804, 619, 422] 联合最小化相对于静止形状的变形能量和图像对应关系的重投影误差。这些方法还被嵌入带鲁棒核的序列数据关联，以检测和剔除离群值 [619, 620, 422]。

非刚性运动恢复结构 (NRSfM)。为处理没有 SfT 强先验的完整 NRSfM 问题，早期方法假设正交相机模型，但该模型只适用于远离相机的运动物体。最早的方法见 [113]，其奠基性工作催生了基于低维基的方法，用于从序列帧计算三维轨迹 [837, 775, 239]。这些方法分别使用空间 [239, 368]、时间 [23]、时空 [20, 394, 395] 或较新的拓扑先验 [989] 作为正则化器。

可变形 SLAM 典型的近距离序列（如医学内窥镜）必须使用透视相机模型。SfT 中首先提出的等距假设在 NRSfM 中也取得了出色结果 [1076, 1127, 196, 197, 843, 844]。其中，等距方法 [843] 对 SLAM 尤其有用，它是一种能够自然处理可变形 SLAM 中常见遮挡和缺失数据的局部方法。在 SfT 和 NRSfM 之间，Agudo 等人 [22] 提出基于 Navier 方程和有限元法 (FEM) 的变形模型，并以 EKF-SLAM 的序列方式处理视频帧。

15.4.1.2 可变形 SLAM

相比之下，可变形 SLAM 研究上述问题的一种变体：场景发生变形，相机也在运动。目标是同时估计观测场景的变形历史和相机轨迹。假设观测到的变形中凡是可以由刚体变换解释的部分，都纳入相机轨迹（图 15.14(b)）。将运动中的刚性和非刚性成分分离是一个不适定问题，所谓“浮动地图歧义” [621] 突出体现了这一点。但可以通过对可变形物体的位移引入先验，实现这种分离。

SLAM 的一个重要挑战是相机通常沿探索轨迹运动，即并非所有帧都观测相同的场景区域（图 15.14(c)）。相机还经常靠近变形表面，此时正交相机假设不再成立。图 15.15 展示了观测变形曼陀罗序列中的典型近距离视图 [620]。

最后一个常见假设是过程必须具有因果性并实时运行，即要在时刻 k 生成地图和相机位姿，只能处理第 1 到第 k 帧。这与通常批量处理全部图像的 NRSfM 形成鲜明对比。

鉴于涉及的困难，第 15.4.2 节先讨论带深度信息的可变形 SLAM，随后在第 15.4.3 节回到使用单目相机的可变形 SLAM。

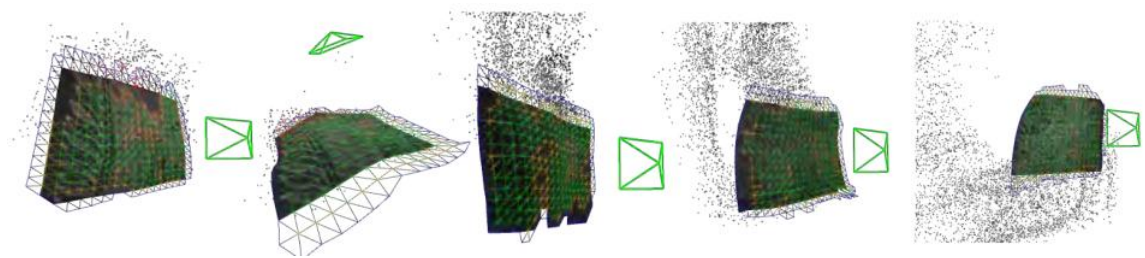


图 15.12: 图 15.15 可变形 SLAM 中的典型探索轨迹 [620]。相机位姿（绿色视锥）与变形场景的局部可变形模板（网格）联合估计。全局地图点以黑色显示。（©2021 IEEE）

15.4.2 带深度信息的可变形 SLAM

RGB-D 或双目相机等三维传感器能够提供深度测量，使估计问题变为超约束，相较纯单目方法大幅降低复杂度。DynamicFusion [802] 是首个基于 RGB-D 数据的可变形三维 SLAM 工作。它构建场景的规范地图（即

静止状态下的形状), 并将其变形以解释当前深度观测, 将嵌入变形图模型 (ED) ^[1049] 与尽可能刚性 (as-rigid-as-possible) 正则化器 ^[1025] 结合。ED 模型在图结构中离散化变形空间, 从而加速稠密重建。DynamicFusion 奠定了后续工作的基础, 例如 VolumeDeform ^[504] 纳入光度信息以改善结果; 随后 KillingFusion ^[1016] 提出强制变形平滑且近似等距, 图 15.16 给出了示例。上述方法都使用 SDF 表示地图, 但其随地图规模扩展性较差, 限制了探索应用。Surfelwarp ^[364] 提出使用 surfel 表示替代经典 SDF, 提高可扩展性。在医疗领域, MIS-SLAM ^[1023] 提出适用于双目系统的可变形 SLAM, 将尽可能刚性变形与 ED 模型结合。在双目医疗应用中, Zhou 和 Jayender ^[1294] 将期望最大化与对偶四元数 (EMDQ) 算法同 SURF 特征结合, 以跟踪相机运动并估计视频帧之间的组织变形。

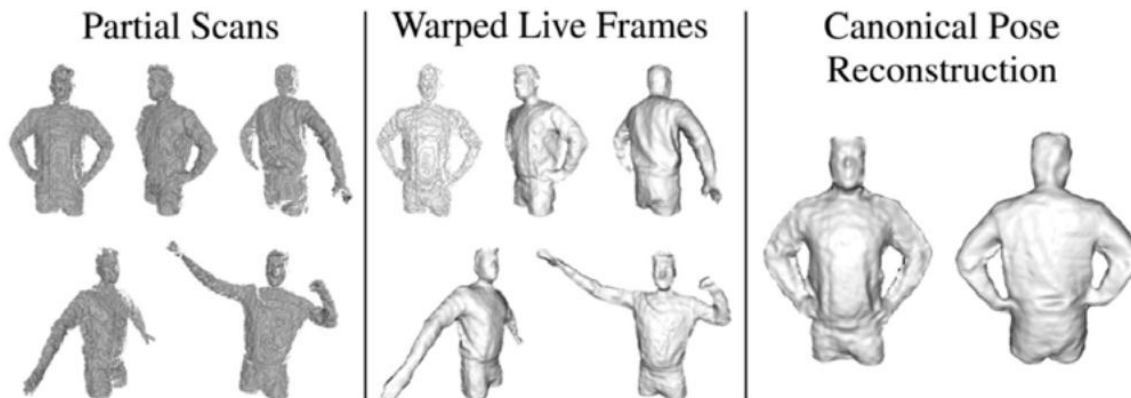


图 15.13: 图 15.16 KillingFusion ^[1016] 从运动 RGB-D 相机观测中恢复非刚性运动物体。通过使用有符号距离函数 (SDF) 和 killing 正则化器, 它能够恢复发生拓扑变化的结构, 例如双手与躯干融合。(©2017 IEEE)

15.4.3 使用单目相机的可变形 SLAM 流水线

典型可变形 SLAM 系统的流水线模仿视觉 SLAM 中以帧频率跟踪、以关键帧频率建图的方式, 但针对变形情形进行了调整, 例如 DefSLAM ^[620]。

跟踪。可变形跟踪类似 SfT 方法。假设探索局部区域的静止形状可用, 如 ^[619] 首次提出的方案, 并使用受基于物理的弹性模型启发的变形能量。随后, Gómez Rodríguez 等人 ^[422] 提出可变形跟踪系统, 使用空间、尽可能刚性以及黏性时间正则化器。这些方法分别集成到完整可变形 SLAM 系统 DefSLAM ^[620] 和 NR-SLAM ^[933] 中。

建图。可变形建图线程以序列中的点跟踪为输入, 负责以关键帧频率重新计算模板。首个可变形 SLAM 系统 DefSLAM ^[620] 使用等距 NRSfM ^[843] 完成该计算。NRSfM 处理一组共视关键帧, 为每个地图关键帧生成一个模板; 在假设场景具有平面拓扑的条件下, 将这些模板与早期局部建图操作中计算的前一批模板对齐。为放宽平面拓扑假设, NR-SLAM ^[933] 引入动态可变形图, 以处理任意场景拓扑。在变形模型方面, 作者比 ^[422] 更进一步, 提出直观的黏弹性变形模型, 同时包含时间和空间正则化。该模型用于局部建图可变形 BA, 构成可变形建图核心, 替代 DefSLAM 中的等距 NRSfM。

特征匹配。DefSLAM 继承 ORB-SLAM ^[787] 的 ORB 特征 ^[953]。但研究表明, 在医学场景的近帧跟踪中, ORB 描述子并不理想; 由于 Lukas-Kanade (LK) 光流 ^[707] 对仿射光照变化不变, 具有更好性能 ^[391]。此外, 作者将 ORB 描述子与 DBoW 词袋 ^[362] 结合进行地点识别, 以在跟踪丢失后实现相机重定位。NR-SLAM ^[933] 的特征跟踪同样使用 LK 跟踪。

15.4.4 初始化与地图扩展

从零初始化始终是所有单目视觉 SLAM 系统的挑战步骤, 在可变形情况下尤为明显。DefSLAM ^[620, 391] 依赖等距 NRSfM 算法 ^[843], 可以从零计算初始关键帧的模板。唯一问题是如何计算关键帧之间的初始匹配,

该问题通过假设场景为平面来解决。对于 NR-SLAM^[933]，可变形 BA 需要初始猜测；初始地图和相机运动使用刚性 SfM 计算。通过分割初始关键帧之间的图像光流，初始化动态变形图。

当相机探索新区域时，需要向地图中加入新点，使其覆盖新区域。对于 DefSLAM，只要能在关键帧中找到这些新点的匹配，等距 NR-SfM 就能自然处理地图扩展。对于 NR-SLAM，则利用已有相机位姿，从新点与已定位帧中的匹配对三角化新地图点。为此，提出模型选择方法：分别使用刚性和非刚性算法对地图点三角化，再为新探索区域选择正确模型。

15.5 延伸阅读与近期趋势

动态、变化和变形场景中的 SLAM 是一个高度活跃、持续发展的研究方向。本章旨在概览该领域的关键挑战和解决方案，但无法涵盖所有正在进行的工作，希望这些内容能促使读者深入阅读相关主题，并在未来参与研究。尽管已经取得显著进展，仍有许多开放问题。下面强调其中一些问题及其他有前景的未来方向。

极端环境下的状态估计。即使不显式建模环境变化和动态，现代 SLAM 系统也已对其具有相当鲁棒性。然而，在存在大面积遮挡^[691, 450]、运动物体占比很高^[897]或强运动先验^[450, 76]时，将动态物体跟踪纳入状态估计可以提高鲁棒性和精度。但这些优势会增加计算成本，当前每种挑战通常分别处理。能够高效且鲁棒地应对多种极端条件（或“边界情形”）的 SLAM 方案仍是开放问题。

可微场景表示。第 14 章介绍的 NeRF 和 3DGS 等可微渲染模型虽然仍较新，但已经显示出作为动态场景表示的巨大潜力。早期工作展示了它们对动态、变形物体进行高保真重建^[1192]以及变化检测^[706]的能力。不过，该领域仍面临实时 SLAM 的内存和计算成本、移动传感器产生的局部且带噪观测，以及实现空间和时间一致地图的规模扩展与地图管理等开放挑战。

数据关联与深度学习先验。数据关联仍是根本且困难的问题，尤其是局部观测、遮挡，以及长期地点识别和实例重定位。为克服这些问题，研究者似乎正转向纳入先验的深度学习方法，从局部观测补全并关联物体^[345, 1307, 1207]。特别是提取 SE(3) 不变或 SE(3) 等变特征的几何深度学习技术，有望解决这些挑战。但在移动部署的泛化能力、可靠性和可扩展性方面，仍有开放问题。

终身运行与持续学习。终身运行 SLAM 方法的可扩展性和性能是另一个开放问题。这涉及从外观到几何变化的一系列感知挑战，以及传感器随时间可能发生的退化。此外，如果机器人能够长时间运行，也应能够持续学习^[1134]，随时间适应并改进。最后，这两个方向都需要某种形式的内存管理，以决定保留哪些信息、如何摘要或边缘化，以及何时遗忘什么信息。

可变形 SLAM。可变形环境中的 SLAM 研究仍处于早期阶段，存在许多开放问题^[492]。例如，在什么条件下可以同时准确估计相机轨迹和地图变形？能否清晰区分可变形物体的刚体运动与相机的刚体运动？能否准确估计可变形 SLAM 结果的不确定性？可变形 SLAM 中是否存在局部极小值？如何判断得到的结果是否为全局极小值？

迈向时空智能。除动态和可变形场景中的 SLAM 外，时空智能旨在构建超越定位和重建的整体场景理解，同时捕获并学习时间模式。

理想情况下，这种地图或“世界模型”将机器人语义先验与观测相结合，提供对场景及其动态的深入、机制化理解。它能够估计一致且完整的过去历史，预测未来结果，并促进高层次、长期的推理和决策。

第十六章 度量-语义 SLAM

作者: Arash Asgharivaskasi、Kevin Doherty、Jens Behley、Nathan Hughes、Yun Chang、John Leonard、Henrik I. Christensen、Luca Carlone 和 Nikolay Atanasov

本章讨论如何将 SLAM 方法生成的几何地图表示增强为语义信息更丰富的表示，以支持更广泛的下游任务，同时提升 SLAM 性能。SLAM 对移动机器人自主性至关重要，因为它使机器人能够在先验未知环境中定位自身并维护一致的地图表示。正如第 1 章所述，SLAM 的另一个同等重要的作用，是通过提供任务目标、运动目标及其约束，为机器人任务规划和运动规划提供信息。稠密地图表示（如第 5 章回顾的表示）能够高效编码机器人导航和操作中的碰撞检查安全约束。例如，可以利用占据表示^[94, 170]、有符号距离函数^[822, 1277]和网格表示^[888, 112]支持运动规划和轨迹优化。

然而，当机器人任务包含更复杂的语义目标和要求（例如“导航到办公室中的笔记本电脑”，或“会议室内有人时不要进入”）时，仅靠环境的几何信息可能并不足够。在机器人导航中，可以利用环境语义（例如地图的一部分是道路、人行道还是植被）指定不同的可通行代价^[910, 666]。更一般地说，SLAM 技术生成的地图可以通过提供关于物体、地点及其关系的语义信息来支持机器人任务描述和规划。这些信息不仅对下游任务有用，对 SLAM 系统本身也常常有用：具有语义意义的特征更容易唯一识别，并且对视点变化更不敏感，从而改善数据关联和回环闭合^[108, 710]。类似地，语义信息有助于处理场景中的动态实体^[943]，因为动态实体通常不利于运动估计（参见第 15 章）。

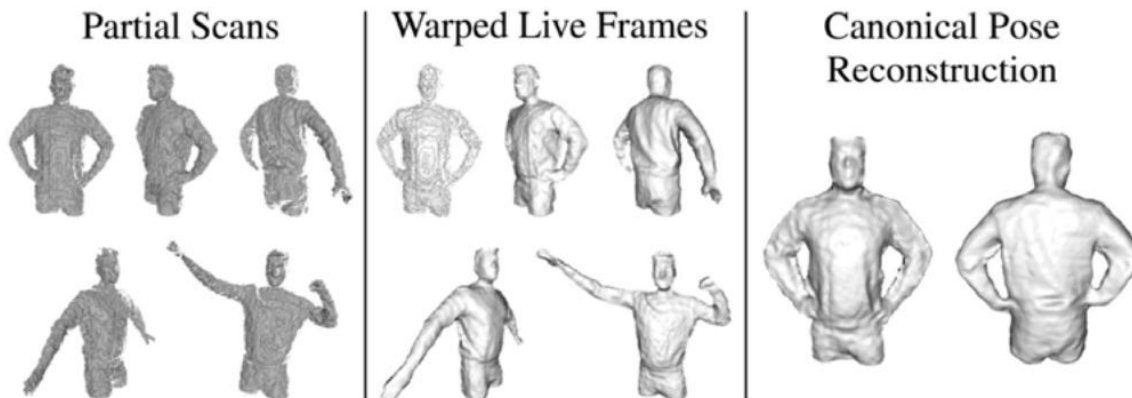
构建语义丰富的地图表示，需要从视觉观测中提取超越基于图像梯度关键点的附加信息。深度学习的发展使我们能够从视觉观测中获取物体检测、物体关键点、语义边缘、语义分割、实例分割、全景分割以及其他语义信息。物体检测是计算机视觉任务，通常通过边界框标注识别并定位图像中的物体。语义关键点^[854]或语义边缘^[1257]可以作为物体的中层部件被识别（例如门、车轮和汽车挡风玻璃）。语义分割^[444, 934]超越了物体检测，为图像中的每个像素分配类别标签。实例分割还会识别并分离属于同一类别的不同物体，通过逐像素掩膜区分每个实例。全景分割^[580]将语义分割与实例分割结合起来，同时分割背景和物体实例，以实现对图像的全面理解。

本章介绍传统 SLAM 的不同扩展：利用视觉前端提供的语义信息，并将后端优化的地图推广为能够累积和融合这些信息的三维空间度量-语义表示。第 16.1 节先给出快速概览；第 16.2 节介绍如何将路标语义信息加入基于路标的 SLAM；第 16.3 节描述第 5 章稠密表示的扩展，以纳入语义信息；第 16.4 节讨论在不同抽象层次捕获语义的分层地图表示。最后，在第 16.5 节讨论近期趋势。本章聚焦于“封闭集”语义，即语义信息限于相对较小（例如 100–1000 个标签）且预定义的语义概念字典（例如“椅子”“桌子”“办公室”）。关于将开放集语义（即语言嵌入）纳入 SLAM 的讨论留到第 17 章。

16.1 从传统 SLAM 到度量-语义 SLAM

将语义信息纳入地图表示有多种方式。例如，可以在基于路标的地图中为路标添加语义，以包含稀疏语义信息；可以在基于稠密表面的地图中嵌入语义标签；甚至可以为自由空间的整个区域赋予语义（例如室内的房间，或室外更一般的区域，如停车场）。对于稀疏地图，需要将通常的三维点路标推广为能够表示物体类别以及可能的位姿和形状的路标。相应的传感器测量模型也需要从低层视觉关键点和特征扩展到边界框^[920]、分割掩膜^[444]或物体部件^[854]等高层检测。这会导出能够同时捕获度量信息和语义信息的新型概率因子及因子图优化技术。

对于稠密地图，可以将占据模型从表示自由空间与占据空间的二值模型扩展为表示语义类别的多类别模



型。更一般地说，前端计算机视觉模型提取的语义特征可以与稠密地图中维护的点、surfel、网格面或体素建立对应关系。除将语义观测变换到三维空间外，还需要为语义信息构造新的观测模型，并采用序贯概率推断技术更新语义地图信息。正如第 16.4 节将看到的，分层建图方法通常将稀疏和稠密的度量-语义地图组合为统一表示，在多个抽象层次描述语义，从稠密的表面地图到物体和区域。

16.2 稀疏度量-语义表示

考虑一个基于路标的 SLAM 问题，其中路标表示环境中的物理物体。我们将这一设置称为稀疏度量-语义 SLAM，因为地图由带有位置、方向和形状等度量属性，以及物体类别等语义属性的物体路标组成。如第 1 章所述，基于路标的 SLAM 通常表述为非线性最小二乘问题，路标观测在目标函数中产生平方项 $r_i(\mathbf{x}_i)^2$ ，残差为

$$r_i(\mathbf{x}_i) = \|\mathbf{z}_i - h_i(\mathbf{T}_i, \ell_i)\|_{\Sigma_i}. \quad (16.1)$$

其中 $\mathbf{z}_i$ 是测量，状态变量 $\mathbf{x}_i = (\mathbf{T}_i, \ell_i)$ 包含机器人位姿 $\mathbf{T}_i$ 和路标状态 ℓ_i 。与前面各章类似，索引 i 指由成对的测量 $\mathbf{z}_i$ 和状态 $\mathbf{x}_i$ 在数据关联已知时引入因子图的因子。在视觉 SLAM（第 7 章）中，路标 ℓ_i 最常表示为三维点，相应测量 $\mathbf{z}_i$ 是路标投影到图像平面的二维像素坐标。在稀疏度量-语义 SLAM 中，路标被推广为能够描述物体及其属性的模型。测量和残差误差也被推广为表示计算机视觉算法产生的物体检测，并度量三维空间中的物体投影到二维图像平面时的误差。下面讨论物体表示、物体测量以及物体残差的定义。

16.2.1 物体表示与因子图建模

我们先将路标从三维点扩展为刚体物体。物体路标不仅由位置决定，还由方向、尺度、形状和语义类别决定。

定义 16.1 物体路标是元组 $\ell = (\sigma, \mathbf{q}, \lambda, \mathbf{s})$ ，其中语义类别 $\sigma \in \mathbb{N}$ （例如“汽车”“椅子”“桌子”），位姿 $\mathbf{q} \in \text{SE}(3)$ ，尺度 $\lambda \in \mathbb{R}_{>0}$ ，形状 $\mathbf{s} \in \mathbb{R}^d$ 。

物体路标 ℓ 的位姿 $\mathbf{q}$ 指定物体坐标系相对于地图世界坐标系的位置和方向。除刚体变换外，物体局部坐标系的定义还需要包含尺度，以允许同一类别、同一形状但尺寸不同的物体存在（例如汽车与玩具汽车）。下面介绍几种物体形状描述，并通过多个示例说明物体路标表示。与 SLAM 中的稠密表面建模（第 5 章）类似，物体形状 $\mathbf{s}$ 可以使用显式表面模型（例如点、基本几何图形或网格）或隐式表面模型（例如占据函数或有符号距离函数场）表示。下面给出物体 SLAM 中常用的物体形状表示。

16.2.1.1 将物体形状表示为语义路标

给定类别的物体形状（定义 16.1）可以建模为稀疏三维点集合 $\{\mathbf{s}_j\}_j$ ，这些点称为语义路标，且 $\mathbf{s}_j \in \mathbb{R}^3$ 。语义路标通常对应物体的显著部件；例如，对汽车可以将语义关键点附着在车轮、后视镜和车牌等部位。虽

然这种形状模型可以简单看作点云，但 SLAM 后端优化必须将三维语义路标与二维视觉观测联系起来，并度量相应残差。因此，语义路标不是一般点云，而是使用可变形（或主动）形状模型^[231]定义在三维计算机辅助设计（Computer Aided Design, CAD）物体模型中的关键点：

$$\mathbf{s}_j = \mathbf{b}_{j,0} + \sum_k c_k \mathbf{b}_{j,k}. \quad (16.2)$$

其中， $\mathbf{b}_{j,0}$ 是类别平均形状模型中的语义路标， $\mathbf{b}_{j,k}$ 是通过 PCA 获得的若干形状变化模态，关联形状变形系数 $c_k \in \mathbb{R}_{\geq 0}$ ，如^[854]所述。例如，可以用语义路标 $\mathbf{b}_{j,0}$ 表示汽车物体的平均形状；为了允许不同品牌和型号的汽车实例存在形状变化，再沿主要变化模态 $\mathbf{b}_{j,k}$ 以系数 c_k 进行形状变形。因此，可以直接优化 $\{\mathbf{s}_j\}_j$ ，或优化形状变形系数 c_k ^[1006]，估计观测物体路标的形状。与优化彼此独立的点相比，式（16.2）的参数化确保语义路标描述合理的物体形状。

将语义路标 $\mathbf{s}_j$ 投影到相机图像平面所得的二维点称为语义关键点 $\mathbf{z}_j \in \mathbb{R}^2$ 。可以使用监督学习训练的卷积神经网络模型检测相机图像中的语义关键点，模型输出以每个关键点为中心的二维高斯热图^[854, 1300]。图 16.1 展示了这一过程。



对于物体路标 ℓ_i ，设其世界坐标系位姿为 $\mathbf{q}_i$ 、尺度为 λ_i 、语义路标形状为 $\{\mathbf{s}_{i,j}\}_j$ ，从相机位姿 $\mathbf{T}_i$ （从相机光学坐标系到世界坐标系的变换）得到对应的语义关键点检测 $\{\mathbf{z}_{i,j}\}_j$ 。使用相机透视投影模型，可将二者之间的残差写为

$$r_i(\mathbf{x}_i)^2 = \sum_j \left\| \mathbf{z}_{i,j} - \pi(\mathbf{q}_i^{-1} \mathbf{T}_i, \lambda_i \mathbf{s}_{i,j}) \right\|_{\Sigma_i}^2. \quad (16.3)$$

其中， $\pi(\mathbf{T}, \mathbf{s}_j)$ 表示将三维点 $\mathbf{s}_j$ 透视投影到位姿为 $\mathbf{T}$ 的相机图像平面二维像素坐标。具体而言，相机在物体坐标系中的位姿为 $\mathbf{q}_i^{-1} \mathbf{T}_i$ ；每个具有物体坐标系坐标的语义路标 $\mathbf{s}_{i,j}$ 先乘以 λ_i ，再从物体坐标系变换到相机坐标系。由此得到的物体语义路标预测像素坐标，通过式（16.3）的残差与语义关键点检测 $\mathbf{z}_{i,j}$ 比较。

图 16.2 展示了在真实 KITTI 数据集^[373]上使用语义路标作为物体形状表示的物体 SLAM 示例。该方法扩展了^[108]的工作，为每个物体使用多个语义路标，而不只是物体质心。

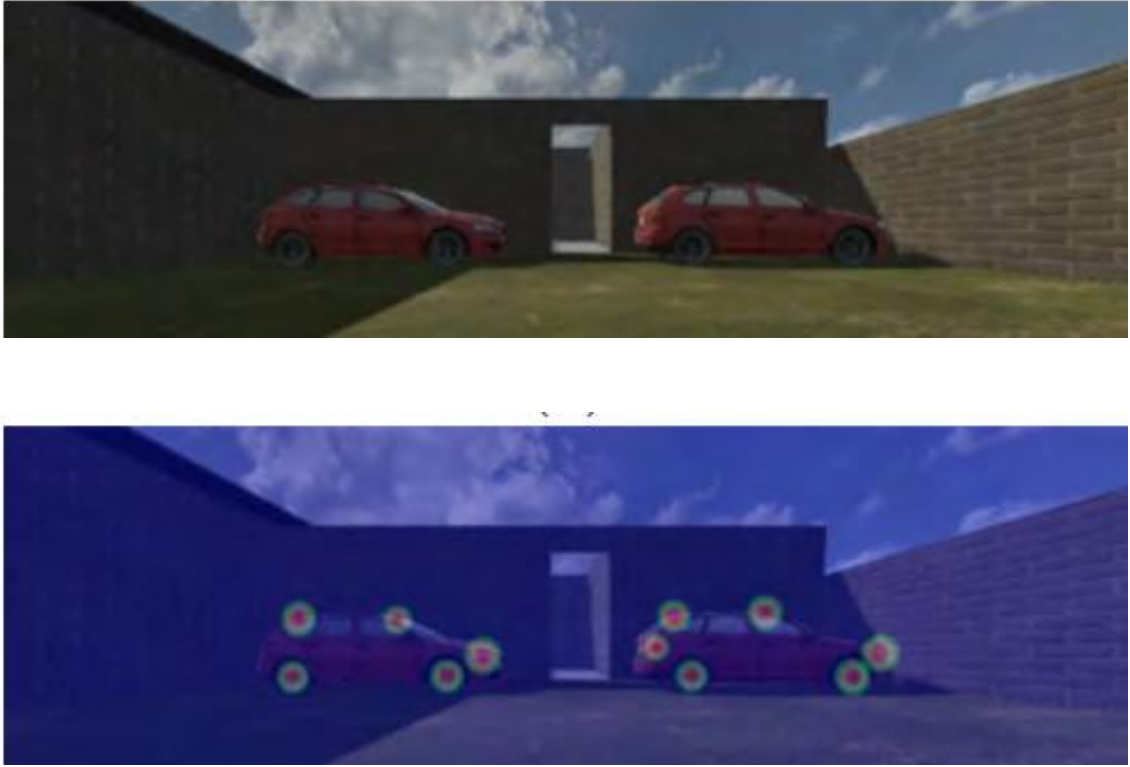


图 16.1: 图 16.1 从 RGB 观测中提取的语义关键点。(a) 包含两辆汽车和一台配备相机的机器人的室外环境；(b) 相机坐标系中的 RGB 观测；(c) RGB 图像中检测到的语义关键点。

16.2.1.2 将物体形状表示为二次曲面

另一种常用的物体形状显式模型基于二次曲面^[805, 821, 995]。这里将物体路标形状建模为椭球 $\mathcal{E}_s \subset \mathbb{R}^3$ ，其半轴长度为 $\mathbf{s} \in \mathbb{R}^3$ ：

$$\mathcal{E}_s = \{\mathbf{x} \in \mathbb{R}^3 \mid \mathbf{x}^\top \text{diag}(\mathbf{s})^{-2} \mathbf{x} \leq 1\}. \quad (16.4)$$

将式 (16.4) 用齐次坐标改写，可以得到由矩阵 $\mathbf{Q}_s \in \mathbb{R}^{4 \times 4}$ 定义的二次曲面^[805]：

$$\mathcal{E}_s = \left\{ \mathbf{x} \in \mathbb{R}^3 \mid \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}^\top \begin{bmatrix} \text{diag}(\mathbf{s})^{-2} & \mathbf{0} \\ \mathbf{0}^\top & -1 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} \leq 0 \right\}. \quad (16.5)$$

椭球形状表示很有吸引力，因为其在相机图像中的投影可以解析求得，进而可使用残差项与边界框物体检测比较（见图 16.3）。三维椭球与其二维圆锥曲线透视投影之间的关系，可以用对偶二次曲面表示解析得到。椭球 $\mathcal{E}_s$ 的对偶是与其相切的所有超平面集合：

$$\mathcal{E}_s^* = \{\mathbf{y} \in \mathbb{R}^3 \mid \mathbf{y}^\top \text{diag}(\mathbf{s})^2 \mathbf{y} \leq 1\} = \left\{ \mathbf{y} \in \mathbb{R}^3 \mid \begin{bmatrix} \mathbf{y} \\ 1 \end{bmatrix}^\top \mathbf{Q}_s^* \begin{bmatrix} \mathbf{y} \\ 1 \end{bmatrix} \leq 0 \right\}. \quad (16.6)$$

其中， $\mathbf{Q}_s^*$ 是 $\mathbf{Q}_s$ 的伴随矩阵。对椭球， $\mathbf{Q}_s$ 可逆，且 $\mathbf{Q}_s^* = \text{adj}(\mathbf{Q}_s) = \det(\mathbf{Q}_s) \mathbf{Q}_s^{-1}$ 。由于式 (16.6) 中对偶二次曲面定义具有尺度不变性，实际可将其简化为 $\mathbf{Q}_s^{-1}$ 。

考虑位姿为 $\mathbf{q}$ 、尺度为 λ 、形状为二次曲面 $\mathbf{Q}_s$ 的物体路标 ℓ 。物体表面可按式缩放，并从局部物体坐标系变换到世界坐标系：

$$\mathbf{Q}_\ell^* \propto \mathbf{q} \begin{bmatrix} \lambda \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix} \mathbf{Q}_s^* \begin{bmatrix} \lambda \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix}^\top \mathbf{q}^\top. \quad (16.7)$$

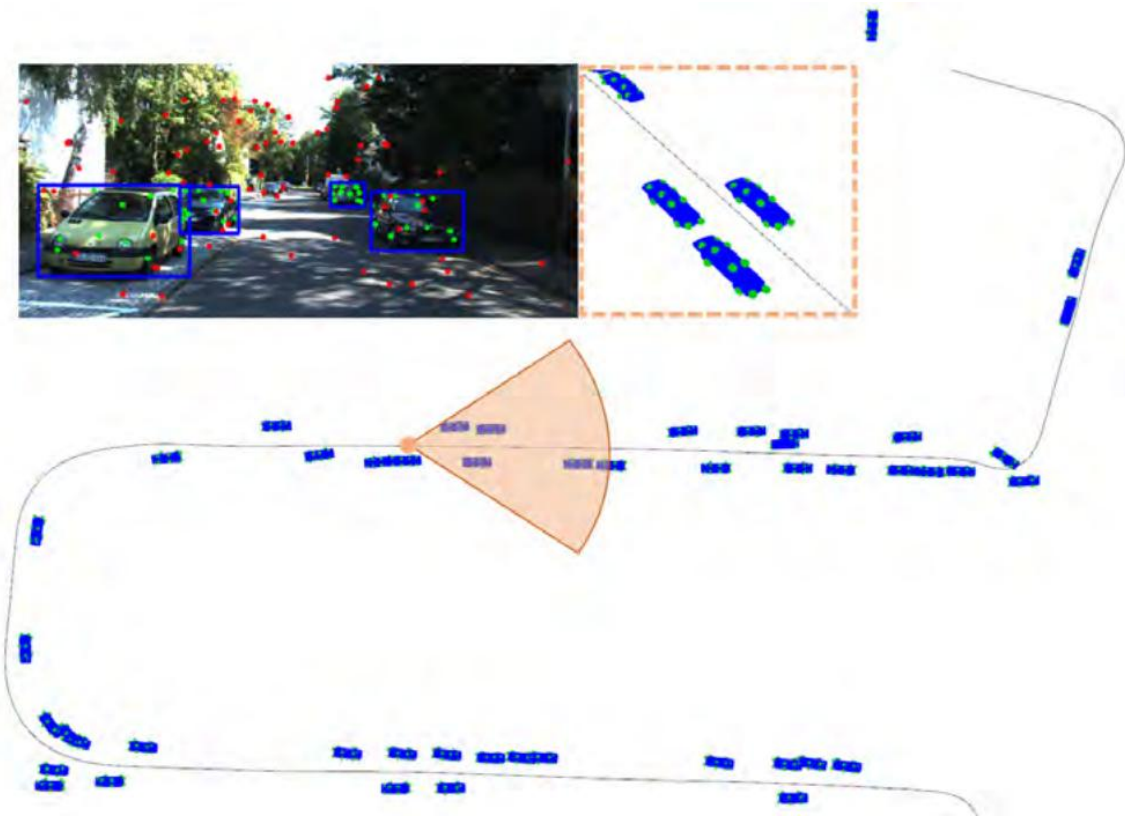


图 16.2: 图 16.2 从 KITTI 数据集^[373] 的 RGB 图像中提取物体检测 (左上蓝色边界框)、语义关键点 (左上绿色二维点) 和普通视觉关键点 (左上红色二维点), 用于估计汽车语义路标 (绿色三维点, 如车轮和前灯)、汽车位姿 (以蓝色网格显示) 以及传感器轨迹 (黑色曲线)。

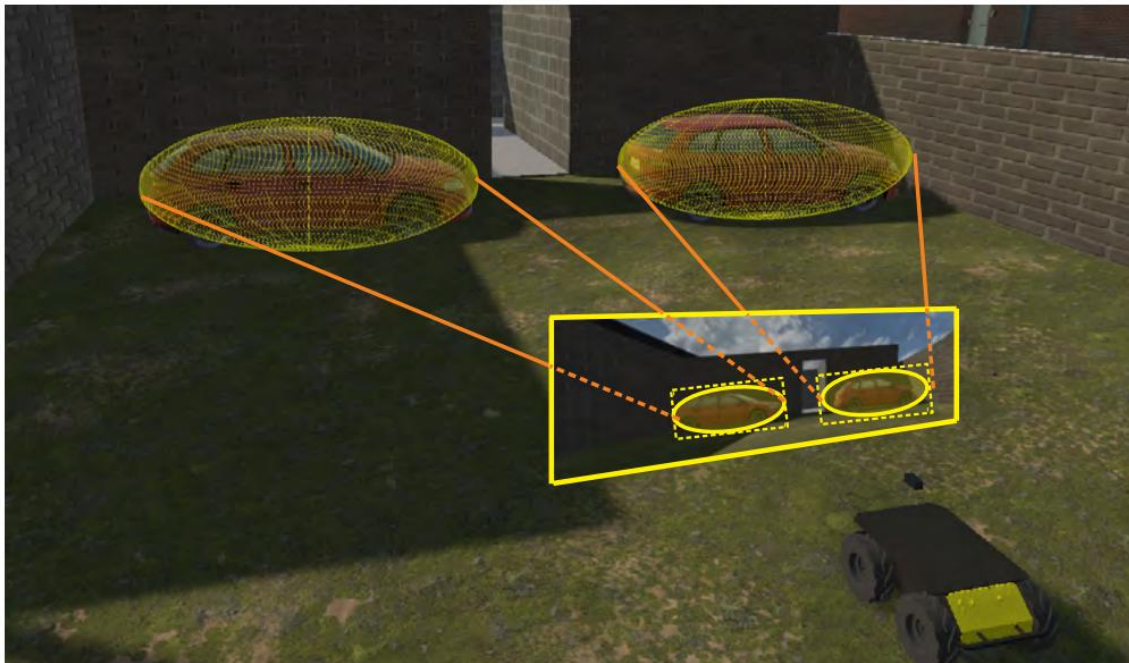


图 16.3: 图 16.3 检测物体的椭球表示。图中同时显示三维椭球投影到机器人相机坐标系的二维结果和检测边界框。

将对偶二次曲面 Q_ℓ^* 从世界坐标系透视投影到位姿为 $T \in \text{SE}(3)$ 的相机图像平面，可以得到对偶圆锥曲线 $C_x^* \in \mathbb{R}^{3 \times 3}$ ：

$$C_x^* \propto \frac{1}{\beta} P T^{-1} Q_\ell^* T^{-\top} P^\top, \quad (16.8)$$

其中， $P = [I_3 \ 0] \in \mathbb{R}^{3 \times 4}$ 是去除最后一维坐标的投影矩阵， β 表示相机深度尺度。下标 x 强调 C_x^* 由状态变量 $x = (T, \ell)$ 决定，其中包含相机位姿和物体路标。

物体边界框检测 $z = [u \ v \ w \ h]^\top \in \mathbb{R}^4$ 包含边界框左下角经相机内参逆矩阵归一化后的像素坐标 (u, v) ，以及边界框宽和高 (w, h) 。令椭圆中心为 $(c_u, c_v) = (u + w/2, v + h/2) \in \mathbb{R}^2$ ，则该边界框内接、轴对齐椭圆的对偶圆锥曲线为

$$C_z^* \propto \begin{bmatrix} 1 & 0 & c_u \\ 0 & 1 & c_v \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} (w/2)^2 & 0 & 0 \\ 0 & (h/2)^2 & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 & c_u \\ 0 & 1 & c_v \\ 0 & 0 & 1 \end{bmatrix}^\top. \quad (16.9)$$

因此，对于位姿为 q_i 、尺度为 λ_i 、二次曲面形状为 Q_{s_i} 的物体路标，以及由相机位姿 T_i 得到的边界框检测 z_j ，可以比较式 (16.9) 由边界框产生的圆锥曲线 $C_{z_j}^*$ 与式 (16.8) 由二次曲面投影产生的 $C_{x_i}^*$ ，得到残差：

$$r_i(x_i) = \left\| C_{z_j}^* - C_{x_i}^* \right\|_{\Sigma_i}. \quad (16.10)$$

为避免定义矩阵范数，并利用圆锥曲线由对称矩阵定义这一事实，可以将上述差异向量化。具体地，令 $b_j = \text{vech}(C_{z_j}^*)$ 为对称矩阵 $C_{z_j}^*$ 的半向量化，并令 $v(T_i, \ell_i) = \text{vech}(T_i^{-1} Q_{\ell_i}^* T_i^{-\top})$ 。再令 $A = D(P \otimes P)E$ ，其中 $\otimes$ 为 Kronecker 积， $D \in \mathbb{R}^{6 \times 9}$ 与 $E \in \mathbb{R}^{16 \times 10}$ 满足 $\text{vech}(Q) = D \text{vec}(Q)$ 及 $\text{vec}(Q) = E \text{vech}(Q)$ 。于是式 (16.10) 可重写为

$$r_i(x_i) = \left\| b_j - \frac{1}{\beta_i} A v(T_i, \ell_i) \right\|_{\Sigma_i}. \quad (16.11)$$

有关细节可参见 [952]，其他残差形式可参见 [805, 821]。图 16.4 给出 OrcVIO 算法 [995] 在真实数据上的物体 SLAM 示例；该算法将语义路标和椭球结合为物体形状表示。

16.2.1.3 将物体形状表示为网格

使用三角网格可获得表达能力更强的物体形状表示。设顶点集合为 $\mathcal{V} = \{s_j\}_j$ ，其中 $s_j \in \mathbb{R}^3$ ，面集合为 $\mathcal{F} \subset \mathcal{V} \times \mathcal{V} \times \mathcal{V}$ 。为了估计具有网格形状的物体路标参数，可以将网格渲染结果与通过物体检测和分割得到的相机图像分割掩膜 $z_i \subset \mathbb{R}^2$ 关联 [444, 920, 1121]，如图 16.5 所示。

要在 SLAM 中优化物体网格形状，必须定义可微的残差。关键难点在于利用可微渲染将网格投影并光栅化到图像平面。给定顶点 $\mathcal{V}$ 和面 $\mathcal{F}$ ，可定义光栅化函数 $\rho(\mathcal{V}, \mathcal{F})$ ：将网格顶点投影到图像平面，若一个像素处存在多个面，则只绘制最前方的面 [733]。Kato 等人 [541] 和 Liu 等人 [680] 最早获得了光栅化函数 ρ 相对于网格顶点的近似梯度；关于可微渲染的综述参见 [542]。

对于位姿 q_i 、尺度 λ_i 、网格形状 $(\mathcal{V}, \mathcal{F})$ 的物体路标 ℓ_i ，以及从机器人位姿 T_i 获得的分割掩膜 z_i ，可以使用网格图像投影与分割掩膜交并比的倒数定义残差：

$$r_i(x_i) = \frac{\rho(\{\pi(q_i^{-1} T_i, \lambda_i s_{i,j})\}_{j,j}, \mathcal{F}) \cup z_i}{\rho(\{\pi(q_i^{-1} T_i, \lambda_i s_{i,j})\}_{j,j}, \mathcal{F}) \cap z_i}. \quad (16.12)$$

其中， $\pi(q_i^{-1} T_i, \lambda_i s_{i,j})$ 与式 (16.3) 相同，表示三维网格顶点到图像平面的透视投影； ρ 是可微网格光栅化函数。与上述语义路标表示一样，网格顶点坐标 $\{s_j\}_j$ 可以直接优化，也可以通过定义类别平均形状并优化形状变化主成分的变形系数间接优化。还可以使用其他残差，例如渲染物体掩膜与分割掩膜 z_i 之间的 ℓ_2 损失或二元交叉熵 [702]。

可微网格渲染已用于 [323] 中的物体 SLAM、[855] 中的人体姿态估计和 [702] 中的机器人姿态估计。图 16.6 给出 [323] 在真实 KITTI 数据集 [373] 上采用网格形状表示的物体 SLAM 示例。图 16.7 给出 [1161] 的类似示例，该方法从存储有符号距离值的体素网格的可变形形状模型 (16.2) 生成可微分割掩膜。

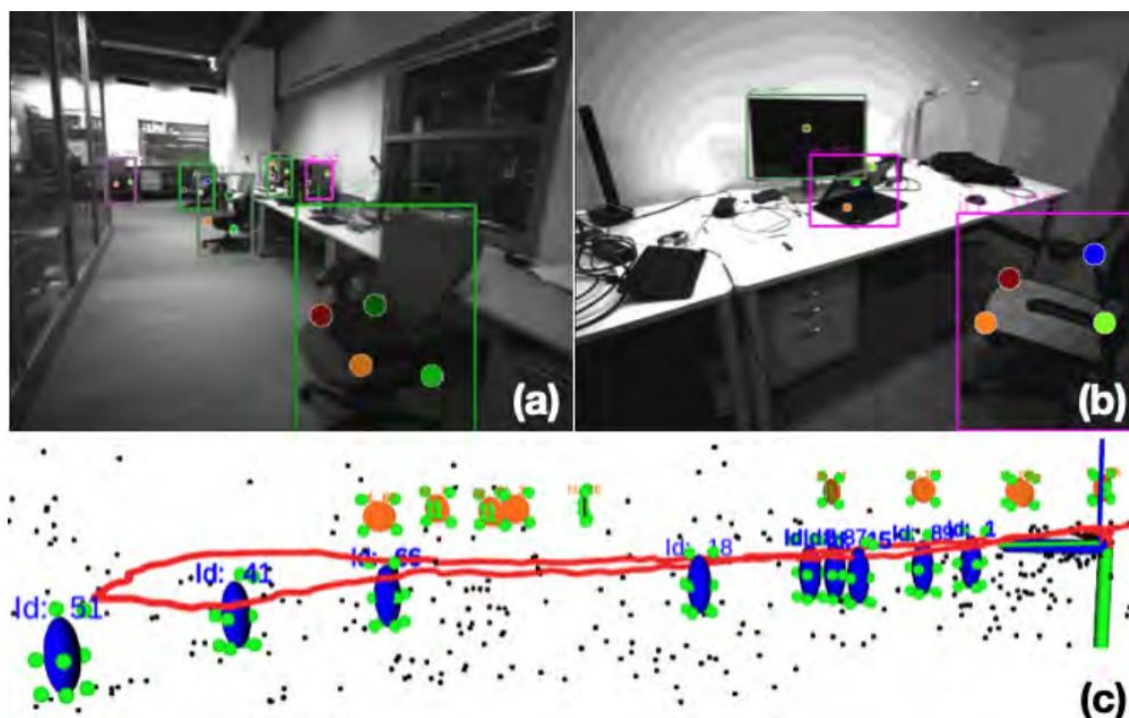


图 16.4: 图 16.4 含椅子和显示器的室内场景中的定位和物体建图。(a) 与 (b) 显示边界框和语义关键点检测；(c) 显示 OrcVIO^[995] 得到的传感器估计轨迹（红色曲线）、几何路标（黑点）、语义路标（绿点）和物体椭球（椅子为蓝色，显示器为橙色）。

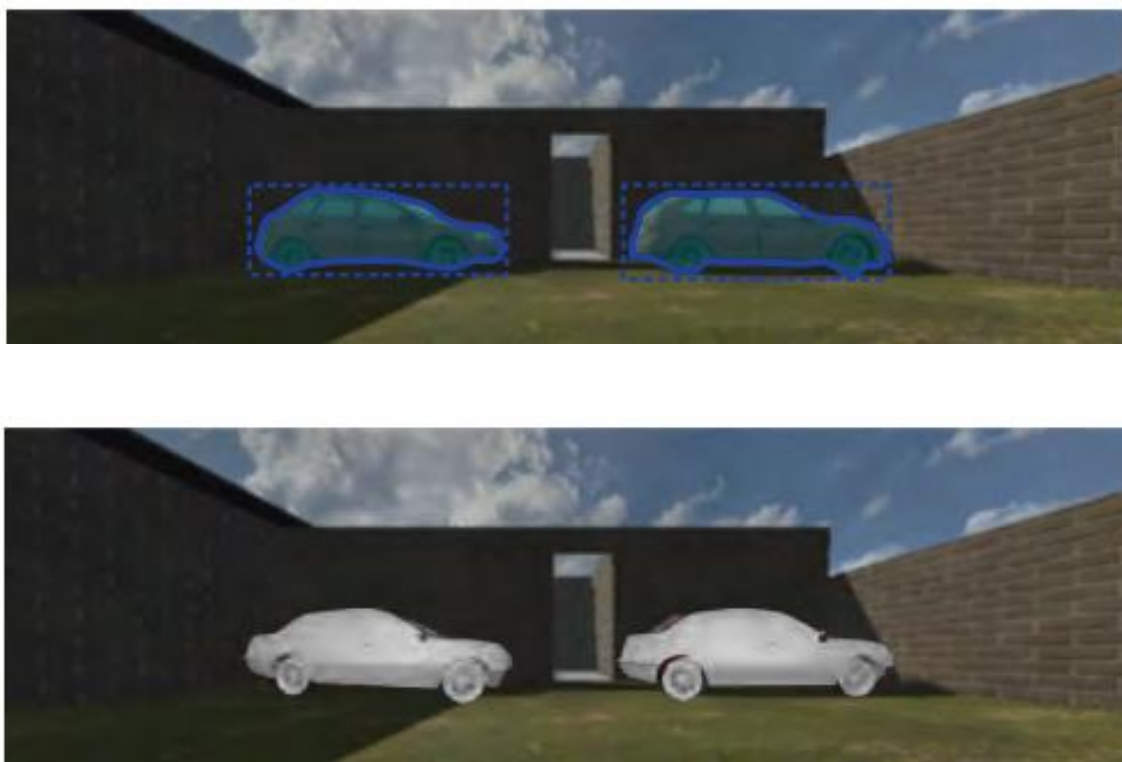


图 16.5: 图 16.5 检测物体的网格形状表示。(a) 相机视图在以蓝色突出显示的检测物体边界框和分割掩膜；(b) 最小化分割掩膜与物体网格投影之间残差的结果。

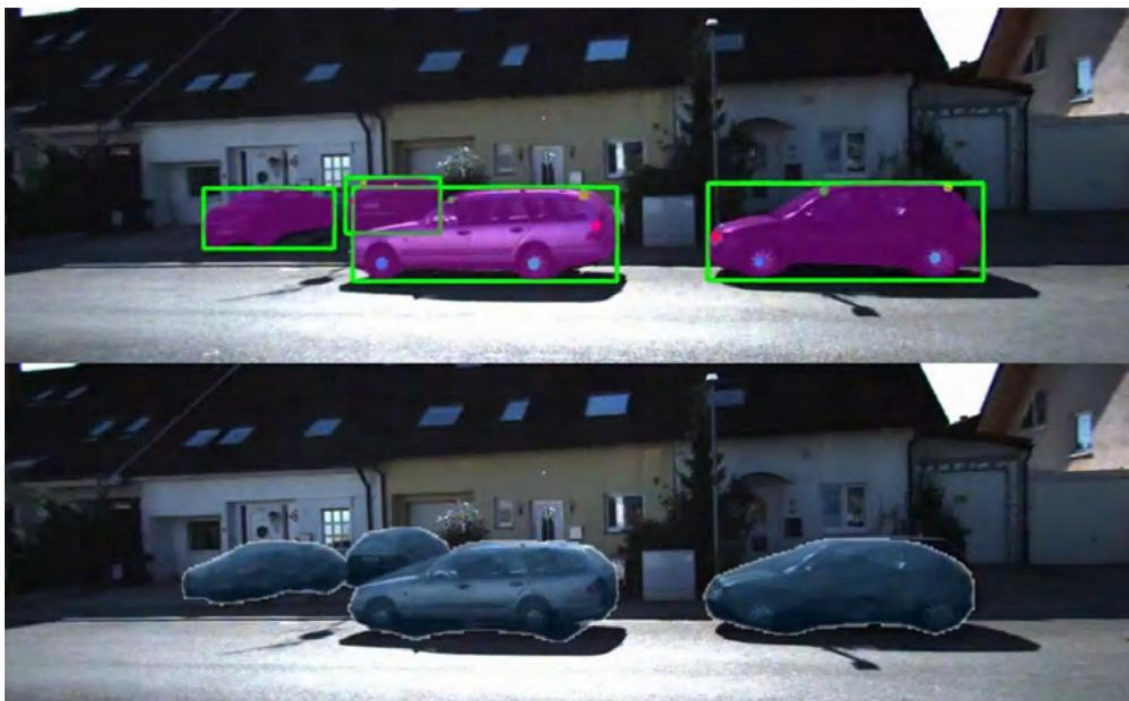


图 16.6: 图 16.6 在 KITTI 里程计数据集^[373]上使用可微分割掩膜^[323]估计三维物体网格形状和位姿的定性结果。方法以边界框（绿色）、分割掩膜（洋红色）和语义关键点（多种颜色）为输入，利用式（16.12）的交并比残差优化物体位姿和网格形状。

16.2.1.4 隐式物体形状表示

上面讨论的语义路标、二次曲面和网格都是显式形状模型，因为它们直接表示物体表面几何。下面讨论隐式形状表示：它将物体建模为空间函数，该函数的某个等值集表示物体表面。常用的隐式形状表示包括占据函数^[759]、有符号距离函数（signed distance function, SDF）^[847]和 NeRF^[1278]。

设物体路标的形状为集合 $\mathcal{S} \subset \mathbb{R}^3$ 。集合 $\mathcal{S}$ 的占据函数 $f_{\mathcal{S}}(\mathbf{x})$ 是二值函数，用来指示点 $\mathbf{x} \in \mathbb{R}^3$ 是否位于 $\mathcal{S}$ 内：

$$f_{\mathcal{S}}(\mathbf{x}) = \begin{cases} -1, & \mathbf{x} \in \mathcal{S}, \\ 1, & \mathbf{x} \notin \mathcal{S}. \end{cases} \quad (16.13)$$

集合 $\mathcal{S}$ 的 SDF $d_{\mathcal{S}}(\mathbf{x})$ 是实值函数，度量点 $\mathbf{x} \in \mathbb{R}^3$ 到 $\mathcal{S}$ 边界的有符号距离：

$$d_{\mathcal{S}}(\mathbf{x}) = \begin{cases} -\inf_{\mathbf{y} \in \partial \mathcal{S}} \|\mathbf{x} - \mathbf{y}\|, & \mathbf{x} \in \mathcal{S}, \\ \inf_{\mathbf{y} \in \partial \mathcal{S}} \|\mathbf{x} - \mathbf{y}\|, & \mathbf{x} \notin \mathcal{S}. \end{cases} \quad (16.14)$$

从传感器测量重建物体形状的隐式模型，可以看作估计占据函数 $f_{\mathcal{S}}(\mathbf{x})$ 或 SDF $d_{\mathcal{S}}(\mathbf{x})$ 的回归问题。这里重点讨论 SDF 形状估计，占据模型也可采用类似方法。为近似 $d_{\mathcal{S}}(\mathbf{x})$ ，引入参数为 θ 的神经网络 $d_{\theta}(\mathbf{x}, \mathbf{s})$ ，其中潜在特征向量 $\mathbf{s} \in \mathbb{R}^d$ 称为形状代码，用于捕获 $\mathcal{S}$ 的特定形状。换言之， d_{θ} 是一个神经网络解码器，用来估计查询点 $\mathbf{x}$ 到具有形状代码 $\mathbf{s}$ 的物体表面的有符号距离。改变 $\mathbf{s}$ 可在同一个解码器下描述不同物体形状，因而适合类别级形状表示^[847, 996]。

为估计物体路标 ℓ_i 的 SDF，设 LiDAR 或深度相机等距离传感器从机器人位姿 $\mathbf{T}_i$ 提供数据 $\{\mathbf{y}_{i,j}, z_{i,j}\}_j$ ，其中 $\mathbf{y}_{i,j}$ 是物体 ℓ_i 表面附近的点测量， $z_{i,j}$ 是到物体表面的距离测量。对于位姿 $\mathbf{q}_i$ 、尺度 λ_i 、形状代码 $\mathbf{s}_i$ 的物体路标，残差通过比较 SDF 预测与测得距离得到：

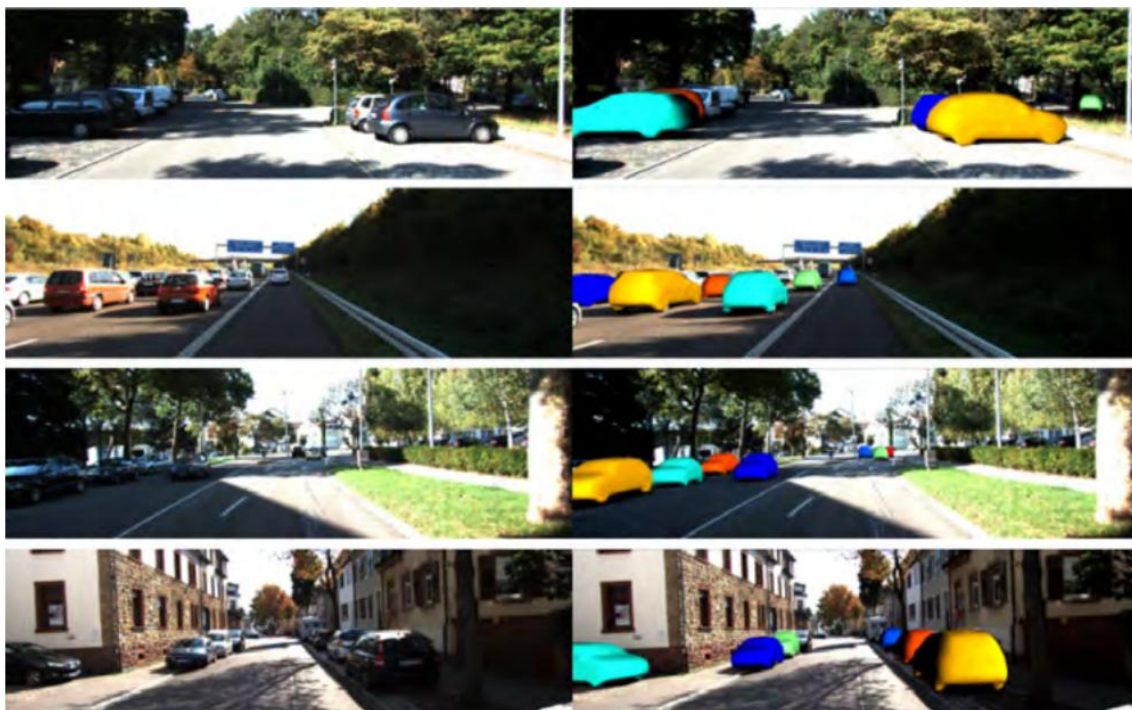


图 16.7: 图 16.7 在 KITTI Stereo 2015 基准^[756]上, 使用由可变形有符号距离形状模型^[1161]得到的可微分割掩膜, 估计三维物体形状和位姿的定性结果。该方法输入具有分割掩膜的立体图像、初始物体位姿和学习得到的物体平均形状; 将物体投影到图像后, 通过轮廓对齐和光度一致性残差度量投影与分割掩膜的一致性, 从而优化物体位姿和形状。

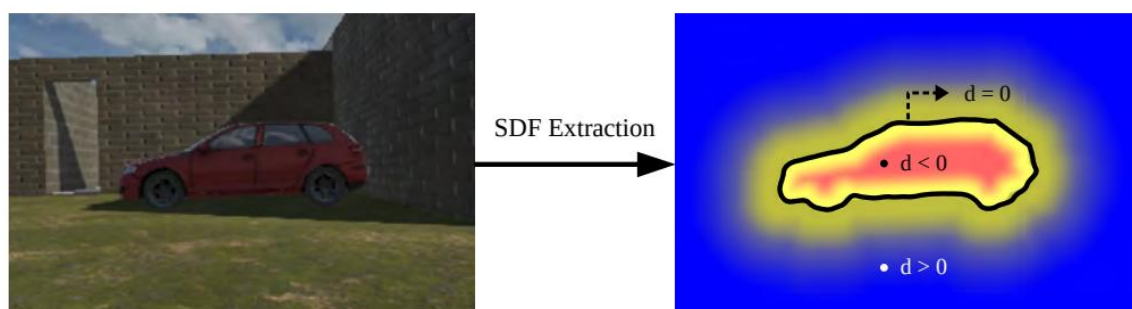


图 16.8: 图 16.8 物体路标的 SDF。按定义, 物体表面处距离为零; 物体内部和外部的距离分别为负和正。

$$r_i(\mathbf{x}_i)^2 = \sum_j |z_{i,j} - d_{\theta}(\lambda_i^{-1} \mathbf{q}_i \mathbf{T}_i^{-1} \mathbf{y}_{i,j}, \mathbf{s}_i)|_{\sigma_i}^2. \quad (16.15)$$

其中, $\lambda_i^{-1} \mathbf{q}_i \mathbf{T}_i^{-1} \mathbf{y}_{i,j}$ 先将点 $\mathbf{y}_{i,j}$ 从机器人坐标系变换到世界坐标系, 再从世界坐标系变换到物体坐标系, 最后乘以 λ_i^{-1} , 从而得到规范物体坐标系中的查询点^[1152]。若 SDF 解码器已经训练完成, 最小化式 (16.15) 的残差即可同时估计机器人位姿 $\mathbf{T}_i$, 以及物体路标 ℓ_i 的位姿 $\mathbf{q}_i$ 、尺度 λ_i 和形状代码 $\mathbf{s}_i$ 。SDF 回归方法常加入其他残差, 例如通过 Eikonal 方程将物体表面法向与 d_{θ} 的梯度联系起来^[404], 或鼓励远离表面的点取正/负值^[834]。

应区分训练阶段和测试阶段。训练阶段使用离线数据优化某个物体类别 SDF 解码器的参数 θ ; 测试阶段使用在线数据、预训练解码器, 优化此前未见同类别物体实例的位姿 $\mathbf{q}_i$ 、尺度 λ_i 和形状代码 $\mathbf{s}_i$ 。训练时通常假设传感器位姿 $\mathbf{T}_i$ 、路标位姿 $\mathbf{q}_i$ 和尺度 λ_i 已知, 针对 θ 与 $\mathbf{s}_i$ 最小化式 (16.15)。解码器参数 θ 通常在整个物体类别内共享, 而形状代码 $\mathbf{s}_i$ 对每个物体实例单独优化。测试时通常假设 θ 已知, 针对包含 $\mathbf{T}_i$ 、 $\mathbf{q}_i$ 、 λ_i 和 $\mathbf{s}_i$ 的状态 $\mathbf{x}_i$ 最小化式 (16.15)。图 16.9 显示 ELLIPSDF^[996] 在真实 ScanNet 数据集^[238] 上结合粗略椭球形状和精细 SDF 形状的物体 SLAM 示例。



图 16.9: 图 16.9 ELLIPSDF^[996] 在 ScanNet 数据集^[238] (场景 0087) 上进行三维物体位姿和隐式 SDF 形状重建的定性结果。图中左侧为场景 RGB 图像, 右上为真值彩色点云重建, 右下为由物体形状代码解码的 SDF 模型和优化后的 SIM(3) 位姿所重建的物体网格。

16.2.2 稀疏度量-语义 SLAM 的混合求解器

基于物体或稀疏度量-语义 SLAM 的表示, 将语义物体类别等离散信息与物体位置、方向和形状等连续信息结合起来。因此, 求解此类问题通常需要联合推理这些往往以复杂方式耦合的状态。例如, 物体的语义

类别可以提供其形状信息^[994]，反之亦然。离散变量也会以其他方式进入稀疏度量-语义 SLAM，例如关注物体具有离散对称性^[705]，或基于学习的物体检测与姿态估计前端给出多个离散位姿假设^[344]。与纯几何路标 SLAM 一样，数据关联（第 3 章）仍是开发稀疏度量-语义 SLAM 系统的关键挑战。

离散状态使 SLAM 的实际求解难度显著上升。将问题写成优化问题时，度量-语义 SLAM 与纯度量 SLAM 一样具有非线性、非凸和高维特性，还具有组合性：必须在离散状态数目呈指数增长的状态空间中搜索。给定将物体检测和分类视为有噪传感器输出的模型，可以按最大后验（MAP）推断同时估计环境路标的潜在语义类别和几何：

$$\hat{X}, \hat{L}, \hat{D} = \arg \max_{X, L, D} p(X, L, D | Z). \quad (16.16)$$

其中， Z 是完整测量集（包括语义测量）， X 是机器人位姿集合， L 是环境路标集合；路标通常将位置、方向、尺寸和形状等几何信息与来自已知固定类别集的离散语义标签结合。 D 是 Z 中测量与 L 中路标之间的关联集合。物体离散类别信息可以自然融入数据关联本已离散的推断问题：类别知识有助于在杂乱环境中将物体同其他物体区分开。该表述统一了语义类别离散模型、几何估计和数据关联；但除标准 SLAM 的非凸高维优化外，还需处理组合优化及学习感知模型的错误。

通常，式 (16.16) 的 MAP 推断计算上不可解 [592, Section 13.1.1]。即使是机器人感知中的纯连续估计问题也往往是 NP-hard 的（见第 6 章）。不过，在拥有良好初始化时，平滑局部优化通常能以较低代价获得良好解。即便假定连续估计可以高效求解，离散变量仍使问题复杂：在最坏情况下，联合 MAP 全局估计要求对每一种离散状态赋值求解一个连续优化子问题，而离散状态空间随离散变量数目指数增长。因此，需要高效的近似解法。

可将式 (16.16) 写成混合离散-连续因子图的分解形式。把连续变量记为 Θ 、离散变量记为 D ，有

$$p(\Theta, D | Z) \propto \prod_k \phi_k(\mathcal{V}_k), \quad \mathcal{V}_k \triangleq \{v_i \in \mathcal{V} \mid (\phi_k, v_i) \in \mathcal{E}\}. \quad (16.17)$$

每个因子 ϕ_k 对应测量似然 $p(z_k | \mathcal{V}_k)$ 或先验 $p(\mathcal{V}_k)$ ， $\mathcal{V}_k$ 是参与该因子的连续或离散变量集合。后验可分为三类因子：仅含离散变量的 $\phi_k(D_k)$ ，仅含连续变量的 $\phi_k(\Theta_k)$ ，以及离散-连续因子 $\phi_k(\Theta_k, D_k)$ 。由此，式 (16.16) 的 MAP 问题等价于最小化负对数后验：

$$\Theta^*, D^* = \arg \max_{\Theta, D} p(\Theta, D | Z) = \arg \min_{\Theta, D} \sum_k -\log \phi_k(\mathcal{V}_k) \triangleq \arg \min_{\Theta, D} \mathcal{L}(\Theta, D). \quad (16.18)$$

许多混合估计问题可以表示为非线性最小二乘，从而可以用第 1 章的 iSAM2^[534] 等工具估计关注的连续状态。具体地，离散-连续因子可写为

$$-\log \phi_k(\Theta_k, D_k) = \|\mathbf{r}_k(\Theta_k, D_k)\|_2^2, \quad \Theta_k \subseteq \Theta, D_k \subseteq D, \quad (16.19)$$

其中 $\mathbf{r}_k$ 对 Θ 通常非线性但一阶可微，上式允许差一个与 Θ, D 无关的常数；只含连续变量的因子也可作类似表示。

这种表述可以利用因子图的条件独立结构实现高效局部推断。若固定任一离散状态赋值，则只剩连续变量，可如第 1 章那样采用平滑优化近似推断；反过来，若固定连续变量估计，则得到离散因子图优化，可用 max-product 变量消元获得全局最优。后者最坏情况下仍需探索指数数量的离散状态，但允许采用成熟的启发式近似策略。

尤其是，将离散状态划分为在给定连续状态时条件独立、且相互不交的子集 $D_j \subseteq D$ ，则

$$p(D | \Theta, Z) \propto \prod_j p(D_j | \Theta, Z), \quad (16.20)$$

并且

$$\max_D p(D | \Theta, Z) \propto \prod_j \left[\max_{D_j} p(D_j | \Theta, Z) \right]. \quad (16.21)$$

这将“乘积的最大值”替换为可独立计算的“各最大值之积”。当离散状态分解为很小的子集 ($|D_j| \ll |D|$) 时, 可以高效推断。点云配准 (含离散数据关联变量)、鲁棒位姿图优化 (含离群值剔除开关变量) 和度量-语义 SLAM (含数据关联变量和离散物体类别) 等许多机器人混合优化问题, 都具有这种相对友好的分解^[276, 618]; 另见第 3 章第 3.3.2 节。

这一思想有多种用法。例如, DC-SAM^[276] 采用交替最小化: 固定初始迭代 $\Theta^{(i)}$ 后, 依次求解

$$D^{(i+1)} = \arg \min_D \mathcal{L}(\Theta^{(i)}, D), \quad (16.22a)$$

$$\Theta^{(i+1)} = \arg \min_{\Theta} \mathcal{L}(\Theta, D^{(i+1)}). \quad (16.22b)$$

重复式 (16.22a) 和 (16.22b), 直至 $\mathcal{L}(\Theta, D)$ 的相对下降足够小或达到最大迭代次数。实践中无需将每个子问题求至最优: 连续子问题可使用常规优化技术求解; 还可将两阶段块坐标下降推广为针对更小变量组迭代, 在每一步复杂度和总优化步数间权衡。

Bowman 等人^[108] 对稀疏度量-语义 SLAM 使用相近策略, 通过期望最大化 (EM) 联合优化: 先固定机器人位姿和路标位置, 计算数据关联概率和路标类别 (E 步); 再固定数据关联概率和路标类别, 以相应路标对应概率为测量加权, 优化机器人位姿和路标位置 (M 步)。该方法为物体赋予“软”关联, 逐渐收敛到式 (16.16) 的局部最优解。E 步概率可由矩阵 permanent 精确恢复; 通过近似 permanent, 可更高效地近似 M 步所需的数据关联概率^[40]。

多假设方法显式搜索 D 的可能赋值来求解式 (16.18), 但状态空间规模随离散状态数目呈指数增长, 因此需要启发式剪枝保持可计算性。MH-iSAM2^[479] 和 iMHS^[519] 等通用方法已用于鲁棒位姿图优化、足式机器人的接触估计等混合估计问题, 但尚未应用于稀疏度量-语义 SLAM。Bernreiter 等人^[74] 提出面向度量-语义建图和定位的多假设方法。

另一种方法是将离散变量的组合推断改写为只对连续变量的优化。早期工作^[1061] 在 K 类语义标注问题中优化语义标签概率, 标签概率定义在 $(K-1)$ 维单位单纯形上。这与第 3 章离群值剔除策略类似, 但扩展至多个标签。还有方法尝试将式 (16.16) 中所有离散状态边缘化, 近似完整后验为混合分布; 由于该后验一般非高斯, 实际需使用近似方法, 参见面向 SLAM 的非高斯求解器^[337, 491] 及度量-语义 SLAM 应用^[274, 41]。

16.3 稠密度量-语义表示

本节讨论将第 5 章地图表示扩展为包含语义信息的稠密度量-语义表示。第 16.3.1 节首先讨论使用语义信息的点和 surfel 地图; 第 16.3.2 节说明如何以概率一致的方式将 OctoMap^[478] 等体素地图扩展为语义表示; 第 16.3.3 节讨论网格表示。与体素地图相比, 网格通常更节省存储, 并且在回环闭合时更容易变形。

16.3.1 基于点和 surfel 的度量-语义 SLAM

大规模稠密标注数据集^[68, 137, 69, 330] 使得直接在点云上学习语义分割^[768] 与全景分割^[767] 成为可能。这些模型提供逐点语义, 可能还提供实例信息; 因此, 若干 LiDAR-SLAM 方法^[187, 652, 522] 直接利用这些信息构建度量-语义地图。正如本章前面所述, 语义有助于识别动态物体、改进数据关联: 可以在里程计对应搜索中利用语义^[652, 522], 也可以构建语义场景描述子以检测回环^[593, 522]。

这类方法的关键挑战, 是如何将相互冲突的逐帧 LiDAR 点云语义分割进行空间融合, 形成统一地图。早期使用语义分割的稠密 LiDAR SLAM 方法 SuMa++^[187], 为每个 surfel 增加语义类别和语义置信度。Chen 等人^[187] 同时利用语义置信度和测量不确定性更新 surfel 置信度, 使冲突的语义标签降低 surfel 的置信度。最终可滤除低置信度 surfel; 这些 surfel 可能来自测量不确定性, 也可能来自语义不确定性。

将语义纳入 surfel 地图使系统能够处理冲突标签, 并更有效地滤除动态物体。Chen 等人^[187] 表明, 仅按语义移除所有潜在运动物体, 效果劣于只根据地图与当前场景语义解释之间的冲突进行处理。其原因是城市环境中车辆普遍存在, 仍可用于位姿估计; 若简单移除所有潜在运动物体, 会同时移除可用于 ICP 对齐的有价值地图信息。

后续方法如 SA-LOAM^[652] 和 SELVO^[522] 仅匹配语义兼容特征^[1263]，将语义纳入位姿估计，也在回环确定时利用语义^[593]。Li 等人^[652] 在 LOAM^[1269] 基础上，采用基于点的表示，在不同语义地图中存储表面型和角点型结构特征；回环时通过逐点语义分割生成的语义图匹配场景。Jiang 等人^[522] 同样利用匹配语义类别的对应关系，并扩展 Scan Context 描述子，使其在场景描述子匹配过程中考虑语义。图 16.10 展示了仅用立体相机图像获得的大规模语义标注点云地图；该方法^[193] 使用立体图像的稠密二维语义标签，结合直接稀疏视觉里程计前端和全局优化后端。

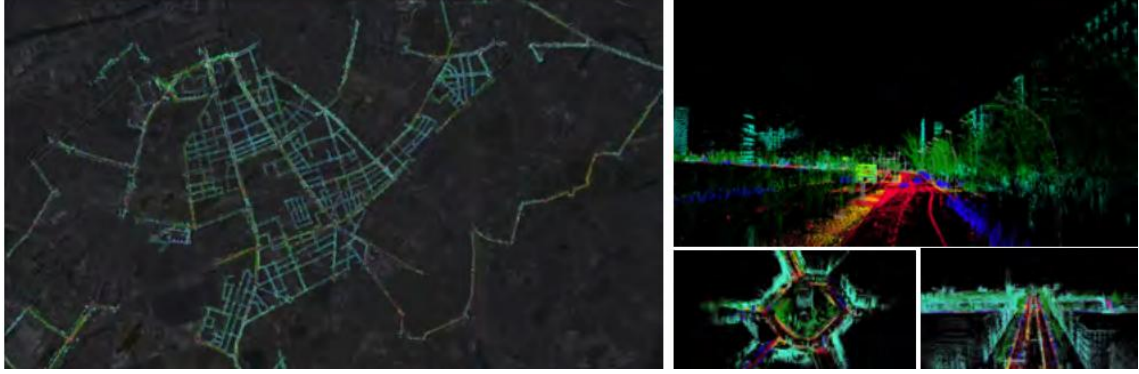


图 16.10: 图 16.10 车辆队列利用^[193]的方法生成的柏林大规模语义地图，覆盖 8 000 km 道路（左）。语义地图局部放大图显示精细三维重建细节（右）。图像来自^[193]（©2022 IEEE）。

16.3.2 基于体素的度量-语义 SLAM

利用体素的稠密环境模型已被广泛用作点云建图的替代方案^[97, 1064, 1124, 347]。设机器人导航环境为 $\mathcal{E}$ 。如第 5 章所述，体素地图 m 通过互不重叠立方体网格划分 $\mathcal{E}$ ；每个体素存储占据、温度等环境属性，并假定该属性在相应三维物理空间内恒定。本节讨论如何在占据或距离信息之外，为体素表示增添语义信息。

要将体素表示扩展为度量-语义地图，第一步是将语义纳入机器人观测。将 $\mathcal{E}$ 划分为不相交集 $\mathcal{E}_k \subset \mathbb{R}^3$ ，每个集合对应语义类别 $k \in \mathcal{K} := \{0, \dots, K\}$ ； $\mathcal{E}_0$ 是自由空间， $k > 0$ 的 $\mathcal{E}_k$ 表示建筑物、汽车和地形等不同物体类别。设机器人传感器沿射线集合 $\{\eta_b\}$ 提供周边物体距离和语义类别信息， b 是射线索引， $\|\eta_b\| = r_{\max}$ ，其中 $r_{\max}$ 为最大测距范围。该集合只是 RGB-D 相机或 LiDAR 等测距传感器长度为 $r_{\max}$ 的射线向量集合。通过对相机^[766] 或 LiDAR^[768] 测量进行语义分割，可为每条射线关联语义标注点测量。

具体地，若机器人在时刻 t 的方向为 $\mathbf{R}_t \in \text{SO}(3)$ 、位置为 $\mathbf{p}_t \in \mathbb{R}^3$ ，则沿传感器射线 $\mathbf{R}_t \eta_b + \mathbf{p}_t$ 的距离-类别观测定义为集合 $\mathbf{Z}_t = \{z_{t,b}\}_b$ ，其中 $\mathbf{z} := (r_{t,b}, y_{t,b}) \in \mathbb{R}_{\geq 0} \times \mathcal{K}$ 。图 16.11 中，RGB 图像每个像素对应一条射线 η_b ，语义分割图和距离图中相应的值分别为类别 $y_{t,b}$ 和距离 $r_{t,b}$ 。基于体素的度量-语义建图目标，是根据流式距离-类别观测增量构建环境 $\mathcal{E}$ 的多类别地图 m 。地图建模为单元 $i \in \mathcal{I} := \{1, \dots, N\}$ 的栅格，每个单元标记一个类别 $m_i \in \mathcal{K}$ 。

令 $p(\mathbf{Z}_t | m, \mathbf{R}_t, \mathbf{p}_t)$ 为描述传感器观测噪声的概率密度函数。利用该观测模型，可通过贝叶斯更新将测量融合到概率地图。令 $p_t(m) := p(m | \mathbf{Z}_{1:t}, \mathbf{R}_{1:t}, \mathbf{p}_{1:t})$ 为给定时刻 t 前机器人轨迹和观测的地图概率质量函数。由位姿 $(\mathbf{R}_{t+1}, \mathbf{p}_{t+1})$ 获得新观测 $\mathbf{Z}_{t+1}$ 后，更新为

$$p_{t+1}(m) \propto p(\mathbf{Z}_{t+1} | m, \mathbf{R}_{t+1}, \mathbf{p}_{t+1}) p_t(m). \quad (16.23)$$

后续为简洁起见省略地图分布和观测模型对机器人位姿的依赖。Asgharivaskasi 和 Atanasov^[37] 通过对数几率占据建图算法^[1088, Ch. 9] 推广到多类别地图，提出在线基于体素的语义建图技术。每个地图单元 m_i 存储 $\mathcal{K}$ 中物体类别的概率。为保证模型复杂度随地图大小 N 线性增长，维护按地图单元分解的概率质量函数：

$$p_t(m) = \prod_{i=1}^N p_t(m_i).$$

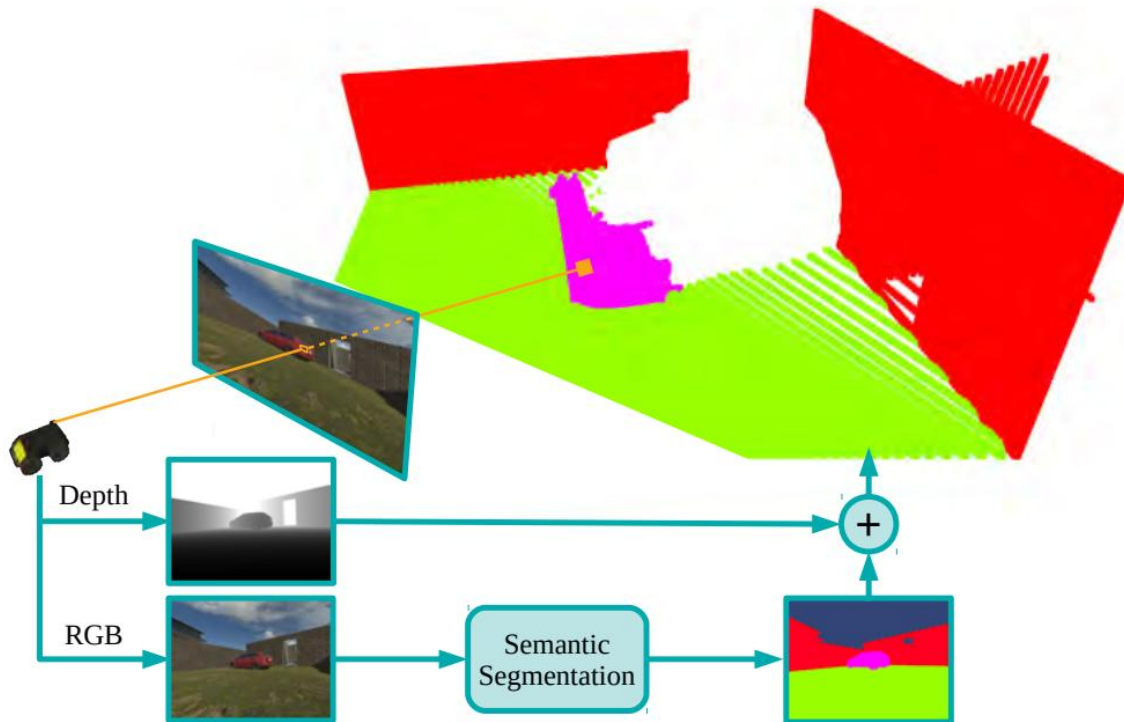


图 16.11: 图 16.11 距离-类别观测的生成过程。机器人配备 RGB-D 传感器，将其 RGB 测量输入语义分割算法；给定相机内参后，结合语义分割图和深度图形成带语义标注的三维点云。

单个单元 PMF $p_t(m_i)$ 的多类别对数几率是定义在语义类别 $\mathcal{K}$ 上的向量，表示相对似然的对数比：

$$\mathbf{h}_{t,i} := \left[\log \frac{p_t(m_i = 0)}{p_t(m_i = 0)} \cdots \log \frac{p_t(m_i = K)}{p_t(m_i = 0)} \right]^\top \in \mathbb{R}^{K+1}.$$

这里以自由类别似然 $p_t(m_i = 0)$ 为枢轴。给定对数几率向量 $\mathbf{h}_{t,i}$ ，可以用 softmax 函数 $\sigma : \mathbb{R}^{K+1} \rightarrow \mathbb{R}^{K+1}$ 恢复单元 m_i 的 PMF：

$$p_t(m_i = k) = \sigma_{k+1}(\mathbf{h}_{t,i}) = \frac{\mathbf{e}_{k+1}^\top \exp(\mathbf{h}_{t,i})}{\mathbf{1}^\top \exp(\mathbf{h}_{t,i})}.$$

在新的测量集合 $\mathbf{Z}_{t+1}$ 上评估距离-类别观测模型，可将式 (16.23) 的贝叶斯更新写为

$$\mathbf{h}_{t+1,i} = \mathbf{h}_{t,i} + \sum_{\mathbf{z} \in \mathbf{Z}_{t+1}} \mathbf{l}_i(\mathbf{z}), \quad (16.24)$$

其中， $\mathbf{l}_i(\mathbf{z})$ 是观测模型对数几率：

$$\mathbf{l}_i(\mathbf{z}) := \left[\log \frac{p(\mathbf{z} | m_i = 0)}{p(\mathbf{z} | m_i = 0)} \cdots \log \frac{p(\mathbf{z} | m_i = K)}{p(\mathbf{z} | m_i = 0)} \right]^\top. \quad (16.25)$$

要完整定义式 (16.24)，还需要具体观测模型。传感器射线会一直传播，直至撞到类别 $\mathcal{K} \setminus \{0\}$ 的障碍物或到达 $r_{\max}$ 。从位置 $\mathbf{p}$ 、方向 $\mathbf{R}$ 获得的带标签距离测量 $\mathbf{z} = (r, y)$ 表明：若测量终点 $\mathbf{p} + (r/r_{\max})\mathbf{R}\boldsymbol{\eta}$ 落在单元 m_i 中，则 m_i 被观测为占据；若 m_i 在射线上但不含终点，则观测为自由；若射线不与 m_i 相交，则不提供占据信息。因此，观测模型必须反映这些性质，可人工调节^[37]或从数据学习^[1165]。例如，^[37]将式 (16.25) 的观测模型对数几率向量参数化为沿射线的分段常数函数：

$$\mathbf{l}_i((r, y)) := \begin{cases} \boldsymbol{\phi}^+ + \mathbf{E}_{y+1}\boldsymbol{\psi}^+, & r \text{ 表示 } m_i \text{ 被占据,} \\ \boldsymbol{\phi}^-, & r \text{ 表示 } m_i \text{ 为空闲,} \\ \mathbf{0}, & \text{其他情况,} \end{cases} \quad (16.26)$$

其中 $\mathbf{E}_k := \mathbf{e}_k \mathbf{e}_k^\top$, $\psi^+, \phi^-, \phi^+ \in \mathbb{R}^{K+1}$ 是参数向量, 其首元素为零, 以确保 $\mathbf{l}_i(\mathbf{z})$ 是合法语义对数几率向量。图 16.12 展示了该观测模型下的贝叶斯多类别建图。

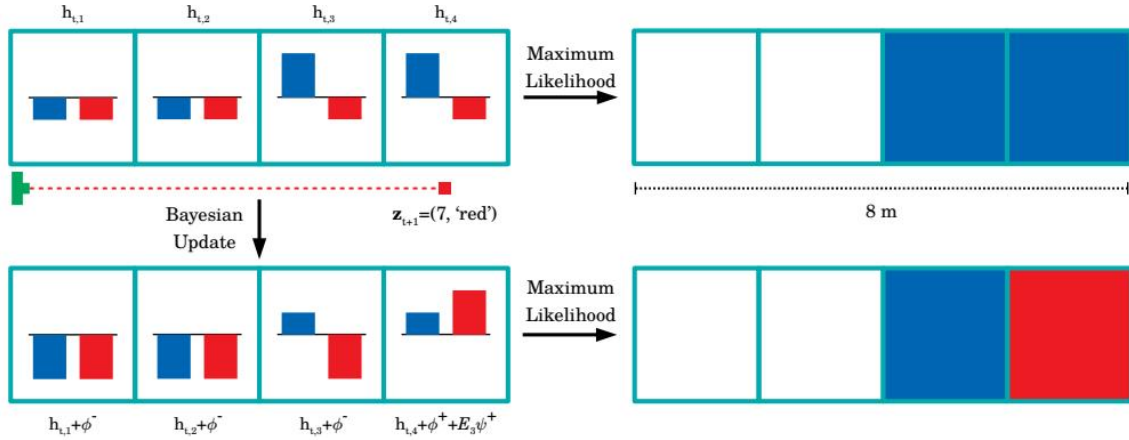


图 16.12: 图 16.12 使用距离-类别传感器测量进行贝叶斯多类别建图的示意, 类别为 $\mathcal{K} = \{\text{free}, \text{blue}, \text{red}\}$ 。左上显示时刻 t 沿观测射线每个单元的语义对数几率向量, 虚线为红色射线; 其对应基于对数几率的最大似然地图。距离-类别观测 $\mathbf{z}_{t+1}$ 在入射点给出 7 m 距离及 “red” 分类。下方显示时刻 $t+1$ 更新后的对数几率和各单元最大似然类别。

使用规则栅格离散化表示 $\mathcal{E}$ 具有高昂的存储和计算需求。真实环境中大量连续区域为空, 因而自适应离散化更有效。Hornung 等人^[478]提出的 OctoMap 使用八叉树数据结构合并具有相同占据概率的体素, 从而提高存储效率。八叉树每个节点表示部分物理环境, 每个节点有 0 个或 8 个子节点; 8 个子节点对应欧氏三维坐标系的 8 个卦限, 共同构成父节点空间的八叉划分。没有子节点的节点称为叶节点, 提供八叉树地图的最高分辨率可视化。

文献^[37]通过引入多类别八叉树, 将 OctoMap 的概率三维建图扩展为度量-语义表示。多类别八叉树的每个节点以语义对数几率向量 $\mathbf{h}_{i,t}$ 形式, 存储物体类别集 $\mathcal{K}$ 上的类别概率分布。要注册新观测, 先在八叉树上执行射线投射^[427, 16]找到被观测的叶节点。若观测需要更新某个叶节点, 则将其递归展开到最小分辨率, 并按式 (16.24) 更新后代节点的语义对数几率。

为了压缩八叉树, 还须定义从子节点向父节点融合信息的规则。不同应用可采用不同策略: 保守做法是把占据概率最高子节点的语义对数几率赋给父节点; 也可以将子节点对数几率向量的平均值赋给父节点, 这等于原概率空间中的贝叶斯信息融合。八叉树能够将相似单元 (叶节点) 合并为较大单元 (内部节点), 这一过程称为剪枝。每次将观测融合进地图后, 自底向上检查节点的剪枝机会; 若内部节点的所有子节点均为叶节点且语义对数几率相同, 则删除子节点并将内部节点变成具有相同语义对数几率的叶节点。这样会将大多数自由单元压缩为少量大单元; 占据单元通常不易剪枝, 因为传感器只观测到其表面, 内部仍是未知区域。

为实现实时性能, 多类别八叉树还需要额外设计。由于传感器噪声, 同一类别 (例如自由或被同一障碍物占据) 的单元不太可能得到完全相同的对数几率。可以为语义对数几率各元素设定上下限, 使单元在数值达到限值后进入稳定状态; 稳定单元更可能共享相同多类别概率分布, 因而更容易剪枝。阈值会在 $p_t(m_i = k) = 1$ 附近造成信息损失, 可通过上下限控制。多类别地图更新的空间和计算复杂度随类别标签数 K 线性增长, 而 K 增大时剪枝概率会指数下降。因此, 可以仅存储最可能的 $\bar{K}$ 个类别标签, 把其余标签汇入一个 “其他” 变量, 从而鼓励更多剪枝并显著降低多类别较多时的更新计算量。

多个机器人可以协同构建多类别八叉树地图。Riemannian Optimization for Active Mapping (ROAM)^[38]在图上构造优化问题: 节点变量表示不同机器人的八叉树地图, 共识约束要求机器人就局部地图达成一致。ROAM 基于分布式黎曼优化, 只依赖一跳通信即可实现共识并给出最优性保证。图 16.15 展示了 ROAM^[38]在真实数据上的多类别八叉树建图结果。

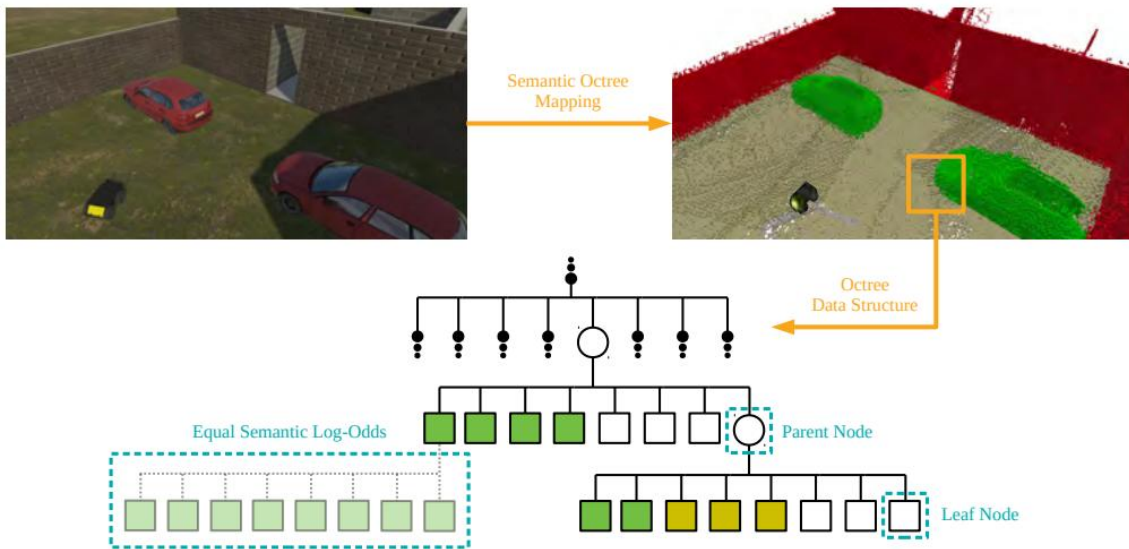


图 16.13: 图 16.13 多类别八叉树建图示例。左上为搭载距离-类别传感器的机器人建图环境；右上为生成的多类别八叉树地图，各类别以不同颜色表示（自由类别透明）；下方为地图局部对应的八叉树结构，圆形和方形节点分别表示父节点和叶节点。每个叶节点要么处于八叉树最小分辨率，要么包含语义对数几率向量相同的子节点。

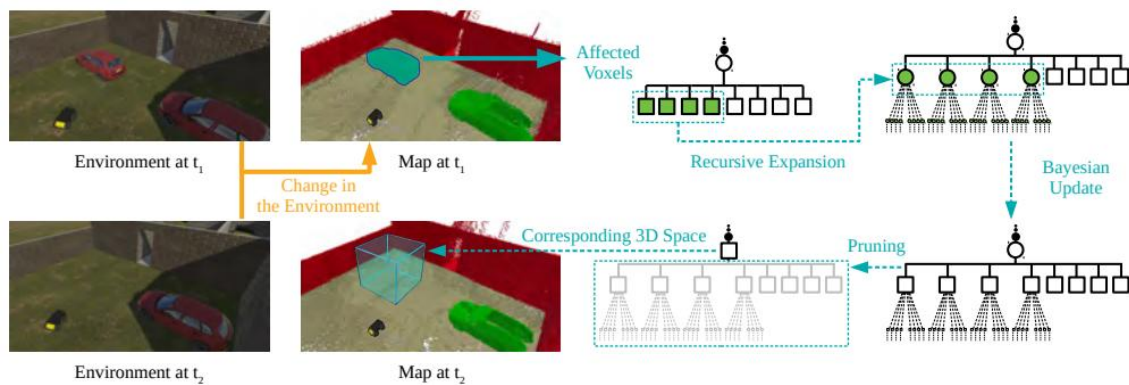


图 16.14: 图 16.14 多类别八叉树地图更新。 $t = t_1$ 到 $t = t_2$ 之间环境发生变化，从而触发地图更新。通过射线投射确定青色高亮体素需要更新，八叉树中对应节点被递归展开，并依据式 (16.24) 更新；同时检查各父节点子节点的剪枝机会。剪枝生成更少但物理尺寸更大的八叉树节点，如 $t = t_2$ 地图中的青色立方体。

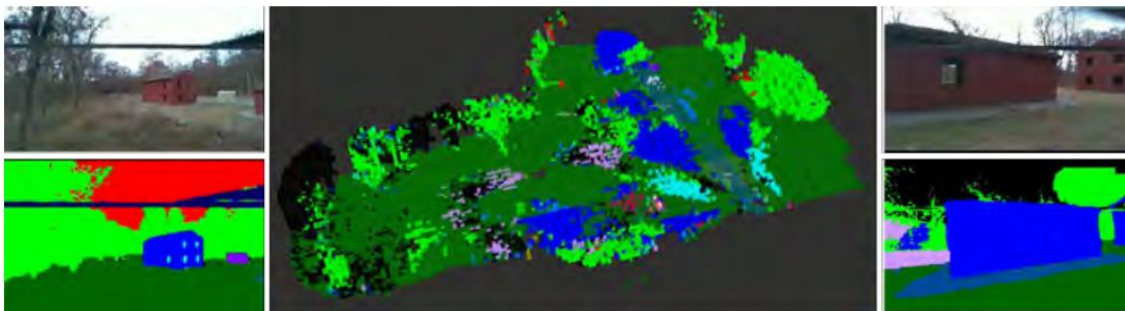


图 16.15: 图 16.15 真实 SLAM 实验的定性结果：两架四旋翼机器人使用 ROAM 算法^[38]构建多类别八叉树地图。左、右分别为两台机器人机载 RGB-D 相机获得的 RGB 图像和语义分割图，中间为生成的多类别八叉树地图。

16.3.3 基于网格的度量-语义 SLAM

体素地图虽然能有效地随时间概率融合信息，但有两个缺点：第一，存储代价较高，尽管可通过八叉树部分缓解；第二，难以随回环闭合编辑。回环闭合会使机器人轨迹和相应地图估计发生大变形，体素地图对此更新代价很高，通常需要将过去观测反积分后再重新积分^[237]。网格表示可以同时回避这两项问题。网格由多边形（通常是三角形）组成，存储复杂度随所表示场景的复杂度而变化。常见做法是先建立体素表示，再用 marching cubes 等标准算法计算网格。更高级的方法^[943]只在机器人周围的空间窗口建立体素地图，并在体素离开活动窗口时增量转换为网格。

更重要的是，网格表示可以在回环闭合时高效变形，因此比体素反积分与重积分更高效地校正地图。这里回顾 Pose-Graph and Mesh Optimization (PGMO)^[943]。PGMO 基于嵌入式形变图概念，该概念最初用于计算机图形学中的形状操作^[1049]。形变图由对原网格下采样得到的控制节点组成，每个控制节点附带局部坐标系；通过求解保持图边局部刚性的优化问题使整个图变形。计算出控制节点的形变后，再用控制节点的新配置插值回原网格。

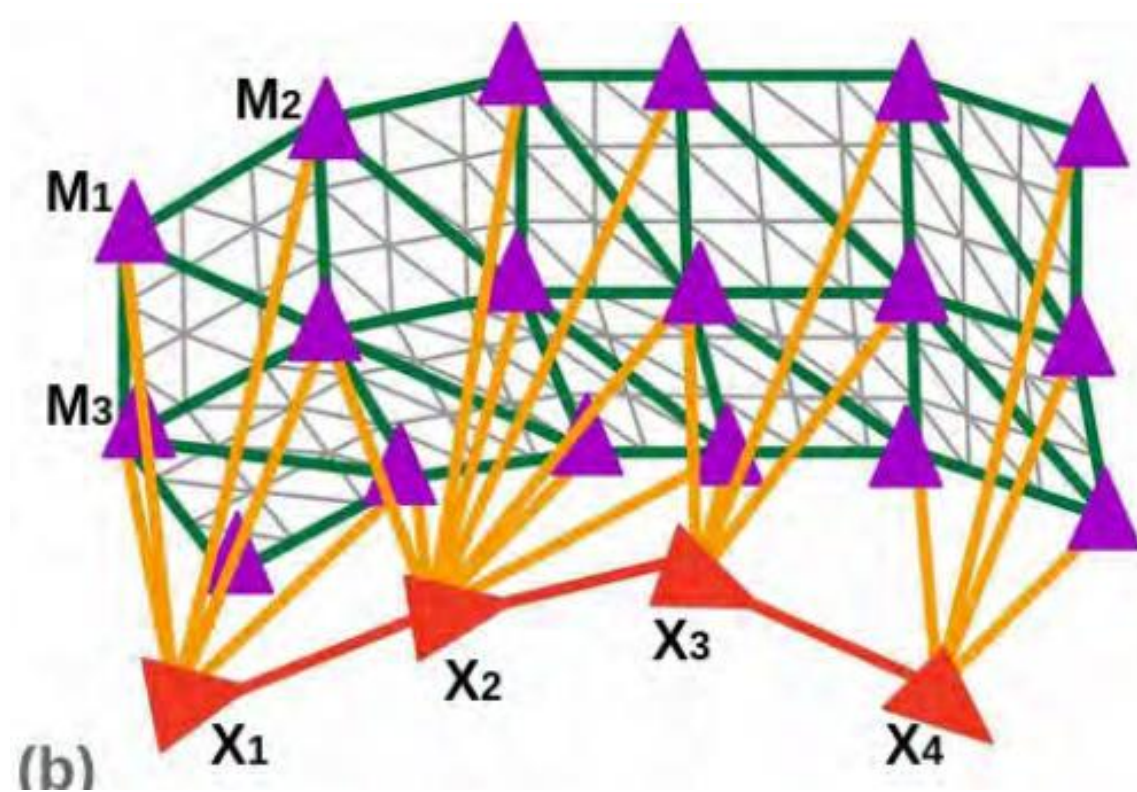
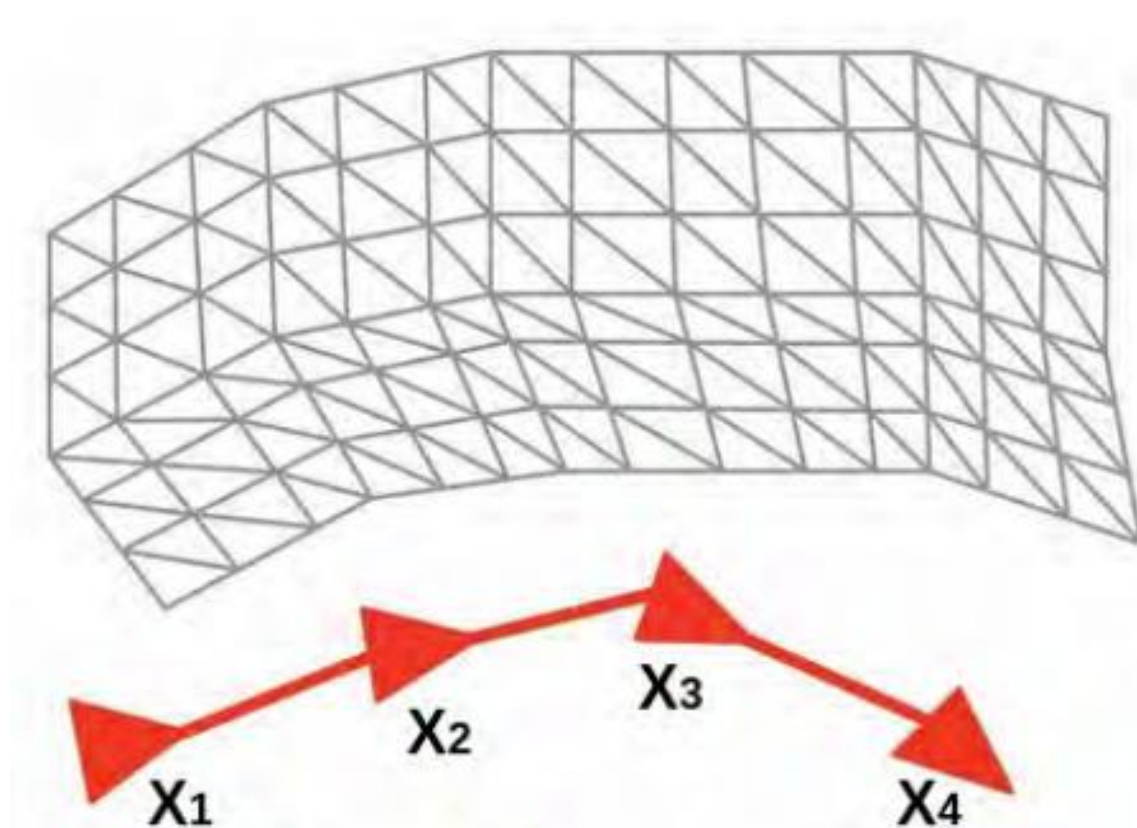
对稠密三维度量-语义 SLAM，可由机器人位姿图和稠密网格的下采样版本创建形变图。形变图的位姿顶点就是机器人位姿图节点，变换为 $\mathbf{X}_i = [\mathbf{R}_i^X, \mathbf{t}_i^X]$ ，边连接关系继承位姿图。形变图的网格顶点来自下采样网格，网格顶点 k 的变换为 $\mathbf{M}_k = [\mathbf{R}_k^M, \mathbf{t}_k^M]$ ，边连接关系继承简化网格。若网格顶点 k 对应传感器在位姿顶点 i 处可见，则在二者间连接一条边。

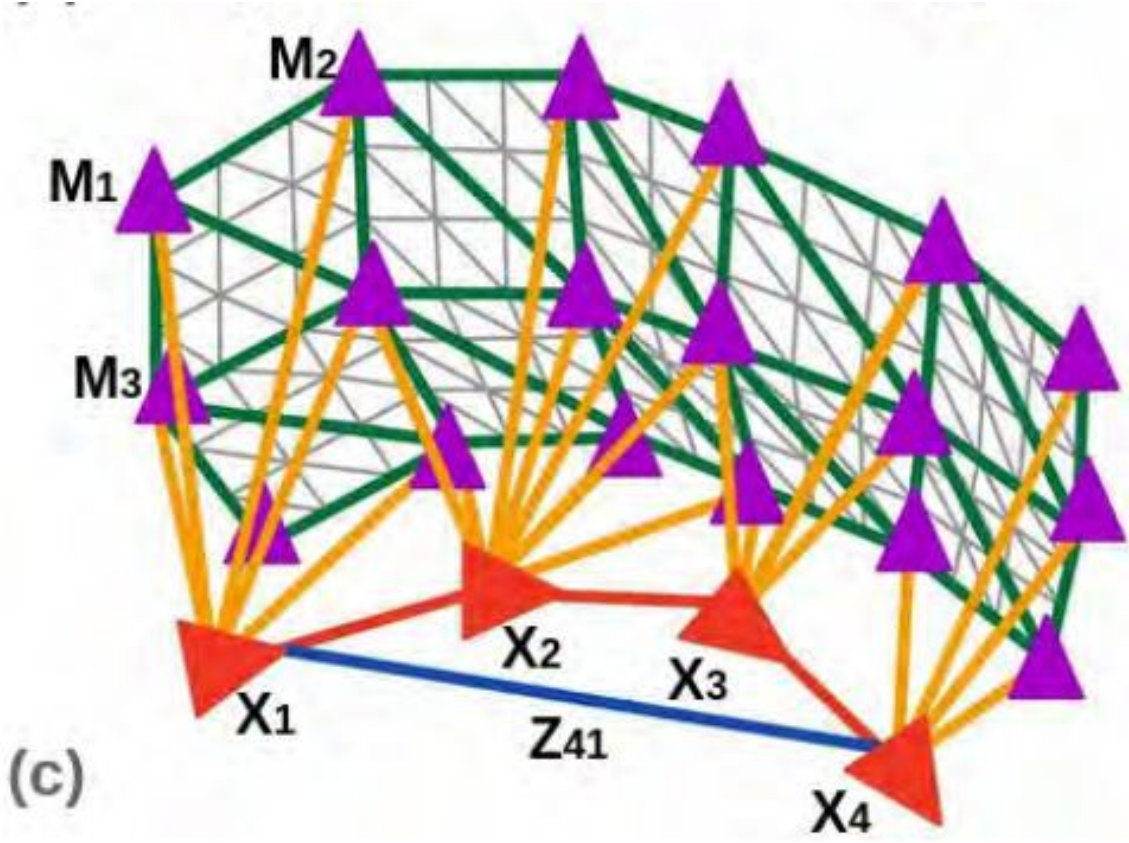
在未变形状态中， $\mathbf{X}_i$ 是节点 i 未优化的位姿， $\mathbf{R}_k^M = \mathbf{I}_3$ ， $\mathbf{t}_k^M = \mathbf{g}_k$ ，其中 $\mathbf{g}_k$ 是顶点 k 原始世界坐标位置。直观地说， $\mathbf{R}_k^M$ 表示以顶点 k 为中心的局部旋转， $\mathbf{t}_k^M - \mathbf{g}_k$ 表示局部平移。检测到回环等触发形变后，优化问题为

$$\begin{aligned} \min_{\mathbf{X}_{1:n}, \mathbf{M}_{1:m} \in \text{SE}(3)} & \sum_{i,j} \|\mathbf{X}_i \mathbf{Z}_{ij} - \mathbf{X}_j\|_{\Omega_{ij}}^2 \\ & + \sum_{k=0}^m \sum_{l \in \mathcal{N}^M(k)} \|\mathbf{R}_k^M (\mathbf{g}_l - \mathbf{g}_k) + \mathbf{t}_k^M - \mathbf{t}_l^M\|_{\Omega_{kl}}^2 \\ & + \sum_{i=0}^n \sum_{l \in \mathcal{N}^M(i)} \|\mathbf{R}_i^X \tilde{\mathbf{g}}_{il} + \mathbf{t}_i^X - \mathbf{t}_l^M\|_{\Omega_{il}}^2. \end{aligned} \quad (16.27)$$

这里 $\mathcal{N}^M(i)$ 是形变图中顶点 i 的相邻网格顶点， $\mathbf{g}_i$ 是形变图网格顶点的初始位置， $\tilde{\mathbf{g}}_{il} = \tilde{\mathbf{R}}_i^\top (\mathbf{g}_l - \mathbf{g}_i)$ 是顶点 l 在节点 i 里程计位姿局部坐标系中的未变形位置。式 (16.27) 第一项是通用 PGO，第二项通过惩罚相连网格顶点间的变形保持局部刚性，第三项保持位姿顶点和网格顶点间的局部刚性。矩阵范数为平方加权 Frobenius 范数：

$$\|\mathbf{A}\|_{\Omega}^2 = \text{tr}(\mathbf{A} \Omega \mathbf{A}^\top), \quad \Omega = \begin{bmatrix} \omega_R \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & \omega_t \end{bmatrix}, \quad (16.28)$$





其中 $\omega_R, \omega_t \geq 0$; 对向量, $\|v\|_{\Omega}^2 = v^T \Omega v$ 是通常的平方 Mahalanobis 范数。依照^[943], 定义

$$G_{ij} = \begin{bmatrix} I_3 & g_j - g_i \\ \mathbf{0} & 1 \end{bmatrix}, \quad \bar{G}_{ij} = \begin{bmatrix} I_3 & \tilde{g}_{il} \\ \mathbf{0} & 1 \end{bmatrix}, \quad (16.29)$$

便可把问题写为

$$\min_{\mathbf{x}_{1:n}, \mathbf{M}_{1:m} \in \text{SE}(3)} \sum_{Z_{ij} \in \mathcal{Z}} \|X_i Z_{ij} - X_j\|_{\Omega_{Z_{ij}}}^2 + \sum_{G_{ij} \in \mathcal{G}} \|M_i G_{ij} - M_j\|_{\Omega_{G_{ij}}}^2 + \sum_{\bar{G}_{ij} \in \bar{\mathcal{G}}} \|X_i \bar{G}_{ij} - M_j\|_{\Omega_{\bar{G}_{ij}}}^2. \quad (16.30)$$

$\mathcal{Z}$ 是位姿图边集合 (图 16.16 中红、蓝边), $\mathcal{G}$ 是简化网格边集合 (绿边), $\bar{\mathcal{G}}$ 是连接位姿顶点与网格顶点的边集合 (黄边)。第二、三项只优化平移, 因此 $\Omega_{G_{ij}}$ 与 $\Omega_{\bar{G}_{ij}}$ 的旋转部分为零。进一步将位姿或网格顶点 i 的变换统记为 T_i , 其边变换统记为 E_{ij} , 便得到经典 PGO:

$$\min_{T_{1:n+m} \in \text{SE}(3)} \sum_{E_{ij}} \|T_i E_{ij} - T_j\|_{\Omega_{ij}}^2. \quad (16.31)$$

优化后, 利用形变图中距离最近的 m 个控制节点仿射变换更新每个网格顶点:

$$\tilde{\mathbf{v}}_i = \sum_{j=1}^m w_j(\mathbf{v}_i) [\mathbf{R}_j^M (\mathbf{v}_i - \mathbf{g}_j) + \mathbf{t}_j^M], \quad (16.32)$$

其中 $\mathbf{v}_i$ 是原位置, $\tilde{\mathbf{v}}_i$ 是第 i 个网格顶点变形后的位置。控制节点权重为

$$w_j(\mathbf{v}_i) = \frac{\bar{w}_j(\mathbf{v}_i)}{\sum_{k=1}^m \bar{w}_k(\mathbf{v}_i)}, \quad \bar{w}_j(\mathbf{v}_i) = \left(1 - \frac{\|\mathbf{v}_i - \mathbf{g}_j\|}{d_{\max}}\right)^2, \quad (16.33)$$

其中 $d_{\max}$ 是该网格顶点到 m 个控制节点中最远节点的距离。

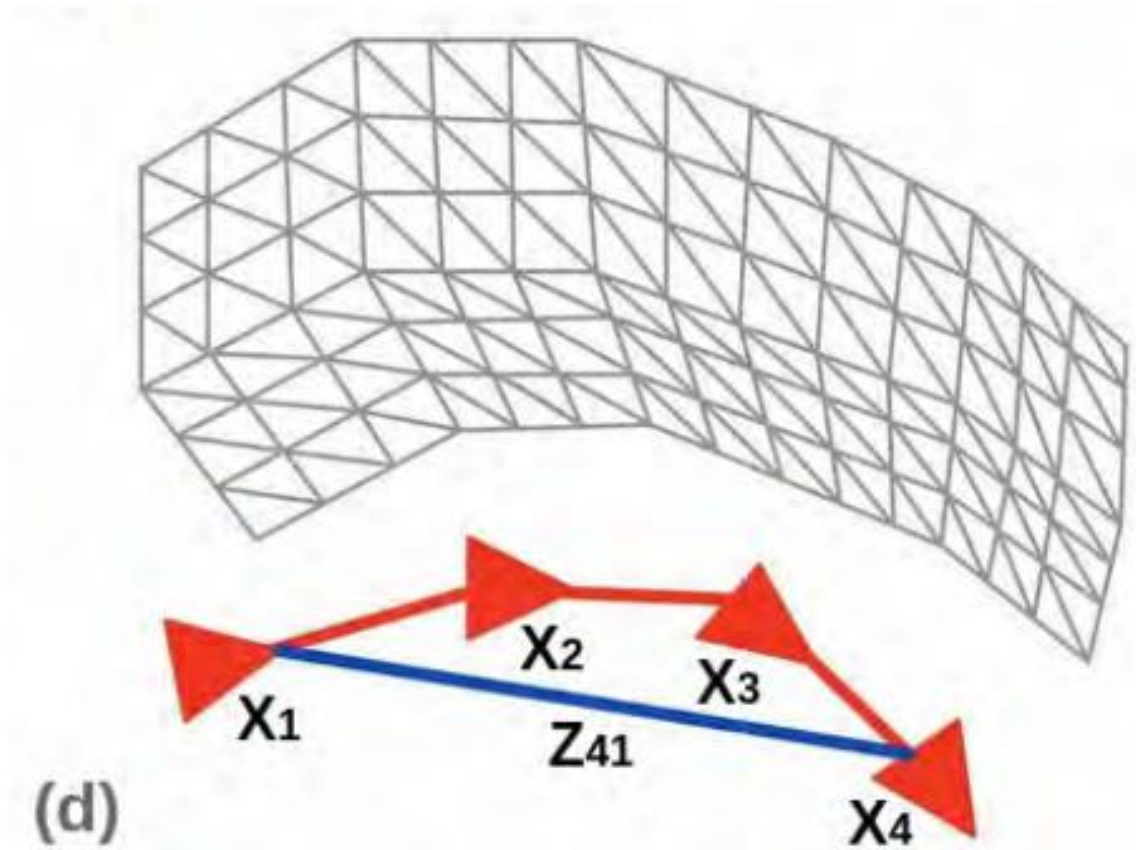


图 16.16: 图 16.16 PGMO 形变。(a) 未变形位姿图和网格；(b) 创建形变图，位姿顶点为红色，网格顶点为紫色，绿色边表示简化网格的连接关系，黄色边按传感器可见性连接位姿和网格顶点；(c) 回环闭合触发位姿图和网格优化，依据结果更新 X_i 和 M_i 并使稠密网格变形；(d) 得到的变形网格和位姿图。

16.4 分层度量-语义表示与三维场景图

第 16.2 节和第 16.3 节分别讨论了将语义信息编码到机器人地图中的稀疏表示和稠密表示。二者各有优势：稀疏表示便于操作物体，稠密体积表示则擅长编码自由空间和可通行性。本节介绍将稠密与稀疏地图组合为统一模型的分层表示。首先讨论空间表示和语义类别的选择如何影响机器人所需内存；随后说明组织信息的层次结构能够节省内存和计算；最后以三维场景图为例，介绍一种度量-语义分层地图。

16.4.1 分层表示与符号落地

先回顾度量-语义建图与符号落地问题之间的关系^[431]。到目前为止，本章主要考虑由学习模型从相机图像或 LiDAR 扫描提供的封闭集语义类别。这些特征是符号的落地，因为它们把对人类具有特定含义的概念¹关联到机器人传感器数据。人类发出的“去拿椅子”等高层指令本质上涉及符号；机器人要正确执行，就必须将“椅子”这一符号落地到其所在的物理位置，这就是符号落地问题。

原则上，可以设计感知系统，直接在带位姿传感器数据中落地符号。例如，相机机器人可将图像像素映射为“椅子”“家具”“厨房”和“公寓”等符号²。但直接在传感器数据中落地符号不可扩展：图像等数据以高频采集，存储代价大，不适合长期运行。因此需要中间（或亚符号）表示，将原始传感器数据压缩为更紧凑的格式，用于落地符号。本章前述内容关注的正是几何亚符号表示，以及将语义信息落地到这些表示的算法。

层次结构与内存。在亚符号表示中落地符号的最直接策略，是对场景中关注的 L 个符号，在每个体素中存储其是否存在。分辨率为 δ 、体积为 V 的体素网格需要

$$m = O\left(L \cdot \frac{V}{\delta^3}\right). \quad (16.34)$$

图 16.17 左图展示了该策略：每个体素最多存储 L 个语义属性。室内环境中关注的符号通常具有层次结构，例如物体、房间和楼层。与其在每个体素中稠密且冗余地存储符号，不如根据层次组织表示：构造一个图，节点表示物体、房间或建筑，边表示包含关系；建筑节点的子节点是其中的房间，房间节点的子节点是其中的物体。此时内存需求降为

$$m = O\left(\frac{V}{\delta^3} + N_{\text{objects}} + N_{\text{rooms}} + \cdots + N_{\text{buildings}}\right). \quad (16.35)$$

该压缩是无损的，原始稠密体素网格可以从分层表示中恢复^[496]。若采用八叉树或自由空间近似聚类更节省内存的亚符号表示，需求还可降低为

$$m = O(N_{\text{sub-sym}} + N_{\text{objects}} + N_{\text{rooms}} + \cdots + N_{\text{buildings}}), \quad (16.36)$$

其中 $N_{\text{sub-sym}}$ 通常远小于 V/δ^3 。这种压缩通常有损：即便符号部分可恢复，原始体素亚符号表示也未必能完全恢复^[496]。图 16.17 右图展示了将自由空间聚类为地点拓扑图的例子。

层次结构与推断。分层表示除节省内存外，还具有有利于计算的性质。定义图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 为分层图，若顶点可划分为 l 层 $\mathcal{V} = \bigcup_{i \in 1:l} \mathcal{V}_i$ ，并满足：

1. **单父节点**：对于层 $\mathcal{V}_i$ 的子节点 v 和层 $\mathcal{V}_{i+1}$ 的父节点 u ，若 $(v, u) \in \mathcal{E}$ ，则不存在另一条 (v, u') 且 $u' \in \mathcal{V}_{i+1}$ 的边。
2. **局部性**：层间边只能连接相邻层；若 $u \in \mathcal{V}_i$ 、 $v \in \mathcal{V}_j$ 且 $|i - j| > 1$ ，则不存在 $(u, v) \in \mathcal{E}$ 。
3. **子节点不相交**：对任意同层节点 $u, v \in \mathcal{V}_{i+1}$ ，其子节点集合 $\mathcal{C}(u)$ 和 $\mathcal{C}(v)$ 不相交，即不存在连接二者子节点的边。

节点 $s \in \mathcal{V}_{i+1}$ 的子节点集合定义为 $\mathcal{C}(s) = \{t : t \in \mathcal{V}_i \wedge (s, t) \in \mathcal{E}\}$ 。该定义意味着，图中任意节点的后代树都与同层其他节点的后代树不相交。Hughes 等人^[496]表明，这一性质允许存在分层树分解，并给出任意分层图树宽的上界：

$$\max_{v \in \mathcal{V}} \text{tw}[\mathcal{G}[\mathcal{C}(v)]] + 1. \quad (16.37)$$

其中 $\mathcal{G}[\mathcal{C}(v)]$ 是由 v 的子节点形成的子图， $\text{tw}[\mathcal{G}]$ 是图树宽。树宽影响图上推断的计算复杂度，也是许多图问题的复杂度度量^[93, 247, 320, 403]。图模型推断一般为 NP-hard^[230]，但若树宽较小，则可以采用高效的

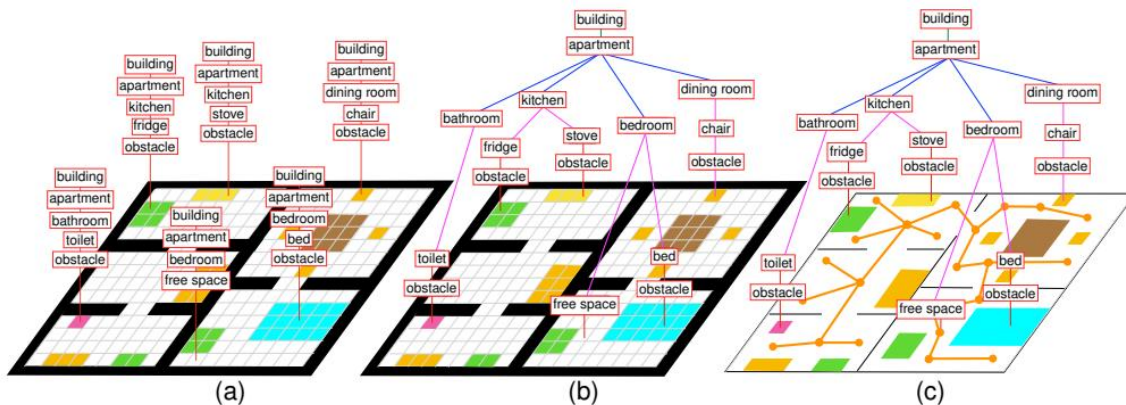


图 16.17: 图 16.17 不同亚符号表示中所关注符号的落地。(a) 每个体素存储语义属性；(b) 按物体、房间和建筑层次组织；(c) 使用更紧凑的亚符号表示，并将自由空间聚类为地点。

多项式时间推断算法。式 (16.37) 因此表明，只要后代子图的树宽保持较小，便能在分层图上高效推断；实践中，三维场景图这一大类分层图通常满足该条件^[496]。

分层地图并非新概念，机器人学自诞生以来就一直使用分层地图^[612, 172, 1090]。早期工作关注二维地图，研究用分层地图弥合度量表示和拓扑表示之间的差异^[956, 355, 1266, 213, 65]；这些工作早于“深度学习革命”，无法利用如今深度神经网络提供的丰富语义。

16.4.2 三维场景图

不同研究社区对三维场景图的定义略有差异。Armeni 等人^[36]首次将三维场景图作为分层地图表示，引入包含物体和房间的语义与空间信息。Kim 等人^[577]独立提出由物体及其关系组成的三维场景图，扩展了二维场景图。Wald 等人^[1141]从稠密场景重建中构造类似的物体关系三维场景图，并最终将其纳入在线 SLAM 系统^[1198]，为^[577]增加隐式层次。Rosinol 等人^[943]提出同时包含动态实体（如人）、物体和自由空间（称为地点）的三维场景图。Hughes 等人^[496]构建了首个在线生成完整分层场景图的系统，其中包含物体、地点和房间，并能够校正三维场景图使其全局一致。图 16.18 展示了^[496]生成的三维场景图示例。

形式上，三维场景图是图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ ，每个节点 $v \in \mathcal{V}$ 关联节点特征或属性 $\mathbf{x}_v$ ，每条边 $e \in \mathcal{E}$ 关联边特征或属性 $\mathbf{x}_e$ 。每个三维场景图都将符号落地到空间，因此节点特征假定包含位置信息³。通常可将场景图视为分层图，将 $\mathcal{V}$ 划分为 l 层；物体及其关系的三维场景图^[577, 1141, 1198]可作为更复杂分层表示中的一层。

三维场景图通常从稠密度量表示^[1141, 1198]或度量-语义表示^[36, 943, 496]通过聚类或分割构造物体，形成物体符号的稀疏、压缩落地，其计算通常比底层稠密表示更易处理。第 496 篇工作中的地点或房间层也是自由空间的稀疏表示。层间边（不同层节点之间的边）常与房间和地点层结合使用，以编码空间包含关系，例如房间与某个物体之间的边表示物体位于房间内，从而支持分层图匹配和高效推断^[496]。边还可以编码“位于……之上”或“邻近”等空间关系^[1198]，且关系可能有方向性。左/右、前/后等二维场景图常用关系依赖视点，在三维场景图中通常不够有意义。

因子图与三维场景图。回环闭合后，高效校正地图表示非常重要。Hughes 等人^[496]扩展第 16.3.3 节的 PGM，同时优化三维场景图与底层稠密表示：除位姿图和网格控制点外，将地点层加入式 (16.27) 并联合优化，为网格控制节点之间的因子提供结构先验。优化结果用于插值和细化其他层，而不必从头重建。另一类方法^[61]在三维场景图不同层之间加入额外因子，从而直接优化整个场景图以校正漂移，无需后续细化；但此时优化问题不再能写成 PGO。

图 16.18: 图 16.18^[496] 生成的三维场景图示例。

16.5 延伸阅读与近期趋势

多模态隐式地图、任务规划与行动模型。一个新兴趋势是将度量-语义建图推广到几何、光度和语义表示之外，编码大型视觉-语言模型（VLM）的特征。近期方法编码高维、语言落地的特征，使机器人能够推理物体可供性和机器人任务。嵌入 CLIP 特征^[899]等语言特征，可以用开放词汇定位物体，为构建更灵活的机器人系统开辟新路径。为 NeRF 和 Gaussian Splatting 等神经场景表示加入语言嵌入也受到广泛关注^[555]。例如，Online Language Splatting^[543]将 CLIP 特征融合进 Gaussian splat，实时构建稠密、具有语言感知能力的场景表示，支持下游查询和落地任务；相关方法将在下一章讨论。

将语义地图中的信息转化为行动^[829, 617]是另一项近期发展。ATLAS Navigator^[829]主动构建嵌入语言特征的空间地图，以支持导航到自然语言描述的物体目标。M3^[1313]引入持久化三维空间多模态记忆，存储与 VLM 对齐的特征，并支持面向长时域任务的基于检索的推理。LUMOS^[798]将世界建模与语言条件模仿学习结合，使机器人能够在 VLM 落地的潜在空间中模拟、想象和规划。GNFactor^[1265]则通过编码可指导控制的丰富感知先验，使可泛化神经特征场连接多任务机器人学习与现实泛化。这些工作体现了向可行动表示的转变：空间记忆、语言和多模态感知紧密耦合，以支持任务规划和技能执行。

然而，这类地图通常作为后处理步骤构建，如何增量构建此类表示仍是开放挑战^[543, 657]。从多个变化视点聚合具有歧义的信息，同时保持一致并区分实例，仍然十分困难。将这些表示与不确定性结合可能是前进方向，能够以原则化方式建模歧义。

三维场景图的前沿。三维场景图已经用于规划^[17, 905, 913]、操作^[520]、地图压缩^[166]、预测^[393, 693]、回环检测^[496]和定位^[530]等问题。当前研究沿多个方向扩展三维场景图。第一，将其扩展到非结构化和室外场景仍是活跃领域，因为通用室外环境中的场景图层次更难定义和构建^[73, 1037]。第二，场景图通常由人预先硬编码，而理想情况是机器人根据任务自动重组层次表示；早期工作已开始研究更灵活、任务驱动的三维场景图^[723, 168]。第三，将语言或语言嵌入引入三维场景图，突破封闭集语义^[1182, 407, 723, 168]；开放集语义建图将在下一章详细讨论。第四，为场景图增加运动、动态性和可供性等语义信息^[1268]。最后，传统 SLAM 中已有成熟的不确定性量化方法（参见第 6 章），但对混合离散和连续变量的分层表示进行不确定性量化，仍是很少被探索的问题。

多机器人语义理解。本章主要关注单机器人度量-语义 SLAM，但近期研究也将这些表示扩展到多机器人团队：多个机器人探索未知环境并构建统一的度量-语义地图。由于信息以分布式方式采集，机器人间共享数据会形成通信瓶颈；采集数据量和问题规模也可能超出单个机器人的处理能力，需要强大的中央服务器或分布式协作。多机器人 SLAM 还缺少所有机器人位姿的可靠初始猜测，因为不同机器人可能使用不同坐标系，这对鲁棒性和离群值剔除提出额外挑战（参见第 3、6 章）。Kimera-Multi^[1095]是首个分布式多机器人度量-语义 SLAM 方法；^[1092]对集中式和分布式 SLAM 的权衡进行了详细实验分析。近期工作还将三维场

景图构建扩展到多机器人团队^[167, 325]。

¹ 例如,“椅子”一词是一个符号,因为人类理解椅子的含义。² 稠密二维语义分割网络就是这种策略的例子。³ ^[496] 给出的树宽界还包含顶层 l 的树宽项;这里为简洁起见假设 $\mathcal{V}_l$ 仅包含一个节点。⁴ 不同领域可能采用不同建模选择:室内三维场景图自然使用 $\mathbb{R}^3$,而面向自动驾驶的场景图^[396]可能更适合使用纬度、经度和海拔。

第十七章 迈向开放世界空间智能

作者：Liam Paull、Sacha Morin、Dominic Maggio、Martin Büchner、Cesar Cadena、Abhinav Valada 和 Luca Carlone

前面各章介绍了多种地图表示：将世界建模为稀疏路标集合的稀疏表示（第 1 章）、存储三维空间详细几何信息的稠密表示（第 5 章），以及试图灵活平衡二者的分层表示（如三维场景图，第 16 章）。这些表示还依赖输入传感器模态，例如视觉（第 7 章）、LiDAR（第 8 章）和雷达（第 9 章）。然而，地图中包含的信息大多仍限于环境的几何属性（或第 14 章所述的光度属性）。

第 16 章展示了现代 SLAM 可以超越几何、编码语义，从而支持某种形式的空间智能，即更抽象、更高层次的推理。这使机器人能够执行以更直观方式指定的复杂任务，例如用高层自然语言而非低层几何指令指定目标。此类表示通常由深度学习训练的感知模型查询得到，但第 16 章中的模型多以封闭集方式训练，潜在类别、物体类型和概念的字典在训练时预先确定，因而固定且有限。

近年来出现了在海量数据集上训练的“基础模型”（foundation model）。它们能够学习开放世界（也称开放集或开放词汇）表示，潜在地泛化到可能遇到的任何类别、物体类型或概念。将 CLIP^[899] 等多模态基础模型的表示嵌入 SLAM 地图，可以用全新的方式交互和查询地图元素，如图 17.1 所示。



图 17.1: 图 17.1 将 CLIP 等多模态基础模型的表示嵌入 SLAM 地图后，可以用文本查询地图。例如查询“一个可以舒适地坐着看电视的地方”，将文本送入文本编码器，再查找与查询嵌入在余弦距离意义下足够接近的地图点，便可找到椅子和沙发。只要不同模态的基础模型具有对齐的嵌入空间，同一方法也适用于其他模态；示例改编自 ConceptFusion^[512]。

本章探讨如何在 SLAM 中利用基础模型。第 17.1 节介绍基础模型研究背景和基本术语；第 17.2 节回顾

多模态基础模型；第 17.3 节讨论基础模型和开放集语义在 SLAM 与空间智能中的近期应用；第 17.4 节总结近期趋势。

17.1 背景与术语

17.1.1 基础模型简史

基础模型是用足够海量的数据训练的机器学习模型，通常可以泛化到广泛运行条件。训练此类模型需要两项关键条件：能够高效更新大量参数的深度学习架构，以及用于训练的超大规模（例如互联网规模）数据集。计算机视觉中最早可能符合这一描述模型是 AlexNet^[608]：它是一个在 ImageNet^[260] 上以监督方式训练的 CNN，数据集包含 320 万幅人工标注图像。传统 CNN 的问题是梯度消失：梯度通过许多层后变得很小，基于该梯度训练参数的效率很低。

残差网络^[443] (ResNet) 通过在层之间引入直接通路，使训练更稳定、高效，缓解了这一问题。基于 ImageNet 预训练 ResNet 的视觉编码器成为闭集视觉感知系统的基础，例如 YOLO^[919]、RCNN^[384] 等物体检测模型。

构建 ImageNet 这样的人工标注数据集十分艰巨。自然语言处理领域则可以自动化标注过程。BERT^[268] 使用语料库自动生成训练任务和标签以学习有意义表示：从句子中删除一个词，再预测缺失词。英语 Wikipedia (约 25 亿词) 和 BookCorpus (约 8 亿词) 是最早使用的语料库，这一过程通常称为无监督学习¹。BERT 还展示了基于注意力机制^[43] 的 Transformer 架构^[1123] 对捕获长期依赖的优势；同期发布的 GPT 也基于 Transformer，并展现了类似能力。

将这种无监督训练扩展到图像等高维输入并不直接，因为词语具有明确语义，而随机图像块未必具有明确含义。结合视觉 Transformer 的对比学习方法^[280] 开始解决这一问题。SimCLR^[185] 等方法通过裁剪、翻转或删除图像块生成增强图像，并构造损失，使原图和增强图在表示空间中接近（正样本），而与其他输入远离（负样本），从而学习对增强不变、适合下游视觉任务的表示。

与此同时，研究者开始将 Transformer 应用于视觉领域^[280]。视觉 Transformer 将图像分割为固定大小的图像块，与位置嵌入一起线性嵌入，并将向量序列输入标准 Transformer 编码器；分类时额外加入可学习的分类 token。

DINO (无标签自蒸馏)^[154] 使用教师-学生蒸馏而非对比学习训练视觉 Transformer 编码器，在学习视觉表示方面非常有效。对比学习还能够对齐不同模态的表示空间：CLIP^[899] 使用互联网图像-文本对，通过对比损失使配对图像和文本在特征空间中接近，而使不匹配样本远离。海量数据、模型和工作流的改进不断增强基础模型对未见任务和场景的泛化能力，但 LiDAR、雷达等 SLAM 常用模态尚缺少同等规模的公开数据。

17.1.2 术语与范围

本章关注的关键区别是“开放世界”。一种范式是在 SLAM 前端使用开放世界感知系统，例如 YOLO-World^[194] 和 OV-DETR^[1264]。它们允许开放词汇文本条件查询，但查询必须在感知模型运行、地图构建时指定。因此地图本身仍是封闭世界：地图表示哪些概念在创建时已经确定。第二种范式是将开放世界表示嵌入 SLAM 地图，使地图在使用时支持开放世界查询；本章第 17.3 节主要关注这一范式。

还需区分定位、建图与 SLAM。基础模型表示已用于视觉地点识别和定位/回环检测，例如 AnyLoc^[547] 研究从 DINO^[154] 的不同层提取并聚合特征，在空间、时间和视点变化下构造视觉地点描述子。本章更直接地关注支持开放世界查询和任务指定的数据结构与算法。后续讨论的大多数方法是纯建图算法，假设已有带位姿的相机帧（例如由前面各章的 SLAM 系统提供），问题转化为如何将基础模型表示嵌入 SLAM 地图。

17.2 面向空间智能的基础模型

本节介绍有助于构建开放世界地图表示的三类基础模型，并区分基于特征和生成式基础模型。CLIP^[899] 等基于特征的模型从文本或图像中提取实值向量，通过特征空间计算完成任务，例如用两个向量之间的距离

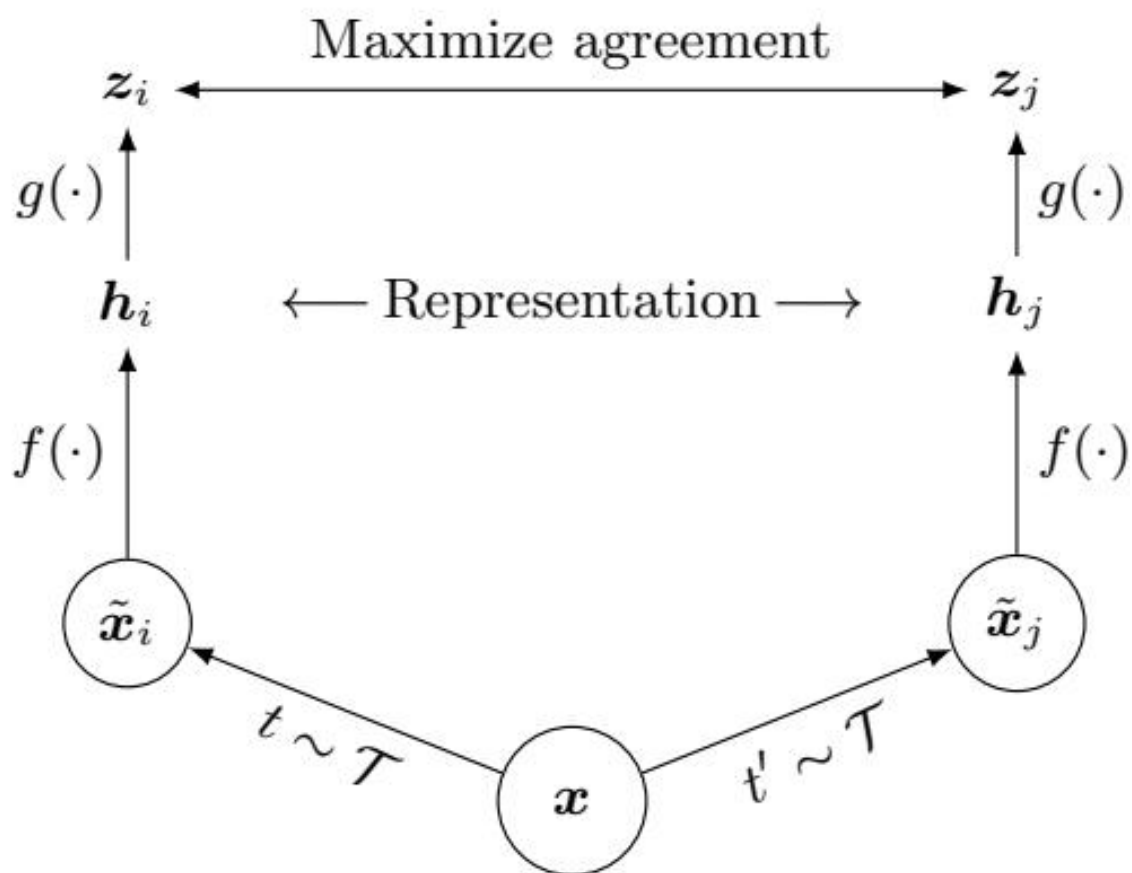


图 17.2: 图 17.2 对比学习示意 (改编自^[185])。输入 x 经两个增强算子 $t \sim \mathcal{T}$ 和 $t' \sim \mathcal{T}$ 产生相关视图, 再由编码器 $f(\cdot)$ 和投影头 $g(\cdot)$ 得到表示, 通过对比损失最大化二者的一致性。

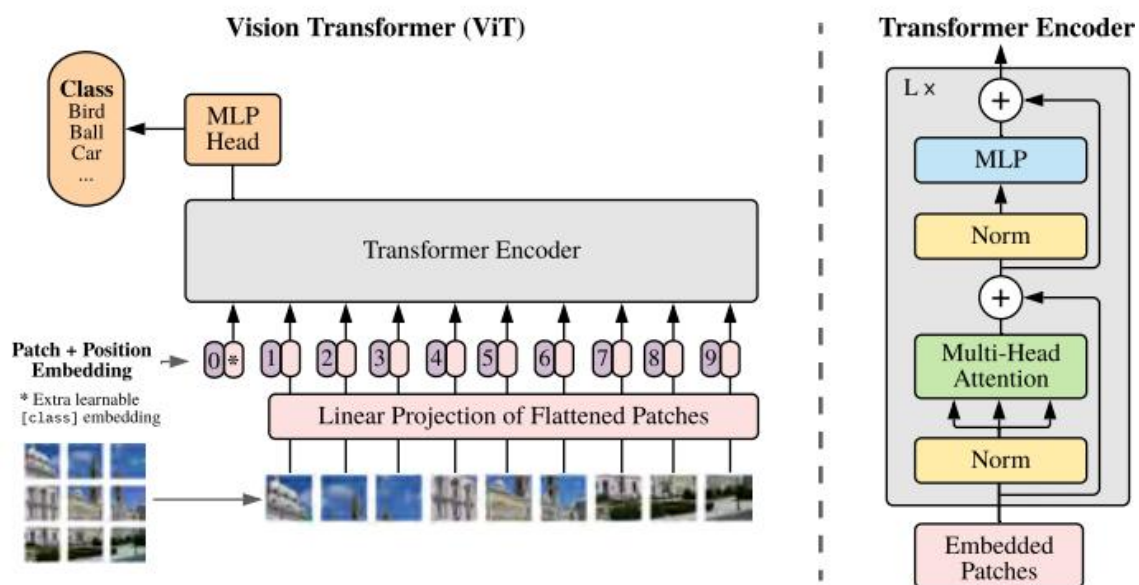


图 17.3: 图 17.3 视觉 Transformer 结构 (改编自^[280])：图像块和位置嵌入经过多头注意力及 MLP 编码, 分类 token 由 MLP 头输出类别。

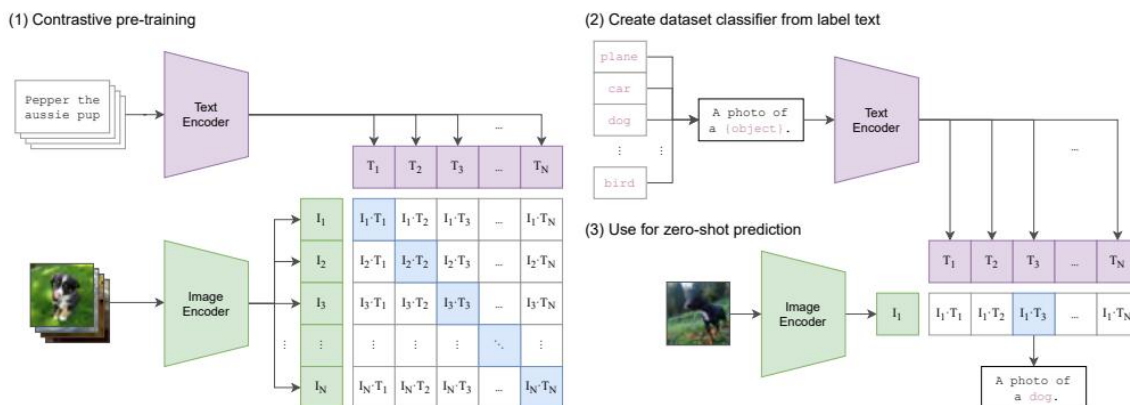


图 17.4: 图 17.4 CLIP 训练和零样本预测 (改编自 [899]): (1) CLIP 以对比学习得到图像和文本特征; (2) 训练后将类别文本标签编码为分类器; (3) 将图像特征与类别特征比较, 实现零样本预测。

衡量文本或图像相似性。生成式基础模型则根据文本和/或图像提示生成新数据。二者并非完全互斥: 基于特征的模型可作为生成式模型的子模块, 生成式模型也能提取特征; 但在地图应用中, 输出形式和典型用途支持这种区分。

17.2.1 基于特征的基础模型

特征也称为嵌入、表示或编码, 是神经网络对输入数据点编码后得到的实值向量。本章只简要提及网络架构和训练目标, 并将不同特征提取器记为函数; 有关深度学习的完整概述可参见 [392]。设模态 X 的编码器 f_X (通常是深度神经网络) 作用于输入 x , 得到 d 维特征 $f_x = f_X(x) \in \mathbb{R}^d$ 。

深度学习的目标, 是学习具有有用性质的编码函数。本章最关心的性质, 是能够方便地测量数据点之间有意义的相似性。特征空间中的距离可以回答“这两段文本有多相似?”或“这段文字说明与这幅图像有多相似?”等查询。

自监督学习目标和深度架构的研究催生了可以作为众多应用基础的特征提取器。现代特征提取器在大规模互联网级自然语言处理和计算机视觉数据集上预训练。虽然可以针对特定任务微调这些模型, 或把特征作为下游分类器和回归器的输入, 但本章讨论的建图方法大多直接使用“原样”的特征。下面简要介绍一些可用的文本、视觉和多模态特征提取器。

自然语言。 BERT [268] 改变了自然语言处理领域, 并引入了以掩码语言建模目标预训练的双向 Transformer。该模型无论是否微调, 在多种下游 NLP 任务上都能取得较强性能。BERT 提供的是 token 级特征 (即输入子字符串的特征); SentenceBERT [924] 增加池化操作, 并对 BERT 进行微调以输出句子级特征, 使特征可以通过余弦相似度高效比较。余弦相似度是把两个嵌入视为从原点出发的向量后, 计算其夹角的余弦。SentenceBERT 已提供多种语言的模型检查点, 并针对各种 NLP 任务进行了微调。

视觉。 DINO [154] 采用自监督方案训练视觉 Transformer [280], 让学生网络匹配教师网络的预测。多裁剪增强是训练的关键: 教师网络看到图像的全局视图 (覆盖超过 50%), 学生网络同时处理同一图像的局部视图 (覆盖少于 50%) 和全局视图, 从而鼓励“局部到全局”的学习。DINOv2 [831] 改进了训练过程, 并引入包含 1.42 亿幅图像的精选数据集, 得到具有强整体图像理解能力的视觉特征提取器。

DINO 输出图像级视觉特征, 可以零样本泛化到图像分类或图像检索等下游任务。例如, 可以利用 DINO 特征空间中的距离构建特定监督任务的 k 近邻分类器; 只要在 DINO 特征上训练, 即使简单的逻辑回归模型也能在图像领域取得较强性能。DINO 背后的 Transformer 架构还允许预测图像块级特征, 例如为图像中每个 8×8 图像块输出一个特征向量。在这些稠密特征上训练额外模型, 可以在语义分割、深度估计和表面法向估计等像素分辨率任务上获得良好性能 [831, 45]。直接使用特征空间距离进行多视图对应估计也已经得到研究 [45]。

多模态。学习得到的特征空间可以用于推理同一模态数据点之间的相似性；不同模态的专用编码器也可以映射到一个共同的、对齐的特征空间，从而实现多模态能力。CLIP^[899] 是视觉-语言领域的代表性方法，训练图像编码器 f_{image} 和文本编码器 f_{text} （图 17.4）。给定 RGB 图像 I 和文本查询 T ，CLIP 提取单位范数特征 $f_I = f_{\text{image}}(I)$ 和 $f_T = f_{\text{text}}(T)$ ，于是可以用余弦相似度 $\langle f_I, f_T \rangle$ 比较图像和文本。

CLIP 编码器在从互联网收集的 4 亿个图像-标题对上采用对比学习训练，得到能够直接编码广泛视觉和文本概念的特征。无需微调或重新训练，CLIP 特征就可以用于许多下游任务。例如，可以用文本形式定义类别集合 $\mathcal{C}$ （也可以使用“a picture of a dog”“a picture of a cat”等模板句），再以 CLIP 相似度预测图像 I 的类别：

$$\hat{k} = \arg \max_{c \in \mathcal{C}} \langle f_I, f_c \rangle, \quad (17.1)$$

要为新问题重新构造分类器，只需通过文本提示定义新的类别集合，因而可以迁移到大量任务。在具有合适编码器的情况下，式 (17.1) 的分类器也可以扩展到声音等其他模态^[421]。从 CLIP 视觉编码器中提取稠密图像块特征也是可能的；但不能假设这些特征与语言对齐，而且一些研究发现，相比 DINO^[831, 45]，它们在视觉和三维相关任务上的能力较弱。

17.2.2 生成式基础模型

大型语言模型（large language model, LLM）是用于文本生成的 Transformer，通常使用自监督目标训练。模型规模通常很大（即具有大量可学习参数），并在海量文本语料库上训练，因此具有令人印象深刻的泛化能力。语言建模的第一步是分词，即按照词、子词、字符或字节等词汇表定义的单位，把输入文本分割为 token。每个 token 被分配唯一整数，并对应一个可学习的嵌入；LLM 将文本作为 token 嵌入序列处理。

LLM 训练的具体细节超出本章范围，核心学习目标是下一个 token 预测。给定前 $T-1$ 个 token 的上下文 $w_{1:T-1}$ ，LLM 学习预测下一个 token 的概率 $p(w_T | w_{1:T-1})$ 。推理时，LLM 接收一段文本指令作为初始上下文（prompt），自回归地预测下一个 token，将其加入上下文并重复此过程，直到满足停止条件。

原则上，只要任务的指令、输入和输出都能够用文本指定，LLM 就可以用于该任务，例如交互式聊天机器人、代码生成、文本摘要或符号规划。提示设计会显著影响任务性能，并推动了思维链（chain-of-thought，即提示模型逐步思考）和上下文学习（in-context learning，即在提示中提供输入-输出示例）等提示工程技术的研究。常用的 LLM 包括 LLaMA 系列^[1104] 和 GPT 系列^[5]，并且更强的新模型正在频繁发布。

研究者还在积极地将 LLM 的一般能力扩展到视觉等其他模态。视觉语言模型（vision-language model, VLM）除接受图像 token 作为输入外，大多共享 LLM 的组件。LLaVA^[648] 将 CLIP 视觉编码器连接到 LLM，并在视觉和语言数据上训练，从而获得多模态能力。近期 GPT 模型也支持视觉输入和输出。在提示恰当时，VLM 可以执行图像分类、图像描述以及更一般的视觉问答。

17.2.3 类别无关的图像分割

图像分割是许多建图方法的关键组件，对于推理图像特定区域的语义必不可少。我们希望检测一组分割掩膜 $\{Z_i \in \{0, 1\}^{H \times W}\}_{i=0}^R$ ，它们表示宽度为 W 、高度为 H 的 RGB 图像 I 中可能存在的物体。关键是，掩膜提议必须与类别无关，才能正确覆盖机器人环境中通常出现的广泛物体，并利用前面介绍的基础模型的开放词汇性质。本章将这一函数记为 $\text{Seg}(\cdot)$ 。

实现该函数的主要工具是 Segment Anything Model (SAM)^[581, 911]。SAM 基于大规模监督学习执行类别无关的图像分割，是一种可提示的分割模型：输入像素坐标、边界框，甚至另一张分割掩膜，就可以预测分割掩膜。SAM 也可以自动处理完整图像，生成描述不同实例的一组掩膜。SAM 可以直接作为独立分割模型使用，也可以处理开放词汇物体检测器的边界框检测结果^[681]，从而获得更好的物体定义并减少推理时间。

17.3 开放世界建图

第 5 章介绍了如何把占据或距离场等几何量存入地图，第 16 章则主要通过增加类别信息将这一做法扩展到语义。虽然类别框架能够对场景语义提供合理的离散近似，但在实际使用中存在许多局限，其中最主要

的是封闭集假设：经典语义地图必须预先知道希望支持的类别词汇表。例如，任何依赖图像分类器或物体检测器的建图系统，都隐含假设世界可以由其感知前端的词汇完全描述。

实际上，预定义物体类别只能对现实提供非常粗略的近似。以工具箱中的常见物体为例，大多数分类器没有足够词汇完整描述工具箱中的物体。“工具”可能是可以接受的概括，但“螺丝刀”或“锤子”等更具体标签显然更理想。即使如此，我们还可能希望编码具体实例的颜色（“红色锤子”“蓝色螺丝刀”）、材料（“金属”“橡胶”）以及通常丢失在单一类别标签中的其他物理属性（“硬”“软”“热”“冷”）。对于机器人，我们还可能希望编码物体典型用途和可供性（“切割”“测量”）。对于带有图像内容的物体（“一幅罗纳尔多的画”），理想地图甚至应同时编码介质（“画”）和内容，即使内容涉及特定文化引用（“罗纳尔多”）。用预定义类别同时满足所有这些目标非常艰巨；即使包含数百个类别的词汇表，对常见物体也可能在某些层面不足，对于原本不太可能进入词汇表的稀有长尾物体，问题会更加严重。物体可能产生的典型声音等连续属性，本身也很难用离散类别标签表示。

为去除封闭集假设并支持更广泛的概念，我们可以用第 17.2 节介绍的基础模型提取的高维连续特征替换地图中的类别标签。训练这些特征提取器时使用的大型数据集，确保特征能够表示数量庞大的开放集物体及其属性。虽然单独的特征不能直接使用，但它们的多模态对齐允许与不同模态的查询进行比较，主要是自然语言查询，从而以零样本方式完成多种任务。

本节主要关注 RGB-D 相机：它们既支持使用基于图像的基础模型提取 RGB 特征，也支持把像素转换为点云等三维表示。我们通常假设静态场景中一系列带位姿的 RGB-D 帧，因此重点是建图问题而不是完整的 SLAM 问题。我们按顺序处理每一帧，增量构建地图。

17.3.1 稠密表示

构建稠密开放词汇地图涉及三个主要组件：选择三维表示；使用基础模型从传感器数据提取开放词汇特征；以及跨帧聚合这些特征并把它们定位到空间中。最后还要讨论如何对地图进行推理或查询。

地图表示。开放词汇建图把基础模型的语义特征嵌入第 5 章介绍的一种稠密空间表示。一般地，设地图表示由地图元素集合 $\mathcal{M} = \{m_k\}$ 组成，每个地图元素 $m_k = \{p_k, f_k\}$ 包含几何分量 p_k （体素位置、三维点、网格面等）和语义分量或特征向量 f_k 。

表示的选择主要由下游应用决定：导航方法倾向于在二维网格中进行单元级特征聚合^[484]，而移动操作流水线可能选择体素网格^[678]。ConceptFusion^[512] 和 OpenScenes^[863] 用无序三维点集合表示空间和语义；Open-Fusion^[1213] 使用带 TSDF 的体积表示；VLMaps^[484] 采用混合方法，先在三维点云中累积信息，再投影到二维网格地图。需要强调，地图元素的几何分量 p_k 被假设由上游 SLAM 系统提供，例如前面章节介绍的系统。

特征提取。接下来需要决定把哪些特征嵌入三维表示，以及如何获得这些特征。常见做法是用基础模型 f （例如第 17.2.1 节介绍的模型之一）从某帧 RGB 数据 I 中提取中间语义特征。问题在于，基础模型通常产生图像级特征，编码相机视锥内所有物体的整体信息。较大范围的上文很有价值，但我们还需要更细粒度、局部的语义，以适当描述图像的不同部分，甚至达到像素级分辨率。

一种方法是使用直接输出像素级^[647]或区域级^[379]特征的基础模型。这些模型通过在分割数据集上微调 CLIP 等基础模型进行训练。OpenScenes 和 VLMaps 通过 LSeg^[647] 直接从 RGB 图像提取 CLIP 特征；Open-Fusion^[1213] 则使用 SEEM^[1312] 直接提取区域特征，以提高计算效率。

然而，近期证据表明，相比原封不动的 CLIP 特征，这些稠密特征包含的长尾概念较少^[512]。ConceptFusion 采用另一种方法，保留原始 CLIP 特征，并通过特征运算构造像素级特征向量，但需要一些启发式方法，在局部和全局语义信息之间取得平衡。具体地：

- * 用 $f^G = f(I)$ 编码整幅图像，得到全局特征 f^G ；* 对类别无关分割方法（例如 SAM^[581]）得到的所有掩膜 $\mathcal{R} = \text{Seg}(I)$ 进行处理。每个掩膜转换为边界框，生成紧致图像裁剪 I_i ，再应用 f 得到掩膜级特征 $f_i^L = f(I_i)$ ；* 像素级掩膜特征为 $f_i^P = w_i f^G + (1 - w_i) f_i^L$ 。线性组合权重 $w_i \in [0, 1]$ 强调两类局部特征：与全局特征不同的特征，以及与其他局部特征相比具有独特性的特征；* 像素级特征 $f_{u,v}^P$ 由覆盖像素 (u, v) 的所有掩膜（即满足 $Z_i[u, v] = 1$ 的掩膜）的 f_i^L 求和得到，并将结果单位化。

当然，还可以采用其他局部和全局特征融合方法；以更有原则的方式进行融合，是未来值得研究的方向。

综上，在针孔相机模型和像素-特征对应关系的假设下，可以按照第 5.1 节的过程生成具有开放词汇语义

增强的有序点云观测。每个点都有相应的语义特征向量，可以通过像素分辨率模型查询或全局/局部特征融合得到。我们把这类观测称为距离-特征观测 (range-feature observation)，对应于第 16 章介绍的距离-类别观测。

特征聚合与空间落地。选定空间表示和生成距离-特征观测的方法后，下一个组件要处理的是：把不同 RGB-D 帧中对应同一三维位置的特征组合起来，形成标注相应地图元素的一个特征。VLMaps 采用直接方法：如果来自所有帧的距离-特征观测投影到同一二维网格单元，就将它们平均^[484]。

ConceptFusion^[512] 将距离-特征观测转换到全局坐标系；随后对地图中的每个元素 m_k ，将与点 p_k 对应的所有像素级特征按特征置信度加权平均。特征 f_k 和置信度 c_k 按以下方式更新：

$$f_k \leftarrow \frac{c_k f_k + \alpha f_{u,v}^P}{c_k + \alpha}, \quad (17.2)$$

$$c_k \leftarrow c_k + \alpha, \quad (17.3)$$

其中 $\alpha = \exp(-\gamma^2/(2\sigma^2))$ 是与该点关联的每个像素-特征所分配的置信度， γ 是到相机中心的径向距离， σ 是缩放项。直观上， α 为更靠近相机中心的特征分配更高置信度，类似于早期三维重建工作^[549]。

OpenScene^[863] 同样对每帧的二维特征求平均，但还会把它们蒸馏成三维特征；平均二维特征与蒸馏三维特征通过集成机制融合。Open-Fusion^[1213] 则采用不同方法：每个占据体素只保存一个特征字典的键，键的选择由时间特征匹配决定。

其他工作研究了替代的特征聚合或集成策略，以避免平均高维特征向量，并提高对异常特征向量的稳健性。例如，^[863] 随时间为所有地图元素积累特征集合 $F_k = \{f_{k,1}, \dots, f_{k,t}\}$ ，并保留缓冲区中与其他特征欧氏距离最小的特征向量 (medoid 向量)。聚类方法可以把 F_k 压缩为少量代表性聚类中心^[704]，或选择最接近多数聚类中心的 F_k 中特征向量，以提高特征选择的稳健性^[1182]。

推理。特征在地图中落地并聚合后，可以将其与从其他模式提取的特征比较，从而完成多种任务。以视觉和语言特征对齐的 CLIP 为例，可以把文本类别标签编码为提示列表 C ，对地图元素 m_k 及其特征 f_k 预测点级标签：

$$y_k = \arg \max_{C \in C} \langle f_k, f_C \rangle, \quad (17.4)$$

其中 $f_C = f_{\text{text}}(C)$ 是文本编码器对提示 C 编码得到的特征。这与第 17.2 节讨论的图像分类器完全对应。关键是，建图时不必知道 C ；可以随时重新定义它以预测不同标签。对于网格地图和体素地图等其他表示，分割过程是类似的。

然而，开放词汇地图最主要的用途可能是查询。开放式查询通过编码一个查询，并将查询特征与所有地图特征比较来完成。在 CLIP 场景中，可以编码自由形式的文本查询 T ，得到 $f_T = f_{\text{text}}(T)$ ，再根据查询相似度 $\langle f_k, f_T \rangle$ 对地图 $\mathcal{M}$ 中的点评分。对相似度设定阈值可以返回一组相关地图元素，还可以对返回元素的空间坐标聚类，以识别不同实例。建图时同样不需要预先知道查询；多模态基础模型的通用性使其能够合理回答即使非常具体的查询。

虽然语言是指定查询的自然方式，但任何具有对齐编码器的模式都可以提取查询特征。查询特征也可以通过用户交互获得。以 ConceptFusion 为例，除了文本，还支持多模态查询 (图 17.1)：点击查询取可视化地图 $\mathcal{M}$ 中被点击点的特征向量 f_k ；图像查询为 $f_I = f_{\text{image}}(I_{\text{query}})$ ；音频查询为 $f_s = f_{\text{audio}}(s)$ ，其中 f_{audio} 是 AudioCLIP^[421] 的声音编码器， s 是声音片段。

只使用视觉特征 (例如 DINO^[154]) 时，零样本推理能力更有限，但此类地图仍支持图像查询、局部图像查询 (例如点击图像) 和地图点击查询。用户可以在参考图像中点击少量物体，以获得地图中物体及其部分的准确分割^[745]。

17.3.2 物体地图与三维场景图

物体级方法首先将类别无关分割区域聚类为物体，再为每个物体保存三维位置、几何形状、图像特征和语言描述。ConceptGraphs^[407] 使用物体描述和位置，通过 LLM 推断“在……上方”“位于……中”等物体

关系，并构建物体场景图。边特征也可以由同时观察两个物体的图像经基础模型编码得到^[588]，图神经网络可用于学习关系^[587, 589]。

HOV-SG^[1182] 将室内环境分解为楼层、区域、物体和覆盖可导航自由空间的 Voronoi 节点；Clio^[723] 则使用区域、地点和物体。HOV-SG 的分层结构和语言级关系如图 17.5 所示。

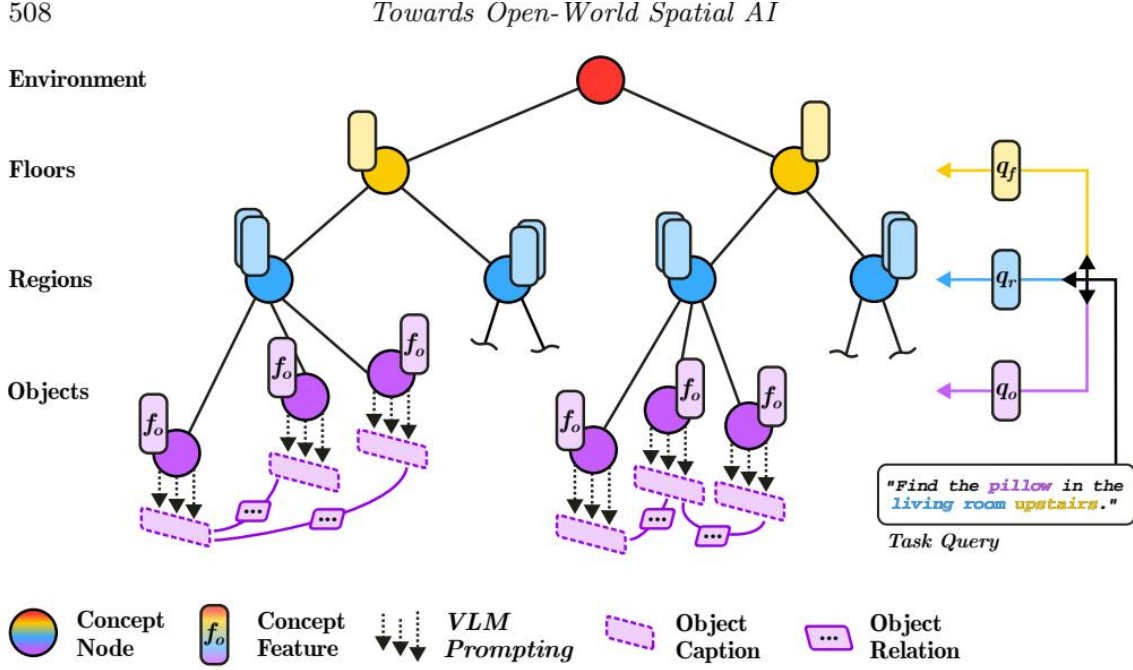


图 17.5: 图 17.5 开放词汇三维场景图: HOV-SG^[1182] 的分层环境划分和 ConceptGraphs^[407] 的语言级物体关系。

虽然稠密开放词汇地图可以以非常细的粒度建模语义，但常用特征向量维度很高，地图会随着环境规模扩大而难以扩展。图像中的邻近像素通常属于同一物体并共享语义；类似地，地图中的邻近元素（点、体素）通常也共享相似语义。因此，可以在物体层级而不是逐点存储语义特征，在信息损失很小的情况下显著降低内存占用。物体中心表示也更适合具身 AI，因为它们天然支持物体检索和以物体为中心的任务规划。

物体地图表示与特征提取。设地图是物体集合 $\mathcal{V}_O = \{o_j\}_{j=1}^J$ 。每个物体 o_j 具有几何表示和特征向量，几何表示可以是物体中心^[178]、三维边界框等稀疏抽象，也可以是三维点云^[407, 1182, 1066]等稠密表示。下面将每个物体表示为三维点云 P_{o_j} 、特征向量 f_{o_j} 和检测次数 n_{o_j} ；处理输入的带位姿 RGB-D 帧时，按需增量添加或初始化物体。

物体中心地图通常使用类别无关分割方法（例如 SAM^[581]）从当前图像提取掩膜集合 $\mathcal{R} = \text{Seg}(I)$ 。每个掩膜被转换为边界框，生成图像裁剪 I_i ，并提取掩膜级特征 $f_i^M = f(I_i)$ ；这与 ConceptFusion 提取局部特征的过程相同。利用深度数据、相机位姿和相机内参，将每个掩膜的像素反投影到三维，得到掩膜点云 P_i^M ，再转换到地图坐标系。此时也可以使用 DBSCAN^[311] 等方法对掩膜点云去噪，最终得到掩膜-特征观测 $\{(P_i^M, f_i^M)\}_{i=0}^R$ 。

物体关联与融合。需要测量掩膜与现有物体集合 $\mathcal{V}_O$ 的相似性，方法通常同时使用几何和语义相似性。几何相似性可用最近邻比率 $\phi_{\text{geo}}(P_i^M, P_{o_j}) \in [0, 1]$ 测量，它表示点云 P_i^M 中在距离阈值 δ_{nn} 内、在 P_{o_j} 中具有最近邻的点所占比例^[407, 1182]。语义相似性可以用特征相似度函数

$$\phi_{\text{sim}}(f_i, f_{o_j}) = \frac{1}{2}(\langle f_i, f_{o_j} \rangle + 1) \in [0, 1]$$

衡量。几何和语义相似性可以组合为总体得分，也可以分别设阈值来识别候选掩膜-物体匹配。贪心策略^[407] 把输入掩膜关联到 $\mathcal{V}_O$ 中一个物体；更复杂的方法允许一个掩膜关联多个物体^[704, 1182]。如果掩膜没有匹配项，就用其几何和特征初始化一个新物体。

当物体 o_j 与第 i 个掩膜匹配时, 需要更新物体。一个简单方法是合并点云并平均特征:

$$f_{o_j} \leftarrow \frac{n_{o_j} f_{o_j} + f_i}{n_{o_j} + 1}, \quad P_{o_j} \leftarrow P_{o_j} \cup P_i^M, \quad n_{o_j} \leftarrow n_{o_j} + 1.$$

随后可以对 P_{o_j} 下采样以删除冗余点。稠密地图中讨论的其他特征聚合策略同样适用于这里。开放词汇方法还会定期 (例如每隔若干帧) 在 $\mathcal{V}_O$ 中的物体之间运行关联和融合算法, 以减少冗余物体^[407, 704]; 检测次数 n_{o_j} 较低的物体也可以在建图过程中定期或处理完全部帧后删除。

另一种方案是不做二维图像分割和掩膜关联, 而是把所有帧累积到全局场景点云中, 再直接在三维中执行物体分割, 可以使用学习模型^[1066]或区域生长等几何技术^[539]。重新投影三维物体掩膜后, 可以找出每个物体的代表性图像并相应提取特征。三维分割有可能更快, 因为避免了逐帧分割 RGB 图像并关联片段; 但它天然不适合增量建图, 因为需要完整的场景点云。

物体查询与语言描述。零样本分割和查询机制与稠密开放词汇建图非常相似: 三维点继承其所属物体的预测类别。若点 $p_k \in P_{o_j}$ 属于具有 CLIP 对齐物体特征 f_{o_j} 的物体, 则点级类别标签为

$$y_k = \arg \max_{C \in \mathcal{C}} \langle f_{o_j}, f_C \rangle.$$

其中 $\mathcal{C}$ 是文本形式的类别集合, f_C 是类别提示的 CLIP 文本特征。物体及其相似度也可以用于实例分割^[1066]。更重要的是, 查询现在可以用于物体检索: 给定文本查询 T 及其特征 f_T , 可以从物体集合中检索

$$o^* = \arg \max_{o \in \mathcal{V}_O} \langle f_o, f_T \rangle.$$

物体检索对于语言驱动的导航和操纵目标指定非常有用。例如, “可以喝的东西” 会与汽水罐的物体特征强烈相关; 对应点云可以传给运动规划流水线, 用于导航到汽水罐、检查汽水罐, 甚至操纵汽水罐。

与短描述查询相比, 语言对齐的物体特征效果最好; 对于包含否定或可供性信息的复杂查询, 它们通常无法捕获细微差别。例如, “除汽水罐之外可以喝的东西” 仍可能与汽水罐特征高度相关, 而且特征本身并不显式, 不能被人类用户或 LLM 直接理解。一种替代方案, 是用自由形式语言直接描述地图中的物体。可以跟踪每个物体最好的图像裁剪 (例如跟踪为物体贡献最多点的掩膜), 把它们与 “描述图像中央物体” 等提示一起输入 VLM; 先为物体的各个视图生成描述, 再用 LLM 得到物体的最终摘要。部分 VLM 也可以直接接受多幅图像作为输入。一些方法同时使用特征和语言描述^[407, 164]。

物体描述比封闭集类别标签表达力强, 也能受益于 VLM 的一般物理解。查询时, 可以指示 LLM 根据物体描述列表和用户查询, 搜索地图中最相关的物体; 实验表明, 这种基于 LLM 的物体检索能处理更复杂的查询, 从而缓解基于特征的物体检索的部分局限^[407]。但 LLM 描述和推理通常比基于特征的方法慢得多; 语言标签等其他文本表示也已被有效使用^[1275]。

物体场景图。更一般地, 可以用 $G = (\mathcal{V}_O, \mathcal{E}_O)$ 表示地图, 其中边集合 $\mathcal{E}_O$ 显式建模 $\mathcal{V}_O$ 中物体之间的成对关系, 从而形成三维场景图。与物体语义类似, 我们希望用语言或特征推断开放词汇的边描述。ConceptGraphs^[407] 依靠物体描述和位置, 用 LLM 推断 “a 在 b 上” “b 在 a 中” 等空间关系, 并讨论把空间边类型扩展到语言模型能够解释的关系, 例如 “背包可以存放在壁橱里” 以及 “纸张可以放进垃圾桶回收”。还可以用基础模型编码两个物体同时可见的图像来得到边特征^[588], 并使用图神经网络学习关系^[587, 589]。物体场景图中的查询需要额外考虑: OVSG^[164] 使用 LLM 将上下文语言查询映射为图结构, 再用图匹配方法与场景图比较。

分层场景图。除了前述物体-物体关系, 许多工作还研究使用开放词汇语义对分层场景进行划分^[1182, 468, 1032]。按照第 16.4 节对分层图的定义, 环境被稀疏建模为图 $G = (V, E)$, 并分为 l 个有序概念层, 节点集合为 $V = \bigcup_{i=1}^l V_i$, 具体形式取决于目标环境。HOV-SG^[1182] 将室内环境分为楼层、区域、物体以及覆盖可导航自由空间的 Voronoi 节点; Clio^[723] 则建模区域、地点和物体。其他工作针对典型城市户外环境, 将其分解为道路、车道以及静态或动态物体^[1032, 263]。

对典型室内场景, 沿用 HOV-SG 的方法。节点和边分别划分为互不相交的集合:

$$V = V_E \cup V_F \cup V_R \cup V_O, \quad E = E_{EF} \cup E_{FR} \cup E_{RO}, \quad (17.5)$$

其中 V_E 表示单一根节点, V_F, V_R, V_O 分别表示楼层、区域和物体节点。边集合 E_{EF} 将楼层与根节点连接, E_{FR} 连接楼层与区域, E_{RO} 连接区域与物体。分层可以识别层次关系、减少内存占用并加快图搜索。HOV-SG 为每个概念节点至少配置一个通过 CLIP 获得的开放词汇概念特征, 从楼层到物体都支持语言落地。

与 ConceptGraphs 类似, HOV-SG 执行增量 RGB-D 建图和类别无关实例预测, 并假设相机位姿准确。它沿场景重力对齐的高度轴对观测点分箱并寻找峰值, 将点云划分为多个楼层; 每个楼层获得“floor {X}”或“upstairs”形式的模板 CLIP 文本特征, 以便通过语言引用。

接下来推断满足几何约束且具有语义一致性的区域。导航任务要求区域原语满足空间连续性和由墙壁、门等表示的结构边界^[468]; 而语义高层推理策略则偏好语义一致的区域落地。例如, 厨房和客厅可能是一个面积很大的连续区域, 但其中包含许多语义明显不同的子区域。HOV-SG 通过对每个楼层的欧氏距离场设阈进行纯几何区域分割; 其他方法根据任务指令, 对广义 Voronoi 图得到的导航节点聚类^[723], 或把门作为区域边界的逻辑指示^[468]。

为了给区域概念节点增加开放词汇特征, 收集落在每个区域内部的所有相机位姿对应的相机视图, 并提取每幅视图的图像级 CLIP 嵌入。使用 k -means 将视图蒸馏为 K 个代表 $\{f_{r,k}\}_{k=1}^K$, 再将代表与用 CLIP 编码的候选区域类别集合 C_R 比较。典型集合可以是“厨房”“浴室”“客厅”“走廊”。每个代表的类别投票由点积得分给出:

$$c_{rk}^* = \arg \max_{c \in C_R} \langle f_{r,k}, f_{R,c} \rangle, \quad (17.6)$$

对所有代表进行多数投票即可确定一个区域类别。

HOV-SG 还展示了在大规模环境中进行自然语言导航的方法: 给定“到一楼客厅的植物处”等复杂文本查询, 生成式基础模型 (GPT) 根据建图阶段规定的概念, 将长查询分解为多个描述 (“植物” “客厅” “一楼”), 再把它们编码到所构建场景图的 CLIP 特征空间。从较高层到较低层 (楼层、区域、物体) 逐层对所有节点评分; 对区域而言, 可以针对一组区域代表 $\{f_{r,k}\}_{k=1}^K$ 查询, 而不是针对代表区域类别的单一文本嵌入, 因此能够更稳健地检索区域。

位姿地图。本节采用以物体为中心的视角, 但也有相关方法在相机位姿或视点层级存储开放词汇语义信息^[540, 1086, 1275]。这样得到的稀疏空间表示可用于房间分割、回环检测^[540]和重定位^[1086]。查询地图特征会返回最相关的视点, 并可用于指定导航目标; 不同视点之间的语义和空间重叠还可以定位特定物体实例^[1275], 实现与显式物体地图相当的物体查询能力。

17.3.3 隐式函数

第 14 章介绍了使用 NeRF^[764]和 3D Gaussian Splatting^[552]创建照片真实感和几何地图。许多工作将开放集语义加入这些表示: NeRF 方法为隐式辐射场增加语言特征, Gaussian Splatting 方法为高斯赋予嵌入并将其光栅化到图像平面。由于为每个高斯存储独立语言嵌入会占用大量内存 (例如 CLIP ViT-L/14 使用 768 维嵌入), 常见做法是学习压缩嵌入, 或将高斯分组后为组分配单一向量。

LERF^[555]为 NeRF 增加给定点 $\mathbf{x}$ 和尺度 s (以 $\mathbf{x}$ 为中心立方体的宽度) 时的语义嵌入输出, 并省略视点依赖以保证不同观察方向的一致输出。语义嵌入像 NeRF 颜色一样渲染; 沿每条射线的视锥体从图像初始尺度 s_{img} 按比例增大。训练时在多个随机图像裁剪尺度上预计算 CLIP 嵌入形成嵌入图像金字塔, 并以随机尺度监督射线; 为获得物体边界更清晰的语义, LERF 还使用 DINO^[154]嵌入监督。对文本嵌入 f_{text} 查询渲染图像时, 相关性可写为

$$\min_i \frac{\exp(\langle f_{\text{render}}, f_{\text{text}} \rangle)}{\exp(\langle f_{\text{render}}, f_{\text{canonical}}^i \rangle) + \exp(\langle f_{\text{render}}, f_{\text{text}} \rangle)}, \quad (17.7)$$

其中 $f_{\text{canonical}}^i$ 来自 “object” “thing” “stuff” 和 “texture” 等规范短语, 用于去噪相关性得分。LangSplat^[895]以类似方式将压缩的语义特征存入 3D 高斯。

与 LERF 在图像金字塔上计算多尺度 CLIP 向量不同, LangSplat^[895]在三个粒度层级上, 对 Segment Anything Model (SAM) 得到的类别无关图像片段计算 CLIP 向量。这样不仅能够捕获局部和全局上下文, 还能使语言嵌入地图的边界更清晰, 因此不必像 LERF 那样额外使用 DINO 监督。

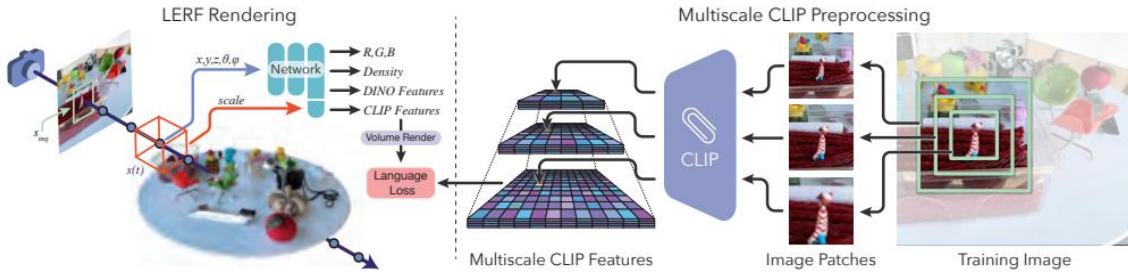


图 17.6: 图 17.6 LERF^[555] 训练神经辐射场以渲染 CLIP 特征 (改编自^[555], ©2013 IEEE)。为了在训练中使用多分辨率的监督语义嵌入, LERF 将不同尺度的图像裁剪输入 CLIP, 预先计算嵌入金字塔。

每个三维高斯在三个语言嵌入层级分别增加一个属性。将 CLIP 向量赋给每个高斯会显著增加内存需求 (LangSplat 使用的 CLIP 模型嵌入维度为 512), 因此 LangSplat 训练一个针对具体场景的网络, 把特定场景的 CLIP 向量从原始 512 维压缩到 3 维, 并训练解码器恢复原始嵌入。其直觉是, 典型场景中的语义变化远小于 CLIP 训练语料中的变化, 因此较小的嵌入向量已经足够。随后, 三个粒度层级的压缩嵌入向量作为三维高斯的训练监督。语义渲染与 RGB 情形相同, 将三个粒度层级的压缩嵌入 splat 到图像平面, 再放大为完整维度的嵌入。查询使用与 LERF 相同的相关性得分; LangSplat 会返回三个得分 (每个粒度层级一个), 定位目标物体时选择得分最高的层级。

17.3.4 任务驱动的表达

开放集建图能够表示比封闭集建图更广泛、描述更丰富的概念, 但也带来了如何定义物体的问题。在封闭集场景中, 物体由“工具”“杯子”等标签列表定义; 在开放集场景中, 物体应采用何种粒度并不明确。例如, 考虑为空房间中的一架钢琴构建开放集语义地图: 如果智能体的任务是移动钢琴, 把整架钢琴作为一个物体是合适的; 如果任务是弹奏钢琴, 则需要把每个琴键和踏板分别建模, 场景中会有 90 多个物体; 如果任务是为钢琴调音, 还需把琴弦、弦轴等建模为数百个物体。同样, 在尚未说明表示需要支持的任务之前, 森林应表示为一整片景观, 还是表示为树枝、树叶和树干等组成部分, 也是一个欠定问题。正确的场景表示取决于智能体必须完成的任务^[732, 1019]; 视觉-语言基础模型出现以前, 如何在实践中纳入任务知识一直不清楚。

前述物体集合地图通常假设, 可按语义或几何相似性聚类区域来定义物体 (例如第 17.3.2 节的 Concept-Graphs)。这种方法在某些场景和应用中可以近似得到正确粒度, 但不足以普遍保证生成适当的开放集地图表示。Clio^[723] 因此正式提出任务驱动开放集建图问题: 给定“清洁背包”“拿取调味包”等自然语言任务集合, 构建一个物体粒度恰好能够支持这些任务的地图。Clio (图 17.7) 用经典信息瓶颈理论^[1098] 将其形式化: 给定细粒度原语观测 X (极端情况下可以是所有视觉观测中的全部像素, 也可以是过分割的类别无关掩膜), 构造压缩场景表示 $\tilde{X}$ (即与任务相关的物体), 同时保留完成期望任务 Y 所需的信息:

$$\min_{p(\tilde{x}|x)} I(X; \tilde{X}) - \beta I(\tilde{X}; Y), \quad (17.8)$$

其中 I 是互信息, β 控制压缩与保留任务信息之间的权衡。信息瓶颈需要定义 $p(y|x)$, 在任务驱动建图中, 它描述原语 x 与给定任务 y 之间的相关性。为使问题易于处理, Clio 将 X 定义为过分割的三维物体原语集合: 把类别无关图图片段 $\text{Seg}(I)$ 投影到三维网格, 并跨帧跟踪。对与每个原语关联的各片段计算 CLIP 嵌入并求平均, 为原语赋予语义嵌入向量; 随后以图像-文本嵌入的余弦得分近似 $p(y|x)$ 。式 (17.8) 可用凝聚式信息瓶颈 (Agglomerative Information Bottleneck) 算法^[1017] 增量求解: 建立以原语为节点、连接邻近原语的图, 并逐步聚类原语, 从而得到硬聚类指派 $p(\tilde{x}|x)$ 。每条边 (i, j) 的权重衡量合并两个原语簇造成的信息失真:

$$d_{ij} = (p(\tilde{x}_i) + p(\tilde{x}_j)) D_{\text{JS}}[p(y|\tilde{x}_i), p(y|\tilde{x}_j)]. \quad (17.9)$$

其中 $p(\tilde{x}_i)$ 和 $p(\tilde{x}_j)$ 由算法计算, 直观上记录分别合并到簇 i 和 j 中的原语数量, D_{JS} 是 Jensen-Shannon 散度。算法在每次迭代中按边权贪心合并, 使邻近原语逐步组成与任务相关的更大物体。Clio 还把任务驱动建图扩展到场景层次结构: 将表示三维自由空间区域的地点原语聚类为房间等任务相关区域, 生成包含物体原语、物体、地点原语和区域等层的分层三维场景图。

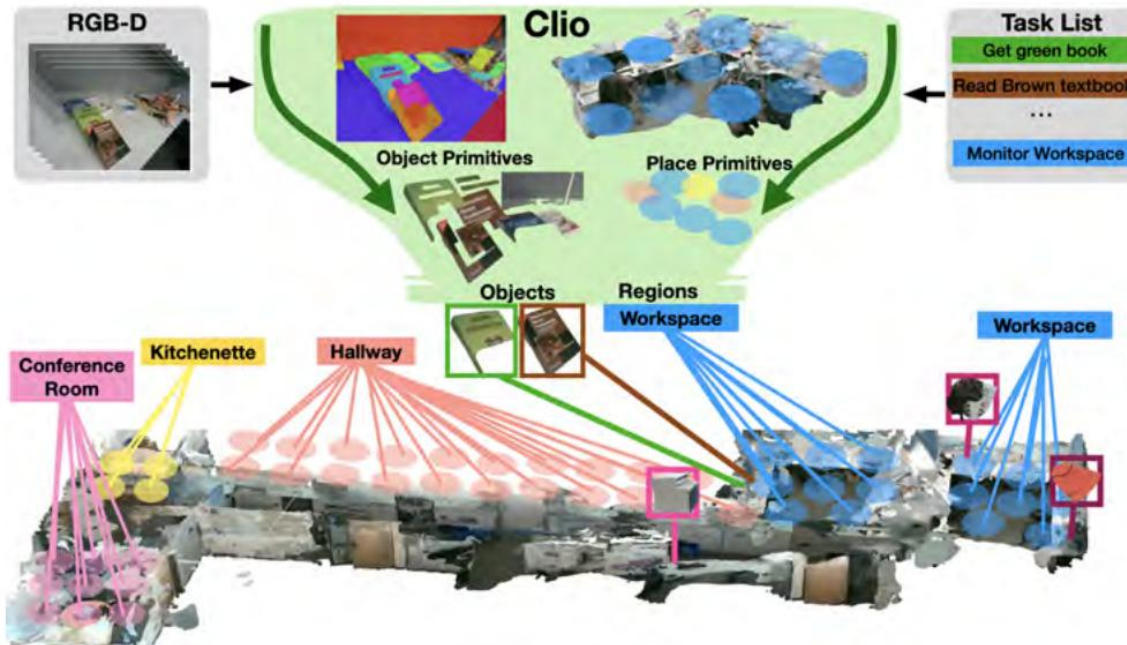


图 17.7: 图 17.7 Clio^[723] 根据 RGB-D 图像和自然语言任务列表生成开放集、任务驱动的三维场景图。它将三维关联的图像分割原语聚类为适合任务的物体, 并将自由空间地点原语聚类为房间等任务相关区域 (©2024 IEEE)。

式 (17.8) 为任务驱动建图给出了通用数学表述, 但实践中, 用图像-文本信息近似 $p(y|x)$ 以及决定如何跨视图聚合语义知识, 都可能引入误差并产生不正确的表示。这两个活跃研究问题在 Bayesian Fields^[722] 中得到讨论; 该方法还提出了更有依据的概率估计方式。Bayesian Fields 与 Clio 使用相似的凝聚式信息瓶颈方法, 但采用 Gaussian Splatting 地图而非粗糙网格, 其中任务相关物体是形成高分辨率、照片真实感物体的三维高斯分组。

目前的任务驱动建图示例要求以“清洁背包”等具体语言提供任务, 这主要受 CLIP 能力限制。更长远地看, 任务驱动建图可以理解为: 机器人根据“保持房屋清洁”或“监控工厂”等高层任务目标, 在线决定如何表示和组织场景中的相关信息。Ashita^[168] 是这一方向的早期工作, 它在把复杂任务分解为子任务的同时, 将这些子任务落地到三维场景图中。

17.4 延伸阅读与近期趋势

本节试图展望未来; 鉴于这一领域发展极其迅速, 这是一项困难的任务。

17.4.1 用地图落地基础模型

第 17.2 节已经看到, 只要把任意文本信息放入提示, LLM 就可以在其条件下完成大量任务。LLM 丰富的世界先验知识和推理能力对空间智能非常有前景; 人们也正积极研究如何把 LLM 落地到物理世界并将其部署为智能体。这产生了一个可能指导未来研究的自然问题: 如何把 LLM 与地图表示充分集成? 下面介绍基础模型与地图表示变得更加紧密的两种方式。

地图提示。有时可以直接以文本形式把地图提供给 LLM。前面介绍的三维场景图就是一个例子：当 VLM 使用自然语言描述物体和边时，场景图可以自然地表示为结构化文本文件（例如 JSON 或 YAML），并加入提示^[407, 726, 1036]。原则上，还可以加入物体的一些稀疏几何描述（例如质心位置、尺寸和边界框坐标），但 LLM 能否有效利用这些信息尚不清楚^[726]。这种把基于 VLM 的地图以文本形式传给 LLM 的模块化系统，可以理解作为一种苏格拉底式模型^[1]，即将多个预训练模型组合起来的模块化框架。

地图 API。文本地图提示无法传达世界的详细几何信息，并且局限于具有稀疏、可用文本表示结构的地图。另一种方案是把地图作为完全独立的模块，通过符号地图 API 暴露部分功能。例如，可以提供 ‘exists(object)’、‘where is(object)’ 或 ‘distance(object A, object B)’ 等查询；输入用自然语言描述，实际实现则依赖第 17.3 节讨论的查询机制。地图函数调用可以与 LLM 调用和机器人 API 调用交错，从而构建复杂而强大的系统。NLMap^[178] 提出了一种规划框架：LLM 将用户查询分解为不同的候选物体，并查询开放词汇地图来验证物体是否存在及其位置。地图 API 也可以用于“工具调用”，即允许 LLM 调用外部函数，并在生成答案时使用函数输出。LLM 可以查询开放词汇地图来生成机器人计划^[1275]，或回答用自然语言表达的多种场景查询^[512, 472]。

17.4.2 重新审视是否需要地图

既然互联网规模数据集训练出的视觉感知表示如此出色，就有必要重新审视本手册第 I 章提出的第一个问题：SLAM 是否需要地图，或者至少是否需要显式地图表示。本节介绍显式地图在某些情况下如何直接被基础模型替代。

长上下文 VLM。Gemini、GPT-4+^[1078, 5] 等模型可以直接处理原本用于构建地图的 RGB 图像序列，但长上下文 VLM 对图像背后空间结构的理解是否一致，尚不清楚。OpenEQA^[726] 对 VLM 与场景图的多种集成方式进行具身问答基准测试，发现直接向 GPT-4V 提供 50 帧提示，在其基准上优于提供显式场景图提示。Mobility VLA^[201] 在导航场景中探索长上下文 VLM：向 VLM 提供完整办公楼参观视频（948 帧），要求它根据用户查询识别高层目标图像，再将目标图像传给基于地图的导航流水线。作者还测试了不使用地图的变体，让 VLM 根据当前观测和办公楼参观视频直接输出导航动作，结果无效。基于这些结果以及其他证据，我们认为，目前物理智能体是否需要显式地图，主要取决于任务的空间和时间范围，仍是一个活跃的研究问题。

物理或几何感知 VLM。其他方法尝试为基础模型增加环境几何和物理属性意识；大多数 VLM 开箱即用缺少这类能力^[45, 1168]。例如，^[177] 从传感器并未具身化（不能被驱动或控制）且只有单一视图的角度出发，仍然展示了增强模型从单个相机图像推理三维空间的强大能力。不过，尽管这些方法在 OpenEQA 等基准上表现出色，它们与在流水线中加入 SLAM 后展现的具身智能水平之间目前仍存在重要差距。模仿学习或强化学习等其他基于学习的范式，可能填补这一差距；下一节将进一步讨论这类方法。

17.4.3 机器人基础模型？

语言和二维视觉基础模型之所以相对容易构建，是因为有可用于训练的互联网规模数据。鉴于这些模型的代表能力带来了惊人进步，自然会问：能否为其他领域生成足够数据，构建相应的基础模型？第 13 章讨论的 Dust3R^[1164] 就是一个很好的例子，它已经对三维视觉领域产生了重要影响。但机器人是否也能直接做到这一点？如果能，是否需要重新考虑显式地图的问题？因此，本章最后考察机器人领域最近尝试构建机器人基础模型的工作。

这类机器人基础模型的目标，是作为基础通用策略，在各种机器人具身形态上执行多种技能。策略需要以某种方式隐式表示世界，并利用该表示执行任务。因此，数据应当由输入-输出对组成，即传感器数据及对应动作。正如第 17.1 节所述，构建基础模型需要海量数据，而在真实机器人硬件上收集数据极其困难。不过，研究者已经开始在真实硬件和仿真中收集这种规模的数据。下面先简要介绍近期大规模机器人数据集的收集工作，再介绍消费这些数据、旨在产生不需要显式世界表示的通用机器人智能体的模型。

数据集和基准。RT1: Robotics Transformer^[118] 是早期数据集工作，包含 13 台机器人在 17 个月内执行 700 种任务的 13 万条机器人轨迹。BridgeData V2^[1142] 进一步提高了任务多样性，包含 24 个环境中的 13 种技能、超过 5 万次演示；尽管硬件固定，但通过随机化外部相机位置来提高稳健性。DROID 数据集

[559] 是多机构合作项目，旨在创建多样的机器人交互数据，主要用于训练操纵策略。上述数据集及其他数据集后来被整合为 Open-X-Embodiment [226]，该数据集包含超过 100 万个 episode，覆盖 22 种机器人具身形态、527 种不同“技能”、34 所大学实验室以及多样环境。

扩大数据收集规模的重要因素，是为人类提供高效、直观的演示 API，并经常配合高保真仿真。例如，DROID [559] 使用 VR 头戴设备进行远程操作；ALOHA [1077] 设备成本较低，由两组同步机械臂组成，人类可以提供演示，机器人硬件随后直接模仿；Universal Manipulation Interface [200] 是一种低成本手持夹具，精确匹配机器人的夹具和手眼传感器配置，因此可以灵活、低成本地收集机器人演示。

上述数据集中的数据大多是传感器-动作对轨迹 $\mathcal{D} = \{z_t, u_t\}_{t=1}^T$ ，其中 T 是轨迹长度。因此，它们主要针对机器人技能学习，例如在多种运行条件和机器人配置下进行抓取与操纵。理想目标是通过越来越复杂的演示（结合语言条件），或通过串联较短的子任务演示，完成复杂任务。

模型类别。为消费上述数据集并产生通用智能体，已经发展出多种建模范式。智能体应当能够在多种环境以及多种机器人配置和形态上，执行大量抽象指定的任务。

一种简单直观的方法是行为克隆（behavior cloning, BC）：以监督方式训练模型，学习从观测到动作的映射 [881]。但这种方法已知较脆弱且容易发散，因为演示中包含的次优数据很少，而动作相对演示产生微小偏差后，模型就会进入分布外状态 [945]。“行为 Transformer”模型即使完全在仿真中训练和评估，也显示出克服部分挑战的初步潜力 [993]。

因此，在这些大规模数据集上训练的大多数候选“机器人基础模型”，都采用与第 17.2 节介绍的基础模型相似的结构，只是 Transformer 的输出 token 现在被直接解释为控制动作，通常称为“视觉-语言-动作”（Vision Language Action, VLA）模型，如图 17.8 所示。

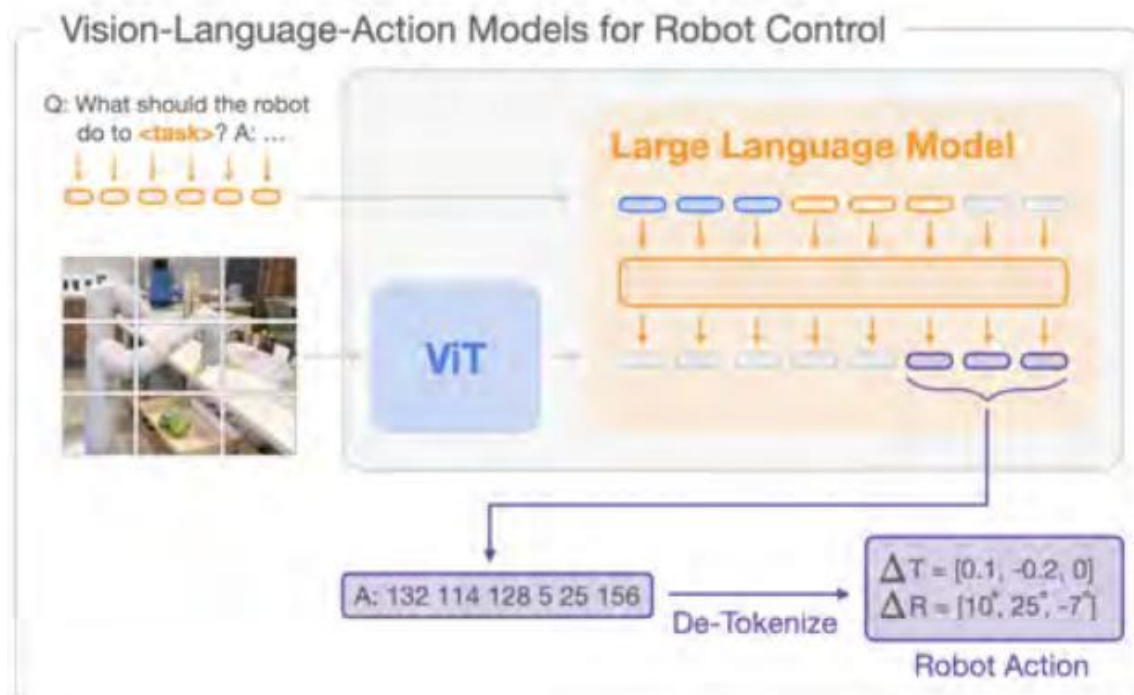


图 17.8: 图 17.8 “视觉-语言-动作”模型示意（改编自 [119]）：Transformer 被修改为直接输出机器人动作。

GATO [921] 可能是第一个此类例子：它通过 token 化输入和输出，展示一个通用智能体可以用单一模型执行大量任务，包括机器人控制。随后，PaLM-E [281] 构建“具身多模态语言模型”，同时在视觉-语言数据和机器人操纵数据上训练，直接完成语言、运动规划和操纵任务。RT1 [118] 使用 FiLM 图像编码器而非 PaLM-E 中的 ViT，但模型结构总体相似。RT2 [119] 将 PaLM-E 和 RT1 的数据结合；RT1-X 和 RT2-X 是在 Open-X-Embodiment 数据集 [226] 上训练的 RT1、RT2 变体。

“Action Chunking Transformer” (ACT) ^[1077] 是标准 VLA 的一种变体, 专为 ALOHA 低成本数据收集平台的精细操纵挑战设计。ACT 输出一段动作序列, 而不是像 RT1 那样只输出单个动作; 随后研究表明, 这是改进 VLA Transformer 架构并获得更好泛化性能的重要修改 ^[84]。

进一步把第 17.2 节介绍的 VLM 和 LLM 的表示能力集成到通用策略的语言和视觉编码器中, 可能继续带来收益。例如 OpenVLA 完全使用开源模型, 包括 VLM (DINOv2) 和 LLM (Llama 2) 组件; 在 64 张 A100 GPU 集群上微调 15 天后, 其机器人操纵性能显著优于仅在 Open-X-Embodiment 数据集上训练的模型 ^[573]。 π_0 ^[84] 也采用了类似方法。

另一种获取通用机器人策略的范式是扩散模型 ^[463]。扩散模型通过逆转逐步加噪过程来生成数据: 训练时从干净数据开始逐步加噪, 直到变成纯噪声; 模型学习逐阶段逆转这一过程, 将带噪输入去噪并恢复原始数据。生成样本时, 从随机噪声开始, 通过迭代细化生成训练数据分布中的样本。这一过程已通过“扩散策略”成功用于生成机器人动作: 将视觉运动策略表示为条件去噪过程 ^[199], 目前仍是非常活跃的研究方向。两种范式并不互斥, 例如 Octo ^[818] 使用大型 Transformer 产生输出, 再将其提供给执行前向扩散过程的“动作头”。

强化学习 (reinforcement learning, RL) 也是训练机器人策略日益流行的范式。不过, 上述大型数据集没有显式奖励概念, 因此需要像逆强化学习那样推断或学习奖励。RL 特别适合在仿真中训练, 因为仿真很容易定义和计算奖励函数, 并且与真实硬件相比, 执行 rollout 的成本较低。RL 还可以作为微调机制, 作用于前述 VLA 或扩散方法生成的策略。

本节绝不是对通用策略训练现状的穷尽性综述; 这一领域发展很快, 更强的新模型频繁发布。随着模型能力增长, 它们隐式编码环境某些表示的能力似乎也在提高, 这可能改变仍需要显式地图表示的场景。

17.4.4 总结性说明

不可否认, 各类基础模型已经像改变其他领域一样改变了机器人学。但这种进步能够持续到什么程度仍有待观察。一方面, 继续扩展机器人数据集和专用基础模型, 并将其与互联网规模的 VLM 和 LLM 结合, 似乎可能产生越来越令人印象深刻的能力, 最终使显式地图表示过时。另一方面, 已有研究表明, 尤其是 Transformer 难以完成组合式或思维链推理 ^[297]。因此, 神经记忆机制正在重新出现, 例如专门设计为具有状态性的“结构化状态空间模型” ^[406], 其方式与循环神经网络相似。但这类神经记忆架构的能力上限, 以及它们如何与通用策略集成, 仍不清楚。

总体而言, 基础模型为 SLAM 地图引入开放词汇语义提供了统一工具, 但当前方法仍多为后处理, 依赖带位姿图像和预训练模型。未来的开放世界空间智能系统需要在建图过程中增量融合语言和视觉特征, 维护实例、关系和不确定性, 并将表示直接连接到任务规划和控制。

¹ 本章沿用文献中的术语, 将不需要人工标注的方法统称为无监督学习, 不再区分自监督学习与严格意义上的无监督学习。

第十八章 空间智能系统的计算结构

作者：Andrew J. Davison

SLAM 算法能够绘制环境地图并估计设备在其中的位置，已经促成了许多重要的新产品和能力，并正在工业自动化、家庭机器人、无人机以及虚拟/增强现实（VR/AR）等领域产生影响。然而，我们认为这只是更大事情的起点：人工设备有能力构建真正丰富而高效的周围环境表示，使其能够以通常意义上的智能方式与环境交互。我们将这种能力定义为**空间智能**（Spatial AI）；这一术语最早在《FutureMapping》^[245]中提出。

本章认为，空间智能能力是本手册前文所述 SLAM 研究的持续演化。我们将预测并分析未来几年空间智能系统的能力和计算结构，尤其关注机器学习以及新型感知与计算硬件将产生的影响。

18.1 从 SLAM 走向空间智能

18.1.1 具身智能设备需要空间智能

空间智能系统的目标不是抽象地理解场景，而是持续构建恰当的表示，以支持实时解释和一般性的智能行动。此类系统的设计，一端由任务对性能的要求界定，另一端则受到设备使用环境和形态所施加的约束。

设想未来面向大众市场的家用机器人产品，任务是监视、清洁和整理若干房间。机器人可以是人形机器人，也可以是具备通用移动和操纵能力的其他平台。任务要求包括清洁或扫除表面并判断何时已清洁；识别、整理、搬运和操纵物体；以及及时而有礼貌地应对人类，例如主动让路或提供帮助。与此同时，由传感器、处理器和算法组成的空间智能系统还要受到价格、外观、尺寸、安全性和功耗等因素的约束，所有这些都必须符合消费产品的范围。

再考虑一个不同领域的例子：不是自主机器人，而是面向人的可穿戴辅助设备（为了对称起见，可以称为“IA”，即 Intelligence Augmentation，智能增强）。我们设想一种未来的增强现实系统，为佩戴者提供关于所遇到的所有地点、物体和人的可靠空间记忆，从而能够轻松找回遗失物体，并在世界中的任意实体上放置虚拟便签或其他注释。这种能力可以让一个人的完整生活上下文自动提供给始终在线的大型语言模型（Large Language Model, LLM）助手，并可能全面增强其智能能力。为了得到广泛采用，该设备应当具有标准眼镜的尺寸、重量和外形（约 65 g），并且全天运行而无需充电（功耗小于 1 W）。

目前，这类强大空间智能系统在有用应用中需要完成的事情，与现实约束下现有技术能够达到的水平之间存在巨大差距。关于所需算法和技术已经有许多很有前景的研究，但即使可以使用昂贵而笨重的传感器和无限的计算资源，要获得稳健性能仍然困难。一旦把真实产品的约束考虑进来，现实与期望性能之间的差距会更加显著。尤其是，目前支持先进机器人视觉的计算处理器，其尺寸、成本和功耗要求还远远无法满足我们设想的这些应用。

18.1.2 场景表示与世界模型

我们认为，空间智能的关键在于表示。表示是保存在计算机内存中的一组数值，它以持久、可适应并且会持续变化的方式描述具身设备及其周围世界的状态：设备获得关于世界的新信息时，或世界自身状态发生变化时，表示都会更新。出于效率原因，设备也可以选择对表示的某些部分进行抽象，甚至将其“遗忘”。

“世界模型”（world model）一词最近在机器学习研究中流行起来^[423]，其含义与场景表示非常接近。许多机器学习研究者将世界模型视为相对较新的研究领域，因为当前训练得到的神经网络架构大多是无状态的：

它们只是处理输入，并将输入确定性地变成输出。机器学习领域目前正在大量研究如何以“端到端”方式训练此类网络执行任务，即在自动驾驶到物体操纵等各种领域，将传感器输入直接转换为动作。这些方法往往极其有效，但难以完成长时域任务。

神经网络中的世界模型研究，旨在为网络配备能够表示系统及其环境变化状态的持久记忆模块，从而支持仿真和更长期的规划。

尽管这是近来的新兴趣，世界模型和场景表示的思想在更广义的人工智能领域当然有悠久历史，而机器人领域的 SLAM 研究正是最清晰的例子：构建持久场景表示，可以实现一致的长时域行为。最直接地说，在自主无人机或扫地机器人等应用中，SLAM 地图可以支持可靠地长期导航至多个航点，或保证覆盖指定区域。

当 SLAM 从为定位构建最小场景表示，扩展到稠密、语义化、对象级甚至能够预测物理状态的场景模型时，设备将可以用一般方式进行仿真和规划。模型预测控制（model-predictive control, MPC）可以在增量构建且准确、最新的机器人环境仿真中运行，即使在物体操纵任务中也能产生最优而富有创造性的行为。

许多机器学习研究者可能会认为，“世界模型”应当比 SLAM 系统通常输出的显式三维场景地图抽象得多。我们同意，仍需要大量研究来确定哪些世界表示能够在保持高效和实用的同时实现真正的空间智能。然而，显式描述场景几何和物体的表示，在效率、可组合性、可解释性和通用性方面具有许多优势。

因此我们提出如下假设：当设备必须长时间运行，执行多种任务（其中并非所有任务都在设计时已知），并与包括人类在内的其他实体通信时，其空间智能系统应构建一种通用且持久的场景表示；该表示至少在局部应接近度量三维几何，并且能够被人类理解。需要明确的是，这一定义为场景表示留下了大量选择空间，表示中可以同时包含学习得到的元素和人工设计的元素；但它排除了使用非常具体的任务专用表示的算法，也排除了完全抽象的“黑箱”世界模型。

我们的第二个假设是：空间智能系统对于广泛任务的有用程度，可以用数量相对较少且具有一般重要性的性能度量来描述。无论系统用于引导配送无人机在狭窄空间中自主飞行、指导家用机器人整理房间，还是让增强现实显示器向场景加入合成物体，虽然这些应用确实具有不同的需求和约束，其空间智能系统总体上应当相似，差别可以由少量性能参数规定。显然的参数包括全局设备定位精度和更新延迟，但我们认为还有一些对应用更有意义的指标，例如表面接触距离预测精度、物体识别精度或跟踪稳健性。第 18.8 节将进一步讨论性能度量。

在本章其余部分，我们将这种模块称为空间智能系统：它从主要由视觉输入构成的数据中，以实时方式增量构建并维护一种接近度量的、具有一般用途的场景表示，并且具有可量化的性能指标。

Judea Pearl 最近在讨论高效的情境学习以及推理因果关系的必要性时指出^[856]：“人类拥有而其他物种缺乏的，是一种心理表示，即他们可以随意操纵的环境蓝图，用来想象替代性的假设环境，从而进行规划和学习。”我们认为，空间智能同样将使先进机器人和设备脱颖而出，把具身智能提升到新的层次。

此外，有证据表明，人类的空间推理能力还被更广泛地用作智能的基本工具：即使面对抽象概念，人类也会通过空间化的方式组织思维^[71]；人工系统可能也能从中受益。

18.1.3 SLAM 正在演化为空间智能

SLAM，尤其是视觉 SLAM 的研究长期以来由实时演示、实时可视化和开源代码发布所推动。MonoSLAM、PTAM、KinectFusion、LSD-SLAM、SVO、ORB-SLAM、iMAP 和 Gaussian Splatting SLAM 等系统的实时演示，与其说是数据集结果，不如说更能代表进展。这是因为数据集通常只能捕获系统性能的有限方面（尤其是精度），却无法突出同样重要、并且可以通过短时实时演示直接评估的稳健性、效率和灵活性。

实时视觉 SLAM 能够实现的场景表示水平逐步提高：从稀疏特征到稠密地图，再到日益丰富的语义标签。商业 SLAM 提供商 Slamcore 分别将稀疏定位、稠密建图和语义标注称为第 1、2、3 级能力。再往上，场景表示发展的最终目的也许是场景图：由准确分割、可进行物理仿真的对象实例组成的几何结构，并以层次方式组织（进一步讨论见第 16 章）。这些都是 SLAM 走向空间智能的连续步骤。观察 SLAM 三十多年稳定而持续的进步后，我们已经确信，当前以及可预见的 SLAM 系统的运行方式，是推断未来空间智能算法结构的最佳依据。

需要强调的是，尽管可以用分级方式理解这一过程，SLAM 系统的进步通常并不是在一个已经工作的系统上简单增加层次。相反，每一次表示方式的改变都会导致整个系统重新设计。例如，一旦拥有稠密场景地

图，就可以用它实现准确而稳健的直接跟踪来完成相机定位；语义场景标注也可以提高稠密重建的质量。这种思维方式是实时系统研究固有的一部分，因为系统的每个部分都会在闭环中影响其他部分。

因此，我们坚信，持续进行的 SLAM 研究是目前发展空间智能最好的模型。

18.2 总体计算结构

当一个至少带有相机和其他传感器的传感器平台在世界中移动时，其运动可能由主动控制产生，也可能由另一名操作者提供。无论空间智能系统属于何种形式，其基本工作方式都可以概括如下：

1. 系统由面向外部的传感器（例如相机）和辅助传感器（例如 IMU）组成；这些传感器与低功耗封装中的处理架构紧密集成，并嵌入机器人或增强现实系统等移动实体。
2. 系统必须实时维护并更新包含几何和语义信息的世界模型，并仅根据或主要根据机载传感器的测量，估计自身在该模型中的位置。
3. 系统应提供大量与任务相关的信息，回答场景中“什么”位于“哪里”。理想情况下，它应提供周围所有物体及其他实体的完整语义级模型，包括身份、位置、形状和运动。
4. 世界模型的表示至少在局部应接近度量表示，从而使人工智能或智能增强系统能够快速推理任意感兴趣的预测和测量。
5. 系统可能只在重点区域保留几何和语义的最高质量表示，最明显的是当前正在观测且与近期交互相关的场景部分。模型的其余部分将以多个残余质量层级存储，重新访问时可以迅速升级。
6. 系统应当总体上保持警觉，即将每一份输入视觉数据都与前向预测的场景模型进行比较，用于跟踪、检测环境变化以及检测独立运动，并能够响应环境变化。

这种方法的关键计算特性是闭环性质：世界模型持久存在并以增量方式更新，以抽象形式表示截至目前获得的所有有用数据，并在实时循环中用于数据关联和跟踪。这与纯视觉里程计等增量估计视觉系统不同；后者只逐帧估计相机运动，不保留长期数据结构。它也不同于只能通过离线、事后批处理来实现全局一致性的系统。闭环空间智能系统并不意味着每项计算都要以固定频率发生；更重要的是，需要时计算必须可用，使整个系统能够无停顿地继续实时运行。

高层思考的下一个要素，是确定如何在低功耗要求下实现上述全部能力并保持高性能。我们认为，空间智能高效处理的关键，是识别算法中计算和数据移动所形成的图，并尽可能使用具有相同性质的处理硬件，或者专门设计这类硬件，尤其要尽量减少数据在架构内部的移动。我们需要回答：什么存储在哪里？什么在哪里处理？什么在何时传送到哪里？

这里不打算在神经科学方面作出强类比，但显然可以把人工空间智能系统的思想和设计，与生物大脑的视觉及空间推理能力和结构联系起来。人脑似乎只用不到 10 W 的功耗，就能实现高性能、完全“嵌入式”的语义和几何视觉；它的结构也确实体现了我们讨论的某些概念。我们把进一步分析这些关系的任务留给其他作者，部分原因是我们缺乏神经科学方面的专业知识，部分原因则是我们认为，虽然人工视觉系统显然仍有很多地方可以向生物学学习，但它们不必被设计成复制大脑的性能或结构。我们从纯工程角度考虑空间智能计算问题，目标是在最小资源下获得应用所需的性能。我们的解决方案有些方面模仿生物进化发现的机制并不奇怪，但在另一些方面，由于两个对比，我们也可能找到非常不同的方法：第一，进化采用的是增量式、必须始终工作的设计路线，而人工智能设计拥有更大的自由度；第二，生物湿件与可用作计算基底的硬件存在差异。

在将这种计算分析具体化、提出空间智能实现的通用设计之前，还需要考虑两个重要主题：(i) 状态估计与机器学习；(ii) 未来处理器和传感器硬件的整体格局。下面两节将讨论这两个主题。

18.3 空间智能中的状态估计与机器学习

本手册的大部分内容介绍了由人类设计的算法和表示，主要是基于概率论的“状态估计”方法。在后面的章节中，我们看到多种机器学习技术逐渐出现。机器学习在 SLAM 中的关键工具是人工神经网络。相对于更广泛的人工智能领域，SLAM 的特点是尚未完全被机器学习统治。定位和稀疏建图的最佳系统仍主要由人工设计；稠密和语义 SLAM 系统则采取折中方式，用机器学习替代状态估计的一部分功能，或在其上增加功能。不过，机器学习技术逐步覆盖空间智能更多功能的趋势是不可否认的。

在机器学习首次用于 SLAM 的一些工作中，学习模块只增加了一层功能，并未直接与状态估计组件交互。例如，SemanticFusion^[747] 使用基于 CNN 的语义图像分割，为 ElasticFusion^[1184] 生成的稠密几何地

图增加标签。随后人们逐渐认识到,学习组件可以在 SLAM 的核心几何估计循环中发挥重要作用,主要形式包括:

1. 训练网络从单幅图像预测几何属性,例如深度图、法向图或物体分割。2. 用组件更新标准几何估计(如位姿和点云),DROID-SLAM 就采用了这种方式;这类网络通常执行匹配或几何优化等运算。3. 最新的网络从多幅图像输入产生质量更高的几何预测,例如 DUST3R 和 VGGT。

尽管神经网络可以直接预测几何(例如从单幅图像预测深度图),但在 SLAM 中使用这些输出一直很困难,因为单视图预测通常包含很大的系统误差,而且其不确定性并不容易理解。一条有希望的路线,是训练网络预测比原始深度稍弱的属性,例如编码深度 CodeSLAM^[91]或深度协方差^[269],再通过低维的多视图优化实现一致的稠密几何融合。

单视图研究最终可能希望得到这样一种网络:它把输入图像转换为一组可靠的、具有语义且类似物体的实体,随后用这些实体填充高效、抽象的三维物体地图或场景图,并通过持续的动态多视图优化和更新来维护。SLAM++^[964]首次探索了这一思路,它直接在已识别的三维物体层级构建高效场景图地图,但要求预先拥有每个物体的三维模型。该思路的现代推广是 SuperPrimitives^[744],它使用通用的单图像物体分割和局部重建,从而支持任意三维物体片段的长期建图。

机器学习在空间智能中的吸引力可能有两个关键方面。第一,神经网络能够学习执行一些很难手工设计的任务,例如场景标注或深度预测。第二,神经网络将迭代计算压缩为高效而计算简单的形式,可以在现代处理器上以前馈方式快速运行。

我们自然会问,未来 SLAM 和空间智能是否最终会完全由神经网络端到端完成,而不再需要人工设计的表示或状态估计。许多研究者确实相信这一点;从端到端视觉里程计早期的 DeepVO^[1163, 1299],到能够直接从一组无位姿图像输出三维点云的 VGGT,已经出现越来越令人印象深刻的工作。

这些工作必然会继续改进,但我们认为,在可预见的未来,结合学习和状态估计的混合方法仍然更可取,理由有若干。首先,网络预测的不确定性仍然难以理解,而系统需要持续更新并融合数据到模型中。假设一个网络能够根据 100 张图像生成三维场景模型,当再获得一张图像时,应当如何更新它?显然不应该把包含 101 张图像的整个网络重新运行一遍。一旦承认需要长期表示和数据融合,就需要概率状态估计工具,也就有充分理由采用模块化场景表示。

从中期来看,空间智能系统必然会把学习组件和人工设计组件紧集成到难以区分的程度。这个状态既可以通过向人工设计系统不断加入学习模块达到,也可以通过向纯学习系统加入结构达到。无论采用哪条路线,仍然重要、并且正是本章关注中心的,是整个系统的存储和计算结构。

18.4 处理器和传感器硬件的未来格局

18.4.1 处理器

多年来,SLAM 研究处在这样一个时代:单核 CPU 处理器的时钟频率(因而串行处理能力)可以可靠地每 1~2 年翻一番。近年来这一点已经不再成立。描述集成电路中晶体管密度翻倍速度的摩尔定律的严格定义,直到当前仍大体成立;改变的是,晶体管密度不再能按比例转换为串行 CPU 性能。这是因为另一条不太知名的经验规律“Dennard 缩放”已经失效:该规律认为,晶体管缩小时其功率密度保持不变。当晶体管缩小到今天的纳米尺度,尺寸已经不再远大于组成它们的原子时,漏电流和发热会变得严重。这堵“功耗墙”把能够在不过热的情况下合理运行的时钟频率限制在约 4 GHz。

因此,处理器设计者越来越需要通过提高时钟以外的方式提升计算性能。正如 Sutter 的在线文章《Welcome to the Jungle》^[1062]所描述的,处理器格局正变得更加复杂、并行且专用;即使是“云”数据中心中的处理器设计,也在变得更加多样和复杂。对于空间智能这类嵌入式应用,摆脱 CPU 的压力更大,因为功耗是关键问题;在满足真实产品功耗限制的前提下,并行、异构、专用处理器似乎是达到空间智能所需计算性能的唯一途径。因此,当前无人机等嵌入式视觉系统经常只用 CPU 实现 SLAM 算法(而不是要求 GPU),但我们认为长期来看这并不是正确方向。虽然当前桌面 GPU 确实耗能较高,空间智能必须充分拥抱并行性。不过,GPU 只是未来几年将出现的广阔处理器设计空间的起点。对于这一领域的大胆思考,我们强烈推荐 Julien Martel 的近期博士论文^[734]。

大约 20 年前, 主流几何计算机视觉开始利用 GPU 形式的并行处理 (例如^[800]); 在空间智能中, 这促成了稠密 SLAM 的突破^[800, 801]。GPU 提供的 SIMT (Single Instruction, Multiple Threads, 单指令多线程) 并行性非常适合实时视觉中的一类操作: 在图像空间或地图空间的规则数组的每个元素上同时执行相同运算。同时, GPU 也是计算机视觉深度学习兴起的核心, 因为它提供了训练足够大规模神经网络所需的计算资源, 使其最终能够在图像分类等重要任务中证明价值。

从 CPU 转向 GPU 作为计算机视觉的主要算力来源, 只是处理技术演化的开始。我们预见未来的嵌入式空间智能系统将采用异构、多单元、专用的架构, 在高性能的同时实现低功耗。十年后的嵌入式视觉标准 SoC (system-on-chip, 片上系统) 可能用于个人移动设备、消费机器人或 AR 头戴设备, 其中仍会有与今天 CPU 和 GPU 设计相似的单元, 因为它们灵活且可以运行数量巨大的实用软件; 但它很可能还会包含多个针对低功耗实时视觉优化的专用处理器。

兼顾速度和低功耗的高效处理, 关键是把计算分配给大量时钟频率相对较低或结构更简单的核心, 并尽量减少核心之间的数据移动。CPU 在计算过程中逐个从独立的主存储器中取出和写回小块数据; 除了对经常使用的数据进行本地缓存外, 它没有太多减少数据流的方法。单 CPU 编程较为直接, 因为任何算法都可以分解为访问单一中心内存的顺序步骤, 但数据在中心内存与处理器之间逐块流动会带来巨大的功耗。

更高效的处理器设计会让处理单元与所操作的数据彼此靠近, 并限制中间结果的传输。理想方式是让处理器/存储器架构的设计与其运行的算法高度匹配。对于许多计算机视觉处理任务, GPU 相比 CPU 具有明显优势, 但归根结底, GPU 最初是为计算机图形学而非视觉和人工智能设计的处理器。它的 SIMT 架构能够高效运行对大量不同数据元素同时执行相同操作的算法; 而在完整的空间智能系统中, 仍有许多方面不适合这种方式, 因此目前需要 CPU/GPU 联合架构, 并在二者之间进行大量数据传输。

虽然设计定制的“加速器”处理器来高效实现某些特定低层算法相对容易, 但直到最近, 人们很少从整体计算结构出发思考空间智能这类闭环嵌入式系统。如今, 低功耗视觉已成为工业界日益重要的目标, 因而出现了 Movidius Myriad 系列等定制处理器。这些处理器将低功耗、类似 CPU 的单元、类似 DSP 的单元和定制单元组合在一个完整封装中。Microsoft 为第一代 HoloLens AR 头戴设备定制的 HPU 也采用了相似设计。更近期的 Apple Vision Pro 使用定制的 Apple R1 协处理器处理实时传感器输入; Meta 的 ARIA Gen 2 智能眼镜研究设备则采用了用于“超低功耗、设备端机器感知”的定制硅片, 使 SLAM、眼动跟踪、手部跟踪和语音识别无需外部电池即可在设备上运行。这些商用芯片及其运行算法的细节属于机密, 但可以设想, 它们是为高效运行小型神经网络、其他图像处理以及一定程度的通用概率优化而定制的。

如果把视线放得更远, 我们可以设想处理器架构, 使其与算法之间更加紧密匹配。与我们的目标特别相关的是, 目前正在开展的重大工作: 用大量相互独立、规格相对较低的核心进行大规模处理, 重点关注通信。曼彻斯特大学的 SpiNNaker^[348] 是一个旨在模拟生物大脑的研究项目; 它已制造出原型机, 每块板上有数百个 ARM 核心, 总数最多可达一百万个。以不同的商业目标提供一种重要的新型 AI 处理器的 Graphcore, 是一家英国公司, 正在开发“智能处理单元”(Intelligence Processing Unit, IPU) 处理器。IPU 在单芯片上包含数千个核心, 每个核心都可以在自己的本地内存上运行独立程序, 并通过高效可编程的互连结构通信。Tenstorrent 和 Cerebras 等其他公司也在设计有趣的新型芯片。

这些项目主要关注神经网络的高效实现。SpiNNaker 和 Graphcore 都认为, 关键在于大量核心的整体拓扑: 每个核心执行不同的操作, 但通过适配使用场景的图结构实现高效互连。这些设计并未对每个核心执行的处理类型或交换的消息类型作出强约束, 希望把这些选择留给未来程序员。这与更明确的神经形态架构相反, 后者试图实现对生物大脑运行方式的特定模型, 例如 IBM 的 Truenorth 项目。

这类架构目前还没有接近嵌入式视觉的实用阶段, 但对于设计空间智能系统具有巨大的长期潜力: 算法和数据存储的图结构可以与处理器上的实现相匹配, 从而获得定制且可能极其高效的系统。后文将进一步讨论这一点。

我们还认为, 不应只考虑把空间智能映射到单个处理器, 即使该处理器内部具有图结构。更一般的图还可以描述与相机及其他传感器、执行器和其他输出的通信; 描述共享同一环境的其他独立机器人和设备; 并且可以扩展到云中的、完全位于设备之外的计算资源。

18.4.2 传感器

相机是空间智能中最重要的传感器。随着传感器设计者不断创新,“相机”的概念如今正变得越来越宽泛;在本章中,我们把本质上能够捕获光测量阵列的任何设备都视为视觉传感器。大多数相机是被动的,即记录和测量从周围环境到达的环境光;另一大类相机则以某种受控方式发出自己的光。在空间智能中已经使用了许多相机类型,最常见的是被动单目和双目相机组,以及基于结构光或飞行时间概念的深度相机。每种相机设计都代表一种权衡:它提供的信息质量(例如空间和时间分辨率、动态范围)与对所用系统施加的约束(例如尺寸和功耗)之间需要取舍。我们在先前工作^[428]中研究了实时跟踪这一空间智能子问题中性能与计算成本之间的部分权衡。

正如本手册已经讨论的,事件相机是空间智能中一种特别有前景的传感器。它只感知并传输强度变化,从而去除冗余。事件流以低得多的比特率编码视频的信息内容,同时在时间灵敏度和强度灵敏度方面具有优势。这很可能只是图像传感器技术即将快速变化的起点;低功耗计算机视觉将成为越来越重要的驱动力。

该领域一个重要的学术项目,是曼彻斯特大学开发的带有平面内处理能力的 SCAMP 系列视觉芯片(介绍见^[735])。SCAMP5 以 1.2 W 运行,图像分辨率为 256×256,每个像素都可以由逐像素处理单元控制 and 处理。利用模拟电流模式电路,可以极其快速、高效地完成求和、减法、除法、平方以及与相邻像素通信等操作,从而完全在芯片上实现相当程度的实时视觉处理。在只需要低更新率的应用中,也已经可以实现超低功耗运行。

正如后文将讨论的,这种图像平面处理器最终能够执行的视觉处理范围还有待充分探索。最直接的用途是前端预处理,例如特征检测、局部运动跟踪或分割。我们认为其长期潜力在于:完整的、基于模型的空间智能仍需要中心处理器,但靠近传感器的处理可以通过低通信量的双向交互与中心处理器完全协同,主要目标是减少传感器与主处理器之间所需的比特率,进而降低通信功耗。

最后,在考虑空间智能计算资源的演化时,绝不能忘记云计算资源将继续扩大容量并降低成本。未来所有空间智能系统很可能大多数时间都连接到云端;从系统角度看,可以假设云端的处理和存储资源近乎无限且免费。真正不免费的,是嵌入式设备与云之间的通信,尤其当传输视频等高带宽数据时,通信可能消耗大量功率。另一个重要因素是时间延迟:将数据发送到云端处理通常会产生显著的几分之一秒延迟。在某些应用中,始终在线的高带宽云连接也不可行,例如太空、水下或地下环境。

18.5 将空间智能图映射到硬件

现在更具体地讨论空间智能设备的架构设计。尽管通过云连接进行共享建图具有明显潜力,本节选择只关注这样一种单设备:它只依靠机载资源在环境中运行。这是能力最通用的配置,可以用于任何应用而无需额外基础设施。

我们认为,在空间智能计算系统的总体架构中,实现高效性能的长期关键,是让算法的自然图结构与物理硬件的配置相匹配。正如前文所见,这种思路会引出靠近传感器的处理(例如平面内图像处理),并通过事件等原则,尽量减少从传感器以及向执行器或其他输出传输的数据。

不过,我们仍然认为,嵌入式空间智能系统的大部分计算最好由相对集中的处理资源完成。关键原因是,增量构建并使用的世界模型始终发挥着核心作用,它表示系统对自身状态和环境状态的认识。来自可能多个相机和传感器的每一份新数据,最终都要与该模型比较:如果与设备的短期或长期目标相关,就用于更新模型;否则就被丢弃。

18.5.1 世界模型处理

我们为这种中心处理资源设想的形式,远不是 CPU 加上类似 RAM 的存储器。CPU 访问 RAM 的任意内容时成本大致相同,但在空间智能中,世界知识数据的局部性以及把新数据纳入其中的计算负载组织,都包含更多结构。

处理和存储的图结构应当内置于中心世界模型计算单元的设计中,并以完全集成的方式将存储与处理结合起来。当前已经有许多重大工作,正在探索如何设计显式且灵活的图状架构。

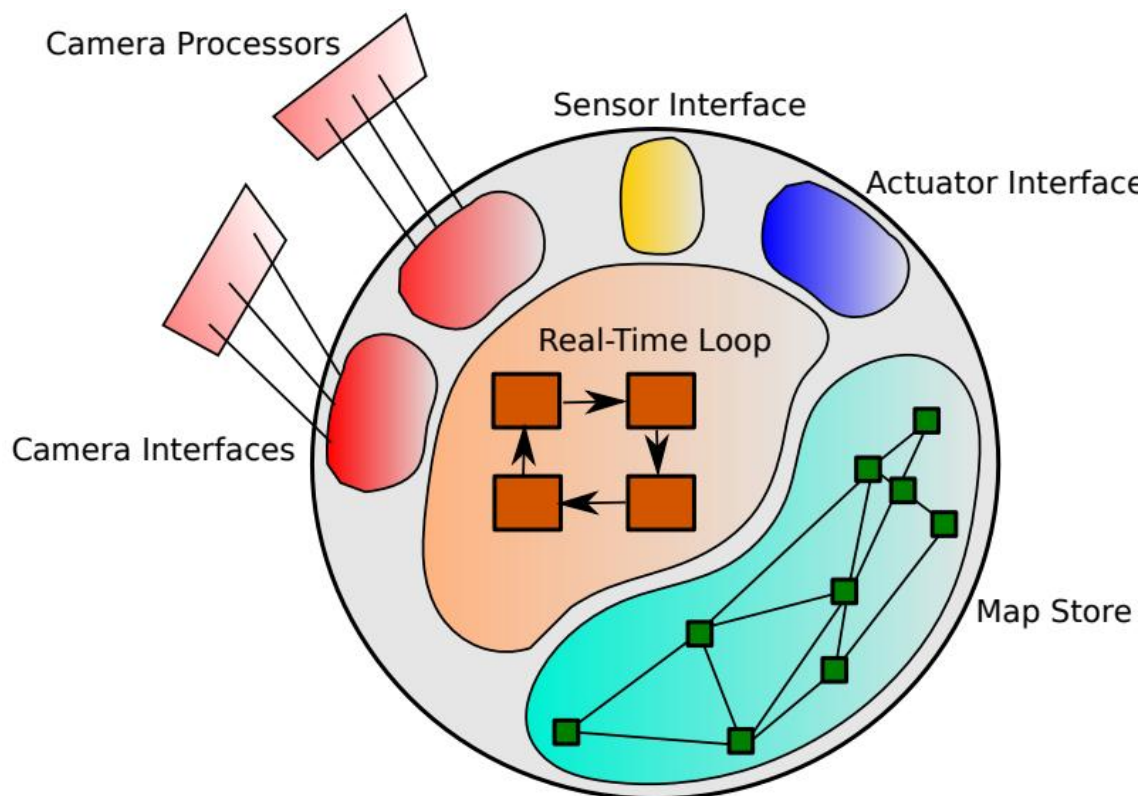


图 18.1: 图 18.1 空间智能大脑：一般空间智能系统的表示图和处理图结构如何映射到图处理器。关键元素包括实时处理循环、基于图的地图存储，以及与传感器和输出执行器连接的模块。我们还设想在视觉传感器中集成额外的“靠近传感器”处理，以降低主处理器与相机之间的数据带宽；这些相机通常位于距离主处理器一定距离的位置，未来通信可能是双向的。

以一种处理器概念为例，Graphcore 的 IPU（或图处理器）设计用于高效执行能够很好地建模为稀疏图操作的 AI 负载，尤其适合所有所需存储都保存在同一图中、而不是外部存储器中的情况。IPU 芯片包含大量独立核心（数以千计），每个核心都可以运行任意程序并拥有自己的本地存储；同时还具有强大的通信基底，使核心之间能够高效发送消息。要在 IPU 上运行程序，首先需要分析它所需的计算和通信模式，并将程序编译成适当形式，然后生成经过优化的操作、数据和通信通道位置的图拓扑。

图 18.1 首次尝试绘制了一个“空间智能大脑”模型，类比 Graphcore 将计算图编译到 IPU 上的方式。圆盘表示我们预计会在主处理器内部运行的模块。两个主要区域之一是地图存储，其中保存当前世界模型。地图存储具有与世界几何相关的内部图结构，也包含相当强的内部处理能力，可以在模型数据的局部运行计算，后文将讨论其作用。第二个主要区域是实时循环，主实时计算在此完成，将输入图像流连接到世界模型。实时循环具有处理图结构，必须支持大规模实时数据流以及对图像/地图结构的并行计算，因此需要针对这些需求进行优化。

主处理器还包含其他模块。每个相机接口都负责对其所连接传感器到达的数据进行建模和预测。接口再与相机本身连接；机器人或其他设备的物理设计可能迫使相机距离主处理器较远。连接可以是串行的，也可以沿多条并行线路进行。

我们进一步设想，每个相机都内置自己的“靠近传感器”处理能力，并通过数据链路与主处理器分开。主处理器对输入建模的目标，是尽量减少实际传输到靠近传感器处理器的数据。靠近传感器处理器可以完全采用图像驱动的计算，类似 SCAMP 项目，并向主处理器提供抽象后的表示；也可以在相机和主处理器之间进行双向数据传输。主处理器把模型预测发送给靠近传感器的处理器后，后者就知道主处理器已经拥有哪些内容，只需报告差异。这是事件相机概念的推广：事件相机只报告强度变化，而未来最优的高效相机应报告接收到的数据与预测不一致的位置。第 18.5.3 节将进一步讨论靠近传感器的处理。

18.5.1.1 世界模型

许多重要的 SLAM 方法已经认识并利用了世界模型的图结构（例如 [534, 595, 964, 307, 754]）。

当机器人在世界中移动并进行观测时，SLAM 算法会检测、跟踪从图像数据中提取的特征，并将它们插入地图。这里“特征”一词采用广义含义，表示场景实体的某种抽象，并不局限于稀疏的点状路标。随着空间智能的抽象程度提高，这些特征很可能变成物体或其他语义实体。场景中的每个特征都有一个可以从中测量它的相机位置区域。当特征位于传感器视场之外、被遮挡，或者由于与首次观测时相比距离相机太远或观测角度差异太大等原因时，该特征就不可测量。

因此，机器人穿过场景时，特征会以具有不同重叠程度的模式变得可观测。对当前可见特征进行测量时，可以改进其位置估计；这些测量也用于估计相机运动。通过不确定的相机状态，同时观测到的特征位置估计彼此之间会产生强相关。就观测它们之间发生的相机运动量而言，“邻近”特征仍然相关，但相关性较弱；而“遥远”特征之间发生了大量运动（并且 SLAM 依赖中间特征），因此只有弱相关。

SLAM 算法输出的特征位置概率联合密度，可以用图高效表示：以强边连接“邻近”特征，以弱边连接“遥远”特征。对概率表示精度设置阈值，就可以切断较弱的边，从而得到只连接“邻近”特征的稀疏图。

SLAM 的一种做法不是显式估计场景特征的状态，而是构建移动相机历史上部分位姿的地图，并使场景地图保持隐式。这通常称为位姿图 SLAM (pose-graph SLAM, 参见第 1 章)；在这种地图中，图结构显而易见，因为我们将能够获得传感器对应关系的位姿连接起来。时间上连续的位姿相连；如果能够检测到经过较长时间后的重新访问，也会把相应位姿连接起来，即形成回环。无论图表示历史位姿还是场景特征，其结构都非常相似：它连接“邻近”的位姿，或连接从这些位姿测得的特征；两种方法在本质上没有区别。

由此我们得出结论：空间智能最可能采用的表示，是用特征图表示三维空间。特征按照与相机运动有关的多尺度模式连接在一起，并共同生成稠密场景预测。在主处理器内部，应当用一个主要区域存储这张地图；地图可以分布在大量独立核心上，而核心之间的连接拓扑仿照地图图拓扑。SLAM 地图是动态定义和增长的，因此处理器中的图也必须能够动态变化，或者一开始就定义足够大的未使用容量，并随着 SLAM 运行逐步填充。

重要的是，大规模 SLAM 的很大一部分相关处理都可以直接构建进这张图。这些处理主要是地图进行自我优化、细化和抽象的“维护”计算，包括：

1. 特征聚类；物体分割与识别。
2. 回环检测。
3. 回环优化。
4. 地图正则化（平滑）。
5. 通过无监督聚类

发现新的语义类别。

随着时间、数据和计算量的增加，一张起初包含稠密几何和低层特征的地图，可以逐渐细化为高效的对象级地图。这些操作将以高度并行且具有数据局部性的方式运行。例如，平滑和分割可以分别在图的各个局部进行；全局地图优化则可以通过消息传递实现，具体见第 18.6 节。最重要的是，所有这些过程都可以在地图存储内部完成。

当前 Graphcore IPU 等图处理器的片上内存完全分布在各处理 tile 之间，总容量较大（约每个 tile 1 MB，1000 个 tile 总计约 1 GB），但并不算巨大，尤其与处理器外部的标准 RAM 相比。因此，应当强调快速将地图存储抽象为高效的长期形式。不过，未来处理器可能拥有容量大得多的分布式存储。

18.5.2 实时循环

我们提出一个假设：未来 SLAM 系统最紧密的实时循环，其核心计算图将包含许多与今天系统相似的元素。这里的中心是 Newcombe 等人^[799, 801, 800]提出的稠密 SLAM 范式，因为这种方法试图以稠密、生成式的细节建模世界，使相机输入的每个新像素都可以与基于模型的预测进行比较。系统因此能够保持总体“感知”：它持续建模相机前方世界的状态，可以检测偏离模型的内容。基于稠密 SLAM 的系统也正在从静态场景走向动态场景的重建和跟踪^[802, 957]。

稠密 SLAM 系统还可以最好地利用场景先验；这些先验今后将越来越多地来自学习，而不是人工设计。实时循环中的主要计算元素包括：

1. 图像到特征的经验标注，通常由神经网络完成。
2. 渲染：将世界地图中的内容生成稠密预测并投影到图像空间。
3. 跟踪：将预测与新图像数据对齐，包括找出异常值和检测独立运动。
4. 融合：把更新后的几何和标签融合回地图。
5. 从运行中的系统进行自监督学习。

这种处理既依赖相机和其他传感器的实时输入，也依赖世界模型，因此从某种意义上说必须位于二者之间。当需要预测场景视图、以便与新图像数据进行跟踪时，需要从地图存储读取数据；当使用新数据获得的几何和标签更新地图内容时，也需要从地图存储读取数据。这通常是 GPU 加速稠密 SLAM 中所熟悉的大规模并行处理。可以把这些函数定义为计算图中的固定元素，使用若干节点并访问主要计算资源的很大一部分。输入图像的分割和标注也可以在这里实现，但正如后文将讨论的，更合理的做法是通过靠近传感器的处理将它们放在主处理器之外。

对实时循环应用图处理最困难的问题，是这些操作与基于图的地图存储中相关部分的对应关系会因运动而持续变化。这样似乎无法为分布式存储器定义高效的固定数据路径。虽然地图存储内部会执行处理，但这些处理重点是维护地图；让数据在地图存储中快速移动，例如始终把当前观测区域放在图中的某个固定位置，似乎并不恰当。

一种可能的解决方案，是定义位于实时循环模块和地图存储之间的专用接口节点。这些节点专注于通信：一方面与实时循环处理的相关组件连接，另一方面与地图图中的多个位置连接；它们在某些方面可能类似哺乳动物大脑中的海马体。如果地图存储具有“小世界”拓扑，即任意部分之间都可以通过少量边跳数连接，那么接口节点就应能够用少量操作访问（复制）任意相关地图数据，并将其提供给实时循环。地图存储中的每个节点也必须参与这种通信过程，有时作为地图数据来回复制的路径。

18.5.3 靠近图像平面的处理

机器人或其他设备会有一个或多个与主处理器连接的相机。我们认为，未来技术将允许大量处理发生在传感器内部或其附近，核心目标是减少从相机流出的冗余数据。组成相机图像传感器的像素具有规则的、通常为矩形的几何结构。虽然每个像素通常独立感知光强，但许多视觉算法利用了邻近像素输出高度相关这一事实：邻近像素往往观测同一场景物体和结构的不同部分。

最常见的做法是使用规则图，将每个像素与它的四个邻居（上、下、左、右）连接起来，以此作为稠密匹配或光流估计等早期视觉问题中平滑操作的基础（例如^[879]）。图像的规则图结构也被 CNN 的早期卷积层用于各种计算机视觉任务。卷积的多个层次同时体现了图像数据中局部规则性通常具有层次结构这一事实。

最直接地，可以使用类似 SCAMP5^[735]的具有平面内处理能力的传感器，对输入图像流执行完全自底向上的处理：抽象、简化和检测特征，将图像压缩为更紧凑且信息更丰富的形式。也可以执行局部跟踪（例

如光流估计)、分割和简单标注。这与视网膜和视神经执行的“早期视觉”处理存在生物学上的类比。

利用平面内处理进行跟踪是一个有趣的问题。平面内处理适合数据访问保持高度局部的任务，因此估计局部图像运动（光流）很合适，因为输出是每个像素处的运动向量。在每次更新时，可以利用基于相邻像素差异施加的局部平滑正则化来增强局部图像变化。如果考察 Pock 的 TV-L1 光流算法^[879]的并行实现，会发现它包含逐像素并行操作：与数据项有关的纯并行步骤，与通过相邻像素之间消息传递实现梯度计算的正则化步骤交替进行。因此，这类算法非常适合在平面内处理器上实现。

SLAM 通常需要更具挑战性的跟踪：我们希望根据局部图像变化估计与模型相关的一致全局运动参数。例如，可以通过整幅图像对齐^[707]估计两帧之间的全局旋转；也可以像稠密 SLAM^[698, 799]那样针对持久场景模型跟踪相机。在标准处理器上实现稠密跟踪时，需要在两种步骤之间交替：对所有像素并行计算误差项；通过归约步骤将所有误差相加并更新全局运动参数。归约步骤施加全局模型，扮演正则化角色；但这里的正则化是全局的而非局部的。

在现代系统中，这种全局跟踪通常最好由 GPU 和 CPU 协同完成，因此需要在二者之间定期（且代价高昂地）传输数据。但如果希望采用平面内处理并保持所有计算和存储的局部性，就不能使用这种方案。保持数据局部性的主要选择，是放弃跟踪解必然具有全局一致性的要求，而是通过局部消息传递逐渐趋近于一致。每个像素都可以保留所关注全局运动参数的自身估计，并在每次迭代后将估计与局部邻居共享。我们预计最终将达到全局收敛；经过一定次数的迭代后，每个像素保存的数值应当足够接近，以至于查询任意单个像素都能得到可用估计。如果平面内处理器还具有像素之间的较长数据传输链路，或者更一般地具有“小世界”互连模式，收敛速度会更快。

至于利用局部处理进行标注，这当然可行；但复杂标注面临的问题是，当前平面内处理器的内存非常小，很难存储学习得到的卷积掩模或类似数据。未来用于自底向上处理的处理器可能不再局限于图像平面，而是采用适合实现 CNN 各层的三维堆叠方式。目前三维堆叠仍然难以制造，并面临散热和成本方面的特殊挑战，但最终提取特征层次必将成为计算机视觉相机的内置能力。Featuremetric tracking^[235]等工作展示了这种能力的广泛用途；我们应当研究一个完全前馈的 CNN 在多任务学习训练下可以输出的全部内容。

一个有趣的问题是，靠近图像平面的处理是否会始终保持纯粹自底向上，或者相机与主处理器之间的双向通信是否值得采用。双向通信可以把世界地图中的模型预测以某种频率发送到相机，从而在相机侧执行更高层次的处理，例如基于模型跟踪相机自身运动或已知物体。

在极限情况下，如果能够把完整的基于模型的预测传送给相机，那么相机只需返回与预测不同的信息。这在某种意义上是事件相机工作方式的极限。事件相机在像素亮度发生变化时输出数据，相当于假设默认情况下相机看到的场景保持不变；一般的“基于模型的事件相机”则会在发生偏离预测的事情时输出数据。这些问题归结为“自底向上与自顶向下”处理之间的关键问题，下一节将进一步讨论。

最后，任何空间智能系统最终都必须提供由任务决定的输出：可以是发送给机器人执行器的命令，也可以是发送给另一台机器人的通信消息，或是在智能增强场景中发送给人类操作员的注释和显示内容。正如“靠近传感器”的处理有助于提高效率，“靠近执行器”的处理也应发挥作用，尤其是因为执行器或通信通道本身具有需要在紧密循环中纳入并融合的感知信号，例如执行器的扭矩反馈、AR 显示器的眼动跟踪或其他测量。

18.6 使用高斯信念传播的收敛式分布式计算

我们已经论证，效率和低功耗要求空间智能系统采用图结构：处理和本地存储被划分为模块，并可能分布在多个硬件处理器和传感器之间。但到目前为止，对于如何使这些存储的表示保持最新，还没有作太多说明。正如本手册各处所述，概率和估计理论为 SLAM 中的状态估计提供了基础背景；尤其是因子图^[250]，一直是灵活表示复杂 SLAM 系统结构的通用工具。

Sutton 著名的 AI “苦涩教训”^[1063]指出：“能够利用计算的通用方法最终最有效，而且优势非常大。”在空间智能中，如果我们确实相信低功耗计算和存储资源会在处理器内部以及处理器之间越来越分布式，那么值得选择的算法就应当能够随这些条件扩展。空间智能问题中最纯粹的知识表示是因子图本身，而不是由因子图推导出的概率分布，因为后者始终需要以某种近似形式存储。我们真正要寻找的是一种算法：将因子图作为主表示，在其上原地运行局部计算和消息传递，按需估计变量，同时考虑全局影响。消息应当持续在大型因子图中“冒泡”传播；因子图也会随着新测量因子和变量节点的加入而不断变化，可能永远无法完全

收敛，但始终以可控方式保持接近收敛。估计过程还可能由注意力驱动：对当前重要的区域或方面投入大量计算，使其达到高质量；当注意力转移后，再以“及时”（just-in-time）方式允许这些区域逐渐退化到不那么最新的状态。

如果希望因子图估计具有完全局部的计算和数据存储性质，就必须放弃一种想法：即任何时候都能获得整个图结构的“上帝视角”。我们应当转向这样的方法：处理和存储图中的每个节点只需了解整个图结构的极少信息，最低限度是只知道自身和邻居的纯局部信息。

这正是概率因子图推断中信念传播（Belief Propagation, BP）算法族的特性。在 BP 中，每个变量节点和因子节点处理消息时，除了直接邻居之外不需要了解图的其余部分；BP 可以在任意异步消息传递调度下收敛，不需要全局协调。

当前几何空间智能估计中最重要的技术，尤其是扩展卡尔曼滤波和束调整（bundle adjustment, BA），都是基于高斯的技术。基于高斯的方法与线性代数有非常紧密的联系，因为优化高斯似然等价于最小二乘最小化，而后者需要求解线性系统。当图中的所有因子和变量都呈高斯形式时，BP 的具体形式称为高斯信念传播（Gaussian Belief Propagation, GBP）；在含有环的图中，GBP 对最优解具有特别强的收敛性质。

Davison 和 Ortiz 在《FutureMapping 2》^[242] 中给出了 GBP 的一般介绍；在线文章《A Visual Introduction to Gaussian Belief Propagation》^[833] 提供了交互式教程。这些文章表明，GBP 完全兼容空间智能问题中无处不在的非线性和稳健因子，并且可用于位姿图优化（PGO）、稀疏 SLAM、曲面拟合和图像平滑等问题。

此前已有工作在单个 CPU 上实现 GBP-SLAM^[906]，展示了它即使在单 CPU 实现中也具有一些有用性质。Ortiz 等人^[832] 则通过在 Graphcore IPU 芯片上使用 GBP 实现高性能 BA，展示了它的长期潜力：完全分布式实现求解小型真实视觉优化问题的速度比 CPU 快 30 倍。图 18.2 展示了如何把 BA 因子图简单地铺设到图处理器的 tile 上，以充分利用其并行性。

Murai 等人^[788] 在 Robot Web 中进一步展示了 GBP 图上分布式计算的通用性，说明它也可以用于多机器人空间智能问题。具体而言，多机器人定位被表述为图 18.3 所示的单一因子图，并被划分、存储在由 Wi-Fi 等临时通信连接的机器人网络中。Robot Web 表明，即使消息传递异步、传感器发生故障并且通信丢包，GBP 仍能得到收敛且准确的估计。当展望机器人处理器的长期发展时，这些性质对单机器人空间智能同样很有希望：未来机器人处理器可能采用低精度数值表示以及异步或事件驱动并行等更接近大脑的机制，从而获得更高的单位能耗性能。

18.7 因子图中的持续学习

第 18.3 节讨论了学习组件在空间智能中日益增加的重要性，但将这些组件与模块化状态估计连接起来仍然笨拙且存在许多局限。展望更远的未来，我们认为空间智能中的学习应当具备以下几项雄心勃勃的性质：

1. 学习将成为空间智能系统的关键部分，但应当与经过验证的人工设计概率推断算法无缝并行；后者能够正确地对多个不确定信息源进行加权和融合。
2. 学习的主要方式应当是自监督的，即正在运行的系统可以在没有外部帮助的情况下学习有用抽象，而不需要人类提供带标签的数据集。
3. 空间智能中的学习应当是持续的，作为运行系统的一部分进行，并使系统性能逐步提高。我们应当摆脱当前人为划分的“训练时间”和“测试时间”。
4. 学习组件最好能够解释，或者至少在模块以及模块之间连接的某个层级上能够被人类理解。

大多数学习算法，尤其是标准神经网络，都是分开训练和使用的。训练阶段通过梯度下降寻找模型参数（权重）的数值，并使用经验数值确定学习率和停止准则；同时还引入了将数据划分为训练集和验证集等临时概念。很难想象这类方法如何能够与概率估计一起，用于真正的持续、自监督学习。

我们首先认为，上述要求促使空间智能采用概率式机器学习方法。在概率机器学习或贝叶斯机器学习中，表示学习模型的参数使用与标准概率推断相同的概率推断规律进行估计。概率推断中的状态估计与贝叶斯机器学习中的模型学习之间不存在根本差别，区别只在于结构和规模。二者使用同一套机制，就可以在标准贝叶斯推断的同时正确处理自监督、持续学习。

高效且可扩展的贝叶斯机器学习是一个具有挑战性的主题，但 Nabarro 等人^[793] 最近提出并初步演示了一种 GBP Learning 方法，它与本章提出的因子图空间智能构想完全兼容。核心思想是把过参数化的学习结构直接构建到因子图中，并使用标准的高斯信念传播机制进行贝叶斯推断，从而完成学习。我们不再使用

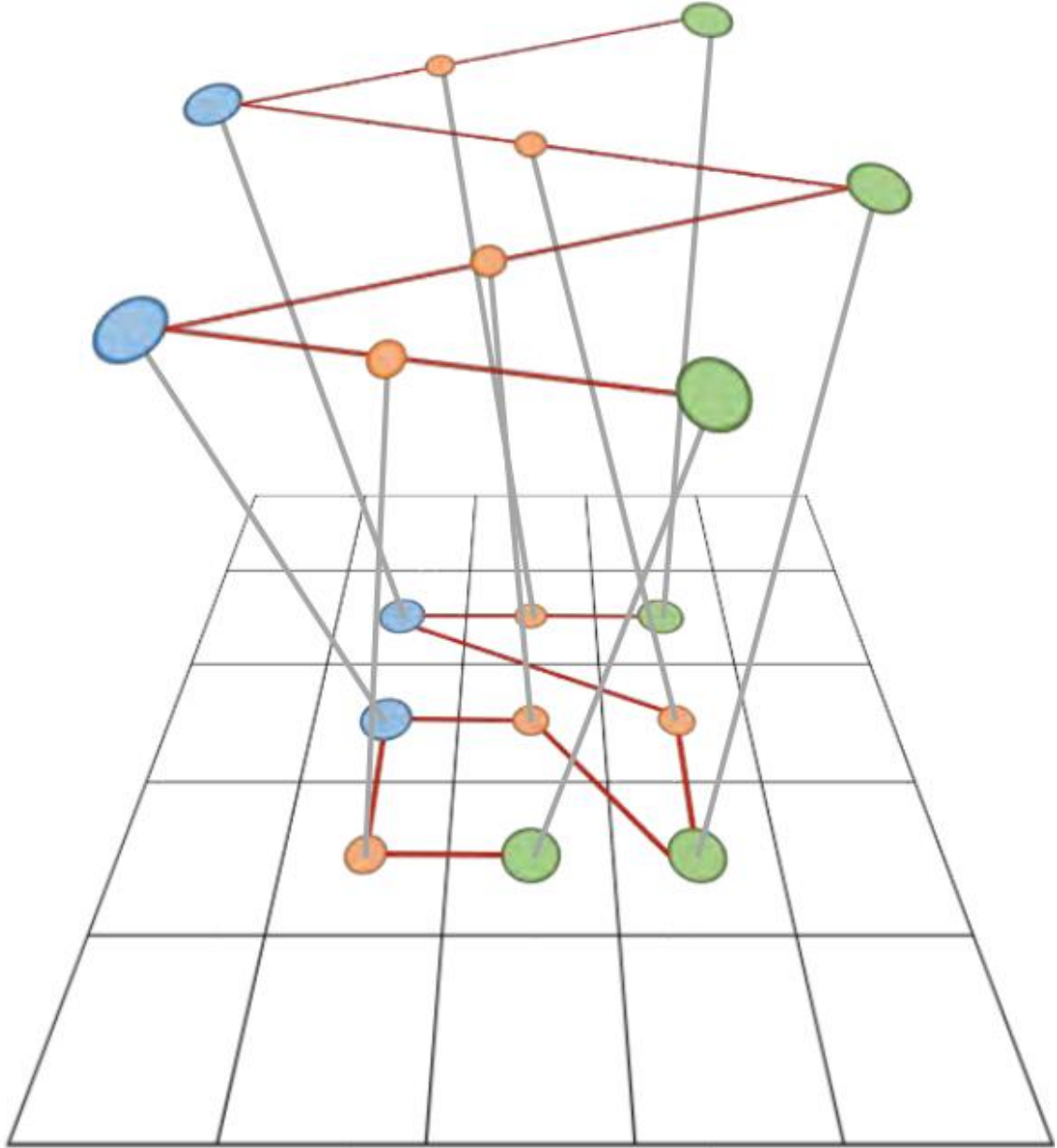


图 18.2: 图 18.2 将束调整因子图映射到图处理器的 tile（核心）上，从而通过高斯信念传播实现快速、分布式、原地推断，改编自^[832]（©IEEE）。这里展示最简单的映射方式：因子图中的每个节点映射到一个任意 tile。关键帧节点为蓝色，路标节点为绿色，测量因子节点为橙色。tile 以同步方式处理，在计算步骤和交换步骤之间交替进行。

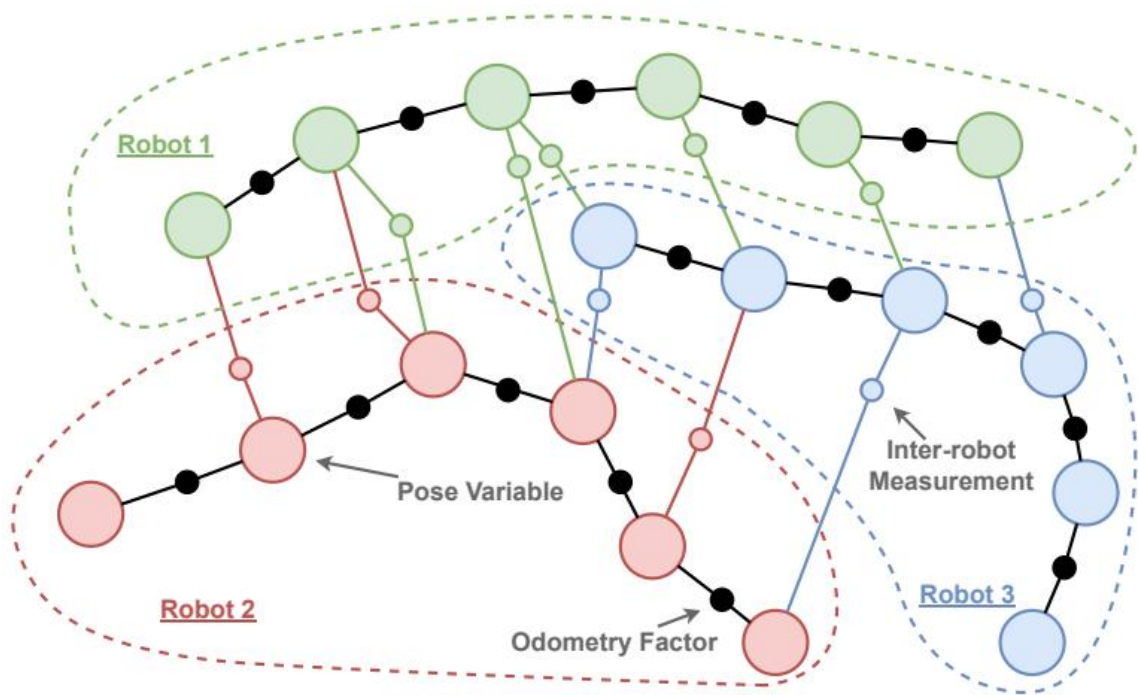


图 18.3: 图 18.3 多机器人定位的因子图, 取自 Robot Web ^[788] (©IEEE)。如彩色区域之间的点线所示, 存储和更新因子的责任分配给参与其中的多个机器人。每个机器人维护自己的位姿变量节点、里程计因子, 以及由自身传感器测得的机器人间测量因子, 并在该图片段上持续运行 GBP。跨越点线边界的消息传递通过各机器人发布和更新的 Robot Web 页面完成, 过程是异步且临时的。

神经网络的神经元和权重，而是构建具有相似结构的变量图和因子图。变量以等价于权重学习的方式推断数值，但这个过程可以持续进行，不需要明确区分训练时间和推理时间。非线性因子扮演神经元的角色，可以软切换，并以不同模式组合来表示复杂函数。图 18.4（取自^[793]）展示了一个具有卷积结构的因子图模块；该模块已用于持续学习图像去噪和分类。

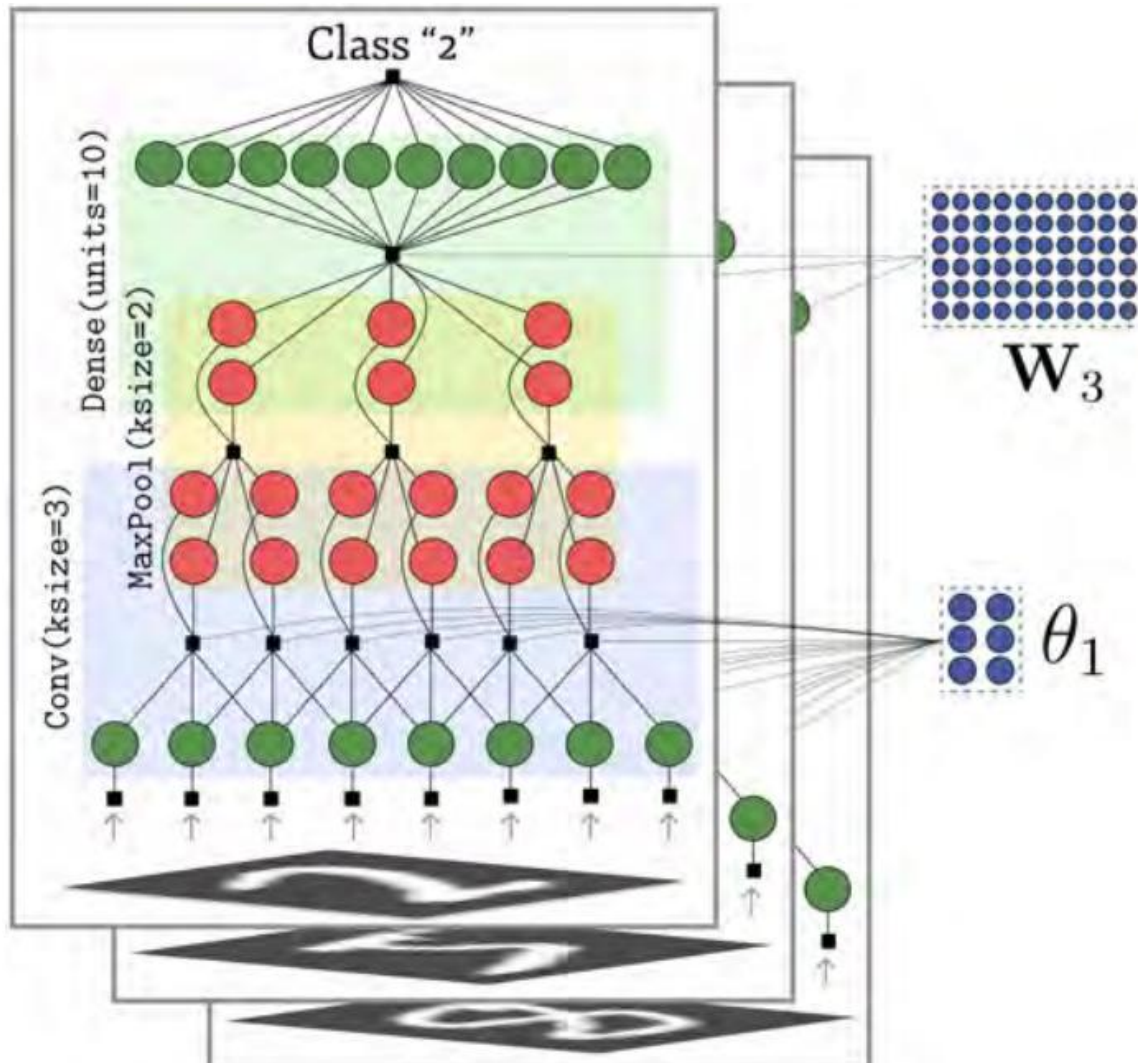


图 18.4: 图 18.4 在 GBP Learning^[793] 中，因子图的结构仿照常见神经网络架构，从而可以用 GBP 进行分布式训练和预测。可学习参数（圆圈）以及输入、输出和激活都作为随机变量加入。参数在所有观测之间共享，而其他变量则为每次观测复制一次。层与层之间的因子（黑色方块）约束表示在局部保持一致；连接输入和输出的因子则鼓励表示与观测相容。层间因子是非线性的，以实现软切换行为。该图像分类示例架构包含卷积层、最大池化层和稠密投影层；如果移除输出观测因子，同一架构也可以在无监督条件下训练。

这种方法或类似方法可能会为一般性的空间智能持续学习打开道路。例如，机器人可以在重建场景的同时学习该场景的平滑模型或分割模型。系统中学习组件和估计组件采用统一的因子图结构后，就可以灵活选择高效或分布式估计方式，也可以轻松地在多个设备之间同步已学习的元素。

18.8 性能度量

本章开头提出的一个假设是：空间智能系统对于广泛任务的有用程度，可以由数量相对较少、且具有普遍重要性的性能度量较好地描述。

SLAM 性能测量最常见的重点是定位精度；已经有多项工作为此创建基准数据集（例如^[1045]）。通常将运动捕获系统提供的外部位姿测量视为真值，并以此比较 SLAM 算法。

我们已经论证，空间智能远不止位姿估计；较新的数据集也在尝试扩展可评估内容的范围。稠密场景建模很难评估，因为需要昂贵且耗时的过程（例如详细激光扫描），才能捕获一个真实场景的完整模型，而且模型必须足够准确和完整，才能被视为真值。对于这一维度以及其他评价维度，另一种选择是使用计算机图形学生成测试数据，例如 ICL-NUIM 数据集^[429]。近来，这种方法还扩展到为语义标注提供真值（例如 SceneNet RGB-D^[746]）；对于真实数据而言，语义标注也是难以获得高质量真值的输出。人们自然会怀疑在合成测试数据上评估的价值，同时也出现了通过众包收集大规模真实建图数据的新方法，例如 ScanNet^[238]。不过，合成数据一直在改进，我们相信其重要性只会不断增长。

退一步说，我们甚至可以质疑空间智能系统使用基准的价值。人们经常评论计算机视觉研究者过度依赖数据集评估：大量精力被用于优化和组合算法，只为在基准上多取得几个百分点，而不是研究新的思想和技术。由于 SLAM 的实时性质，以及不同应用所需的各种有用输出和性能水平，很难用有意义的基准捕获 SLAM。我们一直认为，与任何现有基准上的测量性能相比，亲自运行一个视觉 SLAM 系统约一分钟，通过实时可视化自适应、定性地检查其行为，往往能更多地了解系统的有用性。因此，进展更有意义地体现在高质量实时开源 SLAM 研究系统的发展上，例如 MonoSLAM^[244, 243]、PTAM^[583]、KinectFusion^[801]、LSD-SLAM^[307]、ORB-SLAM^[787]、OKVIS^[645]、SVO^[333]、ElasticFusion^[1184]、DROID-SLAM^[1082]、Gaussian Splatting SLAM^[740] 和 MAST3R-SLAM^[789]，而不是基准分数；人们可以直接对这些系统进行实验。

SLAM 基准之所以令人不满意，是因为它们会对场景类型和形状、相机及其他传感器的选择与安装、帧率和分辨率等作出假设，并且只关注某些评价方面（例如精度），却淡化了效率或稳健性等可能更重要的方面。例如，许多在精度基准上评估算法的论文，会从诸如^[1045]的数据集可用测试序列中进行选择，只报告那些基本能够“工作”的序列上的结果。

这引出一个问题：我们是否仍应建立基准，并努力评估和比较 SLAM 系统？我们的回答仍然是肯定的，但应该扩大空间智能系统中“基准”的含义，并承认不能对基准告诉我们的内容过度信任。

由 PAMELA 研究项目^[795]发布的 SLAMBench 框架，代表了评估完整空间智能系统的早期工作。在 SLAMBench 中，SLAM 算法（具体为 KinectFusion^[801]）在一系列处理器平台上、使用不同语言实现，从精度和计算成本两方面进行测量。从长期来看，我们认为基准测试应当转向能够普遍预测空间智能系统完成任务时表现的度量。这显然是一个多目标指标集合，对 Pareto 前沿^[796]的分析将允许针对特定应用作出选择。

一个可能的指标集合包括：

1. 新探索区域中的局部位姿精度（视觉里程计漂移率）。
2. 已充分建图区域中的长期度量位姿可重复性。
3. 跟踪稳健性百分比。
4. 重定位稳健性百分比。
5. SLAM 系统延迟。
6. 每个像素的稠密距离预测精度。
7. 物体分割精度。
8. 物体分类精度。
9. 用于增强现实的像素配准精度。
10. 场景变化检测精度。
11. 功耗。
12. 设备端数据移动量，以 bit×millimetre（比特 × 毫米）度量。

18.9 延伸阅读与近期趋势

最后，空间智能和 SLAM 研究领域将继续变得更加重要，并向许多不同应用所需的一般三维感知能力演化，最终充分结合人工设计的估计技术和机器学习技术。不过，实际应用的广泛使用需要能力提升同时伴随资源需求下降；这要求对算法、处理器和传感器进行协同思考。高效系统的关键，是识别算法和数据存储中的计算结构，并针对稀疏图模式进行设计，再尽可能使其与即将变得重要的新型硬件相匹配。

人工设计的状态估计和机器学习都将继续重要，并且必须充分集成，以支持灵活而持续的学习。我们相信，二者将出现有趣的趋同：状态估计系统不断加入更多学习元素，而学习网络则引入更多领域知识和结构。无论从哪条路线出发，都应当走向同一个目的地；关于计算结构的同一组研究问题仍然需要解决。